

Mastering Quarkus 3.x

The Complete Developer Guide
Interview Ready

Build-time augmentation · Quarkus REST · Panache · Mutiny
Kafka · OIDC · Native image · Kubernetes

Maratib Ali Khan

linkedin.com/in/maratibali

Java 21 · Jakarta EE 10 · Quarkus 3.33 LTS

Contents

Preface	1
Who this book is for	1
What makes this different from the documentation	1
The running example	1
Conventions	2
How to read it	2
1 Quarkus Fundamentals	3
Learning Objectives	3
1.1 The Problem Quarkus Was Built To Solve	3
1.2 What Quarkus Actually Is	5
1.3 The Central Idea: Shifting Work to Build Time	6
1.4 Augmentation In Detail	9
1.5 Runtime Architecture	13
1.6 JVM Mode vs Native Mode	16
1.7 The Running Example: <code>orders-service</code>	17
1.8 Dev Mode, Live Reload and Dev Services	22
1.9 Extensions	25
1.10 Configuration	26
1.11 Logging	30
1.12 Application Lifecycle	32
1.13 Packaging	34
1.14 Quarkus vs Spring Boot	35
1.15 Best Practices	36
1.16 Common Mistakes	37
1.17 Interview Questions	38
1.18 Cheat Sheet	42
1.19 Summary	43
1.20 References	43
2 Quarkus REST (RESTEasy Reactive)	45
Learning Objectives	45
2.1 Naming: Three Things That Are Easy To Confuse	45
2.2 The Execution Model	46
2.3 Build-Time Route Generation	50
2.4 Resources, Parameters and Responses	51
2.5 Error Handling	53
2.6 Filters and Interceptors	55
2.7 The REST Client	57
2.8 Streaming: <code>Multi</code> and Server-Sent Events	59

2.9	Multipart and File Uploads	61
2.10	HTTP Layer Configuration	62
2.11	OpenAPI and Swagger UI	62
2.12	Testing the REST Layer	63
2.13	Performance Notes	65
2.14	Best Practices	66
2.15	Common Mistakes	66
2.16	Interview Questions	67
2.17	Cheat Sheet	70
2.18	Summary	71
2.19	References	72
3	CDI and ArC	73
	Learning Objectives	73
3.1	CDI in One Page	73
3.2	Why Build-Time DI Changes the Rules	75
3.3	Bean Discovery	76
3.4	Scopes	77
3.5	Client Proxies	80
3.6	Injection Styles	82
3.7	Bean Removal — The Quarkus-Specific Trap	83
3.8	Producers, Disposers and Qualifiers	84
3.9	Conditional Beans	86
3.10	Interceptors and Decorators	86
3.11	Events	88
3.12	Context Propagation	89
3.13	Testing CDI Code	90
3.14	Best Practices	92
3.15	Common Mistakes	92
3.16	Interview Questions	93
3.17	Cheat Sheet	95
3.18	Summary	97
3.19	References	97
4	Configuration and Profiles	98
	Learning Objectives	98
4.1	The Model	98
4.2	Config Sources and Ordinals	100
4.3	Reading Configuration	101
4.4	Types and Converters	104
4.5	Validation	105
4.6	Profiles	106
4.7	Build-Time vs Runtime Properties	107
4.8	Expressions, Defaults and Secrets	110
4.9	YAML Configuration	112
4.10	Custom Config Sources	113
4.11	Configuration in Tests	114
4.12	Troubleshooting	115

4.13 Best Practices	116
4.14 Common Mistakes	116
4.15 Interview Questions	117
4.16 Cheat Sheet	118
4.17 Summary	120
4.18 References	120
5 Hibernate ORM	121
Learning Objectives	121
5.1 What Quarkus Changes About Hibernate	121
5.2 Datasource and Connection Pooling	123
5.3 Entities	124
5.4 Relationships	128
5.5 The Persistence Context	129
5.6 Querying	132
5.7 The N+1 Problem	135
5.8 Transactions and Locking	137
5.9 Caching	138
5.10 Schema Management	139
5.11 Hibernate Reactive	140
5.12 Dev Services and Testing	141
5.13 Performance Checklist	142
5.14 Common Mistakes	143
5.15 Interview Questions	143
5.16 Cheat Sheet	146
5.17 Summary	147
5.18 References	148
6 Panache	149
Learning Objectives	149
6.1 What Panache Is	149
6.2 Active Record	151
6.3 The Repository Pattern	154
6.4 Choosing Between Them	155
6.5 Querying	156
6.6 Updates and Deletes	159
6.7 Transactions	160
6.8 Panache REST Data	161
6.9 Reactive Panache	162
6.10 Testing	163
6.11 Best Practices	164
6.12 Common Mistakes	165
6.13 Interview Questions	165
6.14 Cheat Sheet	168
6.15 Summary	169
6.16 References	169
7 Transactions	170

Learning Objectives	170
7.1 What a Transaction Actually Guarantees	170
7.2 <code>@Transactional</code>	171
7.3 Propagation	173
7.4 Isolation Levels	175
7.5 Rollback Rules	176
7.6 Transactions and the Persistence Context	177
7.7 Programmatic Transactions	180
7.8 Transactions Beyond HTTP	181
7.9 Distributed Transactions	182
7.10 Testing Transactions	186
7.11 Best Practices	187
7.12 Common Mistakes	187
7.13 Interview Questions	188
7.14 Cheat Sheet	191
7.15 Summary	192
7.16 References	192
8 Validation	193
Learning Objectives	193
8.1 Three Kinds of Validation	193
8.2 Getting Started	194
8.3 The Constraint Vocabulary	195
8.4 Where to Validate	197
8.5 Custom Constraints	200
8.6 Validation Groups	202
8.7 Error Responses	203
8.8 Messages and Internationalisation	205
8.9 Programmatic Validation	206
8.10 Testing	206
8.11 Best Practices	208
8.12 Common Mistakes	208
8.13 Interview Questions	208
8.14 Cheat Sheet	211
8.15 Summary	212
8.16 References	212
9 Security, JWT and OIDC	213
Learning Objectives	213
9.1 The Architecture	214
9.2 JWT vs OIDC — Getting This Right	215
9.3 OIDC in Service Mode (Bearer Tokens)	216
9.4 OIDC in Web-App Mode (Authorization Code Flow)	219
9.5 SmallRye JWT	220
9.6 Authorisation	221
9.7 Beyond Roles	223
9.8 Identity Propagation	225
9.9 Transport Security	226

9.10 Secrets	228
9.11 Native Mode Considerations	228
9.12 Testing Security	229
9.13 Best Practices	231
9.14 Common Mistakes	231
9.15 Interview Questions	232
9.16 Cheat Sheet	235
9.17 Summary	236
9.18 References	237
10 Reactive Programming with Mutiny	238
Learning Objectives	238
10.1 What Reactive Actually Buys You	238
10.2 <code>Uni</code> and <code>Multi</code>	239
10.3 The Event-Driven API	241
10.4 Composition	242
10.5 Failure Handling	244
10.6 Backpressure	246
10.7 Threading	247
10.8 Context Propagation	249
10.9 Mutiny or Virtual Threads?	250
10.10 Testing	251
10.11 Best Practices	253
10.12 Common Mistakes	253
10.13 Interview Questions	254
10.14 Cheat Sheet	257
10.15 Summary	258
10.16 References	258
11 Kafka and Messaging	259
Learning Objectives	259
11.1 Kafka in One Page	260
11.2 Setup	261
11.3 Producing	262
11.4 Consuming	263
11.5 Delivery Guarantees	265
11.6 Error Handling	268
11.7 Schemas and Evolution	269
11.8 Other Brokers	270
11.9 Testing	271
11.10 Operating Kafka Consumers	272
11.11 Best Practices	273
11.12 Common Mistakes	273
11.13 Interview Questions	274
11.14 Cheat Sheet	277
11.15 Summary	278
11.16 References	278

12 Scheduling	279
Learning Objectives	279
12.1 The Basics	279
12.2 The Multi-Replica Problem	281
12.3 Concurrent Execution	284
12.4 Time Zones	285
12.5 Programmatic Scheduling	286
12.6 Transactions and Long-Running Work	286
12.7 Observability	287
12.8 Testing	288
12.9 Best Practices	289
12.10 Common Mistakes	290
12.11 Interview Questions	290
12.12 Cheat Sheet	292
12.13 Summary	293
12.14 References	293
13 Caching	294
Learning Objectives	294
13.1 Should You Cache?	294
13.2 The Quarkus Cache Extension	295
13.3 Configuration	297
13.4 The Programmatic API	298
13.5 The Multi-Replica Staleness Problem	299
13.6 Cache Stampede	301
13.7 HTTP Caching	302
13.8 Metrics	303
13.9 Testing	304
13.10 Best Practices	305
13.11 Common Mistakes	305
13.12 Interview Questions	306
13.13 Cheat Sheet	308
13.14 Summary	309
13.15 References	309
14 Metrics and Micrometer	311
Learning Objectives	311
14.1 Setup	311
14.2 The Instruments	312
14.3 Annotations	314
14.4 Cardinality — The Failure Mode	315
14.5 Global Tags and Naming	316
14.6 Built-in Metrics	317
14.7 What to Measure: RED and USE	318
14.8 Percentiles Done Right	319
14.9 Alerting	320
14.10 Exposing Metrics	321
14.11 Testing Instrumentation	321

14.12 Best Practices	322
14.13 Common Mistakes	323
14.14 Interview Questions	323
14.15 Cheat Sheet	325
14.16 Summary	326
14.17 References	327
15 OpenTelemetry and Distributed Tracing	328
Learning Objectives	328
15.1 Why Tracing	328
15.2 The Data Model	329
15.3 Setup	330
15.4 Manual Instrumentation	331
15.5 Context Propagation	333
15.6 Sampling	335
15.7 Correlating Traces, Logs and Metrics	336
15.8 The Cost of Tracing	337
15.9 Testing	337
15.10 Best Practices	338
15.11 Common Mistakes	339
15.12 Interview Questions	339
15.13 Cheat Sheet	342
15.14 Summary	343
15.15 References	343
16 Health Checks	344
Learning Objectives	344
16.1 The Three Probes	344
16.2 SmallRye Health	346
16.3 Writing Checks	347
16.4 Kubernetes Configuration	349
16.5 The Cascading Failure Problem	350
16.6 Health and Graceful Shutdown	353
16.7 Health Checks Are Not Monitoring	354
16.8 Testing	355
16.9 Best Practices	356
16.10 Common Mistakes	356
16.11 Interview Questions	357
16.12 Cheat Sheet	359
16.13 Summary	360
16.14 References	361
17 Testing	362
Learning Objectives	362
17.1 The Test Types	362
17.2 Plain JUnit — the Default	363
17.3 <code>@QuarkusComponentTest</code>	364
17.4 <code>@QuarkusTest</code>	365

17.5 Dev Services in Tests	367
17.6 Test Data and Isolation	368
17.7 What Usually Goes Untested	370
17.8 <code>@QuarkusIntegrationTest</code> and Native	372
17.9 Keeping the Suite Fast	373
17.10 Flaky Tests	374
17.11 Best Practices	374
17.12 Common Mistakes	375
17.13 Interview Questions	375
17.14 Cheat Sheet	378
17.15 Summary	379
17.16 References	380
18 Native Image	381
Learning Objectives	381
18.1 What <code>native-image</code> Does	381
18.2 Building	383
18.3 What Breaks, and Why	384
18.4 Static Initialisation — The Subtle One	386
18.5 The Tracing Agent	387
18.6 Optimising	387
18.7 Debugging	388
18.8 Testing Natively	389
18.9 Deciding	389
18.10 Best Practices	390
18.11 Common Mistakes	391
18.12 Interview Questions	391
18.13 Cheat Sheet	395
18.14 Summary	396
18.15 References	396
19 Docker and Container Images	397
Learning Objectives	397
19.1 Why Layering Matters	397
19.2 The JVM Dockerfile	399
19.3 The Native Dockerfile	400
19.4 Base Images	400
19.5 The Container-Image Extensions	401
19.6 Running Correctly	402
19.7 Supply Chain	404
19.8 Debugging Containers	404
19.9 Multi-Architecture Images	405
19.10 Best Practices	405
19.11 Common Mistakes	406
19.12 Interview Questions	406
19.13 Cheat Sheet	408
19.14 Summary	409
19.15 References	410

20 Kubernetes	411
Learning Objectives	411
20.1 Generating Manifests	411
20.2 A Production Deployment	412
20.3 Resources: Requests, Limits and the CPU Argument	414
20.4 Rollouts Without Dropped Requests	416
20.5 Configuration	417
20.6 Autoscaling	418
20.7 Networking	420
20.8 Database Migrations	421
20.9 Debugging	422
20.10 Best Practices	422
20.11 Common Mistakes	423
20.12 Interview Questions	423
20.13 Cheat Sheet	426
20.14 Summary	427
20.15 References	428
21 OpenShift	429
Learning Objectives	429
21.1 What OpenShift Actually Is	429
21.2 Deploying with the Extension	430
21.3 Build Strategies	431
21.4 Deployment versus DeploymentConfig	432
21.5 Security Context Constraints	432
21.6 Routes	434
21.7 ImageStreams	435
21.8 Built-in Monitoring and Logging	435
21.9 OpenShift Serverless	436
21.10 Migrating from Kubernetes	436
21.11 Best Practices	437
21.12 Common Mistakes	437
21.13 Interview Questions	438
21.14 Cheat Sheet	439
21.15 Summary	440
21.16 References	441
22 Microservice Patterns	442
Learning Objectives	442
22.1 Boundaries	442
22.2 Resilience	444
22.3 Communication	447
22.4 Data Patterns	448
22.5 API Gateway and BFF	451
22.6 Discovery, Configuration and Identity	452
22.7 The Distributed Monolith	452
22.8 The Modular Monolith	453
22.9 Best Practices	454

22.10 Common Mistakes	454
22.11 Interview Questions	455
22.12 Cheat Sheet	458
22.13 Summary	458
22.14 References	459
23 Performance Tuning	460
Learning Objectives	460
23.1 A Method	460
23.2 Load Testing Properly	462
23.3 Profiling	463
23.4 The Four Usual Bottlenecks	464
23.5 JVM Tuning	465
23.6 Startup Optimisation	467
23.7 Memory Optimisation	467
23.8 Does It Actually Help?	468
23.9 Best Practices	468
23.10 Common Mistakes	469
23.11 Interview Questions	469
23.12 Cheat Sheet	472
23.13 Summary	473
23.14 References	474
24 Best Practices and Migration	475
Learning Objectives	475
24.1 Structuring a Codebase	475
24.2 Code Conventions	476
24.3 Convention Checklist	477
24.4 Dependency and Extension Hygiene	478
24.5 Upgrading Quarkus	479
24.6 Spring Boot to Quarkus: the Mapping	479
24.7 The Compatibility Extensions	481
24.8 Planning a Migration	483
24.9 The Paved Road	484
24.10 Anti-Pattern Catalogue	485
24.11 Interview Questions	486
24.12 Cheat Sheet	489
24.13 Summary	490
24.14 References	490
25 Senior Interview Scenarios	491
25.1 How to Answer a Scenario Question	491
25.2 Production Incidents	491
25.3 Architecture and Design	495
25.4 Performance	498
25.5 Data and Consistency	501
25.6 Security	503
25.7 Team, Process and Judgement	505

25.8 Summary	508
25.9 References	508
A Quarkus Annotation Cheat Sheet	509
A.1 REST — Server	509
A.2 CDI and Dependency Injection	513
A.3 Configuration	516
A.4 Validation	517
A.5 Hibernate ORM and Panache	519
A.6 Transactions	521
A.7 Caching	522
A.8 Scheduling	523
A.9 Messaging and Kafka	524
A.10 Security	525
A.11 Fault Tolerance	526
A.12 Observability	527
A.13 Testing	528
A.14 Native Image	529
A.15 Quick Reference by Task	530
A.16 Annotations That Are Easy to Confuse	531
B Glossary	532
A	532
B	532
C	533
D	534
E	534
F	535
G	535
H	535
I	535
J	535
K	536
L	536
M	536
N	536
O	537
P	537
Q	538
R	538
S	538
T	539
U	539
V	539
W	539
X	540
Z	540
Abbreviations	540

Preface

Who this book is for

This book has two readers in mind, and it refuses to choose between them.

The first is someone learning Quarkus. Every chapter starts from first principles, explains *why* a mechanism exists before showing how to use it, and assumes no prior exposure to Jakarta EE or reactive programming.

The second is an experienced engineer preparing for a senior interview, or trying to make an architectural decision they will have to defend. For that reader, every chapter goes past the documentation into internals, trade-offs, failure modes, and the reasoning behind design choices — including the cases where Quarkus is the wrong answer.

If you are the first reader today, you will be the second one eventually. The material is arranged so the same book serves both.

What makes this different from the documentation

The Quarkus guides are excellent and this book does not try to replace them. What documentation cannot do, by its nature, is tell you:

- **Why** a mechanism was designed this way, and what was traded away.
- **How it fails**, and what the failure looks like at 3 a.m. on a dashboard.
- **When not to use it**, which is often the more valuable half of the knowledge.
- **What an interviewer is actually testing** when they ask a question — and what separates a mid-level answer from a senior one.

Every chapter therefore contains internals, performance implications, common mistakes drawn from real projects, and graded interview questions from junior through principal level, each with a model answer.

The running example

Rather than a series of disconnected snippets, the book builds one system: `orders-service`, the order-management microservice of a fictional commerce platform. It starts as a REST resource in Chapter 2 and accumulates persistence, security, messaging, observability, native compilation and a Kubernetes deployment as the chapters progress. Code that appears in Chapter 5 is code you will see again in Chapter 18.

Conventions

Every code example targets **Java 21** and **Quarkus 3.33 LTS** with **Jakarta EE 10**. Where an API changed recently — most notably the Quarkus 3.9 rename of RESTEasy Reactive to Quarkus REST — the book uses the current name and notes the old one, because a great deal of material online (including most LLM-generated examples) is still written against the pre-3.9 names.

Diagrams are written in Mermaid, so the Markdown edition renders them natively on GitHub, GitBook, MkDocs and Docusaurus; the PDF edition ships them as vector artwork.

Callouts in the margin look like this:

A note like this flags something that is easy to get wrong, or that an interviewer is likely to probe.

How to read it

Chapters 1 to 4 are foundational — build-time augmentation, the HTTP layer, CDI, and configuration — and everything later assumes them. After that the book is close to random-access: read Chapter 11 when you need Kafka, Chapter 18 when you need native image.

If you are preparing for an interview under time pressure, read Chapter 1 properly, then the interview-question section of each chapter, then return to the body for whichever answers you could not give confidently.

Quarkus Fundamentals

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Jakarta EE 10 · Maven 3.9 · GraalVM/Mandrel 21

Learning Objectives

By the end of this chapter you will be able to:

1. Explain *why* Quarkus exists in terms of the economics of running Java in a container, not just in terms of feature lists.
2. Describe the **build-time augmentation** model precisely — what a build step is, what a build item is, what a recorder is, and how they compose into a deployable artefact.
3. Distinguish **static init** from **runtime init**, and predict which one a given piece of code runs in.
4. Choose between **JVM mode** and **native mode** using cost, latency and operational criteria rather than folklore.
5. Read a Quarkus `pom.xml` and explain what every element does.
6. Explain how **dev mode**, **live reload**, **Dev Services** and **continuous testing** work internally — including why live reload is not “just a classloader trick”.
7. Describe the anatomy of an extension and why it is split into **runtime** and **deployment** modules.
8. Reason about **configuration ordinals**, and about the difference between build-time-fixed and runtime-overridable properties — the single most common source of production surprises.
9. Trace the **application lifecycle** from `main()` to `StartupEvent` to `ShutdownEvent`.
10. Answer junior through principal-level interview questions on all of the above, and know what the interviewer is actually testing.

1.1 The Problem Quarkus Was Built To Solve

1.1.1 Java's original bet

Java was designed for a world of **long-lived, vertically-scaled processes**. A servlet container started once on a Tuesday and was still running in November. Under those assumptions, a set of engineering trade-offs made complete sense:

- **Scan the classpath at startup.** Startup happens once; amortised over months, a 40-second boot is free.
- **Build object graphs reflectively at runtime.** Reflection is slow, but only during boot.

- **Optimise for peak throughput, not for time-to-first-request.** The JIT will eventually produce excellent machine code, so let it warm up.
- **Treat memory as abundant.** One big heap on one big machine; the JVM's per-process overhead is amortised across thousands of concurrent users.

Every mainstream Java framework — Java EE application servers, Spring, and their descendants — internalised these assumptions. They are not bad assumptions. They were *correct* assumptions for the deployment model of 2005.

1.1.2 What changed

Containers and Kubernetes inverted every one of those assumptions:

Assumption (2005)	Reality (cloud native)
One long-lived process	Pods are cattle: rescheduled, rolled, scaled, evicted constantly
Startup cost amortised over months	Startup cost is paid on every scale-out event and every deploy
Memory is abundant and shared	Memory is the billing unit; each replica pays the full framework overhead
Peak throughput dominates	p99 latency during scale-out and cold start dominates user experience
Vertical scaling	Horizontal scaling: 50 small replicas, not 1 large one

Consider the arithmetic. A traditional Java microservice that needs 400 MB of RSS and takes 12 seconds to start is perfectly serviceable as a single instance. Run 40 replicas of it across 12 services and you are provisioning ~16 GB of memory purely for framework overhead, and every deployment rollout has a 12-second-per-pod latency floor. Under a horizontal pod autoscaler reacting to a traffic spike, those 12 seconds are 12 seconds of *continuing to drop requests*.

Meanwhile Go and Node services in the next namespace start in 30 ms with 20 MB of RSS. The pressure on Java was not aesthetic; it was a line item on a cloud bill and a number on an SLO dashboard.

1.1.3 Where the time and memory actually go

This is the question that produced Quarkus. If you profile the startup of a conventional Java framework, the time is dominated by things that are, in principle, **knowable before the application ever runs**:

```
Classpath scanning ..... find every @Entity, @Path, @Component
Annotation parsing ..... read bytecode, build metadata model
Reflective wiring ..... construct proxies, resolve injection points
Config parsing ..... read properties, bind to objects
Framework bootstrapping .... build ORM metamodel, compile JAXB contexts,
                             validate bean definitions, build routers
```

And the memory is dominated by the *retained cost of having done that work*: reflection metadata caches, annotation objects, `Method/Field` handles, the framework’s own model of your application, dynamic proxy classes, and the class metadata for the framework machinery itself.

Quarkus’s founding observation:

Almost none of this work depends on runtime information. Your `@Path` annotations do not change between deployments. Your entity graph does not change. Your injection points do not change. **This is compile-time-knowable work being performed at runtime, every single time the process starts, on every single replica.**

1.1.4 The Quarkus thesis

Quarkus’s answer is a single idea applied with unusual discipline:

Do it once, at build time. Ship the result. Throw away the machinery that computed it.

That last clause is the part people miss. It is not merely that Quarkus is *faster* at startup because work moved earlier. It is that once the work has been done at build time, **the code that did the work does not need to exist in the runtime image at all**. Hibernate’s annotation processor, the CDI bean resolver, the JAX-RS scanner — all of it runs during the Maven build in a separate classloader and is then discarded. What ships is the *result*: generated bytecode that does the equivalent job with no scanning, no reflection, and no framework model to retain.

This is why Quarkus improves startup time and memory **simultaneously**. A cache would improve one at the cost of the other. Removing the work improves both.

1.2 What Quarkus Actually Is

It is worth being precise, because the marketing description (“Kubernetes-native Java stack”) does not tell you anything actionable.

Quarkus is three things stacked on top of one another:

- 1. A build-time augmentation framework.** A Maven/Gradle plugin plus a build engine that runs a graph of *build steps* which analyse your application and generate bytecode, resources and metadata. Everything else in Quarkus is built on this.
- 2. A curated set of extensions.** Roughly 500+ libraries — Hibernate, Kafka, Keycloak, Vert.x, gRPC, Flyway, Kubernetes client — each wrapped so that its initialisation participates in build-time augmentation and so that it is closed-world-safe for native compilation.
- 3. A developer experience layer.** Dev mode, live reload, Dev Services, continuous testing, Dev UI, and the `quarkus` CLI. This layer is not a nicety; it is a deliberate counterweight, because build-time augmentation could easily have made the inner development loop *worse*, and the team decided that was unacceptable.

Standards-wise, Quarkus 3.x is built on **Jakarta EE 10** (hence the `jakarta.*` package namespace) and **MicroProfile 6/7**, implemented largely through the **SmallRye** project. You write standard

`jakarta.ws.rs`, `jakarta.persistence`, `jakarta.inject` and `org.eclipse.microprofile.*` APIs. Quarkus changes *how* those APIs are implemented and initialised, not what they look like.

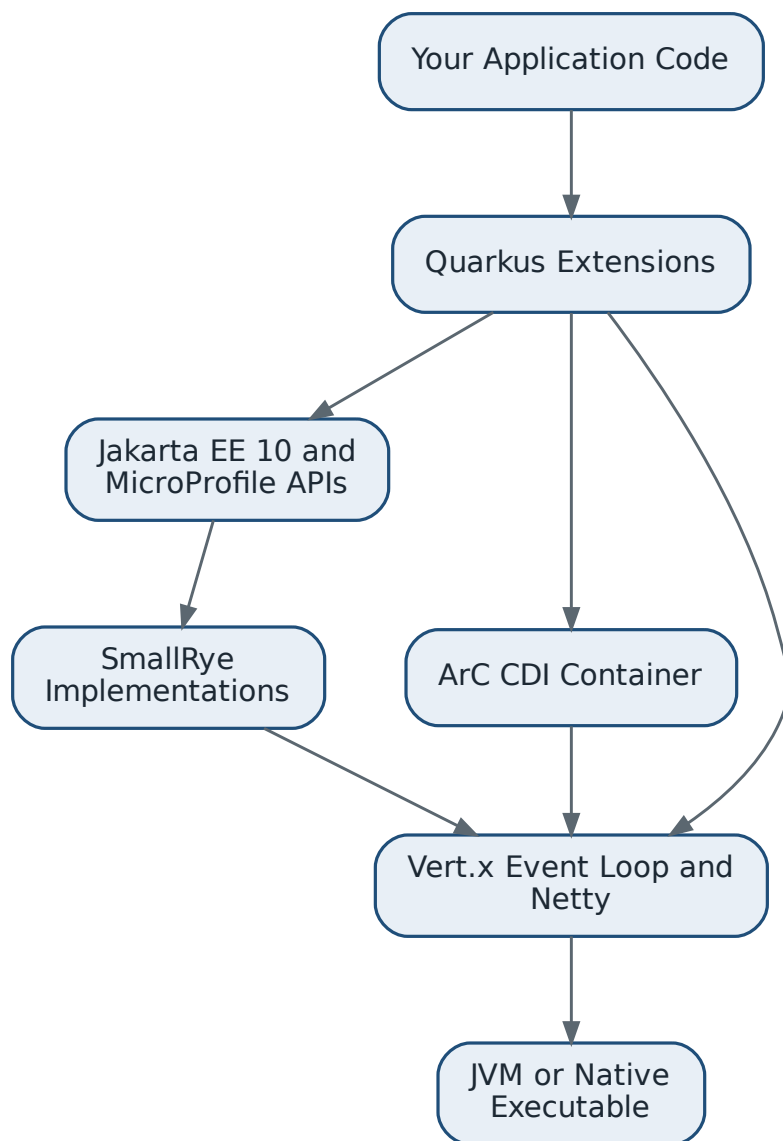


Figure 1.1 — The Quarkus layer cake. Note that Vert.x sits underneath almost everything: Quarkus’s I/O layer is a single reactive engine that both the imperative and the reactive programming models are built on.

1.3 The Central Idea: Shifting Work to Build Time

Maratib Ali Khan

1.3.1 The three phases

Every Quarkus application passes through three distinct phases. Getting these straight is the foundation for everything else in this book, and confusing them is the root cause of most “it works in dev but fails in native” bugs.

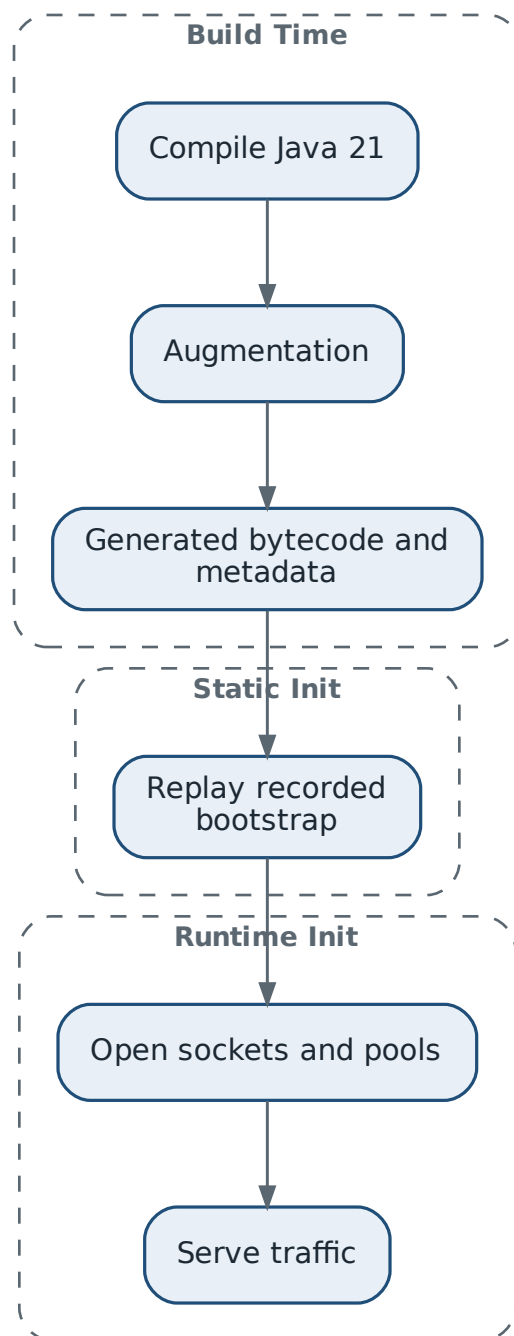


Figure 1.2 — The three phases. In native mode the boundary between augmentation and static init is where the GraalVM image heap is snapshotted.

Phase 1 — Augmentation (build time, inside Maven/Gradle). Runs during `mvn package`. Quarkus indexes your classes with **Jandex**, runs all registered build steps, and emits generated classes, reflection registrations, resource lists and a recorded bootstrap sequence. The deployment-time code that does this runs in an isolated classloader and never ships.

Phase 2 — Static initialisation (JVM: process start; native: build time). Executes the bootstrap actions that were *recorded* during augmentation with `ExecutionTime.STATIC_INIT`. Building an ORM metamodel or a JSON serialiser belongs here: it is deterministic, depends only on your code, and produces objects that are safe to reuse.

The crucial detail: **in native mode, static init runs during the native image build, and the resulting object graph is serialised into the executable’s image heap.** At runtime it is already in memory before `main()` is entered. That is where a large part of native’s sub-millisecond startup comes from.

Phase 3 — Runtime initialisation (always at process start). Anything that touches the outside world or the current environment: opening ports, creating connection pools, reading environment variables, starting threads, seeding randomness. Recorded with `ExecutionTime.RUNTIME_INIT`.

1.3.2 Why the distinction is not academic

Ask yourself: what happens if a database connection is opened during static init, in native mode?

The connection would be established **on the build machine**, and a dead socket handle would be baked into the executable’s image heap. Every container started from that image would come up holding a reference to a TCP connection that was closed weeks ago, to a host that may not be routable from production.

This is precisely why Quarkus is strict about the boundary, and why the native build fails fast with errors such as `Detected an instance of Random/SplittableRandom in the image heap` or `Unsupported feature: No instances of java.net.Socket are allowed in the image heap`. Those messages are not GraalVM being difficult; they are GraalVM catching a genuine correctness bug at build time instead of at 3 a.m.

Interview signal. If you can articulate *why* a `Random` instance in the image heap is a bug — every replica of the image would produce the identical “random” sequence, because the seed was fixed on the build machine — you have demonstrated real understanding of the closed-world model rather than memorised trivia.

1.4 Augmentation In Detail

This section is the deepest part of the chapter. Most Quarkus developers never write a build step, but understanding the model is what separates “I use Quarkus” from “I understand Quarkus” in a senior interview.

1.4.1 Jandex: the index everything is built on

Before any build step runs, Quarkus builds a **Jandex index** of your application and its dependencies. Jandex is an offline bytecode indexer: it parses class files and produces a compact,

queryable index of classes, annotations, method signatures and the class hierarchy — *without loading any classes*.

Not loading classes matters enormously. Traditional classpath scanning must load a class to read its annotations, which triggers static initialisers, pulls in transitive dependencies, and permanently occupies metaspace. Jandex reads the bytes and answers questions like “give me every class annotated with `@Path`” in microseconds, with zero class loading.

```
// The kind of query build steps run against the index.
// (Illustrative – you write this inside an extension, not in application code.)
for (AnnotationInstance ann :
    index.getAnnotations(DotName.createSimple("jakarta.ws.rs.Path"))) {
    ClassInfo resourceClass = ann.target().asClass();
    // ... generate a route for it, no reflection, no class loading
}
```

A dependency JAR is only indexed if it carries a `META-INF/jandex.idx`, or if you list it under `quarkus.index-dependency.*`. This is why a CDI bean or a JPA entity in an external module can be silently invisible to Quarkus — a genuinely common real-world bug we will return to in §1.16.

1.4.2 Build steps and build items

The augmentation engine is a **dependency-injected, parallel build graph**, conceptually similar to a build tool’s task graph.

- A **build step** is a method annotated `@BuildStep`.
- It **consumes** build items as method parameters.
- It **produces** build items as return values or via `BuildProducer<T>`.
- The engine topologically sorts the steps by their item dependencies and executes independent branches in parallel.

There are two flavours of build item:

- `SimpleBuildItem` — at most one instance exists (e.g. `CombinedIndexBuildItem`, the merged Jandex index).
- `MultiBuildItem` — many instances (e.g. `ReflectiveClassBuildItem`, one per class needing reflection).

A minimal, realistic build step:

```
package com.acme.audit.deployment;

import io.quarkus.deployment.annotations.BuildStep;
import io.quarkus.deployment.annotations.ExecutionTime;
import io.quarkus.deployment.annotations.Record;
import io.quarkus.deployment.builditem.CombinedIndexBuildItem;
import io.quarkus.deployment.builditem.FeatureBuildItem;
import io.quarkus.deployment.builditem.nativeimage.ReflectiveClassBuildItem;
import io.quarkus.deployment.annotations.BuildProducer;
import io.quarkus.arc.deployment.SyntheticBeanBuildItem;
import org.jboss.jandex.AnnotationInstance;
import org.jboss.jandex.ClassInfo;
```

```

import org.jboss.jandex.DotName;

class AuditProcessor {

    private static final DotName AUDITED = DotName.createSimple("com.acme.audit.Audited");

    /** Advertises the extension in the startup banner. */
    @BuildStep
    FeatureBuildItem feature() {
        return new FeatureBuildItem("acme-audit");
    }

    /**
     * Finds every @Audited class at build time and registers it for reflection.
     * This runs during `mvn package`, never at runtime.
     */
    @BuildStep
    void registerAuditedTypes(CombinedIndexBuildItem index,
                             BuildProducer<ReflectiveClassBuildItem> reflective) {
        for (AnnotationInstance ann : index.getIndex().getAnnotations(AUDITED)) {
            if (ann.target().kind() != org.jboss.jandex.AnnotationTarget.Kind.CLASS) {
                continue;
            }
            ClassInfo type = ann.target().asClass();
            reflective.produce(
                ReflectiveClassBuildItem.builder(type.name().toString())
                    .methods(true)
                    .fields(true)
                    .reason("Serialised by the @Audited audit trail writer")
                    .build());
        }
    }

    /**
     * Records a runtime action. The recorder call below does NOT execute the
     * method now – it records the intent to call it later.
     */
    @BuildStep
    @Record(ExecutionTime.RUNTIME_INIT)
    SyntheticBeanBuildItem auditSink(AuditRecorder recorder, AuditBuildTimeConfig config) {
        return SyntheticBeanBuildItem
            .configure(AuditSink.class)
            .scope(jakarta.enterprise.context.ApplicationScoped.class)
            .runtimeValue(recorder.createSink(config.destination()))
            .done();
    }
}

```

1.4.3 Recorders: the trick that makes it all work

The recorder pattern is the most elegant idea in Quarkus and the one interviewers use to separate levels.

The problem: a build step runs at build time, in a different JVM and a different classloader from the eventual application. It cannot simply *call* runtime code — the runtime classes may not even be loadable in the deployment classloader, and in native mode there is no runtime yet.

The solution: **method calls on a `@Recorder` class are intercepted and recorded as bytecode rather than executed.**

```
package com.acme.audit.runtime;

import io.quarkus.runtime.annotations.Recorder;
import io.quarkus.runtime.RuntimeValue;

@Recorder
public class AuditRecorder {

    // Looks like a normal method. At build time, calling it does NOT run the body.
    public RuntimeValue<AuditSink> createSink(AuditBuildTimeConfig config) {
        // This body executes at RUNTIME, on the recorded arguments.
        return new RuntimeValue<>(new FileAuditSink(config.destination()));
    }
}
```

When the build step calls `recorder.createSink(config)`, Quarkus’s bytecode recorder does not invoke the method. It appends an instruction to a generated class — roughly:

```
// GENERATED at build time, executed at startup. Simplified.
public class AuditRecorderGenerated {
    public void deploy(StartupContext ctx) {
        AuditRecorder r = new AuditRecorder();
        RuntimeValue v = r.createSink(deserialisedConfigInstance);
        ctx.putValue("audit.sink", v);
    }
}
```

Two consequences follow, and both are common interview questions:

1. **`RuntimeValue<T>` is a deferred handle.** At build time you hold a *proxy* representing “the value that will exist at runtime”. You cannot call methods on it or inspect it during augmentation. If you try, you get the classic `java.lang.IllegalStateException: Cannot invoke method on a runtime value`.
2. **Recorder arguments must be serialisable into bytecode.** Primitives, `String`, collections, and simple JavaBeans work. Arbitrary objects with no no-arg constructor do not. This constraint is *why* build-time config classes are plain, boring data holders.

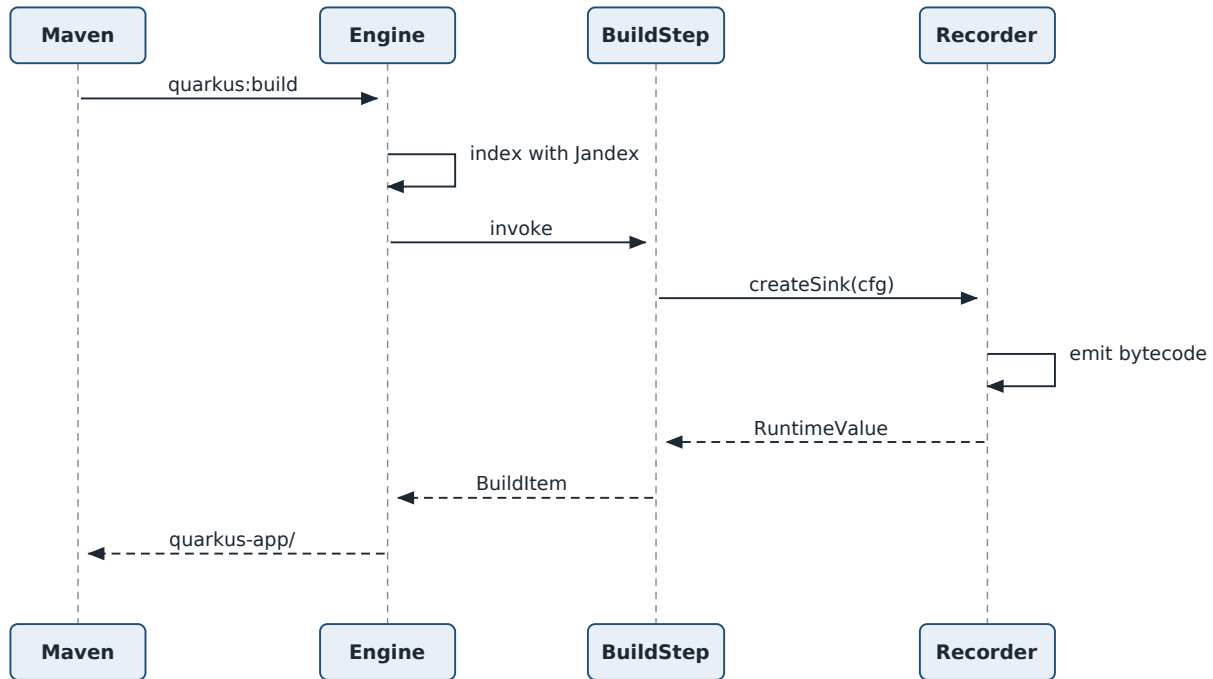


Figure 1.3 — The recorder never executes; it writes bytecode that will execute later.

1.4.4 The output of augmentation

After augmentation, `target/quarkus-app/` contains:

```

target/quarkus-app/
├─ quarkus-run.jar          # tiny launcher, Class-Path manifest only
├─ app/
│   └─ orders-service-1.0.0.jar # YOUR code, unchanged
├─ lib/
│   ├── boot/               # the handful of jars needed to bootstrap
│   └─ main/                 # all other dependencies, one file each
└─ quarkus/
    ├── generated-bytecode.jar # everything augmentation produced
    ├── transformed-bytecode.jar
    └─ quarkus-application.dat # the recorded bootstrap sequence
  
```

This layout — the **fast-jar** format, the default since Quarkus 1.12 — is deliberately container-friendly. Because dependencies live in `lib/` as individual files that change rarely, and your code lives in `app/` and changes on every commit, a Dockerfile can copy them in separate layers. Rebuilding after a code change re-uploads a 50 KB layer rather than a 60 MB uber-jar. We will exploit this properly in Chapter 19.

1.5.1 The startup sequence

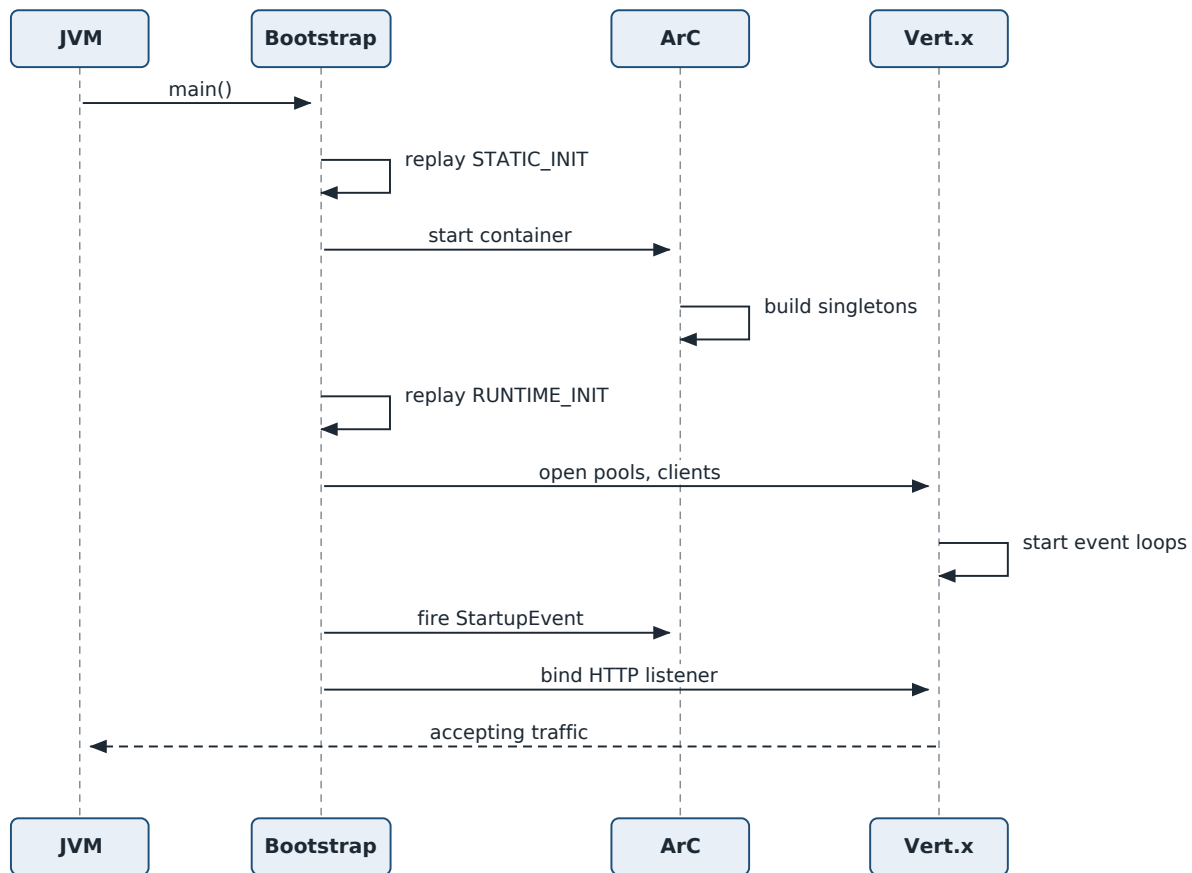


Figure 1.4 — Startup. Note that the HTTP listener binds last, so a pod never accepts traffic before its dependencies are ready.

1.5.2 ArC: CDI without the reflection

Quarkus’s CDI 4.0 implementation is called **ArC**. It is a full CDI container that resolves the bean graph **at build time** and generates a concrete Java class per bean.

For a bean like:

```
@ApplicationScoped
public class OrderService {
    @Inject OrderRepository repository;
}
```

ArC generates something functionally equivalent to:

```
// GENERATED – illustrative, not the literal output
public class OrderService_Bean implements InjectableBean<OrderService> {
    public OrderService create(CreationalContext<OrderService> ctx) {
        OrderService instance = new OrderService();
        instance.repository = (OrderRepository) orderRepositoryBean.get(ctx); // direct call
    }
}
```

```

        return instance;
    }
}

```

There is no reflection, no annotation processing and no bean resolution at runtime — just a direct method call. This has two important consequences:

Consequence 1: unused beans are removed. Because the whole graph is known at build time, ArC deletes beans nothing injects. This is excellent for image size, and it is also the source of the single most common CDI complaint in Quarkus: a bean that is only ever looked up dynamically gets removed. The fix is `@Unremovable`, and we cover the full rules in Chapter 3.

Consequence 2: normal-scoped beans are client proxies. `@ApplicationScoped` and `@RequestScoped` beans are injected as generated subclasses that delegate to the contextual instance. This is why an `@ApplicationScoped` bean must not be `final`, must not have `final` methods you expect to be intercepted, and must have a no-arg constructor.

1.5.3 Vert.x underneath everything

Quarkus's HTTP layer is Eclipse Vert.x on Netty. There is exactly **one** I/O engine, and both programming models sit on top of it:

- **Event loop threads** — a small pool, typically `2 × CPU cores`. Non-blocking work runs here. Blocking one is a serious incident: you have removed a fraction of your entire server's I/O capacity.
- **Worker threads** — an elastic pool for blocking work. Standard imperative JAX-RS endpoints and `@Transactional` JDBC calls run here.

Quarkus decides which pool your endpoint runs on **at build time**, by inspecting its return type and annotations:

```

@Path("/orders")
public class OrderResource {

    // Returns Uni -> assumed non-blocking -> runs on the EVENT LOOP
    @GET
    public Uni<List<OrderView>> listReactive() { ... }

    // Returns a plain type -> assumed blocking -> runs on a WORKER thread
    @GET @Path("/sync")
    public List<OrderView> listBlocking() { ... }

    // Explicit override: force a worker thread even though it returns Uni
    @GET @Path("/mixed")
    @Blocking
    public Uni<String> mixed() { ... }

    // Explicit override: force the event loop even though it returns a plain type
    @GET @Path("/fast")
    @NonBlocking
    public String fast() { return "pong"; }
}

```

This build-time dispatch decision is one of the most frequently asked Quarkus interview questions. The answer that scores is: *Quarkus inspects the signature during augmentation and generates the dispatch code accordingly; `@Blocking` and `@NonBlocking` override the heuristic; putting a `JDBC` call in a `Uni`-returning method without `@Blocking` will block an event loop and degrade the whole server, not just that request.*

1.6 JVM Mode vs Native Mode

1.6.1 What actually differs

	JVM mode (fast-jar)	Native mode (GraalVM/Mandrel)
Startup	0.5 – 1.5 s	0.01 – 0.05 s
RSS at idle	90 – 180 MB	20 – 50 MB
Peak throughput	Highest (C2 JIT)	~10–30 % lower, no JIT
Build time	10 – 40 s	2 – 6 minutes
Build memory	~1 GB	6 – 12 GB
Debugging	Full JVM tooling	Limited; no JDWP, gdb-level
Reflection	Unrestricted	Must be registered at build time
Class loading	Dynamic	Closed world — no dynamic loading
Observability	JFR, async-profiler, JMX	JFR partial, no JMX by default

1.6.2 The closed-world assumption

Native compilation performs whole-programme static analysis and produces an executable containing **only reachable code**. Everything else is deleted. This is what produces the small binary and small heap, and it is also the source of every native-mode difficulty:

- **Reflection** breaks unless the target is registered — the analysis cannot see `Class.forName(someString)`.
- **Dynamic proxies** must be declared up front.
- **Resources** on the classpath are excluded unless explicitly included.
- **JNI**, `MethodHandles` with runtime-computed targets, and service loaders all need configuration.

In practice, Quarkus extensions handle this for you: that is a large part of what an extension *is*. You only hit it when you use a plain library that has no Quarkus extension. Then you reach for:

```
@RegisterForReflection(targets = { LegacyPayload.class, LegacyResponse.class })
public class ReflectionConfig { }
```

Or, when you do not control the class, `reflect-config.json` under `src/main/resources/META-INF/native-image/`. Chapter 18 covers the full toolkit including the tracing agent.

1.6.3 Choosing — a decision framework

Choose **native** when:

- The workload is serverless / scale-to-zero (Lambda, Knative, Cloud Run) where cold start is user-visible.
- You run many replicas of a small service and memory is your dominant cost.
- The service is CLI-like or short-lived (batch jobs, operators, sidecars).
- Security posture benefits from a smaller attack surface and no dynamic class loading.

Choose **JVM** when:

- Throughput per instance is what you optimise for and instances are long-lived.
- Your CI budget cannot absorb multi-minute native builds on every PR.
- You depend on libraries with poor native support and no extension.
- Your team relies on JVM-level production tooling (JMX, live profilers, heap dumps).

The professional answer to “JVM or native?” is “both”. The mature pattern is: develop and test in JVM mode, run the native build in a nightly or pre-release pipeline stage, and make the production decision per service. Quarkus is explicitly designed so the same codebase works in both, and `@QuarkusIntegrationTest` lets you run the *same* test suite against the native binary. Committing to native everywhere on day one is a common and expensive mistake.

1.7 The Running Example: `orders-service`

Throughout this book we build one system rather than a series of disconnected snippets. It is an order-management microservice for a fictional commerce platform:

- **Chapter 1–4** — project setup, REST layer, CDI structure, configuration.
- **Chapter 5–8** — persistence with Hibernate ORM and Panache, transactions, validation.
- **Chapter 9–11** — OIDC security, reactive pipelines, Kafka events.
- **Chapter 12–17** — scheduling, caching, metrics, tracing, health, testing.
- **Chapter 18–21** — native image, Docker, Kubernetes, OpenShift.
- **Chapter 22–25** — patterns, performance, practices, senior scenarios.

1.7.1 Bootstrapping

The `quarkus` CLI is the fastest route (install via SDKMAN: `sdk install quarkus`):

```
quarkus create app com.acme:orders-service:1.0.0-SNAPSHOT \
  --java=21 \
  --extension='rest-jackson,hibernate-orm-panache,jdbc-postgresql,smallrye-health'
cd orders-service
```

The Maven-only equivalent, which is what you will see in CI and in most corporate environments:

```
mvn io.quarkus.platform:quarkus-maven-plugin:3.33.1:create \
  -DprojectGroupId=com.acme \
  -DprojectArtifactId=orders-service \
```

```
-DprojectVersion=1.0.0-SNAPSHOT \
-DjavaVersion=21 \
-Dextensions='rest-jackson,hibernate-orm-panache,jdbc-postgresql,smallrye-health'
```

Note on extension names. In Quarkus 3.9 the reactive stack was renamed: `quarkus-resteasy-reactive` became **`quarkus-rest`**, `quarkus-resteasy-reactive-jackson` became **`quarkus-rest-jackson`**, and `quarkus-rest-client-reactive` became **`quarkus-rest-client`**. Maven relocations keep old coordinates working, but new code should use the new names. Tutorials written before 2024 — and, notably, many LLM-generated examples — still use the old ones. Knowing this distinction is a quick credibility marker in an interview.

1.7.2 Project structure

```
orders-service/
├── pom.xml
├── mvnw / mvnw.cmd          # Maven wrapper – commit these
├── .dockerignore
├── src/
│   ├── main/
│   │   ├── java/com/acme/orders/
│   │   │   ├── api/          # REST resources + DTOs
│   │   │   ├── domain/      # entities + domain services
│   │   │   ├── infra/       # repositories, clients, adapters
│   │   │   └── config/       # @ConfigMapping interfaces
│   │   ├── resources/
│   │   │   ├── application.properties
│   │   │   ├── import.sql    # dev-only seed data
│   │   │   └── META-INF/resources/ # static assets served at /
│   │   └── docker/
│   │       ├── Dockerfile.jvm
│   │       ├── Dockerfile.native
│   │       └── Dockerfile.native-micro
│   └── test/
│       └── java/com/acme/orders/
└── target/
    └── quarkus-app/          # the fast-jar output
```

Two structural notes worth internalising early:

- `src/main/resources/META-INF/resources/` is served as static content at the application root. This is a Quarkus convention that surprises people expecting `src/main/webapp`.
- Package structure is *not* enforced by Quarkus (unlike Spring Boot, which scans downward from the main class). Because ArC uses a Jandex index of the whole application, a bean in any package is discovered. This is freeing, but it means you must impose architectural discipline yourself.

1.7.3 The pom.xml, line by line

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
```

```

<groupId>com.acme</groupId>
<artifactId>orders-service</artifactId>
<version>1.0.0-SNAPSHOT</version>

<properties>
  <!-- Java 21: virtual threads, records, pattern matching, sequenced collections -->
  <maven.compiler.release>21</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <quarkus.platform.group-id>io.quarkus.platform</quarkus.platform.group-id>
  <quarkus.platform.artifact-id>quarkus-bom</quarkus.platform.artifact-id>
  <quarkus.platform.version>3.33.1</quarkus.platform.version>

  <surefire-plugin.version>3.5.2</surefire-plugin.version>
  <compiler-plugin.version>3.13.0</compiler-plugin.version>
</properties>

<dependencyManagement>
  <dependencies>
    <!-- The BOM. This is the single most important element in the file:
         it pins consistent versions for ~500 extensions and their
         transitive dependencies. NEVER override an extension version
         by hand - you will get a combination nobody has ever tested. -->
    <dependency>
      <groupId>${quarkus.platform.group-id}</groupId>
      <artifactId>${quarkus.platform.artifact-id}</artifactId>
      <version>${quarkus.platform.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <!-- Note: no <version> anywhere. The BOM supplies them. -->
  <dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-rest-jackson</artifactId>
  </dependency>
  <dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-hibernate-orm-panache</artifactId>
  </dependency>
  <dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-jdbc-postgresql</artifactId>
  </dependency>
  <dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-smallrye-health</artifactId>
  </dependency>

  <dependency>

```



```

    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-junit5</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- The plugin that runs augmentation. Without it you have a
         pile of jars, not a Quarkus application. -->
    <plugin>
      <groupId>${quarkus.platform.group-id}</groupId>
      <artifactId>quarkus-maven-plugin</artifactId>
      <version>${quarkus.platform.version}</version>
      <extensions>true</extensions>
      <executions>
        <execution>
          <goals>
            <goal>build</goal>           <!-- augmentation -->
            <goal>generate-code</goal>   <!-- e.g. gRPC, OpenAPI clients -->
            <goal>generate-code-tests</goal>
          </goals>
        </execution>
      </executions>
    </plugin>

    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>${compiler-plugin.version}</version>
      <configuration>
        <compilerArgs>
          <!-- Keeps parameter names in bytecode. Quarkus, Jackson and
               Hibernate all benefit; without it you need @JsonProperty
               and @Param annotations everywhere. -->
          <arg>-parameters</arg>
        </compilerArgs>
      </configuration>
    </plugin>

    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>${surefire-plugin.version}</version>
      <configuration>
        <systemPropertyVariables>
          <java.util.logging.manager>org.jboss.logmanager.LogManager</java.util.logging.manager>
          <maven.home>${maven.home}</maven.home>
        </systemPropertyVariables>
      </configuration>
    </plugin>
  </plugins>
</build>

```

```

        </configuration>
      </plugin>
    </plugins>
  </build>

  <profiles>
    <!-- Activated with -Dnative. Keeps the slow build off the default path. -->
    <profile>
      <id>native</id>
      <activation>
        <property><name>native</name></property>
      </activation>
      <properties>
        <skipITs>false</skipITs>
        <quarkus.native.enabled>true</quarkus.native.enabled>
      </properties>
      <build>
        <plugins>
          <plugin>
            <artifactId>maven-failsafe-plugin</artifactId>
            <version>${surefire-plugin.version}</version>
            <executions>
              <execution>
                <goals>
                  <goal>integration-test</goal>
                  <goal>verify</goal>
                </goals>
                <configuration>
                  <systemPropertyVariables>
                    <native.image.path>
                      ${project.build.directory}/${project.build.finalName}-runner
                    </native.image.path>
                  </systemPropertyVariables>
                </configuration>
              </execution>
            </executions>
          </plugin>
        </plugins>
      </build>
    </profile>
  </profiles>
</project>

```

The `java.util.logging.manager` system property in Surefire is not boilerplate you can delete. Quarkus routes all logging through JBoss LogManager. If the JDK's default LogManager is installed first — which happens as soon as any test-scoped class touches `java.util.logging` — Quarkus cannot install its own, and you lose all log output in tests, or get a `LinkageError`. This exact line has cost many teams an afternoon.

1.7.4 First endpoint

Using Java 21 records for DTOs, which is the idiomatic modern style:

```

package com.acme.orders.api;

import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import java.math.BigDecimal;
import java.net.URI;
import java.util.List;

@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class OrderResource {

    private final OrderService orders;

    // Constructor injection: preferred over field injection because it makes
    // the dependency explicit and keeps the class unit-testable without a container.
    public OrderResource(OrderService orders) {
        this.orders = orders;
    }

    @GET
    public List<OrderView> list(@QueryParam("status") @DefaultValue("ALL") String status) {
        return orders.findByStatus(status);
    }

    @GET
    @Path("/{id}")
    public OrderView findById(@PathParam("id") long id) {
        return orders.findById(id)
            .orElseThrow(() -> new NotFoundException("No order " + id));
    }

    @POST
    public Response create(CreateOrderRequest request) {
        OrderView created = orders.create(request);
        return Response.created(URI.create("/orders/" + created.id()))
            .entity(created)
            .build();
    }
}

// Records as DTOs: immutable, concise, and Jackson handles them natively.
record CreateOrderRequest(String customerEmail, List<LineItem> items) { }
record LineItem(String sku, int quantity, BigDecimal unitPrice) { }
record OrderView(long id, String status, BigDecimal total, String customerEmail) { }

```

1.8 Dev Mode, Live Reload and Dev Services

1.8.1 Starting dev mode

```
./mvnw quarkus:dev      # or: quarkus dev
```

1.8.2 How live reload actually works

The common misconception is that Quarkus “hot swaps” classes with the JVM’s `HotSwap` mechanism. It does not — `HotSwap` cannot add methods or fields, which would make it nearly useless.

What actually happens:

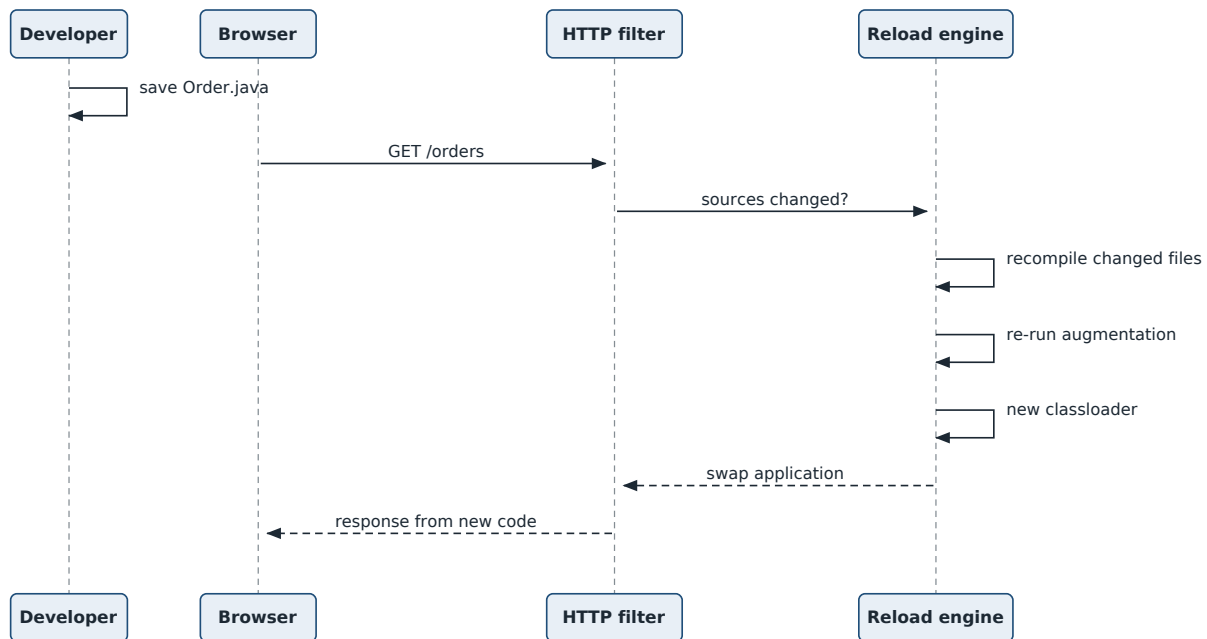


Figure 1.5 — Live reload is request-triggered, not file-watcher-triggered.

The mechanism in detail:

1. Quarkus installs a filter in front of the HTTP handler chain.
2. On each **incoming request** it checks source timestamps and checksums (throttled, so a burst of requests does not thrash).
3. If anything changed, it recompiles *only the changed files*, re-runs augmentation, discards the old application classloader and builds a new one.
4. The request then proceeds against the new code.

Design consequences you should be able to articulate:

- **Reload is lazy.** Saving a file does nothing until you hit an endpoint. This is why reload feels instantaneous — the cost is paid on a request you were about to make anyway.
- **A fresh classloader means no accumulated state.** Unlike a hot-swap approach, there is no drift between what is loaded and what is on disk. Static fields are genuinely reset.

- **Because augmentation re-runs, build-time changes are picked up too** — a new extension in `pom.xml`, a changed build-time config property, a new entity. This is the part competing hot-reload solutions cannot do, and it is the strongest argument for the build-time model actually *improving* the developer loop rather than harming it.
- **It works for resources too**, including `application.properties` and templates.

1.8.3 Dev Services

If an extension needs infrastructure and you have not configured a URL for it, Quarkus starts it for you in a container via Testcontainers.

```
# In dev/test, DELETE this line rather than setting it.
# quarkus.datasource.jdbc.url=jdbc:postgresql://localhost:5432/orders
```

With no URL configured and `quarkus-jdbc-postgresql` on the classpath, `quarkus:dev` starts a PostgreSQL container, wires the datasource to it, and shuts it down on exit. The same applies to Kafka, Redis, MongoDB, Keycloak, Elasticsearch, RabbitMQ, Vault and more.

```
# Reuse the container across restarts for a faster loop
quarkus.devservices.enabled=true
quarkus.datasource.devservices.reuse=true
quarkus.datasource.devservices.image-name=postgres:16-alpine
quarkus.datasource.devservices.port=5433      # pin it if you want to attach a SQL client
```

Interview framing. Dev Services is not “Docker Compose but automatic”. The important property is that **the same mechanism runs in tests**, so `@QuarkusTest` runs against a real PostgreSQL rather than H2. That eliminates an entire category of bug — the one where your HQL works on H2 and fails on Postgres. When asked “how do you test database code?”, the strong answer is: real database, via Dev Services, with the test lifecycle managed by Quarkus.

1.8.4 Continuous testing

Press `r` in the dev-mode console and Quarkus runs the tests affected by your change, continuously, in the background.

```
Tests paused
Press [e] to edit command line args, [r] to resume testing,
      [o] Toggle test output, [:] for the terminal, [h] for more options>
```

It uses the same change-detection machinery as live reload to compute which tests are *affected* by the classes you touched, rather than re-running everything. On a large codebase this is the difference between a 2-second and a 4-minute feedback loop.

1.8.5 Dev UI

At `http://localhost:8080/q/dev-ui/` you get a per-extension console: view the resolved configuration with its source, browse every CDI bean and its injection points, inspect and modify the ArC container, run Flyway migrations, browse the Hibernate persistence units and generated DDL, view scheduled tasks, and open a Swagger UI.

The CDI page in particular is the fastest way to debug “why is my bean not injected” — it shows removed beans and *why* they were removed.

1.8.6 Other dev-mode facilities

```
# Attach a debugger: dev mode listens on 5005 by default
./mvnw quarkus:dev -Ddebug=5005
./mvnw quarkus:dev -Dsuspend          # wait for the debugger before starting

# Add an extension without editing pom.xml
quarkus extension add quarkus-smallrye-openapi

# Upgrade the whole project (Quarkus version, renamed artifacts, deprecated APIs)
quarkus update
```

`quarkus update` is worth knowing about: it applies OpenRewrite recipes that perform mechanical migrations — including the `resteasy-reactive` → `rest` rename discussed above.

1.9 Extensions

1.9.1 What an extension is

An extension is a library plus the build steps that integrate it into the augmentation model. Concretely, it is **two Maven modules**:

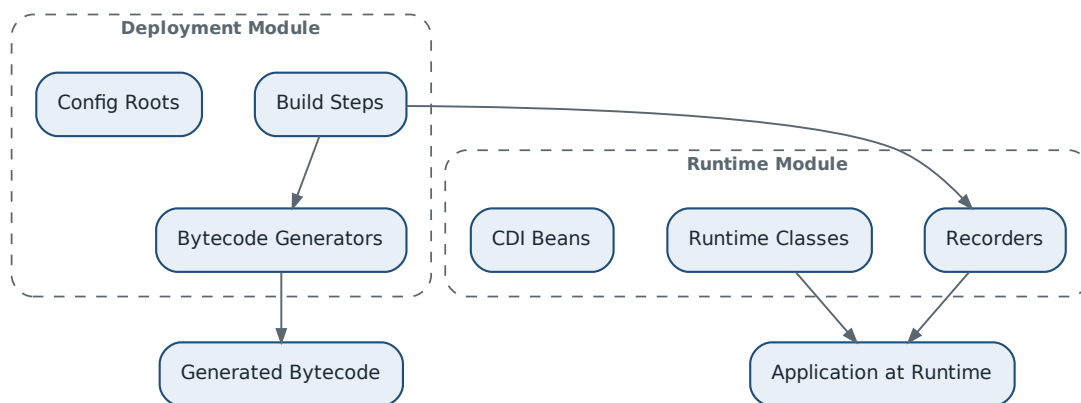


Figure 1.6 — Extension anatomy. The deployment module depends on the runtime module, never the reverse.

- **-deployment** — runs only during augmentation. Contains `@BuildStep` methods. Discarded after the build; never ships.
- **runtime** — ships with the application. Contains `@Recorder` classes, CDI beans, and the actual functionality.

The asymmetry is the point: all the heavy analysis machinery lives in a module that is deleted before the container image is built.

1.9.2 Why extensions exist rather than just “use the library”

You *can* add any Maven dependency to a Quarkus project and it will usually work in JVM mode. The extension buys you four things:

1. **Build-time initialisation** — the library’s setup cost is paid during `mvn package`.
2. **Native-image readiness** — reflection registrations, resource inclusions, substitutions for unsupported code.
3. **Configuration integration** — properties surfaced through SmallRye Config with validation and Dev UI visibility.
4. **Dev Services and Dev UI integration.**

A good interview answer to “what does a Quarkus extension actually do?” is: *it moves a library’s initialisation from runtime to build time and teaches GraalVM what the library does reflectively*. Everything else is convenience.

```
quarkus extension list --installable      # what is available
quarkus extension list --installed
quarkus extension add hibernate-validator
quarkus extension remove smallrye-openapi
quarkus extension categories
```

1.10 Configuration

Configuration in Quarkus is **SmallRye Config**, an implementation of MicroProfile Config with substantial extensions.

1.10.1 Sources and ordinals

Sources are consulted in descending **ordinal** order; the highest ordinal that defines a key wins.

Source	Ordinal
System properties (<code>-Dfoo=bar</code>)	400
Environment variables	300
<code>.env</code> file in the working directory	295
<code>\$PWD/config/application.properties</code>	260
<code>src/main/resources/application.properties</code>	250
<code>application.yaml</code> (with <code>quarkus-config-yaml</code>)	255
<code>META-INF/microprofile-config.properties</code>	100
Extension defaults	~0

The `config/application.properties` entry at 260 is worth remembering: drop a file in a `config/` directory next to your jar and it overrides the packaged configuration **without rebuilding**. That is the intended mechanism for on-premise deployments where a ConfigMap is not available.

Environment variable mapping. `quarkus.datasource.jdbc.url` becomes `QUARKUS_DATASOURCE_JDBC_URL`: uppercase, and every character that is not alphanumeric becomes `_`. For indexed properties, `quarkus.log.category."com.acme".level` becomes `QUARKUS_LOG_CATEGORY__COM_ACME__LEVEL`. This mapping is a reliably popular interview question because it is where Kubernetes deployments actually go wrong.

1.10.2 Reading configuration

The modern, type-safe approach is `@ConfigMapping` — an interface, validated at startup:

```
package com.acme.orders.config;

import io.smallrye.config.ConfigMapping;
import io.smallrye.config.WithDefault;
import io.smallrye.config.WithName;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import java.time.Duration;
import java.util.List;
import java.util.Optional;

@ConfigMapping(prefix = "orders")
public interface OrdersConfig {

    /** orders.max-line-items */
    @WithDefault("50")
    @Min(1) @Max(500)
    int maxLineItems();

    /** orders.checkout-timeout */
    @WithDefault("30s")
    Duration checkoutTimeout();

    /** orders.notify-emails - comma separated */
    List<String> notifyEmails();

    /** orders.legacy-endpoint - explicitly optional */
    Optional<String> legacyEndpoint();

    /** Nested group: orders.retry.* */
    Retry retry();

    interface Retry {
        @WithDefault("3") int maxAttempts();
        @WithDefault("PT1S") Duration backoff();
        @WithName("jitter-ms") @WithDefault("100") int jitterMillis();
    }
}
```



```

@ApplicationScoped
public class CheckoutService {
    private final OrdersConfig config;

    public CheckoutService(OrdersConfig config) {    // injected like any bean
        this.config = config;
    }

    public void validate(List<LineItem> items) {
        if (items.size() > config.maxLineItems()) {
            throw new BadRequestException("Too many line items");
        }
    }
}

```

The older, still-valid property-level style:

```

@ApplicationScoped
public class LegacyStyle {
    @ConfigProperty(name = "orders.max-line-items", defaultValue = "50")
    int maxLineItems;

    @ConfigProperty(name = "orders.legacy-endpoint")
    Optional<String> legacyEndpoint;    // Optional for genuinely optional keys
}

```

Prefer `@ConfigMapping`. It fails at *startup* with a clear message listing every missing or invalid property, rather than at first use. In a Kubernetes rollout, failing at startup means the readiness probe never passes and the rollout is halted automatically — a misconfigured pod never takes traffic. Being able to explain that operational consequence is the difference between a mid-level and a senior answer.

1.10.3 Profiles

Three profiles are built in: **dev** (quarkus:dev), **test** (@QuarkusTest), **prod** (everything else).

```

# Default for all profiles
orders.max-line-items=50
quarkus.http.port=8080

```

```

# Profile-specific overrides
%dev.quarkus.log.category."com.acme".level=DEBUG
%dev.quarkus.hibernate-orm.database.generation=drop-and-create

```

```

%test.quarkus.hibernate-orm.database.generation=drop-and-create
%test.orders.max-line-items=5

```

```

%prod.quarkus.hibernate-orm.database.generation=none
%prod.quarkus.log.console.json=true

```

```

# Multiple profiles share a value
%dev,test.orders.notify-emails=dev-null@acme.com

```

Custom profiles compose naturally:

```
java -Dquarkus.profile=staging -jar target/quarkus-app/quarkus-run.jar
# or
QUARKUS_PROFILE=staging ./target/orders-service-runner
```

```
%staging.quarkus.datasource.jdbc.url=jdbc:postgresql://staging-db:5432/orders
```

Quarkus 3 also supports **profile-aware files** — `application-staging.properties` beside `application.properties` — and **parent profiles**:

```
# staging inherits everything from prod, then overrides
%staging.quarkus.config.profile.parent=prod
```

1.10.4 Build-time vs runtime configuration — the trap

This is the concept most likely to bite you in production, and it is a favourite senior interview question.

Some properties are **fixed at build time**. Because augmentation used them to generate code, changing them later has no effect. Examples:

```
quarkus.datasource.db-kind=postgresql      # BUILD TIME - picks the driver
quarkus.http.root-path=/api                # BUILD TIME - bakes into routes
quarkus.hibernate-orm.packages=com.acme    # BUILD TIME
quarkus.native.additional-build-args=...   # BUILD TIME
```

Others are **runtime overridable**:

```
quarkus.datasource.jdbc.url=...            # RUNTIME
quarkus.datasource.username=...            # RUNTIME
quarkus.http.port=8080                     # RUNTIME
quarkus.log.level=INFO                     # RUNTIME
```

The failure mode: you build an image, deploy it, set `QUARKUS_HTTP_ROOT_PATH=/api/v2` in the Deployment manifest, and nothing changes. No error, no warning — the value was consumed at build time. In native mode Quarkus will at least warn you at startup that a build-time property was changed; in JVM mode it may be silent.

Every property in the Quarkus documentation is marked with a lock icon if it is build-time-fixed. **Check it before you plan a configuration strategy**, and treat build-time properties as part of the image identity, not as deployment configuration.

1.10.5 Secrets and expressions

```
# Reference another property
orders.callback-url=${quarkus.http.host}:${quarkus.http.port}/callbacks

# Default if unset
orders.region=${DEPLOY_REGION:eu-west-1}

# Fail fast if a required secret is missing at startup
quarkus.datasource.password=${DB_PASSWORD}
```

For real secret management, extensions exist for Vault ([quarkus-vault](#)) and Kubernetes Secrets ([quarkus-kubernetes-config](#)). Never commit secrets to `application.properties`; use `.env` locally (and gitignore it).

1.11 Logging

Quarkus uses **JBoss LogManager** as the backend with the **JBoss Logging** facade, and bridges SLF4J, JUL, Log4j and Commons Logging into it. You can use any of these APIs; they all end up in the same place.

```
import org.jboss.logging.Logger;

@ApplicationScoped
public class OrderService {
    private static final Logger LOG = Logger.getLogger(OrderService.class);

    public void ship(long id) {
        LOG.infof("Shipping order %d", id);           // printf-style, no string concat
        LOG.debugv("Payload: {0}", payload);         // MessageFormat style
    }
}
```

Or with Lombok-free Quarkus idiom:

```
import io.quarkus.logging.Log; // static, no field needed

@ApplicationScoped
public class OrderService {
    public void ship(long id) {
        Log.infof("Shipping order %d", id);
    }
}
```

`io.quarkus.logging.Log` is a build-time trick: the static calls are rewritten during augmentation into a per-class logger instance, so there is no runtime lookup cost. It is a neat, concrete example of augmentation used for ergonomics.

Configuration:

```
quarkus.log.level=INFO
quarkus.log.category."com.acme.orders".level=DEBUG
quarkus.log.category."org.hibernate.SQL".level=DEBUG

quarkus.log.console.enable=true
quarkus.log.console.format=%d{HH:mm:ss} %-5p [%c{2.}] (%t) %s%e%n
quarkus.log.console.level=DEBUG

# Structured JSON for production - requires quarkus-logging-json
%prod.quarkus.log.console.json=true
%prod.quarkus.log.console.json.pretty-print=false

# File logging with rotation
quarkus.log.file.enable=true
```

```
quarkus.log.file.path=/var/log/orders/app.log
quarkus.log.file.rotation.max-file-size=10M
quarkus.log.file.rotation.max-backup-index=5
```

| In Kubernetes, log to stdout as JSON and let the platform handle the rest. File logging inside a container is nearly always wrong: it fills the ephemeral layer, is lost when the pod dies, and duplicates work the cluster's log agent already does.

1.12 Application Lifecycle

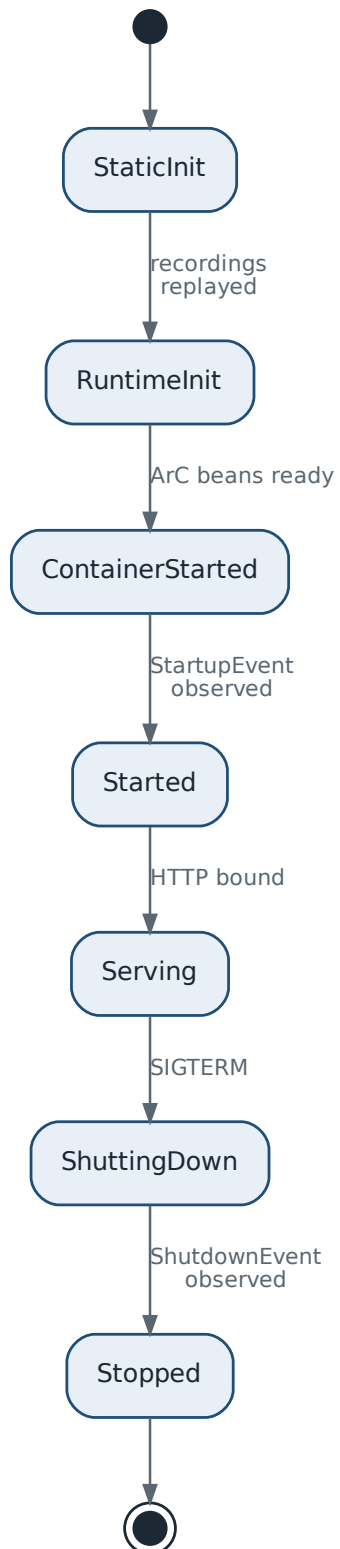


Figure 1.7 — Application lifecycle states.

1.12.1 Lifecycle hooks

```
package com.acme.orders.infra;

import io.quarkus.runtime.StartupEvent;
import io.quarkus.runtime.ShutdownEvent;
import io.quarkus.runtime.Startup;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.event.Observes;
import jakarta.annotation.Priority;
import io.quarkus.logging.Log;

@ApplicationScoped
public class LifecycleHooks {

    // Runs once the CDI container is up and all runtime init has completed.
    void onStart(@Observes StartupEvent event) {
        Log.info("orders-service ready");
    }

    // Ordered startup: lower priority runs first.
    void warmCaches(@Observes @Priority(100) StartupEvent event) {
        Log.info("Warming caches before anything else");
    }

    // Runs on SIGTERM, before the HTTP listener closes.
    void onStop(@Observes ShutdownEvent event) {
        Log.info("orders-service shutting down");
    }
}

// @Startup forces eager instantiation of a normally lazy bean.
// Without it, an @ApplicationScoped bean is only created on first use.
@Startup
@ApplicationScoped
public class SchemaValidator {
    public SchemaValidator(DataSource ds) {
        // runs at startup, not on first request
    }
}
```

1.12.2 Taking over main

For CLI tools, batch jobs and operators:

```
package com.acme.orders;

import io.quarkus.runtime.Quarkus;
import io.quarkus.runtime.QuarkusApplication;
import io.quarkus.runtime.annotations.QuarkusMain;
```

```

@QuarkusMain
public class Main implements QuarkusApplication {

    public static void main(String... args) {
        Quarkus.run(Main.class, args);
    }

    @Override
    public int run(String... args) {
        // Container is up; beans are injectable here.
        System.out.println("Running with args: " + String.join(" ", args));

        // For a server: block until shutdown is requested.
        Quarkus.waitForExit();
        return 0;                // process exit code
    }
}

```

For a job that should exit when done, simply `return 0`; without `waitForExit()`. Combine with `quarkus-picocli` for proper CLI argument handling.

1.12.3 Graceful shutdown

```

# Finish in-flight requests before exiting. Essential in Kubernetes.
quarkus.shutdown.timeout=30S

```

```

# Optional: keep serving for a few seconds after SIGTERM so that the
# Service endpoints controller has time to remove this pod from rotation.
quarkus.shutdown.delay=5S

```

Without `shutdown.delay`, there is a race in Kubernetes: the kubelet sends `SIGTERM` at the same time as the endpoints controller starts removing the pod. If your app exits instantly, requests already routed to it are dropped. A short delay closes that window. This is a favourite “have you actually run this in production?” question.

1.13 Packaging

```

quarkus.package.jar.type=fast-jar    # default; layered, container-friendly
quarkus.package.jar.type=uber-jar    # one fat jar; simple but poor layering
quarkus.package.jar.type=mutable-jar # supports remote dev mode

```

```

./mvnw package                                # fast-jar
java -jar target/quarkus-app/quarkus-run.jar

./mvnw package -Dquarkus.package.jar.type=uber-jar
java -jar target/orders-service-1.0.0-SNAPSHOT-runner.jar

./mvnw package -Dnative                        # needs GraalVM/Mandrel locally
./mvnw package -Dnative -Dquarkus.native.container-build=true # needs only Docker/Podman
./target/orders-service-1.0.0-SNAPSHOT-runner

```

`-Dquarkus.native.container-build=true` is the pragmatic choice for CI: it runs the native build inside a Mandrel builder image, so your agents need Docker but not a GraalVM installation, and the build is reproducible across developer machines and CI.

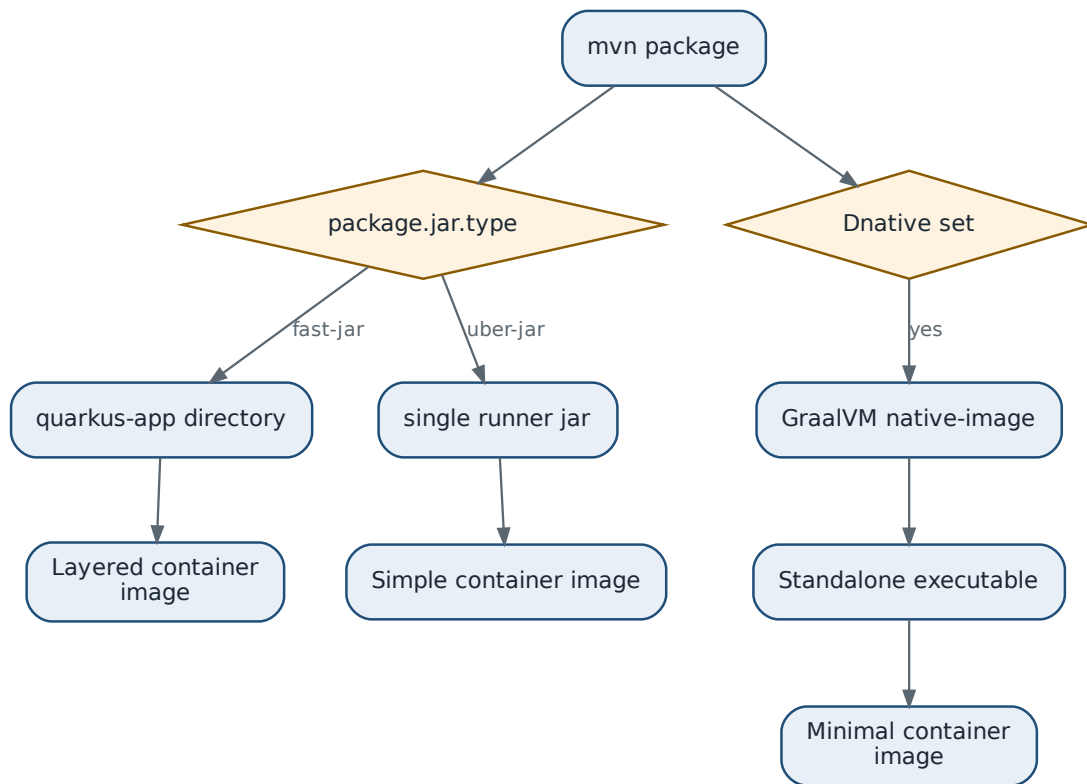


Figure 1.8 — Packaging outputs and their container implications.

1.14 Quarkus vs Spring Boot

Interviewers ask this constantly. A weak answer lists features. A strong answer contrasts **design philosophies** and is honest about trade-offs.

Dimension	Spring Boot	Quarkus
DI resolution	Runtime, reflective	Build time, generated bytecode
Bean discovery	Component scan from main class	Jandex index of the whole app
Startup (typical REST + JPA)	2.5 – 6 s	0.6 – 1.2 s JVM · 0.02 s native
Idle RSS	250 – 450 MB	90 – 180 MB JVM · 20 – 50 MB native

Dimension	Spring Boot	Quarkus
Native support	Spring Boot 3 AOT + GraalVM	First-class since inception
Hot reload	DevTools (restart-based)	Request-triggered re-augmentation
Test infra	Testcontainers, wired by hand	Dev Services, automatic
Reactive story	WebFlux (parallel stack)	Mutiny on Vert.x (one unified stack)
Config	<code>@Value</code> , <code>@Configuration Properties</code>	MicroProfile Config, <code>@ConfigMapping</code>
Ecosystem breadth	Larger	Large and growing; some gaps remain
Hiring pool	Very large	Smaller

Be honest about where Spring wins. Ecosystem maturity, the sheer volume of Stack Overflow answers, and the availability of engineers who already know it are real, material advantages. A senior engineer who cannot name a competitor’s strengths reads as a partisan rather than an architect.

Migration note. Quarkus provides Spring compatibility extensions — `quarkus-spring-di`, `quarkus-spring-web`, `quarkus-spring-data-jpa`, `quarkus-spring-security` — which accept `@Autowired`, `@RestController`, `@RepositoryRestResource` and friends. These exist to **shorten a migration**, not to be a permanent home. They map Spring annotations onto Quarkus mechanisms at build time and do not cover the whole Spring surface. The correct use is: migrate with them, then remove them incrementally. Chapter 24 covers migration strategy properly.

1.15 Best Practices

1. **Pin everything through the BOM.** Never hand-specify a version for a `io.quarkus:*` artifact.
2. **Use `@ConfigMapping`, not `@ConfigProperty`,** for anything with more than a couple of keys. Fail at startup, not at first request.
3. **Know which properties are build-time-fixed** before designing your deployment configuration.
4. **Prefer constructor injection.** It documents dependencies, permits `final` fields, and makes plain-JUnit unit tests possible without a container.
5. **Delete infrastructure URLs from dev/test config** and let Dev Services provide real backing services.
6. **Add `-parameters` to the compiler.** It removes an entire class of “why is my JSON field null” issues.
7. **Keep the `java.util.logging.manager` system property in Surefire.**
8. **Log to stdout as JSON in production,** never to files inside containers.
9. **Build native in CI, not on laptops** — use `-Dquarkus.native.container-build=true` for reproducibility.

10. Set `quarkus.shutdown.timeout` and `quarkus.shutdown.delay` for any service behind a Kubernetes Service.
11. **Index external modules** that contain beans or entities: add the Jandex plugin, or list them under `quarkus.index-dependency`.
12. **Run `quarkus update` for version upgrades** rather than editing the version property by hand; it applies mechanical migrations you would otherwise miss.

1.16 Common Mistakes

1. Beans in an external JAR are silently ignored. No Jandex index means no discovery. Either add the plugin to that module:

```
<plugin>
  <groupId>io.smallrye</groupId>
  <artifactId>jandex-maven-plugin</artifactId>
  <version>3.2.2</version>
  <executions>
    <execution><id>make-index</id><goals><goal>jandex</goal></goals></execution>
  </executions>
</plugin>
```

or index it from the consuming application:

```
quarkus.index-dependency.shared.group-id=com.acme
quarkus.index-dependency.shared.artifact-id=shared-domain
```

2. Expecting a build-time property to be overridable at runtime. See §1.10.4. Symptom: the environment variable is set correctly and has no effect.

3. Blocking an event loop. A JDBC call, `Thread.sleep`, or a synchronous HTTP client inside a `Uni`-returning endpoint without `@Blocking`. Symptom: throughput collapses under load and the log shows `Thread Thread[vert.x-eventloop-thread-0] has been blocked for N ms`. Take that warning seriously — it is telling you a fraction of your total I/O capacity is gone.

4. Assuming JVM behaviour carries to native. Reflection, resource loading, dynamic proxies and service loaders all need registration. Always run the integration test suite against the native binary with `@QuarkusIntegrationTest`.

5. Making an `@ApplicationScoped` bean `final`, or giving it only a `private/arg-only` constructor. Normal-scoped beans need a client proxy, which requires a non-final class and a no-arg constructor (which the compiler supplies only if you declare no other constructor — so add one explicitly, or use `@Singleton` where a proxy is not needed).

6. `@Singleton` vs `@ApplicationScoped` confusion. `@Singleton` has no client proxy: it is created eagerly on first injection and injected directly. `@ApplicationScoped` is lazy and proxied. Injecting an `@ApplicationScoped` bean into a `@Singleton` is fine; the reverse can create the singleton earlier than you expect.

7. Committing `.env`. It has ordinal 295 — above your `application.properties` — so a stale `.env` on a colleague's machine silently overrides real configuration.

8. Using `resteasy-reactive` artifact names in new code. They still resolve via relocation, but they signal to a reviewer that the code was written against pre-3.9 material.

1.17 Interview Questions

Junior

Q1. What is Quarkus? A Kubernetes-native Java framework built on Jakarta EE 10 and MicroProfile that moves framework initialisation from runtime to build time. The result is fast startup and low memory, in both JVM mode and as a GraalVM native executable.

Follow-up: “How is it different from a normal Java framework?” → It does classpath scanning, DI resolution and framework bootstrapping during the Maven build instead of at process start.

Q2. What are the main benefits? Fast startup, low memory footprint, native-image support, live reload with a genuinely fast dev loop, Dev Services, and standards-based APIs (Jakarta EE, MicroProfile) so the knowledge transfers.

Q3. What is dev mode? `./mvnw quarkus:dev` starts the app with live reload, continuous testing, Dev UI and Dev Services. Code changes are picked up on the next HTTP request without a manual restart.

Q4. What is `application.properties` and where does it live? `src/main/resources/application.properties`. It is one of several config sources; system properties and environment variables override it.

Q5. What is a Quarkus extension? A module that integrates a library with Quarkus — moving its initialisation to build time, making it native-image-safe, and exposing its configuration.

Q6. How do you inject a dependency? `@Inject` on a field, or preferably via the constructor, on a CDI bean annotated `@ApplicationScoped`, `@Singleton`, `@RequestScoped` or `@Dependent`.

Q7. What is the difference between JVM mode and native mode? JVM mode produces a jar run by a JVM: higher peak throughput, slower start, more memory. Native mode produces a standalone executable via GraalVM: near-instant start, much lower memory, slightly lower peak throughput, and a closed-world constraint.

Mid-level

Q8. Why does Quarkus do dependency injection at build time? Because the bean graph is fully determined by your code and does not change at runtime. Resolving it at build time removes classpath scanning, reflection and the retained bean metadata from the runtime image. It also enables dead-bean elimination and makes the graph analysable by GraalVM’s closed-world analysis.

Follow-up: “What do you give up?” → Runtime-dynamic bean registration. You cannot add a bean based on a value read from a database at startup; you would use a producer method with a runtime condition instead.

Q9. Explain static init vs runtime init. Static init runs deterministic, environment-independent setup — building an ORM metamodel, compiling serialisers. Runtime init runs anything that touches the environment — opening sockets, reading env vars, creating pools. In native mode static init runs **at image build time** and its object graph is serialised into the executable, which is why a socket or a `Random` in static init is a build failure.

Q10. How does live reload work? On each incoming request, Quarkus checks whether sources changed; if so it recompiles the changed files, re-runs augmentation, and swaps in a fresh application

classloader. It is request-triggered and classloader-based, not JVM [HotSwap](#), which is why it can pick up new extensions and build-time config changes.

Q11. What are Dev Services? When an extension needs infrastructure and no connection URL is configured, Quarkus starts a container for it via Testcontainers in dev and test. It works for PostgreSQL, Kafka, Redis, Keycloak, MongoDB and others, and it means tests run against the real technology.

Q12. How is configuration resolved? SmallRye Config consults sources in descending ordinal order: system properties (400), environment variables (300), [.env](#) (295), [config/application.properties](#) (260), [application.properties](#) (250), [microprofile-config.properties](#) (100). Profiles prefix keys with [%dev.](#), [%test.](#), [%prod.](#) or a custom name.

Q13. What is Arc? Quarkus's build-time CDI 4.0 container. It resolves the bean graph during augmentation, generates a class per bean, removes unused beans, and creates client proxies for normal-scoped beans — with no runtime reflection.

Q14. Why must an [@ApplicationScoped](#) bean not be [final](#)? Normal-scoped beans are injected as generated client proxies that subclass the bean, so the class must be extensible and have a no-arg constructor.

Q15. What is [@RegisterForReflection](#) for? Native-image builds delete unreachable code. If a class is only reached reflectively — say, deserialised by name — the analysis cannot see it. [@RegisterForReflection](#) tells the build to keep it and retain its metadata.

Q16. How do you package for a container? [fast-jar](#) (the default) splits output into [lib/](#), [app/](#) and [quarkus/](#) so Docker layers cache dependencies separately from application code. Copy them in separate [COPY](#) instructions so a code change re-uploads only a small layer.

Q17. What is the Quarkus BOM and why does it matter? [quarkus-bom](#) pins consistent, tested versions for all extensions and their transitive dependencies. Overriding a single version by hand produces a combination that was never tested together, which is a common source of obscure runtime failures.

Senior

Q18. Walk me through what happens during [mvn package](#) on a Quarkus project. Maven compiles sources; [quarkus-maven-plugin:build](#) then runs augmentation in an isolated deployment classloader. Quarkus builds a Jandex index of the application and indexed dependencies, then executes the build-step graph — steps declare their inputs and outputs as build items and the engine sorts them topologically and runs independent branches in parallel. Steps produce generated bytecode (CDI bean classes, JAX-RS route dispatchers, Hibernate metamodel), reflection and resource registrations, and — via recorders — a serialised bootstrap sequence split into static-init and runtime-init instructions. The output is written as [target/quarkus-app/](#). The deployment classloader and everything in it is then discarded.

Q19. Explain the recorder pattern and why it is necessary. A build step runs at build time in a different classloader from the application; it cannot invoke runtime code directly. A [@Recorder](#) class solves this: calls to its methods during augmentation are intercepted by a bytecode recorder and turned into instructions in a generated class, rather than being executed. [RuntimeValue<T>](#) represents a value that will exist only at runtime, so it cannot be inspected during augmentation.

Recorder arguments must be serialisable into bytecode, which is why build-time config classes are simple data holders.

Q20. How would you diagnose a Quarkus service that starts in 5 seconds despite being native? Native startup is normally tens of milliseconds, so 5 seconds means the time is being spent in *runtime* init, not in the framework. I would: (a) enable `quarkus.log.level=DEBUG` for `io.quarkus.runtime` and look at the startup timing breakdown; (b) check for DNS resolution or TLS handshakes against dependencies that are slow or timing out — a common cause is a connection pool eagerly opening N connections to a database behind a cold proxy; (c) check for eager `@Startup` beans doing real work such as cache warming or schema validation; (d) check whether Kubernetes-side latency is being misattributed — image pull time, init containers, and readiness probe `initialDelaySeconds` all show up as “slow startup” on a dashboard; (e) look for a `Random`/entropy stall, historically `/dev/random` blocking in containers. The systematic move is to instrument first — `StartupEvent` timing per bean — rather than guess.

Q21. What are the trade-offs of build-time augmentation? Gains: startup time, memory, native-image feasibility, dead-code elimination, and early failure — misconfiguration surfaces during the build rather than in production. Costs: the build is slower and more complex; some properties become immutable after build, which complicates “build once, deploy everywhere” if you are careless; runtime-dynamic behaviour (registering beans based on runtime data, dynamic class loading) is restricted; and debugging generated bytecode is harder than debugging a reflective framework. The extension model also means a library without an extension may need manual native configuration.

Q22. How does Quarkus decide whether an endpoint runs on an event loop or a worker thread? At build time, from the method signature: reactive return types (`Uni`, `Multi`) imply non-blocking and are dispatched on the event loop; other return types imply blocking and are dispatched to a worker thread. `@Blocking` and `@NonBlocking` override the heuristic. The generated dispatcher encodes the decision, so there is no runtime inspection cost. The failure mode is a blocking call inside a `Uni` method without `@Blocking`, which stalls an event loop thread and degrades throughput server-wide.

Q23. Your team wants “build once, deploy everywhere”. How do build-time properties affect that, and how do you manage it? Build-time-fixed properties become part of the image’s identity. Anything that must differ per environment — database URLs, credentials, external endpoints, log levels, ports — must be a runtime property, supplied by env vars, ConfigMaps or Secrets. I would audit the configuration and classify every key as build-time or runtime; keep build-time properties in `application.properties` under version control; and forbid environment-specific values there. Where a genuinely build-time concern differs by environment (for instance a different database *kind*), that is a signal the environments are not comparable and the difference should be eliminated rather than configured around. I would also add a startup assertion that logs the resolved values of critical properties and their source, so a misconfiguration is visible in the first ten lines of the pod log.

Q24. When would you *not* use Quarkus? When the team has deep Spring expertise and the service is not startup- or memory-sensitive — the migration cost would not pay back. When a critical dependency has no extension and poor native support and you need native. When the organisation’s tooling, security scanning and runbooks are built around a specific stack and the operational cost of divergence exceeds the runtime savings. Also for a monolith that starts once

and runs for months on a large machine, where the metrics Quarkus optimises for simply do not matter.

Principal

Q25. You are migrating 40 Spring Boot microservices to Quarkus. Design the programme. I would not migrate 40 services. I would:

Establish the case. Measure current cost: replica counts, memory requests, cold-start impact on autoscaling and deploy duration. Convert to a monthly figure. If the number is not compelling, stop here — this is the discipline that distinguishes an architect from an enthusiast.

Segment the estate. Rank services by (memory footprint $\times$ replica count) and by startup sensitivity. Serverless and burst-scaled services benefit most; a low-traffic admin service benefits least.

Pilot. Pick two services: one straightforward (REST + JPA) and one that exercises a hard dependency (a proprietary client, a messaging integration). Migrate both fully, including CI, observability, runbooks and on-call documentation. The second service is the one that tells you the real cost.

Build the paved road. A Quarkus starter archetype, shared extensions for the organisation’s cross-cutting concerns, CI templates with native builds, Grafana dashboards, and a migration playbook derived from the pilot. Without this, service 3 through 40 each re-solve the same problems.

Sequence by value and risk. Migrate high-benefit, low-risk services first to build confidence and bank savings early. Use the Spring compatibility extensions as a *transitional* device with an explicit removal deadline, not as a destination.

Accept a permanent mixed estate. Some services will not be worth migrating. That is a correct outcome, not a failure. Set an explicit “we stop here” criterion up front so the programme has a defined end.

Decide native per service, not globally. Default to JVM mode; promote to native where the measurements justify the build-time and tooling cost.

Q26. How would you design an internal Quarkus extension for your organisation, and what would you put in it? The case for one is repeated cross-cutting concern: correlation-ID propagation, a standard audit trail, the organisation’s authentication conventions, standard metrics tags. I would put in it: build steps that discover annotated types via the Jandex index and generate the wiring; recorders for anything needing runtime state; a `@ConfigMapping` config root so it is configurable and Dev-UI-visible; reflection registrations so it works in native; and a Dev Services integration if it depends on infrastructure. Critically, I would keep the deployment module free of anything the runtime needs, so the analysis machinery does not ship. I would also invest in tests using `QuarkusUnitTest`, which runs augmentation in-process and lets you assert on generated behaviour — extensions fail in ways application code does not, and untested build steps are a maintenance liability.

Q27. Argue against adopting Quarkus in a company that is currently all-Spring. The runtime savings are real but bounded; the organisational costs are diffuse and ongoing. You fragment your engineering population into two stacks, which doubles the surface area of shared libraries, security patching, observability integration, and onboarding. Hiring gets harder. Every internal platform tool — CI templates, base images, tracing conventions, lint rules — must be built twice or abstracted, and abstractions across two frameworks tend to be poor. The Spring

ecosystem's depth means that for any given problem there is likely an existing, battle-tested answer; in Quarkus you may be the first person at your company to hit it. Unless memory or cold-start cost is a genuine, quantified constraint — serverless workloads, very high replica counts, or an edge deployment with hard memory limits — the honest recommendation is to stay on one stack and invest the same effort in tuning it. *The ability to make this argument credibly is itself a strong signal in a principal-level interview.*

1.18 Cheat Sheet

CLI

```
quarkus create app com.acme:svc --java=21 --extension='rest-jackson'
quarkus dev                                # dev mode + live reload
quarkus build                              # package
quarkus build --native                     # native executable
quarkus extension list --installable
quarkus extension add hibernate-validator
quarkus update                             # upgrade Quarkus + apply migrations
```

Maven

```
./mvnw quarkus:dev
./mvnw quarkus:dev -Ddebug=5005
./mvnw package
./mvnw package -Dnative
./mvnw package -Dnative -Dquarkus.native.container-build=true
./mvnw quarkus:add-extension -Dextensions='smallrye-openapi'
```

Dev-mode keys

[r] resume/start continuous testing	[o] toggle test output
[s] force restart	[d] open Dev UI
[h] help	[q] quit
[:] enter the interactive terminal	

Essential endpoints

/q/dev-ui/	Dev UI (dev mode only)
/q/health	aggregate health
/q/health/live	liveness
/q/health/ready	readiness
/q/metrics	Micrometer/Prometheus
/q/openapi	OpenAPI document
/q/swagger-ui	Swagger UI
/q/info	build & git info (quarkus-info)

Configuration essentials

```
quarkus.http.port=8080
quarkus.http.root-path=/api                # BUILD TIME
quarkus.log.level=INFO
quarkus.log.category."com.acme".level=DEBUG
quarkus.shutdown.timeout=30S
quarkus.shutdown.delay=5S
```

```
quarkus.package.jar.type=fast-jar
%dev.quarkus.log.console.level=DEBUG
%prod.quarkus.log.console.json=true
```

Lifecycle

```
void onStart(@Observes StartupEvent e) { }
void onStop(@Observes ShutdownEvent e) { }
@Startup @ApplicationScoped class Eager { }
@QuarkusMain class Main implements QuarkusApplication { }
```

Ordinals (highest wins)

```
400 system properties
300 environment variables
295 .env
260 config/application.properties (next to the jar)
255 application.yaml
250 src/main/resources/application.properties
100 META-INF/microprofile-config.properties
```

1.19 Summary

Quarkus is one idea executed with unusual discipline: **framework work that can be done at build time should be done at build time, and the machinery that did it should not ship.**

Everything else follows. Build-time DI resolution gives fast startup and enables dead-bean elimination. Build-time metadata generation removes reflection, which is what makes GraalVM's closed-world analysis tractable. Recorders bridge the two worlds by writing bytecode instead of executing. The static-init/runtime-init split exists because native mode snapshots the heap at build time, so anything environment-dependent must be deferred. Live reload re-runs augmentation rather than hot-swapping classes, which is why it can pick up changes no other reload mechanism can.

Understanding this one idea lets you predict Quarkus's behaviour in situations the documentation does not cover — which is precisely what interviewers are probing for, and precisely what makes you effective on day two of a real project.

Chapter 2 applies it to the HTTP layer: Quarkus REST (formerly RESTEasy Reactive), the build-time route generation that makes it fast, and the blocking/non-blocking execution model that is the most common source of production incidents in Quarkus applications.

1.20 References

- Quarkus Guides — <https://quarkus.io/guides/>
- Quarkus 3.33 LTS announcement — <https://quarkus.io/blog/quarkus-3-33-released/>
- Quarkus 3.9 “Big Reactive Rename” — <https://quarkus.io/blog/quarkus-3-9-1-released/>
- Writing your own extension — <https://quarkus.io/guides/writing-extensions>
- Configuration reference — <https://quarkus.io/guides/config-reference>
- Building a native executable — <https://quarkus.io/guides/building-native-image>

- Dev Services — <https://quarkus.io/guides/dev-services>
- ArC / CDI reference — <https://quarkus.io/guides/cdi-reference>
- Jandex — <https://github.com/smallrye/jandex>
- Red Hat build of Quarkus 3.33 release notes — https://docs.redhat.com/en/documentation/red_hat_build_of_quarkus/3.33/

Quarkus REST (RESTEasy Reactive)

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-rest` · `quarkus-rest-jackson` · `quarkus-rest-client-jackson` · Jakarta REST 3.1

Learning Objectives

By the end of this chapter you will be able to:

1. Explain the naming history — RESTEasy Classic, RESTEasy Reactive, Quarkus REST — and pick the right extension.
2. Describe how Quarkus generates the HTTP routing layer **at build time**, and why that removes reflection from the request path.
3. Predict, from a method signature alone, whether an endpoint runs on an **event loop** or a **worker thread**, and explain the consequences of getting it wrong.
4. Write resources using Java 21 records, `Uni`, `Multi` and `RestResponse<T>` idiomatically.
5. Build a coherent error-handling strategy with `ExceptionHandler` and RFC 7807 problem details.
6. Use filters, interceptors and `@ServerRequestFilter` / `@ServerResponseFilter` without reintroducing reflection.
7. Consume APIs with the declarative **REST Client**, including timeouts, retries and fault tolerance.
8. Stream responses with `Multi` and Server-Sent Events, and know when streaming is the wrong tool.
9. Handle multipart uploads safely.
10. Test the HTTP layer with RestAssured and `@QuarkusTest`, including native-mode verification.
11. Answer 25+ interview questions on the REST layer, including the execution-model questions that separate levels.

2.1 Naming: Three Things That Are Easy To Confuse

This trips people up constantly, including in interviews, so let us fix it first.

Name	Artifact (Quarkus 3.9+)	What it is
RESTEasy Classic	<code>quarkus-resteasy</code>	The original blocking, servlet-derived JAX-RS implementation. Reflection-heavy, thread-per-request. Still supported; not the default.
RESTEasy Reactive	<i>(old name)</i>	A ground-up rewrite for Quarkus: build-time route generation, runs on Vert.x, supports both blocking and non-blocking.
Quarkus REST	<code>quarkus-rest</code>	The same thing as RESTEasy Reactive , renamed in Quarkus 3.9 because “Reactive” wrongly implied you had to write reactive code.

So: *RESTEasy Reactive* and *Quarkus REST* are the **same implementation under two names**. The rename in Quarkus 3.9 also touched the companions:

```

quarkus-resteasy-reactive      -> quarkus-rest
quarkus-resteasy-reactive-jackson -> quarkus-rest-jackson
quarkus-resteasy-reactive-jsonb -> quarkus-rest-jsonb
quarkus-resteasy-reactive-qute -> quarkus-rest-qute
quarkus-rest-client-reactive    -> quarkus-rest-client
quarkus-rest-client-reactive-jackson -> quarkus-rest-client-jackson
quarkus-resteasy              -> quarkus-resteasy-classic

```

Maven relocations keep the old coordinates resolving, so nothing breaks. But new code should use the new names, and `quarkus update` will rewrite them for you.

Which do I choose? `quarkus-rest`, unless you have a hard dependency on a RESTEasy Classic-only feature (some servlet integrations, a few legacy providers). Quarkus REST is faster, uses less memory, works better in native mode, and supports the imperative programming style perfectly well. “Reactive” in the old name was about the *engine*, not about your code.

2.2 The Execution Model

This is the single most important section in the chapter. Most Quarkus production incidents that are not database-related trace back to a misunderstanding here.

2.2.1 One engine, two thread pools

Quarkus REST runs on Vert.x, which runs on Netty. There are two pools that matter:

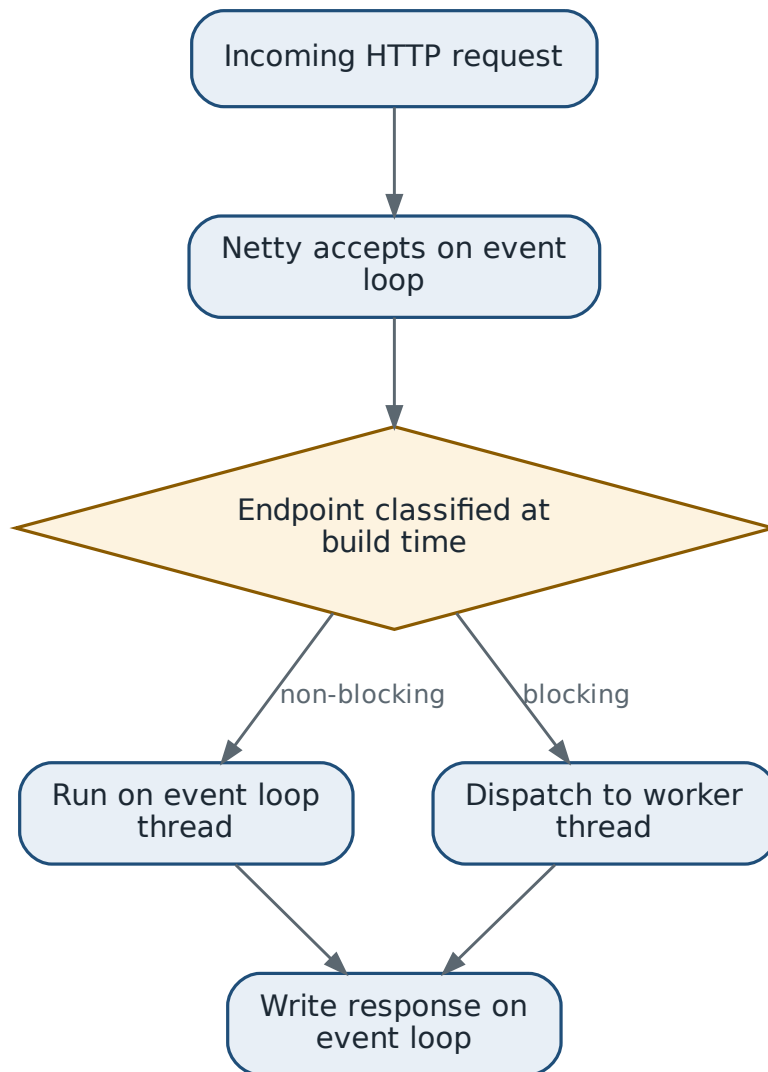


Figure 2.1 — Request dispatch. The classification decision is made during augmentation, not per request.

Event loop threads. A small, fixed pool — by default $2 \times \text{available processors}$. Every socket is bound to exactly one event loop for its lifetime. These threads must never block. Blocking one does not slow down one request; it stalls **every connection assigned to that event loop**.

Worker threads. An elastic pool (default max 200) for blocking work: JDBC, blocking HTTP clients, file I/O, `Thread.sleep`, anything that parks.

```

quarkus.vertx.event-loops-pool-size=8      # default: 2 x cores
quarkus.thread-pool.max-threads=200       # worker pool ceiling
quarkus.thread-pool.core-threads=1
  
```

```
quarkus.thread-pool.queue-size=                # unbounded by default
quarkus.vertx.max-event-loop-execute-time=2S # warn threshold
```

2.2.2 How Quarkus classifies your endpoint

The decision is made **at build time** from the method signature:

Signature	Runs on	Why
<code>Uni<T> m()</code>	Event loop	Reactive return type $\Rightarrow$ assumed non-blocking
<code>Multi<T> m()</code>	Event loop	As above
<code>CompletionStage<T> m()</code>	Event loop	As above
<code>T m()</code> (plain type)	Worker	Assumed to block
<code>void m()</code>	Worker	Assumed to block
<code>@Blocking Uni<T> m()</code>	Worker	Explicit override
<code>@NonBlocking T m()</code>	Event loop	Explicit override
<code>@RunOnVirtualThread T m()</code>	Virtual thread	Java 21 virtual thread per request

```
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
public class OrderResource {

    @Inject OrderRepository repo;           // blocking JDBC
    @Inject ReactiveOrderClient client;     // non-blocking

    // WORKER THREAD. Correct: JDBC blocks.
    @GET @Path("/{id}")
    public OrderView byId(@RestPath long id) {
        return repo.findById(id);
    }

    // EVENT LOOP. Correct: the client never blocks.
    @GET @Path("/{id}/tracking")
    public Uni<TrackingView> tracking(@RestPath long id) {
        return client.track(id);
    }

    // EVENT LOOP - AND THIS IS A BUG.
    // repo.findById is JDBC; it blocks an event loop thread.
    @GET @Path("/{id}/broken")
    public Uni<OrderView> broken(@RestPath long id) {
        return Uni.createFrom().item(repo.findById(id)); // <-- blocks!
    }
}
```

```

// Fixed: force it onto a worker thread.
@GET @Path("/{id}/fixed")
@Blocking
public Uni<OrderView> fixed(@RestPath long id) {
    return Uni.createFrom().item(repo.findById(id));
}

// Better fix: keep it off the event loop but stay reactive.
@GET @Path("/{id}/better")
public Uni<OrderView> better(@RestPath long id) {
    return Uni.createFrom().item(() -> repo.findById(id))
        .runSubscriptionOn(Infrastructure.getDefaultWorkerPool());
}

// Java 21 virtual threads: imperative code, cheap concurrency.
@GET @Path("/{id}/loom")
@RunOnVirtualThread
public OrderView loom(@RestPath long id) {
    return repo.findById(id);
}
}

```

2.2.3 What blocking an event loop actually looks like

Quarkus tells you, loudly:

```

WARN [io.ver.cor.imp.BlockedThreadChecker] Thread Thread[vert.x-eventloop-thread-3,5,main]
has been blocked for 2871 ms, time limit is 2000 ms
io.vertx.core.VertxException: Thread blocked
    at java.base@21.0.5/java.net.SocketInputStream.socketRead0(Native Method)
    ...

```

Treat this as a production incident, not a warning. With 8 event loops, one blocked loop removes 12.5 % of your server's total I/O capacity — for *every* connection pinned to it, not just the request that misbehaved. Under load the symptom is a throughput cliff and a p99 latency explosion that looks nothing like a slow endpoint.

Diagnostics:

```

# Fail fast in dev/test so you catch it before production
%dev.quarkus.vertx.max-event-loop-execute-time=1S
%dev.quarkus.vertx.warning-exception-time=1S

```

2.2.4 Virtual threads (Java 21)

`@RunOnVirtualThread` runs the endpoint on a virtual thread instead of a platform worker thread. You write plain blocking code; the JVM unmounts the carrier thread when you block on I/O.

```

@GET
@Path("/report")
@RunOnVirtualThread
public ReportView report(@RestQuery String period) {
    var orders = repo.findByPeriod(period);    // blocks - fine, unmounts
    var payments = paymentClient.fetch(period); // blocks - fine, unmounts
}

```

```
    return ReportView.of(orders, payments);
}
```

Rules and caveats that matter in an interview:

- **Do not use `synchronized` around blocking I/O** on a virtual thread. Prior to JDK 24 this *pins* the carrier thread, defeating the purpose. Use `ReentrantLock`.
- **Thread pools are pointless** for virtual threads; do not wrap them.
- **`ThreadLocal` still works** but is now per-virtual-thread, so caching heavyweight objects in one is wasteful.
- **JDBC connection pool size becomes the bottleneck.** Virtual threads make it cheap to have 10 000 concurrent requests; your pool of 20 connections does not care. Virtual threads move the constraint, they do not remove it.
- Native-image support for virtual threads works, but profile before assuming a win.

Interview question you will get: “*Virtual threads or reactive?*” The honest answer: virtual threads give you most of the scalability of reactive with none of the cognitive cost, and they are the right default for imperative, database-backed services on Java 21. Reactive still wins where you need **backpressure**, **streaming**, or **fine-grained composition of many concurrent calls** — Mutiny expresses “call these five services, cancel the rest when one fails, with a 200 ms budget” in a way imperative code cannot. Choose per endpoint, not per application.

2.3 Build-Time Route Generation

Classic JAX-RS resolves a request to a method at runtime: match the path, reflect on the method, coerce parameters, invoke reflectively. Quarkus REST does the matching structure and the parameter coercion **at build time**.

During augmentation Quarkus:

1. Finds every `@Path` class in the Jandex index — no classpath scanning.
2. Builds a path-matching trie and generates it as bytecode.
3. Generates a **direct invoker** per endpoint method — a class that calls your method with a normal `invokevirtual`, not `Method.invoke`.
4. Generates parameter extractors that read from the `Vert.x` request without reflection.
5. Generates serialiser/deserialiser wiring for the DTO types it can see.

The runtime effect: the request path contains no reflection, no annotation lookup, and no per-request metadata resolution. It also means the whole REST layer is visible to GraalVM’s reachability analysis, which is why Quarkus REST works in native mode with essentially no configuration.

The practical consequence you must remember: because routes are computed at build time, *you cannot register a JAX-RS resource at runtime*. If you need dynamic routes, use a `Vert.x` route directly:

```
@ApplicationScoped
public class DynamicRoutes {
```

```

void register(@Observes Router router, TenantConfig config) {
    for (String tenant : config.tenants()) {
        router.get("/t/" + tenant + "/status")
            .handler(rc -> rc.response().end(tenant + " ok"));
    }
}
}

```

2.4 Resources, Parameters and Responses

2.4.1 The Quarkus REST parameter annotations

Standard JAX-RS annotations work. Quarkus REST also offers shorter ones that infer the name from the parameter (requires `-parameters`, which the Quarkus archetype sets):

```

import org.jboss.resteasy.reactive.*;

@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class OrderResource {

    // Standard JAX-RS
    @GET @Path("/{id}")
    public OrderView v1(@PathParam("id") long id,
                       @QueryParam("expand") String expand,
                       @HeaderParam("X-Tenant") String tenant) { ... }

    // Quarkus REST shorthand - name inferred from the parameter
    @GET @Path("/{id}/short")
    public OrderView v2(@RestPath long id,
                       @RestQuery String expand,
                       @RestHeader("X-Tenant") String tenant,
                       @RestCookie String session) { ... }

    // Collections, defaults and enums bind automatically
    @GET
    public List<OrderView> search(@RestQuery List<String> status,
                                @RestQuery @DefaultValue("0") int page,
                                @RestQuery @DefaultValue("20") int size,
                                @RestQuery Optional<OrderStatus> filter,
                                @BeanParam PagingParams paging) { ... }
}

// @BeanParam groups repeated parameters into a reusable record-like holder.
public class PagingParams {
    @RestQuery @DefaultValue("0") public int offset;
    @RestQuery @DefaultValue("50") public int limit;
    @RestQuery public String sort;
}

```


2.4.2 Return types

```
// 1. Plain object -> 200 with a JSON body
@GET public List<OrderView> list() { ... }

// 2. jakarta.ws.rs.core.Response -> full control, but untyped
@POST
public Response create(CreateOrderRequest r) {
    OrderView v = service.create(r);
    return Response.created(URI.create("/orders/" + v.id())).entity(v).build();
}

// 3. RestResponse<T> -> full control AND a static type.
// Prefer this: OpenAPI generation and tests both benefit.
@POST @Path("/typed")
public RestResponse<OrderView> createTyped(CreateOrderRequest r) {
    OrderView v = service.create(r);
    return RestResponse.ResponseBuilder
        .create(RestResponse.Status.CREATED, v)
        .location(URI.create("/orders/" + v.id()))
        .header("X-Order-Version", v.version())
        .build();
}

// 4. Uni<T> -> non-blocking
@GET @Path("/{id}/async")
public Uni<OrderView> async(@RestPath long id) { ... }

// 5. Multi<T> -> streaming (see 2.9)
@GET @Path("/stream")
@Produces(MediaType.APPLICATION_JSON) // -> a streamed JSON array
public Multi<OrderView> stream() { ... }
```

2.4.3 Records as DTOs

Java 21 records are the right default for request and response bodies. They are immutable, concise, and Jackson deserialises them natively (given `-parameters`).

```
package com.acme.orders.api;

import com.fasterxml.jackson.annotation.JsonInclude;
import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

public record CreateOrderRequest(
    @NotBlank @Email String customerEmail,
    @NotEmpty @Valid List<LineItemRequest> items,
    @Size(max = 500) String note) { }

public record LineItemRequest(
```

```

        @NotBlank String sku,
        @Positive int quantity) { }

@JsonInclude(JsonInclude.Include.NON_NULL)
public record OrderView(
    long id,
    String status,
    BigDecimal total,
    String customerEmail,
    Instant createdAt,
    List<LineItemView> items) { }

public record LineItemView(String sku, int quantity, BigDecimal unitPrice) { }

```

Never expose a JPA entity as a REST DTO. It couples your wire contract to your schema, drags lazy-loading semantics into serialisation (hello, `LazyInitializationException` at JSON-writing time), and leaks internal fields. Records make the mapping cheap enough that there is no excuse. We revisit this in Chapters 5 and 24.

2.4.4 Jackson configuration

```

quarkus.jackson.fail-on-unknown-properties=false # default false; be deliberate
quarkus.jackson.write-dates-as-timestamps=false # ISO-8601 output
quarkus.jackson.serialization-inclusion=non-null
quarkus.jackson.timezone=UTC

```

For anything beyond that, customise the mapper:

```

@Singleton
public class JacksonCustomiser implements ObjectMapperCustomizer {
    @Override
    public void customize(ObjectMapper mapper) {
        mapper.setPropertyNamingStrategy(PropertyNamingStrategies.SNAKE_CASE);
        mapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
        mapper.registerModule(new JavaTimeModule());
        mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);
    }
}

```

`ObjectMapperCustomizer` is the correct hook. Producing your own `@Produces ObjectMapper` bean replaces Quarkus's, which silently drops the extension-registered modules that other extensions rely on.

2.5 Error Handling

A coherent error strategy is one of the clearest signals of engineering maturity, and interviewers probe it.

2.5.1 Exception mappers

```

package com.acme.orders.api.error;

import jakarta.ws.rs.core.Response;
import org.jboss.resteasy.reactive.RestResponse;
import org.jboss.resteasy.reactive.server.ServerExceptionMapper;

public class ApiExceptionMappers {

    // Quarkus REST style: a method, not a class. Less ceremony, same effect.
    @ServerExceptionMapper
    public RestResponse<ProblemDetail> orderNotFound(OrderNotFoundException e) {
        return RestResponse.status(
            Response.Status.NOT_FOUND,
            ProblemDetail.of("order-not-found", "Order not found", 404, e.getMessage()));
    }

    @ServerExceptionMapper
    public RestResponse<ProblemDetail> conflict(OptimisticLockException e) {
        return RestResponse.status(
            Response.Status.CONFLICT,
            ProblemDetail.of("concurrent-modification",
                "The order was modified by another request", 409,
                "Reload and retry"));
    }

    // Bean Validation failures -> 400 with per-field detail
    @ServerExceptionMapper
    public RestResponse<ProblemDetail> validation(ConstraintViolationException e) {
        var fields = e.getConstraintViolations().stream()
            .collect(Collectors.toMap(
                v -> v.getPropertyPath().toString(),
                ConstraintViolation::getMessage,
                (a, b) -> a + "; " + b));
        return RestResponse.status(
            Response.Status.BAD_REQUEST,
            ProblemDetail.withFields("validation-failed", "Request is invalid", 400, fields));
    }

    // Catch-all. Log the cause; never leak it to the client.
    @ServerExceptionMapper
    public RestResponse<ProblemDetail> unexpected(Throwable t) {
        String ref = UUID.randomUUID().toString();
        Log.errorf(t, "Unhandled error, ref=%s", ref);
        return RestResponse.status(
            Response.Status.INTERNAL_SERVER_ERROR,
            ProblemDetail.of("internal-error",
                "An unexpected error occurred", 500,
                "Reference: " + ref));
    }
}

```

2.5.2 RFC 7807 problem details

Standardise the error body. It is a published standard, clients can parse it generically, and it costs one record:

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ProblemDetail(
    String type,          // URI identifying the problem class
    String title,         // short, human-readable, stable
    int status,
    String detail,        // this occurrence
    String instance,      // URI of this occurrence
    Map<String, String> fields) {

    private static final String BASE = "https://errors.acme.com/orders/";

    public static ProblemDetail of(String type, String title, int status, String detail) {
        return new ProblemDetail(BASE + type, title, status, detail, null, null);
    }

    public static ProblemDetail withFields(String type, String title, int status,
                                           Map<String, String> fields) {
        return new ProblemDetail(BASE + type, title, status, null, null, fields);
    }
}
```

```
{
  "type": "https://errors.acme.com/orders/validation-failed",
  "title": "Request is invalid",
  "status": 400,
  "fields": {
    "createOrder.arg0.customerEmail": "must be a well-formed email address",
    "createOrder.arg0.items": "must not be empty"
  }
}
```

2.5.3 Rules worth stating explicitly

- **Never return a stack trace.** Log it with a correlation reference; return the reference.
- **Map domain exceptions, do not throw `WebApplicationException` from services.** A service that throws `NotFoundException` is coupled to JAX-RS and cannot be reused from a Kafka consumer or a scheduled job.
- **Use the right status code.** 400 for malformed input, 404 for a missing resource, 409 for a state conflict, 422 for semantically invalid but well-formed input, 429 for rate limiting, 503 with `Retry-After` for downstream unavailability.
- **Do not swallow exceptions in a filter.** A filter that catches and returns 200 hides real failures from your metrics.

2.6 Filters and Interceptors

2.6.1 Method-based filters

```
package com.acme.orders.api;

import org.jboss.resteasy.reactive.server.ServerRequestFilter;
import org.jboss.resteasy.reactive.server.ServerResponseFilter;
import jakarta.ws.rs.container.ContainerRequestContext;
import jakarta.ws.rs.container.ContainerResponseContext;

public class CorrelationFilters {

    public static final String HEADER = "X-Correlation-Id";

    @ServerRequestFilter(preMatching = true)
    public void inbound(ContainerRequestContext ctx) {
        String id = ctx.getHeaderString(HEADER);
        if (id == null || id.isBlank()) {
            id = UUID.randomUUID().toString();
            ctx.getHeaders().putSingle(HEADER, id);
        }
        MDC.put("correlationId", id);    // now every log line carries it
    }

    @ServerResponseFilter
    public void outbound(ContainerRequestContext req, ContainerResponseContext res) {
        res.getHeaders().putSingle(HEADER, req.getHeaderString(HEADER));
        MDC.remove("correlationId");
    }

    // A filter can abort the request by returning a Response.
    @ServerRequestFilter
    public Optional<RestResponse<ProblemDetail>> requireTenant(ContainerRequestContext ctx) {
        if (ctx.getUriInfo().getPath().startsWith("/orders")
            && ctx.getHeaderString("X-Tenant") == null) {
            return Optional.of(RestResponse.status(
                Response.Status.BAD_REQUEST,
                ProblemDetail.of("missing-tenant", "X-Tenant header required", 400, null)));
        }
        return Optional.empty();
    }
}
```

`preMatching = true` runs the filter before route matching — necessary if you rewrite the path or method. Without it, the filter runs after the resource method has been selected.

2.6.2 Name-bound filters

Apply a filter to specific endpoints rather than everything:

```
@NameBinding
@Retention(RetentionPolicy.RUNTIME)
public @interface AuditLogged { }
```

```

@Provider
@AuditLogged
public class AuditFilter implements ContainerRequestFilter {
    @Override public void filter(ContainerRequestContext ctx) { ... }
}

@POST
@AuditLogged // only this endpoint is audited
public Response create(CreateOrderRequest r) { ... }

```

2.6.3 The blocking rule applies to filters too

A `@ServerRequestFilter` on a non-blocking endpoint runs on the **event loop**. If your filter does a database lookup to resolve a tenant, you have just blocked an event loop for every request. Either mark it `@Blocking`, or return a `Uni<Void>`:

```

@ServerRequestFilter
public Uni<Void> resolveTenant(ContainerRequestContext ctx) {
    return tenantService.resolve(ctx.getHeaderString("X-Tenant"))
        .invoke(t -> ctx.setProperty("tenant", t))
        .replaceWithVoid();
}

```

2.7 The REST Client

Quarkus's REST Client is the MicroProfile REST Client: declare an interface, get an implementation generated at build time.

2.7.1 Declaring a client

```

package com.acme.orders.infra.payments;

import io.quarkus.rest.client.reactive.ClientExceptionMapper;
import io.smallrye.mutiny.Uni;
import jakarta.ws.rs.*;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;
import org.eclipse.microprofile.faulttolerance.*;

@Path("/payments")
@RegisterRestClient(configKey = "payments-api")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public interface PaymentClient {

    @POST
    @Timeout(2000)
    @Retry(maxRetries = 2, delay = 200, jitter = 100,
        retryOn = {ServerErrorException.class, ProcessingException.class})
    @CircuitBreaker(requestVolumeThreshold = 20, failureRatio = 0.5, delay = 10000)
    @Fallback(fallbackMethod = "capturePending")

```

```

PaymentResult capture(CaptureRequest request);

@GET @Path("/{id}")
Uni<PaymentResult> status(@PathParam("id") String id);

default PaymentResult capturePending(CaptureRequest request) {
    return PaymentResult.pending(request.orderId());
}

// Translate transport-level failures into domain exceptions at the boundary.
@ClientExceptionMapper
static RuntimeException toDomain(Response response) {
    return switch (response.getStatus()) {
        case 402 -> new PaymentDeclinedException(response.readEntity(String.class));
        case 404 -> new PaymentNotFoundException();
        case 429 -> new PaymentRateLimitedException(
            response.getHeaderString("Retry-After"));
        default -> null; // null -> fall through to default handling
    };
}
}

```

```

quarkus.rest-client.payments-api.url=https://payments.internal:8443
quarkus.rest-client.payments-api.scope=jakarta.inject.Singleton
quarkus.rest-client.payments-api.connect-timeout=2000
quarkus.rest-client.payments-api.read-timeout=5000

```

mTLS

```

quarkus.rest-client.payments-api.trust-store=classpath:truststore.p12
quarkus.rest-client.payments-api.trust-store-password=${TRUSTSTORE_PASSWORD}

```

```
%dev.quarkus.rest-client.payments-api.url=http://localhost:9090
```

```
%test.quarkus.rest-client.payments-api.url=http://localhost:${quarkus.wiremock.devservices.port}
↪ t:8089}

```

```

@ApplicationScoped
public class CheckoutService {
    @RestClient PaymentClient payments; // note: @RestClient, not @Inject alone

    public OrderView checkout(long orderId) {
        PaymentResult result = payments.capture(new CaptureRequest(orderId));
        ...
    }
}

```

2.7.2 Propagating headers

```

# Forward these inbound headers to every outbound call
org.eclipse.microprofile.rest.client.propagateHeaders=X-Correlation-Id,X-Tenant

```

Or programmatically, which is more explicit and easier to test:

```

@ApplicationScoped
public class CorrelationHeaderFactory implements ClientHeadersFactory {
    @Override
    public MultivaluedMap<String, String> update(
        MultivaluedMap<String, String> inbound,
        MultivaluedMap<String, String> outbound) {
        outbound.putSingle("X-Correlation-Id",
            Optional.ofNullable(MDC.get("correlationId"))
                .map(Object::toString)
                .orElseGet(() -> UUID.randomUUID().toString()));
        return outbound;
    }
}

```

```

@RegisterRestClient(configKey = "payments-api")
@RegisterClientHeaders(CorrelationHeaderFactory.class)
public interface PaymentClient { ... }

```

2.7.3 Fault tolerance placement — a common design mistake

Putting `@Retry` on a **non-idempotent** operation is a way to charge a customer twice. Retries belong on:

- GET, HEAD, OPTIONS — always safe.
- PUT, DELETE — idempotent by contract, safe if the server honours it.
- POST — **only with an idempotency key** that the server deduplicates on.

```

@POST
@Retry(maxRetries = 3)
PaymentResult capture(@HeaderParam("Idempotency-Key") String key,
    CaptureRequest request);

```

Senior interview question: “Where do you put the circuit breaker — client or server?”

On the **client**, per downstream dependency. Its purpose is to stop *you* wasting threads and latency budget on a dependency that is already failing, and to give it room to recover. A server-side breaker protects the wrong party. Also: set `requestVolumeThreshold` high enough that a handful of unlucky requests during a deploy do not trip it.

2.8 Streaming: Multi and Server-Sent Events

2.8.1 Streaming a JSON array

```

@GET
@Path("/export")
@Produces(MediaType.APPLICATION_JSON)
public Multi<OrderView> export(@RestQuery String period) {
    return repo.streamByPeriod(period);    // emits a streamed JSON array
}

```


The response is written incrementally, so memory stays flat regardless of result size. Compare with returning a `List<OrderView>` of 500 000 rows, which materialises the whole thing on the heap twice — once as objects, once as serialised bytes.

2.8.2 Server-Sent Events

```
@GET
@Path("/{id}/events")
@Produces(MediaType.SERVER_SENT_EVENTS)
@RestStreamElementType(MediaType.APPLICATION_JSON)
public Multi<OrderEvent> events(@RestPath long id) {
    return eventBus.<OrderEvent>consumer("orders." + id)
        .toMulti()
        .map(Message::body)
        .onOverflow().drop();    // ALWAYS declare an overflow policy
}
```

```
// Consuming SSE from another Quarkus service
@GET
@Path("/{id}/events")
@Produces(MediaType.SERVER_SENT_EVENTS)
Multi<OrderEvent> streamEvents(@PathParam("id") long id);
```

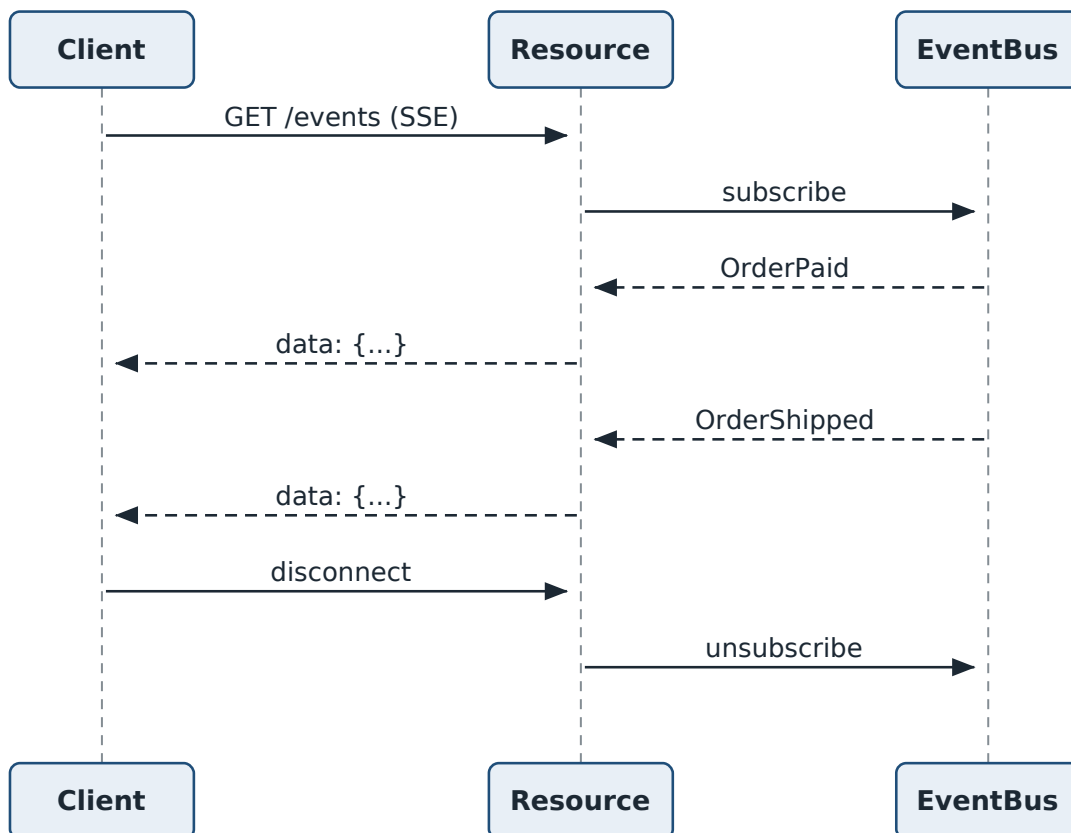


Figure 2.2 — SSE lifecycle. Note the unsubscribe on disconnect: without it you leak a subscription per dropped client.

2.8.3 When streaming is the wrong tool

- **Small, bounded results.** Streaming adds framing overhead and complicates client code for no benefit.
- **Anything behind a buffering proxy.** Many ingress configurations buffer responses, which silently defeats SSE. Verify end to end, not just against `curl` on localhost.
- **When the client needs the total count up front.** Streaming means you do not know the size until you finish.
- **Long-lived SSE at high fan-out.** Each open stream pins a connection. 50 000 subscribers is a WebSocket or a message broker problem, not an SSE problem.

Always set an overflow strategy on a `Multi` fed by an unbounded source — `.onOverflow().drop()`, `.buffer(n)`, or `.latest()`. Without one, a slow consumer causes unbounded buffering and an eventual `OutOfMemoryError`.

2.9 Multipart and File Uploads

```
import org.jboss.resteasy.reactive.multipart.FileUpload;
import org.jboss.resteasy.reactive.RestForm;

@POST
@Path("/{id}/attachments")
@Consumes(MediaType.MULTIPART_FORM_DATA)
@Blocking // file I/O blocks
public RestResponse<AttachmentView> upload(
    @RestPath long id,
    @RestForm("file") FileUpload file,
    @RestForm("description") String description) {

    if (file.size() > 10 * 1024 * 1024) {
        throw new BadRequestException("File exceeds 10 MB");
    }
    // Validate the *actual* content type, not the client-supplied one.
    String declared = file.contentType();
    if (!Set.of("application/pdf", "image/png").contains(declared)) {
        throw new BadRequestException("Unsupported type: " + declared);
    }

    java.nio.file.Path stored = storage.store(id, file.uploadedFile(), file.fileName());
    return RestResponse.ok(new AttachmentView(id, stored.getFileName().toString()));
}
```

```
quarkus.http.body.handle-file-uploads=true
quarkus.http.body.uploads-directory=/tmp/uploads
quarkus.http.body.delete-uploaded-files-on-end=true
quarkus.http.limits.max-body-size=20M
quarkus.http.limits.max-form-attribute-size=2048
```

Security note. Never trust `file.fileName()` — it is client-supplied and may contain `../`. Never trust `contentType()` — sniff the magic bytes if the file type matters. Never write uploads into a directory that is served statically. These three mistakes account for a large share of file-upload vulnerabilities.

2.10 HTTP Layer Configuration

```
# Binding
quarkus.http.port=8080
quarkus.http.host=0.0.0.0
quarkus.http.root-path=/api          # BUILD TIME - see Chapter 1
quarkus.http.non-application-root-path=/q

# Limits
quarkus.http.limits.max-body-size=10M
quarkus.http.limits.max-header-size=20K
quarkus.http.limits.max-initial-line-length=4096
quarkus.http.idle-timeout=30M
quarkus.http.read-timeout=60S

# Compression
quarkus.http.enable-compression=true
quarkus.http.compress-media-types=text/html,text/plain,application/json,application/xml

# Access log - off by default; enable deliberately, it is not free
quarkus.http.access-log.enabled=true
quarkus.http.access-log.pattern=%h %l %u %t "%r" %s %b %D "%{i,X-Correlation-Id}"
%prod.quarkus.http.access-log.exclude-pattern=/q/health.*

# CORS
quarkus.http.cors.enabled=true
quarkus.http.cors.origins=https://app.acme.com,https://admin.acme.com
quarkus.http.cors.methods=GET,POST,PUT,DELETE,OPTIONS
quarkus.http.cors.headers=Content-Type,Authorization,X-Correlation-Id
quarkus.http.cors.exposed-headers=X-Correlation-Id,Location
quarkus.http.cors.access-control-max-age=24H
quarkus.http.cors.access-control-allow-credentials=true

# HTTP/2 is on by default over TLS
quarkus.http.http2=true
```

Never set `quarkus.http.cors.origins=*` together with `access-control-allow-credentials=true`. Browsers reject the combination, and if a proxy “helpfully” rewrites it you have exposed every authenticated endpoint to every origin. List origins explicitly, per environment.

2.11 OpenAPI and Swagger UI

```
<dependency>
  <groupId>io.quarkus</groupId>
```

```
<artifactId>quarkus-smallrye-openapi</artifactId>
</dependency>
```

The document is generated from your resource classes at build time and served at `/q/openapi`. Swagger UI is at `/q/swagger-ui` (dev only unless you enable it).

```
@Path("/orders")
@Tag(name = "Orders", description = "Order lifecycle operations")
public class OrderResource {

    @POST
    @Operation(summary = "Create an order",
        description = "Creates an order in PENDING state and reserves stock.")
    @ApiResponse(responseCode = "201", description = "Created",
        content = @Content(schema = @Schema(implementation = OrderView.class)))
    @ApiResponse(responseCode = "400", description = "Validation failed",
        content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    @ApiResponse(responseCode = "409", description = "Insufficient stock")
    public RestResponse<OrderView> create(@Valid CreateOrderRequest request) { ... }
}
```

```
quarkus.smallrye-openapi.info-title=Orders API
quarkus.smallrye-openapi.info-version=1.0.0
quarkus.smallrye-openapi.servers=https://api.acme.com
quarkus.swagger-ui.always-include=false      # keep it out of prod images
%dev.quarkus.swagger-ui.always-include=true
```

```
# Emit the spec as a build artifact for contract testing / client generation
quarkus.smallrye-openapi.store-schema-directory=target/openapi
```

Committing the generated spec and diffing it in CI is a cheap and effective way to catch accidental breaking changes to your API contract.

2.12 Testing the REST Layer

2.12.1 @QuarkusTest with RestAssured

```
package com.acme.orders.api;

import io.quarkus.test.junit.QuarkusTest;
import org.junit.jupiter.api.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;
import static jakarta.ws.rs.core.Response.Status.*;

@QuarkusTest
class OrderResourceTest {

    @Test
    void createsOrder() {
        given()
            .contentType("application/json")
```

```

        .header("X-Tenant", "acme")
        .body("""
            {
                "customerEmail": "buyer@example.com",
                "items": [{"sku": "SKU-1", "quantity": 2}]
            }
            """)
        .when()
        .post("/orders")
        .then()
        .statusCode(CREATED.getStatusCode())
        .header("Location", matchesPattern(".*orders/\\d+"))
        .body("status", is("PENDING"))
        .body("items", hasSize(1));
    }

    @Test
    void rejectsInvalidEmail() {
        given()
            .contentType("application/json")
            .header("X-Tenant", "acme")
            .body("""
                {
                    "customerEmail": "not-an-email",
                    "items": []
                }
                """)
        .when()
        .post("/orders")
        .then()
        .statusCode(BAD_REQUEST.getStatusCode())
        .body("type", endsWith("validation-failed"))
        .body("fields", hasKey(containsString("customerEmail")));
    }

    @Test
    void returns404ForUnknownOrder() {
        given().header("X-Tenant", "acme")
        .when().get("/orders/999999")
        .then().statusCode(NOT_FOUND.getStatusCode())
            .body("title", is("Order not found"));
    }
}

```

Java 21 text blocks make JSON fixtures readable inline — a small thing that meaningfully improves test maintainability.

2.12.2 Mocking a REST client

```

@QuarkusTest
class CheckoutServiceTest {

    @InjectMock @RestClient PaymentClient payments;
    @Inject CheckoutService checkout;
}

```

```

@Test
void fallsBackWhenPaymentTimesOut() {
    Mockito.when(payments.capture(any()))
        .thenThrow(new ProcessingException("timeout"));

    OrderView result = checkout.checkout(1L);

    assertThat(result.status()).isEqualTo("PAYMENT_PENDING");
}
}

```

2.12.3 Testing against the native binary

```

@QuarkusIntegrationTest    // runs the SAME tests against the packaged artifact
class OrderResourceIT extends OrderResourceTest { }

```

```
./mvnw verify -Dnative
```

This is the single most valuable native-mode practice: reflection problems that only appear in native mode are found by your existing test suite instead of by a customer.

2.12.4 @QuarkusTest vs @QuarkusIntegrationTest

	@QuarkusTest	@QuarkusIntegrationTest
Runs	In the same JVM as the app	Against the built artifact as a separate process
Injection	@Inject works	No injection — black box only
Mocking	@InjectMock works	Not possible
Speed	Fast	Slower (starts the artifact)
Native	No	Yes

The idiomatic pattern is to write the tests once as @QuarkusTest using only HTTP assertions, then subclass with @QuarkusIntegrationTest for the native run — which is exactly what the snippet above does.

2.13 Performance Notes

1. Return **Multi** for large result sets. Constant memory instead of linear.
2. Avoid **Response** when **RestResponse<T>** will do — you keep the static type for OpenAPI and tests at no runtime cost.
3. Enable compression for JSON — typically a 70–85 % reduction on API payloads.
4. Do not enable the access log by default in high-throughput services. It is synchronous formatting work on every request; sample it or rely on the ingress log.

5. **Keep DTOs shallow.** Deep object graphs cost serialisation time and encourage N+1 queries in the layer below.
6. **Prefer `@RunOnVirtualThread` over `@Blocking`** for imperative endpoints on Java 21 — same code, better concurrency, no reactive rewrite.
7. **Size the worker pool against your real bottleneck.** 200 worker threads in front of a 20-connection JDBC pool just moves the queue; it does not add throughput.
8. **Watch the event loop.** Set `max-event-loop-execute-time` low in dev so problems surface early.

2.14 Best Practices

1. Use `quarkus-rest`, not `quarkus-resteasy-classic`, unless something forces your hand.
2. Records for DTOs; **never** expose entities.
3. `RestResponse<T>` over raw `Response`.
4. One `ExceptionHandler` per domain exception class, plus one catch-all that logs and returns a reference.
5. RFC 7807 problem details for every error body.
6. Domain services throw **domain** exceptions, never `WebApplicationException`.
7. `@Blocking` / `@NonBlocking` explicitly wherever the heuristic could be wrong — the annotation is documentation as much as configuration.
8. Version the API in the path (`/api/v1/orders`) and treat the OpenAPI document as a contract diffed in CI.
9. Propagate a correlation ID inbound and outbound, and put it in the MDC.
10. Validate at the boundary with `@Valid`; do not re-validate in every layer.
11. Configure timeouts on **every** REST client. A client with no read timeout will eventually hang a thread forever.
12. Retry only idempotent operations, or use an idempotency key.

2.15 Common Mistakes

1. **Blocking on the event loop.** The defining Quarkus mistake. Symptom: `Thread blocked` warnings, then a throughput cliff.
2. **Returning `Uni.createFrom().item(blockingCall())`.** The blocking call is evaluated *eagerly*, on the current thread, before the `Uni` is even constructed. Use `Uni.createFrom().item(() -> blockingCall())` with a supplier plus `runSubscriptionOn`, or just use `@Blocking`.
3. **Exposing entities.** Produces `LazyInitializationException` during serialisation, leaks internal fields, and welds your API contract to your schema.
4. **Producing your own `ObjectMapper` bean.** Replaces Quarkus's and silently drops extension-registered modules. Use `ObjectMapperCustomizer`.
5. **Forgetting `-parameters`.** `@RestPath long id` and record deserialisation both depend on parameter names being in the bytecode. Symptom: `arg0` in error messages, nulls in deserialised records.
6. **Catching `Exception` in a filter and returning 200.** Hides failures from metrics and alerting.

7. **No timeout on a REST client.** Under a downstream slowdown, worker threads accumulate until the pool is exhausted and the service becomes unavailable — a textbook cascading failure.
8. **@Retry on a non-idempotent POST.** Duplicate charges, duplicate orders.
9. **Unbounded Multi with no overflow strategy.** Slow consumer $\Rightarrow$ unbounded buffer $\Rightarrow$ OOM.
10. **Assuming @RequestScoped state survives a thread switch.** On a reactive endpoint, work can move between threads. Use context propagation (`@ActivateRequestContext`, `quarkus-smallrye-context-propagation`) rather than assuming a thread-bound context.

2.16 Interview Questions

Junior

- Q1. **How do you create a REST endpoint in Quarkus?** Annotate a class with `@Path`, a method with an HTTP verb annotation (`@GET`, `@POST`, ...), and declare media types with `@Produces` / `@Consumes`. Add `quarkus-rest-jackson` for JSON.
- Q2. **What is the difference between @PathParam and @QueryParam?** `@PathParam` binds a template variable in the URI path (`/orders/{id}`); `@QueryParam` binds a query-string parameter (`?status=PAID`).
- Q3. **What does @Produces(MediaType.APPLICATION_JSON) do?** Declares the response content type and drives content negotiation and serialiser selection.
- Q4. **How do you return a 201 with a Location header?** `Response.created(uri).entity(body).build()`, or the typed `RestResponse.ResponseBuilder.create(Status.CREATED, body).location(uri).build()`.
- Q5. **What is a DTO and why not return the entity?** A Data Transfer Object is a type shaped for the wire. Returning entities couples the API to the schema, can trigger lazy-loading errors during serialisation, and leaks internal fields.
- Q6. **How do you test a REST endpoint?** `@QuarkusTest` with `RestAssured`: `given().when().get("/orders").then().statusCode(200)`.
- Q7. **What is quarkus-rest?** The current name of RESTEasy Reactive — Quarkus's build-time-optimised Jakarta REST implementation running on Vert.x.

Mid-level

- Q8. **What is the difference between RESTEasy Classic and Quarkus REST?** Classic is the older servlet-derived implementation: thread-per-request, reflection-heavy runtime dispatch. Quarkus REST generates routes and invokers at build time, runs on Vert.x, supports both blocking and non-blocking endpoints on one engine, and is far better suited to native compilation.
- Q9. **How does Quarkus decide the thread an endpoint runs on?** At build time, from the return type: `Uni/Multi/CompletionStage` $\Rightarrow$ event loop; anything else $\Rightarrow$ worker thread. `@Blocking`, `@NonBlocking` and `@RunOnVirtualThread` override it.

Q10. What happens if you block an event loop thread? Every connection pinned to that event loop stalls. With N event loops you lose 1/N of total I/O capacity. Quarkus logs `Thread blocked` warnings. The symptom under load is a throughput cliff, not a single slow request.

Q11. How do you handle exceptions globally? `@ServerExceptionHandler` methods (or `ExceptionHandler` classes), one per domain exception plus a catch-all that logs with a correlation reference and returns a generic RFC 7807 body.

Q12. How do you call another service? Declare an interface with `@RegisterRestClient`, inject it with `@RestClient`, configure the URL under `quarkus.rest-client.<configKey>.url`. Add `@Timeout`, `@Retry`, `@CircuitBreaker` from MicroProfile Fault Tolerance.

Q13. What is `@BeanParam` for? Grouping repeated parameters (paging, filtering, common headers) into a single reusable holder object, keeping method signatures short.

Q14. How do you stream a large result set? Return `Multi<T>`. The response is written incrementally so memory stays constant. For push-style updates use `@Produces(MediaType.SERVER_SENT_EVENTS)` with `@RestStreamElementType`.

Q15. How do you enable CORS? `quarkus.http.cors.enabled=true` plus an explicit `origins` list. Never combine `*` with `allow-credentials=true`.

Q16. What is the difference between `@QuarkusTest` and `@QuarkusIntegrationTest`? `@QuarkusTest` runs in-JVM with injection and mocking available. `@QuarkusIntegrationTest` runs against the packaged artifact — jar, container or native binary — as a black box, so no injection or mocking, but it validates the thing you actually ship.

Q17. How do you customise Jackson? Implement `ObjectMapperCustomizer`. Do not produce your own `ObjectMapper` bean; that replaces Quarkus's and drops extension-registered modules.

Senior

Q18. Explain how Quarkus REST achieves its performance. Three things. First, routing and invocation are generated at build time: a path-matching trie plus a direct invoker per method, so the request path has no reflection and no annotation lookup. Second, it runs directly on Vert.x/Netty with no servlet layer, so there is no request/response wrapping or thread-per-request model. Third, serialiser wiring is resolved at build time where possible. Secondly, because everything is statically reachable, GraalVM can compile it natively without configuration — so the same design that makes it fast on the JVM is what makes it viable in native mode.

Q19. Walk me through a request through a Quarkus REST endpoint. Netty accepts the connection on an event loop. Vert.x produces an `HttpServerRequest`. The generated router matches the path against the build-time trie and selects the endpoint. Pre-matching filters run, then the endpoint's classification decides whether to stay on the event loop or dispatch to a worker. Generated extractors bind path, query, header and body parameters; the body is deserialised by the pre-wired reader. The generated invoker calls your method directly. The return value goes to the pre-wired writer, response filters run, and the response is written back on the originating event loop.

Q20. Your service exhausts its worker pool under load. Diagnose it. Worker exhaustion means work is arriving faster than it completes, so I look for the thing that got slower rather than the thing that got busier. Steps: (a) thread dump — what are the 200 workers actually doing? Almost always they are all parked on the same thing; (b) if they are on `getConnection`, the JDBC

pool is the real bottleneck and the worker pool is just the queue in front of it; (c) if they are in a socket read, a downstream dependency has slowed and I check whether that client has a read timeout — a missing timeout turns a downstream slowdown into a total outage; (d) if they are in application code, profile it; (e) check whether blocking work is being dispatched from a non-blocking endpoint, which multiplies dispatches. The fixes in priority order are: add timeouts, add a bulkhead per dependency so one slow downstream cannot consume the whole pool, right-size the connection pool, and consider `@RunOnVirtualThread` so blocked requests do not each hold a platform thread.

Q21. When would you use reactive endpoints over virtual threads? When I need backpressure, streaming, or structured concurrency over many calls. Mutiny lets me express “fan out to five services, take the first three, cancel the rest, total budget 200 ms” declaratively, and `Multi` gives real backpressure to a slow consumer. Virtual threads give scalability but not backpressure — 10 000 virtual threads all hitting a downstream will happily overwhelm it. For a straightforward “read the database, map, return” endpoint, virtual threads are simpler and I would prefer them.

Q22. How do you design error handling across a fleet of services? Standardise on RFC 7807 with a shared `ProblemDetail` type and a `type` URI namespace owned centrally, so a client can switch on `type` rather than parse prose. Ship it as an internal library along with the catch-all mapper, so every service behaves identically. Domain services throw domain exceptions; only the boundary knows about HTTP. Every 5xx logs with a correlation ID that is returned to the caller and present in traces, so a customer-reported reference maps to a trace in one query. Enforce it with an architecture test and a contract test in CI.

Q23. How do you version a REST API? URI versioning (`/api/v1/orders`) for major versions, because it is unambiguous in logs, caches and routing rules. Within a major version, only additive changes — new optional fields, new endpoints — and clients must ignore unknown fields. For breaking changes, run v1 and v2 concurrently from the same codebase where the domain permits, with a published deprecation window and metrics per version so you know when v1 traffic has actually stopped. Diff the generated OpenAPI document in CI to catch accidental breaks. Header/content negotiation versioning is more elegant on paper but harder to debug and to cache.

Q24. How would you implement per-tenant rate limiting? I would not implement it in the application if an ingress or API gateway can do it — that is the correct layer, and it protects the service from traffic that never reaches it. If it must be in-process: a `@ServerRequestFilter` resolving the tenant, backed by a token bucket in Redis so the limit is global rather than per-replica, returning 429 with `Retry-After` and `X-RateLimit-*` headers. The filter must be non-blocking or explicitly `@Blocking`. And it needs a fail-open policy: if Redis is unavailable, allow the request and alert, rather than turning a rate-limiter outage into a total outage.

Principal

Q25. Design the HTTP layer conventions for 40 services. What do you standardise and what do you leave free? *Standardise* the things that make services legible to each other and to operators: the error envelope (RFC 7807 with a central `type` namespace), correlation-ID propagation, the health and metrics endpoints, the auth mechanism and token shape, versioning scheme, pagination convention, timeout and retry defaults per dependency class, and the OpenAPI-diff gate in CI. Ship these as an internal Quarkus extension so adoption is a dependency rather than a document — the moment conventions live only in a wiki page they decay.

Leave free the domain modelling: resource naming, DTO shapes, which endpoints exist, whether an endpoint is reactive or virtual-threaded, and internal package structure. Teams need latitude where the domain differs, and standardising it produces lowest-common-denominator APIs.

The test for whether something belongs in the standard set: *does an inconsistency here cost someone outside the owning team?* Error shape does. DTO field ordering does not.

Q26. A team reports that their Quarkus service is slower than the Spring Boot service it replaced. How do you approach it? First, insist on numbers: which percentile, under what load, measured where. “Slower” is usually p99 under a specific condition, and often measured client-side including network. Then: is peak throughput actually lower, or is it latency? Quarkus in JVM mode should be comparable or better; native mode gives up perhaps 10–30 % peak throughput because there is no C2 JIT, so if they moved to native and are comparing peak throughput on a warmed-up JVM, the answer may simply be that they chose the wrong mode for a throughput-bound service. Other common causes: an endpoint blocking an event loop, so throughput collapses non-linearly; a connection pool sized for a thread-per-request model; a missing `-parameters` flag forcing a slow serialisation path; JIT warm-up being measured because the benchmark is too short. I would profile with `async-profiler` in JVM mode, compare flame graphs, and only then discuss architecture. Notably, the correct outcome might be “switch this service back to JVM mode” — native is not the goal, the SLO is.

2.17 Cheat Sheet

Extensions

```
quarkus-rest                <!-- the REST server -->
quarkus-rest-jackson        <!-- JSON via Jackson -->
quarkus-rest-client         <!-- typed HTTP client -->
quarkus-rest-client-jackson
quarkus-smallrye-openapi    <!-- OpenAPI + Swagger UI -->
quarkus-smallrye-fault-tolerance
quarkus-hibernate-validator
```

Execution model

```
public List<X> a()           // worker thread
public Uni<X> b()            // event loop
@Blocking public Uni<X> c()   // worker thread
@NonBlocking public X d()     // event loop
@RunOnVirtualThread public X e() // virtual thread (Java 21)
```

Parameters

```
@RestPath long id           @PathParam("id")
@RestQuery String q          @QueryParam("q")
@RestHeader String h         @HeaderParam("h")
@RestCookie String c         @CookieParam("c")
@RestForm String f           @FormParam("f")
@BeanParam Paging p
```

Responses

```

return view; // 200 + JSON
return RestResponse.ok(view);
return RestResponse.status(Status.CREATED, view);
return RestResponse.ResponseBuilder
    .create(Status.CREATED, view)
    .location(uri).header("X-V", v).build();

```

Error handling

```

@ServerExceptionHandler
public RestResponse<ProblemDetail> map(MyException e) { ... }

```

REST client

```

@registerRestClient(configKey = "svc")
public interface Client { @GET Uni<X> get(); }

@RestClient Client client;

```

```

quarkus.rest-client.svc.url=https://svc:8443
quarkus.rest-client.svc.read-timeout=5000

```

Testing

```

@QuarkusTest // in-JVM, injection + mocks
@QuarkusIntegrationTest // black box against the built artifact
@InjectMock @RestClient Client client;

```

Endpoints

```

/q/openapi    /q/swagger-ui    /q/health    /q/metrics

```

2.18 Summary

Quarkus REST is RESTEasy Reactive under a clearer name. Its performance comes from doing routing, invocation and serialiser wiring at build time — the same idea as Chapter 1, applied to HTTP — which removes reflection from the request path and makes the whole layer statically analysable for native compilation.

The concept you must own is the **execution model**. One engine, two pools, and a classification decided at build time from your method signature. Reactive return type means event loop; anything else means worker thread; `@Blocking`, `@NonBlocking` and `@RunOnVirtualThread` override it. Blocking an event loop is the defining Quarkus production incident, and the ability to explain why it degrades the whole server rather than one request is what an interviewer is listening for.

Around that: records as DTOs, `RestResponse<T>` for typed responses, RFC 7807 for errors, declarative REST clients with timeouts and correctly-placed retries, `Multi` for streaming with an explicit overflow policy, and `@QuarkusIntegrationTest` to run the same suite against the native binary.

Chapter 3 goes underneath the REST layer to **CDI and ArC** — how build-time dependency injection actually works, why beans get removed, what a client proxy really is, and the scope rules that cause the most confusing bugs in Quarkus applications.

2.19 References

- Quarkus REST guide — <https://quarkus.io/guides/rest>
- REST Client — <https://quarkus.io/guides/rest-client>
- Quarkus 3.9 “Big Reactive Rename” — <https://quarkus.io/blog/quarkus-3-9-1-released/>
- Virtual threads in Quarkus — <https://quarkus.io/guides/virtual-threads>
- SmallRye Fault Tolerance — <https://quarkus.io/guides/smallrye-fault-tolerance>
- OpenAPI and Swagger UI — <https://quarkus.io/guides/openapi-swaggerui>
- RFC 7807, Problem Details for HTTP APIs — <https://www.rfc-editor.org/rfc/rfc7807>
- Testing guide — <https://quarkus.io/guides/getting-started-testing>

CDI and ArC

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Jakarta CDI 4.1 · ArC

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what ArC is and how build-time bean resolution differs from runtime CDI.
2. Describe **bean discovery** in Quarkus — including why a bean in an external JAR is silently invisible.
3. Choose correctly between `@ApplicationScoped`, `@Singleton`, `@RequestScoped`, `@SessionScoped` and `@Dependent`, and explain the consequences of each.
4. Explain what a **client proxy** is, why normal scopes need one, and which class-design rules follow from it.
5. Diagnose and fix the **unused bean removal** problem — the most confusing Quarkus-specific CDI failure.
6. Write producers, disposers, qualifiers, interceptors, decorators and observers idiomatically.
7. Use build-time conditional beans (`@IfBuildProfile`, `@UnlessBuildProfile`, `@LookupIfProperty`) instead of runtime branching.
8. Propagate the request context across asynchronous boundaries without leaking it.
9. Test CDI code with `@InjectMock`, `@QuarkusComponentTest` and plain JUnit.
10. Answer 20+ interview questions on CDI, including the scope and proxy questions that reliably separate levels.

3.1 CDI in One Page

CDI (Contexts and Dependency Injection) is the Jakarta EE component model. Three ideas:

- **Beans** — objects whose lifecycle a container manages.
- **Contexts** — the rules for when an instance is created and destroyed (its *scope*).
- **Injection** — the container supplies dependencies rather than you constructing them.

```
@ApplicationScoped                                // this is a bean, one per application
public class OrderService {

    private final OrderRepository repository;        // this is an injection point

    public OrderService(OrderRepository repository) { // constructor injection

```

```
        this.repository = repository;  
    }  
}
```

Quarkus implements CDI 4.1 through **ArC**. ArC is a full CDI container with one enormous difference: **it resolves the entire bean graph during the Maven build**, and generates a Java class for every bean, every producer and every injection point. At runtime there is no reflection, no annotation scanning, and no bean resolution — only direct method calls.

3.2 Why Build-Time DI Changes the Rules

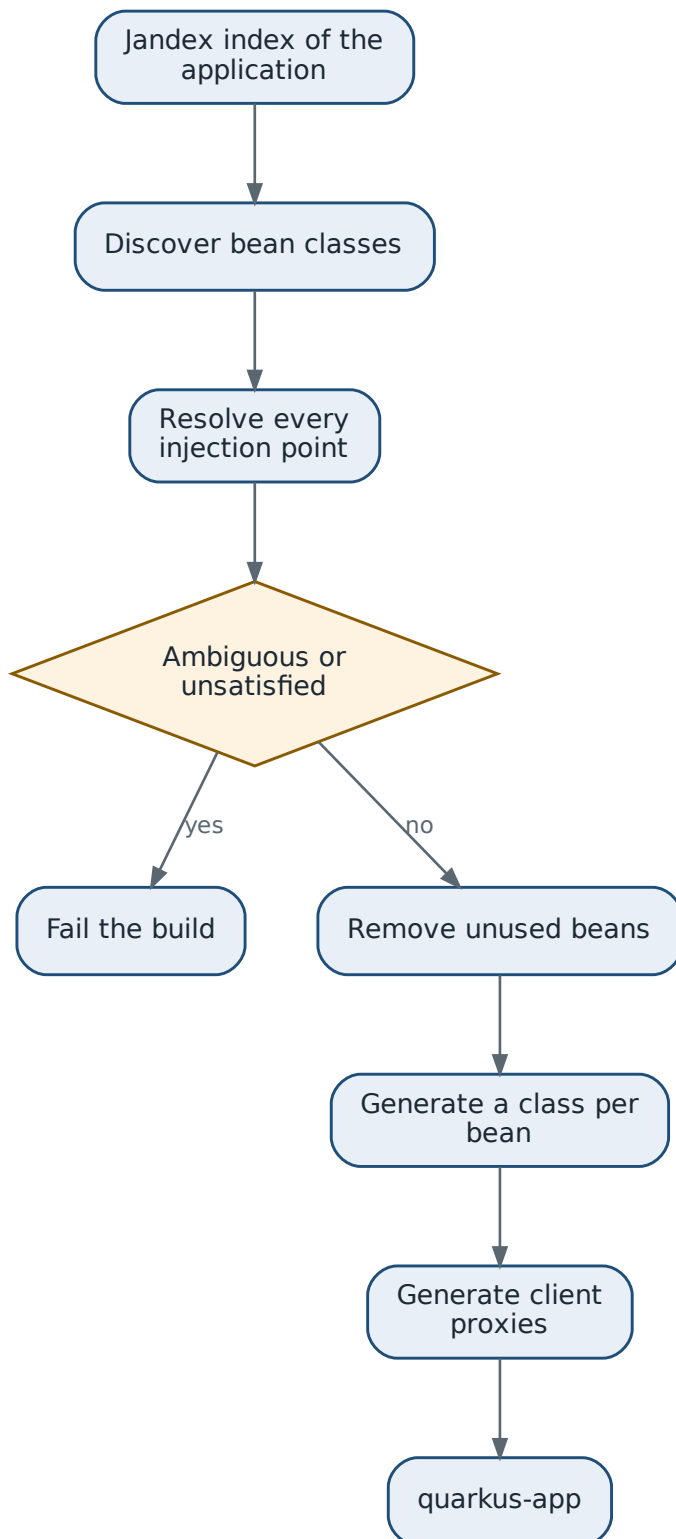


Figure 3.1 — ArC’s build-time pipeline. Note that resolution failures stop the build, not the application.

Four consequences follow, and each is a common interview question:

1. Wiring errors are build errors. An unsatisfied or ambiguous dependency fails `mvn package` with a precise message, not a `NoSuchBeanDefinitionException` at 3 a.m. This is a genuine improvement over runtime resolution.

```
[ERROR] Unsatisfied dependency for type com.acme.orders.PricingEngine
       and qualifiers [@Default]
       - injection target: com.acme.orders.OrderService#pricing
       - declared on CLASS bean [types=[...], qualifiers=[@Default, @Any],
         target=com.acme.orders.OrderService]
```

2. Unused beans are deleted. If nothing injects a bean, ArC removes it. Excellent for image size — and the source of the failure mode in §3.7.

3. The bean set is fixed after the build. You cannot register a bean at runtime based on, say, a row in a database. Use a producer method with a runtime condition instead.

4. Everything is statically reachable. Which is precisely what makes GraalVM’s closed-world analysis able to compile your application.

3.3 Bean Discovery

3.3.1 What counts as a bean

Quarkus uses **annotated-type discovery**. A class becomes a bean if it carries a bean-defining annotation:

- A scope: `@ApplicationScoped`, `@Singleton`, `@RequestScoped`, `@SessionScoped`, `@Dependent`
- A stereotype (e.g. `@Model`)
- `@Interceptor`, `@Decorator`
- Certain Quarkus annotations: `@Path`, `@Scheduled`, `@Startup`, `@GrpcService`, `@Recorder`...

```
@ApplicationScoped           // bean
public class PricingEngine { }

public class NotABean { }      // NOT a bean - no annotation, will not be injected
```

This differs from Spring in a way that trips people up. Spring scans for `@Component` and friends *downward from the main class package*. Quarkus scans a Jandex index of the **entire application** — package structure is irrelevant. That means a bean anywhere in your module is found, but a bean in an unindexed dependency is not found at all.

3.3.2 The external-JAR problem

This is the single most common “my bean is not injected” cause in real projects.

A dependency JAR is only indexed if:

1. It contains a `META-INF/jandex.idx`, **or**
2. It contains a `META-INF/beans.xml` (even empty), **or**
3. It is listed under `quarkus.index-dependency.*`.

Otherwise Jandex never sees it, ArC never discovers its beans, and you get an unsatisfied-dependency build failure with no obvious cause.

Fix A — index at the source (preferred). In the shared module's `pom.xml`:

```
<plugin>
  <groupId>io.smallrye</groupId>
  <artifactId>jandex-maven-plugin</artifactId>
  <version>3.2.2</version>
  <executions>
    <execution>
      <id>make-index</id>
      <goals><goal>jandex</goal></goals>
    </execution>
  </executions>
</plugin>
```

Fix B — index from the consumer. In the application's `application.properties`:

```
quarkus.index-dependency.shared.group-id=com.acme
quarkus.index-dependency.shared.artifact-id=shared-domain
```

Fix A is better: it makes the module self-describing, so every consumer benefits without configuration.

3.4 Scopes

Scope selection is where most CDI bugs originate. Learn this table properly.

Scope	Instances	Proxied	Lazy	Typical use
<code>@Application</code> <code>Scoped</code>	One per application	Yes	Yes	Default for services, repositories
<code>@Singleton</code>	One per application	No	No (eager on first injection)	Config holders, hot-path beans
<code>@RequestScoped</code>	One per HTTP request	Yes	Yes	Per-request state, correlation data
<code>@SessionScoped</code>	One per HTTP session	Yes	Yes	Rare in stateless microservices
<code>@Dependent</code>	One per injection point	No	n/a	Stateful helpers, short-lived objects

3.4.1 @ApplicationScoped vs @Singleton

This is asked in almost every Quarkus interview.

```
@ApplicationScoped
public class LazyService {
    public LazyService() {
        Log.info("created");    // logged on FIRST USE, not at startup
    }
}

@Singleton
public class EagerService {
    public EagerService() {
        Log.info("created");    // logged when first INJECTED, at startup
    }
}
```

	@ApplicationScoped	@Singleton
Client proxy	Yes	No
Instantiation	Lazy, on first method call	Eager, when first injected
Class may be <code>final</code>	No	Yes
No-arg constructor required	Yes	No
Interceptors work	Yes	Yes
Circular references	Tolerated (proxy breaks the cycle)	Fails
Mockable with <code>@InjectMock</code>	Yes	Yes (Quarkus handles it)
Method-call overhead	One extra virtual call	None

Practical guidance: use `@ApplicationScoped` by default. Reach for `@Singleton` when you need `final` fields on the class, when the proxy indirection is measurable in a genuinely hot path, or when the bean must exist eagerly. Do not micro-optimize this — the proxy call is a single virtual dispatch that the JIT will usually inline.

3.4.2 @RequestScoped

```
@RequestScoped
public class RequestContext {
    private String correlationId;
    private String tenant;
    // getters/setters
}

@ApplicationScoped
public class OrderService {
    @Inject RequestContext context;    // proxy - resolves per request

    public void create(...) {
        Log.infof("tenant=%s", context.getTenant());
    }
}
```

```
    }
}
```

The proxy is what makes this work: the same `@ApplicationScoped` bean holds one reference, but each method call resolves to the instance belonging to the *current* request context.

Where the request context exists: HTTP requests, `@Scheduled` methods, Kafka consumers with `@ActivateRequestContext`, and anywhere you activate it explicitly. It does **not** exist on an arbitrary thread you started yourself:

```
// Fails: ContextNotActiveException
new Thread(() -> orderService.create(...)).start();

// Fixed
@ActivateRequestContext
public void createInBackground(...) {
    orderService.create(...);
}
```

3.4.3 @Dependent

The default scope when no annotation is present on a producer. A new instance per injection point, whose lifecycle follows the injecting bean.

```
@Dependent
public class OrderCalculator {
    private BigDecimal runningTotal = BigDecimal.ZERO;    // safe: not shared
}
```

Use it for stateful helpers. Be careful: injecting a `@Dependent` bean into an `@ApplicationScoped` bean gives you **one instance for the whole application lifetime** — its lifecycle follows its injection point, and that injection point lives forever. That is a classic accidental-shared-state bug.

3.4.4 The scope bug that costs people days

```
// BUG: mutable state on an application-scoped bean
@ApplicationScoped
public class OrderProcessor {
    private Order current;                // shared across all threads!

    public void process(Order order) {
        this.current = order;            // race condition
        validate();
        persist();
    }
}
```

Under a single-user test this works perfectly. Under concurrency, requests overwrite each other's state. **Application-scoped beans must be stateless**, or their state must be immutable, or explicitly thread-safe. Pass state as parameters:

```
@ApplicationScoped
public class OrderProcessor {
    public void process(Order order) {           // state is a local
        validate(order);
        persist(order);
    }
}
```

3.5 Client Proxies

3.5.1 What they are

A normal-scoped bean (`@ApplicationScoped`, `@RequestScoped`, `@SessionScoped`) is never injected directly. You receive a generated subclass that forwards every call to the contextual instance:

```
// Conceptually what ArC generates
public class OrderService_ClientProxy extends OrderService {
    @Override
    public OrderView create(CreateOrderRequest r) {
        return arc().contextualInstance().create(r); // resolve, then delegate
    }
}
```

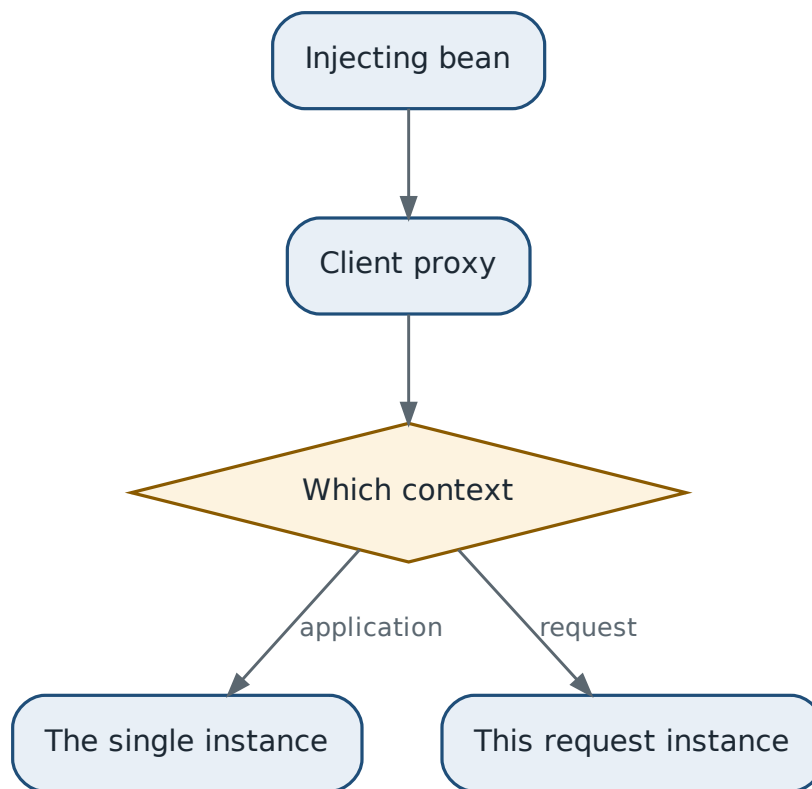


Figure 3.2 — A client proxy resolves the contextual instance on every call.

Proxies exist so that:

- A long-lived bean can hold a reference to a shorter-lived one (`@ApplicationScoped` → `@RequestScoped`).
- Circular dependencies resolve, because the proxy defers the real lookup.
- Lazy instantiation works — the instance is created on first *call*, not on injection.

3.5.2 The rules that follow

A proxyable class must:

- **Not be `final`**, and have no `final` methods you expect to be intercepted.
- **Have a no-arg constructor** (may be `protected`; the compiler only supplies one if you declare no other constructor).
- **Not be `private`.**

```
// FAILS: cannot create a client proxy for a final class
@ApplicationScoped
public final class Broken { }
```

```

// FAILS: no no-arg constructor
@ApplicationScoped
public class AlsoBroken {
    public AlsoBroken(OrderRepository repo) { }    // only constructor
}

// WORKS: explicit no-arg constructor alongside the injecting one
@ApplicationScoped
public class Works {
    private final OrderRepository repo;

    protected Works() { this.repo = null; }        // for the proxy
    @Inject
    public Works(OrderRepository repo) { this.repo = repo; }
}

// SIMPLER: use @Singleton, which needs no proxy
@Singleton
public class Simpler {
    private final OrderRepository repo;
    public Simpler(OrderRepository repo) { this.repo = repo; }
}

```

Common gotcha. Because a client proxy subclasses the bean, calling a method on the proxy that reads a **field** directly returns the proxy's (uninitialised) field, not the real instance's. Always access state through methods. This is the classic “my field is null but the getter works” confusion.

3.6 Injection Styles

```

@ApplicationScoped
public class OrderService {

    // 1. CONSTRUCTOR INJECTION - preferred
    private final OrderRepository repository;
    private final PricingEngine pricing;

    public OrderService(OrderRepository repository, PricingEngine pricing) {
        this.repository = repository;
        this.pricing = pricing;
    }
    // @Inject is optional when there is exactly one constructor

    // 2. FIELD INJECTION - concise, but untestable without a container
    @Inject OrderRepository repo2;

    // 3. METHOD INJECTION - rare; useful for optional collaborators
    private AuditSink audit;
    @Inject

```

```
void setAudit(AuditSink audit) { this.audit = audit; }
}
```

Prefer constructor injection. It makes dependencies explicit and mandatory, permits `final` fields, and lets you write plain JUnit tests with `new OrderService(mockRepo, mockPricing)` — no container, milliseconds instead of seconds. The proxy requirement for a no-arg constructor is the one wrinkle; use `@Singleton` or add a `protected` no-arg constructor.

3.6.1 Injecting collections and optionals

```
@ApplicationScoped
public class NotificationDispatcher {

    // All beans implementing the interface
    @Inject @Any Instance<NotificationChannel> channels;

    // Optional dependency - may or may not be present
    @Inject Instance<MetricsCollector> metrics;

    public void dispatch(Notification n) {
        for (NotificationChannel c : channels) {
            c.send(n);
        }
        if (metrics.isResolvable()) {
            metrics.get().count("notifications");
        }
    }
}
```

`Instance<T>` is also how you break an initialisation cycle you cannot restructure, and how you do conditional lookup — see §3.9.

3.7 Bean Removal — The Quarkus-Specific Trap

ArC removes beans nothing injects. A bean is a **removal candidate** when it is:

- not annotated `@Named`,
- not injected anywhere,
- not a producer/observer target,
- not otherwise marked as required.

This is usually invisible and beneficial. It bites when a bean is only reached dynamically:

```
// Only ever obtained via programmatic lookup -> removed at build time
@ApplicationScoped
public class SmsChannel implements NotificationChannel { }

// At runtime: UnsatisfiedResolutionException
NotificationChannel c = CDI.current()
    .select(NotificationChannel.class, new NamedLiteral("sms")).get();
```


Fix 1 — @Unremovable:

```
@ApplicationScoped
@Unremovable
public class SmsChannel implements NotificationChannel { }
```

Fix 2 — inject Instance<T> somewhere, which counts as a use:

```
@Inject @Any Instance<NotificationChannel> channels; // keeps all of them
```

Fix 3 — make it unremovable globally for a package or annotation, from an extension or via config:

```
quarkus.arc.unremovable-types=com.acme.orders.channels.*
```

Diagnosing it. Turn on the log and read what was removed:

```
quarkus.arc.remove-unused-beans=all # all | none | fwk (framework only)
quarkus.log.category."io.quarkus.arc.processor".level=DEBUG
```

The Dev UI's CDI page lists removed beans and the reason, which is the fastest route to an answer.

Do not set `remove-unused-beans=none` to “fix” this. It disables the optimisation globally, bloating the image and slowing startup, to work around a problem that `@Unremovable` solves precisely. Interviewers notice the difference between a targeted fix and a global switch.

3.8 Producers, Disposers and Qualifiers

3.8.1 Producer methods

For beans you do not own, or that need construction logic:

```
@ApplicationScoped
public class ClientProducers {

    @Produces
    @ApplicationScoped
    public S3Client s3Client(StorageConfig config) {
        return S3Client.builder()
            .region(Region.of(config.region()))
            .endpointOverride(config.endpoint())
            .build();
    }

    // Disposer: called when the bean's context is destroyed
    public void closeS3(@Disposes S3Client client) {
        client.close();
    }

    @Produces
    @Named("reportExecutor")
    @ApplicationScoped
    public ExecutorService reportExecutor() {
```

```

        return Executors.newVirtualThreadPerTaskExecutor(); // Java 21
    }

    public void shutdown(@Disposes @Named("reportExecutor") ExecutorService ex) {
        ex.shutdown();
    }
}

```

A producer without a disposer is a resource leak for anything holding a socket, file handle or thread pool. It is easy to forget because nothing fails immediately — you just leak on every hot reload in dev and on every context destruction in prod.

3.8.2 Qualifiers

When several beans implement the same interface, a qualifier disambiguates. Prefer a typed qualifier over `@Named`, because `@Named` is a `String` and typos are runtime problems.

```

@Qualifier
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.FIELD, ElementType.METHOD, ElementType.PARAMETER})
public @interface Channel {
    Type value();
    enum Type { EMAIL, SMS, PUSH }
}

```

```

@ApplicationScoped
@Channel(Channel.Type.EMAIL)
public class EmailChannel implements NotificationChannel { }

@ApplicationScoped
@Channel(Channel.Type.SMS)
public class SmsChannel implements NotificationChannel { }

@ApplicationScoped
public class Dispatcher {
    @Inject @Channel(Channel.Type.EMAIL) NotificationChannel email;
    @Inject @Channel(Channel.Type.SMS) NotificationChannel sms;
}

```

Selecting dynamically requires an `AnnotationLiteral`:

```

public class ChannelLiteral extends AnnotationLiteral<Channel> implements Channel {
    private final Type value;
    public ChannelLiteral(Type value) { this.value = value; }
    @Override public Type value() { return value; }
}

// usage
NotificationChannel c = channels.select(new ChannelLiteral(Channel.Type.SMS)).get();

```

3.9 Conditional Beans

Runtime `if` statements to pick an implementation are a code smell in Quarkus — the framework can decide at build time and delete the loser entirely.

```
public interface PaymentGateway { PaymentResult charge(Charge c); }

@ApplicationScoped
@IfBuildProfile("prod")
public class StripeGateway implements PaymentGateway { }

@ApplicationScoped
@UnlessBuildProfile("prod")
public class FakeGateway implements PaymentGateway {
    public PaymentResult charge(Charge c) { return PaymentResult.approved("fake"); }
}

// Include only if a build-time property has a given value
@ApplicationScoped
@IfBuildProperty(name = "orders.audit.enabled", stringValue = "true")
public class AuditingOrderListener { }

// Resolve at RUNTIME lookup rather than build time
@ApplicationScoped
@LookupIfProperty(name = "orders.channel", stringValue = "sms")
public class SmsChannel implements NotificationChannel { }
```

Annotation	Decided	Loser is
<code>@IfBuildProfile</code> / <code>@UnlessBuildProfile</code>	Build time	Removed from the image
<code>@IfBuildProperty</code> / <code>@UnlessBuildProperty</code>	Build time	Removed from the image
<code>@LookupIfProperty</code> / <code>@LookupUnlessProperty</code>	Runtime, per lookup	Kept in the image
<code>@Alternative</code> + <code>@Priority</code>	Build time	Kept but not injected

Why this matters beyond tidiness: a build-time decision means the unused implementation — and its transitive dependencies, and its reflection registrations — are absent from the native image. `@IfBuildProfile("prod")` on the real gateway means your test double is not in the production binary at all. That is a security property, not just a size one.

3.10 Interceptors and Decorators

3.10.1 Interceptors — cross-cutting behaviour

```
@InterceptorBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface Timed {
    String value() default "";
}
```

```
@Timed
@Interceptor
@Priority(Interceptor.Priority.APPLICATION)
public class TimedInterceptor {

    @Inject MeterRegistry registry;

    @AroundInvoke
    public Object time(InvocationContext ctx) throws Exception {
        Timed annotation = ctx.getMethod().getAnnotation(Timed.class);
        String name = (annotation != null && !annotation.value().isBlank())
            ? annotation.value()
            : ctx.getTarget().getClass().getSimpleName() + "." + ctx.getMethod().getName();

        Timer.Sample sample = Timer.start(registry);
        try {
            return ctx.proceed();
        } finally {
            sample.stop(registry.timer("method.duration", "method", name));
        }
    }
}
```

```
@ApplicationScoped
public class OrderService {
    @Timed("orders.create")
    public OrderView create(CreateOrderRequest r) { ... }
}
```

Interceptors do not fire on self-invocation. Calling `this.create(...)` from another method in the same class bypasses the proxy entirely, so no interceptor runs. This is true of `@Transactional`, `@CacheResult`, `@Timed` and every other interceptor-based annotation, and it is a very common interview question. If you need interception on an internal call, inject the bean into itself, or extract the method into a separate bean — the latter is usually the better design signal.

```
@ApplicationScoped
public class OrderService {

    @Inject OrderService self;           // legal; the proxy breaks the cycle

    public void batch(List<CreateOrderRequest> rs) {
        for (var r : rs) {
            self.create(r);              // intercepted
        }
    }
}
```

```

        // this.create(r);           // NOT intercepted
    }
}

@Transactional
public OrderView create(CreateOrderRequest r) { ... }
}

```

Common built-in interceptor bindings: `@Transactional`, `@CacheResult`, `@CacheInvalidate`, `@Retry`, `@Timeout`, `@CircuitBreaker`, `@Fallback`, `@Bulkhead`, `@Authenticated`, `@RolesAllowed`, `@Scheduled`, `@Blocking`.

3.10.2 Decorators

A decorator wraps a specific *interface*, giving you the delegate. Use it when the added behaviour is domain logic rather than a cross-cutting concern.

```

@Decorator
@Priority(Interceptor.Priority.APPLICATION)
public abstract class DiscountingPricingEngine implements PricingEngine {

    @Inject @Delegate PricingEngine delegate;
    @Inject LoyaltyService loyalty;

    @Override
    public BigDecimal price(Order order) {
        BigDecimal base = delegate.price(order);
        return base.multiply(loyalty.discountFor(order.customerId()));
    }
}

```

Interceptor or decorator? An interceptor is generic — it does not know or care what method it wraps (logging, timing, transactions). A decorator is type-aware — it implements the interface and can add domain behaviour. If your interceptor is doing `if (method.getName().equals("price"))`, you wanted a decorator.

3.11 Events

CDI events decouple producers from consumers within a single process.

```

public record OrderPlaced(long orderId, String customerEmail, BigDecimal total) { }

@ApplicationScoped
public class OrderService {

    @Inject Event<OrderPlaced> events;

    @Transactional
    public OrderView create(CreateOrderRequest r) {
        Order order = persist(r);
        events.fire(new OrderPlaced(order.id, order.email, order.total)); // synchronous
    }
}

```

```

        return OrderView.of(order);
    }
}

```

```

@ApplicationScoped
public class OrderEventHandlers {

    // Synchronous: runs on the caller's thread, inside the caller's transaction
    void onPlaced(@Observes OrderPlaced event) {
        Log.infof("Order %d placed", event.orderId());
    }

    // Transactional observer: runs only AFTER the transaction commits
    void sendConfirmation(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) {
        mailer.sendConfirmation(e.customerEmail(), e.orderId());
    }

    // Asynchronous: different thread, separate transaction, exceptions isolated
    void indexForSearch(@ObservesAsync OrderPlaced event) {
        searchIndex.index(event.orderId());
    }

    // Ordered: lower priority runs first
    void first(@Observes @Priority(100) OrderPlaced e) { }
    void then(@Observes @Priority(200) OrderPlaced e) { }
}

```

3.11.1 The transactional-observer point

`TransactionPhase.AFTER_SUCCESS` is important and under-used. Consider sending a confirmation email:

- A **synchronous** observer sends the email *inside* the transaction. If the transaction then rolls back, you have emailed a customer about an order that does not exist.
- An **AFTER_SUCCESS** observer sends it only after commit. Correct.

Available phases: `IN_PROGRESS` (default, synchronous), `BEFORE_COMPLETION`, `AFTER_COMPLETION`, `AFTER_FAILURE`, `AFTER_SUCCESS`.

Scope limit worth stating in an interview: CDI events are **in-process only**. They do not cross service boundaries. Reaching for them to integrate two microservices is a category error — that is Kafka's job (Chapter 11). CDI events are for decoupling *within* a deployable unit.

3.12 Context Propagation

On the event loop, work moves between threads. `@RequestScoped` beans, the security identity and the transaction are all context-bound, so they must be propagated explicitly.

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-context-propagation</artifactId>
</dependency>
```

```
@ApplicationScoped
public class ReportService {

    @Inject ManagedExecutor executor;           // propagates CDI, transaction, security
    @Inject ThreadContext threadContext;

    public Uni<Report> generate(String period) {
        return Uni.createFrom()
            .completionStage(executor.supplyAsync(() -> buildReport(period)));
    }

    public void manual() {
        Runnable task = threadContext.contextualRunnable(() -> {
            // request context is available here
        });
        new Thread(task).start();
    }
}
```

```
// Activate a request context on a thread that has none - e.g. a scheduled job
@ActivateRequestContext
public void nightlyReconciliation() {
    orderService.reconcile();           // @RequestScoped beans now work
}
```

Choose what to propagate
quarkus.smallrye-context-propagation.enabled=true

Mutiny propagates context automatically for `Uni`/`Multi` chains when the extension is present. Raw `CompletableFuture` and hand-rolled threads do not.

3.13 Testing CDI Code

3.13.1 Plain JUnit — the fastest option

Constructor injection makes this possible, and it is by far the best feedback loop:

```
class OrderServiceTest {
    @Test
    void appliesDiscount() {
        var repo = mock(OrderRepository.class);
        var pricing = mock(PricingEngine.class);
        when(pricing.price(any())).thenReturn(new BigDecimal("90.00"));

        var service = new OrderService(repo, pricing); // no container at all

        assertThat(service.quote(anOrder()).isEqualByComparingTo("90.00"));
    }
}
```

```
    }
}
```

Milliseconds, not seconds. If your service cannot be tested this way, that is usually a design signal rather than a testing problem.

3.13.2 @QuarkusTest with @InjectMock

```
@QuarkusTest
class OrderServiceIntegrationTest {

    @Inject OrderService service;
    @InjectMock PricingEngine pricing;           // replaces the bean for this test

    @Test
    void usesMockedPricing() {
        when(pricing.price(any())).thenReturn(new BigDecimal("42.00"));
        assertThat(service.quote(anOrder())).isEqualByComparingTo("42.00");
    }
}
```

`@InjectMock` works on normal-scoped beans because the client proxy can be redirected. For `@Singleton` Quarkus handles it too, but `@InjectMock` cannot mock a `@Dependent` bean — there is no single instance to replace.

3.13.3 @QuarkusComponentTest — the middle ground

Starts a minimal CDI container with only the beans you name. Much faster than `@QuarkusTest`, more realistic than plain JUnit:

```
@QuarkusComponentTest
class PricingEngineTest {

    @Inject PricingEngine engine;
    @InjectMock TaxService taxService;

    @Test
    void addsTax() {
        when(taxService.rateFor("GB")).thenReturn(new BigDecimal("0.20"));
        assertThat(engine.price(orderIn("GB"))).isEqualByComparingTo("120.00");
    }
}
```

3.13.4 Alternatives for test doubles

```
@Alternative
@Priority(1)
@ApplicationScoped
public class InMemoryOrderRepository implements OrderRepository { }
```


Place it in `src/test/java` and it replaces the real repository for all tests in the module — useful when many tests need the same double.

3.14 Best Practices

1. **Constructor injection by default.** Explicit, testable, permits `final`.
2. **`@ApplicationScoped` by default; `@Singleton` when you need `final` fields or eager creation.**
3. **Application-scoped beans must be stateless** — or their state immutable or explicitly thread-safe.
4. **Typed qualifiers over `@Named`.** Compile-time safety instead of string typos.
5. **Every producer of a closeable resource needs a `@Disposes` method.**
6. **Build-time conditionals (`@IfBuildProfile`) over runtime `if`.** The loser is removed from the image.
7. **`@Unremovable` for dynamically-looked-up beans** — never `remove-unused-beans=none`.
8. **Index shared modules with the Jandex plugin** at the source.
9. **`TransactionPhase.AFTER_SUCCESS` for any observer with an external side effect.**
10. **Extract, do not self-invoke.** If you need interception on an internal call, the method probably belongs in another bean.
11. **Keep CDI events in-process.** Cross-service communication is a messaging concern.
12. **Prefer `@QuarkusComponentTest` or plain JUnit to `@QuarkusTest`** when you are testing a component rather than the application.

3.15 Common Mistakes

1. **Mutable state on an `@ApplicationScoped` bean.** Works in tests, races in production.
2. **Expecting interceptors on self-invocation.** `this.method()` bypasses the proxy. Affects `@Transactional`, `@CacheResult`, `@Timed`, everything.
3. **`final` class or missing no-arg constructor on a normal-scoped bean.** Build failure: cannot create a client proxy.
4. **Beans in an unindexed JAR.** Unsatisfied dependency with a confusing message. Add the Jandex plugin.
5. **Dynamically-looked-up bean removed at build time.** `UnsatisfiedResolutionException` at runtime. Add `@Unremovable`.
6. **Producer without a disposer.** Leaks connections, file handles and threads — invisibly.
7. **`@Dependent` bean injected into an `@ApplicationScoped` bean.** You get exactly one instance for the application's lifetime, which is almost never what `@Dependent` was chosen for.
8. **Using `@RequestScoped` on a thread with no request context.** `ContextNotActiveException`. Use `@ActivateRequestContext` or `ManagedExecutor`.
9. **Reading a field through a proxy.** Returns the proxy's uninitialised field. Always go through methods.
10. **Circular `@Singleton` dependencies.** No proxy to break the cycle, so the build fails. Switch one side to `@ApplicationScoped`, or — better — reconsider the design; a cycle usually means the responsibility split is wrong.

3.16 Interview Questions

Junior

Q1. What is CDI? Contexts and Dependency Injection, the Jakarta standard component model: the container manages bean lifecycles (contexts) and supplies dependencies (injection) rather than you constructing objects.

Q2. How do you make a class a bean in Quarkus? Annotate it with a scope — `@ApplicationScoped`, `@Singleton`, `@RequestScoped`, `@Dependent` — or another bean-defining annotation such as `@Path`.

Q3. What does `@Inject` do? Marks an injection point. The container supplies a matching bean by type and qualifiers.

Q4. Name the common scopes. `@ApplicationScoped` (one per app), `@Singleton` (one per app, unproxied), `@RequestScoped` (one per request), `@Dependent` (one per injection point), `@SessionScoped` (one per HTTP session).

Q5. What is a qualifier? An annotation that disambiguates between multiple beans of the same type, e.g. `@Channel(EMAIL)`.

Q6. What is a producer method? A `@Produces` method that creates a bean the container cannot instantiate itself — typically a third-party type needing construction logic.

Q7. How do you mock a bean in a test? `@InjectMock` in a `@QuarkusTest`, or construct the class directly with mock collaborators if you used constructor injection.

Mid-level

Q8. What is ArC? Quarkus's CDI 4.1 implementation. It resolves the bean graph at build time and generates a class per bean, so runtime injection involves no reflection and no bean resolution — just direct calls. It also removes unused beans and generates client proxies.

Q9. `@ApplicationScoped` vs `@Singleton`? `@ApplicationScoped` is proxied and lazy: the instance is created on first method call, the class must not be `final` and needs a no-arg constructor, and circular references work. `@Singleton` is unproxied and eager: created when first injected, may be `final`, no no-arg constructor needed, no proxy indirection, but circular references fail.

Q10. Why does a normal-scoped bean need a client proxy? So a long-lived bean can hold a reference to a shorter-lived one and still get the correct contextual instance per call; so lazy instantiation is possible; and so circular references resolve. The proxy resolves the current contextual instance on every invocation.

Q11. Why are my beans not being discovered from a library JAR? Quarkus discovers beans from a Jandex index. A dependency is indexed only if it ships `META-INF/jandex.idx`, contains a `beans.xml`, or is listed under `quarkus.index-dependency.*`. Add the `jandex-maven-plugin` to the library.

Q12. What is bean removal and how do you prevent it? ArC deletes beans nothing injects. If a bean is only obtained by programmatic lookup, it gets removed and lookup fails at runtime. Fix with `@Unremovable`, or by injecting `Instance<T>` somewhere, or `quarkus.arc.unremovable-types`. Do not disable removal globally.

Q13. Why doesn't my `@Transactional` annotation work on an internal call? Interceptors are applied by the proxy. `this.method()` calls the instance directly and bypasses the proxy, so no interceptor runs. Inject the bean into itself and call through the proxy, or move the method to a separate bean.

Q14. Interceptor vs decorator? An interceptor is type-agnostic cross-cutting behaviour (logging, timing, transactions) triggered by an interceptor binding. A decorator implements a specific interface, receives the `@Delegate`, and adds domain behaviour to that type.

Q15. What is `@Dependent` and when would you use it? A pseudo-scope giving a new instance per injection point, with a lifecycle tied to the injecting bean. Use it for stateful short-lived helpers. Be aware that injecting one into an `@ApplicationScoped` bean gives a single instance for the application's lifetime.

Q16. How do CDI events work and when are they the wrong tool? `Event<T>.fire()` synchronously invokes all matching `@Observes` methods; `fireAsync` / `@ObservesAsync` runs them on another thread. `TransactionPhase.AFTER_SUCCESS` defers an observer until after commit. They are in-process only — using them to integrate separate services is a category error.

Senior

Q17. Explain how ArC resolves and generates beans at build time. During augmentation ArC reads the Jandex index and collects classes with bean-defining annotations, producer methods and synthetic beans contributed by extensions. It builds the full bean graph and resolves every injection point by type and qualifiers, failing the build on ambiguity or unsatisfied dependencies. It then computes reachability and removes beans that nothing uses. For each surviving bean it generates an `InjectableBean` implementation containing a `create` method that instantiates the bean and supplies its dependencies through direct calls to other generated beans, plus a client proxy subclass for normal-scoped beans and generated interceptor chains where bindings apply. At runtime the container is a map of pre-built bean instances — no reflection, no scanning, no resolution.

Q18. What are the trade-offs of build-time DI? Gains: no runtime reflection, so faster startup and lower memory; wiring errors become build errors; dead beans are removed, shrinking the image; and the graph is statically analysable, which is what makes native compilation feasible. Costs: the bean set is fixed at build time, so you cannot register beans from runtime data; dynamically-looked-up beans need `@Unremovable`; proxy requirements constrain class design (no `final`, no-arg constructor); and debugging generated bean classes is less transparent than stepping through a reflective container. There is also a conceptual cost — developers must understand that “removed” is a thing that can happen to their bean.

Q19. How do you handle a bean whose implementation must differ per environment? Build-time selection with `@IfBuildProfile` / `@IfBuildProperty`, so the unused implementation is absent from the image entirely — which is a security property, not just a size one: a test double or an in-memory gateway should not exist in a production binary. Where the choice genuinely must be made at runtime — a feature flag toggled without redeploying — `@LookupIfProperty` with `Instance<T>`, accepting that both implementations ship. I would avoid a runtime `if` inside a single bean, because it hides the variation from the container and from anyone reading the wiring.

Q20. A colleague put `quarkus.arc.remove-unused-beans=none` in `application.properties` to fix an injection error. Critique that. It works, and it is the wrong fix. Removal is one of

ArC's main contributions — disabling it globally keeps every unused bean, its interceptor chains and its reflection registrations in the image, increasing size and startup time, and in native mode potentially dragging in whole dependency trees that would otherwise be eliminated. It also hides the actual problem, which is that one specific bean is reached in a way the build cannot see. The targeted fix is `@Unremovable` on that bean, or injecting `Instance<T>` where it is used. More broadly, I would treat the setting as a code smell in review: it converts a precise, local statement about one bean into a global change in build semantics.

Q21. How would you propagate a tenant identifier through an asynchronous pipeline?

The mechanism depends on the boundary. Within a Mutiny chain with `quarkus-smallrye-context-propagation` present, the CDI request context propagates automatically, so a `@RequestScoped TenantContext` bean works. Crossing to a `ManagedExecutor` propagates it too. Crossing to a raw thread or a `CompletableFuture` does not, so I would use `ThreadContext.contextualRunnable`, or pass the tenant explicitly as a parameter. For messaging boundaries — Kafka, HTTP — context does not propagate at all; the tenant must be an explicit header, and the consumer side re-establishes the context. My general preference is to pass it explicitly at boundaries and only rely on context propagation *within* a request, because implicit context that silently becomes null is a hard bug class to find.

Principal

Q22. Your team's services have grown large `@ApplicationScoped` “service” classes with a dozen injected dependencies. How do you address it? The injection count is a symptom, not the disease — a class with twelve collaborators has twelve reasons to change. I would not mandate a limit, because arbitrary limits get gamed by grouping dependencies into a holder object. Instead I would look at whether the class has multiple cohesive clusters of methods that use disjoint subsets of the dependencies; that is the natural seam, and splitting along it usually produces two or three classes with three or four dependencies each. Where the class is genuinely orchestrating a workflow, the dependencies may be legitimate, and the better move is to make the orchestration explicit — a pipeline of steps, each its own bean, composed by the orchestrator. I would introduce this on one class as a worked example rather than as a rule, and use an architecture test to prevent regression only after the team agrees the target shape. Constructor injection helps here, incidentally, because a twelve-parameter constructor is visibly uncomfortable in a way twelve `@Inject` fields are not.

Q23. When would you write a custom CDI extension (a Quarkus extension contributing synthetic beans) rather than ordinary application code? When the same wiring pattern is repeated across many services and the repetition is mechanical: registering a bean per annotated type, contributing configuration-derived beans, or enforcing an organisational convention like automatic audit interception. The test is whether the logic is *about* the wiring rather than about the domain. Concretely I would write one when I find myself asking teams to copy a `Producers` class between repositories — that is a signal that a build step generating synthetic beans would remove the duplication and let the convention evolve centrally. The cost is real: extension code is harder to write, harder to debug and needs its own tests with `QuarkusUnitTest`. So the threshold is roughly three or more consumers plus an expectation that the convention will change. Below that, a shared library with a `@Produces` class is simpler and sufficient.

3.17 Cheat Sheet

Scopes

```

@ApplicationScoped // one per app, proxied, lazy      <- default choice
@Singleton         // one per app, no proxy, eager
@RequestScoped     // one per request, proxied
@Dependent         // one per injection point
@SessionScoped     // one per HTTP session

```

Injection

```

public Svc(Repo r) { }           // constructor (preferred)
@Inject Repo repo;               // field
@Inject void setRepo(Repo r) { } // method
@Inject @Any Instance<Ch> channels; // all implementations
@Inject Instance<Metrics> optional; // optional: isResolvable()

```

Qualifiers

```

@Qualifier @Retention(RUNTIME)
public @interface Channel { Type value(); }

@Inject @Channel(EMAIL) NotificationChannel c;

```

Producers

```

@Produces @ApplicationScoped S3Client s3() { ... }
void close(@Disposes S3Client c) { c.close(); }

```

Conditional beans

```

@IfBuildProfile("prod")           // build time; loser removed
@UnlessBuildProfile("prod")
@IfBuildProperty(name="x", stringValue="y")
@LookupIfProperty(name="x", stringValue="y") // runtime lookup
@Alternative @Priority(1)

```

Interceptors

```

@InterceptorBinding @Retention(RUNTIME) @interface Timed { }

@Timed @Interceptor @Priority(Interceptor.Priority.APPLICATION)
class TimedInterceptor {
    @AroundInvoke Object t(InvocationContext c) throws Exception { return c.proceed(); }
}

```

Events

```

@Inject Event<OrderPlaced> events;  events.fire(e);  events.fireAsync(e);

void on(@Observes OrderPlaced e) { }
void after(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) { }
void async(@ObservesAsync OrderPlaced e) { }

```

Lifecycle

```

@PostConstruct void init() { }
@PreDestroy   void close() { }
void start(@Observes StartupEvent e) { }
@Startup @ApplicationScoped class Eager { }

```

Troubleshooting

```

quarkus.arc.remove-unused-beans=all
quarkus.arc.unremovable-types=com.acme.channels.*
quarkus.log.category."io.quarkus.arc.processor".level=DEBUG
quarkus.index-dependency.shared.group-id=com.acme
quarkus.index-dependency.shared.artifact-id=shared-domain

```

3.18 Summary

ArC is CDI with the resolution moved to build time. That single change gives you compile-time wiring errors, zero-reflection injection, dead-bean elimination and native-image compatibility — and in exchange imposes rules that surprise people coming from runtime containers: beans must be discoverable through a Jandex index, dynamically-looked-up beans must be marked `@Unremovable`, and normal-scoped beans must be proxyable.

The two ideas to internalise are **scopes** and **proxies**. Scope determines how many instances exist and therefore whether shared state is a bug; `@ApplicationScoped` beans must be stateless. Proxies explain almost every otherwise-confusing CDI behaviour in Quarkus: why `final` classes fail, why self-invocation skips interceptors, why field access through a proxy returns null, and why a long-lived bean can safely hold a request-scoped one.

Chapter 4 turns to **configuration and profiles** — SmallRye Config in depth, the build-time versus runtime property distinction that catches teams in production, `@ConfigMapping` validation, secret management, and the configuration strategies that make “build once, deploy everywhere” actually work.

3.19 References

- Quarkus CDI reference — <https://quarkus.io/guides/cdi-reference>
- Introduction to CDI — <https://quarkus.io/guides/cdi>
- ArC on GitHub — <https://github.com/quarkusio/quarkus/tree/main/independent-projects/arc>
- Jakarta CDI 4.1 specification — <https://jakarta.ee/specifications/cdi/4.1/>
- Context propagation — <https://quarkus.io/guides/context-propagation>
- Testing components — <https://quarkus.io/guides/getting-started-testing>
- Jandex — <https://github.com/smallrye/jandex>

Configuration and Profiles

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · SmallRye Config · MicroProfile Config 3.1

Learning Objectives

By the end of this chapter you will be able to:

1. Describe SmallRye Config’s source model and resolve any property by hand, given the ordinals.
2. Map any property name to its environment-variable form — including quoted and indexed keys.
3. Choose between `@ConfigProperty` and `@ConfigMapping`, and explain why the latter is better operationally.
4. Explain the **build-time versus runtime** property distinction and design a configuration strategy around it.
5. Use profiles — built-in, custom, parent — and profile-aware files.
6. Validate configuration so that a misconfigured pod fails its readiness probe instead of serving broken traffic.
7. Wire Kubernetes ConfigMaps, Secrets and Vault into a Quarkus application.
8. Write a custom `ConfigSource` and a custom `Converter`.
9. Override configuration cleanly in tests.
10. Answer 15+ interview questions on configuration, including the “build once, deploy everywhere” design question.

4.1 The Model

Quarkus configuration is **SmallRye Config**, an implementation of MicroProfile Config with substantial Quarkus-specific extensions. The model is deliberately simple:

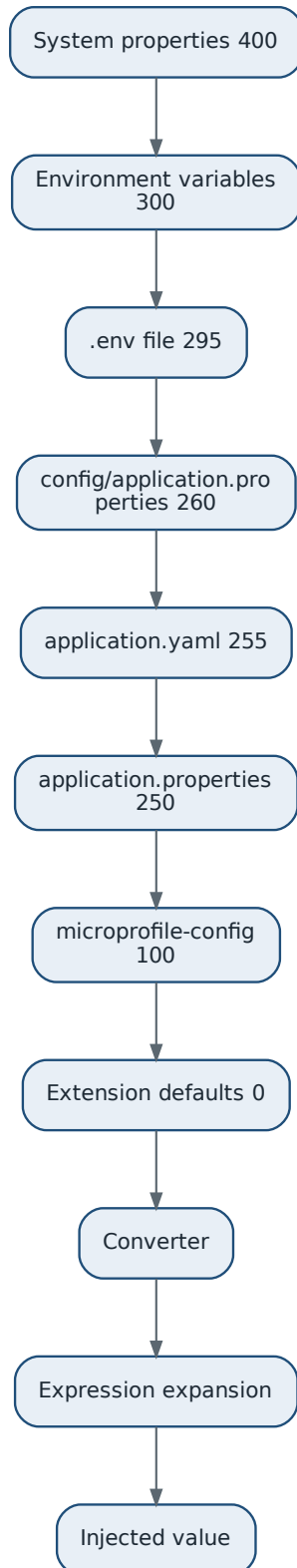


Figure 4.1 — Resolution order. Sources are consulted top to bottom by descending ordinal; the first one that defines the key wins outright. The resolved string is then converted and expression-expanded.

A lookup for `orders.max-line-items` asks every source in descending ordinal order and takes the first hit. There is no merging of values — the highest-ordinal source that defines the key wins outright.

4.2 Config Sources and Ordinals

Source	Ordinal	Notes
System properties	400	<code>-Dorders.max-line-items=10</code>
Environment variables	300	<code>ORDERS_MAX_LINE_ITEMS=10</code>
<code>.env</code> in the working directory	295	Local development only — gitignore it
<code>\$PWD/config/application.properties</code>	260	External override next to the jar; no rebuild needed
<code>application.yaml</code> / <code>.yaml</code>	255	Requires <code>quarkus-config-yaml</code>
<code>src/main/resources/application.properties</code>	250	The packaged default
<code>META-INF/microprofile-config.properties</code>	100	Library-supplied defaults
Extension defaults	~0	What Quarkus ships

4.2.1 The `config/` directory

Ordinal 260 puts `config/application.properties` **above** the packaged file. Deploy layout:

```
/opt/orders/
├─ quarkus-app/
│   └─ quarkus-run.jar
└─ config/
    └─ application.properties      # ops-editable, no rebuild
```

This is the intended mechanism for on-premise and VM deployments where a ConfigMap is not available. It also works next to a native binary.

4.2.2 Environment variable mapping

MicroProfile defines the mapping, and Quarkus applies it strictly. To convert a property name:

1. Uppercase every letter.

2. Replace every character that is **not** A–Z or 0–9 with `_`.

```
quarkus.datasource.jdbc.url      -> QUARKUS_DATASOURCE_JDBC_URL
orders.max-line-items           -> ORDERS_MAX_LINE_ITEMS
orders.retry.jitter-ms          -> ORDERS_RETRY_JITTER_MS
quarkus.log.category."com.acme".level -> QUARKUS_LOG_CATEGORY__COM_ACME__LEVEL
quarkus.rest-client.payments-api.url -> QUARKUS_REST_CLIENT_PAYMENTS_API_URL
```

Note the quoted segment: `."com.acme".` contains `.`, `"`, `c`... — the quotes *and* the dots each become `_`, which is why you see the doubled underscores. Getting this wrong is one of the most common Kubernetes configuration bugs, and it fails silently: the variable is simply never read.

Practical defence. Do not hand-write environment variable names for anything non-trivial. Either use a ConfigMap mounted as `config/application.properties` (ordinal 260, real property syntax, no mapping to get wrong), or log the resolved value of critical properties at startup so a mistake is visible immediately.

4.2.3 Indexed and map properties

```
# Indexed collections
orders.notify-emails[0]=ops@acme.com
orders.notify-emails[1]=oncall@acme.com
# equivalently, comma-separated
orders.notify-emails=ops@acme.com,oncall@acme.com
```

```
# Maps
orders.region-codes.uk=GB
orders.region-codes.germany=DE
```

```
@ConfigMapping(prefix = "orders")
public interface OrdersConfig {
    List<String> notifyEmails();
    Map<String, String> regionCodes();
}
```

To put a literal comma in a list element, escape it: `a\\,b,c` is two elements.

4.3 Reading Configuration

4.3.1 @ConfigMapping — the modern approach

```
package com.acme.orders.config;

import io.smallrye.config.*;
import jakarta.validation.constraints.*;
import java.time.Duration;
import java.util.*;

@ConfigMapping(prefix = "orders")
public interface OrdersConfig {

    @WithDefault("50")
```

```

@Min(1) @Max(500)
int maxLineItems();

@WithDefault("30s")
Duration checkoutTimeout();

@NotEmpty
List<String> notifyEmails();

Optional<String> legacyEndpoint();

Map<String, String> regionCodes();

Retry retry();
Storage storage();

interface Retry {
    @WithDefault("3") @Min(0) @Max(10) int maxAttempts();
    @WithDefault("PT1S") Duration backoff();
    @WithName("jitter-ms") @WithDefault("100") int jitterMillis();
}

interface Storage {
    @WithDefault("s3") Kind kind();
    Optional<String> bucket();
    @WithDefault("eu-west-1") String region();

    enum Kind { S3, GCS, LOCAL }
}
}

```

```

orders.max-line-items=100
orders.checkout-timeout=45s
orders.notify-emails=ops@acme.com,oncall@acme.com
orders.region-codes.uk=GB
orders.retry.max-attempts=5
orders.retry.jitter-ms=250
orders.storage.kind=s3
orders.storage.bucket=acme-orders-prod

```

```

@ApplicationScoped
public class CheckoutService {
    private final OrdersConfig config;

    public CheckoutService(OrdersConfig config) { // injected like any bean
        this.config = config;
    }

    public void validate(List<LineItem> items) {
        if (items.size() > config.maxLineItems()) {
            throw new TooManyLineItemsException(config.maxLineItems());
        }
    }
}

```

```
    }
}
```

Why `@ConfigMapping` is the right default:

- **Validated at startup.** Every property is resolved and validated when the application starts, and *all* failures are reported together:

ERROR: Failed to start application

SRCFG00040: The config property orders.notify-emails is required but it could not be found

SRCFG00039: orders.retry.max-attempts must be less than or equal to 10

- **Operationally correct.** In Kubernetes, failing at startup means the readiness probe never passes and the rollout halts. A misconfigured pod never receives traffic. Compare `@ConfigProperty`, where a missing optional value may not surface until a specific code path runs — possibly hours after deployment, for one unlucky customer.
- **Type-safe and discoverable.** The interface is the documentation. Nested groups keep related settings together.
- **Zero reflection.** The implementation is generated at build time.

4.3.2 @ConfigProperty — the property-level approach

```
@ApplicationScoped
public class LegacyStyle {

    @ConfigProperty(name = "orders.max-line-items", defaultValue = "50")
    int maxLineItems;

    @ConfigProperty(name = "orders.legacy-endpoint")
    Optional<String> legacyEndpoint;           // Optional for genuinely optional keys

    // A Provider re-reads on every call - useful for values that can change
    @ConfigProperty(name = "orders.feature.new-pricing")
    Provider<Boolean> newPricingEnabled;
}
```

Fine for one or two values. Beyond that, `@ConfigMapping`.

4.3.3 Programmatic access

```
@ApplicationScoped
public class DynamicLookup {

    @Inject Config config;

    public String tenantEndpoint(String tenant) {
        return config.getOptionalValue("orders.tenant." + tenant + ".url", String.class)
            .orElseThrow(() -> new UnknownTenantException(tenant));
    }

    public void inspectSources() {
```

```

        SmallRyeConfig smallRye = config.unwrap(SmallRyeConfig.class);
        ConfigValue value = smallRye.getConfigValue("quarkus.datasource.jdbc.url");
        Log.infof("%s = %s (from %s, ordinal %d)",
            value.getName(), value.getValue(),
            value.getConfigSourceName(), value.getConfigSourceOrdinal());
    }
}

```

`ConfigValue.getConfigSourceName()` answers “where did this value come from?”, which is the question you actually have during an incident.

4.4 Types and Converters

Out of the box SmallRye converts to: primitives and wrappers, `String`, `Duration`, `Period`, `LocalDate`, `LocalDateTime`, `OffsetDateTime`, `Instant`, `URL`, `URI`, `Path`, `Pattern`, `Charset`, `InetAddress`, `UUID`, `Currency`, `BigDecimal`, `BigInteger`, enums, `Optional<T>`, `List<T>`, `Set<T>`, `Map<K,V>`, and arrays.

Durations accept both ISO-8601 and a shorthand:

```

orders.checkout-timeout=PT30S      # ISO-8601
orders.checkout-timeout=30s        # shorthand
orders.cache-ttl=5m
orders.session-ttl=2h
quarkus.shutdown.timeout=30S

```

Memory sizes accept suffixes:

```

quarkus.http.limits.max-body-size=10M
quarkus.http.limits.max-header-size=20K

```

4.4.1 A custom converter

```

package com.acme.orders.config;

import org.eclipse.microprofile.config.spi.Converter;
import jakarta.annotation.Priority;

@Priority(300) // higher than the built-in converters
public class CidrConverter implements Converter<Cidr> {
    @Override
    public Cidr convert(String value) {
        if (value == null || value.isBlank()) return null; // null -> treated as absent
        return Cidr.parse(value.trim());
    }
}

```

Register it via the ServiceLoader:

```

src/main/resources/META-INF/services/org.eclipse.microprofile.config.spi.Converter

com.acme.orders.config.CidrConverter

```

```
@ConfigMapping(prefix = "orders.network")
public interface NetworkConfig {
    List<Cidr> allowedRanges();    // "10.0.0.0/8,192.168.0.0/16"
}
```

Native-image note. ServiceLoader-registered converters work in native mode because Quarkus's config extension registers the service file. A converter loaded by any other reflective mechanism would need explicit registration.

4.5 Validation

Add `quarkus-hibernate-validator` and annotate the mapping interface. Validation runs at startup.

```
@ConfigMapping(prefix = "orders")
public interface OrdersConfig {

    @Min(1) @Max(500)
    int maxLineItems();

    @NotBlank
    @Pattern(regexp = "^https://.*", message = "must be an HTTPS URL")
    String callbackUrl();

    @NotEmpty
    List<@Email String> notifyEmails();

    @DurationMin(seconds = 1)
    @DurationMax(minutes = 5)
    Duration checkoutTimeout();
}
```

For cross-field rules, validate in a `StartupEvent` observer:

```
@ApplicationScoped
public class ConfigSanityCheck {

    void verify(@Observes StartupEvent event, OrdersConfig config) {
        if (config.storage().kind() == Storage.Kind.S3
            && config.storage().bucket().isEmpty()) {
            throw new ConfigurationException(
                "orders.storage.bucket is required when orders.storage.kind=s3");
        }
        if (config.retry().backoff().multipliedBy(config.retry().maxAttempts())
            .compareTo(config.checkoutTimeout()) > 0) {
            throw new ConfigurationException(
                "Retry budget exceeds orders.checkout-timeout; the last retry can never complete");
        }
    }
}
```

That second check is the kind of thing that only shows up under load six weeks after deployment. Asserting it at startup costs three lines.

4.6 Profiles

4.6.1 The built-in three

Profile	Active when
dev	quarkus:dev
test	@QuarkusTest / mvn test
prod	Everything else — the default

```
# Applies to every profile
orders.max-line-items=50
quarkus.http.port=8080

%dev.quarkus.log.category."com.acme".level=DEBUG
%dev.quarkus.hibernate-orm.database.generation=drop-and-create
%dev.orders.storage.kind=local

%test.quarkus.hibernate-orm.database.generation=drop-and-create
%test.orders.max-line-items=5

%prod.quarkus.hibernate-orm.database.generation=none
%prod.quarkus.log.console.json=true

# One value for several profiles
%dev,test.orders.notify-emails=dev-null@acme.com
```

4.6.2 Custom profiles

```
java -Dquarkus.profile=staging -jar target/quarkus-app/quarkus-run.jar
QUARKUS_PROFILE=staging ./target/orders-service-runner
./mvnw quarkus:dev -Dquarkus.profile=integration
```

```
%staging.quarkus.datasource.jdbc.url=jdbc:postgresql://staging-db:5432/orders
%staging.orders.notify-emails=staging-alerts@acme.com
```

4.6.3 Parent profiles

```
# staging inherits everything from prod, then overrides
%staging.quarkus.config.profile.parent=prod

%prod.quarkus.log.console.json=true
%prod.quarkus.hibernate-orm.database.generation=none
%staging.quarkus.log.level=DEBUG           # staging = prod + this
```

This removes a real duplication problem: without parent profiles, every non-prod-but-prod-like environment has to restate the whole production block, and they drift.

4.6.4 Profile-aware files

```
src/main/resources/
├─ application.properties          # shared
├─ application-dev.properties      # dev only
├─ application-staging.properties
└─ application-prod.properties
```

Cleaner than a single file with hundreds of `%profile.` prefixes once you have more than a handful of overrides.

4.6.5 Multiple active profiles

```
-Dquarkus.profile=staging, tracing-enabled
```

Later profiles win. Useful for orthogonal toggles, but keep the list short — resolution order becomes hard to reason about quickly.

4.7 Build-Time vs Runtime Properties

This is the most consequential concept in the chapter and a reliable senior interview question.

4.7.1 The distinction

Some properties are consumed during **augmentation** to generate code. Changing them afterwards has no effect, because the generated code is already baked into the artefact.

Build-time fixed (examples):

```
quarkus.datasource.db-kind=postgresql      # selects the driver, generates the pool class
quarkus.http.root-path=/api                # baked into every generated route
quarkus.hibernate-orm.packages=com.acme    # decides which entities are enhanced
quarkus.hibernate-orm.dialect=...          # generates the dialect binding
quarkus.native.additional-build-args=...
quarkus.package.jar.type=fast-jar
quarkus.index-dependency.*
quarkus.arc.remove-unused-beans
quarkus.rest-client.*.scope
```

Runtime overridable (examples):

```
quarkus.datasource.jdbc.url
quarkus.datasource.username
quarkus.datasource.password
quarkus.datasource.jdbc.max-size
quarkus.http.port
quarkus.log.level
quarkus.rest-client.*.url
quarkus.oidc.auth-server-url
```

Every property in the Quarkus documentation carries a **lock icon** when it is fixed at build time. Check it before designing a deployment strategy — not afterwards.

4.7.2 The failure mode

In native mode Quarkus warns at startup that a build-time property was changed:

```
WARN [io.qua.run.con.ConfigRecorder] Build time property cannot be changed at runtime:  
- quarkus.http.root-path was 'api' at build time and is now '/api/v2'
```

In JVM mode it may be silent. Either way, the deployment is subtly wrong.

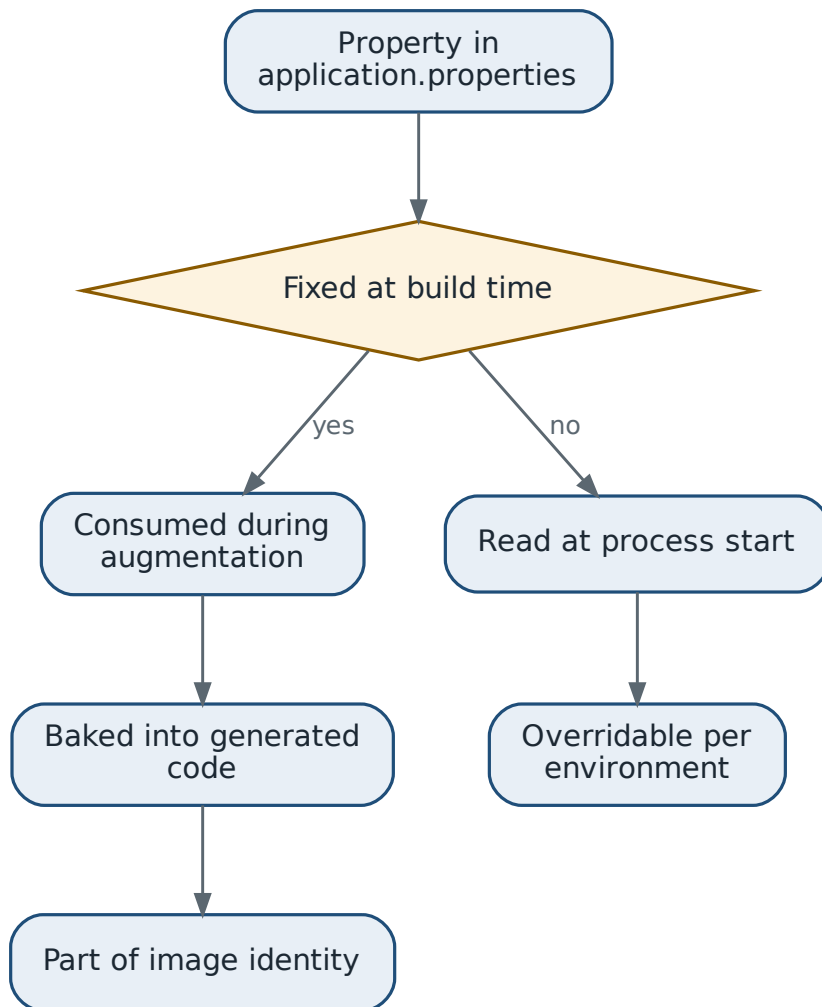


Figure 4.2 — Build-time properties belong to the image, not to the deployment.

4.7.3 Designing around it

The rule that makes “build once, deploy everywhere” work:

Build-time properties are part of the image’s identity. Runtime properties are the deployment’s configuration. Never put an environment-specific value in a build-time property.

Practical steps:

1. **Audit and classify.** Go through `application.properties` and mark each key. Anything build-time that differs by environment is a design problem to fix, not a configuration to manage.
2. **Keep build-time properties in version control**, in `application.properties`, with no profile prefixes for environment-specific values.
3. **Supply runtime properties externally** — environment variables, a mounted ConfigMap, a Secret.
4. **Assert at startup.** Log the resolved value and source of every critical property in the first lines of the pod log:

```
@ApplicationScoped
public class ConfigBanner {

    @Inject Config config;

    void log(@Observes StartupEvent e) {
        SmallRyeConfig cfg = config.unwrap(SmallRyeConfig.class);
        for (String key : List.of("quarkus.datasource.jdbc.url",
                                "quarkus.oidc.auth-server-url",
                                "orders.storage.bucket",
                                "quarkus.http.root-path")) {
            ConfigValue v = cfg.getConfigValue(key);
            Log.infof("config %-38s = %-45s [%s]",
                    key, mask(key, v.getValue()), v.getConfigSourceName());
        }
    }

    private String mask(String key, String value) {
        if (value == null) return "<unset>";
        return key.matches(".*(password|secret|token|key).") ? "*****" : value;
    }
}
```

That block turns “why is it hitting the wrong database?” from an hour of investigation into a five-second log read.

4.7.4 If a build-time property genuinely must vary

Occasionally you have a real need — for example, a different `db-kind` for one environment. Options, in order of preference:

1. **Eliminate the difference.** Use the same database technology everywhere. Environment parity is worth more than the flexibility.
2. **Build a separate image per variant**, tagged accordingly. Honest and explicit.
3. **Use a runtime-selectable abstraction.** Two datasources configured at build time, one chosen at runtime by a `@LookupIfProperty` bean.

4.8 Expressions, Defaults and Secrets

```
# Reference another property
orders.callback-url=https://${quarkus.http.host}:${quarkus.http.port}/callbacks
```

```
# Default when the referenced key is absent
orders.region=${DEPLOY_REGION:eu-west-1}

# Nested defaults
orders.endpoint=${ORDERS_ENDPOINT:${DEFAULT_ENDPOINT:https://localhost:8443}}

# Required: startup fails if DB_PASSWORD is unset
quarkus.datasource.password=${DB_PASSWORD}

# Escape a literal $
orders.template=\${literal}
```

4.8.1 Secrets: what not to do

```
# NEVER
quarkus.datasource.password=Sup3rS3cret
```

Committed secrets are the most common credential-leak vector, and Quarkus makes the alternatives easy.

Local development — `.env` in the project root, gitignored:

```
# .env (ordinal 295, above application.properties)
DB_PASSWORD=localdev
QUARKUS_OIDC_CREDENTIALS_SECRET=dev-secret
```

`.env`

Watch the ordinal. `.env` sits at 295, above `application.properties` at 250. A stale `.env` on a developer's machine silently overrides real configuration and produces bugs that reproduce for exactly one person. If someone reports behaviour nobody else sees, check their `.env` first.

Kubernetes — a Secret projected as environment variables, or mounted:

```
quarkus.kubernetes-config.enabled=true
quarkus.kubernetes-config.secrets.enabled=true
quarkus.kubernetes-config.config-maps=orders-config
quarkus.kubernetes-config.secrets=orders-secrets
quarkus.kubernetes-config.fail-on-missing-config=true
```

Vault:

```
quarkus.vault.url=https://vault.internal:8200
quarkus.vault.authentication.kubernetes.role=orders-service
quarkus.vault.secret-config-kv-path=secret/orders
```

With `quarkus-vault`, secrets appear as ordinary config properties, so application code never knows the difference — which is exactly the property you want from a secrets integration.

4.8.2 Secret rotation

`@ConfigProperty` values are resolved once at injection. For a value that must be re-read:

```

@ApplicationScoped
public class RotatingCredentials {
    // Provider re-reads on every get()
    @ConfigProperty(name = "orders.api-key")
    Provider<String> apiKey;

    public String current() { return apiKey.get(); }
}

```

Note that this only helps if the *source* can change — an environment variable cannot change in a running process, but a Vault-backed source or a periodically-reloaded file source can.

4.9 YAML Configuration

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-config-yaml</artifactId>
</dependency>

```

```

# src/main/resources/application.yaml (ordinal 255)

```

```

quarkus:
  http:
    port: 8080
    cors:
      enabled: true
      origins: "https://app.acme.com"
  datasource:
    db-kind: postgresql
    jdbc:
      max-size: 20
  log:
    category:
      "com.acme.orders":
        level: DEBUG

orders:
  max-line-items: 100
  checkout-timeout: 45s
  notify-emails:
    - ops@acme.com
    - oncall@acme.com
  retry:
    max-attempts: 5
    jitter-ms: 250

"%dev":
  quarkus:
    log:
      console:
        level: DEBUG
  orders:

```

```

storage:
  kind: local

"%prod":
  quarkus:
    log:
      console:
        json: true

```

YAML is better for deeply nested configuration and for lists. It is worse for grepping and for diffing. Pick one and be consistent; mixing both means every “where is this set?” question needs two searches.

4.10 Custom Config Sources

When configuration must come from somewhere Quarkus does not know about — a database table, a feature-flag service, a proprietary store.

```

package com.acme.orders.config;

import org.eclipse.microprofile.config.spi.ConfigSource;
import java.util.*;
import java.util.concurrent.atomic.AtomicReference;

public class FeatureFlagConfigSource implements ConfigSource {

    private final AtomicReference<Map<String, String>> cache =
        new AtomicReference<>(Map.of());

    @Override
    public int getOrdinal() {
        return 310;    // above environment variables, below system properties
    }

    @Override
    public String getName() {
        return "feature-flags";
    }

    @Override
    public Set<String> getPropertyNames() {
        return cache.get().keySet();
    }

    @Override
    public String getValue(String propertyName) {
        if (!propertyName.startsWith("orders.feature.")) {
            return null;    // null means "I don't have it" - fall through
        }
        return cache.get().get(propertyName);
    }
}

```

```

    public void refresh(Map<String, String> flags) {
        cache.set(Map.copyOf(flags));
    }
}

```

Register it:

```

src/main/resources/META-INF/services/org.eclipse.microprofile.config.spi.ConfigSource
com.acme.orders.config.FeatureFlagConfigSource

```

Caution. A config source that performs I/O in `getValue` will be called on the request path, potentially on an event loop. Cache aggressively and refresh on a schedule, as above. A config source that blocks is a subtle way to create a server-wide stall.

For a source that needs CDI beans (a database connection, an HTTP client), use `ConfigSourceFactory` instead — it is invoked later in the lifecycle when the container is available.

4.11 Configuration in Tests

4.11.1 The test profile

```

%test.quarkus.datasource.jdbc.url=jdbc:h2:mem:test
%test.orders.max-line-items=5
%test.orders.storage.kind=local

```

4.11.2 Per-test overrides

```

@QuarkusTest
@TestProfile(StrictLimitsProfile.class)
class StrictLimitsTest {

    public static class StrictLimitsProfile implements QuarkusTestProfile {

        @Override
        public Map<String, String> getConfigOverrides() {
            return Map.of(
                "orders.max-line-items", "2",
                "orders.retry.max-attempts", "0");
        }

        @Override
        public String getConfigProfile() {
            return "test-strict";
        }

        // Restrict the test to a subset of beans - much faster startup
        @Override
        public Set<Class<?>> getEnabledAlternatives() {
            return Set.of(InMemoryOrderRepository.class);
        }
    }
}

```

```

    }

    @Test
    void rejectsThirdLineItem() { ... }
}

```

Each distinct `QuarkusTestProfile` causes a Quarkus restart between test classes, so group tests that share a profile. A suite with twenty different profiles will be slow for reasons that are not obvious.

4.11.3 Overriding a single property inline

```

@QuarkusTest
class TimeoutTest {

    @Test
    @TestProfile(...)           // for a whole class
    void slow() { }
}

// Or, for a quick one-off, a system property in surefire config:
// -Dorders.checkout-timeout=1s

```

4.12 Troubleshooting

Dev UI. <http://localhost:8080/q/dev-ui/> → *Configuration*. It shows every property, its resolved value, its source, and lets you edit values live in dev mode. This is the fastest way to answer “what is it actually using?”.

Programmatic dump.

```

SmallRyeConfig cfg = ConfigProvider.getConfig().unwrap(SmallRyeConfig.class);
for (String name : cfg.getPropertyNames()) {
    ConfigValue v = cfg.getConfigValue(name);
    System.out.printf("%-60s = %-40s [%s:%d]%n",
        name, v.getValue(), v.getConfigSourceName(), v.getConfigSourceOrdinal());
}

```

Common symptoms and causes:

Symptom	Likely cause
Env var has no effect	Wrong name mapping, or the property is build-time fixed
Value differs from <code>application.properties</code>	A higher-ordinal source — check <code>.env</code> and system properties
Works locally, not in the container	<code>.env</code> present locally; missing <code>ConfigMap</code> in the cluster

Symptom	Likely cause
Native build warns “cannot be changed at runtime”	Build-time property being overridden
<code>SRCFG00014: property required but not found</code>	<code>@ConfigMapping</code> with no default and no value — usually correct behaviour

4.13 Best Practices

1. **@ConfigMapping over @ConfigProperty** for anything beyond a couple of keys. Fail at startup, not on first use.
2. **Validate configuration** with Bean Validation annotations plus a `StartupEvent` check for cross-field rules.
3. **Know which properties are build-time fixed** before designing your deployment strategy.
4. **Never commit secrets.** `.env` locally (gitignored), Secrets or Vault in the cluster.
5. **Log the resolved value and source of critical properties at startup**, with masking.
6. **Prefer a mounted `config/application.properties` to a wall of environment variables** — real property syntax, no mapping errors.
7. **Use parent profiles** so staging does not duplicate the production block.
8. **Keep environment-specific values out of the image.** If a build-time property must vary, fix the divergence rather than configure around it.
9. **Optional<T> for genuinely optional properties**; no default for genuinely required ones — an absent required property should fail the boot.
10. **One configuration format.** Properties or YAML, not both.
11. **Group tests by profile.** Each distinct profile restarts Quarkus.
12. **Cache in custom config sources.** Never do I/O per lookup.

4.14 Common Mistakes

1. **Assuming a build-time property is runtime-overridable.** The environment variable is set correctly and nothing happens. See §4.7.
2. **Mis-mapping an environment variable name.** Especially quoted keys: `quarkus.log.category."com.acme".level` → `QUARKUS_LOG_CATEGORY_COM_ACME_LEVEL`.
3. **Committing `.env`.** Ordinal 295 means it overrides `application.properties` — a stale one produces bugs only one developer can reproduce.
4. **Secrets in `application.properties`.** They end up in the image, in the registry, and in every developer’s git history.
5. **Using `@ConfigProperty` without `Optional` for an optional property.** Startup fails with a required-property error, or worse, injects an unexpected default.
6. **Doing I/O in a custom `ConfigSource.getValue`.** Called on the request path; can block an event loop.
7. **A different `QuarkusTestProfile` per test class.** Each one restarts Quarkus; a suite that takes four minutes usually has this problem.

8. Relying on property ordering within a file. There is none — the last definition of a key in a file wins, but across sources only the ordinal matters.

9. Setting `quarkus.http.root-path` and expecting health endpoints to move. They live under `quarkus.http.non-application-root-path (/q)`, which is separate by design so that probes are stable across API path changes.

4.15 Interview Questions

Junior

Q1. Where does Quarkus configuration live? `src/main/resources/application.properties` by default, plus system properties, environment variables, `.env`, an external `config/application.properties`, and optionally `application.yaml`.

Q2. How do you read a property? `@ConfigProperty(name = "...", defaultValue = "...")` on a field, or — better for a group of related properties — a `@ConfigMapping` interface.

Q3. What are profiles? Named sets of overrides. `dev`, `test` and `prod` are built in; prefix a key with `%dev.` to apply it only in that profile.

Q4. How do you set a different database URL per environment? Prefix per profile (`%dev.quarkus.datasource.jdbc.url=...`) for local, and supply the production value at runtime via an environment variable or `ConfigMap`.

Q5. What is the `.env` file for? Local development values, especially secrets. It has ordinal 295 and must be gitignored.

Mid-level

Q6. Explain the config source ordinals. Sources are consulted in descending ordinal order and the first hit wins: system properties 400, environment variables 300, `.env` 295, `config/application.properties` 260, `application.yaml` 255, `application.properties` 250, `microprofile-config.properties` 100, extension defaults ~0.

Q7. How does `quarkus.datasource.jdbc.url` map to an environment variable? `QUARKUS_DATASOURCE_JDBC_URL` — uppercase, and every non-alphanumeric character becomes `_`. Quoted segments produce doubled underscores: `quarkus.log.category."com.acme".level` → `QUARKUS_LOG_CATEGORY__COM_ACME__LEVEL`.

Q8. `@ConfigProperty` vs `@ConfigMapping`? `@ConfigProperty` binds one property to one field. `@ConfigMapping` binds a whole prefix to an interface, supports nested groups, applies Bean Validation, and validates everything at startup, reporting all failures at once. Prefer `@ConfigMapping`.

Q9. What is a build-time-fixed property? One consumed during augmentation to generate code — `quarkus.datasource.db-kind`, `quarkus.http.root-path`, `quarkus.hibernate-orm.packages`. Changing it after the build has no effect because the generated artefact already encodes the decision.

Q10. How do you validate configuration? Bean Validation annotations on a `@ConfigMapping` interface (`@Min`, `@NotBlank`, `@Pattern`) with `quarkus-hibernate-validator`, plus a `StartupEvent` observer for cross-field rules that annotations cannot express.

Q11. How do you handle secrets? Never in `application.properties`. `.env` (gitignored) locally; Kubernetes Secrets via `quarkus-kubernetes-config`, or Vault via `quarkus-vault`, in deployed environments. Both surface secrets as ordinary config properties so application code is unaware.

Q12. What is a parent profile? `%staging.quarkus.config.profile.parent=prod` makes `staging` inherit every `prod` value, then override selectively — avoiding duplication of the production block across prod-like environments.

Senior

Q13. Design a configuration strategy for “build once, deploy everywhere”. Classify every property as build-time or runtime. Build-time properties are part of the image’s identity: they live in `application.properties`, in version control, with no environment-specific values — if one must differ by environment, that is a parity problem to fix rather than a configuration to manage. Runtime properties come entirely from outside the image: a `ConfigMap` mounted as `config/application.properties` (ordinal 260, real property syntax, no env-var mapping to get wrong) plus Secrets for credentials. Every mapping interface is validated at startup so a misconfigured pod fails readiness and the rollout halts automatically. Critical resolved values and their sources are logged in the first lines of the pod log, with masking, so an incident starts with evidence rather than speculation. Finally, a CI check that diffs the build-time property set between releases, because a build-time property quietly becoming environment-specific is exactly the change that breaks the model.

Q14. A property set as an environment variable in Kubernetes has no effect. Walk me through diagnosis. Three candidates, checked in order of likelihood. First, the name mapping — I would `kubectl exec` and print the environment to confirm the variable exists with the exact expected name, since quoted and hyphenated keys are easy to get wrong and fail silently. Second, whether the property is build-time fixed — the documentation’s lock icon, or a native-mode startup warning, will say so; if it is, no runtime override will ever work and the answer is a rebuild. Third, a higher-ordinal source overriding it — a system property in `JAVA_OPTS`, or a `.env` accidentally baked into the image. The decisive move is to ask the application rather than reason about it: `SmallRyeConfig.getConfigValue(key)` reports the value *and* the source name and ordinal, which resolves all three hypotheses at once. That is why I put a startup config banner in every service.

Q15. How would you implement per-tenant configuration for a multi-tenant service? Not with config properties, beyond a small fixed set of tenants. MicroProfile Config is designed for deployment configuration, not for application data — the property set is effectively static, there is no per-request scoping, and a config source doing I/O per lookup risks blocking the event loop. For genuinely dynamic per-tenant settings I would model it as data: a `TenantSettings` table or a settings service, loaded through a cached `@ApplicationScoped` registry with an explicit refresh, and resolved per request from the tenant identity. Config properties would then hold only the *bootstrap* values — where the settings store lives, cache TTL, defaults. The line I would draw is: if operations changes it, it is configuration; if a customer or an admin UI changes it, it is data. Blurring that line is how you end up restarting pods to onboard a customer.

4.16 Cheat Sheet

Ordinals (highest wins)

```

400 system properties      -Dkey=value
300 environment variables KEY_NAME=value
295 .env
260 config/application.properties  (next to the jar)
255 application.yaml
250 src/main/resources/application.properties
100 META-INF/microprofile-config.properties

```

Env var mapping

```

uppercase, non-alphanumeric -> _
orders.max-line-items        -> ORDERS_MAX_LINE_ITEMS
quarkus.log.category."com.acme".level -> QUARKUS_LOG_CATEGORY__COM_ACME__LEVEL

```

@ConfigMapping

```

@ConfigMapping(prefix = "orders")
public interface OrdersConfig {
    @WithDefault("50") @Min(1) int maxLineItems();
    @WithDefault("30s") Duration checkoutTimeout();
    Optional<String> legacyEndpoint();
    List<String> notifyEmails();
    Map<String, String> regionCodes();
    Retry retry();
    interface Retry {
        @WithDefault("3") int maxAttempts();
        @WithName("jitter-ms") @WithDefault("100") int jitterMillis();
    }
}

```

Profiles

```

%dev.key=value
%test.key=value
%prod.key=value
%dev,test.key=value
%staging.quarkus.config.profile.parent=prod

```

```
-Dquarkus.profile=staging    QUARKUS_PROFILE=staging
```

Expressions

```

a=${b}           # reference
a=${B:default}   # default
a=${X:${Y:fallback}} # nested
a=\${literal}    # escape

```

Programmatic

```

@Inject Config config;
config.getOptionalValue("k", String.class);
ConfigValue v = config.unwrap(SmallRyeConfig.class).getConfigValue("k");
v.getValue(); v.getConfigSourceName(); v.getConfigSourceOrdinal();

```

Tests

```

@QuarkusTest @TestProfile(MyProfile.class)
class T { }

public static class MyProfile implements QuarkusTestProfile {
    public Map<String,String> getConfigOverrides() { return Map.of("k","v"); }
    public String getConfigProfile() { return "custom"; }
}

```

4.17 Summary

SmallRye Config is a small model applied consistently: ordered sources, first hit wins, then conversion, then expression expansion. Most day-to-day competence comes from knowing the ordinals and the environment-variable mapping cold.

The engineering content of this chapter is the **build-time versus runtime** distinction. Build-time properties are consumed during augmentation and become part of the image's identity; runtime properties are the deployment's configuration. Confusing the two produces a deployment that is silently wrong — an environment variable set correctly, with no effect and no error. Design your configuration by classifying every key, keep environment-specific values out of the image entirely, validate at startup so a bad config fails readiness rather than serving traffic, and log the resolved values with their sources so incidents begin with evidence.

`@ConfigMapping` is the right default because it turns configuration errors into startup failures, and in Kubernetes a startup failure is a halted rollout — the cheapest possible place for a configuration bug to surface.

Chapter 5 moves to persistence: **Hibernate ORM** in Quarkus, including build-time bytecode enhancement, the datasource and pooling model, the reactive variant, and the query patterns that determine whether your service is fast or spends its life in N+1 queries.

4.18 References

- Configuration reference — <https://quarkus.io/guides/config-reference>
- Mapping configuration to objects — <https://quarkus.io/guides/config-mappings>
- SmallRye Config — <https://smallrye.io/smallrye-config/>
- MicroProfile Config specification — <https://github.com/eclipse/microprofile-config>
- Kubernetes ConfigMaps and Secrets — <https://quarkus.io/guides/kubernetes-config>
- Vault — <https://quarkus.io/guides/vault>
- YAML configuration — <https://quarkus.io/guides/config-yaml>

Hibernate ORM

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Hibernate ORM 6.6 · Jakarta Persistence 3.1 · Agroal · PostgreSQL 16

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what Quarkus does to Hibernate at **build time**, and why that makes startup fast and native compilation possible.
2. Configure a datasource and reason about **connection pool sizing** as a system property rather than a magic number.
3. Model entities correctly with Java 21, including the traps around records, `equals/hashCode` and identifier generation.
4. Choose fetch types and cascade rules deliberately, and explain the consequences of each.
5. Diagnose and eliminate **N+1 queries** — the defining ORM performance failure — with join fetch, entity graphs and batch sizing.
6. Explain the persistence context lifecycle: managed, detached, removed, and where `LazyInitializationException` comes from.
7. Use projections and DTO queries to avoid loading entities you do not need.
8. Configure first-level, second-level and query caches, and know when the second-level cache is a mistake.
9. Manage schema evolution with Flyway, and explain why `hbm2ddl` is not schema management.
10. Decide between blocking Hibernate ORM and Hibernate Reactive.
11. Answer 20+ interview questions on JPA and Hibernate in a Quarkus context.

5.1 What Quarkus Changes About Hibernate

Classic Hibernate bootstrap is expensive: scan for `@Entity` classes, parse annotations reflectively, build the metamodel, resolve the dialect by querying the database, compile named queries, and instantiate proxies via runtime bytecode generation. On a medium application that is several seconds and tens of megabytes of retained metadata.

Quarkus moves nearly all of it to build time.

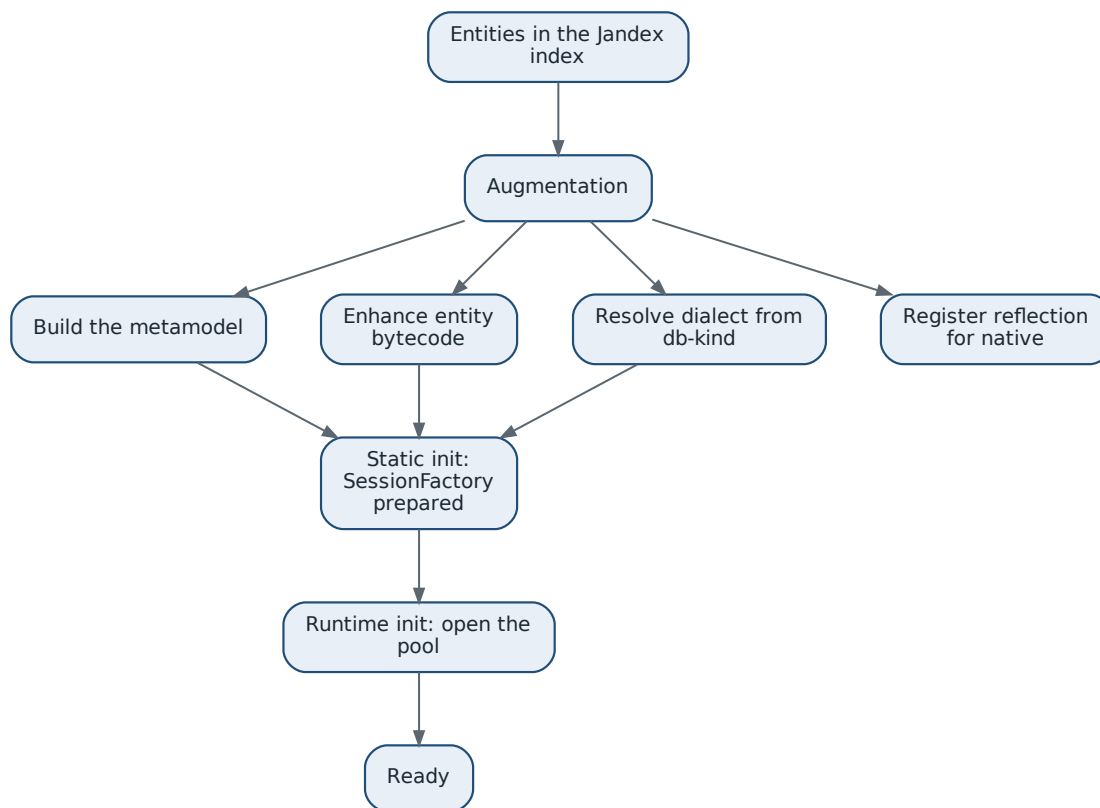


Figure 5.1 — *Hibernate bootstrap in Quarkus. Only the pool opens at runtime.*

Concretely:

- **Entity discovery uses the Jandex index.** No classpath scanning. (And the same caveat as Chapter 3: entities in an unindexed JAR are invisible.)
- **Entities are bytecode-enhanced at build time.** Hibernate normally does this at runtime with a bytecode library. Quarkus does it during `mvn package`, which is why lazy loading works in native mode without extra configuration.
- **The dialect is resolved from `quarkus.datasource.db-kind`,** not by connecting to the database. This is why `db-kind` is a build-time-fixed property (see §4.7) and why startup does not require the database to be reachable.
- **The metamodel is built during static init,** so in native mode it is serialised into the image heap and already exists before `main()` runs.
- **Reflection registrations are generated automatically** for entities, embeddables and enums.

The practical result: a Quarkus + Hibernate application starts in under a second on the JVM and in tens of milliseconds natively, and the entire ORM layer works in native mode with no configuration from you.

5.2 Datasource and Connection Pooling

5.2.1 Configuration

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-orm</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-jdbc-postgresql</artifactId>
</dependency>
```

```
# --- Datasource -----
quarkus.datasource.db-kind=postgresql          # BUILD TIME
quarkus.datasource.username=${DB_USER}
quarkus.datasource.password=${DB_PASSWORD}
quarkus.datasource.jdbc.url=jdbc:postgresql://db:5432/orders

# --- Pool (Agroal) -----
quarkus.datasource.jdbc.min-size=5
quarkus.datasource.jdbc.max-size=20
quarkus.datasource.jdbc.initial-size=5
quarkus.datasource.jdbc.acquisition-timeout=5S
quarkus.datasource.jdbc.background-validation-interval=2M
quarkus.datasource.jdbc.idle-removal-interval=5M
quarkus.datasource.jdbc.max-lifetime=30M
quarkus.datasource.jdbc.leak-detection-interval=10M

# --- Hibernate -----
quarkus.hibernate-orm.database.generation=none # prod: never let Hibernate touch DDL
quarkus.hibernate-orm.log.sql=false
quarkus.hibernate-orm.log.format-sql=true
quarkus.hibernate-orm.jdbc.timezone=UTC
quarkus.hibernate-orm.query.in-clause-parameter-padding=true

%dev.quarkus.hibernate-orm.database.generation=drop-and-create
%dev.quarkus.hibernate-orm.log.sql=true
%dev.quarkus.hibernate-orm.log.bind-parameters=true
```

Quarkus uses **Agroal** rather than HikariCP. It is designed for Quarkus’s transaction integration and works in native mode; the tuning knobs map closely to Hikari’s if you are migrating.

5.2.2 Sizing the pool

The most common mistake is treating `max-size` as “bigger is better”. It is not: the pool is a *queue in front of the database*, and the database has a finite capacity for concurrent work.

A defensible starting point:

```
max-size ≈ number of concurrent requests that actually need the database
         ≈ cores available to PostgreSQL × 2 (a common rule of thumb)
```


For a service with 20 connections against a database with 8 cores, running 10 replicas, you have opened **200 connections** to a database that can usefully execute perhaps 16 queries at once. PostgreSQL's default `max_connections` is 100; you have already exceeded it, and each connection costs memory on the server.

Things to reason about explicitly:

- **Total connections = max-size × replica count.** Autoscaling multiplies your database load in a way that is easy to miss.
- **A larger pool does not add throughput past the database's capacity.** It converts a fast failure (acquisition timeout) into slow queries for everyone.
- **acquisition-timeout should be short.** Five seconds of waiting for a connection is five seconds of a held worker thread. Fail fast, shed load, and let the caller retry.
- **Use a connection proxy (PgBouncer) when replica count is high.** That is the correct fix for connection explosion, not a bigger pool.
- **Virtual threads do not change this.** They make it cheap to have 10 000 concurrent requests; the pool is still 20. Virtual threads move the queue, they do not remove it.

5.2.3 Multiple datasources

```
# Default datasource
quarkus.datasource.db-kind=postgresql
quarkus.datasource.jdbc.url=jdbc:postgresql://primary:5432/orders

# A named datasource for reporting, pointed at a read replica
quarkus.datasource.reporting.db-kind=postgresql
quarkus.datasource.reporting.jdbc.url=jdbc:postgresql://replica:5432/orders
quarkus.datasource.reporting.jdbc.max-size=5

quarkus.hibernate-orm.reporting.datasource=reporting
quarkus.hibernate-orm.reporting.packages=com.acme.orders.reporting
```

```
@Inject
@PersistenceUnit("reporting")
EntityManager reportingEm;

@Inject
@DataSource("reporting")
AgroalDataSource reportingDs;
```

Routing reads to a replica is a legitimate and common pattern. Be aware of replication lag: a read-after-write against the replica may not see the write. Route reads that must be consistent to the primary.

5.3 Entities

5.3.1 A worked entity

```
package com.acme.orders.domain;

import jakarta.persistence.*;
```

```

import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.*;

@Entity
@Table(name = "orders",
    indexes = {
        @Index(name = "idx_orders_customer", columnList = "customer_id"),
        @Index(name = "idx_orders_status_created", columnList = "status,created_at")
    })
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "order_seq")
    @SequenceGenerator(name = "order_seq", sequenceName = "order_id_seq",
        allocationSize = 50)
    private Long id;

    @Column(name = "customer_id", nullable = false)
    private Long customerId;

    @NotBlank @Email
    @Column(name = "customer_email", nullable = false, length = 320)
    private String customerEmail;

    @Enumerated(EnumType.STRING) // never ORDINAL - see below
    @Column(nullable = false, length = 32)
    private OrderStatus status = OrderStatus.PENDING;

    @Column(nullable = false, precision = 19, scale = 2)
    private BigDecimal total = BigDecimal.ZERO;

    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @Column(name = "updated_at")
    private Instant updatedAt;

    @Version // optimistic locking
    private int version;

    @OneToMany(mappedBy = "order",
        cascade = CascadeType.ALL,
        orphanRemoval = true,
        fetch = FetchType.LAZY)
    private List<OrderLine> lines = new ArrayList<>();

    @ManyToOne(fetch = FetchType.LAZY) // ALWAYS lazy on @ManyToOne
    @JoinColumn(name = "customer_id", insertable = false, updatable = false)
    private Customer customer;

    @PrePersist

```

```

    void onCreate() {
        createdAt = Instant.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    void onUpdate() {
        updatedAt = Instant.now();
    }

    // --- Bidirectional helpers: keep both sides in sync -----
    public void addLine(OrderLine line) {
        lines.add(line);
        line.setOrder(this);
        recalculateTotal();
    }

    public void removeLine(OrderLine line) {
        lines.remove(line);
        line.setOrder(null);
        recalculateTotal();
    }

    private void recalculateTotal() {
        total = lines.stream()
            .map(OrderLine::lineTotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    // --- Identity -----
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        // Note: getClass() would break against a Hibernate proxy subclass
        if (!(o instanceof Order other)) return false;
        return id != null && id.equals(other.id);
    }

    @Override
    public int hashCode() {
        return getClass().hashCode(); // constant: stable across the id transition
    }

    // getters/setters omitted
}

```

5.3.2 The rules that matter

@Enumerated(EnumType.STRING), always. The default is **ORDINAL**, which stores the enum's *position*. Insert a new constant in the middle of the enum and every existing row silently changes meaning. This is a data-corruption bug with no error message. There is no situation where the four bytes saved by **ORDINAL** justify it.

fetch = FetchType.LAZY on every **@ManyToOne** and **@OneToOne**. JPA's default for these is **EAGER**, which is almost always wrong: every load of an **Order** drags in the **Customer**, and if **Customer** has an eager association it cascades further. **@OneToMany** and **@ManyToMany** default to **LAZY**, which is correct.

equals/hashCode on entities is genuinely subtle. The problem: an entity's identity changes when it is persisted (**id** goes from **null** to a value), but it may already be in a **HashSet**. The pattern above — **hashCode()** returning a class-based constant, **equals** requiring a non-null id — is the standard safe compromise. Use **instanceof** rather than **getClass()** so it works against Hibernate proxies. A frequently better answer is to give the entity a business key (a UUID assigned in the constructor) and use that.

allocationSize on sequences matters. The default of 50 means Hibernate reserves 50 ids per database round trip. With **allocationSize = 1** you pay a round trip per insert. Just be aware that gaps in the id sequence are normal and expected.

Records cannot be entities. They are final and immutable, and JPA requires a no-arg constructor and mutable state for the proxy and dirty-checking machinery. Records are excellent as **DTOs** and **projections** (§5.6.3) — use them there, not as **@Entity**.

5.3.3 Embeddables and value objects

```
@Embeddable
public record Money(
    @Column(name = "amount", precision = 19, scale = 2) BigDecimal amount,
    @Column(name = "currency", length = 3) String currency) {

    public Money {
        if (amount == null || currency == null) {
            throw new IllegalArgumentException("Money requires amount and currency");
        }
    }
}

@Entity
public class Invoice {
    @Id @GeneratedValue Long id;

    @Embedded
    private Money total;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "amount", column = @Column(name = "tax_amount")),
        @AttributeOverride(name = "currency", column = @Column(name = "tax_currency"))
    })
    private Money tax;
}
```

Hibernate 6 supports **record embeddables**, which is a genuinely nice modern touch: immutable value objects with validation in the compact constructor, mapped to columns on the owning table.

5.4 Relationships

5.4.1 One-to-many

```

@Entity
@Table(name = "order_lines")
public class OrderLine {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;           // the OWNING side

    @Column(nullable = false, length = 64)
    private String sku;

    @Column(nullable = false)
    private int quantity;

    @Column(name = "unit_price", nullable = false, precision = 19, scale = 2)
    private BigDecimal unitPrice;

    public BigDecimal lineTotal() {
        return unitPrice.multiply(BigDecimal.valueOf(quantity));
    }
}

```

Owning side vs inverse side. The side *without* `mappedBy` owns the relationship and controls the foreign key. Changes to the inverse side alone are not persisted:

```

// Does NOTHING to the database - lines is the inverse side
order.getLines().add(line);

// Correct - set the owning side
line.setOrder(order);
order.getLines().add(line);    // keep the in-memory graph consistent too

// Better - a helper that does both
order.addLine(line);

```

5.4.2 Many-to-many

```

@Entity
public class Product {
    @Id @GeneratedValue Long id;

    @ManyToMany
    @JoinTable(name = "product_tags",
        joinColumns = @JoinColumn(name = "product_id"),
        inverseJoinColumns = @JoinColumn(name = "tag_id"))

```

```
private Set<Tag> tags = new HashSet<>();    // Set, not List
}
```

Use `Set`, not `List`, for `@ManyToMany`. With a `List`, Hibernate deletes and re-inserts every join row on any change, because a `List` has ordering semantics it must preserve.

Prefer an explicit join entity to `@ManyToMany`. The moment the relationship needs an attribute — when the tag was added, who added it, a sort order — you must refactor. Modelling it as `ProductTag` with two `@ManyToOne`s from the start costs almost nothing and avoids a migration later. In practice, most `@ManyToMany` relationships acquire an attribute eventually.

5.4.3 Cascade

Cascade	Effect
<code>PERSIST</code>	Saving the parent saves new children
<code>MERGE</code>	Merging the parent merges children
<code>REMOVE</code>	Deleting the parent deletes children
<code>REFRESH</code>	Refreshing the parent refreshes children
<code>DETACH</code>	Detaching the parent detaches children
<code>ALL</code>	All of the above

`orphanRemoval = true` is different from `CascadeType.REMOVE`: it deletes a child that is *removed from the collection*, even if the parent survives.

```
// orphanRemoval = true
order.getLines().remove(line);    // -> DELETE FROM order_lines WHERE id = ?

// CascadeType.REMOVE only
em.remove(order);                 // -> deletes the order AND its lines
```

Never put `CascadeType.REMOVE` on a `@ManyToOne`. `line.setOrder(...)` with cascade `REMOVE` means deleting a line deletes the order — and with `orphanRemoval` on the other side, potentially every other line too. Cascades flow from aggregate root downward, never upward.

5.5 The Persistence Context

5.5.1 Entity states

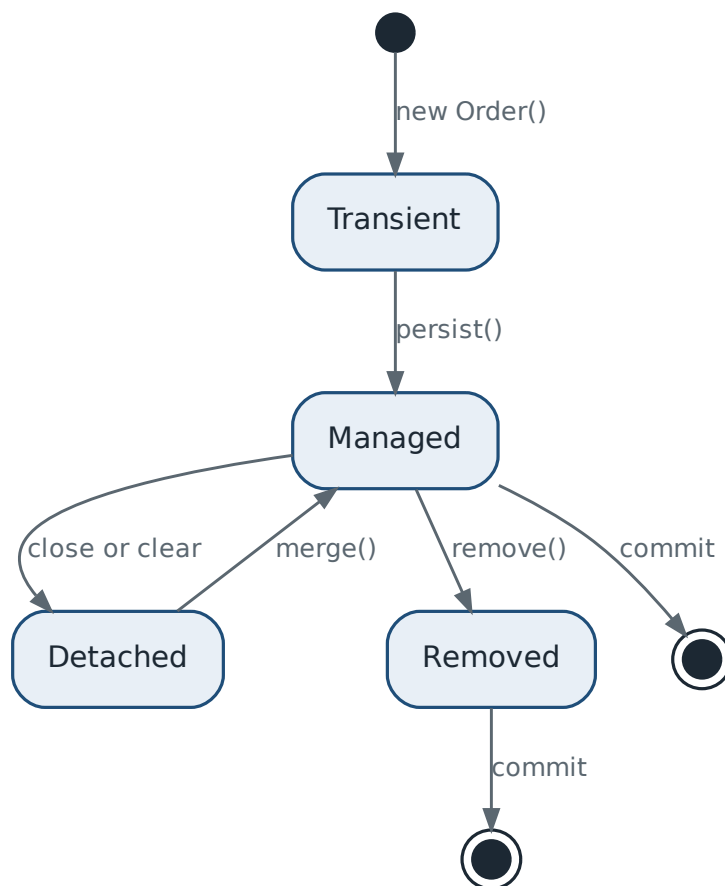


Figure 5.2 — JPA entity states.

- **Transient** — a plain Java object; Hibernate knows nothing about it.
- **Managed** — attached to a persistence context; changes are tracked and flushed automatically.
- **Detached** — was managed, but the context closed. Changes are *not* tracked; lazy associations throw.
- **Removed** — scheduled for deletion at flush.

5.5.2 Dirty checking

```

@Transactional
public void markPaid(long orderId) {
    Order order = em.find(Order.class, orderId);    // now MANAGED
    order.setStatus(OrderStatus.PAID);              // no em.save() needed
} // on commit: Hibernate compares to the loaded snapshot and issues UPDATE
  
```

There is no `save()` in JPA. A managed entity's changes are detected automatically at flush. This surprises people coming from other data-access styles, and it is also why *accidental* mutation of a managed entity persists — a real source of bugs.

5.5.3 LazyInitializationException

The single most-encountered Hibernate exception:

```
// BROKEN
public OrderView findById(long id) {
    Order order = em.find(Order.class, id);
    return OrderView.of(order);          // outside the transaction
}
// -> LazyInitializationException: could not initialize proxy - no Session
```

The transaction ended, the persistence context closed, and `order.getLines()` is an uninitialised proxy.

Four fixes, in order of preference:

1. Fetch what you need, in the query.

```
@Transactional
public OrderView findById(long id) {
    Order order = em.createQuery("""
        SELECT o FROM Order o
        LEFT JOIN FETCH o.lines
        WHERE o.id = :id
        """, Order.class)
        .setParameter("id", id)
        .getSingleResult();
    return OrderView.of(order);
}
```

2. Project directly into a DTO — best of all, because you never load the entity:

```
public OrderSummary summary(long id) {
    return em.createQuery("""
        SELECT new com.acme.orders.api.OrderSummary(o.id, o.status, o.total)
        FROM Order o WHERE o.id = :id
        """, OrderSummary.class)
        .setParameter("id", id)
        .getSingleResult();
}
```

3. Use an entity graph (§5.6.2).

4. Extend the transaction to cover the mapping. Works, but keeps a database connection held while you do CPU work — acceptable for small mappings, wasteful at scale.

Do not “fix” it with `hibernate.enable_lazy_load_no_trans=true`. It silently opens a new session per lazy access, turning one exception into an unbounded number of hidden queries. It converts a loud bug into a quiet performance disaster.

5.6 Querying

5.6.1 JPQL

```
@ApplicationScoped
public class OrderQueries {

    @Inject EntityManager em;

    public List<Order> byStatus(OrderStatus status, int page, int size) {
        return em.createQuery("""
            SELECT o FROM Order o
            WHERE o.status = :status
            ORDER BY o.createdAt DESC
            """, Order.class)
            .setParameter("status", status)
            .setFirstResult(page * size)
            .setMaxResults(size)
            .getResultList();
    }

    public Optional<Order> withLines(long id) {
        return em.createQuery("""
            SELECT o FROM Order o
            LEFT JOIN FETCH o.lines l
            WHERE o.id = :id
            """, Order.class)
            .setParameter("id", id)
            .getResultStream()
            .findFirst();
    }

    public BigDecimal revenueSince(Instant since) {
        return em.createQuery("""
            SELECT COALESCE(SUM(o.total), 0)
            FROM Order o
            WHERE o.status = :paid AND o.createdAt >= :since
            """, BigDecimal.class)
            .setParameter("paid", OrderStatus.PAID)
            .setParameter("since", since)
            .getSingleResult();
    }
}
```

Java 21 text blocks make multi-line JPQL readable, which materially reduces the rate of subtle query bugs.

5.6.2 Entity graphs

Entity graphs declare what to fetch without embedding it in the query string — useful when the same query serves several use cases.

```
@Entity
@NamedEntityGraph(
    name = "Order.withLinesAndCustomer",
    attributeNodes = {
        @NamedAttributeNode("lines"),
        @NamedAttributeNode("customer")
    })
public class Order { ... }
```

```
EntityGraph<?> graph = em.getEntityGraph("Order.withLinesAndCustomer");

Order order = em.find(Order.class, id,
    Map.of("jakarta.persistence.fetchgraph", graph));

// or on a query
List<Order> orders = em.createQuery("SELECT o FROM Order o WHERE o.status = :s", Order.class)
    .setParameter("s", OrderStatus.PENDING)
    .setHint("jakarta.persistence.fetchgraph", graph)
    .getResultList();
```

fetchgraph — fetch exactly these, everything else lazy. **loadgraph** — fetch these *in addition* to the mapping defaults.

5.6.3 DTO projections

The highest-leverage query technique in the chapter. If you only need three columns, do not load the entity.

```
public record OrderSummary(Long id, String status, BigDecimal total, Instant createdAt) { }

public List<OrderSummary> summaries(OrderStatus status) {
    return em.createQuery("""
        SELECT new com.acme.orders.api.OrderSummary(o.id, o.status, o.total, o.createdAt)
        FROM Order o
        WHERE o.status = :status
        ORDER BY o.createdAt DESC
        """, OrderSummary.class)
        .setParameter("status", status)
        .getResultList();
}
```

What you gain: fewer columns transferred, no persistence-context bookkeeping, no dirty-check snapshots, no risk of lazy-loading later, and an immutable result. For read-heavy endpoints this is often a 3–10× improvement over loading entities and mapping them.

Hibernate 6 also supports **tuple** and **interface projections**, and **@IdClass**-free record constructors, but the **SELECT new** form above is portable and explicit.

5.6.4 Criteria API

For queries assembled dynamically:

```
public List<Order> search(OrderSearchCriteria criteria) {
    CriteriaBuilder cb = em.getCriteriaBuilder();
    CriteriaQuery<Order> query = cb.createQuery(Order.class);
    Root<Order> order = query.from(Order.class);

    List<Predicate> predicates = new ArrayList<>();
    if (criteria.status() != null) {
        predicates.add(cb.equal(order.get("status"), criteria.status()));
    }
    if (criteria.minTotal() != null) {
        predicates.add(cb.greaterThanOrEqualTo(order.get("total"), criteria.minTotal()));
    }
    if (criteria.emailContains() != null) {
        predicates.add(cb.like(cb.lower(order.get("customerEmail")),
                                "%" + criteria.emailContains().toLowerCase() + "%"));
    }

    query.where(predicates.toArray(Predicate[]::new))
        .orderBy(cb.desc(order.get("createdAt")));

    return em.createQuery(query)
        .setMaxResults(criteria.limit())
        .getResultList();
}
```

Criteria is verbose. Use it where the query genuinely varies; use JPQL text blocks everywhere else. String concatenation of JPQL to build dynamic queries is how SQL injection and unparseable code both arrive.

5.6.5 Native SQL

```
public List<TopCustomer> topCustomers(int limit) {
    return em.createNativeQuery("""
        SELECT c.id, c.email, SUM(o.total) AS lifetime_value
        FROM customers c
        JOIN orders o ON o.customer_id = c.id
        WHERE o.status = 'PAID'
        GROUP BY c.id, c.email
        ORDER BY lifetime_value DESC
        LIMIT :limit
        """, Tuple.class)
        .setParameter("limit", limit)
        .getResultStream()
        .map(t -> new TopCustomer(
            t.get("id", Long.class),
            t.get("email", String.class),
            t.get("lifetime_value", BigDecimal.class)))
        .toList();
}
```

Native SQL is legitimate and often correct — window functions, CTEs, database-specific features, and heavy analytical queries have no good JPQL expression. The cost is portability, which for most services is a theoretical concern.

Always use named parameters. Never build SQL by string concatenation with user input.

5.7 The N+1 Problem

The defining ORM performance failure. Worth understanding precisely, because it is asked in almost every interview that touches persistence.

5.7.1 What it is

```
List<Order> orders = em.createQuery("SELECT o FROM Order o", Order.class)
    .getResultList();           // 1 query

for (Order order : orders) {
    order.getLines().size();     // 1 query PER ORDER
}
// 100 orders -> 101 queries
```

Each query may take 1 ms, so in development with 5 orders it is invisible. In production with 500 orders it is 500 ms of pure round-trip latency, and it scales linearly with data volume — the classic “it got slow as we grew” performance profile.

5.7.2 Fixes

Join fetch — for a single collection:

```
em.createQuery(
    "SELECT DISTINCT o FROM Order o\n" +
    "LEFT JOIN FETCH o.lines\n" +
    "", Order.class).getResultList();           // 1 query
```

`DISTINCT` is needed because joining a collection multiplies parent rows. In Hibernate 6 the deduplication happens in memory automatically for entity queries, but keeping `DISTINCT` is harmless and portable.

Batch fetching — for multiple collections, or when join fetch would be a cartesian product:

```
@Entity
@BatchSize(size = 50)           // on the entity
public class Order { ... }

@OneToMany(mappedBy = "order")
@BatchSize(size = 50)           // or on the collection
private List<OrderLine> lines;
```

```
quarkus.hibernate-orm.jdbc.statement-batch-size=50
quarkus.hibernate-orm.query.default-null-ordering=none
```

With batching, 500 orders become 1 + 10 queries instead of 1 + 500 — Hibernate loads the collections in batches with `WHERE order_id IN (?, ?, ... )`.

Entity graph — as in §5.6.2.

Projection — sidestep the problem entirely by not loading entities.

5.7.3 You cannot join-fetch two collections

```
// Cartesian product: 10 lines x 5 payments = 50 rows per order
SELECT o FROM Order o
LEFT JOIN FETCH o.lines
LEFT JOIN FETCH o.payments      // MultipleBagFetchException, or a data explosion
```

Hibernate throws `MultipleBagFetchException` when both are `Lists`. Options: change one to a `Set`, fetch one collection and batch the other, or issue two queries against the same persistence context (the second one populates the already-managed entities).

5.7.4 Detecting it

```
%dev.quarkus.hibernate-orm.log.sql=true
%dev.quarkus.hibernate-orm.log.format-sql=true
%dev.quarkus.hibernate-orm.log.bind-parameters=true
```

```
# Fail loudly on queries that exceed a threshold
quarkus.hibernate-orm.log.jdbc-warnings=true
```

Better: assert on query counts in tests, so a regression fails CI rather than a dashboard.

```
@QuarkusTest
class OrderQueryCountTest {

    @Inject EntityManager em;

    @Test
    @Transactional
    void listingOrdersIssuesOneQuery() {
        Statistics stats = em.getEntityManagerFactory()
            .unwrap(SessionFactory.class).getStatistics();

        stats.clear();

        orderQueries.summaries(OrderStatus.PAID);

        assertThat(stats.getPrepareStatementCount()).isEqualTo(1);
    }
}
```

```
quarkus.hibernate-orm.statistics=true      # enable in tests; costs a little in prod
```

This test is worth more than any amount of code review. N+1 regressions are introduced by innocuous-looking changes — adding a field to a DTO mapper, for instance — and a query-count assertion catches them the moment they appear.

5.8 Transactions and Locking

Transactions get a full chapter (Chapter 7); here are the persistence-specific parts.

```
@ApplicationScoped
public class OrderService {

    @Inject EntityManager em;

    @Transactional
    public Order create(CreateOrderRequest request) {
        Order order = new Order();
        order.setCustomerEmail(request.customerEmail());
        request.items().forEach(i -> order.addLine(toLine(i)));
        em.persist(order);
        return order;
    }

    @Transactional(Transaction.TxType.REQUIRES_NEW)
    public void auditIndependently(String message) {
        em.persist(new AuditEntry(message));    // commits even if the caller rolls back
    }

    @Transactional
    public void markPaidWithLock(long id) {
        // Pessimistic: SELECT ... FOR UPDATE
        Order order = em.find(Order.class, id, LockModeType.PESSIMISTIC_WRITE);
        order.setStatus(OrderStatus.PAID);
    }
}
```

5.8.1 Optimistic locking

```
@Entity
public class Order {
    @Version
    private int version;    // or @Version private Instant lastModified;
}
```

On update, Hibernate issues `UPDATE orders SET ..., version = 2 WHERE id = ? AND version = 1`. If another transaction already incremented it, zero rows match and Hibernate throws `OptimisticLockException`.

```
@ServerExceptionHandler
public RestResponse<ProblemDetail> conflict(OptimisticLockException e) {
    return RestResponse.status(Response.Status.CONFLICT,
        ProblemDetail.of("concurrent-modification",
            "The order was modified by another request", 409,
            "Reload and retry"));
}
```

Optimistic locking should be your default. It costs one integer column, adds no contention, and turns a silent lost update into an explicit 409. Pessimistic locking holds database locks for the duration of the transaction and is appropriate only where contention is genuinely high and retries are expensive — inventory decrements, for instance.

5.9 Caching

5.9.1 The three levels

Cache	Scope	Enabled by default
First-level (persistence context)	One transaction	Yes, always
Second-level	The application (or cluster)	No
Query cache	The application	No

The **first-level cache** is the persistence context itself. Two `em.find(Order.class, 1L)` calls in the same transaction issue one query. This also means you cannot “refresh” an entity by finding it again — use `em.refresh(entity)`.

5.9.2 Second-level cache

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-orm</artifactId>
</dependency>
<!-- Caching is provided by the built-in Caffeine-backed cache -->
```

```
@Entity
@Cacheable // opt in per entity
public class Product { ... }

@Entity
public class Order {
    @OneToMany(mappedBy = "order")
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private List<OrderLine> lines;
}
```

```
quarkus.hibernate-orm.second-level-caching-enabled=true
quarkus.hibernate-orm.cache."com.acme.orders.domain.Product".expiration.max-idle=1H
quarkus.hibernate-orm.cache."com.acme.orders.domain.Product".memory.object-count=10000
```

When the second-level cache is right: reference data that is read constantly and written rarely — product catalogues, country codes, tax rates, feature configuration.

When it is a mistake:

- **Any entity written frequently.** Cache invalidation costs more than the query saved.

- **Multi-replica deployments without a distributed cache.** Each pod has its own cache, so a write on pod A leaves pods B–J serving stale data with no error. This is the failure mode that turns into a mystifying support ticket.
- **When the real problem is N+1.** Caching hides the symptom and leaves a design that degrades as data grows.

Interview framing. “We added a second-level cache” is a weaker answer than “we found the query was N+1 and fixed the fetch strategy”. Caching is a legitimate tool, but reaching for it before understanding the query pattern signals that you are treating symptoms. Be ready to say when you would *not* use it.

5.10 Schema Management

5.10.1 hbm2ddl is not schema management

```
quarkus.hibernate-orm.database.generation=none           # PRODUCTION
%dev.quarkus.hibernate-orm.database.generation=drop-and-create
%test.quarkus.hibernate-orm.database.generation=drop-and-create
```

Values: `none`, `create`, `drop-and-create`, `drop`, `update`, `validate`.

Never use `update` in production. It makes additive changes only, never drops or alters, produces no migration history, cannot be reviewed before it runs, and behaves differently across Hibernate versions. It will happily leave you with a column your code no longer uses and no record of why.

`validate` is useful in production as a startup assertion that the schema matches the entities:

```
%prod.quarkus.hibernate-orm.database.generation=validate
```

5.10.2 Flyway

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-flyway</artifactId>
</dependency>
```

```
quarkus.flyway.migrate-at-start=true
quarkus.flyway.baseline-on-migrate=true
quarkus.flyway.baseline-version=1.0.0
quarkus.flyway.locations=db/migration
quarkus.flyway.validate-on-migrate=true
%dev.quarkus.flyway.clean-at-start=true
```

```
-- src/main/resources/db/migration/V1.0.0__create_orders.sql
CREATE TABLE orders (
  id          BIGINT PRIMARY KEY,
  customer_id BIGINT   NOT NULL,
  customer_email VARCHAR(320) NOT NULL,
  status      VARCHAR(32) NOT NULL,
  total       NUMERIC(19,2) NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL,
```



```

        updated_at    TIMESTAMPTZ,
        version       INTEGER    NOT NULL DEFAULT 0
    );

    CREATE SEQUENCE order_id_seq INCREMENT BY 50;

    CREATE INDEX idx_orders_customer      ON orders (customer_id);
    CREATE INDEX idx_orders_status_created ON orders (status, created_at DESC);

```

```

-- V1.1.0__add_order_lines.sql
CREATE TABLE order_lines (
    id          BIGSERIAL PRIMARY KEY,
    order_id    BIGINT      NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    sku         VARCHAR(64) NOT NULL,
    quantity    INTEGER     NOT NULL CHECK (quantity > 0),
    unit_price  NUMERIC(19,2) NOT NULL
);

CREATE INDEX idx_order_lines_order ON order_lines (order_id);

```

5.10.3 migrate-at-start in a multi-replica deployment

With ten replicas starting simultaneously, ten Flyway instances race. Flyway takes a database lock, so it is *safe* — but nine pods wait, and a long migration can exceed the readiness probe deadline and cause a rollback loop.

For anything non-trivial, run migrations as a **Kubernetes Job** or an **init container** before the rollout, and set `migrate-at-start=false` in production. Chapter 20 covers the manifest.

```

%prod.quarkus.flyway.migrate-at-start=false
%prod.quarkus.hibernate-orm.database.generation=validate

```

Also: **make migrations backward-compatible** during a rolling deployment. For a window, old and new pods run simultaneously against the same schema. Dropping a column in the same release that stops using it will break the old pods still serving traffic. The standard pattern is `expand-migrate-contract` across two releases.

5.11 Hibernate Reactive

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-reactive-panache</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-reactive-pg-client</artifactId>
</dependency>

```

```

quarkus.datasource.db-kind=postgresql
quarkus.datasource.reactive.url=postgresql://db:5432/orders
quarkus.datasource.reactive.max-size=20

```

```

@ApplicationScoped
public class ReactiveOrderService {

    @Inject Mutiny.SessionFactory sessionFactory;

    public Uni<Order> findById(long id) {
        return sessionFactory.withSession(session ->
            session.find(Order.class, id));
    }

    @WithTransaction
    public Uni<Order> create(CreateOrderRequest request) {
        Order order = new Order();
        order.setCustomerEmail(request.customerEmail());
        return order.persistAndFlush();
    }
}

```

Choosing between them:

	Hibernate ORM (blocking)	Hibernate Reactive
Driver	JDBC	Vert.x reactive client
Thread	Worker or virtual thread	Event loop
API	Synchronous	Uni / Multi
Maturity	Very high	Good, smaller surface
Lazy loading	Works	Requires explicit fetch
Second-level cache	Yes	Limited
Learning curve	Familiar	Steep

My recommendation, and a defensible interview answer: on Java 21, use **blocking Hibernate ORM with `@RunOnVirtualThread`** as the default. You get the concurrency benefit without the reactive rewrite, keep lazy loading, keep the mature tooling, and keep code that a new team member can read. Choose Hibernate Reactive when you are already fully reactive end to end, or when you have measured that the worker-thread model is your bottleneck — which, given that the connection pool is usually the real constraint, is rarer than people expect.

5.12 Dev Services and Testing

5.12.1 Dev Services

Delete the URL and Quarkus starts a real PostgreSQL container for dev and test:

```

# quarkus.datasource.jdbc.url=... <- omit in dev/test
quarkus.datasource.devservices.image-name=postgres:16-alpine
quarkus.datasource.devservices.reuse=true
quarkus.datasource.devservices.port=5433

```

Testing against real PostgreSQL rather than H2 eliminates a whole class of bug: dialect differences, **NUMERIC** precision, **TIMESTAMPZ** semantics, case sensitivity, and any use of a PostgreSQL-specific feature.

5.12.2 Test structure

```
@QuarkusTest
@TestTransaction // each test runs in a rolled-back transaction
class OrderRepositoryTest {

    @Inject EntityManager em;
    @Inject OrderQueries queries;

    @Test
    void findsByStatus() {
        em.persist(anOrder(OrderStatus.PAID));
        em.persist(anOrder(OrderStatus.PENDING));
        em.flush();

        assertThat(queries.byStatus(OrderStatus.PAID, 0, 10)).hasSize(1);
    }
    // rolled back automatically - no cleanup needed
}
```

`@TestTransaction` is the cleanest isolation mechanism: each test method runs in a transaction that is rolled back afterwards, so tests never see each other's data and there is no teardown code.

```
// Seed data for dev mode only
// src/main/resources/import.sql
```

```
INSERT INTO orders (id, customer_id, customer_email, status, total, created_at, version)
VALUES (1, 1, 'demo@acme.com', 'PENDING', 99.00, now(), 0);
```

5.13 Performance Checklist

1. **Project into DTOs** for read paths. Do not load entities you will not mutate.
2. **Lazy on every @ManyToOne / @OneToOne**. The JPA default is wrong for microservices.
3. **Fix N+1 with join fetch, entity graphs or @BatchSize** — and assert query counts in tests.
4. **@Enumerated(EnumType.STRING)** always.
5. **Index every foreign key and every column you filter or sort by**. Hibernate will not do this for you.
6. **Size the pool against the database's capacity**, not against your request concurrency.
7. **Batch writes**: `quarkus.hibernate-orm.jdbc.statement-batch-size=50` plus ordered inserts/updates.
8. **Paginate everything**. An unbounded `findAll` is a production incident waiting for enough rows.
9. **setReadOnly(true)** (or a read-only transaction) for query-only paths — Hibernate skips dirty-check snapshots.

10. Stream large result sets with `getResultStream()` and a scroll, not `getResultList()`.
11. Enable **statistics** in load tests and look at query counts, not just latency.
12. Use **@Version** for optimistic locking by default.

5.14 Common Mistakes

1. **@Enumerated** left at the **ORDINAL** default. Silent data corruption when the enum changes.
2. Eager **@ManyToOne**. Loads half the object graph on every query.
3. **N+1 queries**. Linear degradation with data volume; invisible in development.
4. Exposing entities as **REST DTOs**. **LazyInitializationException** during serialisation, leaked internal fields, API coupled to schema.
5. **hibernate.database.generation=update** in production. No migration history, no review, no rollback.
6. **getClass()** in entity **equals**. Fails against Hibernate proxies. Use **instanceof**.
7. Mutable **hashCode** based on the **id**. Breaks **HashSet** membership when the entity is persisted.
8. **CascadeType.REMOVE** on a **@ManyToOne**. Deleting a child deletes the parent.
9. **List** for **@ManyToMany**. Delete-and-reinsert of every join row on any change. Use **Set**.
10. Unbounded queries. **SELECT o FROM Order o** with no **setMaxResults**.
11. Second-level cache on a write-heavy entity in a multi-replica deployment. Stale reads with no error.
12. **enable_lazy_load_no_trans=true**. Turns a loud exception into unbounded hidden queries.
13. Business logic that mutates a managed entity accidentally. Dirty checking persists it. If a method should not write, load read-only.

5.15 Interview Questions

Junior

- Q1. What is JPA and what is Hibernate?** JPA (Jakarta Persistence) is the specification for object-relational mapping in Java. Hibernate is the most widely used implementation, and the one Quarkus ships.
- Q2. What does @Entity do?** Marks a class as mapped to a database table. It requires an **@Id** and a no-arg constructor.
- Q3. What is the difference between @OneToMany and @ManyToOne?** **@ManyToOne** is the many side and normally owns the foreign key. **@OneToMany** is the one side and is usually the inverse (**mappedBy**).
- Q4. What is lazy loading?** Deferring the load of an association until it is accessed. Hibernate returns a proxy; touching it triggers a query.
- Q5. What is @Transactional for?** It wraps a method in a database transaction: everything commits together or rolls back together.

Q6. How do you write a query? JPQL through `em.createQuery(...)`, the Criteria API for dynamic queries, or native SQL through `em.createNativeQuery(...)`.

Q7. What is a persistence context? The set of entities Hibernate is currently managing, scoped to a transaction. It is also the first-level cache and the source of dirty checking.

Mid-level

Q8. What is the N+1 problem and how do you fix it? Loading N parents with one query, then one query per parent for an association — N+1 total. It scales linearly with data and is invisible on small datasets. Fixes: `LEFT JOIN FETCH`, a named entity graph, `@BatchSize` to load associations in batches, or — best — projecting into a DTO so the association is never loaded.

Q9. Why do I get `LazyInitializationException`? The persistence context closed and you touched an uninitialised proxy — typically mapping an entity to a DTO outside the transaction. Fix by fetching what you need in the query, or by projecting straight into a DTO. Do not enable `enable_lazy_load_no_trans`.

Q10. Optimistic vs pessimistic locking? Optimistic uses a `@Version` column: the update carries the expected version and fails with `OptimisticLockException` if another transaction won. No locks held, no contention, good default. Pessimistic takes a database lock (`SELECT ... FOR UPDATE`) for the transaction's duration — appropriate only under genuinely high contention where retrying is expensive.

Q11. What does Quarkus do to Hibernate at build time? Discovers entities from the Jandex index, performs bytecode enhancement during `mvn package` rather than at runtime, resolves the dialect from `db-kind` without connecting to the database, builds the metamodel during static init so it lands in the native image heap, and generates the reflection registrations native mode needs.

Q12. Why is `@Enumerated(EnumType.STRING)` important? The default `ORDINAL` stores the enum's position. Reordering or inserting a constant silently changes the meaning of existing rows — data corruption with no error.

Q13. When would you use the second-level cache? For reference data read constantly and written rarely — catalogues, tax rates, country codes. Not for write-heavy entities, and not in a multi-replica deployment without a distributed cache, because each pod caches independently and serves stale data after another pod writes.

Q14. `hbm2ddl.auto=update` in production — thoughts? No. It only adds, never alters or drops; it leaves no migration history; it cannot be reviewed before it runs; and its behaviour varies across Hibernate versions. Use Flyway or Liquibase, and set `generation=validate` in production as a startup assertion.

Q15. How do you size a connection pool? Against the database's capacity to do concurrent work, not against your request concurrency — roughly database cores $\times$ 2 as a starting point, then measured. Remember that total connections are `max-size` $\times$ replicas, so autoscaling multiplies it. Keep `acquisition-timeout` short so overload fails fast rather than holding worker threads.

Senior

Q16. Walk me through what happens when you call `em.persist(order)`. Hibernate makes the instance managed by adding it to the persistence context, and — if the id generator is a sequence with an allocation size — assigns an id, possibly without a database round trip if ids are

pre-allocated. With `IDENTITY` it must issue the insert immediately to obtain the id, which is one reason `IDENTITY` disables JDBC batching. No SQL is otherwise issued yet; the insert is queued in the action queue. Cascades to associated entities marked `CascadeType.PERSIST` are applied transitively. At flush — triggered by transaction commit, by an explicit `flush()`, or before a query whose results the pending changes could affect — Hibernate orders the actions, applies the insert, and updates the entity's snapshot. At commit the transaction ends and the persistence context closes, detaching everything.

Q17. A page is slow. Walk me through diagnosis. Measure before theorising. Turn on `quarkus.hibernate-orm.statistics` and SQL logging in a load test and look first at *query count*, because a count that grows with the result-set size is N+1 and is the most likely cause. If the count is fine, look at *query duration* and take the plan from the database — the usual finding is a missing index on a foreign key or on a sort column, which Hibernate never creates for you. If the plan is fine, look at how much data is being loaded: a query returning entities with twenty columns when the page shows three is doing unnecessary work in the driver, the persistence context and the serialiser, and a DTO projection removes all three. Then look at the pool: `acquisition-timeout` waits show the bottleneck is connections, not queries. Only after all that would I consider caching, because caching a slow query hides a design problem that returns as the data grows.

Q18. How do you evolve a schema safely during a rolling deployment? Assume old and new pods run concurrently against one schema for the duration of the rollout, so every migration must be compatible with both. That forces expand–migrate–contract across releases: release 1 adds the new nullable column and writes to both old and new; release 2 backfills and switches reads to the new column; release 3 drops the old one. Renames become add-copy-drop. I would run migrations as a Job or init container ahead of the rollout rather than `migrate-at-start`, because ten replicas racing on a long migration can exceed readiness deadlines and produce a rollback loop, and because a failed migration should stop the deployment before any new pod starts. In production I would keep `generation=validate` so a schema drift fails the boot loudly.

Q19. When would you choose native SQL over JPQL? When the database can express something JPQL cannot, or can do it far better: window functions, recursive CTEs, `INSERT .. ON CONFLICT`, full-text search, `JSONB` operators, bulk operations, and analytical aggregations. Also when a hand-written query with a known plan is materially faster than what the ORM generates. The cost is portability, which for a service that will run on PostgreSQL for its whole life is a theoretical cost worth paying. I would keep such queries in a clearly-named repository class, parameterised, and covered by tests against a real database via Dev Services — never assembled by string concatenation.

Q20. Hibernate ORM or Hibernate Reactive for a new service on Java 21? Blocking Hibernate ORM with `@RunOnVirtualThread`, in most cases. Virtual threads give the concurrency benefit of reactive without the rewrite: the code stays imperative and debuggable, lazy loading works, the second-level cache works, and the tooling is mature. Hibernate Reactive requires explicit fetching everywhere, has a smaller feature surface, and imposes reactive style on the whole call chain. I would choose Reactive when the service is already reactive end to end — a streaming pipeline, say — or when measurement shows the worker-thread model is the bottleneck. Usually it is not: the connection pool is, and neither model changes that.

Principal

Q21. Your platform has 40 services all talking to one PostgreSQL cluster, and you are hitting connection limits. How do you approach it? The arithmetic first: connections are `max-size` × replicas × services, and that number almost certainly exceeds what the database can usefully serve, so the immediate goal is to reduce it without reducing throughput. Short term, audit pool sizes — most will have been set to a copied default rather than measured, and halving them typically costs nothing because the database was never executing that many queries concurrently anyway. Introduce PgBouncer in transaction mode as a proxy, which is the standard fix and decouples application-side pool sizing from server-side connection limits; note that transaction mode forbids session-level state, so prepared-statement handling and any use of temporary tables needs checking. Medium term, the real question is architectural: 40 services sharing one database is a shared-database anti-pattern, and connection pressure is a symptom of it rather than the disease. I would look for the services with the highest connection cost and least coupling to the shared schema and give them their own database, and route reporting traffic to read replicas. Longer term, I would put a per-service connection budget into the platform’s paved road and make it visible on a dashboard, so growth is noticed before it becomes an incident. Throughout, I would resist the instinct to raise `max_connections`, which trades a hard failure for degraded performance across every service.

Q22. A team wants to introduce an ORM-free approach — plain SQL with a lightweight mapper — arguing Hibernate is the source of their performance problems. Evaluate. I would take the complaint seriously and the diagnosis sceptically. Hibernate does make certain mistakes easy — N+1, over-fetching, accidental writes through dirty checking — and a team that has been burned by them is reacting to something real. But the failures are usually caused by using it as a magic object graph rather than as a query tool, and the same team writing raw SQL will hit different problems: hand-written mapping code, no dirty checking so update statements are written by hand, no first-level cache so duplicate loads within a request, and a large surface for injection bugs. I would ask for the specific slow endpoints and look at whether DTO projections and fixed fetch strategies solve them, because in my experience they solve most of them, and that is a much smaller change than replacing the persistence layer. Where they are right is on the read side: complex reporting and analytical queries genuinely do not belong in JPQL, and a hybrid — Hibernate for the transactional write model, plain SQL or jOOQ for reads — is a legitimate and increasingly common architecture. So my likely recommendation is not “keep Hibernate” or “drop it”, but “split the read and write paths”, which gives them the control they want where it matters and keeps the safety where it helps.

5.16 Cheat Sheet

Configuration

```
quarkus.datasource.db-kind=postgresql          # BUILD TIME
quarkus.datasource.jdbc.url=jdbc:postgresql://db:5432/orders
quarkus.datasource.jdbc.max-size=20
quarkus.datasource.jdbc.acquisition-timeout=5S

quarkus.hibernate-orm.database.generation=none
quarkus.hibernate-orm.jdbc.statement-batch-size=50
quarkus.hibernate-orm.statistics=true
%dev.quarkus.hibernate-orm.log.sql=true
```


Entity essentials

```
@Entity @Table(name = "orders", indexes = @Index(columnList = "status"))
public class Order {
    @Id @GeneratedValue(strategy = SEQUENCE, generator = "s")
    @SequenceGenerator(name = "s", sequenceName = "order_id_seq", allocationSize = 50)
    Long id;

    @Enumerated(EnumType.STRING) OrderStatus status;    // never ORDINAL
    @Version int version;                               // optimistic locking

    @ManyToOne(fetch = LAZY) Customer customer;         // always LAZY
    @OneToMany(mappedBy = "order", cascade = ALL, orphanRemoval = true)
    List<OrderLine> lines = new ArrayList<>();
}
```

Fixing N+1

```
"SELECT DISTINCT o FROM Order o LEFT JOIN FETCH o.lines"    // join fetch
@BatchSize(size = 50)                                       // batch
@NamedEntityGraph(name = "g", attributeNodes = @NamedAttributeNode("lines"))
"SELECT new com.acme.OrderSummary(o.id, o.total) FROM Order o" // projection
```

Locking

```
@Version int version;                                       // optimistic
em.find(Order.class, id, LockModeType.PESSIMISTIC_WRITE);  // SELECT FOR UPDATE
```

Flyway

```
quarkus.flyway.migrate-at-start=true          # dev only
%prod.quarkus.flyway.migrate-at-start=false
%prod.quarkus.hibernate-orm.database.generation=validate

src/main/resources/db/migration/V1.0.0__create_orders.sql
```

Testing

```
@QuarkusTest @TestTransaction                // rolled back automatically
class RepoTest { @Inject EntityManager em; }
```

5.17 Summary

Quarkus does not change what Hibernate *is*; it changes *when* Hibernate does its work. Entity discovery, bytecode enhancement, dialect resolution and metamodel construction all move to build time, which is why a Quarkus service with a full ORM starts in under a second on the JVM and in tens of milliseconds natively, with no native-image configuration from you.

Everything else in this chapter is ordinary JPA discipline, and it is where the actual engineering lies. Lazy on every `@ManyToOne`. `STRING` on every `@Enumerated`. Projections instead of entities on read paths. Join fetch, entity graphs or batch sizing instead of N+1. Flyway instead of `hbm2ddl`. Optimistic locking by default. Pool sized against the database's capacity rather than your request concurrency.

The two things an interviewer will probe hardest are **N+1** and **LazyInitializationException**, because both reveal whether you understand the persistence context or merely use it. The strongest answers to both converge on the same advice: decide what you need before you query, and prefer projecting into a DTO over loading an entity graph you will not mutate.

Chapter 6 layers **Panache** on top of this — the active-record and repository patterns, the query shorthand, and where Panache’s convenience genuinely helps versus where it hides the JPA you still need to understand.

5.18 References

- Hibernate ORM in Quarkus — <https://quarkus.io/guides/hibernate-orm>
- Datasources — <https://quarkus.io/guides/datasource>
- Hibernate Reactive — <https://quarkus.io/guides/hibernate-reactive>
- Flyway — <https://quarkus.io/guides/flyway>
- Hibernate ORM 6 user guide — https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate_User_Guide.html
- Jakarta Persistence 3.1 — <https://jakarta.ee/specifications/persistence/3.1/>
- Agroal — <https://agroal.github.io/>

Panache

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · [quarkus-hibernate-orm-panache](#) · Hibernate ORM 6.6

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what Panache actually is — and, crucially, what it is *not*.
2. Choose between the **active record** and **repository** patterns with reasons you can defend in a design review.
3. Write Panache queries using the simplified HQL syntax, named parameters and [Parameters](#).
4. Paginate, sort, stream and project with Panache without loading more than you need.
5. Use [PanacheQuery](#) correctly — including the parts that surprise people, like [count\(\)](#) and [list\(\)](#) behaviour.
6. Generate a REST layer with Panache REST Data, and explain why you probably should not use it in production.
7. Understand the build-time bytecode generation that makes [Order.findById\(1L\)](#) work on a class with no such method.
8. Avoid the traps: N+1 through convenience, entity-as-DTO, transaction boundaries in the wrong place.
9. Use the reactive Panache API and know when it is warranted.
10. Answer 20+ interview questions on Panache, including “would you use active record in a large codebase?”

6.1 What Panache Is

Panache is a **thin layer of syntactic sugar over Hibernate ORM**, generated at build time. It is not a new ORM, not a replacement for JPA, and not an abstraction that lets you avoid understanding the persistence context.

Compare. Plain JPA:

```
@ApplicationScoped
public class OrderRepository {

    @Inject EntityManager em;

    public Optional<Order> findById(Long id) {
```

```
        return Optional.ofNullable(em.find(Order.class, id));
    }

    public List<Order> findByStatus(OrderStatus status) {
        return em.createQuery("SELECT o FROM Order o WHERE o.status = :status", Order.class)
            .setParameter("status", status)
            .getResultList();
    }

    public long countPending() {
        return em.createQuery("SELECT COUNT(o) FROM Order o WHERE o.status = :s", Long.class)
            .setParameter("s", OrderStatus.PENDING)
            .getSingleResult();
    }

    @Transactional
    public void save(Order order) { em.persist(order); }
}
```

The same thing with Panache:

```
@ApplicationScoped
public class OrderRepository implements PanacheRepository<Order> {

    public List<Order> findByStatus(OrderStatus status) {
        return list("status", status);
    }

    public long countPending() {
        return count("status", OrderStatus.PENDING);
    }

    // findById, persist, delete, listAll, count... all inherited
}
```

The saving is real, and it is entirely in boilerplate. Everything underneath — the persistence context, dirty checking, lazy loading, N+1, transaction semantics — is unchanged. **Every problem from Chapter 5 still applies.** Panache makes it faster to write the query; it does not make the query correct.

| Say this in an interview and you will stand out: *“Panache reduces boilerplate. It does not reduce the amount of JPA you need to understand — and because it makes the easy thing so easy, it can make N+1 and over-fetching more likely, not less.”*

6.1.1 How it works

Panache is another application of build-time augmentation. `PanacheEntity` declares methods like `findById` as `public static` with a body that throws:

```
// In the Panache runtime module - conceptually
public static <T extends PanacheEntityBase> T findById(Object id) {
```

```
throw new UnsupportedOperationException("Should be replaced by the build-time enhancer");
}
```

During augmentation, Quarkus finds every subclass of `PanacheEntityBase` and **rewrites the bytecode** of every call site, replacing the static call with a real implementation bound to the concrete entity type. That is why:

- The static methods know which entity they belong to, despite Java having no such mechanism.
- There is no reflection and no runtime proxying.
- It works in native mode with no configuration.
- Calling `Order.findById(1L)` from Kotlin, or reflectively, does **not** work — the enhancement happens at Java call sites during your build.

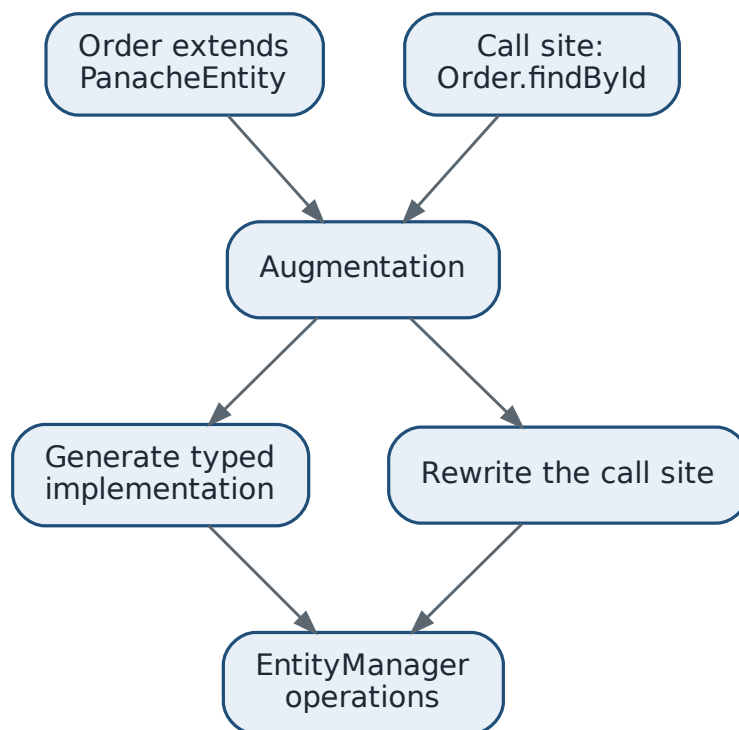


Figure 6.1 — Panache is bytecode rewriting at build time, not runtime magic.

6.2 Active Record

```
package com.acme.orders.domain;

import io.quarkus.hibernate.orm.panache.PanacheEntity;
```

```

import io.quarkus.panache.common.Page;
import io.quarkus.panache.common.Parameters;
import io.quarkus.panache.common.Sort;
import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

@Entity
@Table(name = "orders")
public class Order extends PanacheEntity {           // supplies a Long id

    @Column(name = "customer_email", nullable = false)
    public String customerEmail;                     // public fields - see below

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 32)
    public OrderStatus status = OrderStatus.PENDING;

    @Column(nullable = false, precision = 19, scale = 2)
    public BigDecimal total = BigDecimal.ZERO;

    @Column(name = "created_at", nullable = false, updatable = false)
    public Instant createdAt;

    @Version
    public int version;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    public List<OrderLine> lines = new ArrayList<>();

    @PrePersist
    void onCreate() { createdAt = Instant.now(); }

    // --- Domain queries live on the entity -----

    public static List<Order> findPending() {
        return list("status", OrderStatus.PENDING);
    }

    public static List<Order> findByCustomer(String email, Page page) {
        return find("customerEmail = ?1 order by createdAt desc", email)
            .page(page)
            .list();
    }

    public static long countSince(Instant since) {
        return count("createdAt >= ?1", since);
    }

    public static List<Order> highValue(BigDecimal threshold) {
        return find("total > :t and status = :s",
            Parameters.with("t", threshold).and("s", OrderStatus.PAID))
    }

```

```
        .list();
    }
}
```

```
// Usage
@Transactional
public Order create(CreateOrderRequest request) {
    Order order = new Order();
    order.customerEmail = request.customerEmail();
    order.persist();           // no repository needed
    return order;
}

Order order = Order.findById(1L);
List<Order> pending = Order.findPending();
long total = Order.count();
Order.deleteById(1L);
```

6.2.1 The public-fields question

Panache entities use `public` fields, which looks like a violation of encapsulation. It is not, quite: during augmentation Quarkus **generates getters and setters and rewrites every field access** to go through them. So `order.total` at a call site becomes `order.getTotal()` in bytecode.

That means:

- You get field syntax with method semantics.
- If you *write* a getter or setter yourself, Panache uses yours — so you can add validation or computed logic where it matters.
- Lazy loading still works, because access goes through the enhanced accessor.

```
@Entity
public class Order extends PanacheEntity {
    public BigDecimal total;

    // Explicit setter: Panache uses this instead of generating one
    public void setTotal(BigDecimal total) {
        if (total.signum() < 0) {
            throw new IllegalArgumentException("Total cannot be negative");
        }
        this.total = total;
    }
}
```

The real objection to public fields is not encapsulation but **discoverability**: someone reading `order.total = x` has no signal that a setter with validation exists. That is a genuine cost on a large team.

6.2.2 PanacheEntity vs PanacheEntityBase

```
// PanacheEntity: gives you a Long id with GenerationType.SEQUENCE
public class Order extends PanacheEntity { }

// PanacheEntityBase: you define the id yourself
public class Product extends PanacheEntityBase {
    @Id
    @Column(length = 64)
    public String sku;                // natural key

    public static Product findBySku(String sku) {
        return findById(sku);
    }
}
```

Use `PanacheEntityBase` whenever you need a composite key, a natural key, a UUID id, or a specific generation strategy. `PanacheEntity`'s generated id uses a sequence with the default allocation size — fine for most cases, but you cannot tune it without switching to `PanacheEntityBase`.

6.3 The Repository Pattern

```
package com.acme.orders.infra;

import io.quarkus.hibernate.orm.panache.PanacheRepository;
import jakarta.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class OrderRepository implements PanacheRepository<Order> {

    public List<Order> findPending() {
        return list("status", OrderStatus.PENDING);
    }

    public Optional<Order> findByReference(String reference) {
        return find("reference", reference).firstResultOptional();
    }

    public List<Order> page(int index, int size) {
        return findAll(Sort.by("createdAt").descending())
            .page(Page.of(index, size))
            .list();
    }

    public long markStale(Instant cutoff) {
        return update("status = ?1 where status = ?2 and createdAt < ?3",
            OrderStatus.EXPIRED, OrderStatus.PENDING, cutoff);
    }
}
```

```
// The entity stays a plain JPA entity - no Panache base class
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue Long id;
    private String customerEmail;    // private fields, real getters/setters
    // ...
}
```

For a non-Long id, use `PanacheRepositoryBase<Entity, IdType>`:

```
@ApplicationScoped
public class ProductRepository implements PanacheRepositoryBase<Product, String> { }
```

6.4 Choosing Between Them

This is the design question of the chapter, and it comes up in interviews constantly.

	Active record	Repository
Boilerplate	Least	Slightly more
Where queries live	On the entity	In a repository class
Entity is a POJO	No — extends a framework class	Yes
Mocking in unit tests	Hard (static methods)	Easy (inject a mock)
Multiple persistence strategies	Awkward	Natural
Domain-driven design fit	Poor — entity knows about persistence	Good
Discoverability of queries	Scattered across entities	Grouped by aggregate
Learning curve	Lowest	Low

6.4.1 A defensible position

Active record is excellent for small services, prototypes, CRUD-heavy admin tools, and anywhere the domain model is thin. The reduction in ceremony is real and the code reads well.

Repository is the better default for anything that will live for years with more than a couple of engineers on it, for three concrete reasons:

1. **Testability.** Active-record statics are rewritten at build time, so mocking them means `@InjectMock` on a Panache mock helper (`PanacheMock.mock(Order.class)`) rather than plain Mockito. It works, but it is friction on every test, and it forces `@QuarkusTest` where a plain JUnit test would do.
2. **The entity stops being a domain object.** `Order extends PanacheEntity` means your domain type has a persistence supertype. Any code that touches an `Order` transitively depends on Hibernate. If you ever want the domain model in a module with no persistence dependency — a common and healthy architecture — active record blocks it.

3. **Query discoverability.** In a service with 30 entities, “where are all the queries against `Order`?” has an obvious answer with a repository and a diffuse one with active record, where they accumulate as static methods interleaved with domain logic.

The interview answer that lands: *“Active record for small services, repository for anything I expect to maintain. The deciding factor is not style — it is that active record couples the domain type to the persistence framework and makes unit testing require a container. Neither matters at 3 entities; both matter at 30.”*

Mixing is fine. Panache does not care. A repository for the complex aggregate and active record for a simple lookup table is a perfectly reasonable codebase.

6.5 Querying

6.5.1 Simplified HQL

Panache lets you omit the `SELECT ... FROM Entity` prefix:

```
Order.find("status", OrderStatus.PENDING);
Order.find("status = ?1 and total > ?2", OrderStatus.PAID, new BigDecimal("100"));
Order.find("customerEmail like ?1", "%@acme.com");
Order.find("order by createdAt desc");
Order.find("status in ?1", List.of(OrderStatus.PAID, OrderStatus.SHIPPED));

// Full HQL still works when you need it
Order.find("SELECT o FROM Order o LEFT JOIN FETCH o.lines WHERE o.id = ?1", id);
```

Two parameter styles:

```
// Positional - ?1, ?2 (1-based)
Order.find("status = ?1 and total > ?2", status, threshold);

// Named, via Parameters
Order.find("status = :status and total > :total",
    Parameters.with("status", status).and("total", threshold));

// Named, via Map
Order.find("status = :status", Map.of("status", status));
```

Prefer `Parameters` for anything with more than two parameters — positional parameters in a long query are a readability and bug risk.

6.5.2 Terminal operations

```
PanacheQuery<Order> query = Order.find("status", OrderStatus.PENDING);

List<Order>    all      = query.list();           // materialise everything
Stream<Order>  stream   = query.stream();         // lazy, use inside a transaction
Order         first    = query.firstResult();    // null if empty
Optional<Order> firstOpt = query.firstResultOptional();
```

```

Order          single  = query.singleResult();           // throws unless exactly 1
Optional<Order> singleOpt= query.singleResultOptional();
long           count   = query.count();                 // a separate COUNT query
boolean        any     = query.count() > 0;
int            pages   = query.pageCount();

```

Shortcuts on the entity/repository skip the intermediate `PanacheQuery`:

```

List<Order> pending = Order.list("status", OrderStatus.PENDING);
long        n       = Order.count("status", OrderStatus.PENDING);
Order       one     = Order.find("reference", ref).firstResult();
boolean     exists  = Order.count("reference", ref) > 0;

```

`count()` issues a second query. `query.list()` followed by `query.count()` is two round trips. If you need both — the classic paginated response with a total — that is correct and unavoidable. If you only need to know whether anything matched, `firstResultOptional().isPresent()` is one query instead of a full count.

6.5.3 Paging and sorting

```

import io.quarkus.panache.common.Page;
import io.quarkus.panache.common.Sort;

PanacheQuery<Order> query = Order.find("status", OrderStatus.PAID);

// Index-based paging (0-based)
List<Order> firstPage = query.page(Page.of(0, 20)).list();
List<Order> nextPage  = query.nextPage().list();

// Range-based
List<Order> range = query.range(0, 19).list();    // inclusive

// Metadata
int  pageCount = query.pageCount();
long total    = query.count();
boolean hasNext = query.hasNextPage();

// Sorting
Order.find("status", Sort.by("createdAt").descending(), OrderStatus.PAID);
Order.findAll(Sort.by("status").and("createdAt", Sort.Direction.Descending));
Sort.by("total", Sort.Direction.Descending, Sort.NullPrecedence.NULLS_LAST);

```

A production-shaped paginated endpoint:

```

public record PageResponse<T>(List<T> items, int page, int size, long total, int pages) { }

@GET
public PageResponse<OrderSummary> list(
    @RestQuery @DefaultValue("0") int page,
    @RestQuery @DefaultValue("20") int size,

```

```

    @RestQuery @DefaultValue("createdAt") String sort) {

    int capped = Math.min(size, 100);           // never trust the client
    PanacheQuery<Order> query = Order.find(
        "status = ?1", Sort.by(sort).descending(), OrderStatus.PAID);

    List<OrderSummary> items = query.page(Page.of(page, capped))
        .project(OrderSummary.class)
        .list();

    return new PageResponse<>(items, page, capped, query.count(), query.pageCount());
}

```

Capping `size` matters: without it, `?size=1000000` is a denial-of-service vector against your own database.

6.5.4 Projections

Panache's `.project()` maps query results directly into a record, generating a `SELECT` of only the needed columns:

```

import io.quarkus.hibernate.orm.panache.commonProjectedFieldName;

public record OrderSummary(
    Long id,
    String status,
    BigDecimal total,
    @ProjectedFieldName("customerEmail") String email) { }

List<OrderSummary> summaries = Order.find("status", OrderStatus.PAID)
    .project(OrderSummary.class)
    .list();

// SQL: SELECT o.id, o.status, o.total, o.customer_email FROM orders o WHERE ...

```

Nested projections work through the relationship:

```

public record OrderWithCustomer(
    Long id,
    BigDecimal total,
    @ProjectedFieldName("customer.name") String customerName,
    @ProjectedFieldName("customer.email") String customerEmail) { }

```

This is the single most valuable Panache feature. It gives you Chapter 5's DTO-projection benefit — fewer columns, no persistence-context bookkeeping, no lazy-loading risk, immutable results — with one method call. On read-heavy endpoints it is routinely several times faster than loading entities and mapping them. Use it by default on any endpoint that only reads.

6.5.5 Streaming large results

```
@Transactional
public void exportAll(Writer out) {
    try (Stream<Order> stream = Order.streamAll()) {           // close it
        stream.forEach(order -> write(out, order));
    }
}
```

Two rules: the stream must be consumed **inside the transaction**, and it must be **closed** (use try-with-resources). Otherwise you leak a database cursor. For very large exports, also clear the persistence context periodically or it will grow to hold every entity you have touched:

```
@Inject EntityManager em;

@Transactional
public void exportAll(Writer out) {
    try (Stream<Order> stream = Order.streamAll()) {
        AtomicInteger n = new AtomicInteger();
        stream.forEach(order -> {
            write(out, order);
            if (n.incrementAndGet() % 500 == 0) {
                em.flush();
                em.clear();           // otherwise the context grows without bound
            }
        });
    }
}
```

6.6 Updates and Deletes

```
// Bulk update - a single UPDATE statement, bypasses the persistence context
@Transactional
public long expireStale(Instant cutoff) {
    return Order.update("status = ?1 where status = ?2 and createdAt < ?3",
        OrderStatus.EXPIRED, OrderStatus.PENDING, cutoff);
}

// Bulk delete
@Transactional
public long purge(Instant before) {
    return Order.delete("createdAt < ?1", before);
}

@Transactional
public void deleteAllOrders() {
    Order.deleteAll();
}
```

Bulk operations bypass the persistence context, the second-level cache and lifecycle callbacks. `Order.update(...)` will not fire `@PreUpdate`, will not cascade, and will leave any entity already loaded in the current persistence context holding a stale value. They are the right tool for large-scale maintenance operations, and the wrong tool inside a request that has already loaded the affected entities. If you must do both, `em.clear()` afterwards.

Instance-level operations behave normally:

```
@Transactional
public void markPaid(Long id) {
    Order order = Order.findById(id);
    order.status = OrderStatus.PAID;    // dirty checking persists it - no save()
}

@Transactional
public void cancel(Long id) {
    Order order = Order.findById(id);
    order.delete();                    // cascades and fires callbacks
}
```

6.7 Transactions

Panache does **not** manage transactions. `@Transactional` is still required for any write.

```
@ApplicationScoped
public class OrderService {

    @Transactional
    public Order create(CreateOrderRequest request) {
        Order order = new Order();
        order.customerEmail = request.customerEmail();
        order.persist();
        return order;
    }

    // Reads do not need @Transactional... but see below
    public List<OrderSummary> listPaid() {
        return Order.find("status", OrderStatus.PAID)
            .project(OrderSummary.class)
            .list();
    }
}
```

Two nuances worth knowing:

Reads outside a transaction work, because Hibernate opens a short-lived session per operation. But two reads outside a transaction are two sessions, so they see potentially different data and cannot share the first-level cache. For a read that issues several queries which must be consistent, wrap it in `@Transactional`.

`persist()` outside a transaction throws. The message is clear (`Transaction is not active`), but it is the most common first-day Panache error.

Transactions get full treatment in Chapter 7, including propagation, rollback rules, and why `@Transactional` on a `private` method does nothing.

6.8 Panache REST Data

Quarkus can generate a complete CRUD REST layer from a Panache entity.

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-rest-data-panache</artifactId>
</dependency>
```

```
public interface OrderResource extends PanacheEntityResource<Order, Long> { }
```

That interface alone generates:

```
GET    /order          list, with paging, sorting and filtering
GET    /order/{id}    get one
POST   /order        create
PUT    /order/{id}   update or create
DELETE /order/{id}   delete
```

```
@ResourceProperties(
  path = "orders",
  rolesAllowed = "admin",
  paged = true,
  hal = true)
public interface OrderResource extends PanacheEntityResource<Order, Long> {

  @MethodProperties(exposed = false)
  void delete(Long id);                // hide the delete endpoint

  @MethodProperties(rolesAllowed = "order-writer")
  Order add(Order order);
}
```

When this is appropriate: internal admin tools, prototypes, a data-browsing endpoint behind authentication, and demos.

When it is not — which is most production APIs:

- **It exposes your entities as the wire contract.** Every schema change is an API change, and lazy associations serialise unpredictably. This is exactly what Chapters 2 and 5 told you not to do.
- **There is no place for domain logic.** Creating an order usually means reserving stock, calculating tax and emitting an event. A generated `POST` does `persist()`.
- **Validation and authorisation are coarse.** Field-level rules and per-record ownership checks have nowhere to live.
- **The URL and payload shapes are the framework's,** not your API design's.

An interviewer asking about Panache REST Data is usually checking whether you reach for convenience without thinking about the contract. The strong answer names a legitimate use case *and* the reason it does not generalise.

6.9 Reactive Panache

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-reactive-panache</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-reactive-pg-client</artifactId>
</dependency>
```

```
import io.quarkus.hibernate.reactive.panache.PanacheEntity;
import io.quarkus.hibernate.reactive.panache.common.WithTransaction;
import io.smallrye.mutiny.Uni;
```

```
@Entity
public class Order extends PanacheEntity {
    public String customerEmail;
    public OrderStatus status;

    public static Uni<List<Order>> findPending() {
        return list("status", OrderStatus.PENDING);
    }
}
```

```
@Path("/orders")
public class OrderResource {

    @GET
    public Uni<List<Order>> list() {
        return Order.findPending();
    }

    @POST
    @WithTransaction // NOT @Transactional
    public Uni<Order> create(CreateOrderRequest request) {
        Order order = new Order();
        order.customerEmail = request.customerEmail();
        return order.persist();
    }
}
```

Differences that catch people out:

- **@WithTransaction**, not **@Transactional**. The blocking annotation does not work with a reactive session.

- **Every operation returns `Uni`.** There is no synchronous escape hatch.
- **Lazy loading does not work implicitly.** There is no thread to block on, so you must fetch explicitly — `Uni.chain, withTransaction(session -> session.fetch(...))`, or a join fetch in the query.
- **The second-level cache support is limited.**

The recommendation from Chapter 5 stands: on Java 21, blocking Panache with `@RunOnVirtualThread` is the better default unless you are already reactive end to end.

6.10 Testing

6.10.1 Repository pattern — easy

```
@QuarkusTest
@TestTransaction
class OrderRepositoryTest {

    @Inject OrderRepository repository;

    @Test
    void findsPending() {
        Order o = new Order();
        o.customerEmail = "a@acme.com";
        o.status = OrderStatus.PENDING;
        repository.persist(o);

        assertThat(repository.findPending()).hasSize(1);
    }
}
```

```
// Unit test with a mocked repository - no container
class OrderServiceTest {
    @Test
    void rejectsEmptyOrder() {
        var repo = mock(OrderRepository.class);
        var service = new OrderService(repo);
        assertThatThrownBy(() -> service.create(emptyRequest()))
            .isInstanceOf(EmptyOrderException.class);
    }
}
```

6.10.2 Active record — needs PanacheMock

```
@QuarkusTest
class OrderActiveRecordTest {

    @Test
    void mocksStaticFind() {
        PanacheMock.mock(Order.class);
        Order fake = new Order();
    }
}
```



```
        fake.id = 1L;
        fake.status = OrderStatus.PAID;

        Mockito.when(Order.findById(1L)).thenReturn(fake);

        assertThat(service.statusOf(1L)).isEqualTo(OrderStatus.PAID);

        PanacheMock.verify(Order.class).findById(1L);
    }
}
```

This works, but note what it costs: `@QuarkusTest` (seconds of startup) where the repository version needed no container at all. Multiplied across a test suite, that is the practical argument for repositories.

6.10.3 Asserting query counts

Carry the discipline from Chapter 5 forward — Panache’s convenience makes N+1 easier to introduce, so guard against it:

```
%test.quarkus.hibernate-orm.statistics=true
```

```
@Test
@TestTransaction
void listingDoesNotNPlusOne() {
    seed(50);
    var stats = sessionFactory.getStatistics();
    stats.clear();

    Order.find("status", OrderStatus.PAID).project(OrderSummary.class).list();

    assertThat(stats.getPrepareStatementCount()).isEqualTo(1);
}
```

6.11 Best Practices

1. **Repository by default** on anything you expect to maintain; active record for small or throwaway services.
2. **.project()** on every read-only endpoint. It is the highest-value feature Panache offers.
3. **Cap page size server-side**. Never let the client choose an unbounded `size`.
4. **Parameters over positional placeholders** once you have more than two.
5. **@Transactional** on writes, and on multi-query reads that must be consistent.
6. **Close and bound streams** — try-with-resources, plus periodic `flush()/clear()` for large exports.
7. **Keep entities out of the API**. Panache REST Data is for internal tools, not public contracts.
8. **Put domain logic in a service, not in the entity’s static methods**. Query methods on the entity are fine; business rules are not.
9. **Assert query counts in tests** — convenience makes N+1 easier to introduce.
10. Use **PanacheEntityBase** whenever the id is not a generated `Long`.

11. `em.clear()` after bulk `update/delete` if the current context holds affected entities.
12. **Do not treat Panache as an abstraction over JPA.** It is sugar. The persistence context is still there and still the thing that will surprise you.

6.12 Common Mistakes

1. Forgetting `@Transactional` on a write. `Transaction is not active`.
2. **Returning entities from REST endpoints.** Especially easy with Panache, because the entity is right there and already looks like a DTO. Lazy associations then blow up during serialisation.
3. **N+1 through convenience.** `Order.listAll()` then `order.lines.size()` in a loop. Panache does not change Chapter 5's rules; use join fetch, `@BatchSize` or projections.
4. **Expecting bulk `update/delete` to fire callbacks or update loaded entities.** They bypass the persistence context entirely.
5. **Unbounded `listAll()`.** Fine with 100 rows in dev, fatal with 10 million in production.
6. **Streaming outside a transaction, or without closing the stream.** Leaked cursors.
7. **Using active record then discovering the test suite needs `@QuarkusTest` everywhere.** A design cost that only appears at scale.
8. `@Transactional` on a reactive Panache method. Use `@WithTransaction`.
9. `singleResult()` when zero or many rows are possible. Throws. Use `firstResult Optional()` unless exactly one is a genuine invariant.
10. **Assuming `count()` is free.** It is a separate query. On a paginated endpoint that is fine; in a loop it is not.

6.13 Interview Questions

Junior

- Q1. What is Panache?** A layer over Hibernate ORM that removes boilerplate, offering active-record and repository styles with a simplified query syntax. It is generated at build time.
- Q2. What is the difference between active record and repository in Panache?** Active record: the entity extends `PanacheEntity` and carries static query methods. Repository: the entity is a plain JPA entity and a separate class implements `PanacheRepository<T>`.
- Q3. How do you find an entity by id?** `Order.findById(1L)` with active record, or `repository.findById(1L)` with a repository.
- Q4. How do you save a new entity?** `order.persist()` (or `repository.persist(order)`) inside a `@Transactional` method.
- Q5. Do you still need `@Transactional`?** Yes. Panache does not manage transactions. Any write needs it.
- Q6. How do you paginate?** `Order.find(...).page(Page.of(pageIndex, pageSize)).list()`, with `count()` and `pageCount()` for metadata.

Q7. Why do Panache entities use public fields? Quarkus generates getters and setters at build time and rewrites field accesses to use them, so you get field syntax with method semantics — and lazy loading still works.

Mid-level

Q8. How does Panache work internally? It is build-time bytecode transformation. The base classes declare static methods that throw; during augmentation Quarkus generates typed implementations per entity and rewrites every call site to use them. No reflection, no runtime proxies, and it works natively.

Q9. When would you choose repository over active record? Whenever the codebase is expected to live and grow. Three reasons: the entity stays a plain POJO rather than inheriting a persistence supertype; unit tests can mock the repository without a container, whereas active-record statics need `PanacheMock` and `@QuarkusTest`; and queries are grouped by aggregate rather than scattered across entities.

Q10. What does `.project()` do and why does it matter? It maps query results straight into a record, generating a `SELECT` of only the projected columns. You avoid loading entities, avoid persistence-context bookkeeping, avoid lazy-loading risk, and get an immutable result. It is usually several times faster on read paths.

Q11. What is the difference between `Order.update(...)` and modifying a loaded entity? `update` issues a bulk `UPDATE` that bypasses the persistence context, the second-level cache and lifecycle callbacks. Modifying a managed entity relies on dirty checking, fires callbacks and cascades. Bulk operations are right for maintenance, wrong inside a request that has already loaded the rows.

Q12. Does Panache prevent N+1? No — arguably it makes it more likely, because `order.lines` is so easy to touch. The Chapter 5 remedies all still apply: join fetch, entity graphs, `@BatchSize`, or projections.

Q13. How do you test a Panache repository? `@QuarkusTest` with `@TestTransaction` for repository-level tests against a real database via Dev Services, and plain JUnit with a mocked repository for service-level tests.

Q14. What is `PanacheEntityBase` for? When you do not want Panache's generated `Long id` — a natural key, a UUID, a composite key, or a specific generation strategy.

Senior

Q15. Critique Panache REST Data for a public API. It generates CRUD endpoints directly from entities, which makes the entity the wire contract. That couples the API to the schema, so every migration is a breaking change; it serialises lazy associations unpredictably; it leaves nowhere for domain logic, so creating an order becomes a bare `persist` rather than reserving stock and emitting an event; and validation and per-record authorisation have no natural home. It is genuinely useful for internal admin tools and prototypes where the schema *is* the contract and the audience is trusted. For a public API I would hand-write the resources with explicit DTOs.

Q16. Panache is described as reducing boilerplate. What does it not reduce? The amount of JPA you have to understand. The persistence context, dirty checking, flush ordering, lazy loading, cascade semantics, N+1 and locking are all unchanged — Panache is a call-site rewrite, not an abstraction. The risk is that by making queries trivially easy to write it lowers the barrier

to writing *bad* queries: `listAll()` on a large table, or a loop that touches a lazy collection. I treat Panache as a typing convenience and keep the same review standards I would apply to hand-written JPA, including query-count assertions in tests.

Q17. How would you structure a large Quarkus codebase’s persistence layer? Repository per aggregate root, not per table — the repository is the boundary of a consistency unit, so `OrderRepository` handles orders and their lines, and there is no `OrderLineRepository`. Repositories return domain objects or projections, never leak `PanacheQuery` to callers, and expose intention-revealing methods (`findOverdueFor(customer)`) rather than generic predicates. Read models use `.project()` into records that live with the API layer, so the read path never loads entities. Services own transactions and domain logic; repositories own queries. Bulk maintenance operations live in clearly named methods so it is obvious they bypass the context. I would also keep the domain entities free of Panache base classes, so the domain module could in principle be compiled without Hibernate — a constraint that keeps persistence concerns from leaking upward even if you never actually split the module.

Q18. A team’s Panache-based service degrades as data grows. Where do you look? The shape of the problem — degradation proportional to data volume rather than to traffic — points at unbounded queries and $N+1$ before anything else. First, look for `listAll()` and `find()` without paging; these are invisible until the table grows and are the most common cause. Second, enable Hibernate statistics under load and compare query counts against result-set sizes; a count that scales with rows is $N+1$, typically from a mapper touching a lazy collection. Third, check whether read endpoints load entities where a projection would do, since the cost of over-fetching also scales with data. Fourth, look at indexes on the columns the Panache queries filter and sort by, because the simplified syntax makes it easy to write a query without thinking about the plan. Only then would I look at caching or pool sizing. The fix that generalises is a test-level query-count assertion plus a hard server-side page cap, so the next instance of this fails in CI instead of in production.

Principal

Q19. Your organisation is standardising a persistence approach across 40 Quarkus services. What do you mandate, and what do you leave to teams? I would mandate the things whose inconsistency costs people outside the owning team, and leave style choices alone. Mandated: a hard server-side page-size cap and a standard paginated response envelope, because unbounded endpoints are a shared-database risk and inconsistent pagination is a client burden; Flyway for schema with the expand–migrate–contract convention, since migrations affect the shared cluster; per-service connection-pool budgets, visible on a dashboard; and a query-count assertion helper in the shared test library, because $N+1$ is the failure that most reliably escapes review. Left free: active record versus repository, projection record placement, and whether a given service uses Panache at all — some will legitimately want plain JPA or raw SQL for a reporting service. I would publish the repository pattern as the recommended default with the reasoning, rather than as a rule, because the cost of the wrong choice is borne locally and teams should be able to weigh it. The test I apply throughout: mandate where a mistake leaks into shared infrastructure or another team’s work, recommend everywhere else.

Q20. When would you argue against Panache entirely? Three situations. First, when the service is read-dominated and analytical — dashboards, reporting, aggregations over large tables — where the queries are window functions and CTEs that have no good HQL expression, and the ORM adds a mapping layer between you and the plan you actually care about; there, plain SQL

or jOOQ is a better fit. Second, when the team is building a domain model in the DDD sense with rich aggregates and invariants, and wants the domain module free of persistence dependencies; Panache's repository style is compatible with that, but active record is not, and the pull toward active record is strong enough that some teams prefer to remove the option. Third — and this is the honest one — when the team does not know JPA well. Panache's low friction lets a team ship a lot of code before they encounter the persistence context, and the debt arrives all at once. In that case I would rather they write explicit `EntityManager` code for a while, precisely because it is more visible. None of these are arguments that Panache is bad; they are arguments that convenience is only a benefit once you understand what it is hiding.

6.14 Cheat Sheet

Active record

```
@Entity
public class Order extends PanacheEntity {
    public String customerEmail;
    public static List<Order> findPending() { return list("status", PENDING); }
}

Order.findById(1L);      Order.listAll();      Order.count();
order.persist();         order.delete();      Order.deleteById(1L);
```

Repository

```
@ApplicationScoped
public class OrderRepository implements PanacheRepository<Order> { }
// non-Long id:
public class ProductRepository implements PanacheRepositoryBase<Product, String> { }
```

Queries

```
find("status", PENDING)
find("status = ?1 and total > ?2", PAID, amount)
find("status = :s", Parameters.with("s", PAID))
find("order by createdAt desc")
find("SELECT o FROM Order o LEFT JOIN FETCH o.lines WHERE o.id = ?1", id)
```

Terminals

```
.list() .stream() .firstResult() .firstResultOptional()
.singleResult() .singleResultOptional() .count() .pageCount()
```

Paging and sorting

```
.page(Page.of(0, 20)) .nextPage() .range(0, 19)
Sort.by("createdAt").descending()
Sort.by("status").and("createdAt", Sort.Direction.Descending)
```

Projections

```
public record OrderSummary(Long id, String status,
    @ProjectedFieldName("customerEmail") String email) { }

Order.find("status", PAID).project(OrderSummary.class).list();
```

Bulk

```
Order.update("status = ?1 where createdAt < ?2", EXPIRED, cutoff);
Order.delete("createdAt < ?1", cutoff);
Order.deleteAll();
```

Reactive

```
@WithTransaction // not @Transactional
Uni<Order> create(...) { return order.persist(); }
```

Testing

```
@QuarkusTest @TestTransaction class RepoTest { }
PanacheMock.mock(Order.class); // active record only
```

6.15 Summary

Panache is build-time bytecode rewriting that removes JPA boilerplate. It offers two styles — active record for brevity, repository for maintainability — and the choice is a real architectural decision, not a matter of taste: active record couples your domain type to the persistence framework and forces a container into unit tests, and both costs scale with codebase size.

The feature to reach for constantly is `.project()`, which gives you Chapter 5's DTO-projection benefit in one call and is the single biggest read-path win available. The trap to guard against is that Panache's convenience makes over-fetching and N+1 *easier* to write, not harder — so the discipline of paging everything, capping page size server-side, and asserting query counts in tests matters more here, not less.

Panache REST Data will generate a whole CRUD API from an entity. That is genuinely useful for internal tools and genuinely wrong for a public contract, and knowing which is which is the point.

Chapter 7 goes underneath all of this to **transactions** — propagation, isolation, rollback rules, the interaction with the persistence context, and the distributed-transaction question that every microservices interview eventually reaches.

6.16 References

- Simplified Hibernate ORM with Panache — <https://quarkus.io/guides/hibernate-orm-panache>
- Panache REST Data — <https://quarkus.io/guides/rest-data-panache>
- Reactive Panache — <https://quarkus.io/guides/hibernate-reactive-panache>
- Panache with Kotlin — <https://quarkus.io/guides/hibernate-orm-panache-kotlin>
- MongoDB with Panache — <https://quarkus.io/guides/mongodb-panache>

Transactions

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Narayana JTA · Jakarta Transactions 2.0 · Agroal

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what a transaction guarantees, in terms of ACID, and which of those guarantees your database actually gives you by default.
2. Use `@Transactional` correctly — including the three ways it silently does nothing.
3. Choose a propagation type deliberately and predict the behaviour of nested calls.
4. Reason about isolation levels and the anomalies each one permits.
5. Control rollback with `rollbackOn` / `dontRollbackOn`, and explain the default rule that surprises everyone.
6. Explain how the transaction interacts with the **persistence context** and when a flush actually happens.
7. Manage transaction boundaries around HTTP, messaging and scheduled work without holding connections too long.
8. Explain why **distributed transactions are the wrong answer** in microservices, and describe the Saga and Outbox patterns that replace them.
9. Test transactional behaviour, including rollback paths.
10. Answer 15+ interview questions, including the distributed-transaction question that appears in nearly every microservices interview.

7.1 What a Transaction Actually Guarantees

ACID is recited more often than it is understood. Precisely:

- **Atomicity** — all the statements commit or none do. The unit is all-or-nothing.
- **Consistency** — the transaction moves the database from one valid state to another, where “valid” means your constraints hold. This is largely *your* responsibility; the database only enforces the constraints you declared.
- **Isolation** — concurrent transactions do not observe each other’s intermediate states. **How much** isolation you get depends on the level, and the default is usually not full isolation.
- **Durability** — once committed, it survives a crash.

The one to think hardest about is isolation, because the default in PostgreSQL, Oracle and SQL Server is `READ_COMMITTED`, which permits non-repeatable reads and phantom reads. Most application bugs attributed to “a race condition” are in fact correct database behaviour at the configured isolation level.

7.2 @Transactional

Quarkus uses **Narayana**, a JTA implementation, exposed through `jakarta.transaction.Transactional`.

```
package com.acme.orders.domain;

import jakarta.transaction.Transactional;
import jakarta.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class OrderService {

    @Inject OrderRepository orders;
    @Inject StockService stock;

    @Transactional
    public Order create(CreateOrderRequest request) {
        Order order = new Order();
        order.customerEmail = request.customerEmail();
        request.items().forEach(i -> order.addLine(toLine(i)));

        orders.persist(order);
        stock.reserve(order);    // same transaction - both commit or neither

        return order;
    }
}
```

The transaction begins when the method is entered through the CDI proxy and commits when it returns normally. On an unchecked exception it rolls back.

7.2.1 The three ways @Transactional silently does nothing

This is the most-asked mid-level transaction question, and it has three parts.

1. Self-invocation. `@Transactional` is an interceptor binding, and interceptors are applied by the client proxy. A call through `this` bypasses the proxy.

```
@ApplicationScoped
public class OrderService {

    public void processAll(List<CreateOrderRequest> requests) {
        for (var r : requests) {
            create(r);    // NOT transactional - direct call
        }
    }
}
```



```
@Transactional
public Order create(CreateOrderRequest r) { ... }
}
```

Fix by calling through the proxy, or — better — by moving the method to another bean:

```
@Inject OrderService self;
// ...
self.create(r);                // transactional
```

2. Non-public methods. Interceptors do not apply to `private`, `protected` or package-private methods. The annotation is legal and inert.

```
@Transactional
private void create() { }      // does nothing
```

3. The method is not on a CDI bean. A new `OrderService()` is not managed, so there is no proxy and no interceptor.

In an interview, name all three. Most candidates name self-invocation. Naming the visibility rule and the “must be a bean” rule signals you have actually debugged this.

7.2.2 Where to put the boundary

The transaction should span exactly one unit of business work — usually a **service** method.

```
// WRONG: transaction on the resource. Serialisation and HTTP writing happen
// inside it, holding a database connection while you format JSON.
@Path("/orders")
public class OrderResource {
    @POST
    @Transactional
    public OrderView create(CreateOrderRequest r) { ... }
}

// WRONG: transaction on the repository. Each call is its own transaction,
// so a multi-step operation is not atomic.
@ApplicationScoped
public class OrderRepository {
    @Transactional
    public void persist(Order o) { ... }
}

// RIGHT: transaction on the service, spanning the business operation.
@ApplicationScoped
public class OrderService {
    @Transactional
    public Order create(CreateOrderRequest r) {
        orders.persist(order);
        stock.reserve(order);
        payments.authorise(order);
    }
}
```

```

    return order;
}
}

```

The rule of thumb: **a transaction should be as short as possible while remaining a complete unit of work.** Holding a connection while you call an external HTTP service, format a response, or write a file is how connection pools get exhausted.

7.3 Propagation

```

@Transactional(Transaction.TxType.REQUIRED)           // default
@Transactional(Transaction.TxType.REQUIRES_NEW)
@Transactional(Transaction.TxType.MANDATORY)
@Transactional(Transaction.TxType.SUPPORTS)
@Transactional(Transaction.TxType.NOT_SUPPORTED)
@Transactional(Transaction.TxType.NEVER)

```

Type	Caller has a transaction	Caller has none
REQUIRED (default)	Join it	Start one
REQUIRES_NEW	Suspend it, start a new one	Start one
MANDATORY	Join it	Throw <code>TransactionException</code>
SUPPORTS	Join it	Run without one
NOT_SUPPORTED	Suspend it, run without	Run without one
NEVER	Throw	Run without one

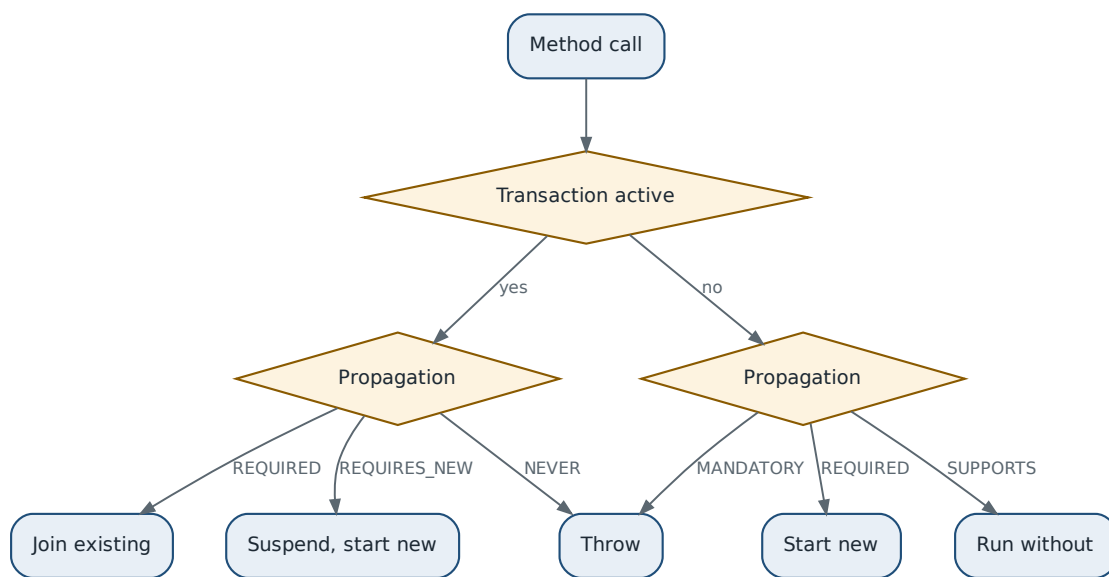


Figure 7.1 — Propagation decision.

7.3.1 REQUIRES_NEW — the audit case

The canonical use: something that must persist **even when the surrounding transaction rolls back**.

```
@ApplicationScoped
public class AuditService {

    @Transactional(Transaction.TxType.REQUIRES_NEW)
    public void record(String action, String detail) {
        auditRepository.persist(new AuditEntry(action, detail, Instant.now()));
    }
}

@Transactional
public Order create(CreateOrderRequest request) {
    audit.record("ORDER_ATTEMPT", request.customerEmail()); // its own transaction
    validate(request); // may throw
    orders.persist(order); // outer rolls back
    return order;
}
```

The audit row survives even though the order does not. That is exactly the desired behaviour for audit and for rate-limit counters.

The cost is a second connection. While the new transaction runs, the outer one is suspended but still holds its connection. A method that calls `REQUIRES_NEW` in a loop over 50 items will hold two connections and do 50 commits. Under load that is a pool exhaustion waiting to happen, and it is the reason `REQUIRES_NEW` should be rare and deliberate.

```
@ApplicationScoped
public class OrderRepository {

    // This must never start its own transaction; the caller owns the boundary.
    @Transactional(Transaction.TxType.MANDATORY)
    public void persist(Order order) {
        em.persist(order);
    }
}
```

This is a useful architectural constraint: it makes it a *runtime error* for someone to call a repository outside a service transaction, which prevents the accidental one-transaction-per-repository-call pattern.

7.4 Isolation Levels

```
@Transactional
@Transactional(timeout = 30)
public void process() { ... }
```

Jakarta Transactions has no isolation attribute, so in Quarkus you set it at the datasource level or per connection:

```
quarkus.datasource.jdbc.transaction-isolation-level=read-committed
```

Level	Dirty read	Non-repeatable read	Phantom read
<code>READ_UNCOMMITTED</code>	possible	possible	possible
<code>READ_COMMITTED</code>	no	possible	possible
<code>REPEATABLE_READ</code>	no	no	possible*
<code>SERIALIZABLE</code>	no	no	no

* PostgreSQL’s `REPEATABLE READ` is snapshot isolation and does prevent phantoms, but permits write skew. Its `SERIALIZABLE` uses predicate locking and can abort transactions with a serialisation failure that you must retry. Isolation semantics are database-specific; verify against yours rather than the table.

7.4.1 The anomalies, concretely

Non-repeatable read — the same row read twice in one transaction gives different values because another transaction committed in between.

```
@Transactional
public void check(long id) {
    Order a = em.find(Order.class, id);      // status = PENDING
    externalCall();                          // another transaction commits PAID
    em.clear();
    Order b = em.find(Order.class, id);      // status = PAID
}
```

Note the `em.clear()`: without it, Hibernate’s first-level cache returns the same instance, which masks the anomaly at the application level. That is a first-level-cache effect, not isolation.

Phantom read — the same query returns a different *set* of rows.

Write skew — two transactions each read a consistent snapshot, each make a decision that is valid given what they read, and together they violate an invariant. The classic case: two doctors each check “at least one doctor is on call”, each sees two, each goes off call. `SERIALIZABLE` prevents this; `REPEATABLE READ` on PostgreSQL does not.

7.4.2 What to actually do

Raising the isolation level is rarely the right first move — it increases contention and, at `SERIALIZABLE`, introduces retryable failures your code must handle. In practice:

- **Optimistic locking (@Version)** handles most lost-update problems with no contention cost, and surfaces conflicts as an explicit `OptimisticLockException` you can map to a 409.
- **Pessimistic locking (SELECT ... FOR UPDATE)** handles genuinely contended hot rows — inventory decrements, sequence counters.
- **A database constraint** handles invariants better than any isolation level: a unique index makes duplicate insertion impossible regardless of concurrency.

```
// Optimistic - the default choice
@Entity
public class Order {
    @Version int version;
}

// Pessimistic - for a contended row
@Transactional
public void decrementStock(long sku, int qty) {
    StockLevel level = em.find(StockLevel.class, sku, LockModeType.PESSIMISTIC_WRITE);
    if (level.available < qty) throw new InsufficientStockException(sku);
    level.available -= qty;
}
```

Interview answer worth having ready: “I reach for a constraint first, optimistic locking second, pessimistic locking third, and isolation level last. Raising isolation is a global change to fix a local problem, and at `SERIALIZABLE` it makes every transaction potentially retryable — which is a correctness burden on code that has nothing to do with the contended row.”

```
@Transactional
public void process() {
    orders.persist(order);
    throw new RuntimeException("boom");    // ROLLS BACK
}

@Transactional
public void process() throws BusinessException {
    orders.persist(order);
    throw new BusinessException("boom");    // COMMITS - checked exception!
}
```

Unchecked exceptions (`RuntimeException`, `Error`) roll back. Checked exceptions commit.

This is inherited from EJB and it is almost never what people expect. A checked `Insufficient FundsException` thrown after a partial write will commit the partial write.

```

@Transactional(rollbackOn = BusinessException.class)
public void rollsBackOnChecked() throws BusinessException { ... }

@Transactional(dontRollbackOn = OrderNotFoundException.class)
public void commitsAnyway() { ... }

@Transactional(rollbackOn = Exception.class)           // roll back on everything
public void alwaysRollBack() throws Exception { ... }

```

Programmatic marking:

```

@Inject TransactionManager tm;

@Transactional
public void conditional() {
    orders.persist(order);
    if (somethingIsWrong()) {
        tm.setRollbackOnly();    // will roll back at commit time
    }
}

```

Once marked rollback-only, any attempt to commit throws `RollbackException`. Note that the *rest of the method still runs* — `setRollbackOnly()` does not throw or return.

Practical recommendation: make your domain exceptions unchecked. A `RuntimeException` hierarchy for domain failures means the default rollback behaviour is correct everywhere, and you never have to remember `rollbackOn`. Checked exceptions in a service layer are, in my experience, more trouble than they are worth in a Quarkus application.

7.6 Transactions and the Persistence Context

The interaction between the JTA transaction and Hibernate’s persistence context is where the subtle bugs live.

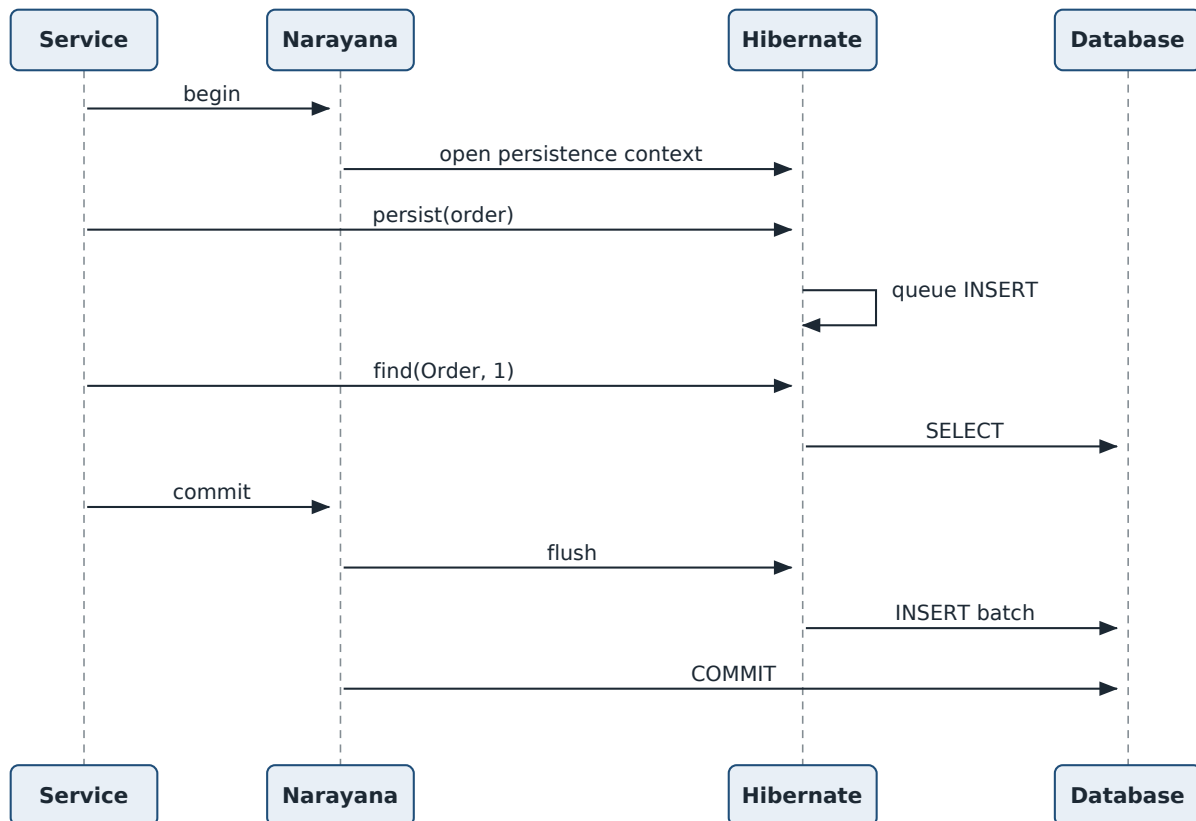


Figure 7.2 — Writes are queued and flushed at commit, not at the persist call.

7.6.1 When a flush happens

Hibernate does **not** issue your `INSERT` when you call `persist()`. It queues an action and flushes at:

1. **Transaction commit** — always.
2. **An explicit `em.flush()`.**
3. **Before a query whose results the pending changes could affect** (`FlushModeType.AUTO`, the default).

Point 3 causes a genuinely confusing class of bug:

```

@Transactional
public void surprise() {
    Order order = new Order();
    order.total = new BigDecimal("100");
    em.persist(order);                                // no SQL yet

    // This query touches Order, so Hibernate flushes the pending INSERT first
    long count = Order.count("total > ?1", BigDecimal.ZERO); // count includes it

    // ... whereas a query on an unrelated entity does not trigger the flush
}
  
```

7.6.2 Constraint violations surface at flush, not at persist

```
@Transactional
public void confusing() {
    try {
        em.persist(orderWithDuplicateReference);
    } catch (ConstraintViolationException e) {
        // NEVER REACHED - no SQL has been issued yet
        log.error("caught", e);
    }
} // <- the exception is thrown HERE, at commit, outside your try block
```

To handle it where you expect, flush explicitly:

```
@Transactional
public void better() {
    try {
        em.persist(order);
        em.flush(); // forces the INSERT now
    } catch (PersistenceException e) {
        throw new DuplicateOrderException(order.reference);
    }
}
```

But note: once a `PersistenceException` has been thrown, the persistence context is in an undefined state and the transaction is typically marked rollback-only. You cannot catch it, “fix” the entity and continue in the same transaction. The correct pattern is to check the invariant *before* persisting (a query, or better, catch it at a boundary and retry in a fresh transaction).

7.6.3 Long transactions hold connections

```
// BAD: the HTTP call happens inside the transaction
@Transactional
public Order createAndNotify(CreateOrderRequest r) {
    Order order = persist(r);
    notificationClient.send(order); // 2 seconds, holding a DB connection
    return order;
}

// BETTER: commit first, then notify
public Order createAndNotify(CreateOrderRequest r) {
    Order order = orderService.create(r); // transaction commits here
    notificationClient.send(order); // no connection held
    return order;
}

// BEST: an after-commit observer, so notification is not the caller's problem
void notify(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced event) {
    notificationClient.send(event.orderId());
}
```


Never call an external service inside a transaction. The connection is held for the duration of a network call you do not control; a downstream slowdown becomes pool exhaustion, which becomes a total outage. This is one of the most common causes of cascading failure in service architectures.

7.7 Programmatic Transactions

For dynamic boundaries, `QuarkusTransaction` is the modern, readable API:

```
import io.quarkus.narayana.jta.QuarkusTransaction;

@ApplicationScoped
public class BatchImporter {

    public void importAll(List<Row> rows) {
        // Chunked commits - one transaction per 500 rows
        for (List<Row> chunk : partition(rows, 500)) {
            QuarkusTransaction.requiringNew()
                .timeout(120)
                .run(() -> chunk.forEach(this::importRow));
        }
    }

    public Order withResult(CreateOrderRequest r) {
        return QuarkusTransaction.joiningExisting().call(() -> orderService.create(r));
    }

    public void withRollbackRules() {
        QuarkusTransaction.requiringNew()
            .exceptionHandler(t -> t instanceof BusinessException
                ? RunOptions.ExceptionResult.ROLLBACK
                : RunOptions.ExceptionResult.COMMIT)
            .run(this::doWork);
    }
}
```

Available entry points: `begin()`, `joiningExisting()`, `requiringNew()`, `disallowingExisting()`, `suspendingExisting()`.

The older `UserTransaction` API still works:

```
@Inject UserTransaction tx;

public void manual() throws Exception {
    tx.begin();
    try {
        doWork();
        tx.commit();
    } catch (Exception e) {
        tx.rollback();
        throw e;
    }
}
```

Prefer `QuarkusTransaction` — the try/catch/rollback dance is easy to get subtly wrong, particularly around the state of `tx` after a failed commit.

7.8 Transactions Beyond HTTP

7.8.1 Scheduled jobs

```
@ApplicationScoped
public class ReconciliationJob {

    @Scheduled(cron = "0 0 2 * * ?")
    @Transactional
    void nightly() {
        // Careful: a long-running transaction holds a connection all night
    }

    // Better: chunked, so each commit releases the connection
    @Scheduled(cron = "0 0 2 * * ?")
    void nightlyChunked() {
        List<Long> ids = findStaleIds(); // no transaction
        for (List<Long> chunk : partition(ids, 500)) {
            QuarkusTransaction.requiringNew().run(() -> process(chunk));
        }
    }
}
```

7.8.2 Kafka consumers

```
@Incoming("orders-in")
@Transactional
public void consume(OrderEvent event) {
    orders.persist(toOrder(event));
}
```

This is **not** exactly-once. The Kafka offset commit and the database commit are separate systems; a crash between them replays the message. Handle it with idempotency — a unique constraint on the event id, or an upsert:

```
@Incoming("orders-in")
@Transactional
public void consume(OrderEvent event) {
    if (processedEvents.exists(event.id())) return; // idempotent guard
    processedEvents.persist(new ProcessedEvent(event.id()));
    orders.persist(toOrder(event));
}
```

The unique constraint on `ProcessedEvent.id` is what makes this safe under concurrency — the check-then-insert is racy, but the constraint is not.

7.9 Distributed Transactions

7.9.1 XA, and why not

Narayana supports XA two-phase commit across multiple resources:

```
quarkus.datasource.jdbc.transactions=xa
quarkus.transaction-manager.enable-recovery=true
quarkus.transaction-manager.object-store.directory=/var/lib/orders/tx
```

```
@Transactional
public void acrossTwoDatabases() {
    ordersEm.persist(order);           // datasource A
    ledgerEm.persist(entry);           // datasource B - one XA transaction
}
```

XA is legitimate in a narrow set of cases: two databases you own, in the same data centre, with a durable transaction log and recovery configured. It is the wrong answer for microservices, for reasons worth being able to state:

- **It requires synchronous coupling.** Every participant must be available for the whole two-phase protocol.
- **Locks are held across the network** for the duration of both phases.
- **The coordinator is a single point of failure**, and its log must be durable and recovered — which conflicts with treating pods as cattle.
- **Most modern participants do not support it.** Kafka, Redis, most HTTP APIs, most cloud services.
- **Throughput is poor** and blocking under partition.

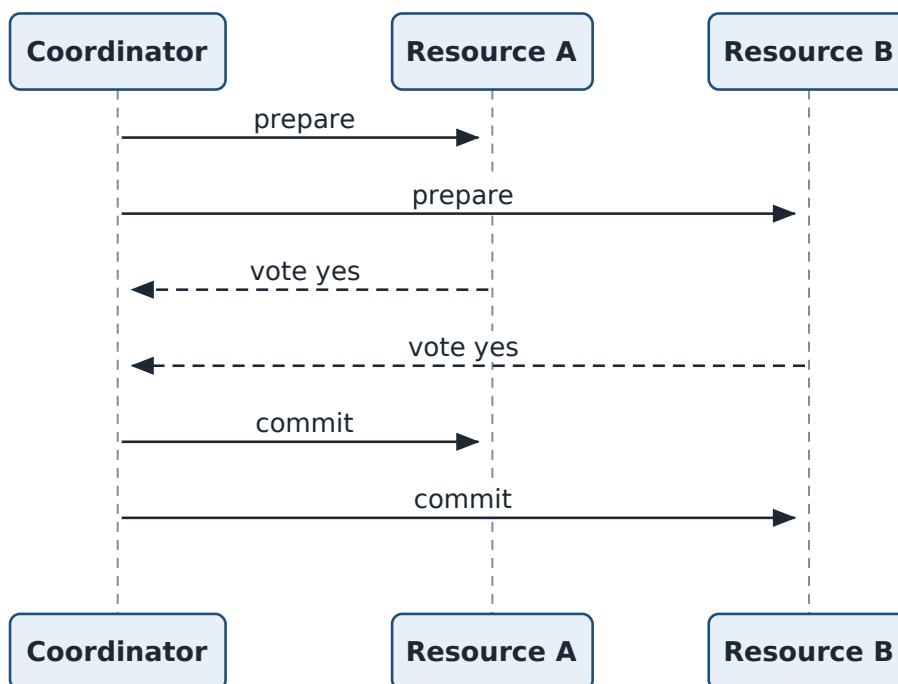


Figure 7.3 — Two-phase commit. If the coordinator fails between prepare and commit, both resources hold locks until recovery.

7.9.2 The Saga pattern

Replace one distributed transaction with a sequence of **local** transactions, each with a compensating action.

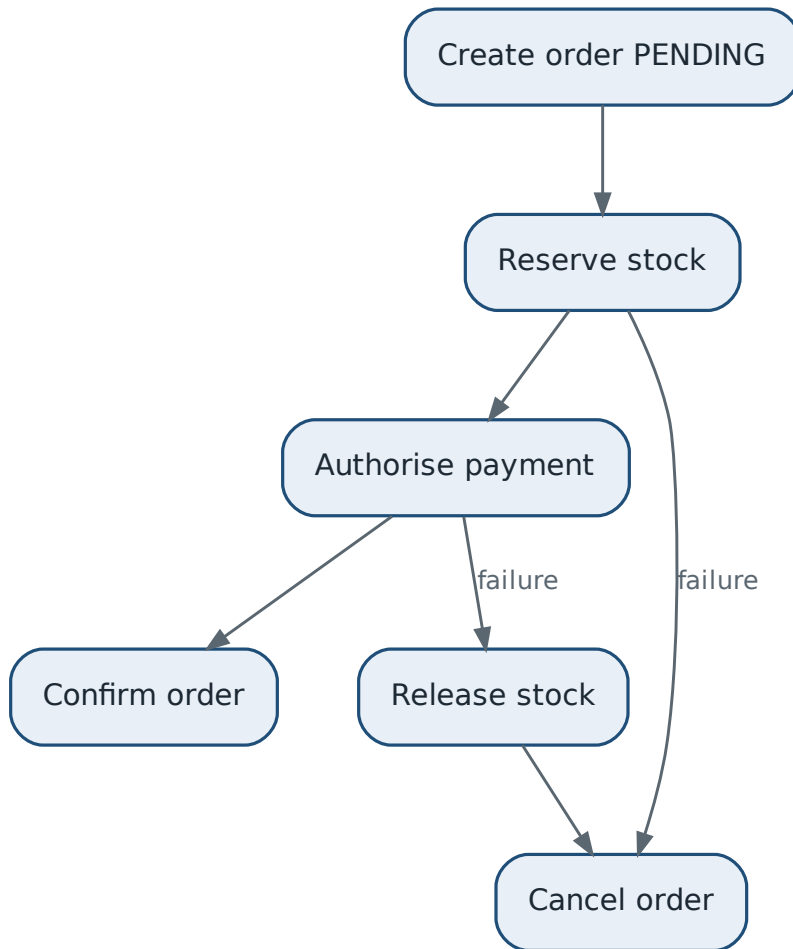


Figure 7.4 — Choreographed saga with compensations.

```

@ApplicationScoped
public class CheckoutSaga {

    @Transactional
    public void start(CreateOrderRequest request) {
        Order order = new Order();
        order.status = OrderStatus.PENDING;
    }
}
  
```

```

        orders.persist(order);
        events.fire(new OrderCreated(order.id));    // local transaction only
    }

    void onOrderCreated(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderCreated e) {
        stockClient.reserve(e.orderId());           // triggers StockReserved or StockFailed
    }

    @Transactional
    public void onStockFailed(StockReservationFailed e) {
        Order order = Order.findById(e.orderId());
        order.status = OrderStatus.CANCELLED;       // compensation
        events.fire(new OrderCancelled(order.id));
    }
}

```

Properties you must accept, and should name in an interview:

- **No isolation.** Intermediate states are visible. An order is genuinely **PENDING** for a while, and other parts of the system will see it.
- **Compensations are not rollbacks.** You cannot un-send an email; you send an apology. Compensating actions are domain operations with their own semantics.
- **Everything must be idempotent**, because steps will be retried.
- **Eventual consistency is a business decision**, not a technical one. Someone must agree that an order can be **PENDING** for thirty seconds.

Quarkus supports **LRA** (Long Running Actions) for orchestrated sagas:

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-narayana-lra</artifactId>
</dependency>

```

```

@Path("/stock")
public class StockResource {

    @POST @Path("/reserve")
    @LRA(LRA.Type.REQUIRED)
    public Response reserve(ReserveRequest r) { ... }

    @PUT @Path("/compensate")
    @Compensate
    public Response compensate(@HeaderParam(LRA.LRA_HTTP_CONTEXT_HEADER) URI lraId) {
        stock.release(lraId);
        return Response.ok().build();
    }

    @PUT @Path("/complete")
    @Complete
    public Response complete(@HeaderParam(LRA.LRA_HTTP_CONTEXT_HEADER) URI lraId) {
        return Response.ok().build();
    }
}

```

```
    }
}
```

7.9.3 The Transactional Outbox

The problem: you need to update the database **and** publish an event, atomically. You cannot — they are different systems.

The solution: write the event to an outbox table **in the same local transaction**, and publish it separately.

```
@Entity
@Table(name = "outbox", indexes = @Index(columnList = "published,createdAt"))
public class OutboxEvent extends PanacheEntity {
    public String aggregateType;
    public String aggregateId;
    public String eventType;
    @Column(columnDefinition = "jsonb")
    public String payload;
    public Instant createdAt;
    public boolean published = false;
}
```

```
@ApplicationScoped
public class OrderService {

    @Transactional
    public Order create(CreateOrderRequest request) {
        Order order = new Order();
        orders.persist(order);

        OutboxEvent event = new OutboxEvent();
        event.aggregateType = "Order";
        event.aggregateId   = order.id.toString();
        event.eventType      = "OrderCreated";
        event.payload        = json.writeValueAsString(OrderCreated.from(order));
        event.createdAt      = Instant.now();
        event.persist();     // SAME transaction - atomic with the order

        return order;
    }
}
```

```
@ApplicationScoped
public class OutboxPublisher {

    @Inject @Channel("orders-out") Emitter<String> emitter;

    @Scheduled(every = "1s", concurrentExecution = SKIP)
    void publish() {
        List<OutboxEvent> batch = OutboxEvent
            .find("published = false order by createdAt")
    }
}
```

```

        .page(Page.ofSize(100))
        .list();

        for (OutboxEvent event : batch) {
            emitter.send(event.payload);           // at-least-once
            markPublished(event.id);
        }
    }

    @Transactional
    void markPublished(Long id) {
        OutboxEvent.update("published = true where id = ?1", id);
    }
}

```

The production version uses **change data capture** — Debezium reading the outbox table from the database’s write-ahead log — which removes the polling loop and the publisher’s own failure modes. Either way, delivery is **at-least-once**, so consumers must be idempotent.

This is the single most useful pattern in this chapter. “Update the database and publish an event” is a requirement in almost every event-driven system, and the naive implementation — persist, then `emitter.send()` — loses events on crash and publishes phantom events on rollback. Being able to explain the outbox and why it is necessary is a strong senior signal.

7.10 Testing Transactions

```

@QuarkusTest
class OrderServiceTransactionTest {

    @Inject OrderService service;
    @Inject OrderRepository orders;

    @Test
    @TestTransaction           // rolled back after the test
    void createsOrder() {
        service.create(validRequest());
        assertEquals(1, orders.count());
    }

    @Test
    void rollsBackOnFailure() {
        // NOT @TestTransaction - we need the real rollback to happen
        assertThrows(() -> service.create(invalidRequest()),
            .assertInstanceOf(InsufficientStockException.class);

        assertEquals(0, orders.count());    // nothing persisted
    }

    @Test
    void auditSurvivesRollback() {

```

```

    assertThatThrownBy(() -> service.create(invalidRequest()))
        .assertInstanceOf(InsufficientStockException.class);

    // REQUIRES_NEW means the audit row committed independently
    assertThat(AuditEntry.count("action", "ORDER_ATTEMPT")).isEqualTo(1);
}
}

```

Note the pattern: `@TestTransaction` is convenient for isolation, but it makes it impossible to test *actual* rollback behaviour, because the whole test is already inside a transaction that will be rolled back. For rollback tests, run without it and clean up explicitly.

7.11 Best Practices

1. **Transaction boundary on the service**, not the resource and not the repository.
2. **Keep transactions short**. Never hold one across an external call.
3. **Make domain exceptions unchecked**, so the default rollback rule is correct.
4. **Optimistic locking by default**; pessimistic only for genuinely contended rows; constraints in preference to both.
5. **REQUIRES_NEW** sparingly — it consumes a second connection while the outer one is suspended.
6. **MANDATORY** on repositories to enforce that the caller owns the boundary.
7. **TransactionPhase.AFTER_SUCCESS** for any side effect that must not happen on rollback.
8. **The outbox pattern** whenever a database write and a message publish must be atomic.
9. **Idempotent consumers**, always — messaging is at-least-once.
10. **Chunked commits** for batch work, so connections are released regularly.
11. **em.flush()** explicitly when you need to handle a constraint violation where you can see it.
12. **Set a transaction timeout** for anything that could run long: `@TransactionConfiguration(timeout = 30)`.

7.12 Common Mistakes

1. **@Transactional** on a self-invoked method. Silently non-transactional.
2. **@Transactional** on a private method. Silently non-transactional.
3. **Expecting a checked exception to roll back**. It commits. Use `rollbackOn` or unchecked exceptions.
4. **An external HTTP call inside a transaction**. Holds a connection for the duration of a network call you do not control.
5. **Catching PersistenceException and continuing**. The context is poisoned and the transaction is rollback-only.
6. **Expecting a constraint violation at persist()**. It surfaces at flush, usually at commit, outside your try block.
7. **@Transactional** on a Kafka consumer and calling it exactly-once. It is not; offsets and the database are separate systems.

- 8. **XA across microservices.** Synchronous coupling, held locks, a coordinator SPOF, and most participants do not support it.
- 9. **Persist-then-publish without an outbox.** Loses events on crash, publishes phantoms on rollback.
- 10. **A single transaction over a million-row batch.** Long-held connection, huge undo log, and a timeout that rolls back hours of work.
- 11. **Raising the isolation level to fix one contended row.** A global change for a local problem; at `SERIALIZABLE` it makes every transaction retryable.

7.13 Interview Questions

Junior

- Q1. What is a transaction?** A unit of work that either commits entirely or has no effect, with guarantees about atomicity, consistency, isolation and durability.
- Q2. How do you make a method transactional in Quarkus?** `@Transactional` from `jakarta.transaction` on a public method of a CDI bean.
- Q3. What happens if an exception is thrown?** An unchecked exception rolls the transaction back. A checked exception does **not** — it commits, unless you declare `rollbackOn`.
- Q4. Where should the `@Transactional` annotation go?** On the service method that represents one business operation — not on the REST resource and not on individual repository methods.
- Q5. What is `@Version` for?** Optimistic locking. Hibernate includes the expected version in the `UPDATE`; if another transaction changed the row first, zero rows match and you get an `OptimisticLockException`.

Mid-level

- Q6. Name three reasons `@Transactional` might silently do nothing.** Self-invocation through `this` bypasses the CDI proxy that applies the interceptor; the method is not `public`; or the object is not a CDI bean at all.
- Q7. Explain `REQUIRED` versus `REQUIRES_NEW`.** `REQUIRED` joins an existing transaction or starts one. `REQUIRES_NEW` always suspends any existing transaction and starts a fresh one, which commits or rolls back independently — useful for audit records that must survive the caller's rollback, at the cost of holding a second connection.
- Q8. What isolation level does Quarkus use by default, and what does it permit?** Whatever the datasource is configured for; for PostgreSQL that is `READ_COMMITTED`, which prevents dirty reads but permits non-repeatable reads and phantom reads.
- Q9. When does Hibernate actually issue the `INSERT`?** At flush, which happens at commit, on an explicit `em.flush()`, or before a query whose results the pending change could affect. Not at `persist()`.
- Q10. Why should you not call an external service inside a transaction?** The database connection is held for the duration of a network call you do not control. A downstream slowdown becomes connection-pool exhaustion, which takes the whole service down — a cascading failure.

Q11. How do you handle “save to the database and publish an event” atomically? You cannot, directly — they are separate systems. Use the transactional outbox: write the event to an outbox table in the same local transaction, then publish it asynchronously, either by polling or with change data capture. Delivery is at-least-once, so consumers must be idempotent.

Q12. What is `setRollbackOnly()`? It marks the current transaction so that it can only roll back. The method continues executing; the rollback happens when commit is attempted.

Senior

Q13. Walk me through what happens when a `@Transactional` method is invoked. The call goes through the bean’s client proxy, which applies the transactional interceptor. The interceptor asks the transaction manager whether a transaction is active and applies the propagation rule — for `REQUIRED`, join or begin. Beginning enrolls the Aoplo datasource, which takes a connection from the pool and associates it with the transaction. Hibernate opens a persistence context bound to that transaction. Your method body runs; `persist` and entity mutations queue actions rather than issuing SQL. On normal return the interceptor commits: Hibernate flushes — ordering inserts, updates and deletes, and applying JDBC batching — then the transaction manager commits, the connection returns to the pool, and the persistence context closes, detaching every entity. On an unchecked exception the interceptor rolls back instead, and any entity you are still holding is detached and possibly in an inconsistent state.

Q14. Why are distributed transactions the wrong answer in microservices, and what do you use instead? XA requires every participant to be available for both phases of the protocol, holds locks across the network for that whole duration, needs a durable coordinator log that must survive restarts — which fits badly with ephemeral pods — and most modern participants such as Kafka and HTTP services do not support it at all. It also converts an availability problem in one service into an availability problem in all of them. The replacement is a saga: a sequence of local transactions, each with a compensating action, coordinated either by choreography through events or by orchestration through something like LRA. The trade you accept is isolation — intermediate states are visible, so an order really is `PENDING` for a while — and that compensations are domain operations rather than rollbacks, because you cannot un-send an email. That trade has to be agreed as a business decision, not made silently in code.

Q15. Design an idempotent, exactly-once-effective message consumer. Exactly-once delivery is not available, so the goal is exactly-once *effect*. The consumer takes the event’s unique id and, in the same local transaction as its business write, inserts a row into a `processed_events` table with a unique constraint on that id. If the insert violates the constraint, the message is a duplicate and the transaction rolls back harmlessly. The unique constraint — not the check — is what makes this correct under concurrency, because a check-then-insert is racy. The business write and the dedup row commit atomically, so there is no window where one exists without the other. If the consumer also publishes events, those go to an outbox in the same transaction. Operationally I would age out old `processed_events` rows on a schedule, sized against the broker’s retention so a replay cannot outlive the dedup window, and I would make the consumer’s business operation naturally idempotent where possible — an upsert rather than an insert — because that is cheaper than dedup bookkeeping when the domain permits it.

Q16. A team reports intermittent `OptimisticLockException` in production. How do you respond? First, establish whether it is a bug or the system working correctly — optimistic locking exists to surface concurrent modification, and an occasional conflict on a genuinely contended row

is expected behaviour, not a failure. So I look at the rate and the distribution: a low rate spread across many rows is normal, a high rate concentrated on a few rows means real contention that needs a design answer. If it is normal, the fix is at the API boundary: map it to a 409 with a clear message so clients retry rather than seeing a 500, and make sure it is not being logged as an error and paging someone. If it is concentrated contention — a counter, an inventory row, a shared aggregate — optimistic locking is the wrong tool, because every conflict wastes the whole transaction; I would move to a pessimistic lock on that row, or better, restructure so the contended value is not a single row at all: an append-only ledger with a periodic rollup, or a database-side atomic decrement. I would also check whether the aggregate is too coarse, since a common cause is unrelated fields sharing one `@Version`, so two users editing different parts of the same order collide unnecessarily.

Principal

Q17. Your platform is moving from a monolith with a single database to services with their own databases. How do you handle the transactions that currently span what will become service boundaries? The first move is to inventory them and challenge each one, because in a monolith it is free to wrap unrelated work in one transaction, and a substantial fraction of the existing boundaries are accidental rather than required. For each genuine cross-boundary transaction I would ask what invariant it protects and whether that invariant must hold *immediately* or merely *eventually* — and this is a business conversation, not a technical one. Most turn out to tolerate eventual consistency once someone states plainly what the intermediate state looks like to a user. Those become sagas with explicit compensations, and the compensations get designed with the domain experts, because “release the reservation” and “refund the payment” have business semantics that engineers should not invent. Where the invariant genuinely must be immediate — and there usually are a few — that is strong evidence the split is wrong, and the right answer is to keep those entities in the same service rather than to reach for XA. I would also introduce the outbox pattern early and platform-wide, because every one of these services will need to write and publish atomically, and a shared implementation with CDC is much better than eight teams each discovering the problem. Finally I would insist on idempotency as a platform-level requirement with a shared dedup helper, since at-least-once delivery is now the substrate everything sits on. The failure mode I would guard hardest against is a team implementing a saga without compensations, which is not a saga — it is a distributed transaction that silently loses data.

Q18. Argue for keeping a shared database and avoiding sagas. The case is stronger than microservices orthodoxy admits. A single database gives you real transactions, so invariants are enforced by the engine rather than by hand-written compensations that must be designed, implemented, tested for every failure ordering, and maintained forever. Sagas move complexity from the database — which is very good at this and has had forty years of work put into it — into application code written under deadline. They also make the system harder to reason about: intermediate states become visible to users and to other services, every consumer must be idempotent, and debugging a partially-completed saga across five services is materially harder than reading one transaction log. Operationally, one well-run database cluster is cheaper and better understood than eight, and cross-entity reporting stops requiring a data pipeline. The genuine costs of a shared database are coupling at the schema level and a shared blast radius, and those are real — but they are addressable with schema ownership conventions, per-service credentials and separate schemas, which is a much smaller change than distributed sagas. My honest position is that the split should be driven by team autonomy and independent deployability where those are actually constrained, not by a default assumption, and that a modular monolith with clear

internal boundaries is the right answer far more often than the industry's rhetoric suggests. Where I would split without hesitation: genuinely different scaling profiles, genuinely different availability requirements, or a compliance boundary that requires data separation.

7.14 Cheat Sheet

Basics

```
@Transactional                                // REQUIRED (default)
@Transactional(TxType.REQUIRES_NEW)           // suspend + new
@Transactional(TxType.MANDATORY)              // throw if none
@Transactional(rollbackOn = BusinessException.class)
@Transactional(dontRollbackOn = NotFoundException.class)
@TransactionalConfiguration(timeout = 30)
```

Rollback rule

```
RuntimeException / Error -> ROLLBACK
checked Exception       -> COMMIT   (override with rollbackOn)
```

Programmatic

```
QuarkusTransaction.requiringNew().timeout(120).run(() -> work());
QuarkusTransaction.joiningExisting().call(() -> result());
tm.setRollbackOnly();
```

Locking

```
@Version int version;                                // optimistic
em.find(E.class, id, LockModeType.PESSIMISTIC_WRITE); // SELECT FOR UPDATE
em.find(E.class, id, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
```

Isolation

```
quarkus.datasource.jdbc.transaction-isolation-level=read-committed
```

Silently non-transactional

1. this.method() - bypasses the proxy
2. private/protected - interceptors do not apply
3. not a CDI bean - no proxy at all

After-commit side effects

```
void notify(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) { }
```

Outbox

```
@Transactional
void create() {
    order.persist();
    outboxEvent.persist();    // same transaction
}
// publish separately, at-least-once, consumers idempotent
```

7.15 Summary

`@Transactional` looks simple and hides three sharp edges: it does nothing on self-invoked or non-public methods, checked exceptions commit rather than roll back, and the SQL you think happened at `persist()` actually happens at flush — usually at commit, outside whatever try block you wrote.

The boundary belongs on the service, spanning exactly one business operation, and it should be as short as possible. The most damaging transaction bug in practice is not a correctness bug at all: it is holding a connection across an external call, which converts a downstream slowdown into pool exhaustion and a full outage.

For concurrency, prefer a database constraint, then optimistic locking, then pessimistic locking, and treat raising the isolation level as a last resort — it is a global change to fix a local problem.

Across service boundaries there are no transactions. XA exists and is the wrong answer for microservices. What replaces it is the saga, with compensating actions designed with the business, and the **transactional outbox**, which is how you write to the database and publish an event atomically. Both make delivery at-least-once, which makes **idempotency** a non-negotiable property of every consumer you write.

Chapter 8 covers **validation** — Bean Validation, where to apply it, how it interacts with the transaction boundary, and how to produce error responses that clients can actually act on.

7.16 References

- Transactions in Quarkus — <https://quarkus.io/guides/transaction>
- Narayana — <https://narayana.io/>
- Jakarta Transactions 2.0 — <https://jakarta.ee/specifications/transactions/2.0/>
- Narayana LRA — <https://quarkus.io/guides/latest/lra>
- Saga pattern — <https://microservices.io/patterns/data/saga.html>
- Transactional outbox — <https://microservices.io/patterns/data/transactional-outbox.html>
- Debezium outbox event router — <https://debezium.io/documentation/reference/transformations/outbox-event-router.html>

Validation

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-hibernate-validator` · Jakarta Bean Validation 3.0

Learning Objectives

By the end of this chapter you will be able to:

1. Apply Bean Validation constraints to records, entities, method parameters and configuration.
2. Explain **where** validation belongs in a layered architecture, and why “validate everywhere” is a design failure.
3. Distinguish **syntactic** validation from **semantic** validation and from **domain invariants**, and place each correctly.
4. Write custom constraints, including cross-field and database-backed ones.
5. Use validation groups and `@GroupSequence` for staged and conditional rules.
6. Produce error responses that a client can act on programmatically.
7. Understand the interaction between validation, the transaction boundary and Hibernate’s lifecycle callbacks.
8. Validate configuration, REST clients and Kafka payloads — not just HTTP request bodies.
9. Test validation properly, including the negative cases people skip.
10. Answer 15+ interview questions on validation and API error design.

8.1 Three Kinds of Validation

Most confusion about validation comes from treating it as one thing. It is three, and they belong in different places.

Kind	Question it answers	Where it belongs
Syntactic	Is this well-formed?	The API boundary
Semantic	Is this meaningful in context?	The service layer
Invariant	Is the domain object in a legal state?	The domain object itself

```
// SYNTACTIC - Bean Validation at the boundary
public record CreateOrderRequest(
    @NotBlank @Email String customerEmail,
    @NotEmpty @Valid List<LineItemRequest> items) { }
```

```

// SEMANTIC - requires context the annotation cannot see
@Transactional
public Order create(CreateOrderRequest request) {
    if (!customers.exists(request.customerEmail())) {
        throw new UnknownCustomerException(request.customerEmail());
    }
    if (!stock.availableFor(request.items())) {
        throw new InsufficientStockException();
    }
    ...
}

// INVARIANT - the object refuses to exist in an illegal state
public record Money(BigDecimal amount, String currency) {
    public Money {
        if (amount == null || currency == null) {
            throw new IllegalArgumentException("Money requires amount and currency");
        }
        if (amount.scale() > 2) {
            throw new IllegalArgumentException("Money supports at most 2 decimal places");
        }
    }
}

```

The mistake to avoid is validating the same thing at every layer. It produces duplicated rules that drift apart, error messages that vary by call path, and a codebase where nobody knows which check is authoritative. Validate syntax once, at the boundary. Enforce invariants in constructors. Put contextual rules in the service, where the context lives.

8.2 Getting Started

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-validator</artifactId>
</dependency>

```

Quarkus wires Hibernate Validator into REST resources, CDI beans, configuration mappings and JPA lifecycle events. As with everything else, the constraint metadata is built at augmentation time, so there is no runtime scanning and it works in native mode.

```

@Path("/orders")
public class OrderResource {

    @POST
    public RestResponse<OrderView> create(@Valid CreateOrderRequest request) {
        // If validation fails, the method is never entered
        return RestResponse.status(Status.CREATED, service.create(request));
    }
}

```

`@Valid` on the parameter triggers validation of the request body. Without it, the constraints on the record are inert — a genuinely common oversight, because the code looks correct.

8.3 The Constraint Vocabulary

8.3.1 Built-in constraints

```
public record CreateOrderRequest(

    // Presence
    @NotNull           String required,           // not null (may be blank)
    @NotEmpty          List<String> nonEmpty,      // not null, size > 0
    @NotBlank          String text,               // not null, has non-whitespace

    // Size and range
    @Size(min = 3, max = 64) String sku,
    @Min(1) @Max(999)      int quantity,
    @Positive             BigDecimal price,
    @PositiveOrZero        BigDecimal discount,
    @Negative              int adjustment,
    @DecimalMin("0.01") @DecimalMax("99999.99") BigDecimal amount,
    @Digits(integer = 10, fraction = 2) BigDecimal money,

    // Format
    @Email                String customerEmail,
    @Pattern(regexp = "[A-Z]{2}-\\d{6}$", message = "must look like GB-123456")
                          String reference,

    // Time
    @Past                  LocalDate birthDate,
    @PastOrPresent         Instant createdAt,
    @Future                LocalDate deliverBy,
    @FutureOrPresent       Instant validFrom,

    // Boolean
    @AssertTrue(message = "terms must be accepted") boolean termsAccepted,
    @AssertFalse           boolean suppressed,

    // Cascade into nested objects - REQUIRED, not implicit
    @Valid @NotEmpty       List<LineItemRequest> items,
    @Valid                  AddressRequest shippingAddress
) { }
```

Hibernate Validator adds useful non-standard ones:

```
@Length(min = 2, max = 5)           // String-specific @Size
@Range(min = 1, max = 100)          // combined @Min/@Max
@URL                                 // well-formed URL
@CreditCardNumber                   // Luhn check
@ISBN
```



```
@UUID
@CodePointLength(max = 100)      // counts code points, not chars - emoji-safe
@DurationMin(seconds = 1) @DurationMax(minutes = 5)
```

8.3.2 @Valid does not cascade automatically

```
// BROKEN: LineItemRequest's constraints are never checked
public record CreateOrderRequest(
    @NotEmpty List<LineItemRequest> items) { }

// CORRECT
public record CreateOrderRequest(
    @NotEmpty @Valid List<LineItemRequest> items) { }
```

This catches people constantly. Cascading is opt-in at every level.

8.3.3 Constraints on containers

Jakarta Bean Validation 3.0 supports type-argument constraints:

```
public record Filters(
    List<@NotBlank String> tags,
    Map<@NotBlank String, @Positive Integer> quantities,
    Optional<@Email String> contact,
    List<@Valid LineItemRequest> items) { }
```

`List<@NotBlank String>` validates each **element**; `@NotEmpty List<String>` validates the **list**. They compose:

```
@NotEmpty List<@NotBlank String> tags      // non-empty list of non-blank strings
```

8.4 Where to Validate

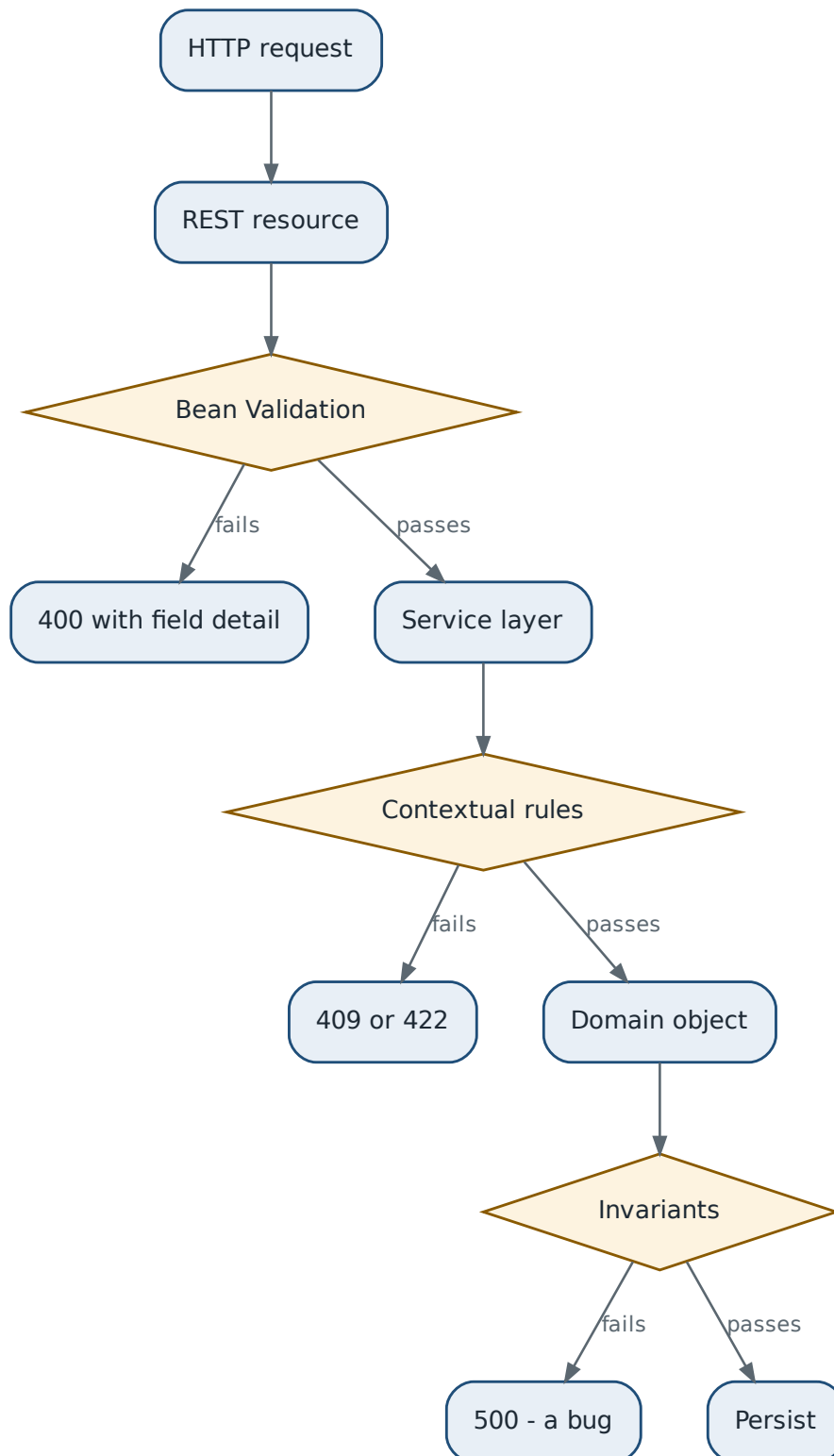


Figure 8.1 — Each layer answers a different question. An invariant violation reaching the domain is a bug, not a user error.

8.4.1 The boundary

```
@Path("/orders")
public class OrderResource {

    @POST
    public RestResponse<OrderView> create(@Valid CreateOrderRequest request) { ... }

    @GET
    public List<OrderSummary> list(
        @RestQuery @Min(0) int page,
        @RestQuery @Min(1) @Max(100) int size) { ... }    // params too
}
```

8.4.2 The service layer, via method validation

Constraints work on any CDI bean method, not just REST resources:

```
@ApplicationScoped
public class OrderService {

    @Transactional
    public @NotNull Order create(@Valid @NotNull CreateOrderRequest request) {
        ...
    }

    public List<Order> findByCustomer(@NotBlank @Email String email,
        @Min(0) int offset,
        @Min(1) @Max(100) int limit) {
        ...
    }
}
```

A violation throws `ConstraintViolationException`. Note the distinction the exception mapper must handle: a violation on a **parameter** is the caller's fault (400), while a violation on a **return value** is your own bug (500). We handle that in §8.7.

8.4.3 Entities

```
@Entity
public class Order extends PanacheEntity {

    @NotBlank @Email
    @Column(nullable = false)
    public String customerEmail;

    @NotNull @PositiveOrZero
```

```
public BigDecimal total;
}
```

Hibernate validates on `@PrePersist` and `@PreUpdate` by default. This is a **safety net, not your primary validation** — it fires at flush, deep inside a transaction, producing a `ConstraintViolationException` at a point where mapping it to a helpful HTTP response is awkward and the transaction is already poisoned.

```
# Control which lifecycle events validate
quarkus.hibernate-orm.validation.enabled=true
```

Treat entity constraints as an assertion that your service layer did its job. If one fires in production, a validation gap upstream let bad data through — that is a bug report, not a user error.

8.4.4 Configuration

Already covered in Chapter 4, and worth restating because it is the highest-value application of validation:

```
@ConfigMapping(prefix = "orders")
public interface OrdersConfig {
    @Min(1) @Max(500) int maxLineItems();
    @NotBlank @Pattern(regexp = "^https://.*") String callbackUrl();
    @NotEmpty List<@Email String> notifyEmails();
}
```

Failing at startup means a misconfigured pod never passes readiness, so a bad config halts the rollout rather than serving broken traffic.

8.4.5 REST clients and messaging

```
@RegisterRestClient(configKey = "payments-api")
public interface PaymentClient {
    @POST
    PaymentResult capture(@Valid CaptureRequest request);    // outbound
}
```

```
@Incoming("orders-in")
@Transactional
public void consume(@Valid OrderEvent event) {                // inbound messages
    orders.persist(toOrder(event));
}
```

Validating inbound messages matters more than validating HTTP, not less: a malformed message from another team's service will otherwise fail somewhere far from its cause, and a poison message can block a partition indefinitely.

8.5 Custom Constraints

8.5.1 A field-level constraint

```
package com.acme.orders.validation;

import jakarta.validation.*;
import java.lang.annotation.*;

@Documented
@Constraint(validatedBy = SkuValidator.class)
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.RECORD_COMPONENT,
        ElementType.TYPE_USE})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidSku {
    String message() default "{com.acme.orders.ValidSku.message}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};

    /** Restrict to a product family, e.g. "BOOK". */
    String family() default "";
}

public class SkuValidator implements ConstraintValidator<ValidSku, String> {

    private static final Pattern SKU = Pattern.compile("^[A-Z]{3}-\\d{4}-[A-Z0-9]{2}$");
    private String family;

    @Override
    public void initialize(ValidSku annotation) {
        this.family = annotation.family();
    }

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) return true;           // @NotNull is a separate concern

        if (!SKU.matcher(value).matches()) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                "must match AAA-9999-XX").addConstraintViolation();
            return false;
        }
        if (!family.isEmpty() && !value.startsWith(family)) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                "must belong to family " + family).addConstraintViolation();
            return false;
        }
        return true;
    }
}
```

Return `true` for `null`. A constraint validator should be concerned with one thing; nullability is `@NotNull`'s job. A validator that rejects `null` makes the field implicitly required, which is invisible at the declaration site.

8.5.2 A class-level (cross-field) constraint

```
@Documented
@Constraint(validatedBy = DateRangeValidator.class)
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidDateRange {
    String message() default "end date must be after start date";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```
public class DateRangeValidator
    implements ConstraintValidator<ValidDateRange, DateRangeRequest> {

    @Override
    public boolean isValid(DateRangeRequest value, ConstraintValidatorContext ctx) {
        if (value == null || value.start() == null || value.end() == null) {
            return true;    // field-level constraints handle the nulls
        }
        if (!value.end().isAfter(value.start())) {
            ctx.disableDefaultConstraintViolation();
            ctx.buildConstraintViolationWithTemplate("must be after start")
                .addPropertyNode("end")    // attach to the RIGHT field
                .addConstraintViolation();
            return false;
        }
        return true;
    }
}
```

```
@ValidDateRange
public record DateRangeRequest(
    @NotNull LocalDate start,
    @NotNull LocalDate end) { }
```

`addPropertyNode("end")` matters: it attaches the message to a specific field so the client can highlight the right input, rather than reporting a violation against the whole object.

8.5.3 A validator that injects a bean

Validators are CDI beans, so they can inject:

```
@ApplicationScoped
public class UniqueReferenceValidator
    implements ConstraintValidator<UniqueReference, String> {

    @Inject OrderRepository orders;
```

```

@Override
public boolean isValid(String reference, ConstraintValidatorContext ctx) {
    if (reference == null) return true;
    return orders.count("reference", reference) == 0;
}
}

```

Be careful with database-backed constraints. They issue a query on every validation, which is on the request path and potentially on an event loop. Worse, they are inherently racy: two concurrent requests both pass the check and both insert. A uniqueness *check* is a usability affordance; a unique *index* is the correctness mechanism. Implement both, and catch the constraint violation at the boundary as a 409.

8.6 Validation Groups

Groups let the same object be validated differently in different situations.

```

public interface OnCreate { }
public interface OnUpdate { }

public record OrderRequest(
    @Null(groups = OnCreate.class)           // must be absent on create
    @NotNull(groups = OnUpdate.class)        // must be present on update
    Long id,

    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    @Email(groups = {OnCreate.class, OnUpdate.class})
    String customerEmail,

    @NotEmpty(groups = OnCreate.class)
    @Valid List<LineItemRequest> items) { }

```

```

@POST
public RestResponse<OrderView> create(
    @ConvertGroup(to = OnCreate.class) @Valid OrderRequest request) { ... }

@PUT @Path("/{id}")
public RestResponse<OrderView> update(
    @RestPath long id,
    @ConvertGroup(to = OnUpdate.class) @Valid OrderRequest request) { ... }

```

8.6.1 Ordered groups

`@GroupSequence` runs groups in order and **stops at the first group with a violation**. This is how you avoid running an expensive check when a cheap one has already failed.

```

public interface Format { }
public interface Business { }

```

```
@GroupSequence({Format.class, Business.class})
public interface FullValidation { }
```

```
public record TransferRequest(
    @NotBlank(groups = Format.class)
    @Pattern(regexp = "[A-Z]{2}\\d{2}[A-Z0-9]{1,30}$", groups = Format.class)
    String iban,

    @Positive(groups = Format.class)
    @SufficientBalance(groups = Business.class)           // hits the database
    BigDecimal amount) { }
```

With a malformed IBAN, the `Format` group fails and `SufficientBalance` never runs — no wasted query.

Groups are powerful and easy to overuse. Three groups is usually a signal that you actually want two different request types. `CreateOrderRequest` and `UpdateOrderRequest` as separate records is often clearer than one record with `@ConvertGroup` at every call site, and it makes the OpenAPI document honest.

8.7 Error Responses

Validation failures are the most common error your API will return, so their shape matters more than any other.

```
package com.acme.orders.api.error;

import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import jakarta.validation.ElementKind;
import jakarta.validation.Path;
import org.jboss.resteasy.reactive.RestResponse;
import org.jboss.resteasy.reactive.server.ServerExceptionHandler;

public class ValidationExceptionHandler {

    @ServerExceptionHandler
    public RestResponse<ProblemDetail> handle(ConstraintViolationException e) {

        // A violation on a RETURN VALUE is our bug, not the caller's.
        boolean isReturnValue = e.getConstraintViolations().stream()
            .anyMatch(v -> lastKind(v.getPropertyPath()) == ElementKind.RETURN_VALUE);

        if (isReturnValue) {
            String ref = UUID.randomUUID().toString();
            Log.errorf(e, "Return-value constraint violated, ref=%s", ref);
            return RestResponse.status(Response.Status.INTERNAL_SERVER_ERROR,
                ProblemDetail.of("internal-error", "An unexpected error occurred",
                    500, "Reference: " + ref));
        }
    }
}
```



```

    Map<String, List<String>> fields = new LinkedHashMap<>();
    for (ConstraintViolation<?> v : e.getConstraintViolations()) {
        fields.computeIfAbsent(fieldName(v.getPropertyPath()), k -> new ArrayList<>())
            .add(v.getMessage());
    }

    return RestResponse.status(Response.Status.BAD_REQUEST,
        ProblemDetail.validation("Request validation failed", fields));
}

/**
 * Turn "create.request.items[0].sku" into "items[0].sku".
 * The method and parameter prefix is an implementation detail the client
 * cannot act on, and leaking it couples your API to your method names.
 */
private static String fieldName(Path path) {
    List<Path.Node> nodes = new ArrayList<>();
    path.forEach(nodes::add);
    int start = 0;
    for (int i = 0; i < nodes.size(); i++) {
        if (nodes.get(i).getKind() == ElementKind.PARAMETER) {
            start = i + 1;
            break;
        }
    }
    StringBuilder sb = new StringBuilder();
    for (int i = start; i < nodes.size(); i++) {
        Path.Node node = nodes.get(i);
        if (node.getName() == null) continue;
        if (!sb.isEmpty()) sb.append('.');
        sb.append(node.getName());
        if (node.getIndex() != null) sb.append('[').append(node.getIndex()).append(']');
    }
    return sb.isEmpty() ? "request" : sb.toString();
}

private static ElementKind lastKind(Path path) {
    ElementKind kind = null;
    for (Path.Node node : path) kind = node.getKind();
    return kind;
}
}

```

The resulting body, which is RFC 7807 with a `fields` extension:

```

{
  "type": "https://errors.acme.com/orders/validation-failed",
  "title": "Request validation failed",
  "status": 400,
  "fields": {
    "customerEmail": ["must be a well-formed email address"],
    "items": ["must not be empty"],

```

```
"items[0].quantity": ["must be greater than 0"]
}
}
```

8.7.1 Why the path cleanup matters

The raw path from Bean Validation for a REST parameter looks like:

```
create.arg0.items[0].sku
```

That leaks your method name and, without `-parameters`, `arg0`. A client cannot map it to a form field. Cleaning it to `items[0].sku` makes the response programmatically useful, which is the entire point of returning field-level detail.

8.7.2 400 versus 422

Both are defensible; be consistent.

- **400 Bad Request** — the request is malformed: wrong types, missing required fields, unparseable JSON.
- **422 Unprocessable Content** — the request is well-formed but semantically wrong: a date range that ends before it starts, an order for a discontinued product.

A workable convention: Bean Validation failures are 400, service-layer semantic failures are 422, and state conflicts are 409. Document it once and apply it everywhere.

8.8 Messages and Internationalisation

```
public record CreateOrderRequest(
    @NotBlank(message = "Customer email is required")
    @Email(message = "{com.acme.orders.email.invalid}") // resource bundle key
    String customerEmail,

    @Size(max = 500, message = "Note cannot exceed {max} characters")
    String note,

    @Min(value = 1, message = "Quantity must be at least {value}")
    int quantity) { }
```

```
# src/main/resources/ValidationMessages.properties
com.acme.orders.email.invalid=Please provide a valid email address
com.acme.orders.sku.invalid=SKU must look like AAA-9999-XX
```

```
# src/main/resources/ValidationMessages_de.properties
com.acme.orders.email.invalid=Bitte geben Sie eine gültige E-Mail-Adresse an
```

```
quarkus.hibernate-validator.locale=en-GB
quarkus.hibernate-validator.locales=en-GB,de-DE,fr-FR
quarkus.hibernate-validator.fail-fast=false # true = stop at the first violation
```

With `Accept-Language` sent by the client, Quarkus resolves the message from the matching bundle automatically.

Do not translate messages for a machine-to-machine API. Localised strings in a JSON error body are for humans; a service-to-service client needs a stable `type` URI and a stable field path. Put the human-readable text in the client, keyed off the machine-readable code.

8.9 Programmatic Validation

```
@ApplicationScoped
public class BulkImporter {

    @Inject Validator validator;

    public ImportReport importRows(List<RowDto> rows) {
        List<RowError> errors = new ArrayList<>();
        List<RowDto> accepted = new ArrayList<>();

        for (int i = 0; i < rows.size(); i++) {
            Set<ConstraintViolation<RowDto>> violations = validator.validate(rows.get(i));
            if (violations.isEmpty()) {
                accepted.add(rows.get(i));
            } else {
                errors.add(new RowError(i, violations.stream()
                    .map(v -> v.getPropertyPath() + ": " + v.getMessage())
                    .toList()));
            }
        }
        service.saveAll(accepted);
        return new ImportReport(accepted.size(), errors);
    }
}
```

This is the right pattern for bulk operations: annotation-driven validation aborts on the first failure, whereas a CSV import needs to report *every* bad row so the user can fix them in one pass.

```
// Validate a single property, or a value before assigning it
validator.validateProperty(order, "customerEmail");
validator.validateValue(CreateOrderRequest.class, "customerEmail", "bad-input");
validator.validate(request, OnCreate.class);    // with a group
```

8.10 Testing

```
@QuarkusTest
class OrderValidationTest {

    @Inject Validator validator;

    @Test
    void rejectsBlankEmail() {
        var request = new CreateOrderRequest("", List.of(validItem()), null);
    }
}
```

```

        Set<ConstraintViolation<CreateOrderRequest>> violations =
            validator.validate(request);

        assertThat(violations)
            .extracting(v -> v.getPropertyPath().toString())
            .contains("customerEmail");
    }

    @Test
    void cascadesIntoLineItems() {
        var request = new CreateOrderRequest(
            "a@acme.com",
            List.of(new LineItemRequest("SKU-1", 0)),    // quantity must be positive
            null);

        assertThat(validator.validate(request))
            .extracting(v -> v.getPropertyPath().toString())
            .contains("items[0].quantity");
    }

    @Test
    void acceptsValidRequest() {
        assertThat(validator.validate(validRequest())).isEmpty();
    }
}

```

End-to-end, asserting on the response shape rather than the violation set:

```

@QuarkusTest
class OrderResourceValidationTest {

    @Test
    void returnsFieldLevelErrors() {
        given()
            .contentType("application/json")
            .body("""
                {"customerEmail": "not-an-email", "items": []}
                """)
        .when()
            .post("/orders")
        .then()
            .statusCode(400)
            .body("type", endsWith("validation-failed"))
            .body("fields.customerEmail", hasItem("must be a well-formed email address"))
            .body("fields.items", hasItem("must not be empty"));
    }
}

```

Test the negative cases. A test suite that only asserts the happy path leaves your most-returned response — the 400 — completely unverified. The field paths in particular are easy to break with a refactor and are exactly what clients depend on.

8.11 Best Practices

1. **Validate syntax once, at the boundary.** Do not repeat the same rule in three layers.
2. **@Valid on every nested object and collection** you want cascaded — it is never implicit.
3. **Enforce invariants in constructors**, especially with records' compact constructors.
4. **Contextual rules go in the service**, where the context exists.
5. **Entity constraints are a safety net**, not the primary mechanism.
6. **Validate configuration** with `@ConfigMapping` so bad config fails the boot.
7. **Validate inbound messages**, not just HTTP bodies.
8. **Clean the property path** before returning it, so clients get `items[0].sku`, not `create.arg0.items[0].sku`.
9. **Return all violations at once**, not the first — one round trip to fix a form.
10. **Custom validators return true for null.** Nullability is `@NotNull`'s concern.
11. **Back every uniqueness check with a unique index.** The check is usability; the index is correctness.
12. **Distinguish parameter violations (400) from return-value violations (500).** The latter is your bug.

8.12 Common Mistakes

1. **Forgetting @Valid on the parameter.** The constraints are inert and the code looks right.
2. **Forgetting @Valid on a nested collection.** `@NotEmpty List<LineItemRequest>` validates the list but never its elements.
3. **Relying on a database-backed uniqueness validator alone.** Racy under concurrency; you need the index.
4. **Validating in the entity only.** The failure surfaces at flush, inside a transaction, far from where you can produce a good response.
5. **Leaking method and parameter names in the error body.** `create.arg0.customerEmail` is not actionable.
6. **fail-fast=true on a form submission.** The user fixes one field, resubmits, and finds another — repeatedly.
7. **Custom validators rejecting null.** Makes the field implicitly required, invisibly.
8. **Doing I/O in a validator on a non-blocking endpoint.** Blocks an event loop.
9. **Translating error codes for machine clients.** They need stable identifiers, not prose.
10. **Group proliferation.** More than two or three groups usually means you want separate request types.
11. **Testing only the happy path.** The 400 response is your most frequent response and often the least tested.

8.13 Interview Questions

Junior

Q1. What is Bean Validation? The Jakarta standard for declarative constraints on objects — `@NotNull`, `@Size`, `@Email` — implemented in Quarkus by Hibernate Validator.

Q2. How do you validate a REST request body? Annotate the DTO's fields with constraints and put `@Valid` on the resource method parameter.

Q3. What is the difference between `@NotNull`, `@NotEmpty` and `@NotBlank`? `@NotNull` only rejects null. `@NotEmpty` also requires size greater than zero (strings, collections, maps, arrays). `@NotBlank` is string-only and additionally requires at least one non-whitespace character.

Q4. How do you write a custom constraint? Define an annotation meta-annotated with `@Constraint(validatedBy = ...)` and implement `ConstraintValidator<A, T>`.

Q5. What happens when validation fails? A `ConstraintViolationException` is thrown; an exception mapper turns it into a 400 with per-field detail.

Mid-level

Q6. Why does nested validation need `@Valid`? Cascading is opt-in at every level, by design — otherwise validating one object could traverse an arbitrarily deep graph, including lazy JPA associations. You must mark each boundary you want crossed.

Q7. Where should validation live in a layered architecture? Syntactic validation at the API boundary with Bean Validation; contextual or semantic rules in the service, where the necessary context exists; invariants in the domain object's constructor. Duplicating the same rule across layers causes drift and inconsistent messages.

Q8. What are validation groups for? Applying different rules to the same type in different situations — for example, an id that must be absent on create and present on update. `@GroupSequence` orders groups and stops at the first failing one, so an expensive check can be skipped when a cheap one already failed.

Q9. Should you validate entities? As a safety net. It fires on `@PrePersist` / `@PreUpdate`, which is deep inside a transaction and awkward to map to a useful response. A violation there means the service layer let bad data through — a bug, not a user error.

Q10. How do you produce a good validation error response? RFC 7807 problem details with a stable `type` URI, plus a `fields` map keyed by cleaned property paths (`items[0].sku`, not `create.arg0.items[0].sku`), containing every violation rather than just the first.

Q11. How do you validate configuration? Bean Validation annotations on a `@ConfigMapping` interface. It runs at startup, so a bad value prevents the application from booting — which in Kubernetes means the readiness probe never passes and the rollout halts.

Senior

Q12. Design validation for a public API used by external partners. The error contract matters as much as the validation. I would return RFC 7807 with a stable `type` URI per error class, a machine-readable code per field violation rather than only prose, and cleaned property paths that map onto the request body's JSON structure so a client can highlight fields directly. Every violation is returned at once, not the first, because partners integrate against it in batch. The `type` URIs are versioned and documented alongside the OpenAPI spec, and the spec declares the error

schema per status code so client generators produce typed errors. I would draw the status codes explicitly — 400 for malformed, 422 for well-formed but semantically invalid, 409 for state conflicts — and document them, because inconsistency here generates more support tickets than any other API design choice. Human-readable text stays in English and is not localised, since partners build their own UI. Finally, I would contract-test the error responses in CI, because the field paths are exactly what partners depend on and exactly what a refactor silently breaks.

Q13. A uniqueness constraint is implemented as a custom validator that queries the database. Critique it. It is racy and it is on the request path. Racy because two concurrent requests both run the query, both see no existing row, and both insert — the check-then-act has no atomicity. On the request path because it issues a query per validation, and if the endpoint is non-blocking that query runs on an event loop. The correct design uses a unique index as the actual correctness mechanism, catches the resulting constraint violation at the boundary and maps it to a 409. The validator can stay as a usability affordance — it gives a nicer message in the common non-concurrent case — but it must not be the thing you rely on, and it should be documented as such so nobody removes the index thinking the validator covers it. This is a specific instance of a general rule: validation is advisory, constraints are authoritative.

Q14. How do you handle validation for a bulk import of 50,000 rows? Annotation-driven validation is the wrong tool because it aborts on the first violation, and a user importing a spreadsheet needs every bad row reported in one pass. I would inject `Validator` and validate each row programmatically, accumulating errors with row numbers, then persist the accepted rows in chunked transactions so a single commit is not holding a connection for the whole import. The response is a report — accepted count plus a list of row errors — not a 400, because a partial success is the honest outcome. For a file that large the operation is asynchronous: accept the upload, return 202 with a job id, process in the background, and expose the report at a status endpoint. I would also validate the file's shape before starting — column headers, row count against a limit — so a wrong file fails in a second rather than after ten minutes. And the import needs to be idempotent by an import id, because the client will retry the upload when their connection drops.

Principal

Q15. Your platform has 40 services with inconsistent validation error formats, and client teams are complaining. How do you fix it? The technical fix is easy and the adoption problem is the real one, so I would optimise for adoption. I would define one error contract — RFC 7807, stable `type` namespace, `fields` map with cleaned paths, documented status-code semantics — and ship it as an internal Quarkus extension that registers the exception mappers automatically, so a service gets correct behaviour by adding a dependency rather than by copying a class. That matters: a convention that lives in a wiki page decays, and one that lives in a shared library drifts as teams patch it locally, whereas an extension can be versioned and its adoption measured. I would then instrument rather than mandate: a contract test in the shared test library that asserts the error shape, plus a platform dashboard showing which services emit conforming errors, so the gap is visible without anyone policing it. Migration would start with the services the complaining clients actually call, because that converts the effort into visible relief quickly and buys goodwill for the rest. I would explicitly *not* try to unify the human-readable messages or the field names across domains — that is where standardisation stops paying and starts producing lowest-common-denominator APIs. The test I apply throughout is the same one from Chapter 2: standardise what costs people outside the owning team, and leave the rest alone.

8.14 Cheat Sheet

Common constraints

```
@NotNull @NotEmpty @NotBlank
@Size(min=, max=) @Length(min=, max=)
@Min @Max @Positive @PositiveOrZero @Negative
@DecimalMin @DecimalMax @Digits(integer=, fraction=)
@email @Pattern(regexp=) @URL @UUID
@Past @PastOrPresent @Future @FutureOrPresent
@AssertTrue @AssertFalse
@Valid // cascade - never implicit
```

Containers

```
@NotEmpty List<@NotBlank String> tags
Map<@NotBlank String, @Positive Integer> quantities
Optional<@Email String> contact
List<@Valid LineItemRequest> items
```

Custom constraint

```
@Constraint(validatedBy = SkuValidator.class)
@Target({FIELD, PARAMETER, RECORD_COMPONENT}) @Retention(RUNTIME)
public @interface ValidSku {
    String message() default "invalid SKU";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

class SkuValidator implements ConstraintValidator<ValidSku, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) {
        return v == null || PATTERN.matcher(v).matches(); // null -> true
    }
}
```

Groups

```
@Null(groups = OnCreate.class) @NotNull(groups = OnUpdate.class) Long id;

@GroupSequence({Format.class, Business.class}) interface Full { }

@ConvertGroup(to = OnCreate.class) @Valid OrderRequest request
```

Programmatic

```
@Inject Validator validator;
validator.validate(obj);
validator.validate(obj, OnCreate.class);
validator.validateProperty(obj, "email");
validator.validateValue(Dto.class, "email", "value");
```

Configuration


```
quarkus.hibernate-validator.fail-fast=false
quarkus.hibernate-validator.locale=en-GB
quarkus.hibernate-orm.validation.enabled=true
```

Status codes

```
400 malformed request (Bean Validation)
422 well-formed but semantically invalid
409 conflicts with current state
```

8.15 Summary

Validation is three different jobs wearing one name. Bean Validation answers “is this well-formed?” and belongs at the API boundary. Contextual questions — does this customer exist, is there stock — belong in the service, because that is where the context is. Invariants belong in constructors, so an illegal domain object cannot exist. Validating the same rule in all three places is not defence in depth; it is duplication that drifts.

Two mechanical details cause most of the bugs: `@Valid` never cascades implicitly, so every nested object and collection needs it explicitly; and constraint validators should treat `null` as valid, leaving nullability to `@NotNull`.

The part that is genuinely engineering rather than annotation is the **error response**. Validation failures are the most frequent error your API returns, so the body should carry every violation at once, keyed by a cleaned property path that maps onto the client’s own data structure, under a stable `type` URI that a client can switch on. And uniqueness is worth stating separately: a validator that queries the database is a usability affordance, while the unique index is the correctness mechanism — you need both, and only one of them is authoritative.

Chapter 9 turns to **security** — authentication with JWT and OIDC, Keycloak integration, authorisation models, and the security failures that are specific to a build-time framework.

8.16 References

- Validation with Hibernate Validator — <https://quarkus.io/guides/validation>
- Jakarta Bean Validation 3.0 — <https://jakarta.ee/specifications/bean-validation/3.0/>
- Hibernate Validator reference — https://docs.jboss.org/hibernate/validator/8.0/reference/en-US/html_single/
- RFC 7807, Problem Details for HTTP APIs — <https://www.rfc-editor.org/rfc/rfc7807>
- RFC 9110, HTTP Semantics (status codes) — <https://www.rfc-editor.org/rfc/rfc9110>

Security, JWT and OIDC

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-oidc` · `quarkus-smallrye-jwt` · Keycloak 26 · Jakarta Security

Learning Objectives

By the end of this chapter you will be able to:

1. Describe Quarkus's security architecture: authentication mechanisms, identity providers and `SecurityIdentity`.
2. Choose correctly between **JWT verification** and **full OIDC**, and explain the difference precisely.
3. Configure `quarkus-oidc` in both `service` (bearer token) and `web-app` (authorization code) modes.
4. Implement authorisation with annotations, configuration-driven policies and programmatic checks — and know when each is right.
5. Explain why **role-based access control breaks down** and what to use instead.
6. Enforce **ownership and row-level** authorisation, which annotations cannot express.
7. Propagate identity across service boundaries without creating a confused-deputy vulnerability.
8. Configure TLS, mTLS and security headers correctly.
9. Test secured endpoints, including negative cases, with `@TestSecurity` and a real Keycloak via Dev Services.
10. Answer 30 interview questions on authentication, authorisation and API security.

9.1 The Architecture

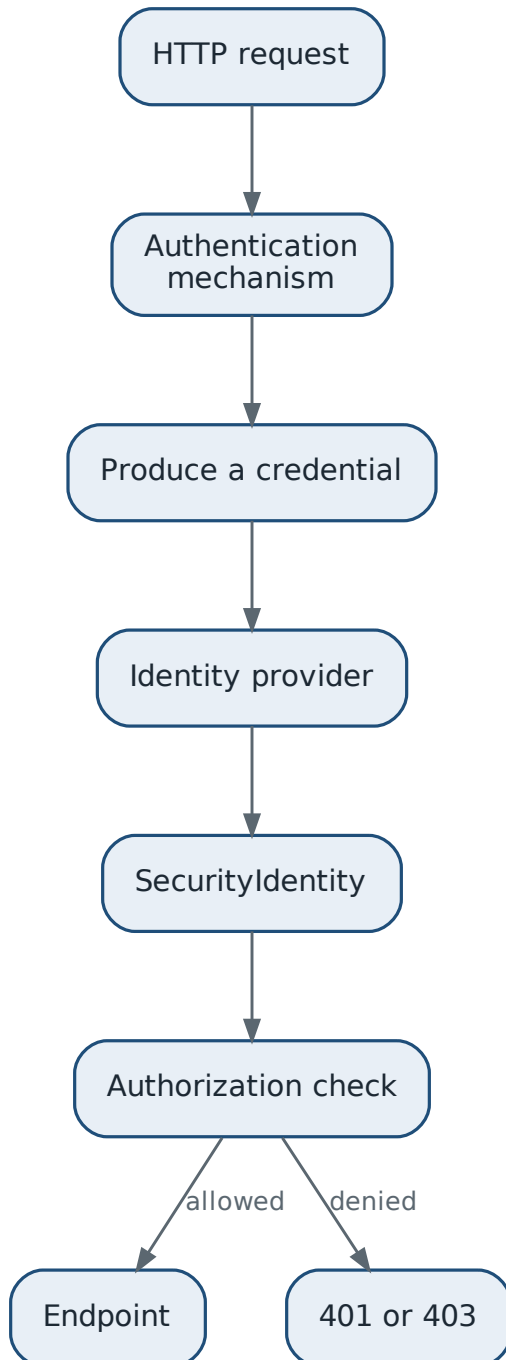


Figure 9.1 — Quarkus security pipeline. Mechanisms authenticate; providers build the identity; authorisation happens afterwards.

Three collaborating pieces:

Authentication mechanism — extracts a credential from the request. `HttpAuthenticationMechanism` implementations exist for bearer tokens, basic auth, form login, mTLS certificates and the OIDC authorization-code flow.

Identity provider — turns a credential into a `SecurityIdentity`. `IdentityProvider<T>` implementations verify a JWT signature, check a password against a database, or exchange a code at the token endpoint.

SecurityIdentity — the authenticated principal plus its roles and attributes. It is `@RequestScoped` and injectable everywhere.

```
@ApplicationScoped
public class OrderService {

    @Inject SecurityIdentity identity;

    public void audit(String action) {
        String user = identity.getPrincipal().getName();
        boolean isAdmin = identity.hasRole("admin");
        JsonWebToken jwt = identity.getAttribute("quarkus.identity.credential");
        ...
    }
}
```

As with everything else in Quarkus, the security metadata — which endpoints have which annotations, which policies apply to which paths — is resolved during augmentation, so there is no reflective annotation lookup on the request path.

9.2 JWT vs OIDC — Getting This Right

These are conflated constantly, including by people who have shipped both.

A JWT is a token format. A signed (and optionally encrypted) JSON payload with claims. `quarkus-smallrye-jwt` *verifies* a JWT: it checks the signature against a public key, validates `iss`, `aud`, `exp` and `nbf`, and maps claims to roles. That is all it does.

OIDC is a protocol. It defines how a client obtains a token in the first place: discovery, the authorization code flow, refresh tokens, the userinfo endpoint, logout, and key rotation via JWKS.

	<code>quarkus-smallrye-jwt</code>	<code>quarkus-oidc</code>
What it does	Verifies a token you already have	Obtains and verifies tokens
Talks to the IdP	No (static public key)	Yes (discovery, JWKS, token endpoint)
Key rotation	Manual	Automatic via JWKS
Token introspection	No	Yes (for opaque tokens)

	quarkus-smallrye-jwt	quarkus-oidc
Login flows	No	Authorization code, hybrid
Refresh tokens	No	Yes
Logout	No	Yes, including back-channel
Right for	Internal service-to-service with a fixed key	Anything involving an identity provider

Use **quarkus-oidc** unless you have a specific reason not to. The usual reason to prefer **smallrye-jwt** is an internal service mesh where tokens are minted by an internal issuer whose key never rotates, and where you want no network dependency on an IdP at startup. Everywhere else, OIDC's automatic JWKS refresh alone justifies it — a manually-managed public key is an outage waiting for the next key rotation.

9.3 OIDC in Service Mode (Bearer Tokens)

This is the configuration for an API that receives an **Authorization: Bearer** header. It is what almost every microservice needs.

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-oidc</artifactId>
</dependency>
```

```
quarkus.oidc.auth-server-url=https://keycloak.acme.com/realms/acme
quarkus.oidc.client-id=orders-service
quarkus.oidc.application-type=service           # bearer token, no login redirect
```

```
# Validate the audience - see the warning below
quarkus.oidc.token.audience=orders-service
```

```
# Clock skew tolerance between your pod and the IdP
quarkus.oidc.token.lifespan-grace=10
```

```
# Where roles live in the token
quarkus.oidc.roles.source=accesstoken
quarkus.oidc.roles.role-claim-path=realm_access/roles
```

```
# JWKS cache and refresh
quarkus.oidc.token.forced-jwk-refresh-interval=10M
quarkus.oidc.connection-timeout=5s
quarkus.oidc.connection-retry-count=3
```

```
# Do not fail startup if the IdP is briefly unreachable
quarkus.oidc.tenant-enabled=true
```

```
@Path("/orders")
@Authenticated                                     // every method requires a valid token
public class OrderResource {
```

```

@Inject JwtToken jwt;
@Inject SecurityIdentity identity;

@GET
@RolesAllowed("order-reader")
public List<OrderSummary> list() {
    String subject = jwt.getSubject();
    String tenant = jwt.getClaim("tenant_id");
    return service.listFor(tenant);
}

@POST
@RolesAllowed({"order-writer", "admin"})
public RestResponse<OrderView> create(@Valid CreateOrderRequest request) { ... }

@DELETE @Path("/{id}")
@RolesAllowed("admin")
public void delete(@RestPath long id) { ... }

@GET @Path("/public/health-summary")
@PermitAll // explicitly public
public HealthSummary summary() { ... }
}

```

9.3.1 Always validate the audience

```
quarkus.oidc.token.audience=orders-service
```

Without this, **any token your IdP issued for any client is accepted**. A token minted for a low-privilege frontend application, or for a third-party integration, will authenticate against your service. This is one of the most common real-world OIDC misconfigurations, it produces no error, and it is invisible in testing because your own tokens work fine.

The same applies to the issuer, which Quarkus validates against `auth-server-url` by default — do not disable it.

9.3.2 Reading claims

```

@Path("/profile")
@Authenticated
public class ProfileResource {

    @Inject JwtToken jwt;

    // Inject a claim directly
    @Inject @Claim("tenant_id") String tenantId;
    @Inject @Claim("email") Optional<String> email;
    @Inject @Claim(standard = Claims.groups) Set<String> groups;

    @GET
    public ProfileView me() {

```

```

        return new ProfileView(
            jwt.getSubject(),
            jwt.getName(),
            jwt.<String>getClaim("preferred_username"),
            jwt.getGroups(),
            Instant.ofEpochSecond(jwt.getExpirationTime()));
    }
}

```

`@Claim` injection requires a request-scoped context, so it works in resources and request-scoped beans but not in `@ApplicationScoped` beans accessed outside a request. Inject `JsonWebToken` (which is itself a proxy over the request-scoped token) instead.

9.3.3 Mapping roles

Where roles live depends on your IdP's configuration. Keycloak's defaults:

```

# Realm roles: {"realm_access": {"roles": ["admin"]}}
quarkus.oidc.roles.role-claim-path=realm_access/roles

# Client roles: {"resource_access": {"orders-service": {"roles": ["order-writer"]}}}
quarkus.oidc.roles.role-claim-path=resource_access/orders-service/roles

# A custom flat claim
quarkus.oidc.roles.role-claim-path=permissions

# Multiple sources
quarkus.oidc.roles.role-claim-path=realm_access/roles,resource_access/orders-service/roles

# Space-delimited scope string -> roles
quarkus.oidc.roles.role-claim-separator=" "

```

For anything more complex, augment the identity in code:

```

@ApplicationScoped
public class RoleAugmentor implements SecurityIdentityAugmentor {

    @Inject EntitlementService entitlements;

    @Override
    public Uni<SecurityIdentity> augment(SecurityIdentity identity,
                                         AuthenticationRequestContext context) {
        if (identity.isAnonymous()) {
            return Uni.createFrom().item(identity);
        }
        // Blocking lookup must be dispatched off the event loop
        return context.runBlocking(() -> {
            var builder = QuarkusSecurityIdentity.builder(identity);
            String subject = identity.getPrincipal().getName();
            entitlements.rolesFor(subject).forEach(builder::addRole);
            builder.addAttribute("tenant", entitlements.tenantFor(subject));
            return builder.build();
        });
    }
}

```

```
@Override
public int priority() { return 10; }
}
```

context.runBlocking is not optional here. The augmentor runs on the event loop for a non-blocking endpoint; a database lookup inside it blocks an event loop thread on **every authenticated request**, which is about the worst possible place to put blocking I/O. Cache the result aggressively as well — this code runs on every request, not once per session.

9.4 OIDC in Web-App Mode (Authorization Code Flow)

For a server-rendered application or a BFF that logs users in itself.

```
quarkus.oidc.auth-server-url=https://keycloak.acme.com/realms/acme
quarkus.oidc.client-id=orders-ui
quarkus.oidc.credentials.secret=${OIDC_CLIENT_SECRET}
quarkus.oidc.application-type=web-app

quarkus.oidc.authentication.scopes=openid,profile,email
quarkus.oidc.authentication.redirect-path=/callback
quarkus.oidc.authentication.restore-path-after-redirect=true
quarkus.oidc.authentication.cookie-path=/
quarkus.oidc.authentication.cookie-same-site=lax

# Session cookie hardening
quarkus.oidc.authentication.cookie-force-secure=true
quarkus.oidc.token-state-manager.split-tokens=true    # smaller cookies
quarkus.oidc.token-state-manager.encryption-required=true

# Silent refresh before expiry
quarkus.oidc.authentication.session-age-extension=30M
quarkus.oidc.token.refresh-expired=true
quarkus.oidc.token.refresh-token-time-skew=30S

# Logout
quarkus.oidc.logout.path=/logout
quarkus.oidc.logout.post-logout-path=/
quarkus.oidc.logout.backchannel.path=/back-channel-logout
```

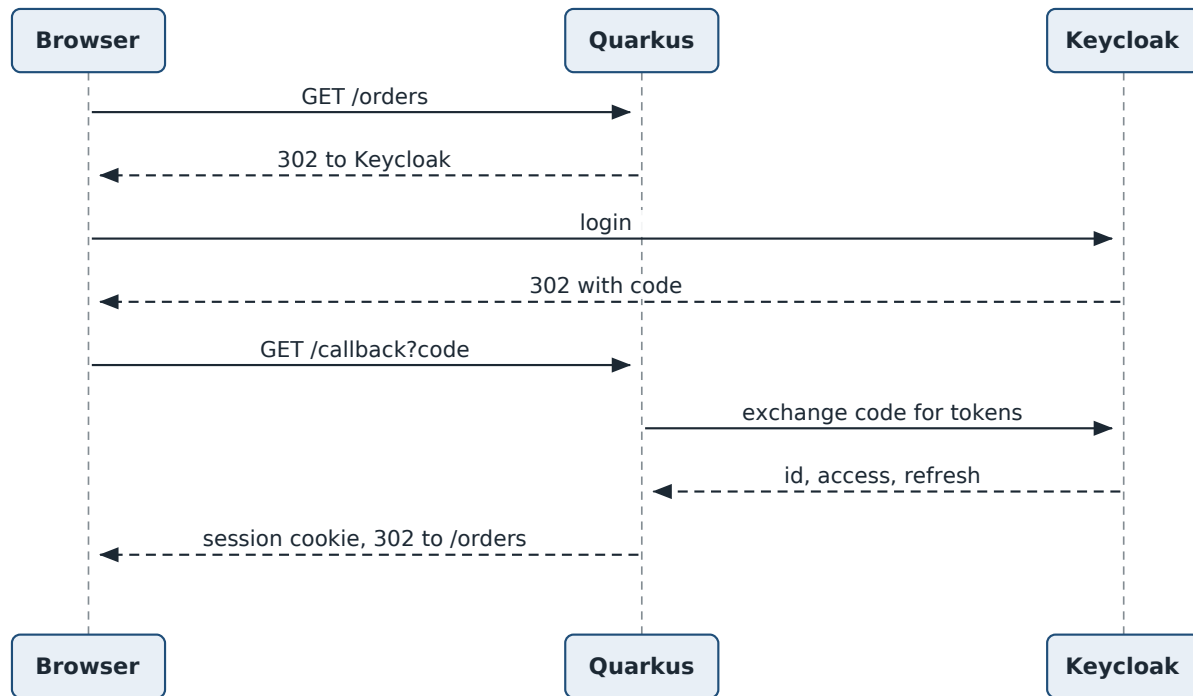



Figure 9.2 — Authorization code flow. Tokens stay server-side; the browser only holds a session cookie.

9.4.1 The important property of this flow

Tokens never reach the browser. Quarkus stores them in an encrypted cookie (or a server-side store) and the browser holds only a session reference. That eliminates the entire class of vulnerability where an XSS bug exfiltrates an access token from `localStorage`.

If you are building a single-page application, the modern recommendation is the **backend-for-frontend** pattern: the SPA talks to a Quarkus BFF in `web-app` mode over a session cookie, and the BFF holds the tokens and calls downstream services. Storing tokens in browser storage is no longer considered acceptable practice.

9.5 SmallRye JWT

When you only need to verify tokens minted elsewhere:

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-jwt</artifactId>
</dependency>
```

```
mp.jwt.verify.publickey.location=https://auth.internal/.well-known/jwks.json
mp.jwt.verify.issuer=https://auth.internal
mp.jwt.verify.audiences=orders-service
mp.jwt.verify.publickey.algorithm=RS256
```

```

smallrye.jwt.token.header=Authorization
smallrye.jwt.token.schemes=Bearer
smallrye.jwt.path.groups=realm_access/roles
smallrye.jwt.claims.groups=default-role          # fallback if the claim is absent
smallrye.jwt.verify.aud=orders-service

```

Issuing tokens — for tests, or for an internal issuer:

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-jwt-build</artifactId>
</dependency>

```

```

String token = Jwt.issuer("https://auth.internal")
    .upn("service-account-orders")
    .groups(Set.of("order-writer"))
    .claim("tenant_id", "acme")
    .expiresIn(Duration.ofMinutes(5))
    .sign();                                     // uses smallrye.jwt.sign.key.location

```

Never mint tokens for humans in your own service. Issuing tokens means owning key management, rotation, revocation and the token lifecycle — all of which an identity provider already does correctly. Minting short-lived internal service tokens is a legitimate narrower case; minting user tokens is re-implementing an IdP badly.

9.6 Authorisation

```

@Path("/orders")
public class OrderResource {

    @GET @PermitAll          public List<OrderSummary> publicList() { ... }
    @GET @Authenticated      public List<OrderSummary> myOrders()  { ... }
    @POST @RolesAllowed("order-writer") public void create() { ... }
    @PUT  @RolesAllowed({"order-writer", "admin"}) public void update() { ... }
    @DELETE @DenyAll         public void neverCallable() { ... }
}

```

Annotations also work on any CDI bean method, so you can enforce at the service layer:

```

@ApplicationScoped
public class AdminService {

    @RolesAllowed("admin")
    public void purgeAll() { ... }
}

```

9.6.2 Deny by default

The single most valuable security configuration in Quarkus:

```
quarkus.security.jaxrs.deny-unannotated-endpoints=true
```

With this set, **an endpoint with no security annotation is denied**. Without it, forgetting `@RolesAllowed` on a new endpoint silently makes it public — and nothing in code review reliably catches an *absent* annotation.

```
# Alternative: default to authenticated rather than denied
quarkus.security.jaxrs.default-roles-allowed=**      # ** means any authenticated user
```

Turn this on at the start of a project. Retrofitting it later means auditing every endpoint at once, which is exactly the kind of task that gets deferred. If an interviewer asks how you prevent accidentally-public endpoints, this is the answer.

9.6.3 Configuration-driven policies

Path-based rules, useful for coarse decisions and for rules that must be changed without a rebuild:

```
quarkus.http.auth.permission.public.paths=/q/health/*,/q/openapi,/public/*
quarkus.http.auth.permission.public.policy=permit
```

```
quarkus.http.auth.permission.metrics.paths=/q/metrics
quarkus.http.auth.permission.metrics.policy=role-metrics
quarkus.http.auth.policy.role-metrics.roles-allowed=monitoring
```

```
quarkus.http.auth.permission.admin.paths=/admin/*
quarkus.http.auth.permission.admin.policy=role-admin
quarkus.http.auth.policy.role-admin.roles-allowed=admin
```

```
quarkus.http.auth.permission.authenticated.paths=/*
quarkus.http.auth.permission.authenticated.policy=authenticated
```

```
# Restrict by method too
quarkus.http.auth.permission.write.paths=/orders/*
quarkus.http.auth.permission.write.methods=POST,PUT,PATCH,DELETE
quarkus.http.auth.permission.write.policy=role-writer
quarkus.http.auth.policy.role-writer.roles-allowed=order-writer
```

Path policies and annotations both apply; the request must satisfy both. Prefer annotations for endpoint-specific rules — they live next to the code they protect — and path policies for cross-cutting concerns like locking down `/q/metrics`.

9.6.4 Programmatic checks

Annotations cannot express “the caller owns this order”. That check is data-dependent and must be in code.

```
@ApplicationScoped
public class OrderAuthorizationService {

    @Inject SecurityIdentity identity;
```

```

public void requireAccess(Order order) {
    if (identity.hasRole("admin")) return;

    String subject = identity.getPrincipal().getName();
    if (!order.customerId.equals(subject)) {
        // 404, not 403 - see the note below
        throw new NotFoundException();
    }
}
}

```

Return 404, not 403, for a resource the caller may not see. A 403 confirms the resource exists, which leaks information: an attacker can enumerate valid order ids by observing 403 versus 404. Use 403 only when the caller is allowed to know the resource exists but not to perform the operation.

9.7 Beyond Roles

9.7.1 Why RBAC breaks down

Role-based access control works until the rules acquire context. The symptom is role names that encode conditions:

```

order-reader
order-reader-eu
order-reader-eu-tier2
order-reader-eu-tier2-readonly-except-pii

```

At that point the role name is a serialised policy and the model has failed. Roles cannot express:

- **Ownership** — “the customer who placed it”.
- **Relationship** — “the account manager for this customer”.
- **Attributes** — “orders under £1000”, “during business hours”, “from a corporate IP”.
- **Delegation** — “acting on behalf of”.

9.7.2 Permission-based access control

Quarkus supports Jakarta’s `@PermissionsAllowed`, which is a meaningful improvement over roles:

```

@Path("/orders")
public class OrderResource {

    @GET
    @PermissionsAllowed("orders:read")
    public List<OrderSummary> list() { ... }

    @POST
    @PermissionsAllowed("orders:write")
    public RestResponse<OrderView> create(@Valid CreateOrderRequest r) { ... }
}

```

```

@DELETE @Path("/{id}")
@PermissionsAllowed({"orders:delete", "orders:admin"}) // OR by default
public void delete(@RestPath long id) { ... }

@PUT @Path("/{id}/refund")
@PermissionsAllowed(value = {"orders:write", "payments:refund"},
    inclusive = true) // AND
public void refund(@RestPath long id) { ... }
}

```

```

@ApplicationScoped
public class PermissionAugmentor implements SecurityIdentityAugmentor {

    @Override
    public Uni<SecurityIdentity> augment(SecurityIdentity identity,
        AuthenticationRequestContext context) {
        if (identity.isAnonymous()) return Uni.createFrom().item(identity);

        var builder = QuarkusSecurityIdentity.builder(identity);
        for (String role : identity.getRoles()) {
            permissionsForRole(role).forEach(p ->
                builder.addPermission(new StringPermission(p)));
        }
        return Uni.createFrom().item(builder.build());
    }
}

```

Roles then become a *packaging* mechanism for permissions, and code checks permissions. Adding a capability means editing a role-to-permission mapping, not touching every endpoint.

9.7.3 Custom permissions with parameters

```

public class OrderPermission extends Permission {

    private final String action;
    private final Set<String> tenants;

    public OrderPermission(String name, String... actions) {
        super(name);
        this.action = actions.length > 0 ? actions[0] : "read";
        this.tenants = Set.of();
    }

    @Override
    public boolean implies(Permission required) {
        if (!(required instanceof OrderPermission other)) return false;
        return getName().equals(other.getName())
            && (action.equals("admin") || action.equals(other.action));
    }

    @Override public String getActions() { return action; }
    @Override public boolean equals(Object o) { ... }
}

```

```
@Override public int hashCode() { ... }
}
```

```
@PermissionsAllowed(value = "orders:read", permission = OrderPermission.class)
public OrderView get(@RestPath long id) { ... }
```

9.7.4 Externalising policy

For genuinely complex authorisation, keep the policy out of the code entirely — Open Policy Agent, Keycloak Authorization Services, or Cedar. The application asks “may this subject do this action on this resource?” and the policy engine answers.

```
@ApplicationScoped
public class PolicyClient {

    @RestClient OpaClient opa;

    public boolean allowed(String subject, String action, String resource) {
        return opa.evaluate(new PolicyInput(subject, action, resource)).result();
    }
}
```

The trade is real and worth stating in an interview. Externalising policy gives you auditable, testable, independently-deployable rules and lets non-engineers change them. It costs a network hop on every check (mitigable by running OPA as a sidecar), a new operational component, and a split between code and policy that makes “why was this denied?” harder to answer. I would reach for it when authorisation rules change more often than the code, or when compliance requires that policy be reviewable separately — not before.

9.8 Identity Propagation

Service A calls service B on behalf of a user. Getting this wrong produces the **confused deputy** vulnerability: B trusts A’s identity and performs an action the user was never entitled to.

9.8.1 Token propagation

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-oidc-token-propagation-reactive</artifactId>
</dependency>
```

```
@RegisterRestClient(configKey = "inventory-api")
@AccessToken // forward the caller's bearer token
public interface InventoryClient {
    @GET @Path("/stock/{sku}")
    StockLevel stock(@PathParam("sku") String sku);
}
```

The incoming token is attached to the outbound call, so the downstream service sees the **original user's** identity and applies its own authorisation. This is the correct default.

9.8.2 Client credentials

For genuinely system-to-system calls with no user context — a scheduled reconciliation job, say:

```
quarkus.oidc-client.auth-server-url=https://keycloak.acme.com/realms/acme
quarkus.oidc-client.client-id=orders-service
quarkus.oidc-client.credentials.secret=${OIDC_CLIENT_SECRET}
quarkus.oidc-client.grant.type=client
```

```
@RegisterRestClient(configKey = "inventory-api")
@OidcClientFilter // attach the service's own token
public interface InventorySystemClient {
    @POST @Path("/stock/reconcile")
    void reconcile(ReconcileRequest request);
}
```

9.8.3 Token exchange

When the downstream service should see a *narrower* identity than the caller holds — a token scoped to exactly the operation being performed:

```
quarkus.oidc-client.grant.type=exchange
quarkus.oidc-client.grant-options.exchange.audience=inventory-service
```

The rule: propagate the user's identity for user-initiated work, use client credentials only for genuinely system-initiated work. The anti-pattern is a service that authenticates the user, then calls downstream with its own powerful service account — the downstream service now has no idea who is really acting, cannot enforce user-level rules, and a bug in the upstream service becomes a full privilege escalation. If you take one thing from this section, take that.

9.9 Transport Security

9.9.1 TLS

Quarkus 3.x has a centralised TLS registry, which is the modern way to configure certificates once and reference them everywhere:

```
quarkus.tls.key-store.p12.path=server-keystore.p12
quarkus.tls.key-store.p12.password=${KEYSTORE_PASSWORD}
quarkus.http.insecure-requests=redirect # or 'disabled'
quarkus.http.ssl-port=8443

# A named configuration for an outbound client
quarkus.tls.payments.trust-store.p12.path=payments-truststore.p12
quarkus.tls.payments.trust-store.p12.password=${TRUSTSTORE_PASSWORD}
quarkus.rest-client.payments-api.tls-configuration-name=payments
```

In Kubernetes, TLS is usually terminated at the ingress and the pod speaks plaintext inside the mesh — or mTLS is handled transparently by the mesh. Terminating TLS in the application is

appropriate for direct exposure, for compliance requirements that mandate end-to-end encryption, and for mTLS between services that are not in a mesh.

9.9.2 Mutual TLS

```
quarkus.http.ssl.client-auth=required
quarkus.tls.key-store.p12.path=server-keystore.p12
quarkus.tls.trust-store.p12.path=client-truststore.p12
quarkus.http.auth.certificate-role-properties=cert-roles.properties

# cert-roles.properties : CN -> roles
CN=orders-client,OU=platform,O=acme=order-writer,order-reader
```

```
@Inject SecurityIdentity identity;

public void whoCalled() {
    CertificateCredential cred = identity.getCredential(CertificateCredential.class);
    X509Certificate cert = cred.getCertificate();
    String cn = cert.getSubjectX500Principal().getName();
}
```

9.9.3 Security headers

Quarkus does not set these for you. For any endpoint a browser touches:

```
@ApplicationScoped
public class SecurityHeadersFilter {

    @ServerResponseFilter
    public void addHeaders(ContainerResponseContext response) {
        var h = response.getHeaders();
        h.putSingle("Strict-Transport-Security", "max-age=31536000; includeSubDomains");
        h.putSingle("X-Content-Type-Options", "nosniff");
        h.putSingle("X-Frame-Options", "DENY");
        h.putSingle("Referrer-Policy", "strict-origin-when-cross-origin");
        h.putSingle("Content-Security-Policy",
            "default-src 'self'; frame-ancestors 'none'; base-uri 'self'");
        h.putSingle("Permissions-Policy", "geolocation=(), camera=(), microphone=()");
        h.putSingle("Cache-Control", "no-store");    // for authenticated responses
    }
}
```

For a pure JSON API consumed only by machines these matter less, but **X-Content-Type-Options** and **Cache-Control: no-store** on authenticated responses are worth having regardless.

9.9.4 CSRF

Only relevant when authentication is cookie-based — that is, **web-app** mode. Bearer tokens are not automatically attached by the browser, so a bearer API is not CSRF-vulnerable.

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-csrf-reactive</artifactId>
</dependency>
```



```
quarkus.csrf-reactive.cookie-name=csrf-token
quarkus.csrf-reactive.cookie-http-only=false      # the SPA must read it
quarkus.csrf-reactive.token-header-name=X-CSRF-TOKEN
quarkus.csrf-reactive.require-form-url-encoded=false
quarkus.oidc.authentication.cookie-same-site=lax
```

`SameSite=Lax` on the session cookie is itself a strong CSRF defence for most flows; the token is defence in depth.

9.10 Secrets

Chapter 4 covered the mechanics; the security-specific rules:

```
# NEVER
quarkus.oidc.credentials.secret=abc123

# Environment / Secret-backed
quarkus.oidc.credentials.secret=${OIDC_CLIENT_SECRET}

# Vault
quarkus.vault.url=https://vault.internal:8200
quarkus.vault.authentication.kubernetes.role=orders-service
quarkus.vault.secret-config-kv-path=secret/orders
```

Additional practices:

- **Prefer workload identity to a shared secret.** Kubernetes service account tokens with OIDC federation remove the secret entirely — there is nothing to rotate or leak.
- **Never log a token.** Add a filter that redacts `Authorization` headers from access logs, and audit any code that logs request headers wholesale.
- **Never put a token in a URL.** Query strings end up in access logs, browser history, and `Referer` headers.
- **Scan images and dependencies.** `quarkus-container-image-*` builds are reproducible enough that a scan in CI is meaningful.

9.11 Native Mode Considerations

Security in native mode mostly just works, with three things to know:

Cryptography needs the security services flag, which Quarkus sets for you when a security extension is present:

```
quarkus.native.additional-build-args=--enable-all-security-services
```

Trust stores must be reachable at runtime, not baked at build time. Reading a keystore during static init would embed the build machine's certificates in the image:

```
quarkus.native.add-all-charsets=true
quarkus.tls.trust-store.p12.path=/etc/tls/truststore.p12    # a mounted file
```

Entropy. Historically, containers could stall on `/dev/random` at startup during key generation. Modern kernels and base images have largely fixed this, but if a native binary hangs for seconds at startup with a security extension present, this is the first thing to check.

9.12 Testing Security

9.12.1 @TestSecurity

```
@QuarkusTest
class OrderResourceSecurityTest {

    @Test
    @TestSecurity(user = "alice", roles = {"order-reader"})
    void readerCanList() {
        given().when().get("/orders").then().statusCode(200);
    }

    @Test
    @TestSecurity(user = "alice", roles = {"order-reader"})
    void readerCannotCreate() {
        given().contentType("application/json").body(validRequest())
            .when().post("/orders")
            .then().statusCode(403);
    }

    @Test
    void anonymousIsRejected() {
        given().when().get("/orders").then().statusCode(401);
    }

    @Test
    @TestSecurity(user = "alice", roles = "order-reader")
    @JwtSecurity(claims = {
        @Claim(key = "tenant_id", value = "acme"),
        @Claim(key = "email", value = "alice@acme.com")
    })
    void readsTenantFromToken() {
        given().when().get("/orders")
            .then().statusCode(200)
            .body("[0].tenant", is("acme"));
    }
}
```

9.12.2 Real Keycloak via Dev Services

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-test-keycloak-server</artifactId>
  <scope>test</scope>
</dependency>
```

```
%test.quarkus.keycloak.devservices.realm-path=acme-realm.json
%test.quarkus.keycloak.devservices.users.alice=alice
%test.quarkus.keycloak.devservices.roles.alice=order-reader
%test.quarkus.keycloak.devservices.users.admin=admin
%test.quarkus.keycloak.devservices.roles.admin=admin,order-writer
```

```
@QuarkusTest
class OrderResourceKeycloakTest {

    @TestHTTPResource String url;

    @Test
    void adminCanDelete() {
        given().auth().oauth2(getAccessToken("admin"))
            .when().delete("/orders/1")
            .then().statusCode(204);
    }

    private String getAccessToken(String user) {
        return given()
            .param("grant_type", "password")
            .param("username", user)
            .param("password", user)
            .param("client_id", "quarkus-app")
            .param("client_secret", "secret")
            .when()
                .post(keycloakUrl + "/protocol/openid-connect/token")
            .as(AccessTokenResponse.class).getToken();
    }
}
```

`@TestSecurity` is fast and right for testing *your* authorisation logic. A real Keycloak is right for testing the *integration* — claim mapping, audience validation, role paths — which is where the misconfigurations actually live.

9.12.3 Test the negatives

```
@Test @TestSecurity(user = "bob", roles = "order-reader")
void cannotReadAnotherUsersOrder() {
    // alice's order - bob is a reader, but not the owner
    given().when().get("/orders/" + aliceOrderId)
        .then().statusCode(404); // 404, not 403 - no existence leak
}

@Test
void rejectsExpiredToken() { ... }

@Test
void rejectsTokenForAnotherAudience() { ... }

@Test
void rejectsTokenSignedByAnotherKey() { ... }
```

The negative tests are the security tests. A suite that only proves an admin can delete tells you nothing about whether a reader can. Every `@RolesAllowed` deserves a test that the *wrong* role gets 403, and every ownership check deserves a test that a different user gets 404.

9.13 Best Practices

1. **deny-unannotated-endpoints=true from day one.** Absent annotations are invisible in review.
2. **Always set `quarkus.oidc.token.audience`.** Otherwise any token from your IdP is accepted.
3. **Use `quarkus-oidc`, not manual JWT verification,** wherever an IdP is involved — JWKS rotation alone justifies it.
4. **Propagate the user's token** for user-initiated downstream calls; client credentials only for system-initiated work.
5. **404, not 403,** for resources whose existence the caller may not know about.
6. **Permissions, not roles,** once role names start encoding conditions.
7. **Ownership checks in code.** Annotations cannot express row-level rules.
8. **Never store tokens in browser storage.** Use `web-app` mode or a BFF with a session cookie.
9. **Cache identity augmentation** and never block the event loop in an augmentor.
10. **Test the negative cases** — wrong role, wrong owner, wrong audience, expired token.
11. **Set security headers explicitly** for anything a browser touches.
12. **Secrets from Vault or Kubernetes Secrets,** never from `application.properties`; prefer workload identity to a shared secret.
13. **Redact `Authorization` headers from logs,** and never put tokens in URLs.
14. **Short token lifetimes** with refresh, rather than long-lived tokens — revocation is hard, expiry is easy.

9.14 Common Mistakes

1. **No audience validation.** Any token the IdP issued works. Silent, and invisible in testing.
2. **Forgetting an annotation on a new endpoint.** Public by default without `deny-unannotated-endpoints`.
3. **Returning 403 where 404 is correct.** Leaks resource existence and enables enumeration.
4. **The confused deputy.** Service A authenticates the user, then calls B with its own privileged service account. B cannot enforce user-level rules.
5. **Blocking I/O in a `SecurityIdentityAugmentor`.** Runs on the event loop on every authenticated request.
6. **Storing access tokens in `localStorage`.** One XSS bug becomes full account takeover.
7. **Roles that encode policy.** `order-reader-eu-tier2` is a serialised rule, not a role.
8. **Checking authorisation only at the REST layer.** A Kafka consumer or scheduled job reaching the same service method bypasses it entirely.
9. **Long-lived tokens without revocation.** A leaked 24-hour token is a 24-hour breach.

10. Logging request headers wholesale. Tokens end up in your log aggregator, which usually has broader access than the service.

11. Trusting a client-supplied user id. `?userId=` in a query parameter is not authentication; the identity must come from the token.

12. Disabling issuer or signature validation to “make it work in dev”. These settings survive into production more often than anyone expects.

9.15 Interview Questions

Junior

Q1. What is a JWT? A JSON Web Token: a base64url-encoded header, payload of claims, and signature. It is self-contained, so a service can verify it without calling the issuer.

Q2. What is OIDC? OpenID Connect, an identity layer on top of OAuth 2.0. It defines how a client authenticates a user and obtains tokens — discovery, the authorization code flow, refresh, logout.

Q3. How do you secure an endpoint in Quarkus? `@Authenticated` to require a valid identity, `@RolesAllowed("role")` to require a role, `@PermitAll` for explicitly public, `@DenyAll` to block.

Q4. How do you read the current user? Inject `SecurityIdentity` and call `getPrincipal().getName()`, or inject `JsonWebToken` for claims.

Q5. What is the difference between 401 and 403? 401 means not authenticated — no credential, or an invalid one. 403 means authenticated but not authorised for this operation.

Q6. Where do you put the client secret? Never in `application.properties`. An environment variable backed by a Kubernetes Secret, or Vault. Better still, workload identity so there is no secret at all.

Q7. How do you test a secured endpoint? `@TestSecurity(user = "alice", roles = "order-reader")` on a `@QuarkusTest` method, plus a real Keycloak via Dev Services for integration-level checks.

Mid-level

Q8. `quarkus-oidc` versus `quarkus-smallrye-jwt` — when do you use each? `smallrye-jwt` only verifies a token against a configured public key. `quarkus-oidc` speaks the protocol: discovery, automatic JWKS refresh, token introspection for opaque tokens, the authorization code flow, refresh and logout. Use OIDC whenever an identity provider is involved; `smallrye-jwt` only for an internal issuer with a fixed key and no desire for a startup dependency on the IdP.

Q9. What is `application-type=service` versus `web-app`? `service` expects a bearer token in the `Authorization` header and returns 401 when it is missing — correct for APIs. `web-app` runs the authorization code flow, redirecting an unauthenticated browser to the IdP and storing tokens server-side behind a session cookie — correct for server-rendered applications and BFFs.

Q10. How do you map IdP roles to Quarkus roles? `quarkus.oidc.roles.role-claim-path`, pointing at the claim — `realm_access/roles` for Keycloak realm roles, `resource_access/<client>/roles` for client roles. For anything more complex, implement a `SecurityIdentityAugmentor`.

Q11. Why is validating the audience important? Without it, any token your IdP issued for any client is accepted. A token minted for a low-privilege frontend will authenticate against your service. It fails silently and your own tokens work fine, so testing does not catch it.

Q12. How do you call a downstream service on behalf of the user? Add `quarkus-oidc-token-propagation-reactive` and annotate the REST client interface with `@AccessToken`, which forwards the caller's bearer token so the downstream service sees the original user and enforces its own rules.

Q13. What is `SecurityIdentityAugmentor` for? Enriching the identity after authentication — adding roles or permissions from a database, attaching a tenant attribute. It runs on every authenticated request, so any blocking work must go through `context.runBlocking` and the result must be cached.

Q14. How do you make endpoints secure by default? `quarkus.security.jaxrs.deny-unannotated-endpoints=true`. An endpoint with no security annotation is then denied rather than public, which is the safe failure mode — code review does not reliably catch a *missing* annotation.

Q15. Is a bearer-token API vulnerable to CSRF? No. CSRF depends on the browser automatically attaching a credential; bearer tokens must be set explicitly by JavaScript. Cookie-based authentication — `web-app` mode — is vulnerable, and needs `SameSite` plus a CSRF token.

Q16. Why 404 instead of 403 for a resource the user does not own? 403 confirms the resource exists, which lets an attacker enumerate valid ids. 404 reveals nothing. Use 403 only when the caller may legitimately know the resource exists but not perform the operation.

Senior

Q17. Walk me through the OIDC authorization code flow. The browser requests a protected page. Quarkus, in `web-app` mode, redirects to the IdP's authorization endpoint with the client id, requested scopes, a redirect URI, a `state` value to bind the response to the session, and a PKCE challenge. The user authenticates at the IdP, which redirects back to the application's callback with an authorization code. Quarkus verifies `state`, then exchanges the code at the token endpoint — server to server, with the client secret or PKCE verifier — for an id token, an access token and a refresh token. It validates the id token's signature against the JWKS, checks issuer, audience, nonce and expiry, builds a `SecurityIdentity`, stores the tokens in an encrypted cookie or a server-side store, and redirects to the originally requested path. The browser thereafter holds only a session reference, never a token. The reason for the code exchange rather than returning tokens directly is that the code is single-use, short-lived and passed through the browser only once, while the tokens themselves travel over a back channel.

Q18. Design authorisation for a multi-tenant SaaS. Tenancy is not a role, so I would carry it as a claim in the token — `tenant_id` — and enforce it as a mandatory filter rather than as a permission check. Concretely: a request-scoped `TenantContext` populated from the token by an augmentor, and every repository query filtered by it, ideally by a Hibernate filter or row-level security in PostgreSQL so that forgetting the filter fails closed rather than leaking across tenants. That last part matters more than anything else in the design — a manual `where tenant_id = ?` in forty query methods will eventually be omitted once, and the consequence is a cross-tenant data leak. Within a tenant, permissions rather than roles, with roles as a packaging mechanism the customer's own admin can compose. Cross-tenant access for support staff is a separate, explicitly-audited path with its own permission and a logged justification, never an ambient “super admin”

role. And I would write the test that a user from tenant A gets 404 on tenant B's resource, and run it against every resource type, because that is the test that catches the regression.

Q19. How would you handle token revocation? Honestly, by mostly not needing it. Self-contained JWTs cannot be revoked without introducing state, which sacrifices the property that made them attractive. So the primary answer is short lifetimes — five to fifteen minutes — with refresh tokens, so a leaked access token has a small window, and refresh tokens that *are* revocable because they are checked at the IdP. Where genuine immediate revocation is required — a compromised account, an employee termination — the options are a denylist of `jti` values in a shared cache that every service consults, which costs a lookup per request and reintroduces a shared dependency; or token introspection against the IdP, which is authoritative but adds a network call per request and can be cached briefly; or back-channel logout, where the IdP notifies each service. I would choose based on the actual requirement: for most systems short lifetimes are sufficient and the complexity of revocation is not justified, and I would push back on a blanket “we need instant revocation” requirement until someone names the scenario and the acceptable latency.

Q20. A colleague suggests storing the access token in `localStorage` so the SPA can attach it. Respond. I would not agree, and the reason is specific rather than dogmatic: `localStorage` is readable by any JavaScript running on the origin, so a single XSS vulnerability anywhere on the site — including in a third-party script or a dependency — becomes full token exfiltration, and the attacker then holds a credential valid until expiry that they can use from anywhere. That converts a bounded vulnerability into account takeover. The current recommendation is a backend-for-frontend: the SPA authenticates against a Quarkus service in `web-app` mode, tokens stay server-side, and the browser holds an `HttpOnly, Secure, SameSite` session cookie that JavaScript cannot read. The BFF calls downstream services with the real token. The costs are real — a stateful component and a CSRF consideration that bearer tokens do not have — and I would name them rather than pretend the choice is free. What I would not accept is the halfway position of an in-memory token with a refresh token in a cookie, which is better than `localStorage` but still exposes the access token to any script on the page.

Q21. How do you secure service-to-service communication inside a cluster? Layered, because no single mechanism is sufficient. At the transport layer, mTLS — from a service mesh if one is present, since it handles rotation and is transparent to the application, otherwise configured directly with short-lived certificates from an internal CA. At the identity layer, either the caller's propagated user token for user-initiated work, or the service's own client-credentials token for system-initiated work, with the audience of each token set to the specific callee so a token for one service cannot be replayed against another. At the network layer, NetworkPolicies so that only the services that should talk to each other can open a connection at all, which limits the blast radius of a compromise regardless of what happens above. And each service enforces its own authorisation rather than trusting that the caller already did — the mesh proves *who* is calling, not *whether they should*. The mistake I would be watching for is treating mTLS as authorisation: it establishes identity, and teams frequently stop there.

Principal

Q22. Design the authentication and authorisation strategy for a platform of 40 Quarkus services. The organising principle is that every service enforces its own authorisation, and the platform makes doing that correctly the path of least resistance. Concretely: one identity provider, one token format, one audience convention (each service's audience is its own name, so tokens are not replayable across services), short access-token lifetimes with refresh handled at the

edge, and a shared internal extension that configures OIDC, sets `deny-unannotated-endpoints`, registers the identity augmentor, adds the security headers filter and redacts `Authorization` from logs — so a service gets the baseline by adding a dependency rather than by copying configuration. Authorisation is permission-based, with a central role-to-permission mapping owned by the security team and roles as the customer-facing packaging. Row-level rules stay in the services, because only they know their data model, but the platform ships a tenant-filter helper so the common case is hard to get wrong. Identity propagation is the default for user-initiated calls and client credentials is an explicit, reviewable exception — I would make that a lint rule, because the confused-deputy pattern is easy to introduce and nearly invisible in review. Verification is where I would spend the most effort: a shared test library with negative-case helpers, a contract test that every endpoint has an explicit security annotation, and a periodic automated sweep that enumerates endpoints across all 40 services and flags any that are unauthenticated. Finally, an incident path: a documented, rehearsed procedure for revoking a compromised credential and rotating a key, because the strategy is only as good as its worst day.

Q23. Argue against adopting a service mesh purely for security. A mesh gives you mTLS with automatic certificate rotation and transparent policy enforcement, which is genuinely valuable and hard to replicate by hand across 40 services. But adopting one *purely* for security is a poor trade, because the security benefit is narrower than it appears and the cost is broad. The narrow part: mTLS establishes that a call came from a particular workload, which is authentication, not authorisation — every service still has to decide what that workload may do, so the mesh removes none of the application-level security work. It also does nothing about the vulnerabilities that actually cause breaches, which are missing authorisation checks, injection, and leaked credentials, not unencrypted intra-cluster traffic. The cost: a sidecar per pod with its own memory and latency overhead, a substantial new operational surface with its own failure modes and upgrade cycle, debugging that now involves an extra hop nobody on the application team understands, and a control plane that becomes a cluster-wide dependency. Cheaper alternatives cover much of the benefit — NetworkPolicies for isolation, cert-manager plus a shared TLS configuration in a platform extension for mTLS, and the OIDC layer for identity. My position would be that a mesh is worth adopting when you also want its traffic management, observability and progressive-delivery capabilities, because then the operational cost is amortised across several needs. For security alone, I would spend the same effort on making authorisation correct in the services, which is where the actual risk is.

9.16 Cheat Sheet

Extensions

<code>quarkus-oidc</code>	<code><!-- OIDC: bearer + code flow --></code>
<code>quarkus-smallrye-jwt</code>	<code><!-- verify JWTs only --></code>
<code>quarkus-smallrye-jwt-build</code>	<code><!-- issue JWTs --></code>
<code>quarkus-oidc-client</code>	<code><!-- obtain tokens (client credentials) --></code>
<code>quarkus-oidc-token-propagation-reactive</code>	<code><!-- forward the caller's token --></code>
<code>quarkus-elytron-security-jdbc</code>	<code><!-- users in a database --></code>
<code>quarkus-test-keycloak-server</code>	<code><!-- test scope --></code>

Bearer-token API

```
quarkus.oidc.auth-server-url=https://kc/realms/acme
quarkus.oidc.client-id=orders-service
```



```
quarkus.oidc.application-type=service
quarkus.oidc.token.audience=orders-service      # DO NOT OMIT
quarkus.oidc.roles.role-claim-path=realm_access/roles
quarkus.security.jaxrs.deny-unannotated-endpoints=true
```

Annotations

```
@Authenticated          @PermitAll          @DenyAll
@RolesAllowed("admin")   @RolesAllowed({"a", "b"})
@PermissionsAllowed("orders:read")
@PermissionsAllowed(value = {"a", "b"}, inclusive = true) // AND
```

Path policies

```
quarkus.http.auth.permission.public.paths=/q/health/*
quarkus.http.auth.permission.public.policy=permit
quarkus.http.auth.permission.admin.paths=/admin/*
quarkus.http.auth.permission.admin.policy=role-admin
quarkus.http.auth.policy.role-admin.roles-allowed=admin
```

Identity

```
@Inject SecurityIdentity identity;
@Inject JsonWebToken jwt;
@Inject @Claim("tenant_id") String tenant;

identity.getPrincipal().getName();
identity.hasRole("admin");
jwt.getClaim("tenant_id");
```

Propagation

```
@RegisterRestClient @AccessToken      interface UserCall { } // forward user token
@RegisterRestClient @OidcClientFilter interface SysCall { }   // service token
```

Testing

```
@TestSecurity(user = "alice", roles = {"order-reader"})
@JwtSecurity(claims = @Claim(key = "tenant_id", value = "acme"))
```

Status codes

```
401 no or invalid credential
403 authenticated, not permitted (existence is not secret)
404 authenticated, not permitted, existence IS secret
```

9.17 Summary

Quarkus's security model is three pieces: a mechanism extracts a credential, a provider turns it into a `SecurityIdentity`, and authorisation runs against that identity. The metadata is resolved at build time like everything else, so there is no reflection on the request path.

The distinction to get right first is **JWT versus OIDC**: one is a token format you verify, the other is the protocol that produces tokens. Use `quarkus-oidc` whenever an identity provider is involved,

and **always set `token.audience`** — omitting it silently accepts every token your IdP issues, and nothing in testing will reveal it.

For authorisation, turn on **`deny-unannotated-endpoints`** on day one, because a missing annotation is invisible in review and produces a public endpoint. Move from roles to permissions as soon as role names start encoding conditions. Accept that ownership and row-level rules cannot be expressed as annotations and must live in code — and return 404 rather than 403 when the caller should not know a resource exists.

Across services, propagate the user’s identity for user-initiated work. The **`confused deputy`** — authenticating a user, then calling downstream with a privileged service account — is the failure that turns one bug into a full privilege escalation, and it is common precisely because it looks like a convenience.

Finally: the negative tests *are* the security tests. Wrong role, wrong owner, wrong audience, expired token. A suite that only proves the admin can delete has verified nothing.

Chapter 10 changes register entirely, moving to **reactive programming with Mutiny** — the type system, backpressure, and the composition patterns that make the reactive model worth its cognitive cost.

9.18 References

- Security overview — <https://quarkus.io/guides/security-overview>
- OIDC bearer token authentication — <https://quarkus.io/guides/security-oidc-bearer-token-authentication>
- OIDC code flow — <https://quarkus.io/guides/security-oidc-code-flow-authentication>
- SmallRye JWT — <https://quarkus.io/guides/security-jwt>
- Authorization — <https://quarkus.io/guides/security-authorize-web-endpoints-reference>
- OIDC token propagation — <https://quarkus.io/guides/security-openid-connect-client-reference>
- TLS registry — <https://quarkus.io/guides/tls-registry-reference>
- Testing security — <https://quarkus.io/guides/security-testing>
- OWASP API Security Top 10 — <https://owasp.org/API-Security/>
- OAuth 2.0 for Browser-Based Apps — <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-browser-based-apps>

Reactive Programming with Mutiny

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Mutiny 2.x · Vert.x · SmallRye Context Propagation

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what problem reactive programming solves, and — equally important — what it does not solve.
2. Distinguish **Uni** from **Multi** and choose correctly between them.
3. Read and write Mutiny’s event-driven API fluently, including the `onItem()` / `onFailure()` group navigation that makes it unusual.
4. Compose asynchronous work: sequential chaining, parallel combination, and racing.
5. Handle failures with recovery, retry with exponential backoff and jitter, and timeouts.
6. Explain **backpressure** precisely and apply overflow strategies to unbounded streams.
7. Reason about threading: which thread a subscriber runs on, and how to move work deliberately.
8. Propagate CDI, transaction and security context across asynchronous boundaries.
9. Decide between **Mutiny** and **Java 21 virtual threads** with an argument you can defend.
10. Test reactive code deterministically, and answer 25 interview questions on the reactive model.

10.1 What Reactive Actually Buys You

The word “reactive” is used for several distinct things. Being precise about which one you mean is the first step to a good interview answer.

Reactive as an execution model. Do not block a thread while waiting for I/O. A small number of threads handles a large number of concurrent operations, because a thread is released the moment it starts waiting rather than parked until the answer arrives.

Reactive as a programming style. Express computation as a pipeline of transformations over asynchronous events rather than as sequential statements.

Reactive Streams as a specification. A standard for asynchronous stream processing with non-blocking **backpressure** — the consumer signals how much it can accept.

Quarkus gives you all three, but they are separable. The execution model is what delivers the performance benefit, and on Java 21 **virtual threads deliver the same execution-model**

benefit without the programming style. That reframing matters, and we return to it in §10.9.

10.1.1 What it does not buy you

- **It is not faster for a single request.** A reactive endpoint and a blocking one have similar latency for one call. Reactive improves *throughput under concurrency* by using fewer threads.
- **It does not remove your bottleneck.** If your service is limited by a 20-connection database pool, going reactive moves the queue; it does not shorten it.
- **It does not help CPU-bound work.** Reactive is about not blocking on I/O. A slow computation is slow on any thread, and running it on an event loop is actively harmful.

A strong interview answer names the cost. Reactive code is harder to read, harder to debug — stack traces do not correspond to the logical flow — and harder to hire for. It is worth that cost when you need high concurrency with few threads, real backpressure, or fine-grained composition of many concurrent calls. It is not worth it as a default.

10.2 Uni and Multi

Mutiny has exactly two types, and the distinction is deliberately stark.

	Uni<T>	Multi<T>
Emits	At most one item, then completes	Zero to many items, then completes
Analogous to	<code>CompletableFuture<T></code>	<code>Flow.Publisher<T></code> / <code>Flux<T></code>
Backpressure	Not applicable	Yes
Typical use	An HTTP call, a database read, a computation	A query result set, a Kafka topic, SSE

```
// Uni: one result
Uni<Order> order = orderService.findById(1L);
Uni<Void> done = orderService.delete(1L);           // Uni<Void> for "completed, no value"

// Multi: a stream
Multi<Order> orders = orderService.streamAll();
Multi<Tick> ticks = Multi.createFrom().ticks().every(Duration.ofSeconds(1));
```

10.2.1 Nothing happens until you subscribe

This is the single most common source of “my reactive code does nothing”.

```
// BUG: creates a pipeline and discards it. No HTTP call is made.
public void notify(Order order) {
    notificationClient.send(order);           // returns Uni - never subscribed
}
```

```

// Correct in a resource: return it, and the framework subscribes
@POST
public Uni<Void> notify(Order order) {
    return notificationClient.send(order);
}

// Correct in fire-and-forget code: subscribe explicitly
public void notifyAsync(Order order) {
    notificationClient.send(order)
        .subscribe().with(
            ok -> Log.debugf("notified %d", order.id),
            err -> Log.errorf(err, "notification failed for %d", order.id));
}

```

`Uni` and `Multi` are **cold and lazy**: they describe work rather than perform it. Building one has no side effect. Quarkus subscribes for you when you return one from a resource method, a `@Incoming/@Outgoing` method, or a scheduled method — everywhere else, subscription is your responsibility.

10.2.2 Creating them

```

// From a known value
Uni.createFrom().item("value");
Uni.createFrom().item(() -> expensiveComputation()); // deferred until subscription
Uni.createFrom().nullItem();
Uni.createFrom().failure(new IllegalStateException("nope"));
Uni.createFrom().voidItem();

// From other async types
Uni.createFrom().completionStage(() -> someCompletableFuture());
Uni.createFrom().future(() -> someFuture());
Uni.createFrom().publisher(somePublisher);

// From a callback-style API
Uni.createFrom().emitter(emitter ->
    legacyClient.fetchAsync(id, new Callback() {
        public void onSuccess(Result r) { emitter.complete(r); }
        public void onError(Throwable t) { emitter.fail(t); }
    }));

// Multi
Multi.createFrom().items("a", "b", "c");
Multi.createFrom().iterable(List.of(1, 2, 3));
Multi.createFrom().range(0, 100);
Multi.createFrom().ticks().every(Duration.ofSeconds(1));
Multi.createFrom().emitter(em -> { em.emit(1); em.emit(2); em.complete(); },
    BackPressureStrategy.BUFFER);

```

`Uni.createFrom().item(value)` versus `item(supplier)` is a real trap. The first evaluates `value` *immediately*, when the pipeline is built. `Uni.createFrom().item(blockingCall())`

therefore blocks the current thread before any reactivity happens — a bug we already met in Chapter 2. The supplier form defers evaluation until subscription.

10.3 The Event-Driven API

Mutiny’s API is deliberately different from RxJava and Reactor. Instead of a flat list of a hundred operators, you first name the **event** you care about, then the action.

```
uni.onItem().transform(...)           // an item arrived
uni.onFailure().recoverWithItem(...) // it failed
uni.onCancellation().invoke(...)      // the subscriber went away
uni.onSubscription().invoke(...)      // someone subscribed
uni.onTermination().invoke(...)       // it ended, however
```

The reasoning: `flatMap` versus `concatMap` versus `switchMap` versus `mergeMap` is exactly the sort of API that requires memorisation. `onItem().transformToUniAndConcatenate()` is longer but says what it does. Once you internalise “event group first, then action”, the API is discoverable by autocompletion.

10.3.1 Transforming

```
// Synchronous transformation: T -> R
Uni<String> name = orderUni.onItem().transform(order -> order.customerEmail);

// Asynchronous transformation: T -> Uni<R> (this is flatMap)
Uni<Customer> customer = orderUni
    .onItem().transformToUni(order -> customerClient.byId(order.customerId));

// Shorthand aliases - identical behaviour, less typing
Uni<String> name2 = orderUni.map(order -> order.customerEmail);
Uni<Customer> cust2 = orderUni.chain(order -> customerClient.byId(order.customerId));

// Side effect without changing the value
Uni<Order> logged = orderUni.invoke(order -> Log.infof("got %d", order.id));

// Asynchronous side effect - waits for it, then passes the ORIGINAL item through
Uni<Order> audited = orderUni.call(order -> auditClient.record(order.id));

// Replace the value entirely
Uni<Void> ignored = orderUni.replaceWithVoid();
```

The `invoke` / `call` distinction is worth committing to memory:

- **invoke** takes a `Consumer` — synchronous, fire-and-forget, does not wait.
- **call** takes a `Function<T, Uni<?>>` — asynchronous, **waits for completion**, then emits the original item.

```
// BUG: the audit call is never subscribed, so it never happens
order.onItem().invoke(o -> auditClient.record(o.id));
```

```
// Correct
order.onItem().call(o -> auditClient.record(o.id));
```

10.3.2 Multi-specific operators

```
Multi<Order> orders = repo.streamAll();

orders.filter(o -> o.total.compareTo(THRESHOLD) > 0)
    .map(OrderSummary::of)
    .select().first(100)
    .select().distinct()
    .skip().first(10)
    .group().intoLists().of(50)           // batches of 50
    .onItem().transformToUniAndConcatenate(batch -> process(batch))
    .collect().asList();                 // Multi<T> -> Uni<List<T>>

// Time-based grouping - useful for batching writes
orders.group().intoLists().every(Duration.ofSeconds(1))
    .onItem().transformToUniAndConcatenate(this::bulkInsert);
```

10.3.3 Concatenate vs merge — an important choice

```
// CONCATENATE: sequential. Order preserved. One at a time.
multi.onItem().transformToUniAndConcatenate(id -> client.fetch(id));

// MERGE: concurrent. Order NOT preserved. All in flight at once.
multi.onItem().transformToUniAndMerge(id -> client.fetch(id));

// MERGE with a concurrency limit - almost always what you actually want
multi.onItem().transformToUni(id -> client.fetch(id)).merge(8);
```

Unbounded `merge` over a large stream will open as many concurrent calls as there are items. Against a downstream service that is a self-inflicted denial of service; against a connection pool it is instant exhaustion. **Always bound the concurrency.**

10.4 Composition

10.4.1 Sequential

```
public Uni<OrderView> checkout(long orderId) {
    return orderRepository.findById(orderId)
        .onItem().ifNull().failWith(() -> new OrderNotFoundException(orderId))
        .chain(order -> stockClient.reserve(order).replaceWith(order))
        .chain(order -> paymentClient.capture(order).map(p -> order.withPayment(p)))
        .chain(order -> orderRepository.persist(order))
        .map(OrderView::of);
}
```

10.4.2 Parallel

All three calls start at subscription and run concurrently. Total latency is the slowest, not the sum — the single clearest win of the reactive model over sequential blocking code.

```
// Collect a dynamic list
List<Uni<StockLevel>> lookups = skus.stream()
    .map(stockClient::level)
    .toList();

Uni<List<StockLevel>> all = Uni.combine().all().unis(lookups)
    .with(results -> results.stream().map(StockLevel.class::cast).toList());

// Do not abort on the first failure - collect them all
Uni<List<StockLevel>> tolerant = Uni.combine().all().unis(lookups)
    .collectFailures()
    .with(results -> ...);
    .with((u, p, v) -> new Dashboard(u, p, v));
}
```

10.4.3 Racing

```
// First to respond wins; the others are cancelled
Uni<Quote> fastest = Uni.combine().any().of(
    carrierA.quote(request),
    carrierB.quote(request),
    carrierC.quote(request));
```

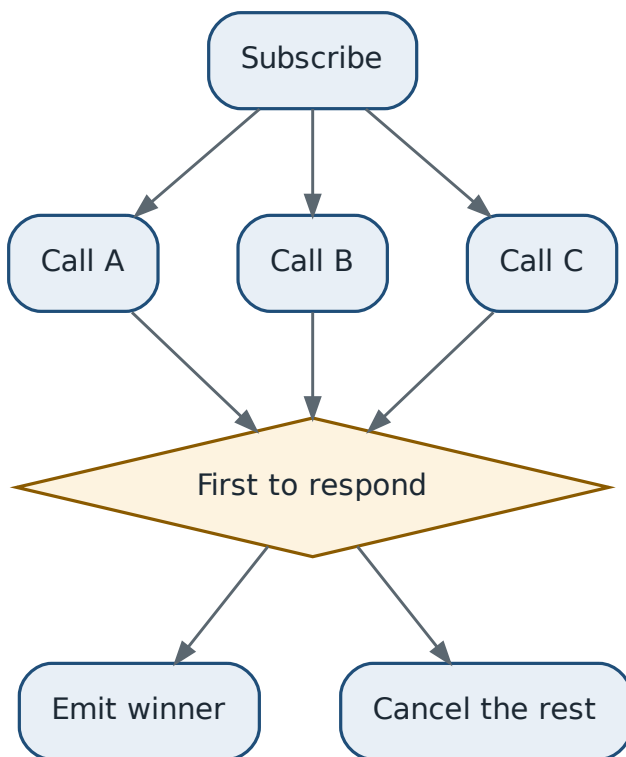



Figure 10.1 — `Uni.combine().any()`. Cancellation of the losers is automatic — a genuine advantage over a naive thread-based implementation.

10.5 Failure Handling

```
Uni<Order> resilient = orderClient.fetch(id)

// Substitute a value
.resFailure().recoverWithItem(Order.EMPTY)
.resFailure(NotFoundException.class).recoverWithNull()

// Substitute another pipeline
.resFailure(TimeoutException.class).recoverWithUni(() -> cache.get(id))

// Translate the exception
.resFailure(WebApplicationException.class)
    .transform(t -> new OrderServiceUnavailableException(t))

// Observe without handling
.resFailure().invoke(t -> Log.errorf(t, "fetch failed for %d", id))
```

```
// Always run
.onTermination().invoke((item, failure, cancelled) -> metrics.record(failure));
```

10.5.1 Retry, done properly

```
Uni<Order> retried = orderClient.fetch(id)
    .onFailure(this::isRetryable)
    .retry()
    .withBackOff(Duration.ofMillis(200), Duration.ofSeconds(5))
    .withJitter(0.3)
    .atMost(4);

private boolean isRetryable(Throwable t) {
    if (t instanceof TimeoutException || t instanceof ConnectException) return true;
    if (t instanceof WebApplicationException w) {
        int status = w.getResponse().getStatus();
        return status >= 500 || status == 429; // never retry a 4xx you caused
    }
    return false;
}
```

Three things this gets right that naive retry does not:

Exponential backoff. Retrying immediately against a struggling service adds load exactly when it can least absorb it.

Jitter. Without it, every client that failed at the same moment retries at the same moment — a synchronised thundering herd that can keep a recovering service down. `withJitter(0.3)` randomises each delay by $\pm 30\%$.

A selective predicate. Retrying a 400 or a 404 is pure waste; retrying a non-idempotent operation is a correctness bug (Chapter 2, §2.7.3).

```
// Retry until a condition holds - for polling an eventually-consistent resource
Uni<JobStatus> polled = jobClient.status(jobId)
    .onItem().transform(s -> { if (!s.isTerminal()) throw new NotDoneYet(); return s; })
    .onFailure(NotDoneYet.class).retry()
    .withBackOff(Duration.ofSeconds(1), Duration.ofSeconds(10))
    .expireIn(Duration.ofMinutes(5).toMillis());
```

10.5.2 Timeouts

```
Uni<Order> bounded = orderClient.fetch(id)
    .ifNoItem().after(Duration.ofSeconds(2)).fail()
    .onFailure(TimeoutException.class).recoverWithItem(Order.EMPTY);

// Or fall back rather than fail
Uni<Order> withFallback = orderClient.fetch(id)
    .ifNoItem().after(Duration.ofSeconds(2)).recoverWithItem(Order.EMPTY);
```

Every outbound call needs a timeout, and the budgets must nest. If your endpoint has a 3-second SLO and calls three services sequentially, each cannot have a 2-second timeout — the arithmetic does not work. Set an overall deadline and derive per-call budgets from it. A service with per-call timeouts that sum to more than its own timeout will exhaust threads under a downstream slowdown, which is the cascading-failure mechanism from Chapter 2.

10.6 Backpressure

This is what genuinely distinguishes Reactive Streams from “async with callbacks”, and it is what virtual threads do *not* give you.

The problem: a producer emits faster than a consumer can process. Without a mechanism, items accumulate in an unbounded buffer until the process dies.

The solution: the subscriber *requests* a number of items. The publisher may emit no more than that. Demand flows upstream; data flows downstream.

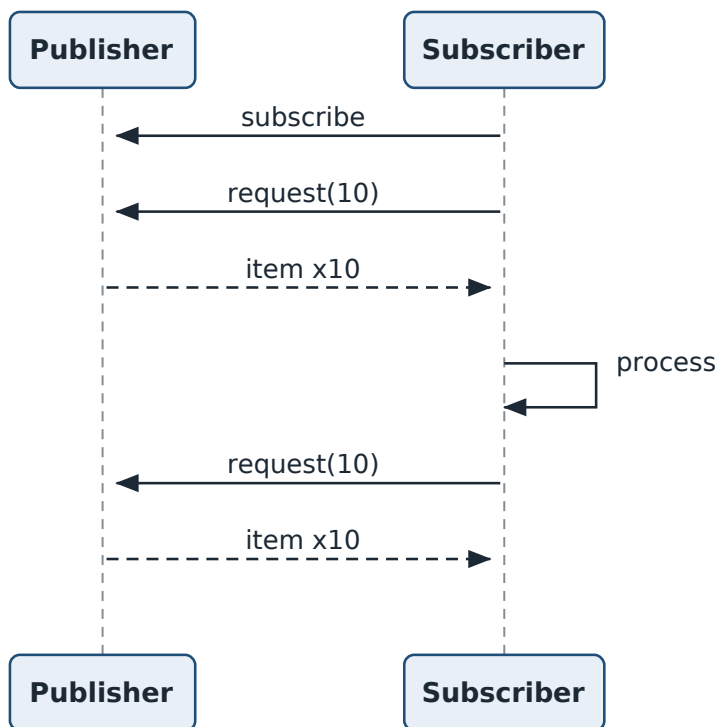


Figure 10.2 — Demand flows upstream. The publisher never sends more than was requested.

Backpressure works end to end when every stage honours it — a database cursor, a Kafka consumer, an HTTP response. It breaks when a source cannot be slowed down: a timer, a UI event, a message broker with no flow control. For those you must declare an overflow strategy.

```
Multi<Event> events = eventBus.consume("orders");

events.onOverflow().buffer(1000)           // buffer, then fail if exceeded
events.onOverflow().dropPreviousItems()    // keep only the newest
```

```
events.onOverflow().drop() // discard new items when full
events.onOverflow().dropPreviousItems().onOverflow().buffer(100)
events.onOverflow().invoke(item -> metrics.count("dropped")).drop();
```

```
// Explicit demand control
multi.paceDemand().using(new FixedDemandPacer(10, Duration.ofSeconds(1)));
```

An unbounded **Multi** with no overflow strategy is an **OutOfMemoryError** waiting for a slow consumer. Any **Multi** fed by a source you do not control — SSE subscribers, an event bus, a message broker without flow control — needs an explicit policy, and the policy is a product decision: is it better to drop events, to keep only the latest, or to fail loudly?

10.7 Threading

10.7.1 The default rule

Mutiny does **not** introduce threads. A pipeline runs on whatever thread emits the event — usually the event loop of the I/O operation that produced it.

```
@GET
public Uni<OrderView> get(@RestPath long id) {
    return client.fetch(id) // I/O on event loop N
        .map(this::toView); // ALSO runs on event loop N
}
```

That is why a slow **map** is dangerous: it occupies an event loop thread.

10.7.2 Moving work deliberately

```
import io.smallrye.mutiny.infrastructure.Infrastructure;

// runSubscriptionOn: where the SOURCE executes
Uni<Report> report = Uni.createFrom().item(() -> buildReportBlocking())
    .runSubscriptionOn(Infrastructure.getDefaultWorkerPool());

// emitOn: where DOWNSTREAM operators execute
Uni<OrderView> view = client.fetch(id)
    .emitOn(Infrastructure.getDefaultWorkerPool())
    .map(this::expensiveMapping); // now on a worker thread

// Java 21 virtual threads as the executor
Executor vthreads = Executors.newVirtualThreadPerTaskExecutor();
Uni<Report> onVirtual = Uni.createFrom().item(() -> buildReportBlocking())
    .runSubscriptionOn(vthreads);
```

The distinction is exam-worthy:

- **runSubscriptionOn** controls the thread on which subscription happens, and therefore where the *source* does its work. Use it to move a blocking source off the event loop.

- **emitOn** controls the thread on which *downstream* operators receive events. Use it to move expensive processing off the event loop after a non-blocking source.

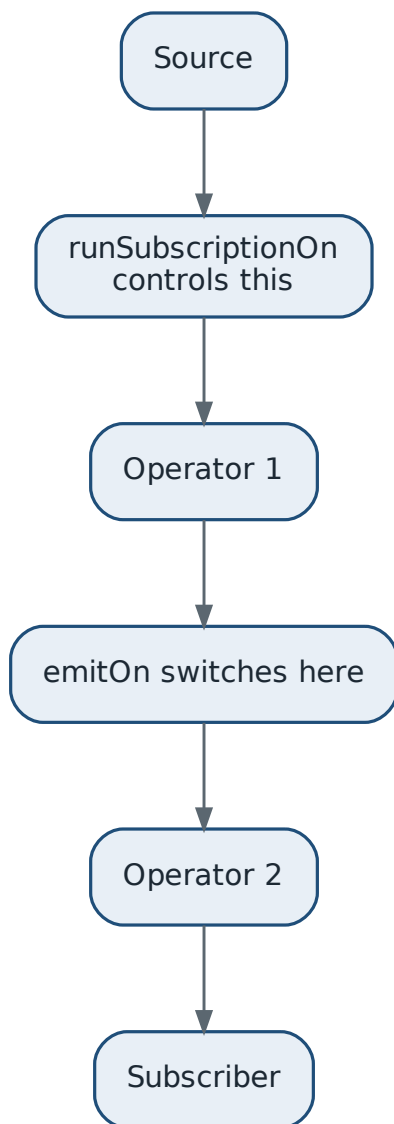


Figure 10.3 — *runSubscriptionOn* affects upstream; *emitOn* affects downstream.

10.7.3 Detecting event-loop blocking

```
%dev.quarkus.vertx.max-event-loop-execute-time=15  
quarkus.vertx.warning-exception-time=25
```

```
WARN [io.vertx.cor.imp.BlockedThreadChecker] Thread Thread[vert.x-eventloop-thread-2]  
has been blocked for 3021 ms
```

Treat it as an incident, per Chapter 2.

10.8 Context Propagation

CDI request context, transactions and the security identity are all thread-bound. When work moves between threads they must be carried explicitly.

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-context-propagation</artifactId>
</dependency>
```

With the extension present, Mutiny propagates context automatically along `Uni`/`Multi` chains. Where it does not reach:

```
@ApplicationScoped
public class ReportService {

    @Inject ManagedExecutor executor;      // propagates CDI, tx, security
    @Inject ThreadContext threadContext;

    public Uni<Report> generate(String period) {
        return Uni.createFrom()
            .completionStage(executor.supplyAsync(() -> build(period)));
    }

    public void rawThread() {
        Runnable task = threadContext.contextualRunnable(() -> {
            // request context is available here
        });
        new Thread(task).start();
    }
}
```

Mutiny also has its own lightweight context for carrying values along a pipeline:

```
Uni<Order> withContext = Uni.createFrom().context(ctx -> {
    String tenant = ctx.get("tenant");
    return orderClient.fetchFor(tenant);
})
.attachContext()
.map(itemWithContext -> {
    String tenant = itemWithContext.context().get("tenant");
    return itemWithContext.get();
});
```

My practical advice: pass identity and tenant explicitly as parameters across asynchronous boundaries. Context propagation works, but implicit context that silently becomes null in one code path is among the hardest bug classes to find, and the failure is usually a security one — a query that runs without a tenant filter. Explicit parameters are verbose and boring, which is what you want here.

10.9 Mutiny or Virtual Threads?

On Java 21, this is the design question, and interviewers know it.

```
// Reactive
@GET @Path("/{id}")
public Uni<Dashboard> reactive(@RestPath String id) {
    return Uni.combine().all()
        .unis(orderClient.recentFor(id), profileClient.byId(id), billingClient.balanceFor(id))
        .with(Dashboard::new);
}

// Virtual threads
@GET @Path("/{id}/vt")
@RunOnVirtualThread
public Dashboard virtual(@RestPath String id) {
    var orders = orderClient.recentForBlocking(id);
    var profile = profileClient.byIdBlocking(id);
    var balance = billingClient.balanceForBlocking(id);
    return new Dashboard(orders, profile, balance);
}
```

Note that the virtual-thread version above runs the three calls **sequentially** — it is simpler but slower. Structured concurrency fixes that:

```
@GET @Path("/{id}/structured")
@RunOnVirtualThread
public Dashboard structured(@RestPath String id) throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        var orders = scope.fork(() -> orderClient.recentForBlocking(id));
        var profile = scope.fork(() -> profileClient.byIdBlocking(id));
        var balance = scope.fork(() -> billingClient.balanceForBlocking(id));

        scope.join().throwIfFailed();
        return new Dashboard(orders.get(), profile.get(), balance.get());
    }
}
```

10.9.1 The comparison

	Mutiny	Virtual threads
Concurrency with few OS threads	Yes	Yes
Backpressure	Yes	No
Streaming	Native (Multi)	Awkward
Cancellation propagation	Built in	StructuredTaskScope
Readability	Lower	Higher
Debuggability / stack traces	Poor	Normal
Learning curve	Steep	Minimal
Library support	Needs reactive clients	Any blocking library

	Mutiny	Virtual threads
Pinning risk	n/a	<code>synchronized</code> around I/O

10.9.2 A defensible position

Default to virtual threads for request/response endpoints that call a database and a few services. The code is imperative, debuggable and readable by anyone, and the concurrency benefit is nearly identical.

Use Mutiny where its distinctive capabilities matter:

- **Streaming** — SSE, a large export, a Kafka pipeline. `Multi` is the right abstraction and virtual threads have nothing equivalent.
- **Backpressure** — a fast producer and a slow consumer. Virtual threads make it *cheap* to overwhelm a downstream service; only backpressure stops you.
- **Complex composition** — “call five services, take the first three, cancel the rest, 200 ms budget”. Mutiny expresses that declaratively.
- **Existing reactive infrastructure** — if you are already on Hibernate Reactive and reactive clients, mixing models adds cost without benefit.

Do not mix them carelessly. `@RunOnVirtualThread` on a method that returns `Uni` is confused; pick one per endpoint. And remember the pinning rule: `synchronized` around blocking I/O on a virtual thread pins the carrier thread. Use `ReentrantLock`.

10.10 Testing

10.10.1 UniAssertSubscriber

```
import io.smallrye.mutiny.helpers.test.UniAssertSubscriber;
import io.smallrye.mutiny.helpers.test.AssertSubscriber;

class OrderServiceTest {

    @Test
    void emitsOrder() {
        UniAssertSubscriber<Order> subscriber = service.findById(1L)
            .subscribe().withSubscriber(UniAssertSubscriber.create());

        subscriber.awaitItem()
            .assertItem(expectedOrder)
            .assertCompleted();
    }

    @Test
    void failsForUnknownId() {
        service.findById(999L)
            .subscribe().withSubscriber(UniAssertSubscriber.create())
            .awaitFailure()
            .assertFailedWith(OrderNotFoundException.class, "999");
    }
}
```



```
}

@Test
void streamsInOrder() {
    AssertSubscriber<Order> subscriber = service.streamAll()
        .subscribe().withSubscriber(AssertSubscriber.create(10));    // request 10

    subscriber.awaitCompletion()
        .assertItems(order1, order2, order3);
}

@Test
void respectsBackpressure() {
    AssertSubscriber<Order> subscriber = service.streamAll()
        .subscribe().withSubscriber(AssertSubscriber.create(2));    // request only 2

    subscriber.awaitNextItems(2).assertNotTerminated();
    subscriber.request(1).awaitNextItem();
}
}
```

10.10.2 Blocking in tests

```
@Test
void quickAssertion() {
    Order order = service.findById(1L).await().atMost(Duration.ofSeconds(5));
    assertThat(order.status).isEqualTo(OrderStatus.PENDING);
}
```

`await()` is fine in tests. **Never use it in production code** — it blocks the calling thread, which on an event loop is exactly the failure this whole chapter exists to avoid.

10.10.3 Virtual time

```
@Test
void retriesWithBackoff() {
    AtomicInteger attempts = new AtomicInteger();

    Uni<String> unstable = Uni.createFrom().item(() -> {
        if (attempts.incrementAndGet() < 3) throw new RuntimeException("boom");
        return "ok";
    })
        .onFailure().retry().withBackOff(Duration.ofMillis(10)).atMost(5);

    assertThat(unstable.await().indefinitely()).isEqualTo("ok");
    assertThat(attempts.get()).isEqualTo(3);
}
```

Keep test backoff durations tiny. A retry test with production backoff values makes your suite take minutes.

10.11 Best Practices

1. **Never block in a reactive pipeline.** Use `runSubscriptionOn` for a blocking source, or `@Blocking` on the endpoint.
2. **Always subscribe.** A `Uni` you build and discard does nothing.
3. **`item(supplier)`, not `item(value)`,** when the value comes from work you want deferred.
4. **Bound merge concurrency.** Unbounded `merge` over a large stream is a self-inflicted denial of service.
5. **Declare an overflow strategy** on any `Multi` from a source you cannot slow down.
6. **Timeout every outbound call**, with per-call budgets that fit inside the endpoint's overall deadline.
7. **Retry with exponential backoff and jitter**, on a selective predicate, only for idempotent operations.
8. **`call`, not `invoke`,** for asynchronous side effects.
9. **Pass identity and tenant explicitly** across asynchronous boundaries rather than relying on implicit context.
10. **Prefer virtual threads for request/response**; use Mutiny for streaming, backpressure and complex composition.
11. **Never `await()` in production code.**
12. **Test with `AssertSubscriber`**, including a backpressure test for any `Multi` you expose.

10.12 Common Mistakes

1. **Building a `Uni` and not subscribing.** The most common reactive bug; it fails silently.
2. **`Uni.createFrom().item(blockingCall())`.** Evaluated eagerly on the current thread — usually an event loop.
3. **Blocking inside `map`.** A JDBC call in a reactive pipeline stalls an event loop.
4. **`invoke` for an asynchronous side effect.** The returned `Uni` is never subscribed, so the effect never happens.
5. **Unbounded `merge`.** Opens one concurrent call per item.
6. **No overflow strategy on an unbounded `Multi`.** Slow consumer $\Rightarrow$ unbounded buffer $\Rightarrow$ OOM.
7. **Retrying non-idempotent operations.** Duplicate charges.
8. **Retry without jitter.** Synchronised thundering herd on a recovering service.
9. **`await()` in production.** Blocks the calling thread and defeats the whole model.
10. **Assuming `@RequestScoped` state survives a thread hop.** It does not without context propagation.
11. **Mixing `@RunOnVirtualThread` with `Uni` returns.** Pick one model per endpoint.
12. **Adopting reactive because it is “faster”.** It is not faster per request; it is more concurrent per thread. If the bottleneck is a connection pool, nothing improves.

10.13 Interview Questions

Junior

Q1. What is Mutiny? Quarkus's reactive programming library, with two types: `Uni<T>` for at most one item and `Multi<T>` for a stream.

Q2. Uni versus Multi? `Uni` emits at most one item then completes — an HTTP call, a database read. `Multi` emits zero to many with backpressure — a result set, a Kafka topic, SSE.

Q3. What happens if you do not subscribe? Nothing. `Uni` and `Multi` are lazy descriptions of work. Returning one from a resource method makes Quarkus subscribe for you; elsewhere you must subscribe explicitly.

Q4. How do you transform a value? `.onItem().transform(fn)` for a synchronous transformation, or the alias `.map(fn)`. For an asynchronous one, `.onItem().transformToUni(fn)` or `.chain(fn)`.

Q5. How do you handle a failure? `.onFailure().recoverWithItem(default)`, `.onFailure(SomeException.class).recoverWithUni(...)`, or `.onFailure().transform(...)` to translate it.

Q6. How do you add a timeout? `.ifNoItem().after(Duration.ofSeconds(2)).fail()`, or `.recoverWithItem(fallback)`.

Mid-level

Q7. Explain invoke versus call. `invoke` takes a `Consumer` and runs a synchronous side effect without waiting. `call` takes a function returning a `Uni`, subscribes to it, waits for it to complete, then emits the original item. Using `invoke` for an asynchronous action means the returned `Uni` is never subscribed and the action never happens.

Q8. How do you run several calls in parallel? `Uni.combine().all().unis(a, b, c).with((x, y, z) -> ...)`. All subscribe at once, so total latency is the slowest rather than the sum. `Uni.combine().any().of(...)` races them and cancels the losers.

Q9. What is backpressure? A protocol where the subscriber requests a bounded number of items and the publisher may not exceed it. Demand flows upstream, data flows downstream. It prevents a fast producer from overwhelming a slow consumer, and it is the property that distinguishes Reactive Streams from callbacks — and the one virtual threads do not provide.

Q10. What happens with an unbounded source? Sources like timers or a broker without flow control cannot be slowed, so you must declare an overflow strategy: `buffer(n)`, `drop()`, `dropPreviousItems()`. Without one, items accumulate until the process dies.

Q11. runSubscriptionOn versus emitOn? `runSubscriptionOn` sets the thread on which subscription — and therefore the source's work — happens; use it to move a blocking source off the event loop. `emitOn` sets the thread on which downstream operators receive events; use it to move expensive processing off the event loop after a non-blocking source.

Q12. Why does retry need jitter? Without it, every client that failed at the same instant retries at the same instant, producing a synchronised load spike that can prevent a recovering service from recovering. Jitter randomises each delay so retries spread out.

Q13. How do you propagate the request context? `quarkus-smallrye-context-propagation` propagates CDI, transaction and security context along Mutiny chains automatically. Crossing to a `ManagedExecutor` also works. Raw threads and plain `CompletableFuture` do not; use `ThreadContext.contextualRunnable`.

Q14. How do you test reactive code? `UniAssertSubscriber` and `AssertSubscriber` with `awaitItem`, `assertItem`, `assertFailedWith`, and explicit `request(n)` to test backpressure. `await().atMost(...)` is acceptable in tests for simple assertions but never in production code.

Senior

Q15. Explain Mutiny’s API design and why it differs from Reactor. Reactor and RxJava expose a large flat operator surface where the distinction between `flatMap`, `concatMap`, `switchMap` and `mergeMap` must be memorised and is easy to get wrong. Mutiny instead makes you name the event group first — `onItem()`, `onFailure()`, `onCancellation()`, `onTermination()` — and then the action, so the API is navigable by autocompletion and the code states its intent: `onItem().transformToUniAndConcatenate()` says sequential where `concatMap` requires you to know it. The trade is verbosity, which Mutiny mitigates with aliases like `map`, `chain` and `call` for the common cases. There is also a deliberate design choice to avoid a `Single/Maybe/Completable` split: `Uni<Void>` covers completion and `Uni<T>` may emit null, which removes a category of type-juggling.

Q16. When would you use Mutiny over virtual threads on Java 21? Three cases, and outside them I would default to virtual threads. First, streaming: `Multi` with backpressure is the right abstraction for SSE, large exports and Kafka pipelines, and virtual threads have no equivalent. Second, backpressure specifically: virtual threads make it cheap to have ten thousand concurrent requests, which makes it cheap to overwhelm a downstream service; only a demand protocol prevents that. Third, complex composition with cancellation — “fan out to five services, take the first three, cancel the rest, total budget two hundred milliseconds” — which Mutiny expresses declaratively and which is fiddly even with `StructuredTaskScope`. For a normal endpoint that reads a database and calls one or two services, virtual threads win on readability, debuggability and hireability at essentially no performance cost, and I would not pay the reactive tax for it.

Q17. Your reactive service has high latency under load but low CPU. Diagnose it. Low CPU with high latency means threads are waiting, and in a reactive service the first hypothesis is always that something is blocking an event loop — so I would check for `BlockedThreadChecker` warnings before anything else, because a single blocking call in a `map` stalls every connection pinned to that loop and produces exactly this signature. If the event loops are clean, the next candidate is a bounded resource behind the reactive layer: a connection pool, or a downstream service with limited concurrency, where the reactive code is simply queueing efficiently on something that has not got faster. I would look at pool acquisition times and at downstream latency percentiles to distinguish those. Third, unbounded `merge` — if the code fans out per item without a concurrency limit, the service may be overwhelming its own downstream and the latency is self-inflicted; the fix is a bounded merge, not more capacity. Fourth, missing or badly-nested timeouts, where a slow downstream holds pipelines open far longer than the endpoint’s SLO. I would instrument with Micrometer timers around each stage before theorising further, because reactive stack traces are unhelpful and guessing is expensive.

Q18. How do you design a reactive pipeline that consumes from Kafka and writes to a database? The shape is: consume, batch, write, commit — with backpressure end to end and idempotency at the sink. Concretely, a `Multi` from the Kafka connector with the consumer’s

own `max.poll.records` acting as the primary flow control, then `group().intoLists().of(n)` or `.every(duration)` to batch, then `transformToUniAndConcatenate` to write each batch so writes are sequential and ordering is preserved within a partition. The database write is a bulk upsert keyed on the event id, which makes it idempotent, because delivery is at-least-once and a redelivery after a crash is guaranteed to happen eventually. Offsets are committed only after the write succeeds — throttled commit strategy rather than latest — so a crash replays rather than skips. Failures need an explicit policy: retry with backoff for transient database errors, and a dead-letter topic for poison messages, because a message that will never succeed must not block the partition forever. I would bound concurrency deliberately at one batch in flight per partition, since going wider trades ordering for throughput and that is a decision the domain should make, not the framework. Finally, metrics on lag, batch size and write latency, because consumer lag is the signal that tells you whether the pipeline is keeping up.

Principal

Q19. A team wants to rewrite a working blocking service as fully reactive to “improve performance”. Evaluate. I would ask for the number they are trying to move, because in my experience this request usually arrives without one. If the service is latency-bound, reactive will not help — it does not make a single request faster. If it is throughput-bound, the question is what the actual constraint is: if it is a twenty-connection database pool, reactive moves the queue from threads to the pipeline and throughput is unchanged, and I have seen teams spend a quarter discovering that. Reactive helps when threads themselves are the scarce resource, which in practice means high concurrency with long waits on I/O — and on Java 21, virtual threads address that case with a fraction of the change and none of the readability cost. So my first counter-proposal is almost always: measure, then try `@RunOnVirtualThread` on the hot endpoints, which is a one-line change and can be reverted. Against the benefit I would put the real costs: the codebase becomes harder to read and much harder to debug because stack traces stop corresponding to logical flow; onboarding slows; and the team acquires a category of subtle bug — unsubscribed pipelines, missing overflow strategies, accidental blocking — that they have no experience detecting. I would support a reactive rewrite where the service genuinely streams, needs backpressure, or fans out to many services with cancellation semantics. I would not support it as a performance project without a measurement showing threads are the constraint.

Q20. How do you make reactive code maintainable across a team with mixed experience? The failure mode is not that people write incorrect pipelines; it is that they write pipelines nobody else can safely change. So I would optimise for legibility over cleverness. Concretely: a house style that keeps pipelines short and names intermediate steps as private methods rather than chaining twenty operators, so each line reads as a sentence; a rule that every pipeline has an explicit timeout and, for `Multi`, an explicit overflow strategy, enforced in review because both failures are invisible until production; and a small set of shared helpers for the recurring patterns — retry-with-jitter, bounded fan-out, per-call budget derived from a request deadline — so people compose vetted pieces rather than reinventing them. I would restrict reactive to the parts of the system that need it and use virtual threads elsewhere, because a codebase where every endpoint is reactive “for consistency” imposes the cost everywhere while gaining the benefit in a few places. Testing matters disproportionately: `AssertSubscriber` tests including a backpressure case for anything streaming, because those are the bugs that do not reproduce locally. And I would invest in observability early — timers per pipeline stage and structured logging with a correlation id — since the usual debugging tools degrade sharply here and the team’s ability to operate the system is the real constraint on how much reactive code you can afford.

10.14 Cheat Sheet

Creating

```
Uni.createFrom().item("x");           Uni.createFrom().item(() -> compute());
Uni.createFrom().failure(new E());     Uni.createFrom().nullItem();
Uni.createFrom().completionStage(() -> cf);
Multi.createFrom().items(1, 2, 3);     Multi.createFrom().range(0, 100);
Multi.createFrom().ticks().every(Duration.ofSeconds(1));
```

Transforming

```
.onItem().transform(fn)                .map(fn)
.onItem().transformToUni(fn)           .chain(fn)
.onItem().invoke(consumer)             // sync side effect
.onItem().call(fn -> Uni)              // async side effect, waits
.replaceWith(x) .replaceWithVoid()
```

Multi

```
.filter(p) .select().first(n) .select().distinct() .skip().first(n)
.group().intoLists().of(50)
.group().intoLists().every(Duration.ofSeconds(1))
.collect().asList()                  // Multi -> Uni<List>
.onItem().transformToUniAndConcatenate(fn) // sequential
.onItem().transformToUniAndMerge(fn)      // concurrent, UNBOUNDED
.onItem().transformToUni(fn).merge(8)     // concurrent, bounded
```

Combining

```
Uni.combine().all().unis(a, b, c).with((x,y,z) -> r);
Uni.combine().all().unis(list).collectFailures().with(...);
Uni.combine().any().of(a, b, c);           // race, cancel losers
```

Failure

```
.onFailure().recoverWithItem(fallback)
.onFailure(E.class).recoverWithUni(() -> alt)
.onFailure().transform(t -> new MyEx(t))
.onFailure(pred).retry().withBackOff(min, max).withJitter(0.3).atMost(4)
.ifNoItem().after(Duration.ofSeconds(2)).fail()
.onTermination().invoke(i, f, c) -> cleanup()
```

Backpressure

```
.onOverflow().buffer(1000)
.onOverflow().drop()
.onOverflow().dropPreviousItems()
.paceDemand().using(new FixedDemandPacer(10, Duration.ofSeconds(1)))
```

Threading

```
.runSubscriptionOn(Infrastructure.getDefaultWorkerPool()) // source thread  
.emitOn(Infrastructure.getDefaultWorkerPool())           // downstream thread
```

Testing

```
uni.subscribe().withSubscriber(UniAssertSubscriber.create())  
    .awaitItem().assertItem(x).assertCompleted();  
multi.subscribe().withSubscriber(AssertSubscriber.create(2))  
    .awaitNextItems(2).request(1);  
uni.await().atMost(Duration.ofSeconds(5)); // TESTS ONLY
```

10.15 Summary

Mutiny has two types and one organising idea: name the event, then the action. **Uni** for at most one item, **Multi** for a stream with backpressure. Both are lazy, so nothing happens until something subscribes — which is the source of the most common reactive bug and the first thing to check when async code silently does nothing.

The capability that justifies the model is **backpressure**: a subscriber requests a bounded number of items and the publisher may not exceed it. That is what separates Reactive Streams from callbacks, and it is precisely what virtual threads do not offer. Virtual threads give you the execution-model benefit — high concurrency without many OS threads — with imperative, debuggable code. So on Java 21 the honest division is: **virtual threads for request/response, Mutiny for streaming, backpressure and complex composition with cancellation**.

Whichever you choose, the operational disciplines are the same and non-negotiable: a timeout on every outbound call with budgets that nest inside the endpoint's deadline; retries with exponential backoff and jitter, on a selective predicate, only for idempotent operations; bounded concurrency on any fan-out; and an explicit overflow strategy on any stream from a source you cannot slow down.

Chapter 11 applies all of this to **Kafka and messaging** — the reactive messaging model, delivery guarantees, offset strategies, dead-letter handling, and the exactly-once question that every event-driven interview reaches.

10.16 References

- Mutiny — <https://smallrye.io/smallrye-mutiny/>
- Mutiny guides — <https://smallrye.io/smallrye-mutiny/latest/guides/>
- Quarkus reactive architecture — <https://quarkus.io/guides/quarkus-reactive-architecture>
- Getting started with reactive — <https://quarkus.io/guides/getting-started-reactive>
- Virtual threads in Quarkus — <https://quarkus.io/guides/virtual-threads>
- Context propagation — <https://quarkus.io/guides/context-propagation>
- Reactive Streams specification — <https://www.reactive-streams.org/>
- JEP 444, Virtual Threads — <https://openjdk.org/jeps/444>

Kafka and Messaging

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · SmallRye Reactive Messaging 4.x
· Apache Kafka 3.9 · Avro / Apicurio

Learning Objectives

By the end of this chapter you will be able to:

1. Explain Kafka’s model — topics, partitions, offsets, consumer groups — well enough to reason about ordering and scaling.
2. Produce and consume with SmallRye Reactive Messaging using `@Outgoing`, `@Incoming` and `Emitter`.
3. Choose an **acknowledgement and commit strategy** deliberately, and explain the failure mode of each.
4. State the delivery guarantees honestly, and design for **at-least-once with idempotent consumers**.
5. Implement dead-letter handling and understand why a poison message blocks a partition.
6. Design partition keys so that ordering guarantees match your domain’s requirements.
7. Use schemas — Avro or JSON Schema — with a registry, and evolve them compatibly.
8. Apply the **outbox pattern** from Chapter 7 to publish reliably.
9. Test Kafka messaging with in-memory connectors and with a real broker via Dev Services.
10. Answer 25 interview questions, including the exactly-once question that appears in every event-driven interview.

11.1 Kafka in One Page

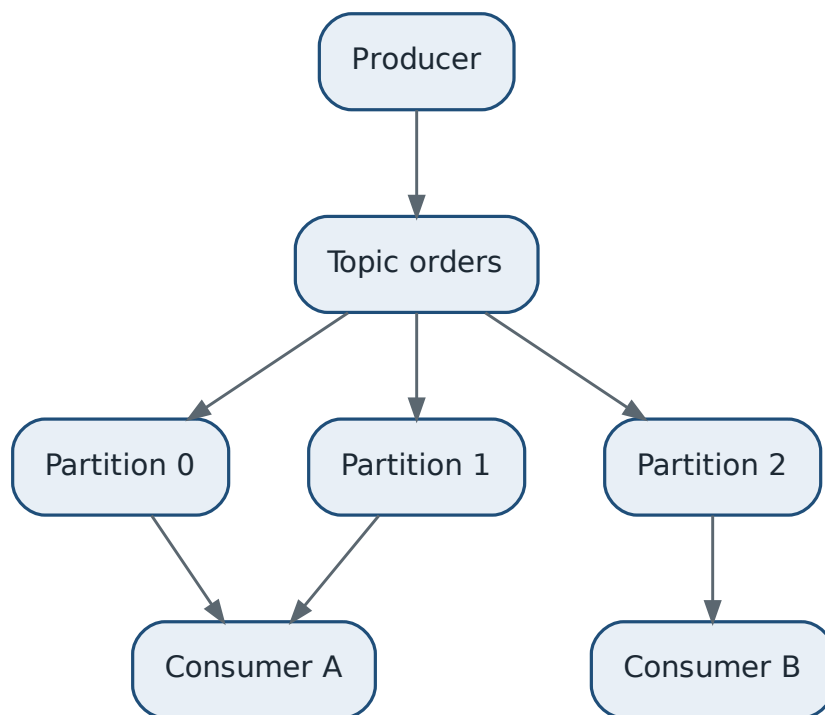


Figure 11.1 — A topic is split into partitions; each partition is assigned to exactly one consumer in a group.

The facts that determine every design decision:

- **A topic is an append-only log**, split into **partitions**.
- **Ordering is guaranteed within a partition**, never across a topic.
- **The partition is chosen by the message key** — same key, same partition, therefore ordered relative to each other.
- **A consumer group** divides partitions among its members. Each partition goes to exactly **one** consumer in the group.
- **Therefore: parallelism within a group is capped by the partition count.** Ten consumers on a three-partition topic means seven idle consumers.
- **An offset** is a consumer group's bookmark per partition. Committing it is what makes progress durable.
- **Messages are retained by policy**, not by consumption. Multiple groups read the same data independently.

The consequence people miss: ordering and parallelism are in direct tension. Strict ordering across all orders means one partition means one consumer. Ordering *per customer* means keying

by customer id, which gives you as much parallelism as you have customers, spread across partitions. Choosing the key is choosing your ordering guarantee, and it is very hard to change later because it repartitions all your data.

11.2 Setup

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-messaging-kafka</artifactId>
</dependency>
```

```
kafka.bootstrap.servers=kafka-1:9092,kafka-2:9092,kafka-3:9092
```

```
# --- Outgoing channel -----
mp.messaging.outgoing.orders-out.connector=smallrye-kafka
mp.messaging.outgoing.orders-out.topic=orders
mp.messaging.outgoing.orders-out.key.serializer=org.apache.kafka.common.serialization.StringSe
  ↪ rializer
mp.messaging.outgoing.orders-out.value.serializer=io.quarkus.kafka.client.serialization.Object
  ↪ MapperSerializer
mp.messaging.outgoing.orders-out.acks=all
mp.messaging.outgoing.orders-out.enable.idempotence=true
mp.messaging.outgoing.orders-out.max.in.flight.requests.per.connection=5
mp.messaging.outgoing.orders-out.compression.type=snappy
mp.messaging.outgoing.orders-out.linger.ms=20
mp.messaging.outgoing.orders-out.batch.size=32768

# --- Incoming channel -----
mp.messaging.incoming.orders-in.connector=smallrye-kafka
mp.messaging.incoming.orders-in.topic=orders
mp.messaging.incoming.orders-in.group.id=orders-processor
mp.messaging.incoming.orders-in.auto.offset.reset=earliest
mp.messaging.incoming.orders-in.key.deserializer=org.apache.kafka.common.serialization.StringD
  ↪ eserializer
mp.messaging.incoming.orders-in.value.deserializer=com.acme.orders.OrderEventDeserializer
mp.messaging.incoming.orders-in.enable.auto.commit=false
mp.messaging.incoming.orders-in.commit-strategy=throttled
mp.messaging.incoming.orders-in.failure-strategy=dead-letter-queue
mp.messaging.incoming.orders-in.dead-letter-queue.topic=orders-dlq
mp.messaging.incoming.orders-in.max.poll.records=100
mp.messaging.incoming.orders-in.max.poll.interval.ms=300000
```

Three settings above are load-bearing and worth memorising:

- **acks=all** — the leader waits for all in-sync replicas. **acks=1** loses messages on leader failure. There is essentially no reason to use anything else for business data.
- **enable.idempotence=true** — the producer deduplicates its own retries, so a network-level retry does not create a duplicate. Free, and it also enables safe ordering with in-flight requests.
- **enable.auto.commit=false** — auto-commit commits on a timer regardless of whether you processed the message, which turns at-least-once into at-most-once and loses data on crash. Always disable it and let the connector manage offsets.

11.3 Producing

```
@ApplicationScoped
public class TickProducer {

    @Outgoing("heartbeats-out")
    public Multi<String> heartbeat() {
        return Multi.createFrom().ticks().every(Duration.ofSeconds(30))
            .map(tick -> Instant.now().toString());
    }
}
```

Useful for genuinely stream-shaped sources. Most application code produces in response to something instead, which means `Emitter`.

```
@ApplicationScoped
public class OrderEventPublisher {

    @Inject
    @Channel("orders-out")
    MutinyEmitter<OrderEvent> emitter;

    public Uni<Void> publish(Order order) {
        OrderEvent event = OrderEvent.from(order);

        Message<OrderEvent> message = Message.of(event)
            .addMetadata(OutgoingKafkaRecordMetadata.<String>builder()
                .withKey(order.customerId.toString()) // ordering per customer
                .withTopic("orders")
                .withHeaders(new RecordHeaders()
                    .add("event-type", "OrderCreated".getBytes(UTF_8))
                    .add("correlation-id", correlationId().getBytes(UTF_8))
                    .add("schema-version", "2".getBytes(UTF_8)))
                .build());

        return emitter.sendMessage(message);
    }
}
```

```
# Backpressure on the emitter - what happens when the broker is slow
mp.messaging.outgoing.orders-out.overflow-strategy=buffer
mp.messaging.outgoing.orders-out.overflow-buffer-size=1024
# alternatives: latest, drop, fail, unbounded-buffer (never use the last one)
```

Emitter versus MutinyEmitter. `Emitter.send()` returns a `CompletionStage`; `MutinyEmitter.send()` returns a `Uni`. Prefer `MutinyEmitter` in Quarkus for consistency with the rest of the

stack. In both cases, **the returned handle completes when the broker acknowledges** — ignoring it means you do not know whether the message was published, which is usually a bug.

11.3.3 The publish-after-commit problem

```
// BROKEN: if the transaction rolls back, the event has already been published.
// If the process dies after commit but before send, the event is lost.
@Transactional
public Order create(CreateOrderRequest request) {
    Order order = persist(request);
    emitter.send(OrderEvent.from(order));    // not atomic with the commit
    return order;
}
```

There is no way to make a database commit and a Kafka publish atomic. The correct answer is the **transactional outbox** from Chapter 7:

```
@Transactional
public Order create(CreateOrderRequest request) {
    Order order = persist(request);
    OutboxEvent.record("Order", order.id, "OrderCreated", json(order)); // same tx
    return order;
}
```

...with a separate publisher (polling, or Debezium CDC) draining the outbox. Delivery becomes at-least-once, which is fine because your consumers are idempotent.

A partial mitigation when an outbox is genuinely too heavy: publish from an after-commit observer, which at least removes the phantom-event case.

```
void publish(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) {
    emitter.send(e);    // never publishes on rollback; may still be lost on crash
}
```

11.4 Consuming

11.4.1 Signatures

```
@ApplicationScoped
public class OrderEventConsumer {

    // Simplest: payload in, void out. Auto-acknowledged on return.
    @Incoming("orders-in")
    @Blocking                                // JDBC work needs a worker thread
    @Transactional
    public void consume(OrderEvent event) {
        orderRepository.upsert(event);
    }

    // Message: full access to metadata and manual acknowledgement
}
```

```

@Incoming("orders-in")
public Uni<Void> consumeMessage(Message<OrderEvent> message) {
    var metadata = message.getMetadata(IncomingKafkaRecordMetadata.class).orElseThrow();
    String key      = (String) metadata.getKey();
    int    partition = metadata.getPartition();
    long    offset   = metadata.getOffset();

    return process(message.getPayload())
        .onItem().transformToUni(v -> Uni.createFrom().completionStage(message.ack()))
        .onFailure().recoverWithUni(t ->
            Uni.createFrom().completionStage(message.nack(t)));
}

// Transform: consume from one topic, produce to another
@Incoming("orders-in")
@Outgoing("enriched-out")
public OrderEvent enrich(OrderEvent event) {
    return event.withCustomer(customerCache.get(event.customerId()));
}

// Stream processing
@Incoming("orders-in")
@Outgoing("batched-out")
public Multi<List<OrderEvent>> batch(Multi<OrderEvent> stream) {
    return stream.group().intoLists().every(Duration.ofSeconds(5));
}
}

```

11.4.2 The blocking rule applies here too

A `@Incoming` method that returns a plain type runs on an **event loop** by default — this surprises people who expect the Chapter 2 heuristic to be reversed for messaging. A JDBC write inside it blocks an event loop on every message.

```

@Incoming("orders-in")
@Blocking                                     // dispatch to a worker thread
@Transactional
public void consume(OrderEvent event) { ... }

@Incoming("orders-in")
@RunOnVirtualThread                         // Java 21 alternative
@Transactional
public void consumeVirtual(OrderEvent event) { ... }

// Ordered blocking - the default for @Blocking; process one at a time per partition
@Incoming("orders-in")
@Blocking(ordered = true)
public void ordered(OrderEvent event) { ... }

// Named worker pool with bounded concurrency
@Incoming("orders-in")

```

```
@Blocking("order-pool")
public void pooled(OrderEvent event) { ... }
```

```
smallrye.messaging.worker.order-pool.max-concurrency=8
```

@Blocking(ordered = false) is a correctness decision, not a performance tweak. It allows messages from the same partition to be processed concurrently, which discards Kafka's ordering guarantee. If your domain requires that `OrderUpdated` is applied after `OrderCreated`, unordered processing will corrupt state under load — and it will work perfectly in testing.

11.4.3 Acknowledgement strategies

```
@Incoming("orders-in")
@Acknowledgment(Acknowledgment.Strategy.POST_PROCESSING) // default: ack after success
public void afterProcessing(OrderEvent e) { }

@Incoming("orders-in")
@Acknowledgment(Acknowledgment.Strategy.PRE_PROCESSING) // ack first: at-most-once
public void beforeProcessing(OrderEvent e) { }

@Incoming("orders-in")
@Acknowledgment(Acknowledgment.Strategy.MANUAL) // you call ack()/nack()
public CompletionStage<Void> manual(Message<OrderEvent> m) {
    return process(m.getPayload()).thenCompose(v -> m.ack());
}
```

`POST_PROCESSING` is the default and correct choice: the offset advances only after your method succeeds, so a crash replays the message. That is at-least-once, which is what you want.

11.4.4 Commit strategies

```
mp.messaging.incoming.orders-in.commit-strategy=throttled # default
# alternatives: latest, ignore
mp.messaging.incoming.orders-in.throttled.unprocessed-record-max-age.ms=60000
```

- **throttled** — tracks acknowledgements and periodically commits the highest offset below which everything is acknowledged. Efficient and safe. It will fail the channel if a record goes unacknowledged for longer than `unprocessed-record-max-age.ms`, which is a deliberate safety valve against a stuck consumer silently falling behind forever.
- **latest** — commits after every message. Safer on paper, materially slower, and rarely necessary.
- **ignore** — you manage offsets yourself. Only with `enable.auto.commit=true`, which you should not use.

11.5 Delivery Guarantees

The question every event-driven interview reaches.

Guarantee	Achievable?	How
At-most-once	Yes	Commit before processing. Loses messages on crash.
At-least-once	Yes — the default	Commit after processing. Duplicates on crash.
Exactly-once <i>delivery</i>	No	Impossible across a network with failures.
Exactly-once <i>effect</i>	Yes	At-least-once delivery + idempotent consumer.

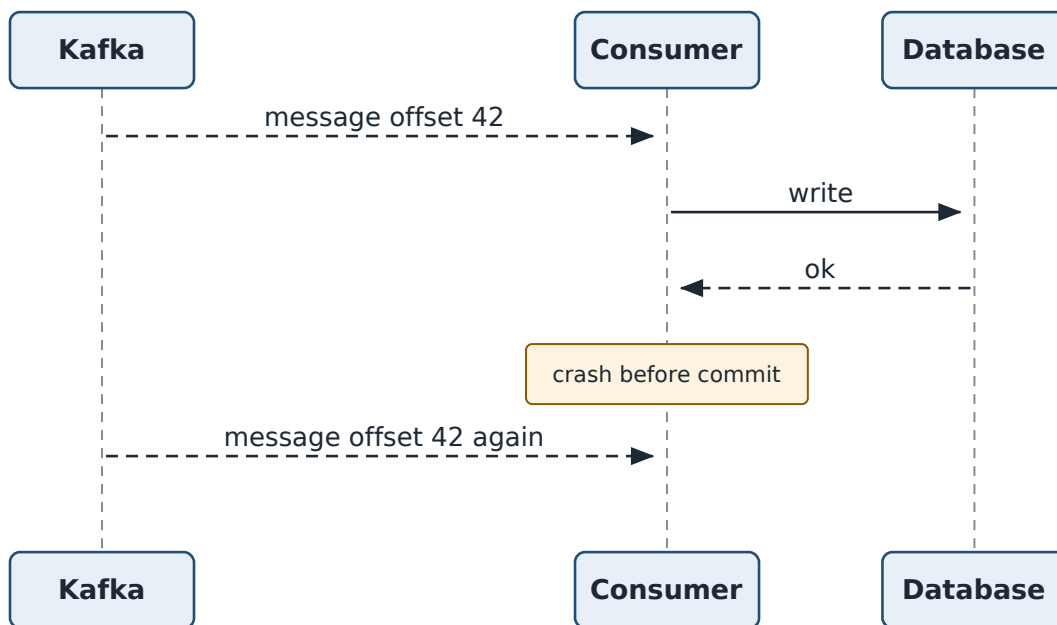


Figure 11.2 — The window between processing and offset commit is why at-least-once is the realistic guarantee.

11.5.1 The idempotent consumer

```

@Entity
@Table(name = "processed_events",
        uniqueConstraints = @UniqueConstraint(columnNames = "event_id"))
public class ProcessedEvent extends PanacheEntityBase {
    @Id @Column(name = "event_id", length = 64)
    public String eventId;
    public Instant processedAt;
}

```

```

@ApplicationScoped
public class IdempotentOrderConsumer {

```

```

@Incoming("orders-in")
@Blocking
@Transactional
public void consume(OrderEvent event) {
    try {
        ProcessedEvent marker = new ProcessedEvent();
        marker.eventId      = event.eventId();
        marker.processedAt = Instant.now();
        marker.persistAndFlush();           // flush forces the constraint check NOW
    } catch (PersistenceException e) {
        Log.debugf("Duplicate event %s ignored", event.eventId());
        return;                             // already processed
    }

    orderRepository.apply(event);           // same transaction as the marker
}

```

Two details that matter:

persistAndFlush, not persist. Without the flush, the constraint violation surfaces at commit — outside your `try` block, per Chapter 7 §7.6.2.

The unique constraint is the mechanism, not the check. A `if (exists) return;` before the insert is racy: two concurrent deliveries both see nothing and both proceed. The constraint is what makes it correct.

Better still, where the domain allows: make the operation **naturally idempotent** and skip the bookkeeping.

```

// An upsert keyed on the business id is idempotent by construction
@Incoming("orders-in")
@Blocking
@Transactional
public void consume(OrderEvent event) {
    Order.update("""
        status = ?1, total = ?2, updatedAt = ?3
        where externalId = ?4 and updatedAt < ?3
        """, event.status(), event.total(), event.occurredAt(), event.orderId());
}

```

The `updatedAt < ?3` guard also makes it safe against **out-of-order** delivery, which deduplication alone does not.

11.5.2 Kafka transactions

Kafka does support transactions for the consume-transform-produce pattern *within Kafka*:

```

@Inject KafkaTransactions<OrderEvent> transaction;

@Incoming("orders-in")
public Uni<Void> transform(Message<InputEvent> message) {
    return transaction.withTransaction(emitter -> {

```



```

        emitter.send(Message.of(transform(message.getPayload())));
        return Uni.createFrom().voidItem();
    }).onItem().transformToUni(v -> Uni.createFrom().completionStage(message.ack()));
}

```

This gives exactly-once semantics **between Kafka topics**, because the offset commit and the produce are in one Kafka transaction. It does **not** extend to your database — that boundary is still at-least-once, and still needs idempotency.

11.6 Error Handling

11.6.1 Failure strategies

```

mp.messaging.incoming.orders-in.failure-strategy=dead-letter-queue
mp.messaging.incoming.orders-in.dead-letter-queue.topic=orders-dlq
mp.messaging.incoming.orders-in.dead-letter-queue.key.serializer=...
mp.messaging.incoming.orders-in.dead-letter-queue.value.serializer=...

```

Strategy	Behaviour
<code>fail</code> (default)	Stop the channel. The application keeps running but consumes nothing.
<code>ignore</code>	Log and skip. Silently loses the message.
<code>dead-letter-queue</code>	Publish to a DLQ topic and continue.
<code>delayed-retry-topic</code>	Retry via delay topics, then DLQ.

The default is `fail`, and it is a poor default for production. One malformed message stops the channel; the pod stays alive and healthy-looking while consumer lag grows without bound. Configure `dead-letter-queue` explicitly, and alert on DLQ depth.

11.6.2 Why a poison message blocks a partition

Ordering within a partition means the consumer cannot skip a message and come back to it — the offset is a single high-water mark. A message that always fails therefore blocks every subsequent message in that partition indefinitely. That is why a DLQ is not optional at scale: it is the mechanism that lets the partition make progress.

11.6.3 Retry before dead-lettering

```

@Incoming("orders-in")
public Uni<Void> consume(Message<OrderEvent> message) {
    return process(message.getPayload())
        .onFailure(this::isTransient)
        .retry().withBackOff(Duration.ofMillis(200), Duration.ofSeconds(5))
        .withJitter(0.3).atMost(3)
}

```

```

    .onItem().transformToUni(v -> Uni.createFrom().completionStage(message.ack()))
    .onFailure().recoverWithUni(t -> {
        Log.errorf(t, "Dead-lettering event %s", message.getPayload().eventId());
        return Uni.createFrom().completionStage(message.nack(t));
    });
}

```

Distinguish clearly:

- **Transient failures** (database down, downstream timeout) — retry. Dead-lettering these would move a recoverable problem into a queue nobody drains.
- **Permanent failures** (malformed payload, unknown schema, business rule violation) — dead-letter immediately. Retrying is pure waste and delays the partition.

Delayed retry topics: progressive backoff without blocking the main partition

mp.messaging.incoming.orders-in.failure-strategy=delayed-retry-topic

mp.messaging.incoming.orders-in.delayed-retry-topic.topics=orders-retry-1s,orders-retry-10s,orders-retry-1m

mp.messaging.incoming.orders-in.delayed-retry-topic.max-retries=3

11.6.4 Draining the DLQ

A DLQ that nobody reads is a data-loss mechanism with extra steps. Minimum viable operations:

- **Alert on DLQ depth**, not just on rate.
- **Keep the failure reason** — SmallRye adds `dead-letter-reason` and `dead-letter-cause` headers.
- **Provide a replay path** — an admin endpoint or job that reads the DLQ and republishes to the source topic after the bug is fixed.

11.7 Schemas and Evolution

JSON with Jackson is fine for internal, low-stakes topics. For anything crossing team boundaries, use a schema registry.

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-apicurio-registry-avro</artifactId>
</dependency>

```

mp.messaging.outgoing.orders-out.value.serializer=io.apicurio.registry.serde.avro.AvroKafkaSerializer

mp.messaging.incoming.orders-in.value.deserializer=io.apicurio.registry.serde.avro.AvroKafkaDeserializer

mp.messaging.connector.smallrye-kafka.apicurio.registry.url=http://apicurio:8080/apis/registry/v2

mp.messaging.connector.smallrye-kafka.apicurio.registry.auto-register=true

mp.messaging.incoming.orders-in.apicurio.registry.use-specific-avro-reader=true

```

{
  "type": "record",

```

```

"name": "OrderEvent",
"namespace": "com.acme.orders.events",
"fields": [
  {"name": "eventId", "type": "string"},
  {"name": "orderId", "type": "long"},
  {"name": "status", "type": {"type": "enum", "name": "Status",
    "symbols": ["PENDING", "PAID", "SHIPPED", "CANCELLED"]}},
  {"name": "total", "type": {"type": "bytes", "logicalType": "decimal",
    "precision": 19, "scale": 2}},
  {"name": "occurredAt", "type": {"type": "long", "logicalType": "timestamp-millis"}},
  {"name": "note", "type": ["null", "string"], "default": null}
]
}

```

11.7.1 Compatible evolution

Change	Backward compatible?
Add a field with a default	Yes
Add a field without a default	No
Remove a field that has a default	Yes
Rename a field	No — use an alias
Change a type	Only for widening (<code>int</code> → <code>long</code>)
Add an enum symbol	Only with a default on the enum

Set the registry’s compatibility rule to **BACKWARD and enforce it in CI.** An incompatible schema deployed to production breaks every consumer at once, and because consumers are decoupled you will not find out from a failing build — you will find out from consumer lag alerts across several teams. Checking compatibility at build time is a five-minute setup that prevents a category of cross-team incident.

11.8 Other Brokers

The same programming model works across connectors — only configuration changes:

```

# RabbitMQ / AMQP 1.0
mp.messaging.outgoing.orders-out.connector=smallrye-rabbitmq
mp.messaging.outgoing.orders-out.exchange.name=orders
mp.messaging.outgoing.orders-out.routing-keys=order.created

# Pulsar
mp.messaging.incoming.orders-in.connector=smallrye-pulsar

# MQTT
mp.messaging.incoming.sensors.connector=smallrye-mqtt

# In-memory (tests, or intra-application decoupling)
mp.messaging.outgoing.orders-out.connector=smallrye-in-memory

```

Choosing a broker briefly:

- **Kafka** — high throughput, retention, replay, stream processing. Ordering per partition. The default for event-driven architectures.
- **RabbitMQ** — flexible routing, per-message acknowledgement, priorities, delayed delivery. Better for task queues and work distribution.
- **Pulsar** — Kafka-like with multi-tenancy and tiered storage built in.

The distinction that usually decides it: Kafka is a **log** you can replay; RabbitMQ is a **queue** that forgets. If a new consumer needs to read the last month of events, that is Kafka.

11.9 Testing

11.9.1 In-memory connector

```
@QuarkusTest
class OrderEventConsumerTest {

    @Inject @Any InMemoryConnector connector;
    @Inject OrderRepository orders;

    @BeforeAll
    static void switchToInMemory() {
        InMemoryConnector.switchIncomingChannelsToInMemory("orders-in");
        InMemoryConnector.switchOutgoingChannelsToInMemory("orders-out");
    }

    @AfterAll
    static void restore() {
        InMemoryConnector.clear();
    }

    @Test
    void persistsIncomingOrder() {
        InMemorySource<OrderEvent> source = connector.source("orders-in");
        source.send(new OrderEvent("evt-1", 1L, "PENDING", TEN, Instant.now()));

        await().atMost(5, SECONDS)
            .untilAsserted(() -> assertThat(orders.count()).isEqualTo(1));
    }

    @Test
    void ignoresDuplicates() {
        InMemorySource<OrderEvent> source = connector.source("orders-in");
        OrderEvent event = new OrderEvent("evt-2", 2L, "PENDING", TEN, Instant.now());

        source.send(event);
        source.send(event);           // same eventId

        await().atMost(5, SECONDS)
            .untilAsserted(() -> assertThat(orders.count("externalId", 2L)).isEqualTo(1));
    }
}
```

```

@Test
void publishesOnCreate() {
    InMemorySink<OrderEvent> sink = connector.sink("orders-out");
    orderService.create(validRequest());

    await().untilAsserted(() -> assertThat(sink.received()).hasSize(1));
    assertThat(sink.received().get(0).getPayload().status()).isEqualTo("PENDING");
}
}

```

```

%test.mp.messaging.incoming.orders-in.connector=smallrye-in-memory
%test.mp.messaging.outgoing.orders-out.connector=smallrye-in-memory

```

11.9.2 A real broker via Dev Services

```

# Omit kafka.bootstrap.servers in dev/test and Quarkus starts Redpanda or Kafka
%dev.quarkus.kafka.devservices.provider=redpanda
%dev.quarkus.kafka.devservices.topic-partitions.orders=3

```

The in-memory connector tests **your logic**. A real broker tests **serialisation, partitioning, offset behaviour and rebalancing** — which is where the interesting bugs live. Use both: in-memory for the bulk of the suite, a real broker for a handful of integration tests.

The duplicate test above is the one that matters most. Idempotency is the property your correctness depends on, it is easy to break with an innocuous refactor, and it never fails in manual testing because you do not naturally send the same message twice.

11.10 Operating Kafka Consumers

```

# The knobs that actually matter in production
mp.messaging.incoming.orders-in.max.poll.records=100
mp.messaging.incoming.orders-in.max.poll.interval.ms=300000 # rebalance if exceeded
mp.messaging.incoming.orders-in.session.timeout.ms=45000
mp.messaging.incoming.orders-in.heartbeat.interval.ms=3000
mp.messaging.incoming.orders-in.fetch.min.bytes=1024
mp.messaging.incoming.orders-in.partitions=3 # consumers per instance

```

max.poll.interval.ms is the one that bites. If processing a batch takes longer than this, Kafka assumes the consumer is dead and rebalances — which reassigns partitions, replays uncommitted messages, and can produce a rebalance loop where nothing ever completes. If your processing is slow, either reduce **max.poll.records** or raise the interval; do not leave both at defaults and hope.

Consumer lag is the single most important metric. It is the answer to “is my consumer keeping up?”, and it is what you alert on:

```

quarkus.micrometer.binder.kafka.enabled=true

kafka_consumer_fetch_manager_records_lag_max
kafka_consumer_fetch_manager_records_consumed_rate

```

Scaling. More consumer instances help only up to the partition count. Beyond that they sit idle. Increasing partitions is possible but repartitions future data — messages for a given key may land in a different partition than before, which breaks ordering across the change. Plan partition counts with headroom.

11.11 Best Practices

1. **acks=all** and **enable.idempotence=true** for any business data.
2. **enable.auto.commit=false**. Auto-commit turns at-least-once into at-most-once.
3. **Idempotent consumers, always**. Delivery is at-least-once and no configuration changes that.
4. **Key by the ordering unit** — customer, order, aggregate — not randomly and not by a monotonically increasing id.
5. **Configure a DLQ explicitly**. The **fail** default silently stops consumption.
6. **Alert on consumer lag and DLQ depth**, and provide a replay path.
7. **@Blocking** on any consumer doing JDBC. Otherwise it runs on an event loop.
8. **Do not use @Blocking(ordered = false)** unless you have decided ordering does not matter.
9. **Schema registry with BACKWARD compatibility, enforced in CI**, for cross-team topics.
10. **The outbox pattern** for database-plus-publish atomicity.
11. **Retry transient failures, dead-letter permanent ones**. Distinguishing them is your job.
12. **Watch max.poll.interval.ms** against real processing time to avoid rebalance loops.
13. **Test duplicates explicitly**, and test with a real broker at least once.

11.12 Common Mistakes

1. **Assuming exactly-once delivery**. It does not exist. Idempotency is the answer.
2. **Auto-commit enabled**. Commits on a timer regardless of success; loses data on crash.
3. **No DLQ**. One poison message stops the channel; the pod stays “healthy” while lag grows.
4. **@Incoming** doing JDBC without **@Blocking**. Blocks an event loop per message.
5. **Publishing inside a transaction**. Not atomic; phantom events on rollback, lost events on crash.
6. **Random or high-cardinality-but-wrong keys**. Loses the ordering you needed, or creates hot partitions.
7. **More consumers than partitions**. The extras idle.
8. **Incompatible schema change**. Breaks every consumer at once, discovered via lag alerts across teams.
9. **Retrying permanent failures forever**. Blocks the partition; the message will never succeed.
10. **max.poll.interval.ms** shorter than real processing time. Rebalance loop; throughput collapses to zero.
11. **Ignoring the emitter’s returned Uni/CompletionStage**. You do not know whether the publish succeeded.

12. A DLQ nobody drains. Data loss with a paper trail.

11.13 Interview Questions

Junior

Q1. What is Kafka? A distributed append-only log. Producers write to topics, which are split into partitions; consumers read from partitions and track their position with offsets.

Q2. What is a partition? A subdivision of a topic. Ordering is guaranteed within a partition, and each partition is consumed by exactly one member of a consumer group.

Q3. What is a consumer group? A set of consumers that share the work of a topic. Kafka assigns each partition to one member, so the group's parallelism is capped by the partition count.

Q4. How do you consume a message in Quarkus? `@Incoming("channel")` on a method, with the channel bound to a topic in `application.properties`.

Q5. How do you publish? Inject a `MutinyEmitter<T>` with `@Channel("channel")` and call `send`, or return a `Multi` from an `@Outgoing` method.

Q6. What is an offset? A consumer group's position in a partition. Committing it records progress so a restart resumes from the right place.

Mid-level

Q7. What delivery guarantee does Kafka give? At-least-once in practice. At-most-once is possible by committing before processing, which loses messages on crash. Exactly-once *delivery* is not achievable across a network with failures; exactly-once *effect* is, by combining at-least-once delivery with an idempotent consumer.

Q8. How do you make a consumer idempotent? Either make the operation naturally idempotent — an upsert keyed on the business id, ideally with a timestamp guard so out-of-order delivery is also safe — or insert a row into a `processed_events` table with a unique constraint on the event id, in the same transaction as the business write. The constraint, not a preceding existence check, is what makes it correct under concurrency.

Q9. Why disable auto-commit? Auto-commit advances the offset on a timer regardless of whether processing succeeded. A crash after commit but before processing loses the message. With it disabled, the connector commits only acknowledged offsets.

Q10. What happens to a message that always fails? With the default `fail` strategy, the channel stops and consumption halts while the pod appears healthy. Because a partition's offset is a single high-water mark, the consumer cannot skip it. Configure `dead-letter-queue` so the partition can make progress.

Q11. How do you guarantee ordering? Only within a partition, so key messages by the unit you need ordered — customer id for per-customer ordering. Global ordering requires a single partition and therefore a single consumer. Also avoid `@Blocking(ordered = false)`, which processes a partition's messages concurrently and discards the guarantee.

Q12. How do you scale a consumer? Add instances up to the partition count; beyond that they idle. Increasing partitions adds capacity but changes key-to-partition mapping for future messages, which breaks ordering across the change — so provision partitions with headroom.

Q13. What is a schema registry for? Storing message schemas centrally so producers and consumers agree on structure, and enforcing compatibility rules so a schema change cannot break existing consumers. With `BACKWARD` compatibility you may add fields with defaults and remove fields that have them, but not rename or narrow types.

Senior

Q14. Design an event-driven order pipeline. Order creation writes the order and an outbox row in one local transaction, because a database commit and a Kafka publish cannot be atomic. A separate publisher — Debezium reading the outbox via CDC, or a polling job — drains it to an `orders` topic keyed by customer id, so all events for a customer are ordered relative to each other while different customers process in parallel. Downstream services consume with `enable.auto.commit=false`, `POST_PROCESSING` acknowledgement and a throttled commit strategy, and each consumer is idempotent by an upsert with a timestamp guard, since delivery is at-least-once and out-of-order redelivery is possible after a rebalance. Transient failures retry with backoff and jitter; permanent ones go to a DLQ with the reason in headers, and there is an operator-triggered replay path. Schemas live in a registry with `BACKWARD` compatibility checked in CI. The metrics that matter are consumer lag per partition and DLQ depth, and both are alerted. The design decision I would flag explicitly to the team is the partition key, because it fixes the ordering guarantee and is expensive to change afterwards.

Q15. Explain exactly-once semantics honestly. Exactly-once *delivery* is not achievable: the consumer and the broker cannot atomically agree that a message was both processed and committed, so there is always a window in which a crash forces a choice between replaying and losing. What is achievable is exactly-once *effect*, and it comes from two independent pieces. On the producer side, `enable.idempotence=true` deduplicates the producer's own retries using a sequence number per partition, which removes duplicates caused by network-level retries. On the consumer side, at-least-once delivery combined with an idempotent operation means a redelivery has no additional effect. Kafka transactions extend this to the consume-transform-produce pattern within Kafka, because the offset commit and the produced records land in one Kafka transaction — but that boundary stops at Kafka, so a consumer writing to a database is still at-least-once and still needs idempotency. When someone says “we have exactly-once”, the useful follow-up is which of these three things they mean.

Q16. A consumer's lag grows continuously. Diagnose it. Lag growing means arrival rate exceeds processing rate, so I would first establish which side changed — a producer-side spike looks different from a consumer-side slowdown, and the consumed-rate metric distinguishes them. If consumption slowed, the candidates in order: the consumer is blocking an event loop, which in messaging shows up as very low throughput with low CPU; the database or a downstream call got slower, which shows in the per-message processing time; a rebalance loop caused by processing exceeding `max.poll.interval.ms`, which is visible as repeated partition reassignments in the logs and is particularly nasty because it produces zero net progress; or the consumer is stuck retrying a message that will never succeed, which blocks the partition and shows as lag on one partition rather than all. If it is genuinely capacity, the fix is more consumers — but only up to the partition count, and if we are already at it then the answer is repartitioning or reducing per-message work, not more pods. I would also check whether `@Blocking(ordered = false)` is an option, since serialising per

partition is often the real constraint and unordered processing may be acceptable for that particular topic.

Q17. When would you choose RabbitMQ over Kafka? When the workload is task distribution rather than event streaming. RabbitMQ gives per-message acknowledgement and redelivery, so a slow or failing message does not block its peers the way a poison message blocks a Kafka partition — that alone makes it much better suited to a work queue where messages are independent. It also has native support for priorities, delayed delivery, TTLs and flexible topic-exchange routing, all of which require workarounds in Kafka. Kafka’s advantages are retention and replay — a new consumer can read the last month of history — plus much higher throughput and ordering guarantees. So the question I would ask is whether anyone will ever need to re-read the stream. If the messages are commands to be executed once and then forgotten, RabbitMQ is simpler and a better fit. If they are facts other services may want to consume later, or in a different way, Kafka’s log semantics are the whole point.

Principal

Q18. Design the event platform standards for an organisation of 40 services. The goal is that a team can produce an event other teams can consume safely without a meeting. That means standardising the contract and the operational envelope, and leaving the domain alone. On contracts: a schema registry with **BACKWARD** compatibility enforced in CI, a topic naming convention encoding domain and event type, and a mandatory envelope carrying event id, event type, schema version, occurred-at timestamp, correlation id and producing service — because those six fields are what makes debugging across services tractable, and retrofitting them is painful. On operations: the outbox pattern as the default publishing mechanism, shipped as a shared library with a CDC deployment so teams do not each build a polling loop; a required DLQ per consumer with standard reason headers and a shared replay tool; and consumer lag plus DLQ depth as platform-level SLIs visible on a common dashboard. On semantics: idempotency is a stated requirement, with a shared dedup helper, and the platform documentation states plainly that delivery is at-least-once so nobody designs assuming otherwise. What I would leave free: event granularity, payload shape beyond the envelope, whether a service uses Kafka Streams, and partition counts within a published range. The thing I would guard hardest is schema compatibility, because an incompatible change breaks consumers a team has never heard of, and the failure surfaces as lag alerts on someone else’s rota rather than as a failing build on the producer’s.

Q19. Argue against event-driven architecture for a team considering it. The costs are real and usually underestimated because the benefits are easy to articulate and the costs are diffuse. You lose the ability to reason about the system by reading code: with synchronous calls the call graph is in the source, whereas with events the flow lives in configuration and broker topology, and a new engineer cannot answer “what happens when an order is created?” without tooling you have to build. Debugging degrades similarly — a failure that would be one stack trace becomes a correlation-id search across five services and a broker. You inherit eventual consistency everywhere, which means every read path must handle “not there yet”, and product and support have to understand that too. You must make every consumer idempotent, which is a discipline that decays without enforcement, and the failures from a broken one are silent data corruption rather than an exception. And you acquire a new critical operational component with its own scaling, upgrade and partition-count decisions. Against that, the benefits — decoupling, independent scaling, replay, temporal decoupling during outages — are genuine but only pay off at a certain size and rate of change. My honest position is that a well-factored synchronous system with clear ownership is

easier to operate than most event-driven systems of the same size, and that the strongest reason to go event-driven is a specific need: multiple independent consumers of the same facts, replay for rebuilding read models, or a producer that must not be coupled to consumer availability. Adopting it because it is the architecture one is supposed to have is how teams end up with a distributed monolith plus a broker.

11.14 Cheat Sheet

Producer config

```
mp.messaging.outgoing.out.connector=smallrye-kafka
mp.messaging.outgoing.out.topic=orders
mp.messaging.outgoing.out.acks=all
mp.messaging.outgoing.out.enable.idempotence=true
mp.messaging.outgoing.out.compression.type=snappy
mp.messaging.outgoing.out.overflow-strategy=buffer
```

Consumer config

```
mp.messaging.incoming.in.connector=smallrye-kafka
mp.messaging.incoming.in.topic=orders
mp.messaging.incoming.in.group.id=orders-processor
mp.messaging.incoming.in.enable.auto.commit=false
mp.messaging.incoming.in.commit-strategy=throttled
mp.messaging.incoming.in.failure-strategy=dead-letter-queue
mp.messaging.incoming.in.dead-letter-queue.topic=orders-dlq
mp.messaging.incoming.in.max.poll.records=100
mp.messaging.incoming.in.auto.offset.reset=earliest
```

Code

```
@Inject @Channel("out") MutinyEmitter<OrderEvent> emitter;
emitter.sendMessage(Message.of(event).addMetadata(
    OutgoingKafkaRecordMetadata.<String>builder().withKey(key).build()));

@Incoming("in") @Blocking @Transactional
public void consume(OrderEvent e) { }

@Incoming("in") @Outgoing("out")
public OrderEvent transform(OrderEvent e) { return enrich(e); }

@Incoming("in") @Acknowledgment(Strategy.MANUAL)
public CompletionStage<Void> manual(Message<OrderEvent> m) { ... m.ack(); }
```

Guarantees

at-most-once	commit before processing	-> loses on crash	
at-least-once	commit after processing	-> duplicates on crash	<- default
exactly-once	delivery: impossible		
	effect: at-least-once + idempotent consumer		

Testing

```
InMemoryConnector.switchIncomingChannelsToInMemory("in");
connector.source("in").send(event);
connector.<OrderEvent>sink("out").received();
```

Metrics to alert on

```
kafka_consumer_fetch_manager_records_lag_max
DLQ topic depth
rebalance rate
```

11.15 Summary

Kafka's model determines your design more than any framework choice: ordering exists only within a partition, the partition is chosen by the key, and a group's parallelism is capped by the partition count. Choosing the key is therefore choosing your ordering guarantee, and it is expensive to change.

On delivery, be honest. **At-least-once is what you get**, and no configuration changes that. Exactly-once *effect* is reachable by making consumers idempotent — ideally by construction, with an upsert and a timestamp guard, otherwise with a unique constraint on the event id in the same transaction as the business write. Kafka transactions give exactly-once *between Kafka topics* and stop at your database.

Two operational things are not optional. **Configure a DLQ**, because the `fail` default silently halts consumption while the pod looks healthy, and a poison message otherwise blocks its partition forever. And **alert on consumer lag**, because it is the one metric that answers whether the system is keeping up.

Finally, publishing to Kafka from inside a database transaction is not atomic and never will be. The **outbox pattern** is the answer, and it is worth building once, properly, as shared infrastructure.

Chapter 12 covers **scheduling** — cron and interval jobs, the clustered-scheduler problem that catches every team running more than one replica, and how to make scheduled work idempotent and observable.

11.16 References

- Quarkus Kafka guide — <https://quarkus.io/guides/kafka>
- Kafka reference — <https://quarkus.io/guides/kafka-reference>
- SmallRye Reactive Messaging — <https://smallrye.io/smallrye-reactive-messaging/>
- Using Apache Kafka with schema registry and Avro — <https://quarkus.io/guides/kafka-schema-registry-avro>
- Apache Kafka documentation — <https://kafka.apache.org/documentation/>
- Debezium outbox event router — <https://debezium.io/documentation/reference/transformations/outbox-event-router.html>
- Transactional outbox pattern — <https://microservices.io/patterns/data/transactional-outbox.html>

Scheduling

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-scheduler` · `quarkus-quartz`

Learning Objectives

By the end of this chapter you will be able to:

1. Schedule work with `@Scheduled` using intervals, cron expressions and configuration-driven values.
2. Explain the **multi-replica problem** — the single most common scheduling bug in Kubernetes — and its solutions.
3. Configure the clustered Quartz scheduler for single-execution-across-replicas.
4. Control concurrent execution, skip predicates and time zones.
5. Register jobs programmatically at runtime.
6. Manage transactions and long-running work inside scheduled methods without exhausting the connection pool.
7. Make scheduled jobs observable — metrics, health, and knowing when one silently stopped running.
8. Decide between an in-application scheduler and a Kubernetes `CronJob`.
9. Test scheduled logic deterministically.
10. Answer 12 interview questions on scheduling in a distributed system.

12.1 The Basics

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-scheduler</artifactId>
</dependency>
```

```
package com.acme.orders.jobs;

import io.quarkus.scheduler.Scheduled;
import jakarta.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class OrderJobs {
```

```

// Fixed interval, measured from the end of the previous run
@Scheduled(every = "30s")
void pollPayments() { ... }

// Delay the first execution until after startup
@Scheduled(every = "1m", delayed = "30s")
void warmCaches() { ... }

// Cron: second minute hour day-of-month month day-of-week [year]
@Scheduled(cron = "0 0 2 * * ?") // 02:00 every day
void nightlyReconciliation() { ... }

@Scheduled(cron = "0 */15 * * * ?") // every 15 minutes
void refreshExchangeRates() { ... }

@Scheduled(cron = "0 0 9 ? * MON-FRI") // 09:00 on weekdays
void weekdayReport() { ... }

// Driven by configuration - the right default for anything operational
@Scheduled(cron = "{orders.reconciliation.cron}")
void configurable() { ... }

// Disable without removing code: set the property to "off"
@Scheduled(cron = "{orders.optional-job.cron:off}")
void optional() { ... }

// Receive execution metadata
@Scheduled(every = "5m")
void withContext(ScheduledExecution execution) {
    Log.infof("Trigger %s fired at %s, scheduled for %s",
        execution.getTrigger().getId(),
        execution.getFireTime(),
        execution.getScheduledFireTime());
}
}

```

A scheduled method must be on a CDI bean, must return `void` or `Uni<Void>`, and may take zero arguments or a single `ScheduledExecution`.

| `cron = "{property}"` should be your default, not a hard-coded expression. Operations will eventually need to change a schedule, disable a job during an incident, or stagger jobs across environments. A property makes that a config change; a literal makes it a deployment.

12.1.1 Cron syntax

Quarkus uses **Quartz-style** cron by default, which has six or seven fields — note the leading seconds field, which trips up anyone used to Unix cron.

```

┌────────── second (0-59)
├──┬──────── minute (0-59)
│ │ ┌──────── hour (0-23)
│ │ │ ┌──────── day of month (1-31)

```

```

| | | | | month (1-12 or JAN-DEC)
| | | | | day of week (1-7 or SUN-SAT, ? = no specific value)
| | | | | year (optional)
* * * * ? *

```

Switch to Unix (5-field) or CRON4J syntax if you prefer
 quarkus.scheduler.cron-type=quartz # quartz | unix | cron4j | spring

? in either the day-of-month or day-of-week field means “no specific value” and is required because specifying both is ambiguous. `0 0 2 * * ?` is valid; `0 0 2 * * *` is not.

12.2 The Multi-Replica Problem

This is the section that matters. Everything else in the chapter is mechanics.

```

@Scheduled(cron = "0 0 2 * * ?")
void sendMonthlyInvoices() {
    invoiceService.generateAndSend();
}

```

Deploy this with `replicas: 3` and **every pod runs it**. Three sets of invoices, three emails per customer, three sets of charges.

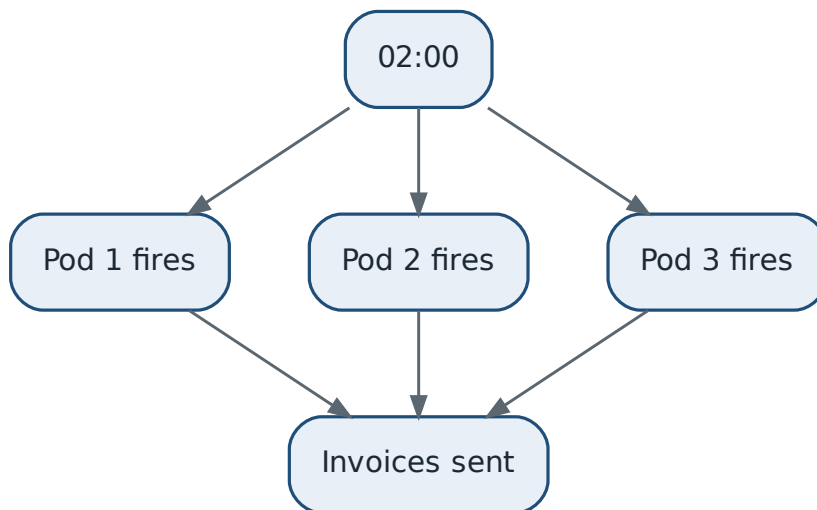


Figure 12.1 — The default in-memory scheduler is per-pod. Every replica runs every job.

The default `quarkus-scheduler` is an **in-memory, per-JVM** scheduler. It has no knowledge of other replicas. This is fine — correct, even — for idempotent maintenance work like refreshing a local cache. It is a serious bug for anything with an external side effect.

12.2.1 Solution 1: the clustered Quartz scheduler

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-quartz</artifactId>
</dependency>
```

```
quarkus.quartz.clustered=true
quarkus.quartz.store-type=jdbc-cmt          # requires the Quartz tables
quarkus.quartz.start-mode=forced
quarkus.quartz.instance-name=orders-scheduler
quarkus.quartz.cluster-checkin-interval=15000
```

```
# Quartz needs its own tables - create them with Flyway
quarkus.flyway.migrate-at-start=true
```

With clustering on, Quartz takes a database lock before firing a trigger, so **exactly one replica executes each occurrence**. Which one is unspecified and varies between runs, which is correct — the job must not depend on running anywhere in particular.

The Quartz schema ships with Quartz; add it as a migration:

```
-- V1.5.0__quartz_tables.sql (the standard Quartz PostgreSQL DDL)
CREATE TABLE qrtz_job_details (...);
CREATE TABLE qrtz_triggers (...);
CREATE TABLE qrtz_locks (...);
-- ... and the rest
```

```
// With Quartz, you can also declare misfire behaviour
@Scheduled(cron = "0 0 2 * * ?",
            concurrentExecution = Scheduled.ConcurrentExecution.SKIP)
void nightly() { ... }
```

Misfire handling is the reason to think about Quartz beyond clustering: if all replicas were down at 02:00, should the job run when they come back, or be skipped? Quartz lets you choose; the in-memory scheduler simply loses the occurrence.

12.2.2 Solution 2: an application-level lock

Lighter weight than Quartz when you have one or two jobs:

```
@ApplicationScoped
public class LockedJobs {

    @Inject EntityManager em;

    @Scheduled(cron = "0 0 2 * * ?")
    void nightly() {
        if (!tryAcquire("nightly-reconciliation", Duration.ofHours(1))) {
            Log.debug("Another replica holds the lock; skipping");
            return;
        }
        try {
```

```

        reconciliationService.run();
    } finally {
        release("nightly-reconciliation");
    }
}

@Transactional(Transactional.TxType.REQUIRES_NEW)
boolean tryAcquire(String name, Duration ttl) {
    // A unique constraint on (name) makes this atomic
    try {
        em.createNativeQuery("""
            INSERT INTO job_locks (name, holder, acquired_at, expires_at)
            VALUES (?1, ?2, now(), now() + ?3::interval)
            ON CONFLICT (name) DO UPDATE
                SET holder = ?2, acquired_at = now(), expires_at = now() + ?3::interval
            WHERE job_locks.expires_at < now()
            """)
            .setParameter(1, name)
            .setParameter(2, instanceId)
            .setParameter(3, ttl.toSeconds() + " seconds")
            .executeUpdate();
        return true;
    } catch (PersistenceException e) {
        return false;
    }
}
}

```

The `expires_at` clause is essential: without a TTL, a pod that crashes mid-job holds the lock forever and the job never runs again. That failure is worse than the duplicate-execution problem you were solving.

12.2.3 Solution 3: a Kubernetes CronJob

Take the schedule out of the application entirely.

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: orders-reconciliation
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid          # never overlap
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  startingDeadlineSeconds: 300
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: Never
          containers:

```



```

- name: reconciliation
  image: acme/orders-service:1.4.0
  args: ["reconcile"]
  env:
    - name: QUARKUS_PROFILE
      value: job

```

```

@QuarkusMain
public class Main implements QuarkusApplication {

    @Inject ReconciliationService reconciliation;

    public static void main(String... args) { Quarkus.run(Main.class, args); }

    @Override
    public int run(String... args) {
        if (args.length > 0 && args[0].equals("reconcile")) {
            reconciliation.run();
            return 0; // exits; no waitForExit()
        }
        Quarkus.waitForExit();
        return 0;
    }
}

```

My recommendation. Use `@Scheduled` for frequent, cheap, idempotent, in-process work — refreshing a cache, publishing a heartbeat, draining an outbox. Use a Kubernetes `CronJob` for infrequent, expensive, externally-visible batch work — invoicing, reports, data exports. The `CronJob` gets you single execution for free, isolated resource limits, independent retry and history, and a failure that does not consume capacity from your request-serving pods. In native mode the startup cost is negligible, which removes the traditional objection. Reach for clustered Quartz when you need dynamic schedules, misfire policies, or job state that a `CronJob` cannot express.

12.3 Concurrent Execution

```

// Default: PROCEED. A slow run overlaps with the next trigger.
@Scheduled(every = "10s")
void mayOverlap() { slowWork(); } // 15s work -> two concurrent runs

// SKIP: if the previous run is still going, skip this occurrence
@Scheduled(every = "10s", concurrentExecution = Scheduled.ConcurrentExecution.SKIP)
void neverOverlaps() { slowWork(); }

```

`SKIP` should be your default for anything that mutates state. The `PROCEED` default is a foot-gun: a job that gradually slows down eventually runs continuously with several instances in flight, which looks like a memory leak and a database load spike with no obvious cause.

12.3.1 Skip predicates

```
@ApplicationScoped
public class MaintenanceWindowPredicate implements Scheduled.SkipPredicate {
    @Inject MaintenanceState state;

    @Override
    public boolean test(ScheduledExecution execution) {
        return state.isInMaintenance();
    }
}
```

```
@Scheduled(cron = "0 */5 * * * ?", skipExecutionIf = MaintenanceWindowPredicate.class)
void skippableJob() { ... }

// Built-in: skip while the application is not yet fully started
@Scheduled(every = "10s", skipExecutionIf = Scheduled.ApplicationNotRunning.class)
void onlyWhenRunning() { ... }
```

`ApplicationNotRunning` is worth knowing: without it, a job with a short interval can fire during startup, before the beans it depends on are ready.

12.3.2 Execution thread

```
// Default: a worker thread from the scheduler's pool
@Scheduled(every = "1m")
void blocking() { jdbcWork(); }

// Non-blocking: runs on an event loop - must not block
@Scheduled(every = "1s")
Uni<Void> reactive() { return client.ping(); }

// Java 21 virtual thread
@Scheduled(every = "1m")
@RunOnVirtualThread
void virtual() { jdbcWork(); }
```

```
quarkus.scheduler.overdue-grace-period=1S
quarkus.quartz.thread-count=10
quarkus.quartz.thread-priority=5
```

12.4 Time Zones

```
@Scheduled(cron = "0 0 9 * * ?", timeZone = "Europe/London")
void londonMorning() { ... }

@Scheduled(cron = "0 0 9 * * ?", timeZone = "{orders.report.timezone:UTC}")
void configurable() { ... }
```

Run schedules in UTC unless a human-facing time genuinely matters. A job at 02:00 Europe/London runs twice on the day clocks go back and not at all on the day they go forward — a real, annual, entirely predictable incident. If the job must align with local business hours, use the zone and make the job idempotent so the duplicate occurrence is harmless.

12.5 Programmatic Scheduling

```
@ApplicationScoped
public class DynamicJobs {

    @Inject Scheduler scheduler;

    void onStart(@Observes StartupEvent event, TenantConfig config) {
        for (Tenant tenant : config.tenants()) {
            scheduler.newJob("report-" + tenant.id())
                .setCron(tenant.reportCron())
                .setConcurrentExecution(Scheduled.ConcurrentExecution.SKIP)
                .setTask(execution -> reportService.generate(tenant.id()))
                .schedule();
        }
    }

    public void unschedule(String tenantId) {
        scheduler.unscheduleJob("report-" + tenantId);
    }

    public void pauseAll() { scheduler.pause(); }
    public void resumeAll() { scheduler.resume(); }
    public boolean isRunning() { return scheduler.isRunning(); }
}
```

`scheduler.pause()` is genuinely useful operationally: expose it behind an authenticated admin endpoint so an on-call engineer can stop background work during an incident without redeploying.

12.6 Transactions and Long-Running Work

```
// BAD: one transaction spanning the whole job.
// Holds a connection for hours; a failure at the end rolls back everything.
@Scheduled(cron = "0 0 2 * * ?")
@Transactional
void badNightly() {
    List<Order> stale = Order.list("status = ?1 and createdAt < ?2", PENDING, cutoff());
    stale.forEach(this::expire);           // could be a million rows
}

// GOOD: chunked, so each commit releases the connection and bounds the rollback
@Scheduled(cron = "0 0 2 * * ?", concurrentExecution = SKIP)
void goodNightly() {
    List<Long> ids = findStaleIds();           // no transaction
}
```

```

    for (List<Long> chunk : partition(ids, 500)) {
        QuarkusTransaction.requiringNew()
            .timeout(60)
            .run(() -> expireChunk(chunk));
    }
}

```

Chapter 7's rules apply, with two additions specific to scheduled work:

- **A scheduled method needs `@ActivateRequestContext`** if it uses `@RequestScoped` beans — there is no HTTP request to establish one.
- **Bound the total runtime.** A job that can run longer than its own interval will either overlap (with `PROCEED`) or be skipped forever (with `SKIP`), and both are silent.

```

@Scheduled(every = "1h", concurrentExecution = SKIP)
@ActivateRequestContext
void needsRequestScope() { ... }

```

12.7 Observability

A scheduled job that stops running produces **no error**. Nothing fails; work simply stops happening. That is the failure mode to design against.

```

@ApplicationScoped
public class InstrumentedJobs {

    @Inject MeterRegistry registry;
    private final AtomicReference<Instant> lastSuccess = new AtomicReference<>();

    @Scheduled(cron = "0 0 2 * * ?", concurrentExecution = SKIP)
    void nightly() {
        Timer.Sample sample = Timer.start(registry);
        try {
            int processed = reconciliation.run();
            registry.counter("job.records", "job", "nightly").increment(processed);
            lastSuccess.set(Instant.now());
            registry.counter("job.runs", "job", "nightly", "outcome", "success").increment();
        } catch (Exception e) {
            registry.counter("job.runs", "job", "nightly", "outcome", "failure").increment();
            Log.error("Nightly reconciliation failed", e);
            throw e;
        } finally {
            sample.stop(registry.timer("job.duration", "job", "nightly"));
        }
    }

    void registerGauge(@Observes StartupEvent e) {
        Gauge.builder("job.seconds_since_success",
            () -> lastSuccess.get() == null
                ? Double.NaN
                : Duration.between(lastSuccess.get(), Instant.now()).toSeconds())
    }
}

```

```

        .tag("job", "nightly")
        .register(registry);
    }
}

```

```

// A health check that fails when a job has not succeeded recently
@Liveness
@ApplicationScoped
public class JobFreshnessCheck implements HealthCheck {

    @Inject InstrumentedJobs jobs;

    @Override
    public HealthCheckResponse call() {
        Duration since = jobs.timeSinceLastSuccess();
        return HealthCheckResponse.named("nightly-job")
            .status(since.compareTo(Duration.ofHours(26)) < 0)
            .withData("secondsSinceSuccess", since.toSeconds())
            .build();
    }
}

```

Alert on `seconds_since_success`, not on failure rate. A failure produces a log line someone might see. A job that silently stopped being scheduled — because a property was set to `off`, or the clustered lock is stuck, or the trigger was unscheduled — produces nothing at all. The staleness gauge is the only signal that catches it.

The Quarkus scheduler also exposes CDI events you can observe centrally:

```

void onSuccess(@Observes SuccessfulExecution execution) { ... }
void onFailure(@Observes FailedExecution execution) { ... }
void onSkipped(@Observes SkippedExecution execution) { ... }

```

12.8 Testing

Scheduling is time-dependent, which makes it awkward to test directly. The solution is to not test the scheduling.

```

@ApplicationScoped
public class ReconciliationJob {

    @Inject ReconciliationService service;    // all the logic lives here

    @Scheduled(cron = "{orders.reconciliation.cron:0 0 2 * * ?}")
    void run() {
        service.reconcile();                // the job is a two-line adapter
    }
}

```

```
// Test the service directly - fast, deterministic, no clock involved
class ReconciliationServiceTest {
    @Test
    void expiresStaleOrders() {
        var service = new ReconciliationService(repo, clock);
        service.reconcile();
        verify(repo).expire(anyList());
    }
}
```

```
# Disable schedulers entirely in tests so they do not fire mid-test
%test.quarkus.scheduler.enabled=false
```

```
// When you must test the trigger itself
@QuarkusTest
@TestProfile(FastScheduleProfile.class)
class ScheduledTriggerTest {

    public static class FastScheduleProfile implements QuarkusTestProfile {
        public Map<String, String> getConfigOverrides() {
            return Map.of("quarkus.scheduler.enabled", "true",
                "orders.reconciliation.cron", "0/1 * * * * ?"); // every second
        }
    }

    @InjectSpy ReconciliationService service;

    @Test
    void firesOnSchedule() {
        await().atMost(5, SECONDS)
            .untilAsserted(() -> verify(service, atLeastOnce()).reconcile());
    }
}
```

Inject a **Clock** into any service whose behaviour depends on time. It makes “expire orders older than 30 days” testable without waiting 30 days, and it costs nothing.

```
@ApplicationScoped
public class ClockProducer {
    @Produces @Singleton
    Clock clock() { return Clock.systemUTC(); }
}
```

12.9 Best Practices

1. **Decide the multi-replica behaviour before writing the job.** Idempotent-everywhere, clustered Quartz, or a **CronJob** — but decide.
2. **concurrentExecution = SKIP** for anything that mutates state.
3. **Cron from configuration**, with an **off** default for optional jobs.
4. **UTC schedules** unless a human-facing local time is genuinely required.
5. **Chunked transactions**, never one transaction spanning the whole job.

6. A staleness gauge and an alert on it. A job that stops running produces no error.
7. Keep the `@Scheduled` method a two-line adapter over a testable service.
8. Inject a `Clock` so time-dependent logic is testable.
9. `skipExecutionIf = ApplicationNotRunning.class` on short-interval jobs.
10. `quarkus.scheduler.enabled=false` in tests.
11. Expose pause/resume behind an admin endpoint for incident response.
12. Prefer a Kubernetes `CronJob` for infrequent, expensive, externally-visible batch work.

12.10 Common Mistakes

1. **Running a side-effecting job on every replica.** Duplicate invoices, duplicate emails, duplicate charges.
2. Leaving `concurrentExecution` at `PROCEED`. A gradually-slowing job eventually runs several instances concurrently.
3. **One transaction for the whole job.** Holds a connection for hours; one failure discards all the work.
4. **Hard-coded cron expressions.** Changing a schedule becomes a deployment.
5. **Local time zones.** Jobs run twice or not at all on daylight-saving transitions.
6. **No alerting on job staleness.** The failure is silence, and silence does not page anyone.
7. `@RequestScoped` beans without `@ActivateRequestContext`. `ContextNotActiveException`.
8. **An application lock without a TTL.** A crashed pod holds it forever and the job never runs again.
9. **Business logic inside the `@Scheduled` method.** Untestable without waiting for the clock.
10. **Schedulers running during tests.** Jobs fire mid-test and corrupt fixtures, producing flaky failures.
11. **A job whose runtime exceeds its interval.** Silent overlap or silent permanent skipping.

12.11 Interview Questions

Junior

- Q1. How do you schedule a job in Quarkus?** `@Scheduled(every = "30s")` or `@Scheduled(cron = "0 0 2 * * ?")` on a method of a CDI bean, with `quarkus-scheduler` on the classpath.
- Q2. What is the difference between `every` and `cron`?** `every` is a fixed interval measured from the end of the previous run. `cron` is a calendar expression, so it fires at specific clock times.
- Q3. Why does Quarkus cron have six fields?** It uses Quartz syntax, which adds a leading seconds field. `0 0 2 * * ?` means 02:00:00 daily. The `?` means “no specific value” and is required in one of the day fields because specifying both is ambiguous.
- Q4. How do you make a schedule configurable?** `@Scheduled(cron = "{orders.job.cron}")`, referencing a config property. Setting the property to `off` disables the job.

Mid-level

Q5. What happens to a scheduled job when you run three replicas? The default scheduler is in-memory and per-JVM, so all three replicas run it. For idempotent local work that is harmless; for anything with an external side effect it means triple execution. The fixes are clustered Quartz, an application-level lock with a TTL, or moving the job to a Kubernetes [CronJob](#).

Q6. How does clustered Quartz prevent duplicate execution? It stores triggers in a database and takes a row lock before firing, so exactly one instance executes each occurrence. Which instance is unspecified, so the job must not depend on running on a particular pod.

Q7. What is `concurrentExecution` and what should it be? It controls what happens when a trigger fires while the previous run is still going. The default `PROCEED` overlaps; `SKIP` skips the occurrence. `SKIP` should be the default for anything that mutates state, because a slowly-degrading job under `PROCEED` ends up running several instances concurrently.

Q8. How do you handle a long-running scheduled job? Chunk it: query the ids without a transaction, then process in batches of a few hundred inside `QuarkusTransaction.requiringNew()`, so each commit releases the connection and a failure loses only one chunk. Set `SKIP`, bound the total runtime, and make the work resumable so a partial run is not a problem.

Q9. How do you test a scheduled job? Keep the `@Scheduled` method a thin adapter over a service, and test the service directly with an injected `Clock`. Disable the scheduler in tests with `quarkus.scheduler.enabled=false`. If the trigger itself needs testing, use a test profile with a one-second cron and `Awaitility`.

Senior

Q10. Application scheduler or Kubernetes `CronJob`? It depends on frequency, cost and visibility of the side effect. `@Scheduled` is right for frequent, cheap, in-process work — draining an outbox every second, refreshing a cache — where the overhead of starting a pod per execution would dominate and where duplicate execution across replicas is harmless. A `CronJob` is right for infrequent, expensive, externally-visible batch work: it gives single execution without any clustering machinery, its own resource limits so a heavy job cannot starve request-serving pods, independent retry and backoff, and a run history operators can inspect. The traditional objection — JVM startup cost per run — is largely gone with Quarkus, and entirely gone in native mode. I would reach for clustered Quartz specifically when I need dynamic schedules created at runtime, misfire policies, or persistent job state, which neither of the other options gives.

Q11. How do you know a scheduled job stopped running? You cannot rely on failure signals, because the failure mode is absence: nothing throws, nothing logs, work simply stops. So I instrument for staleness rather than for errors — a gauge recording seconds since the last successful completion, exported to Prometheus, with an alert threshold slightly above the expected interval. That single metric catches every cause: a property set to `off` during an incident and never restored, a stuck clustered lock, an unscheduled trigger, a pod that is running but whose scheduler thread died, and a job silently skipping every occurrence because the previous run never finished. I would also record run count by outcome and duration, so a job that is running but taking longer each night is visible before it crosses its interval. And for critical jobs I would add a liveness or readiness check keyed on the same staleness value, so the platform itself reacts rather than waiting for a human.

Q12. Design scheduled work for a multi-tenant SaaS where each tenant has its own schedule. Static `@Scheduled` annotations cannot express per-tenant schedules, so this needs programmatic scheduling: register a job per tenant at startup via the `Scheduler` API, keyed by tenant id, and add or remove jobs as tenants are onboarded and offboarded. That immediately raises the coordination question, because with N replicas each would register the same jobs — so this needs the clustered Quartz store, where the trigger definitions live in the database and are shared. At that point the design is really a job registry: tenant schedules stored as data, a reconciliation loop that makes the scheduler's state match the table, and idempotent execution so a rebalance or a double-fire is harmless. I would cap concurrency deliberately, because ten thousand tenants with a midnight schedule means ten thousand simultaneous jobs; staggering by hashing the tenant id into the minute field spreads the load and is the sort of thing that is easy to add early and painful to retrofit. Per-tenant observability matters too — a staleness metric tagged by tenant, since “the job is running” is no longer a single boolean. If the tenant count grows large, I would move from a job-per-tenant model to a single sweeper job that processes tenants due for work, because tens of thousands of Quartz triggers is an operational burden that a `WHERE next_run_at < now()` query is not.

12.12 Cheat Sheet

Annotations

```
@Scheduled(every = "30s")
@Scheduled(every = "1m", delayed = "30s")
@Scheduled(cron = "0 0 2 * * ?")
@Scheduled(cron = "{job.cron:off}")
@Scheduled(cron = "0 0 9 * * ?", timeZone = "Europe/London")
@Scheduled(every = "10s", concurrentExecution = ConcurrentExecution.SKIP)
@Scheduled(every = "10s", skipExecutionIf = MyPredicate.class)
@Scheduled(every = "1m") @RunOnVirtualThread
```

Quartz cron fields

sec	min	hour	day-of-month	month	day-of-week	[year]	
0	0	2	*	*	?		-> 02:00 daily
0	*/15	*	*	*	?		-> every 15 min
0	0	9	?	*	MON-FRI		-> 09:00 weekdays

Clustering

```
quarkus.quartz.clustered=true
quarkus.quartz.store-type=jdbc-cmt
quarkus.quartz.start-mode=forced
```

Programmatic

```
@Inject Scheduler scheduler;
scheduler.newJob("id").setCron("...").setTask(exec -> work()).schedule();
scheduler.unscheduleJob("id");
scheduler.pause(); scheduler.resume();
```

Events

```
void ok(@Observes SuccessfulExecution e) { }
void bad(@Observes FailedExecution e) { }
void skip(@Observes SkippedExecution e) { }
```

Testing

```
%test.quarkus.scheduler.enabled=false
```

12.13 Summary

Scheduling in Quarkus is a one-annotation affair, and the annotation is the least interesting part. The question that determines whether your job is correct is **what happens when there are three replicas** — because the default scheduler is per-JVM and every pod fires every trigger. For idempotent local work that is fine; for anything with an external side effect it is triple execution, and it is the most common scheduling bug in Kubernetes.

Three answers: clustered Quartz with a database lock, an application-level lock with a TTL, or moving the job out to a Kubernetes [CronJob](#). The last is usually the best choice for infrequent, expensive, externally-visible batch work, and Quarkus’s fast startup removes the traditional objection to it.

Beyond that: [SKIP](#) for anything stateful, chunked transactions rather than one long one, UTC schedules, and configuration-driven cron so operations can change or disable a job without a deployment.

The observability point is worth repeating because it is counter-intuitive. A scheduled job that stops running **produces no error**. Alert on seconds-since-last-success, not on failure rate — the staleness gauge is the only signal that catches a job which quietly stopped being scheduled at all.

Chapter 13 covers **caching** — the local cache, the multi-replica staleness problem that mirrors this chapter’s multi-replica execution problem, cache stampedes, and when caching is the wrong answer to a query problem.

12.14 References

- Scheduling periodic tasks — <https://quarkus.io/guides/scheduler>
- Scheduler reference — <https://quarkus.io/guides/scheduler-reference>
- Quartz — <https://quarkus.io/guides/quartz>
- Kubernetes CronJob — <https://kubernetes.io/docs/concepts/workloads/controllers/cron-job/>
- Quartz cron expression reference — <https://www.quartz-scheduler.org/documentation/quartz-2.3.0/tutorials/crontrigger.html>

Caching

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-cache` (Caffeine) · `quarkus-redis-cache` · Infinispan

Learning Objectives

By the end of this chapter you will be able to:

1. Decide **whether to cache at all** — and recognise when caching is masking a design problem.
2. Use `@CacheResult`, `@CacheInvalidate`, `@CacheInvalidateAll` and `@CacheKey` correctly.
3. Configure eviction, expiry and sizing with reasons rather than round numbers.
4. Explain the **multi-replica staleness problem** and choose between local and distributed caching.
5. Prevent **cache stampedes** and understand why they happen.
6. Use the programmatic cache API for cases annotations cannot express.
7. Apply HTTP caching — `ETag`, `Cache-Control` — which is often better than an application cache.
8. Instrument caches so hit ratio and eviction rate are visible.
9. Test cached code without the cache making tests lie.
10. Answer 15 interview questions on caching, invalidation and consistency.

13.1 Should You Cache?

Caching is the most frequently misapplied performance tool, because it reliably makes a benchmark look better while making the system harder to reason about. Before adding a cache, work through this:

1. **Is the operation actually slow?** Measure. A 3 ms query does not need a cache; a 300 ms one might.
2. **Why is it slow?** If the answer is N+1, a missing index, or loading entities where a projection would do, **fix that instead**. Caching a badly-shaped query preserves the bad shape and hides it until the data grows enough to overwhelm the cache too.
3. **Is the data read far more than it is written?** Caching write-heavy data spends more on invalidation than it saves on reads.
4. **Can you tolerate staleness, and for how long?** This is a product question, not a technical one. “Is it acceptable for a price change to take 60 seconds to appear?” needs an answer from someone who owns the outcome.

5. What happens when the cache is cold or unavailable? If the system cannot survive a cold start, the cache is not an optimisation — it is a load-bearing dependency, and it needs the availability treatment that implies.

The strongest interview answer about caching is usually about not caching. “We found the endpoint was doing an N+1 query; fixing the fetch strategy took it from 400 ms to 20 ms and we did not need a cache” demonstrates more than any amount of cache configuration. Reach for caching when the work is genuinely irreducible — an expensive aggregation, a third-party API call, a computation — not as a first response to slowness.

13.1.1 What caches well

- **Reference data** — country codes, tax rates, product catalogues, feature configuration.
- **Expensive third-party calls** — geocoding, currency rates, credit checks.
- **Expensive computations** — pricing rules, report aggregations.
- **Authorisation lookups** — the permission set for a subject, which is read on every request.

13.1.2 What does not

- **Write-heavy entities.** Invalidation churn exceeds the benefit.
- **User-specific data with low reuse.** A cache with a 2 % hit rate is pure overhead.
- **Anything requiring strong consistency.** A cache is by definition a consistency compromise.
- **Large objects, in quantity.** You have moved a database problem into a heap problem.

13.2 The Quarkus Cache Extension

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-cache</artifactId>
</dependency>
```

The default backend is **Caffeine**, an in-process, per-JVM cache. Remember that: *per-JVM*. It is the source of §13.5.

13.2.1 Annotations

```
package com.acme.orders.pricing;

import io.quarkus.cache.CacheInvalidate;
import io.quarkus.cache.CacheInvalidateAll;
import io.quarkus.cache.CacheKey;
import io.quarkus.cache.CacheResult;

@ApplicationScoped
public class TaxRateService {

    // The method runs once per distinct key; afterwards the cached value is returned.
    @CacheResult(cacheName = "tax-rates")
```

```

    public TaxRate rateFor(String countryCode) {
        return taxApi.lookup(countryCode);           // slow third-party call
    }

    // Multiple parameters: the key is a composite of all of them
    @CacheResult(cacheName = "tax-rates-regional")
    public TaxRate rateFor(String countryCode, String region) { ... }

    // @CacheKey restricts the key to the annotated parameters
    @CacheResult(cacheName = "tax-rates")
    public TaxRate rateFor(@CacheKey String countryCode, RequestContext ignored) { ... }

    // Remove one entry
    @CacheInvalidate(cacheName = "tax-rates")
    public void refresh(String countryCode) { ... }

    // Clear the whole cache
    @CacheInvalidateAll(cacheName = "tax-rates")
    public void refreshAll() { ... }

    // Update: invalidate then repopulate in one call
    @CacheInvalidate(cacheName = "tax-rates")
    @CacheResult(cacheName = "tax-rates")
    public TaxRate forceRefresh(String countryCode) {
        return taxApi.lookup(countryCode);
    }
}

```

13.2.2 Key construction rules

The composite key is built from **all** method parameters unless **@CacheKey** is present, in which case only the annotated ones are used. Two consequences catch people:

```

// BUG: the key includes `now`, so it is never the same twice.
// Every call misses, and the cache grows without bound until eviction.
@CacheResult(cacheName = "rates")
public Rate rate(String code, Instant now) { ... }

// Fixed
@CacheResult(cacheName = "rates")
public Rate rate(@CacheKey String code, Instant now) { ... }

// BUG: a key type without a stable equals/hashCode.
// Every call is a miss because the default identity comparison never matches.
@CacheResult(cacheName = "quotes")
public Quote quote(QuoteRequest request) { ... }    // if QuoteRequest is a plain class

```

Key types must implement **equals** and **hashCode**. Records do so automatically, which is one more reason to use them for parameter objects.

13.2.3 The self-invocation rule applies

`@CacheResult` is an interceptor binding, so — exactly as with `@Transactional` in Chapter 7 — calling the method through `this` bypasses it entirely:

```
@ApplicationScoped
public class PricingService {

    public BigDecimal priceAll(List<Item> items) {
        return items.stream()
            .map(i -> rateFor(i.country()))    // NOT cached - direct call
            .reduce(...);
    }

    @CacheResult(cacheName = "tax-rates")
    public TaxRate rateFor(String country) { ... }
}
```

The symptom is a cache with a 0 % hit rate and no obvious explanation. Inject the bean into itself, or move the cached method to another bean — the latter is usually the better design.

13.3 Configuration

```
# Per-cache configuration
quarkus.cache.caffeine."tax-rates".initial-capacity=100
quarkus.cache.caffeine."tax-rates".maximum-size=10000
quarkus.cache.caffeine."tax-rates".expire-after-write=1H
quarkus.cache.caffeine."tax-rates".expire-after-access=30M
quarkus.cache.caffeine."tax-rates".metrics-enabled=true

# Defaults for every cache
quarkus.cache.caffeine.maximum-size=1000
quarkus.cache.caffeine.expire-after-write=10M

# Disable caching entirely - useful in tests
%test.quarkus.cache.enabled=false
```

13.3.1 expire-after-write versus expire-after-access

- **expire-after-write** — the entry expires a fixed time after it was created, regardless of use. This bounds **staleness**, which is what you usually care about.
- **expire-after-access** — the entry expires a fixed time after it was last read. This bounds **memory** by evicting unused entries, but a hot entry can stay cached indefinitely and become arbitrarily stale.

Use **expire-after-write** as your primary control, and add **expire-after-access** only to reclaim memory from cold entries.

```
# Bound staleness at 5 minutes; also drop entries nobody has read for an hour
quarkus.cache.caffeine."tax-rates".expire-after-write=5M
quarkus.cache.caffeine."tax-rates".expire-after-access=1H
```

13.3.2 Sizing

maximum-size is an entry count, not a memory limit. A cache of 10 000 entries holding 1 MB objects is 10 GB. Size it from the object size and your heap budget, not from a round number, and verify with a heap dump under load if the objects are non-trivial.

An unbounded cache is a memory leak with good intentions. Always set **maximum-size**. The default in Quarkus is unbounded unless you configure one, and a cache keyed by something unexpectedly high-cardinality — a user id, a request id — will fill the heap.

13.4 The Programmatic API

Annotations cannot express conditional caching, dynamic cache names, or bulk operations.

```
import io.quarkus.cache.Cache;
import io.quarkus.cache.CacheName;
import io.quarkus.cache.CaffeineCache;

@ApplicationScoped
public class PricingService {

    @Inject
    @CacheName("tax-rates")
    Cache cache;

    // get() is atomic per key: concurrent callers for the same key
    // wait for a single computation rather than all computing it.
    public Uni<TaxRate> rateFor(String country) {
        return cache.get(country, key -> taxApi.lookup(key));
    }

    public Uni<TaxRate> rateForAsync(String country) {
        return cache.getAsync(country, key -> taxApi.lookupAsync(key));
    }

    public Uni<Void> invalidate(String country) {
        return cache.invalidate(country);
    }

    public Uni<Void> invalidateAll() {
        return cache.invalidateAll();
    }

    // Conditional caching: only cache results worth caching
    public Uni<TaxRate> conditional(String country) {
        return taxApi.lookupAsync(country)
            .onItem().call(rate -> rate.isStable()
                ? cache.as(CaffeineCache.class).put(country, completedFuture(rate))
                : Uni.createFrom().voidItem());
    }

    // Inspect - useful for admin endpoints and diagnostics
```

```

public Set<Object> keys() {
    return cache.as(CaffeineCache.class).keySet();
}

```

`cache.get(key, loader)` is the important one. It computes atomically per key, so if fifty concurrent requests miss on the same key, the loader runs **once** and the other forty-nine wait. That is the built-in defence against the stampede in §13.6, and it is why you should prefer it to a manual `if (cached == null) { compute(); put(); }` pattern, which has exactly that race.

13.5 The Multi-Replica Staleness Problem

This mirrors Chapter 12’s multi-replica execution problem, and it is just as commonly missed.

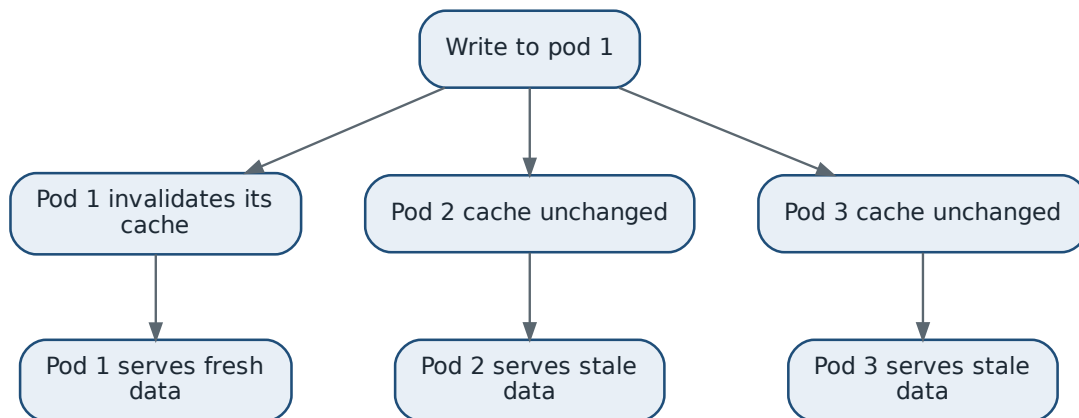


Figure 13.1 — A local cache invalidation reaches one pod. The others keep serving the old value until their own TTL expires.

Caffeine is per-JVM. `@CacheInvalidate` clears the entry **on the pod that handled the write**. Every other replica keeps serving the old value until its TTL expires.

The symptom is maddening: a user updates a setting, refreshes, and sees the new value; refreshes again and sees the old one; refreshes again and it is new. They are hitting different pods. It is intermittent, unreproducible locally, and looks like a bug in the write path.

13.5.1 Option 1: accept it, with a short TTL

If a few seconds of staleness is acceptable, a short `expire-after-write` bounds the inconsistency and costs nothing operationally.

```
quarkus.cache.caffeine."feature-flags".expire-after-write=30S
```

This is the right answer far more often than people expect. Be explicit that it is a decision, document the staleness window, and make sure the product owner knows.

13.5.2 Option 2: a distributed cache

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-redis-cache</artifactId>
</dependency>
```

```
quarkus.cache.type=redis
quarkus.redis.hosts=redis://redis:6379
quarkus.cache.redis.expire-after-write=10M
quarkus.cache.redis."tax-rates".expire-after-write=1H
quarkus.cache.redis."tax-rates".value-type=com.acme.orders.pricing.TaxRate
```

The same annotations now work against Redis, so invalidation is global. What you pay:

- **A network round trip per lookup.** A Redis hit is perhaps 1 ms; a Caffeine hit is nanoseconds. For a cache in front of a 3 ms query, Redis may be slower than no cache at all.
- **Serialisation cost** on every read and write.
- **A new critical dependency.** Decide explicitly whether a Redis outage should fail the request or fall through to the source.

13.5.3 Option 3: two-tier

The pattern used by most mature systems: a small local cache in front of a shared one.

```
@ApplicationScoped
public class TwoTierCache {

    @Inject @CacheName("local-tax-rates") Cache local;    // Caffeine, 10s TTL
    @Inject ReactiveRedisDataSource redis;                // shared, 1h TTL

    public Uni<TaxRate> rateFor(String country) {
        return local.get(country, key ->
            redis.value(TaxRate.class).get("tax:" + key)
                .onItem().ifNull().switchTo(() ->
                    taxApi.lookupAsync(key)
                        .call(rate -> redis.value(TaxRate.class)
                            .setex("tax:" + key, 3600, rate)))
                .await().indefinitely());    // inside the loader
    }
}
```

The local tier absorbs the request-rate load; the shared tier bounds cross-pod divergence to the local TTL. Ten seconds of local TTL means at most ten seconds of divergence, with Redis hit only once per pod per ten seconds.

13.5.4 Option 4: invalidation broadcast

Publish invalidation events on a Kafka topic or Redis pub/sub, and have every pod clear its local entry.

```
@ApplicationScoped
public class CacheInvalidationListener {
```

```

@Inject @CacheName("tax-rates") Cache cache;

@Incoming("cache-invalidations")
public Uni<Void> onInvalidation(InvalidationEvent event) {
    return "tax-rates".equals(event.cacheName())
        ? cache.invalidate(event.key())
        : Uni.createFrom().voidItem();
}

```

This gives near-immediate global invalidation with local-cache read performance. The cost is a messaging dependency and the fact that invalidation is now best-effort and asynchronous — a pod that was down during the broadcast comes back with a stale entry, so you still want a TTL as a backstop.

13.6 Cache Stampede

A popular key expires. Fifty concurrent requests miss simultaneously. All fifty execute the expensive load. The downstream service, which was comfortably handling a cached load of one request per minute, receives fifty at once.

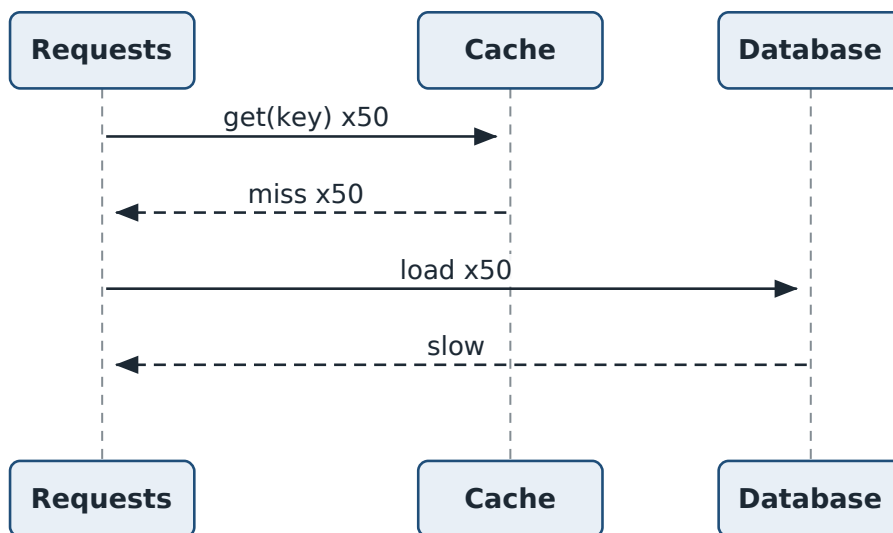


Figure 13.2 — Stampede. The expiry of one hot key becomes a load spike downstream.

Defence 1 — atomic loading. `cache.get(key, loader)` already does this: concurrent callers for the same key share one computation. Use it rather than a manual check-then-load.

Defence 2 — jittered TTLs. If many entries were populated at the same moment — at startup, or after a bulk invalidation — they will all expire at the same moment. Add randomness:

```

public Uni<TaxRate> rateFor(String country) {
    return cache.get(country, key -> {
        TaxRate rate = taxApi.lookup(key);
    });
}

```

```

        // Vary the effective lifetime by writing an expiry into the value
        return rate.withExpiry(Instant.now()
            .plusSeconds(3600 + ThreadLocalRandom.current().nextInt(600)));
    });
}

```

Defence 3 — refresh ahead. Serve the stale value while refreshing in the background, so no request ever waits.

```

@ApplicationScoped
public class RefreshAheadCache {

    @Inject @CacheName("tax-rates") Cache cache;

    @Scheduled(every = "50m", concurrentExecution = SKIP) // before the 1h TTL
    void refresh() {
        for (String country : activeCountries()) {
            taxApi.lookupAsync(country)
                .subscribe().with(rate ->
                    cache.as(CaffeineCache.class).put(country, completedFuture(rate)));
        }
    }
}

```

Defence 4 — never let the cache be load-bearing. If the source cannot survive a full cache miss, the cache is a single point of failure. Rate-limit the loader, or keep a stale-but-usable fallback.

13.7 HTTP Caching

Frequently better than an application cache, because the work is avoided entirely rather than made faster.

```

@Path("/products")
public class ProductResource {

    @GET
    @Path("/{id}")
    public RestResponse<ProductView> get(@RestPath long id,
        @HeaderParam("If-None-Match") String ifNoneMatch) {

        ProductView product = service.findById(id);
        String etag = "\"" + product.version() + "\"";

        if (etag.equals(ifNoneMatch)) {
            return RestResponse.status(Status.NOT_MODIFIED); // 304, no body
        }

        return RestResponse.ResponseBuilder.ok(product)
            .header("ETag", etag)
            .header("Cache-Control", "private, max-age=60, must-revalidate")
            .build();
    }
}

```

```

@GET
@Path("/catalogue")
public RestResponse<List<ProductView>> catalogue() {
    return RestResponse.ResponseBuilder.ok(service.catalogue())
        .header("Cache-Control", "public, max-age=3600, stale-while-revalidate=600")
        .build();
}
}

```

```

// Quarkus's declarative shorthand
@GET
@CacheControl(maxAge = 3600, noStore = false)
public List<ProductView> cached() { ... }

```

Why this often wins: a 304 response transfers no body, and a CDN or reverse proxy can serve **public** responses without your service being involved at all. **stale-while-revalidate** lets the proxy serve a slightly-stale response immediately while refreshing in the background — the refresh-ahead pattern, implemented by infrastructure you already have.

Never set **Cache-Control: public** on an authenticated response. A shared proxy will cache one user's data and serve it to another. Authenticated responses should be **private** at most, and **no-store** when they contain anything sensitive.

13.8 Metrics

```

quarkus.cache.caffeine."tax-rates".metrics-enabled=true
quarkus.micrometer.enabled=true

```

```

cache_gets_total{cache="tax-rates",result="hit"}
cache_gets_total{cache="tax-rates",result="miss"}
cache_puts_total{cache="tax-rates"}
cache_evictions_total{cache="tax-rates"}
cache_size{cache="tax-rates"}

```

What to watch, and what each number means:

- **Hit ratio.** Below roughly 80 % on a cache you added deliberately, question whether it is earning its complexity. Near 0 % usually means a key problem — a timestamp in the key, or a key type without **equals/hashCode**, or self-invocation.
- **Eviction rate.** High evictions with a low hit ratio means the cache is too small for the working set; you are paying the cost and getting the benefit for almost nothing.
- **Size against maximum-size.** Consistently at the limit means the working set exceeds capacity.
- **Load time.** If the loader is slow, misses are expensive and the tail latency of your endpoint is dominated by them.

```

@ApplicationScoped
public class CacheHealth {

    @Inject MeterRegistry registry;
}

```

```

@Scheduled(every = "5m")
void report() {
    double hits    = counter("cache_gets_total", "result", "hit");
    double misses = counter("cache_gets_total", "result", "miss");
    double ratio   = hits / Math.max(hits + misses, 1);
    if (ratio < 0.5) {
        Log.warnf("Cache hit ratio is %.1f%% - review the key or the TTL", ratio * 100);
    }
}
}

```

13.9 Testing

Disable caching in most tests so they exercise real code paths
 %test.quarkus.cache.enabled=false

That single line prevents a category of confusing test failure, where a test passes because a previous test populated the cache, or fails because it did.

```

@QuarkusTest
class TaxRateServiceCacheTest {

    @Inject TaxRateService service;
    @InjectMock TaxApiClient taxApi;
    @Inject @CacheName("tax-rates") Cache cache;

    @BeforeEach
    void clear() {
        cache.invalidateAll().await().indefinitely();
        Mockito.reset(taxApi);
    }

    @Test
    void callsTheApiOnlyOnce() {
        when(taxApi.lookup("GB")).thenReturn(new TaxRate("GB", new BigDecimal("0.20")));

        service.rateFor("GB");
        service.rateFor("GB");
        service.rateFor("GB");

        verify(taxApi, times(1)).lookup("GB");    // cached after the first
    }

    @Test
    void invalidationForcesReload() {
        when(taxApi.lookup("GB")).thenReturn(rate("0.20"), rate("0.21"));

        assertThat(service.rateFor("GB").value()).isEqualByComparingTo("0.20");
        service.refresh("GB");
        assertThat(service.rateFor("GB").value()).isEqualByComparingTo("0.21");
    }
}

```

```

@Test
void differentKeysAreCachedSeparately() {
    service.rateFor("GB");
    service.rateFor("DE");
    verify(taxApi).lookup("GB");
    verify(taxApi).lookup("DE");
}
}

```

Note the profile requirement: with `%test.quarkus.cache.enabled=false` globally, these cache-specific tests need a profile that re-enables it.

13.10 Best Practices

1. **Fix the query before adding a cache.** N+1 and missing indexes are the usual real cause.
2. **Always set `maximum-size`.** An unbounded cache is a memory leak.
3. **`expire-after-write` to bound staleness; `expire-after-access` only to reclaim memory.**
4. **Decide the multi-replica story explicitly** — short TTL, distributed cache, two-tier, or broadcast invalidation.
5. **`cache.get(key, loader)`, not manual check-then-load** — it is stampede-safe.
6. **Records as key types.** Stable `equals/hashCode` for free.
7. **`@CacheKey` when not every parameter belongs in the key.**
8. **Never cache in a self-invoked method.** The interceptor is bypassed.
9. **Prefer HTTP caching where the client is a browser or a CDN** — the work is avoided entirely.
10. **Never `Cache-Control: public` on an authenticated response.**
11. **Instrument every cache** and alert on a collapsing hit ratio.
12. **Disable caching in tests by default.**
13. **Assume the cache can be empty.** If the system cannot survive that, the cache is a dependency, not an optimisation.

13.11 Common Mistakes

1. **Caching to hide an N+1 query.** The design problem survives and returns at larger scale.
2. **No `maximum-size`.** Heap exhaustion under an unexpected key cardinality.
3. **A timestamp or request id in the cache key.** Every call misses; the cache only consumes memory.
4. **A key type without `equals/hashCode`.** Same symptom, harder to spot.
5. **Self-invocation.** `this.cachedMethod()` bypasses the interceptor. 0 % hit rate, no explanation.
6. **Assuming `@CacheInvalidate` is global.** It clears one pod. The rest serve stale data until TTL.
7. **Ignoring the stampede.** A hot key expiring becomes a downstream load spike.
8. **Synchronised TTLs.** Entries populated together expire together. Jitter them.

9. Caching mutable objects. A caller mutates the cached instance and every subsequent reader sees the change. Cache immutable values — records.

10. Cache-Control: public on authenticated responses. A shared proxy serves one user's data to another.

11. A cache the system cannot run without. That is a dependency with an availability requirement, not an optimisation.

12. Caching enabled in tests. Cross-test pollution and order-dependent failures.

13.12 Interview Questions

Junior

Q1. How do you cache a method result in Quarkus? `@CacheResult(cacheName = "name")` on the method, with `quarkus-cache` on the classpath. The key is built from the method parameters.

Q2. How do you evict an entry? `@CacheInvalidate(cacheName = "name")` on a method whose parameters form the key, or `@CacheInvalidateAll` to clear the whole cache.

Q3. What is `@CacheKey` for? Restricting the key to specific parameters when not all of them are part of the identity of the result.

Q4. What cache does Quarkus use by default? Caffeine, an in-process per-JVM cache.

Q5. What is `expire-after-write`? An entry expires a fixed time after it was created, regardless of how often it is read — which is how you bound staleness.

Mid-level

Q6. What happens to `@CacheInvalidate` when you run three replicas? It clears the entry only on the pod that handled the request. The other two keep serving the old value until their own TTL expires. The symptom is intermittent stale reads that depend on which pod the client happens to hit.

Q7. How do you solve that? Four options: accept it with a short TTL and document the staleness window; use a distributed cache such as Redis so invalidation is global, paying a network round trip per lookup; use a two-tier cache with a small local TTL in front of a shared one; or broadcast invalidation events so every pod clears its own entry. The right choice depends on how much staleness the product can tolerate and how sensitive the read path is to latency.

Q8. What is a cache stampede? A popular key expires and many concurrent requests miss at once, all executing the expensive load simultaneously and spiking the downstream system. `cache.get(key, loader)` mitigates it by computing once per key while other callers wait; jittered TTLs prevent synchronised expiry; refresh-ahead avoids the miss entirely.

Q9. Why does my cache have a 0 % hit rate? Usually one of three things: the key includes something that varies every call, such as a timestamp; the key type has no stable `equals/hashCode`; or the method is being called through `this` from within the same bean, which bypasses the interceptor entirely.

Q10. `expire-after-write` versus `expire-after-access`? `expire-after-write` bounds staleness — the entry dies at a fixed age. `expire-after-access` bounds memory — unused entries are evicted,

but a frequently-read entry can live indefinitely and become arbitrarily stale. Use the former as the primary control.

Q11. When would you use HTTP caching instead? When the client is a browser or sits behind a CDN. An `ETag` with a 304 avoids transferring the body, and a `public Cache-Control` lets a proxy serve the response without touching your service at all — which is strictly better than making your service faster. `stale-while-revalidate` gives you refresh-ahead behaviour for free.

Senior

Q12. Design caching for a product catalogue read by 40 services. The catalogue is read-heavy and written rarely, which is the ideal profile, so caching is genuinely appropriate here rather than a workaround. I would use a two-tier design: a small Caffeine cache in each service with a short TTL — tens of seconds — in front of a shared Redis tier with a much longer one. The local tier absorbs the request rate so Redis is hit once per pod per local TTL rather than once per request, and the local TTL bounds cross-pod divergence to a window the business can be told about. On writes, the catalogue service invalidates Redis and publishes an invalidation event on Kafka that all consumers apply to their local caches, so the common case converges in milliseconds while the TTL remains the backstop for a pod that missed the event. I would cache immutable record types to avoid shared-mutable-state bugs, set an explicit `maximum-size` derived from measured object size, and instrument hit ratio per tier. The design decision I would put to the product owner explicitly is the local TTL, because it *is* the staleness guarantee, and nobody should discover it from an incident.

Q13. A team added Redis caching and latency got worse. Explain. Almost certainly because the cached operation was faster than the cache. A Redis lookup is a network round trip plus serialisation — typically around a millisecond — so putting it in front of a 2 ms indexed primary-key query adds latency on hits and adds latency on misses too, since a miss now costs the Redis round trip *plus* the query. The hit ratio matters enormously here: at 50 % you pay the Redis cost on every request and avoid the query on half of them, which is a losing trade for a fast query. I would look at the hit ratio and the source query time first, and I would expect the fix to be either removing the cache entirely, or moving it in-process with Caffeine where a hit costs nanoseconds. There is a second possibility worth checking: if the cached objects are large, serialisation and network transfer may dominate, in which case caching a smaller projection rather than the full object changes the arithmetic. The general lesson is that a cache is only a win when the cached work is substantially more expensive than the cache lookup, and people rarely measure that before adding one.

Q14. How do you decide the TTL for a cache? Backwards, from the staleness the business can tolerate — not from a technical guess. I would ask what the consequence of serving a value N seconds out of date is, and for what N that consequence becomes unacceptable; that gives an upper bound. Then I would check whether the resulting hit ratio and load reduction actually justify the cache at that TTL, because a TTL short enough to be safe is sometimes short enough to be pointless, and that is a useful thing to discover early. Where the tolerance is genuinely tight but the read volume is high, the answer is usually not a shorter TTL but active invalidation — event-driven or write-through — with the TTL as a backstop rather than the primary mechanism. I would also stagger TTLs with jitter so entries do not expire in lockstep, and I would document the chosen window somewhere the product owner will see it, because “the price took two minutes to update” is a support ticket that should be answerable by pointing at a decision rather than by investigating.

Principal

Q15. Your platform's services each cache independently and users report inconsistent data across features. How do you address it? The visible problem is inconsistency, but the underlying one is that caching decisions were made locally without anyone owning the end-to-end staleness contract, so I would start by making the contract explicit rather than by standardising the technology. Concretely: inventory what is cached where and with what TTL, and map it against the user journeys where inconsistency is being reported — usually a small number of shared entities, such as a customer profile or a product, cached in six services with six different TTLs, so a user sees a change reflected in one screen and not another. For those entities I would define a single owning service, a published staleness budget, and an invalidation event on a shared topic that every consumer honours; the TTL then becomes a backstop rather than the mechanism, and the budget is a number the product organisation has agreed to rather than an accident of configuration. Technically I would ship this as a shared library — a cache wrapper that registers the invalidation listener and exposes hit-ratio metrics with consistent naming — so adoption is a dependency rather than a document, and so the platform dashboard can show staleness by entity across all services. I would resist the instinct to mandate one cache technology, because a local Caffeine cache for reference data and a Redis tier for shared state are both correct in their places. The thing I would mandate is that any cached shared entity has a named owner and a published budget, because the inconsistency users are reporting is a governance failure rather than a technical one.

13.13 Cheat Sheet

Annotations

```
@CacheResult(cacheName = "n")           // cache the return value
@CacheInvalidate(cacheName = "n")        // evict one key
@CacheInvalidateAll(cacheName = "n")      // clear the cache
@CacheKey                                 // restrict the key to these parameters
```

Configuration

```
quarkus.cache.caffeine."n".maximum-size=10000
quarkus.cache.caffeine."n".expire-after-write=1H
quarkus.cache.caffeine."n".expire-after-access=30M
quarkus.cache.caffeine."n".metrics-enabled=true
%test.quarkus.cache.enabled=false
```

Redis backend

```
quarkus.cache.type=redis
quarkus.redis.hosts=redis://redis:6379
quarkus.cache.redis."n".expire-after-write=1H
quarkus.cache.redis."n".value-type=com.acme.TaxRate
```

Programmatic

```
@Inject @CacheName("n") Cache cache;

cache.get(key, k -> load(k));           // atomic per key - stampede-safe
cache.getAsync(key, k -> loadUni(k));
cache.invalidate(key);
cache.invalidateAll();
```

```
cache.as(CaffeineCache.class).keySet();
cache.as(CaffeineCache.class).put(key, completedFuture(value));
```

HTTP caching

```
RestResponse.ResponseBuilder.ok(body)
    .header("ETag", etag)
    .header("Cache-Control", "private, max-age=60, must-revalidate")
    .build();

// 304 when the ETag matches
RestResponse.status(Status.NOT_MODIFIED);
```

Metrics

```
cache_gets_total{cache,result}   cache_evictions_total{cache}   cache_size{cache}
```

Diagnosing a 0 % hit rate

1. Does the key vary every call (timestamp, request id)?
2. Does the key type have equals/hashCode? (use a record)
3. Is the method self-invoked through this? (interceptor bypassed)

13.14 Summary

The most useful thing to know about caching is when not to do it. A cache in front of an N+1 query or a missing index preserves the underlying problem and defers it until the data outgrows the cache too — so measure, understand *why* the operation is slow, and fix the shape before adding a layer.

When caching is warranted, three mechanics matter. **Keys:** built from all parameters unless `@CacheKey` narrows them, requiring stable `equals/hashCode`, and bypassed entirely by self-invocation — which accounts for most mysterious 0 % hit rates. **Bounds:** always set `maximum-size`, and use `expire-after-write` to bound staleness rather than `expire-after-access`, which bounds only memory. **Loading:** use `cache.get(key, loader)`, which computes once per key under concurrency and is your built-in stampede defence.

The design question is the same one Chapter 12 posed about scheduling: **what happens with three replicas?** Caffeine is per-JVM, so `@CacheInvalidate` clears one pod and the others serve stale data until their TTL expires. Choose deliberately between a short TTL, a distributed cache, a two-tier arrangement, or broadcast invalidation — and treat the resulting staleness window as a product decision that someone has agreed to, not an accident of configuration.

Finally, prefer HTTP caching where the client permits it. A 304 or a CDN hit avoids the work entirely, which beats making the work faster.

Chapter 14 begins the observability sequence with **metrics and Micrometer** — what to measure, how to name it, and the cardinality mistake that takes down monitoring systems.

13.15 References

- Application data caching — <https://quarkus.io/guides/cache>

- Redis cache backend — <https://quarkus.io/guides/cache-redis-reference>
- Infinispan client — <https://quarkus.io/guides/infinispan-client>
- Caffeine — <https://github.com/ben-manes/caffeine/wiki>
- RFC 9111, HTTP Caching — <https://www.rfc-editor.org/rfc/rfc9111>
- `stale-while-revalidate` — <https://www.rfc-editor.org/rfc/rfc5861>

Metrics and Micrometer

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-micrometer-registry-prometheus` · Micrometer 1.14 · Prometheus · Grafana

Learning Objectives

By the end of this chapter you will be able to:

1. Choose the right instrument — counter, gauge, timer, distribution summary — for a given question.
2. Explain **cardinality** and why it is the failure mode that takes down monitoring systems.
3. Instrument application code with annotations and programmatically, without leaking implementation detail into metric names.
4. Read and configure the built-in Quarkus metrics: HTTP, JVM, datasource, Kafka, cache.
5. Define **SLIs** and derive alerts from them, rather than alerting on every metric you happen to have.
6. Use histograms and percentiles correctly — including why averaging percentiles across pods is wrong.
7. Apply the RED and USE methods to decide what to measure.
8. Expose metrics safely, and understand the scraping model.
9. Test instrumentation.
10. Answer 15 interview questions on metrics and monitoring design.

14.1 Setup

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-micrometer-registry-prometheus</artifactId>
</dependency>
```

```
quarkus.micrometer.enabled=true
quarkus.micrometer.export.prometheus.enabled=true
quarkus.micrometer.export.prometheus.path=/q/metrics
```

```
# Built-in binders
quarkus.micrometer.binder.jvm=true
quarkus.micrometer.binder.system=true
quarkus.micrometer.binder.http-server.enabled=true
quarkus.micrometer.binder.http-client.enabled=true
```

```
quarkus.micrometer.binder.kafka.enabled=true
quarkus.micrometer.binder.vertx.enabled=true
quarkus.micrometer.binder.grpc-server.enabled=true
```

```
# Tags applied to every metric emitted by this service
quarkus.micrometer.export.prometheus.default-registry=true
```

Quarkus uses **Micrometer**, a vendor-neutral facade — the SLF4J of metrics. You instrument once and choose the backend by dependency: Prometheus, Datadog, New Relic, OTLP, SignalFx, StatsD.

MicroProfile Metrics (`quarkus-smallrye-metrics`) is deprecated. If you encounter `@Counted`, `@Timed` from `org.eclipse.microprofile.metrics`, that is the old stack. Micrometer is the current answer, and the annotations look similar but come from `io.micrometer.core`. annotation.

14.2 The Instruments

Instrument	Answers	Example
Counter	How many, cumulatively?	Orders created, errors, cache misses
Gauge	What is it right now?	Queue depth, active connections, pool size
Timer	How long, and how many?	Request duration, query duration
Distribution summary	How big, and how many?	Payload size, batch size, order value
Long task timer	How long has this been running?	An in-flight import job

14.2.1 Counters

```
@ApplicationScoped
public class OrderMetrics {

    @Inject MeterRegistry registry;

    private Counter created;
    private Counter rejected;

    @PostConstruct
    void init() {
        created = Counter.builder("orders.created")
            .description("Orders successfully created")
            .baseUnit("orders")
            .tag("service", "orders")
            .register(registry);
    }
}
```

```

        rejected = Counter.builder("orders.rejected")
            .description("Orders rejected before creation")
            .register(registry);
    }

    public void recordCreated(String channel) {
        // A tag varying per call needs a per-tag counter; Micrometer caches these
        registry.counter("orders.created", "channel", channel).increment();
    }
}

```

A counter only goes up. Prometheus computes rates from it: `rate(orders_created_total[5m])`. Never use a counter for something that can decrease — that is a gauge.

14.2.2 Gauges

```

@ApplicationScoped
public class QueueMetrics {

    @Inject MeterRegistry registry;
    private final Queue<Task> queue = new ConcurrentLinkedQueue<>();

    @PostConstruct
    void init() {
        // The registry holds a WEAK reference to the object
        Gauge.builder("orders.queue.depth", queue, Collection::size)
            .description("Pending tasks in the in-process queue")
            .register(registry);

        registry.gaugeCollectionSize("orders.queue.size", Tags.empty(), queue);
    }
}

```

The weak-reference rule is the classic gauge bug. Micrometer holds the gauged object weakly so that instrumentation cannot cause a memory leak. If nothing else holds a strong reference, the object is collected and the gauge silently reports `NaN` forever. Keep the object in a field, as above — a gauge over a local variable will appear to work in testing and stop working in production once GC runs.

14.2.3 Timers

```

@ApplicationScoped
public class PaymentService {

    @Inject MeterRegistry registry;

    public PaymentResult capture(Order order) {
        Timer.Sample sample = Timer.start(registry);
        String outcome = "success";
    }
}

```

```

    try {
        return paymentClient.capture(order);
    } catch (PaymentDeclinedException e) {
        outcome = "declined";
        throw e;
    } catch (Exception e) {
        outcome = "error";
        throw e;
    } finally {
        sample.stop(Timer.builder("payments.capture")
            .description("Payment capture duration")
            .tag("outcome", outcome)
            .publishPercentiles(0.5, 0.95, 0.99)
            .publishPercentileHistogram() // enables server-side quantiles
            .register(registry));
    }
}

// Convenience wrappers
public Order timed(long id) {
    return registry.timer("orders.load").record(() -> repository.findById(id));
}
}

```

14.2.4 Distribution summaries

```

DistributionSummary orderValue = DistributionSummary.builder("orders.value")
    .description("Monetary value of created orders")
    .baseUnit("GBP")
    .publishPercentiles(0.5, 0.95, 0.99)
    .register(registry);

orderValue.record(order.total.doubleValue());

```

Use these for non-time quantities where the distribution matters: payload sizes, batch sizes, order values. A counter of total value plus a counter of orders gives you a mean; a summary gives you the shape, which is usually the interesting part.

14.3 Annotations

```

import io.micrometer.core.annotation.Counted;
import io.micrometer.core.annotation.Timed;

@ApplicationScoped
public class OrderService {

    @Timed(value = "orders.create",
        description = "Time to create an order",
        extraTags = {"layer", "service"},
        percentiles = {0.5, 0.95, 0.99})

```

```

@Transactional
public Order create(CreateOrderRequest request) { ... }

@Counted(value = "orders.cancelled",
    description = "Cancellation attempts",
    recordFailuresOnly = false)
public void cancel(long id) { ... }
}

```

Annotations are interceptor-based, so — as with `@Transactional` and `@CacheResult` — **self-invocation is not instrumented**. If a `@Timed` method shows zero invocations, that is the first thing to check.

Annotations are convenient for coarse timing. For anything where you need an outcome tag or conditional recording, the programmatic API is clearer and no more code.

14.4 Cardinality — The Failure Mode

This is the section that matters most, and the one interviewers use to distinguish people who have run a monitoring system from people who have added metrics to one.

Every distinct combination of tag values creates a separate time series. Prometheus stores each series independently, with its own memory, index entry and disk footprint.

```

// CATASTROPHIC: one series per user
registry.counter("orders.created", "user_id", userId).increment();

// CATASTROPHIC: one series per order
registry.counter("orders.created", "order_id", orderId).increment();

// BAD: one series per URL, and URLs contain ids
registry.counter("http.requests", "path", request.getUri().getPath()).increment();

// FINE: bounded, small, and useful for slicing
registry.counter("orders.created", "channel", channel, "region", region).increment();

```

The arithmetic: 100 000 users × 5 endpoints × 3 outcomes = 1.5 million series from **one metric**. Prometheus holds roughly 3–8 KB of memory per active series, so that is 5–12 GB — for a single counter. The usual outcome is that the monitoring system falls over, taking with it visibility into every other service at exactly the moment you need it.

14.4.1 Rules

1. **Tag values must come from a bounded set.** Status codes, methods, outcomes, regions, tenants (if you have tens, not millions).
2. **Never tag with an identifier** — user id, order id, request id, session id, trace id.
3. **Never tag with a raw URI.** Use the *template*: `/orders/{id}`, not `/orders/12345`.
4. **Never tag with an exception message.** Use the exception class name, and only if the set is bounded.
5. **Estimate before you ship.** Multiply the cardinality of every tag. If the product exceeds a few hundred, think again.


```
# Quarkus does path templating for you when the route is a JAX-RS resource
quarkus.micrometer.binder.http-server.match-patterns=/orders/.*/orders/{id}
quarkus.micrometer.binder.http-server.ignore-patterns=/q/.*
```

```
# A hard safety limit - meters beyond this are rejected rather than accepted
quarkus.micrometer.binder.http-server.max-uri-tags=100
```

```
// A global filter that denies high-cardinality meters at the registry level
@ApplicationScoped
public class CardinalityGuard implements MeterFilter {

    private static final Set<String> FORBIDDEN =
        Set.of("user_id", "order_id", "request_id", "trace_id", "session_id");

    @Produces
    @Singleton
    public MeterFilter denyHighCardinalityTags() {
        return MeterFilter.denyUnless(id ->
            id.getTags().stream().noneMatch(t -> FORBIDDEN.contains(t.getKey())));
    }
}
```

If an interviewer asks “what is the most dangerous mistake with metrics?”, this is the answer. High-cardinality tags do not fail loudly at the point of the mistake — they fail later, in the monitoring system, affecting every team.

14.5 Global Tags and Naming

```
@ApplicationScoped
public class MetricsConfiguration {

    @ConfigProperty(name = "quarkus.application.name") String application;
    @ConfigProperty(name = "quarkus.application.version") String version;

    @Produces
    @Singleton
    public MeterFilter commonTags() {
        return MeterFilter.commonTags(Tags.of(
            "application", application,
            "version", version,
            "env", ConfigProvider.getConfig()
                .getOptionalValue("deploy.env", String.class).orElse("local")));
    }

    // Rename or drop meters centrally
    @Produces
    @Singleton
    public MeterFilter renameLegacy() {
        return MeterFilter.renameTag("http.server.requests", "uri", "path");
    }
}
```

```

// Apply percentile histograms to a family of meters without touching the code
@Produces
@Singleton
public MeterFilter histogramsForHttp() {
    return new MeterFilter() {
        @Override
        public DistributionStatisticConfig configure(Meter.Id id,
                                                    DistributionStatisticConfig config) {
            if (id.getName().startsWith("http.server.requests")) {
                return DistributionStatisticConfig.builder()
                    .percentilesHistogram(true)
                    .percentiles(0.5, 0.95, 0.99)
                    .serviceLevelObjectives(
                        Duration.ofMillis(100).toNanos(),
                        Duration.ofMillis(500).toNanos(),
                        Duration.ofSeconds(1).toNanos())
                    .build().merge(config);
            }
            return config;
        }
    };
}

```

Naming. Micrometer uses dot-separated names (`orders.created`) and translates to each backend's convention — Prometheus gets `orders_created_total`. Conventions worth following:

- **Lower case, dot-separated, hierarchical:** `area.subarea.measurement`.
- **A base unit in the name or the `baseUnit`:** `duration` in seconds, `size` in bytes.
- **Nouns, not verbs:** `orders.created`, not `create.order`.
- **Do not encode tag values in the name:** `orders.created` with a `channel` tag, not `orders.created.web` and `orders.created.mobile`.

That last one matters: a name is not queryable the way a tag is. `sum by (channel) (rate(orders_created_total[5m]))` only works if `channel` is a tag.

14.6 Built-in Metrics

Quarkus provides a substantial set out of the box.

HTTP server — the most important:

```

http_server_requests_seconds_count{method,uri,outcome,status}
http_server_requests_seconds_sum{...}
http_server_requests_seconds_max{...}
http_server_bytes_read / http_server_bytes_written
http_server_active_requests

```

JVM:

```

jvm_memory_used_bytes{area,id}
jvm_gc_pause_seconds{action,cause}
jvm_threads_live_threads

```

```
jvm_classes_loaded_classes
process_cpu_usage
system_load_average_1m
```

Datasource (Agroal) — worth watching closely, per Chapter 5:

```
agroal_active_count      agroal_available_count
agroal_awaiting_count    agroal_blocking_time_average_ms
agroal_acquire_count     agroal_creation_count
```

Kafka, per Chapter 11:

```
kafka_consumer_fetch_manager_records_lag_max
kafka_consumer_fetch_manager_records_consumed_rate
kafka_producer_record_send_rate
```

Cache, per Chapter 13:

```
cache_gets_total{cache,result}  cache_evictions_total{cache}  cache_size{cache}
```

14.7 What to Measure: RED and USE

Two frameworks that turn “what metrics should I add?” into a checklist.

14.7.1 RED — for request-driven services

- **Rate** — requests per second.
- **Errors** — failed requests per second.
- **Duration** — the distribution of request latency.

```
# Rate
sum(rate(http_server_requests_seconds_count{app="orders"}[5m]))

# Error ratio
sum(rate(http_server_requests_seconds_count{app="orders",status=~"5.."}[5m]))
/
sum(rate(http_server_requests_seconds_count{app="orders"}[5m]))

# p99 latency, correctly aggregated across pods
histogram_quantile(0.99,
  sum by (le, uri) (rate(http_server_requests_seconds_bucket{app="orders"}[5m])))
```

14.7.2 USE — for resources

- **Utilisation** — percentage of time busy.
- **Saturation** — how much work is queued.
- **Errors** — error count.

Applied to a connection pool: utilisation is `agroal_active_count / max`, saturation is `agroal_awaiting_count`, errors are acquisition timeouts. Saturation is the leading indicator — utilisation hits 100 % and stays there, while a growing queue tells you how far past capacity you are.

14.7.3 Business metrics

Technical metrics tell you the system is up. Business metrics tell you it is *working*.

```

@ApplicationScoped
public class BusinessMetrics {

    @Inject MeterRegistry registry;

    void onOrderPlaced(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) {
        registry.counter("business.orders.placed",
            "channel", e.channel(),
            "region", e.region()).increment();
        registry.summary("business.order.value", "currency", e.currency())
            .record(e.total().doubleValue());
    }

    void onPaymentFailed(@Observes PaymentFailed e) {
        registry.counter("business.payments.failed", "reason", e.reason()).increment();
    }
}

```

A drop in orders-per-minute detects outages that no technical metric catches — a broken frontend deployment, an expired third-party API key, a validation rule that rejects everything. Every service should emit at least one metric that a product owner would recognise. Note the tags above are all bounded: channel, region, currency, reason.

14.8 Percentiles Done Right

This is a common interview probe because it is a genuine and widespread mistake.

```

// Client-side percentiles: computed per pod, exported as a value
.publishPercentiles(0.5, 0.95, 0.99)

// Server-side histogram: exports buckets, percentiles computed at query time
.publishPercentileHistogram()

```

You cannot average percentiles. If pod A reports p99 = 100 ms and pod B reports p99 = 200 ms, the fleet's p99 is not 150 ms — it is unknowable from those two numbers. `avg(http_server_requests_seconds{quantile="0.99"})` is a query you will see on real dashboards, and it is meaningless.

Histograms aggregate correctly. Export buckets and let Prometheus compute the quantile across pods:

```

histogram_quantile(0.99,
    sum by (le) (rate(http_server_requests_seconds_bucket[5m])))

```

The cost is cardinality — one series per bucket per label combination — so use `serviceLevelObjectives` to define only the buckets you care about rather than the full default set:

```

.serviceLevelObjectives(
    Duration.ofMillis(50).toNanos(),
    Duration.ofMillis(200).toNanos(),

```

```
Duration.ofMillis(500).toNanos(),
Duration.ofSeconds(2).toNanos())
```

Use client-side percentiles only when you genuinely need per-instance numbers and will not aggregate them.

14.9 Alerting

The purpose of metrics is to make good decisions, and the highest-value decision is “should someone be woken up?”.

14.9.1 Alert on symptoms, not causes

```
# GOOD: a user-visible symptom
- alert: OrdersHighErrorRate
  expr: |
    sum(rate(http_server_requests_seconds_count{app="orders",status=~"5.."}[5m]))
    /
    sum(rate(http_server_requests_seconds_count{app="orders"}[5m])) > 0.01
  for: 5m
  labels: {severity: page}
  annotations:
    summary: "Orders service error rate above 1%"
    runbook: "https://runbooks.acme.com/orders/high-error-rate"

# GOOD: latency SLO breach
- alert: OrdersLatencySLO
  expr: |
    histogram_quantile(0.99,
      sum by (le) (rate(http_server_requests_seconds_bucket{app="orders"}[5m]))) > 1
  for: 10m
  labels: {severity: page}

# GOOD: a leading indicator of saturation
- alert: OrdersPoolSaturation
  expr: agroal_awaiting_count{app="orders"} > 5
  for: 5m
  labels: {severity: warn}

# BAD: a cause that may not affect anyone
- alert: HighCpu
  expr: process_cpu_usage > 0.8
```

High CPU is not a problem if latency is fine. Paging on it produces alert fatigue, which is how real alerts get ignored.

14.9.2 Error budgets

```
# Availability SLI over 30 days
1 - (
  sum(increase(http_server_requests_seconds_count{app="orders",status=~"5.."}[30d]))
```

```

/
sum(increase(http_server_requests_seconds_count{app="orders"}[30d]))
)

```

With a 99.9 % objective, the budget is 0.1 % of requests over the window. Alerting on **budget burn rate** rather than on instantaneous error rate is the current best practice: a fast burn pages immediately, a slow burn raises a ticket. It avoids both the false pages from a brief spike and the silent slow degradation that an instantaneous threshold misses.

14.10 Exposing Metrics

```

quarkus.micrometer.export.prometheus.path=/q/metrics
quarkus.http.non-application-root-path=/q

```

`/q/metrics` is on the non-application root path, so it is unaffected by `quarkus.http.root-path`. That separation is deliberate: probes and scrape endpoints stay stable when you version your API.

Restrict it — metric names and label values leak information about your architecture and traffic:

```

quarkus.http.auth.permission.metrics.paths=/q/metrics
quarkus.http.auth.permission.metrics.policy=role-monitoring
quarkus.http.auth.policy.role-monitoring.roles-allowed=monitoring

```

Or bind it to a separate port that is not exposed by the Service:

```

quarkus.management.enabled=true
quarkus.management.port=9000
quarkus.micrometer.export.prometheus.path=/metrics

```

The management interface is the cleaner option in Kubernetes: scrape port 9000 from Prometheus, expose only 8080 through the Service and Ingress.

```

metadata:
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9000"
    prometheus.io/path: "/metrics"

```

14.11 Testing Instrumentation

```

@QuarkusTest
class OrderMetricsTest {

    @Inject MeterRegistry registry;
    @Inject OrderService service;

    @BeforeEach
    void clear() {
        registry.clear();
    }

    @Test
    void countsCreatedOrders() {

```

```

        service.create(validRequest());
        service.create(validRequest());

        Counter counter = registry.find("orders.created").counter();
        assertThat(counter).isNotNull();
        assertThat(counter.count()).isEqualTo(2.0);
    }

    @Test
    void tagsFailuresWithOutcome() {
        assertThatThrownBy(() -> service.create(invalidRequest()))
            .isInstanceOf(ValidationException.class);

        assertThat(registry.find("orders.create")
            .tag("outcome", "error")
            .timer().count()).isEqualTo(1);
    }

    @Test
    void doesNotUseHighCardinalityTags() {
        service.create(validRequest());

        Set<String> forbidden = Set.of("user_id", "order_id", "request_id");
        assertThat(registry.getMeters())
            .allSatisfy(meter -> assertThat(meter.getId().getTags())
                .noneMatch(tag -> forbidden.contains(tag.getKey())));
    }
}

```

That last test is worth having in a shared test library. Cardinality mistakes are easy to introduce and expensive to discover in production.

```
%test.quarkus.micrometer.export.prometheus.enabled=false # keep the test registry simple
```

14.12 Best Practices

1. **Estimate cardinality before shipping a tag.** Multiply the distinct values of every tag.
2. **Never tag with an identifier**, a raw URI or an exception message.
3. **Histograms over client-side percentiles** when you will aggregate across pods.
4. **Emit at least one business metric** per service.
5. **Alert on symptoms** — error rate, latency, saturation — not on causes like CPU.
6. **Common tags via a `MeterFilter`**, not repeated at every call site.
7. **Keep the object referenced** when you register a gauge; the registry holds it weakly.
8. **Names are nouns and hierarchical**; dimensions are tags, not name segments.
9. **Expose metrics on the management port** rather than the application port.
10. **Watch pool saturation (`agroal_awaiting_count`)** — it leads latency.
11. **Define SLIs first**, then build the dashboard from them.
12. **Test for forbidden tags** in CI.

14.13 Common Mistakes

1. **High-cardinality tags.** The one that takes down the monitoring system for everybody.
2. **Averaging percentiles across pods.** Mathematically meaningless; the number on the dashboard is fiction.
3. **A gauge over an unreferenced object.** Reports NaN after the first GC.
4. **Encoding dimensions in the metric name.** `orders.created.web` cannot be aggregated by channel.
5. **A counter for something that decreases.** Rates go negative and become nonsense.
6. **Alerting on CPU or memory.** Causes, not symptoms. Alert fatigue follows.
7. **Only technical metrics.** The system is up and nobody can place an order.
8. **@Timed on a self-invoked method.** Not intercepted; reports zero.
9. **Exposing /q/metrics publicly.** Leaks architecture, traffic volume and endpoint names.
10. **Recording a timer without an outcome tag.** You cannot distinguish fast failures from fast successes, and a service that fails instantly looks healthy.
11. **Percentile histograms on everything.** Bucket series multiply cardinality; enable them where you have an SLO.

14.14 Interview Questions

Junior

Q1. How do you add metrics to a Quarkus service? Add `quarkus-micrometer-registry-prometheus`, inject `MeterRegistry`, and register counters, gauges and timers — or use `@Timed` and `@Counted`. Metrics are exposed at `/q/metrics`.

Q2. Counter versus gauge? A counter only increases and is used with `rate()` to get a per-second value. A gauge is a point-in-time reading that can go up or down — queue depth, active connections.

Q3. What does a timer record? Both a count and a duration distribution, so one timer answers “how many” and “how long”.

Q4. What is Micrometer? A vendor-neutral metrics facade. You instrument once and select the backend — Prometheus, Datadog, OTLP — by dependency.

Q5. Where are metrics exposed? `/q/metrics` by default, on the non-application root path, in Prometheus text format.

Mid-level

Q6. What is cardinality and why does it matter? Every distinct combination of tag values is a separate time series with its own memory and index cost. Tagging with an identifier — user id, order id, request id — produces unbounded series growth that can exhaust the monitoring system’s memory, which affects every team, not just yours.

Q7. How do you avoid high cardinality on HTTP metrics? Tag with the route template rather than the concrete URI — `/orders/{id}`, not `/orders/12345`. Quarkus does this automatically for JAX-RS resources, and `match-patterns` plus `max-uri-tags` handle the rest.

Q8. Why can you not average percentiles? A percentile is a property of a distribution, not a value that composes. Averaging pod A's p99 of 100 ms with pod B's p99 of 200 ms produces a number corresponding to no real quantity. Export histogram buckets instead and compute the quantile at query time with `histogram_quantile`.

Q9. What is the RED method? Rate, Errors, Duration — the three signals to measure for any request-driven service. USE is the counterpart for resources: Utilisation, Saturation, Errors.

Q10. What should you alert on? Symptoms visible to users — error ratio, latency against an SLO, saturation as a leading indicator — not causes like CPU or memory, which may be high without anyone being affected. Alerting on causes produces fatigue and real alerts get ignored.

Q11. Why does my `@Timed` method report nothing? Most likely self-invocation: `@Timed` is interceptor-based, so calling the method through `this` from within the same bean bypasses the proxy. The same rule applies to `@Transactional` and `@CacheResult`.

Senior

Q12. Design the observability strategy for a new service. I would start from the questions we need to answer rather than from the metrics available. Three tiers. First, RED on every endpoint — rate, error ratio, and a latency histogram with buckets chosen around the SLO rather than the defaults, because bucket choice is where cardinality is won or lost. Second, USE on every bounded resource: connection pool utilisation and, more importantly, saturation, since `agroal_awaiting_count` rises before latency does and is the metric that gives you warning. Third, at least one business metric a product owner would recognise — orders per minute — because it catches failures that are invisible technically, like a frontend deploy that silently stops submitting. On top of that, common tags applied by a `MeterFilter` so every service is queryable the same way, a cardinality guard filter denying identifier-like tag keys, and a test in CI asserting no forbidden tags. Alerts derive from SLIs only, with burn-rate alerting rather than instantaneous thresholds, and every alert carries a runbook link. What I would deliberately *not* do is build a dashboard of everything the binders expose; an unread dashboard is worse than none because it creates false confidence.

Q13. Your Prometheus is running out of memory. Diagnose it. Memory in Prometheus is dominated by active series, so the question is which metric grew. `topk(10, count by (__name__)(__name__=~".+"))` gives the answer directly, and in my experience it is almost always one metric with an unbounded label — a user id, an order id, a raw URI containing an id, or an exception message used as a label. The immediate mitigation is a `metric_relabel_config` on the scrape job to drop the offending label or the metric entirely, which stops the bleeding without a deploy. Then fix the source: replace the identifier with a bounded dimension, or move that information to traces and logs where per-request detail belongs. Structurally, I would add a `MeterFilter` denying identifier-like tag keys across the platform's shared library, plus a CI test, because this recurs — and I would set a per-service series budget with an alert, so the next occurrence is caught by the team that caused it rather than by whoever is on call for the monitoring platform. The framing worth stating is that metrics are for aggregates and traces are for individual requests; high cardinality is a symptom of using the wrong tool.

Q14. How do you set an SLO and alert on it? The SLO has to come from what users need, not from what the system currently achieves, so I would start with the product owner: what proportion of requests must succeed within what latency for the feature to be considered working. That gives an SLI definition — say, the fraction of requests returning non-5xx within 500 ms — which I would then express as a Prometheus query over histogram buckets, choosing the bucket boundaries to include the threshold exactly so the query is precise rather than interpolated. From the objective comes an error budget: 99.9 % over 30 days is roughly 43 minutes of full unavailability, or an equivalent partial degradation. I would alert on burn rate rather than on the instantaneous ratio, with a fast-burn rule that pages when we would exhaust the month's budget in a few hours and a slow-burn rule that raises a ticket for sustained low-level degradation. That combination avoids paging for a thirty-second blip while still catching a slow decline that a static threshold would never trip. Finally, the budget needs to mean something organisationally — if it is exhausted, reliability work takes priority over features — otherwise it is a dashboard rather than a decision-making tool.

Principal

Q15. Standardise observability across 40 services. What do you mandate? I would mandate the things that make services comparable and queryable together, and leave the domain-specific instrumentation alone. Mandated: a common tag set — application, version, environment — applied by a shared `MeterFilter` shipped in a platform extension, so every query works across every service without per-team knowledge; a metric naming convention; a cardinality guard denying identifier-like labels, enforced both at runtime and by a CI test, because this is the failure that damages other teams; the management-port pattern so scraping is uniform and metrics are never publicly exposed; and RED plus at least one business metric per service, since a service with no business metric cannot be reasoned about by anyone outside the owning team. Also mandated: every alert has a runbook link and an owning rota, because an alert without either is noise that trains people to ignore alerting generally. Left free: which business metrics, dashboard layout, and any domain-specific instrumentation — those are local knowledge and standardising them produces lowest-common-denominator results. The delivery mechanism matters as much as the content: shipping this as a dependency rather than a wiki page is the difference between adoption and decay, and I would measure conformance on a platform dashboard rather than policing it. The single hardest thing to retrofit is the common tag set, so I would do that first even if nothing else lands.

14.15 Cheat Sheet

Instruments

```
registry.counter("name", "tag", "value").increment();
Gauge.builder("name", obj, Obj::size).register(registry); // keep obj referenced
Timer.Sample s = Timer.start(registry);
s.stop(Timer.builder("name").tag("outcome", o).register(registry));
registry.summary("name").record(value);
```

Annotations

```
@Timed(value = "name", percentiles = {0.5, 0.95, 0.99})
@Counted(value = "name")
// interceptor-based: NOT applied on self-invocation
```

Configuration

```
quarkus.micrometer.export.prometheus.path=/q/metrics
quarkus.micrometer.binder.jvm=true
quarkus.micrometer.binder.http-server.ignore-patterns=/q/.*
quarkus.micrometer.binder.http-server.max-uri-tags=100
quarkus.management.enabled=true
quarkus.management.port=9000
```

Key built-in metrics

```
http_server_requests_seconds_{count,sum,bucket}{method,uri,status,outcome}
jvm_memory_used_bytes      jvm_gc_pause_seconds    process_cpu_usage
agroal_active_count        agroal_awaiting_count
kafka_consumer_fetch_manager_records_lag_max
cache_gets_total{cache,result}
```

PromQL

```
sum(rate(http_server_requests_seconds_count[5m]))
sum(rate(...{status=~"5.."}[5m])) / sum(rate(...[5m]))
histogram_quantile(0.99, sum by (le) (rate(http_server_requests_seconds_bucket[5m])))
topk(10, count by (__name__)(__name__=~".+"))      -- find cardinality offenders
```

Cardinality rules

NEVER tag with: `user_id`, `order_id`, `request_id`, `trace_id`, raw URI, exception message
ALWAYS tag with: bounded sets - `status`, `method`, `outcome`, `region`, `channel`
ESTIMATE: multiply the distinct values of every tag before shipping

14.16 Summary

Micrometer gives you four instruments — counter, gauge, timer, distribution summary — and the skill is choosing the right one and, far more importantly, choosing the right **tags**.

Cardinality is the concept that separates people who have added metrics from people who have operated a monitoring system. Every tag-value combination is a time series, so an identifier in a tag produces unbounded growth that exhausts Prometheus and removes visibility from every team at once. Tag with bounded dimensions; put per-request detail in traces, where it belongs.

The second technical point worth internalising is that **percentiles do not average**. Export histogram buckets and compute quantiles at query time, choosing bucket boundaries around your SLO so you pay cardinality only where it buys something.

For deciding what to measure, RED for request-driven services and USE for resources give you a checklist that covers most of it — plus at least one business metric per service, because a drop in orders per minute catches outages that no technical signal does. Alert on those symptoms, not on causes like CPU, and derive alerts from SLIs with burn-rate rules rather than instantaneous thresholds.

Chapter 15 continues with **OpenTelemetry** — distributed tracing, context propagation across services, and the sampling decisions that determine whether tracing is affordable.

14.17 References

- Micrometer metrics in Quarkus — <https://quarkus.io/guides/telemetry-micrometer>
- Micrometer Prometheus registry — <https://quarkus.io/guides/telemetry-micrometer-tutorial>
- Micrometer documentation — <https://micrometer.io/docs>
- Prometheus best practices — <https://prometheus.io/docs/practices/naming/>
- Google SRE Workbook, alerting on SLOs — <https://sre.google/workbook/alerting-on-slos/>
- The RED method — <https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/>
- The USE method — <https://www.brendangregg.com/usemethod.html>

OpenTelemetry and Distributed Tracing

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · `quarkus-opentelemetry` · OTLP · Jaeger / Tempo · W3C Trace Context

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what tracing answers that metrics and logs cannot.
2. Describe the OpenTelemetry data model: traces, spans, context, attributes, events, links.
3. Configure Quarkus to export traces via OTLP and understand what is instrumented automatically.
4. Create manual spans and add attributes without creating cardinality or cost problems.
5. Explain **context propagation** — W3C `traceparent` — across HTTP, messaging and asynchronous boundaries.
6. Choose a **sampling strategy** and defend the trade-off between cost and diagnostic value.
7. Correlate traces with logs and metrics so an incident starts with evidence rather than search.
8. Trace across Kafka, where the parent-child relationship is not synchronous.
9. Understand the performance cost of tracing and how to bound it.
10. Answer 15 interview questions on tracing and observability design.

15.1 Why Tracing

The three pillars answer different questions, and understanding the division prevents a lot of wasted instrumentation:

Signal	Answers	Granularity	Cost
Metrics	Is the system healthy? How much? How fast?	Aggregate	Cheap, bounded
Logs	What happened in this component?	Per event	Moderate, unbounded
Traces	Where did <i>this one request</i> spend its time, across services?	Per request, cross-service	Expensive, sampled

In a monolith, a stack trace tells you the call path. In a distributed system there is no stack — a request touches six services, three of them asynchronously, and the interesting question is which of them took 800 ms. That is what a trace answers, and neither metrics nor logs can.

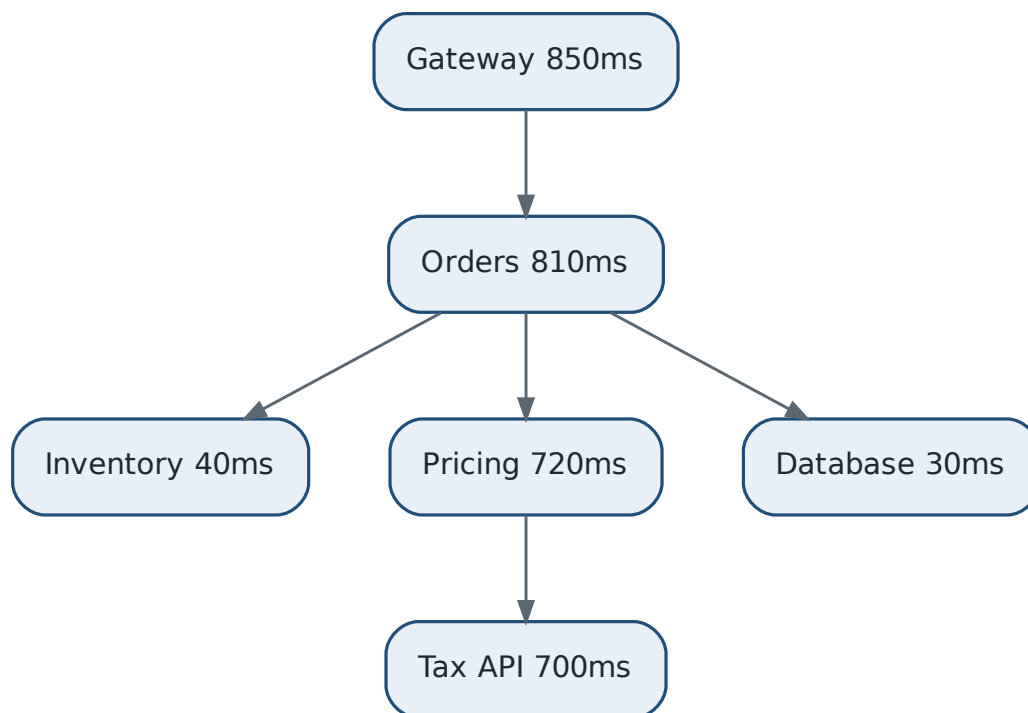


Figure 15.1 — A trace attributes latency to a specific downstream call. The problem is the tax API, which no single service’s metrics would have revealed.

15.2 The Data Model

Trace — the whole journey of one request. Identified by a 16-byte trace id.

Span — one unit of work within a trace: an HTTP handler, a database query, an outbound call. It has a span id, a parent span id, a name, a start and end time, attributes, events and a status.

Context — the trace id plus the current span id, propagated between processes.

Attributes — key/value metadata on a span. Unlike metric tags, **high cardinality is fine here** — a span is per-request by construction, so `order.id` is exactly the sort of thing that belongs.

Events — timestamped annotations within a span, for things that happen at a moment rather than over a duration.

Links — a relationship to another trace, used where there is no simple parent-child relationship — notably batch processing and messaging fan-in.

Trace 4bf92f3577b34da6a3ce929d0e0e4736

```

└─ Span: POST /orders      850ms (root, service=gateway)
  └─ Span: POST /orders    810ms (service=orders)
    └─ Span: GET /stock    40ms (service=inventory)
      └─ Span: POST /price  720ms (service=pricing)
        └─ Span: GET tax-api 700ms (external)
          └─ Span: INSERT orders 30ms (db)

```

The cardinality rule inverts between metrics and traces, and this is worth stating explicitly because people carry Chapter 14’s discipline over incorrectly. A metric tag with 100 000 values creates 100 000 time series. A span attribute with 100 000 values creates nothing extra — each span already exists per request. Put identifiers in span attributes; keep them out of metric tags.

15.3 Setup

```

<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-opentelemetry</artifactId>
</dependency>

```

```

quarkus.otel.enabled=true
quarkus.otel.exporter.otlp.endpoint=http://otel-collector:4317
quarkus.otel.exporter.otlp.protocol=grpc          # or http/protobuf on 4318
quarkus.otel.exporter.otlp.headers=api-key=${OTEL_API_KEY}

# Resource attributes - identify this service in the backend
quarkus.otel.resource.attributes=service.name=orders-service,\
service.version=${quarkus.application.version},\
deployment.environment=${DEPLOY_ENV:local}

# Sampling
quarkus.otel.traces.sampler=parentbased_traceidratio
quarkus.otel.traces.sampler.arg=0.1              # 10 %

# Batching - bounds the export overhead
quarkus.otel.bsp.schedule.delay=5s
quarkus.otel.bsp.max.queue.size=2048
quarkus.otel.bsp.max.export.batch.size=512
quarkus.otel.bsp.export.timeout=30s

# Reduce noise
quarkus.otel.traces.suppress-non-application-uris=true # skip /q/*

%dev.quarkus.otel.traces.sampler=always_on
%test.quarkus.otel.enabled=false

```

15.3.1 What is instrumented automatically

Adding the extension gives you, with no code changes:

- **Inbound HTTP** — a server span per request, with method, route, status.
- **Outbound HTTP** — REST Client and Vert.x client spans, with propagation headers attached.
- **JDBC** — a span per query, with the statement (see the note below).
- **Kafka** — producer and consumer spans, with context carried in message headers.
- **gRPC, Redis, MongoDB, reactive messaging.**
- **CDI methods annotated `@WithSpan`.**

```
# JDBC spans need the datasource telemetry flag
quarkus.datasource.jdbc.telemetry=true
```

Be careful with SQL in span attributes. The statement text is enormously useful for diagnosis and is also a place where personal data leaks — a query with inlined literals may contain an email address or a card number. Quarkus records the statement without bind parameters by default, which is the right balance; do not turn parameter capture on in production without thinking about what is in them.

15.4 Manual Instrumentation

Automatic instrumentation covers the boundaries. Manual spans cover the work *inside* a service, which is where unexplained latency usually hides.

```
package com.acme.orders.pricing;

import io.opentelemetry.api.trace.Span;
import io.opentelemetry.api.trace.SpanKind;
import io.opentelemetry.api.trace.StatusCode;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.context.Scope;
import io.opentelemetry.instrumentation.annotations.SpanAttribute;
import io.opentelemetry.instrumentation.annotations.WithSpan;

@ApplicationScoped
public class PricingService {

    @Inject Tracer tracer;

    // Declarative: one span per invocation
    @WithSpan("pricing.calculate")
    public BigDecimal price(@SpanAttribute("order.id") long orderId,
                           @SpanAttribute("order.lines") int lineCount) {
        Span.current().setAttribute("pricing.strategy", strategy.name());
        return calculate(orderId);
    }

    // Programmatic: full control over lifecycle and nesting
    public PricingResult priceWithBreakdown(Order order) {
        Span span = tracer.spanBuilder("pricing.breakdown")
            .setSpanKind(SpanKind.INTERNAL)
            .setAttribute("order.id", order.id)
            .setAttribute("order.total", order.total.doubleValue())
```



```

        .setAttribute("customer.tier", order.customerTier())
        .startSpan();

    try (Scope scope = span.makeCurrent()) {
        span.addEvent("rules.loaded",
            Attributes.of(AttributeKey.longKey("rule.count"), (long) rules.size()));

        PricingResult result = applyRules(order);

        span.setAttribute("pricing.discount", result.discount().doubleValue());
        span.setStatus(StatusCode.OK);
        return result;

    } catch (Exception e) {
        span.recordException(e);
        span.setStatus(StatusCode.ERROR, e.getMessage());
        throw e;
    } finally {
        span.end(); // ALWAYS, or the span leaks
    }
}

```

Three rules for manual spans:

span.end() in a finally. An unended span never exports, so the trace shows a gap where the work was — worse than no span at all, because it looks like the request skipped that code.

makeCurrent() in try-with-resources. Without an open scope, child spans attach to the wrong parent, or become roots of their own traces.

recordException and setStatus(ERROR) on failure, so error traces are findable in the backend by status rather than by guessing.

15.4.1 Attribute conventions

OpenTelemetry defines **semantic conventions** — standard attribute names that backends understand and can build UI around.

```

// Use the conventions where one exists
span.setAttribute("http.request.method", "POST");
span.setAttribute("url.path", "/orders");
span.setAttribute("http.response.status_code", 201);
span.setAttribute("db.system", "postgresql");
span.setAttribute("messaging.system", "kafka");
span.setAttribute("messaging.destination.name", "orders");

// Namespace your own attributes so they cannot collide
span.setAttribute("acme.order.id", orderId);
span.setAttribute("acme.tenant.id", tenantId);
span.setAttribute("acme.pricing.strategy", strategy);

```

Never put a secret, a token, a password or a full request body in an attribute. Traces are typically stored with broader access than application logs, and they are retained.

15.5 Context Propagation

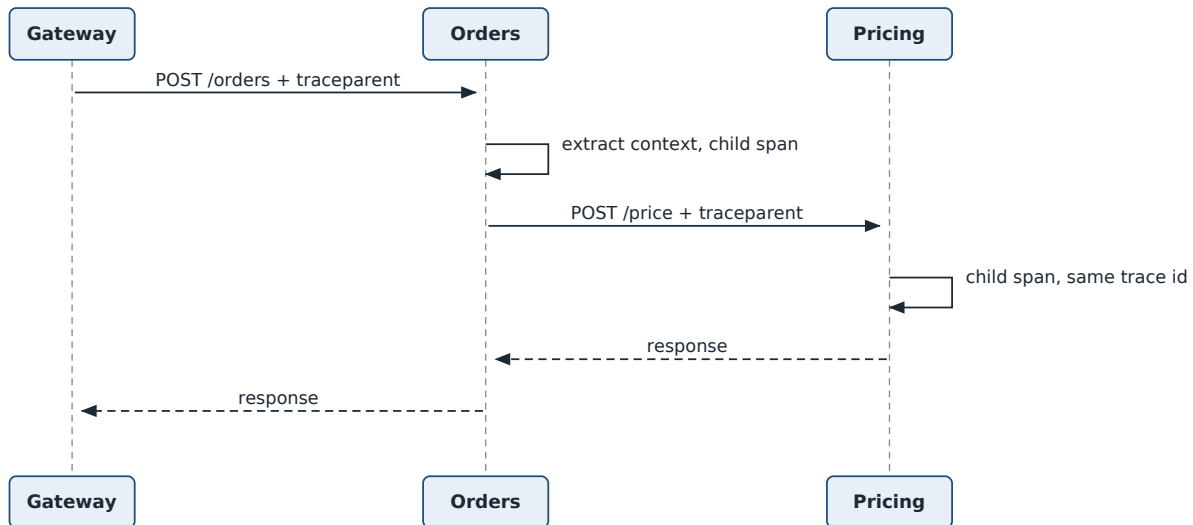


Figure 15.2 — The `traceparent` header carries the trace id and parent span id across process boundaries.

The **W3C Trace Context** header:

```

traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
              | |                               | |
              | trace id (16 bytes)             | span id (8)   | flags (sampled)
              | version
tracestate: acme=t6lrcWkgMzE,vendor=opaque
  
```

For HTTP this is automatic in both directions. Where it needs attention:

15.5.1 Asynchronous boundaries

```

// Mutiny: context propagates with quarkus-smallrye-context-propagation present
public Uni<Order> process(long id) {
    return repository.findById(id)
        .chain(order -> pricingClient.price(order)); // same trace
}

// A raw thread loses it - capture and restore explicitly
public void manual() {
    Context context = Context.current();
    executor.submit(() -> {
        try (Scope scope = context.makeCurrent()) {
            doWork(); // now in the same trace
        }
    });
}
  
```

```

}

// ManagedExecutor propagates OTel context too
@Inject ManagedExecutor executor;

```

15.5.2 Kafka: links, not parents

Messaging breaks the parent-child model. A consumer may process a message minutes after it was produced, and a batch consumer processes many messages whose producers are unrelated to each other.

```

@ApplicationScoped
public class OrderEventConsumer {

    @Inject Tracer tracer;

    @Incoming("orders-in")
    @WithSpan("orders.consume")
    @Blocking
    public void consume(OrderEvent event) {
        // The producer's context arrives in Kafka headers and Quarkus links it
        Span.current().setAttribute("acme.order.id", event.orderId());
        Span.current().setAttribute("acme.event.id", event.eventId());
        process(event);
    }

    // A batch: link to every producing trace rather than picking one parent
    @Incoming("orders-batch")
    public void consumeBatch(List<Message<OrderEvent>> batch) {
        SpanBuilder builder = tracer.spanBuilder("orders.consume.batch")
            .setSpanKind(SpanKind.CONSUMER)
            .setAttribute("messaging.batch.message_count", batch.size());

        for (Message<OrderEvent> message : batch) {
            extractContext(message).ifPresent(ctx ->
                builder.addLink(Span.fromContext(ctx).getSpanContext()));
        }

        Span span = builder.startSpan();
        try (Scope scope = span.makeCurrent()) {
            batch.forEach(m -> process(m.getPayload()));
        } finally {
            span.end();
        }
    }
}

```

Making a consumer span a *child* of the producer is usually wrong. It makes the trace’s duration span the whole queue latency, so a message sitting in a topic for an hour produces a one-hour trace that distorts every latency percentile in the backend. A **link** expresses “caused by” without implying “contained within”, which is the honest relationship.

15.6 Sampling

Tracing every request is affordable at low volume and ruinous at high volume. Sampling is where the cost decision lives.

```
quarkus.otel.traces.sampler=always_on           # everything (dev)
quarkus.otel.traces.sampler=always_off         # nothing
quarkus.otel.traces.sampler=traceidratio
quarkus.otel.traces.sampler.arg=0.1           # 10 %
quarkus.otel.traces.sampler=parentbased_always_on # follow the caller
quarkus.otel.traces.sampler=parentbased_traceidratio
quarkus.otel.traces.sampler.arg=0.05
```

15.6.1 Why parentbased matters

A sampling decision must be **consistent across the whole trace**. If the gateway samples a request but the orders service independently decides not to, you get a trace with holes — which is worse than no trace, because it looks like the request skipped a service.

`parentbased_*` samplers honour the incoming `traceparent` sampled flag and only make an independent decision for root spans. **This should always be your production setting.** `traceidratio` alone is consistent too — because the decision derives from the trace id, which is shared — but `parentbased` is more explicit and handles the case where an upstream service uses a different rate.

15.6.2 Head-based versus tail-based

Head-based (what the SDK does) decides at the root, before anything has happened. It is cheap and simple, and it discards 90 % of traces including 90 % of the errors and slow requests — exactly the ones you wanted.

Tail-based (done in the OpenTelemetry Collector) buffers a whole trace, then decides. It can keep every error and every slow trace while dropping most successful fast ones.

```
# otel-collector-config.yaml
processors:
  tail_sampling:
    decision_wait: 10s
    num_traces: 100000
    policies:
      - name: errors
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: slow
        type: latency
        latency: {threshold_ms: 1000}
      - name: baseline
        type: probabilistic
        probabilistic: {sampling_percentage: 5}
```

The cost is that the collector must buffer every span of every trace until the decision, which needs memory and a collector topology where all spans of a trace reach the same instance. For most

organisations that is worth it: keeping 100 % of errors and 5 % of successes is dramatically more useful per byte stored than a flat 10 %.

This is a strong senior interview answer. “We sample 10 %” is a configuration. “We use head-based sampling at the edge for volume control and tail-based in the collector so every error and every request over one second is retained, with a 5 % baseline for comparison” is a design.

15.7 Correlating Traces, Logs and Metrics

The value of observability comes from moving between signals: an alert on a metric leads to a trace, which leads to the log lines for that request.

15.7.1 Trace ids in logs

```
quarkus.log.console.format=%d{HH:mm:ss} %-5p traceId=%X{traceId} spanId=%X{spanId} \
sampled=%X{sampled} [%c{2.}] (%t) %s%n
```

```
# JSON logging carries the MDC automatically
%prod.quarkus.log.console.json=true
```

Quarkus puts `traceId`, `spanId` and `sampled` into the MDC automatically when the OpenTelemetry extension is present, so structured logs are queryable by trace id with no code change. That single line of format configuration is the highest-value piece of correlation you can do.

```
Log.infof("Order %d priced at %s", orderId, total);
// -> {"timestamp":"...", "message":"Order 42 priced at 99.00",
//      "traceId":"4bf92f3577b34da6a3ce929d0e0e4736", "spanId":"00f067aa0ba902b7"}
```

15.7.2 Returning the trace id to the caller

```
@ServerResponseFilter
public void addTraceId(ContainerResponseContext response) {
    SpanContext context = Span.current().getSpanContext();
    if (context.isValid()) {
        response.getHeaders().putSingle("X-Trace-Id", context.getTraceId());
    }
}
```

Now a customer-reported “it failed at 14:32 with reference abc123” maps to a trace in one query. Combine it with the error-reference pattern from Chapter 2:

```
@ServerExceptionHandler
public RestResponse<ProblemDetail> unexpected(Throwable t) {
    String traceId = Span.current().getSpanContext().getTraceId();
    Log.errorf(t, "Unhandled error");
    return RestResponse.status(Status.INTERNAL_SERVER_ERROR,
        ProblemDetail.of("internal-error", "An unexpected error occurred", 500,
            "Reference: " + traceId));
}
```

Using the trace id as the customer-facing reference is better than a random UUID, because it is already the key to everything else.

15.7.3 Exemplars

Exemplars link a metric bucket to an example trace, so you can click a latency spike on a Grafana panel and open a trace from it.

```
quarkus.micrometer.export.prometheus.enabled=true
quarkus.otel.enabled=true
# Prometheus must be started with --enable-feature=exemplar-storage
```

15.8 The Cost of Tracing

Tracing is not free, and being able to quantify it is a senior-level signal.

CPU — span creation, attribute recording and serialisation. Typically 1–3 % at full sampling for a normal service; negligible at 10 %.

Memory — the batch span processor’s queue. `max.queue.size=2048` spans, each a few hundred bytes, is a bounded and small cost.

Network — spans are exported continuously. A service at 1000 rps with 20 spans per trace at 10 % sampling produces 2000 spans/second; at roughly 500 bytes each that is 1 MB/s per instance, which is not nothing.

Storage — usually the dominant cost, and the reason sampling exists at all.

Bounding it:

```
# Batch rather than exporting per span
quarkus.otel.bsp.schedule.delay=5s
quarkus.otel.bsp.max.export.batch.size=512

# Drop spans rather than block the application if the collector is unavailable
quarkus.otel.bsp.max.queue.size=2048

# Do not trace health checks and metrics scrapes
quarkus.otel.traces.suppress-non-application-uris=true
```

Tracing must never block or fail the application. The batch processor drops spans when its queue is full, which is the correct behaviour — losing telemetry is always better than losing requests. If you write a custom exporter, preserve that property.

15.9 Testing

```
%test.quarkus.otel.enabled=false          # default: off, so tests are not slowed
```

```
@QuarkusTest
@TestProfile(TracingEnabledProfile.class)
class TracingTest {

    public static class TracingEnabledProfile implements QuarkusTestProfile {
```

```

    public Map<String, String> getConfigOverrides() {
        return Map.of("quarkus.otel.enabled", "true",
            "quarkus.otel.traces.sampler", "always_on",
            "quarkus.otel.exporter.otlp.enabled", "false");
    }
}

@Inject InMemorySpanExporter exporter;

@BeforeEach void clear() { exporter.reset(); }

@Test
void createsSpanForPricing() {
    given().contentType("application/json").body(validRequest())
        .when().post("/orders")
        .then().statusCode(201);

    await().atMost(5, SECONDS).untilAsserted(() ->
        assertThat(exporter.getFinishedSpanItems())
            .extracting(SpanData::getName)
            .contains("POST /orders", "pricing.calculate"));
}

@Test
void recordsErrorStatus() {
    given().contentType("application/json").body(invalidRequest())
        .when().post("/orders")
        .then().statusCode(400);

    await().untilAsserted(() ->
        assertThat(exporter.getFinishedSpanItems())
            .anySatisfy(span ->
                assertThat(span.getStatus().getStatusCode()).isEqualTo(StatusCode.ERROR)));
}

@Test
void doesNotLeakSecretsInAttributes() {
    given().header("Authorization", "Bearer secret-token")
        .when().get("/orders").then().statusCode(200);

    await().untilAsserted(() ->
        assertThat(exporter.getFinishedSpanItems())
            .allSatisfy(span -> assertThat(span.getAttributes().asMap().values())
                .noneMatch(v -> v.toString().contains("secret-token"))));
}
}

```

15.10 Best Practices

1. **parentbased_*** samplers in production, so the sampling decision is consistent across the trace.

2. **Tail-based sampling in the collector** where volume justifies it — keep all errors and slow traces.
3. `span.end()` in a **finally**. An unended span shows as a gap.
4. `makeCurrent()` in **try-with-resources**, or children attach to the wrong parent.
5. **Trace ids in the log format**, and returned to the caller as a reference.
6. **Span attributes may be high cardinality** — put ids there rather than in metric tags.
7. **Never put secrets, tokens or bodies in attributes.**
8. **Semantic conventions where they exist**; namespace your own attributes.
9. **Links, not parent-child, for messaging**, so queue latency does not distort trace duration.
10. **Suppress `/q/*` spans** — health and metrics traffic is noise.
11. **Manual spans around expensive internal work**, not around every method.
12. **Disable tracing in tests** by default; enable per-profile where you assert on spans.

15.11 Common Mistakes

1. **Independent sampling per service.** Traces with holes; looks like a service was skipped.
2. **Forgetting `span.end()`.** The span never exports; the trace shows a gap where the work happened.
3. **No `makeCurrent()`.** Child spans become orphan roots.
4. **Consumer spans as children of producer spans.** Queue latency becomes trace duration and distorts every percentile.
5. **Secrets in attributes.** Traces are retained and often have broader access than logs.
6. **Tracing everything at 100 % in production.** Storage cost, and rarely more useful than tail-based sampling.
7. **No trace id in logs.** The signals cannot be joined, so an incident starts with a search rather than a link.
8. **A span per method.** Thousands of spans per trace, unreadable in the UI, expensive to store.
9. **Assuming context crosses a raw thread.** It does not; capture and restore explicitly.
10. **Blocking on export.** Telemetry must never fail requests.
11. **Applying metric cardinality rules to span attributes.** Overly cautious — identifiers are exactly what makes a trace useful.

15.12 Interview Questions

Junior

Q1. What is distributed tracing? Following one request across service boundaries, recording where its time was spent. A trace is composed of spans, each representing a unit of work.

Q2. What is a span? One operation within a trace: an HTTP handler, a query, an outbound call. It has an id, a parent, a name, a duration, attributes and a status.

Q3. How do you enable tracing in Quarkus? Add `quarkus-opentelemetry` and set `quarkus.otel.exporter.otlp.endpoint`. HTTP, JDBC, Kafka and REST Client instrumentation is automatic.

Q4. How is context passed between services? The W3C `traceparent` header, carrying the trace id, the parent span id and a sampled flag. Quarkus injects and extracts it automatically for HTTP.

Q5. How do you create a span manually? `@WithSpan` on a method, or `tracer.spanBuilder("name").startSpan()` with `makeCurrent()` in try-with-resources and `end()` in a `finally`.

Mid-level

Q6. What is sampling and why do you need it? Recording every trace is prohibitively expensive at volume, so only a fraction is kept. Head-based sampling decides at the root before anything has happened; tail-based sampling decides in the collector after the trace completes, which allows keeping all errors and slow traces.

Q7. Why must you use a `parentbased` sampler? So the decision is consistent for the whole trace. If each service sampled independently, you would get traces missing the middle services, which looks like the request skipped them and is worse than having no trace.

Q8. Can span attributes be high cardinality? Yes, and they should be. A span exists per request already, so adding `order.id` costs nothing extra. This is the opposite of metric tags, where each distinct value creates a time series.

Q9. How do you correlate traces with logs? Quarkus puts `traceId` and `spanId` into the MDC when the extension is present. Include them in the log format, or use JSON logging which carries the MDC automatically. Returning the trace id as an `X-Trace-Id` header lets a customer-reported reference map directly to a trace.

Q10. How does tracing work across Kafka? The producer's context is written into message headers and extracted by the consumer. The consumer span should be **linked** to the producer's rather than made a child, because a message may be consumed long after production and a parent-child relationship would make the trace duration include the queue time.

Q11. What does tracing cost? CPU of one to three per cent at full sampling, a bounded memory queue for the batch processor, continuous network export, and storage — which usually dominates and is why sampling exists. The export path must drop spans rather than block, so telemetry can never fail requests.

Senior

Q12. Design a tracing strategy for 40 services. The non-negotiables first: one collector topology, OTLP everywhere, `parentbased` sampling so decisions are consistent, and resource attributes — service name, version, environment — set uniformly, because a trace whose spans cannot be attributed to a deployment is much less useful during an incident. Sampling would be two-stage: modest head-based sampling at the edge to bound volume, then tail-based policies in the collector keeping every error, everything over a latency threshold, and a small probabilistic baseline for comparison. That combination keeps the traces that matter while discarding the bulk. On correlation, trace ids go into the log MDC and into the error response as the customer-facing reference, so an incident begins with a link rather than a search. Instrumentation would be

automatic at boundaries plus manual spans only around expensive internal work, with a shared library providing the response filter and the exception mapper so it is consistent. What I would guard against is span explosion — a per-method span policy produces traces nobody can read and storage nobody wants to pay for — and secret leakage in attributes, which I would test for in CI. Finally, the collector must be treated as infrastructure with its own capacity planning, because tail-based sampling requires buffering whole traces and a routing layer that sends all spans of a trace to the same collector instance.

Q13. A request takes 3 seconds and you do not know why. Walk me through using traces. I would find a slow trace first — filtered by endpoint and latency, which is exactly what tail-based sampling guarantees is retained — and read the waterfall for the largest span. The common shapes are recognisable. One long child span means the time is in a specific downstream call, and I would then open that service's own span to see whether it is the service or something further down. A long root span with short children means the time is *between* the children, which is application code that is not instrumented — a serialisation step, an in-memory computation, or a lock — and the next step is a manual span or a profiler. Many short repeated child spans with the same name is N+1, either against the database or against a downstream service, and the fix is Chapter 5's. A gap at the start of a span often means queueing — waiting for a worker thread or a connection — which I would confirm against the pool saturation metric rather than the trace. And a span with a long duration but no children at all, in a service that should be calling others, usually means an unended span or a missing `makeCurrent()`, which is an instrumentation bug rather than a latency one. The point I would make is that the trace narrows the search from forty services to one code path in seconds, which is what no other signal does.

Q14. Head-based or tail-based sampling? Both, at different points. Head-based is cheap, requires no infrastructure and is the only option at the SDK, so it is the right tool for bounding raw volume at the edge — but its fundamental weakness is that it decides before it knows anything, so at ten per cent it discards ninety per cent of your errors and slow requests, which are the only traces anyone ever looks at. Tail-based inverts that: the collector buffers the complete trace and applies policies, so you can retain every error, everything over a latency threshold, and a probabilistic baseline for healthy comparison. Per byte stored it is dramatically more useful. The costs are real and worth naming: the collector needs memory proportional to trace volume times the decision window, and it needs a routing layer that guarantees all spans of a trace reach the same collector instance, which is a load-balancing constraint people discover late. For a small system head-based alone is fine. Past a certain volume the storage saving from tail-based pays for the collector several times over, and I would treat the crossover as the point at which someone starts asking to reduce the sampling rate.

Principal

Q15. Your organisation has metrics, logs and traces in three separate systems and incidents still take hours. What is wrong? Having three signals is not the same as being able to move between them, and the hours are almost certainly spent on the joins rather than on the analysis. So I would look first at whether the correlation identifiers exist end to end: is the trace id in every log line, is it returned to the caller, do metric exemplars link to traces, and can someone go from an alert to a trace to the logs for that request in three clicks. In most organisations with this symptom the answer is that each signal was adopted by a different team at a different time with different conventions, so service names do not match between systems, environments are labelled differently, and the trace id is present in some services' logs and not others. That is a naming and

plumbing problem, not a tooling problem, and it is fixed by a shared instrumentation library that sets resource attributes and the log format identically everywhere — which is far cheaper than the platform migration people usually propose. The second thing I would examine is whether the alerts point anywhere: an alert with no runbook and no linked dashboard means every incident starts with orientation, and that is often the bulk of the time. Third, whether sampling is discarding the evidence — head-based sampling at a low rate means the trace for the incident frequently does not exist, and people stop trying. I would resist consolidating onto one vendor as the first move, because it is expensive, slow, and does not by itself fix conventions; I would do it only if the correlation work reveals genuine technical incompatibility. The measurable goal I would set is time-from-alert-to-first-relevant-trace, because it is the part of the incident that is pure friction and it is the part that improves fastest.

15.13 Cheat Sheet

Setup

```
quarkus.otel.exporter.otlp.endpoint=http://otel-collector:4317
quarkus.otel.resource.attributes=service.name=orders,service.version=1.0
quarkus.otel.traces.sampler=parentbased_traceidratio
quarkus.otel.traces.sampler.arg=0.1
quarkus.otel.traces.suppress-non-application-uris=true
quarkus.datasource.jdbc.telemetry=true
%test.quarkus.otel.enabled=false
```

Manual spans

```
@WithSpan("name")
public X m(@SpanAttribute("order.id") long id) { }

Span span = tracer.spanBuilder("name")
    .setSpanKind(SpanKind.INTERNAL)
    .setAttribute("acme.order.id", id)
    .startSpan();
try (Scope scope = span.makeCurrent()) {
    span.addEvent("checkpoint");
    ...
} catch (Exception e) {
    span.recordException(e);
    span.setStatus(StatusCode.ERROR, e.getMessage());
    throw e;
} finally {
    span.end(); // ALWAYS
}
```

Context

traceparent: 00-<trace-id-32hex>-<span-id-16hex>-<flags>

```
Context ctx = Context.current();
try (Scope s = ctx.makeCurrent()) { ... } // restore on another thread
Span.current().getSpanContext().getTraceId();
```

Correlation

```
quarkus.log.console.format=... traceId=%X{traceId} spanId=%X{spanId} ...
%prod.quarkus.log.console.json=true
```

Samplers

```
always_on  always_off  traceidratio  parentbased_always_on  parentbased_traceidratio
```

Metrics vs traces

```
metric tags  -> MUST be low cardinality
span attrs   -> MAY be high cardinality (ids belong here)
```

15.14 Summary

Tracing answers the question neither metrics nor logs can: for *this* request, across *these* services, where did the time go. In a distributed system there is no stack trace, and the trace is its replacement.

Two configuration decisions carry most of the value. Use a **parentbased** sampler, so the sampling decision is consistent and you never see a trace with holes. And **put the trace id in the log MDC and in the error response**, because that single change turns an incident from a search across three systems into a link.

Two modelling decisions distinguish good instrumentation from noise. **Span attributes may be high cardinality** — this inverts Chapter 14’s rule, and identifiers belong here rather than in metric tags. And **messaging uses links, not parent-child**, because making a consumer span a child of its producer makes trace duration include queue latency and distorts every percentile in the backend.

Beyond that: end every manual span in a **finally**, make it current in try-with-resources, instrument expensive internal work rather than every method, never put secrets in attributes, and let the export path drop spans rather than block — telemetry must never be able to fail a request.

At volume, move the sampling decision from the SDK to the collector: tail-based policies that retain every error and every slow trace, with a probabilistic baseline, are worth far more per byte than a flat percentage.

Chapter 16 completes the observability sequence with **health checks** — liveness against readiness, the distinction that causes more Kubernetes outages than any other misconfiguration.

15.15 References

- OpenTelemetry in Quarkus — <https://quarkus.io/guides/opentelemetry>
- Tracing guide — <https://quarkus.io/guides/opentelemetry-tracing>
- OpenTelemetry specification — <https://opentelemetry.io/docs/specs/otel/>
- Semantic conventions — <https://opentelemetry.io/docs/specs/semconv/>
- W3C Trace Context — <https://www.w3.org/TR/trace-context/>
- Tail sampling processor — <https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor/tailsamplingprocessor>
- Jaeger — <https://www.jaegertracing.io/>
- Grafana Tempo — <https://grafana.com/oss/tempo/>

Health Checks

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · [quarkus-smallrye-health](#) · MicroProfile Health 4.0 · Kubernetes

Learning Objectives

By the end of this chapter you will be able to:

1. State precisely what **liveness**, **readiness** and **startup** probes mean, and what Kubernetes does in response to each.
2. Explain why checking dependencies in a liveness probe causes outages, and why checking them in readiness can cause cluster-wide cascading failure.
3. Write custom health checks that are cheap, bounded and meaningful.
4. Configure Kubernetes probes with values derived from the application's actual behaviour.
5. Use startup probes to eliminate the slow-start-versus-fast-detection trade-off.
6. Coordinate health checks with graceful shutdown so no request is dropped during a rollout.
7. Distinguish **health checks** from **monitoring** and know what belongs in each.
8. Test health endpoints.
9. Answer 15 interview questions, including the cascading-failure question that reliably separates levels.

16.1 The Three Probes

This is the whole chapter. Everything else is mechanics.

Probe	Question	Kubernetes response to failure
Liveness	Is this process broken beyond recovery?	Kill and restart the container
Readiness	Can this pod serve traffic <i>right now</i> ?	Remove it from Service endpoints
Startup	Has the application finished booting?	Kill it, but only after <code>failureThreshold × periodSeconds</code>

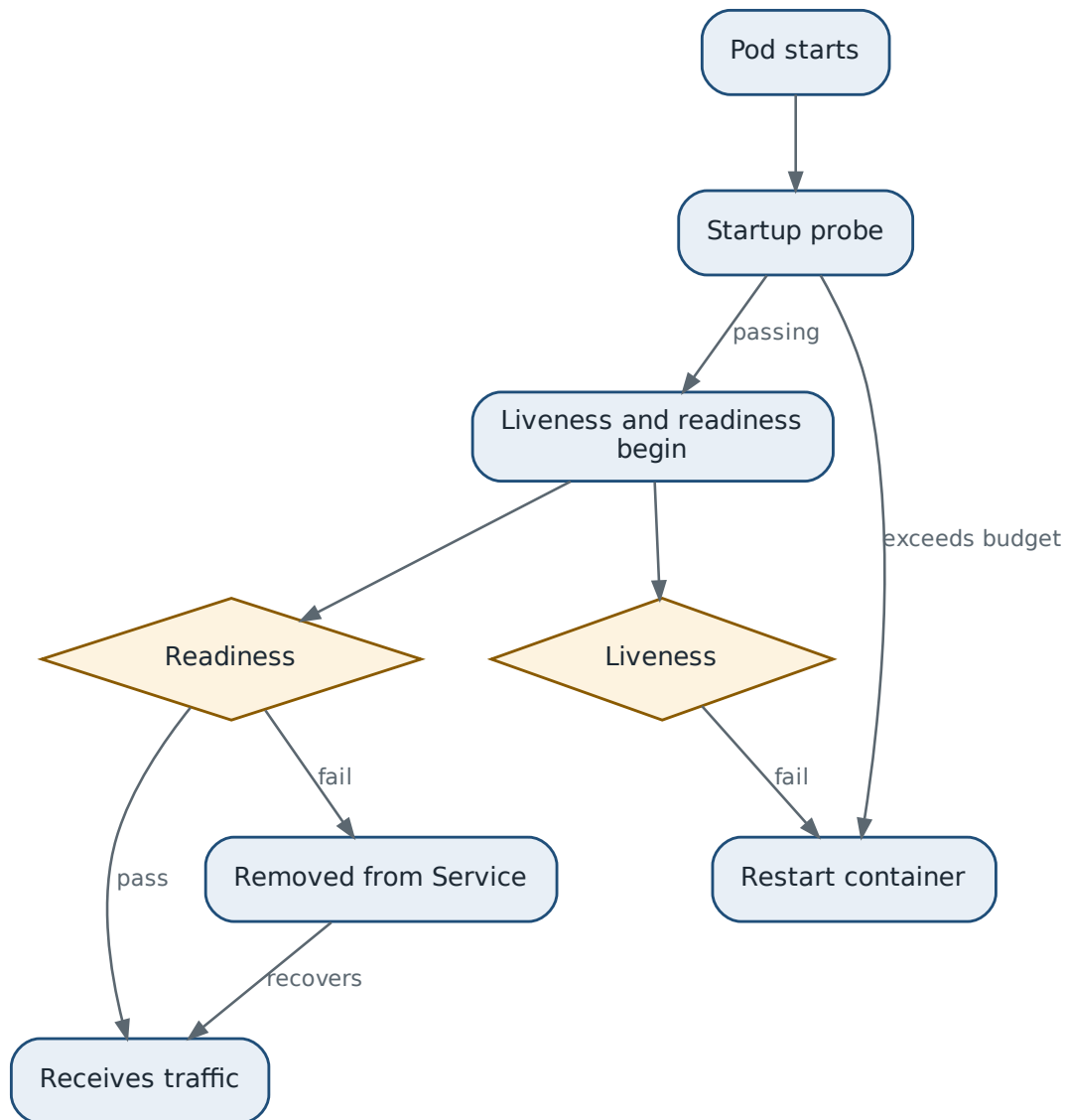


Figure 16.1 — The startup probe suppresses the other two until boot completes.

16.1.1 The rule that prevents most outages

Liveness answers “would a restart fix this?” **Readiness** answers “should I get traffic?”

A liveness probe must therefore check **only this process’s own state** — never a dependency. If the database is down, restarting your pod does not help; it makes things worse by throwing away warm caches, connection pools and in-flight work, and by adding a cold-start stampede to a system that is already struggling.

The failure this prevents is common and severe. A liveness probe that checks the database means: database has a five-minute blip → every pod in every service fails liveness → Kubernetes restarts all of them → the database comes back to a thundering herd of cold pods all opening connections at once → it falls over again. A brief dependency wobble becomes a multi-service outage, and the restart loop makes recovery slower.

16.2 SmallRye Health

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-smallrye-health</artifactId>
</dependency>
```

Endpoints, on the non-application root path so they are unaffected by `quarkus.http.root-path`:

/q/health	aggregate of everything
/q/health/live	liveness checks only
/q/health/ready	readiness checks only
/q/health/started	startup checks only
/q/health-ui	a browser UI (dev mode)

```
{
  "status": "UP",
  "checks": [
    {"name": "Database connections health check", "status": "UP"},
    {"name": "Kafka connection health check", "status": "UP",
      "data": {"orders-in": "OK"}}
  ]
}
```

A response is 200 when UP and 503 when DOWN.

16.2.1 Automatic checks

Several extensions register readiness checks automatically:

Extension	Check
quarkus-agroal	Datasource connectivity
quarkus-messaging-kafka	Broker connectivity and channel state
quarkus-redis-client	Redis ping
quarkus-mongodb-client	MongoDB ping
quarkus-elasticsearch-rest-client	Cluster health
quarkus-keycloak-authorization	Keycloak reachability
quarkus-grpc	gRPC server state

```
# These default to readiness, which is usually right - but see 16.5
quarkus.datasource.health.enabled=true
quarkus.kafka.health.enabled=true
```

```
quarkus.redis.health.enabled=false           # disable if Redis is non-critical
quarkus.mongodb.health.enabled=true
```

Audit the automatic checks before going to production. Each one turns a dependency outage into “this pod leaves the load balancer”. For a critical dependency that is correct. For an optional one — a cache, a recommendation service — it means a non-essential outage takes down a service that could have degraded gracefully. Disable those explicitly.

16.3 Writing Checks

```
package com.acme.orders.health;

import org.eclipse.microprofile.health.*;

// LIVENESS: only this process's own state
@Liveness
@ApplicationScoped
public class DeadlockCheck implements HealthCheck {

    private final ThreadMXBean threads = ManagementFactory.getThreadMXBean();

    @Override
    public HealthCheckResponse call() {
        long[] deadlocked = threads.findDeadlockedThreads();
        return HealthCheckResponse.named("deadlock")
            .status(deadlocked == null || deadlocked.length == 0)
            .withData("deadlockedThreads", deadlocked == null ? 0 : deadlocked.length)
            .build();
    }
}
```

```
// READINESS: can this pod do useful work?
@Readiness
@ApplicationScoped
public class SchemaReadinessCheck implements HealthCheck {

    @Inject AgroalDataSource dataSource;

    @Override
    public HealthCheckResponse call() {
        try (Connection connection = dataSource.getConnection();
            Statement statement = connection.createStatement()) {
            statement.setQueryTimeout(2);           // ALWAYS bound the check
            statement.execute("SELECT 1");
            return HealthCheckResponse.up("database-schema");
        } catch (SQLException e) {
            return HealthCheckResponse.builder()
                .name("database-schema")
                .down()
                .withData("error", e.getMessage())
            .build();
        }
    }
}
```



```

        .build();
    }
}

```

```

// STARTUP: one-time boot conditions
@Startup
@ApplicationScoped
public class MigrationCompleteCheck implements HealthCheck {

    @Inject Flyway flyway;

    @Override
    public HealthCheckResponse call() {
        MigrationInfo current = flyway.info().current();
        boolean applied = current != null
            && current.getState() == MigrationState.SUCCESS;
        return HealthCheckResponse.named("migrations")
            .status(applied)
            .withData("version", current == null ? "none" : current.getVersion().toString())
            .build();
    }
}

```

16.3.1 Rules for check implementations

Bound every check. A health check with no timeout will hang when its dependency hangs, which means the probe times out, which means Kubernetes takes the action it takes on failure — for liveness, a restart. Set a query timeout, a client timeout, or wrap the check.

Keep them cheap. A probe runs every few seconds on every pod. A check that issues a real query is fine; one that counts rows in a large table is not.

No side effects. A check must be safe to call at any frequency.

Return useful data. 503 alone tells an operator nothing. Include the reason.

```

// Bound an unbounded dependency check
@Readiness
@ApplicationScoped
public class DownstreamCheck implements HealthCheck {

    @RestClient InventoryClient inventory;

    @Override
    public HealthCheckResponse call() {
        try {
            inventory.ping()
                .await().atMost(Duration.ofSeconds(2)); // hard bound
            return HealthCheckResponse.up("inventory");
        } catch (TimeoutException e) {
            return HealthCheckResponse.down("inventory");
        }
    }
}

```

```
    }
}
```

16.3.2 Asynchronous checks

```
@Readiness
@ApplicationScoped
public class ReactiveCheck implements AsyncHealthCheck {

    @Inject ReactiveRedisDataSource redis;

    @Override
    public Uni<HealthCheckResponse> call() {
        return redis.value(String.class).get("health")
            .ifNoItem().after(Duration.ofSeconds(1)).fail()
            .map(v -> HealthCheckResponse.up("redis"))
            .onFailure().recoverWithItem(HealthCheckResponse.down("redis"));
    }
}
```

Prefer `AsyncHealthCheck` for anything doing I/O — a synchronous check blocks a worker thread for its duration, and probes run frequently.

16.4 Kubernetes Configuration

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: orders
          image: acme/orders-service:1.4.0

          startupProbe:
            httpGet: {path: /q/health/started, port: 9000}
            periodSeconds: 5
            failureThreshold: 30      # allows up to 150s to boot
            timeoutSeconds: 3

          livenessProbe:
            httpGet: {path: /q/health/live, port: 9000}
            periodSeconds: 10
            timeoutSeconds: 3
            failureThreshold: 3      # ~30s of failure before restart
            successThreshold: 1
```

```

    readinessProbe:
      httpGet: {path: /q/health/ready, port: 9000}
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 2      # ~10s before removal from the Service
      successThreshold: 1

    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "sleep 5"]

    terminationGracePeriodSeconds: 45

```

16.4.1 Choosing the numbers

Startup probe budget = `periodSeconds` × `failureThreshold`. It must comfortably exceed the worst realistic boot time — including a slow image pull warm-up, a cold JIT, and Flyway migrations. 150 seconds is generous for a Quarkus service (which usually boots in under two) and costs nothing, because the probe stops as soon as it passes.

Liveness must be forgiving. `failureThreshold: 3` at `periodSeconds: 10` means roughly 30 seconds of continuous failure before a restart. Being trigger-happy here converts a transient GC pause or a brief CPU starvation into a restart loop.

Readiness should be responsive. `failureThreshold: 2` at `periodSeconds: 5` removes a struggling pod from rotation in about 10 seconds. Failing readiness is cheap and reversible; failing liveness is not.

Always set `timeoutSeconds`. The default is 1 second, which a JVM under GC pressure can easily exceed, producing spurious failures.

16.4.2 Using the management port

```

quarkus.management.enabled=true
quarkus.management.port=9000
quarkus.management.root-path=/q

```

Serving health and metrics on a separate port means they are not exposed through the Service or Ingress, are unaffected by application-level authentication filters, and remain reachable even if the application port's thread pool is saturated. In Kubernetes this is the better default.

16.5 The Cascading Failure Problem

This is the senior-level content of the chapter.

Adding a downstream dependency to a **readiness** check feels responsible. It is often catastrophic.

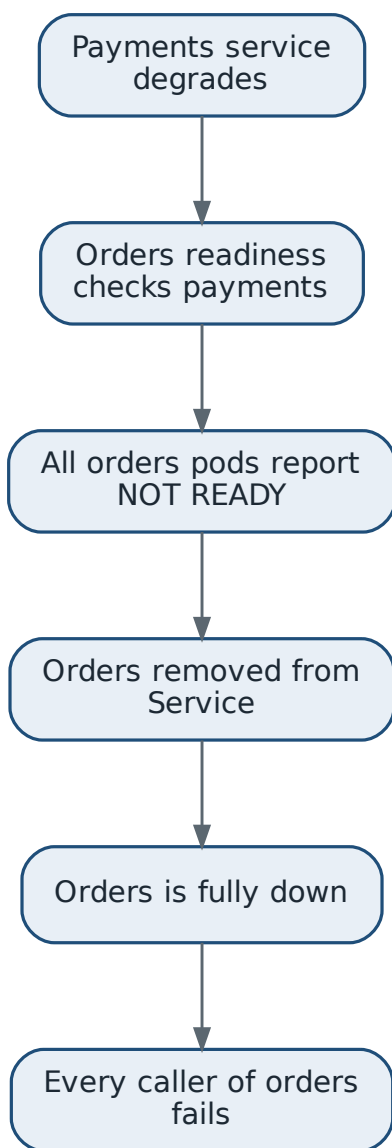


Figure 16.2 — A readiness check on a downstream dependency propagates its outage into your own.

The scenario: `orders` has a readiness check on `payments`. `payments` has one on `ledger`. `ledger` has a five-minute problem. Now `payments` is not ready, therefore `orders` is not ready, therefore the gateway is not ready. A partial degradation in one leaf service has become a total outage across the estate — and crucially, `orders` could have kept serving every read request perfectly well.

16.5.1 The principle

Readiness should reflect whether *this pod* can do its own job — not whether the whole system is healthy.

Concretely:

Check dependencies you cannot function at all without. If every endpoint requires the database, a database check in readiness is defensible — although even then, consider whether returning 503 from the affected endpoints is better than removing the pod, because at least the health endpoint and any cached reads stay available.

Do not check optional dependencies. A cache, a recommendation engine, a notification service. Degrade instead: catch the failure, serve without the enrichment, and record a metric.

Never check a dependency in liveness. Restarting cannot fix someone else's outage.

Prefer circuit breakers to readiness checks for downstream failure. A circuit breaker (Chapter 2) stops you wasting threads on a failing dependency while keeping the rest of the service serving. That is graceful degradation; readiness failure is all-or-nothing.

```
// Instead of a readiness check on payments:
@ApplicationScoped
public class CheckoutService {

    @RestClient PaymentClient payments;

    public OrderView checkout(Order order) {
        try {
            return complete(order, payments.capture(order));
        } catch (CircuitBreakerOpenException | TimeoutException e) {
            metrics.counter("checkout.degraded").increment();
            return queueForLaterCapture(order);    // degrade, do not disappear
        }
    }
}
```

16.5.2 A defensible readiness policy

```
@Readiness
@ApplicationScoped
public class ReadinessCheck implements HealthCheck {

    @Inject AgroalDataSource dataSource;    // required
    @Inject FeatureFlags flags;

    @Override
    public HealthCheckResponse call() {
        HealthCheckResponseBuilder builder = HealthCheckResponse.named("orders-ready");

        // Required: no endpoint works without it
        boolean db = canQuery(dataSource, Duration.ofSeconds(2));
        builder.withData("database", db ? "UP" : "DOWN");

        // Optional: reported, but does not affect the verdict
    }
}
```

```

    builder.withData("recommendations", flags.recommendationsHealthy() ? "UP" :
"DEGRADED");

    return builder.status(db).build();
}
}

```

Reporting the optional dependency in `data` without letting it affect the status gives operators visibility without giving it the power to take the service down.

16.6 Health and Graceful Shutdown

The two interact, and getting it wrong drops requests on every deploy.

The problem: when a pod is deleted, Kubernetes does two things **concurrently** — sends `SIGTERM` to the container, and begins removing the pod from Service endpoints. Endpoint propagation to every kube-proxy and every ingress takes time. If the application exits immediately on `SIGTERM`, requests routed during that window are dropped.

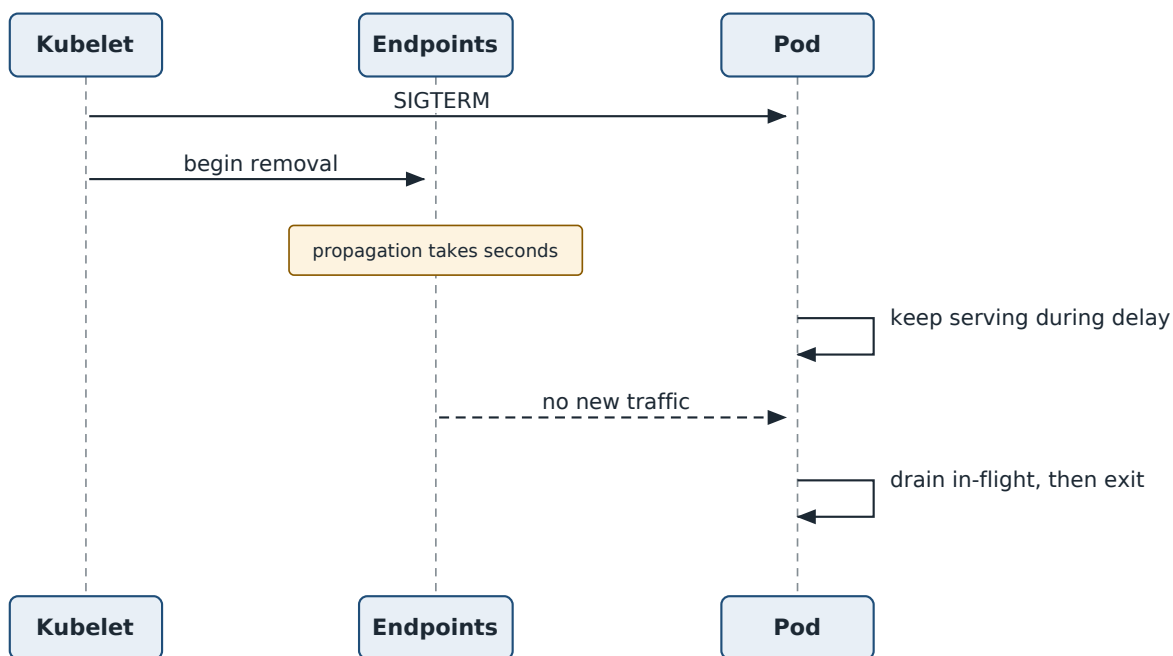


Figure 16.3 — The shutdown delay covers the endpoint-propagation window.

```
# Keep serving for a few seconds after SIGTERM so endpoint removal propagates
quarkus.shutdown.delay=5S
```

```
# Then finish in-flight requests before exiting
quarkus.shutdown.timeout=30S
```

```

lifecycle:
  preStop:
    exec:

```

```
command: ["sh", "-c", "sleep 5"]
terminationGracePeriodSeconds: 45      # > delay + timeout
```

Either `quarkus.shutdown.delay` or a `preStop` sleep will do; using both is belt and braces. What matters is that `terminationGracePeriodSeconds` exceeds `delay + timeout`, or Kubernetes will `SIGKILL` mid-drain and you lose the requests you were carefully finishing.

```
@ApplicationScoped
public class ShutdownCoordinator {

    private volatile boolean shuttingDown = false;

    void onShutdown(@Observes ShutdownEvent event) {
        shuttingDown = true;
        Log.info("Shutdown initiated; readiness will report DOWN");
    }

    public boolean isShuttingDown() { return shuttingDown; }
}
```

```
@Readiness
@ApplicationScoped
public class DrainingCheck implements HealthCheck {

    @Inject ShutdownCoordinator coordinator;

    @Override
    public HealthCheckResponse call() {
        return HealthCheckResponse.named("draining")
            .status(!coordinator.isShuttingDown())
            .build();
    }
}
```

16.7 Health Checks Are Not Monitoring

A common confusion worth stating plainly.

	Health checks	Monitoring
Audience	The orchestrator	Humans
Question	Restart? Route traffic?	Is it working well?
Output	Binary UP/DOWN	Continuous measurement
Frequency	Every few seconds	Continuous
Action	Automatic	Investigation

Health checks are an **actuator interface for Kubernetes**. They should be binary, cheap, fast and boring. Putting business logic, aggregate statistics or “is the error rate acceptable” into a

health check gives an automated system the power to restart your pods based on a judgement it is not equipped to make.

```
// BAD: gives Kubernetes the power to restart pods based on downstream error rates
@Liveness
public HealthCheckResponse call() {
    return HealthCheckResponse.named("error-rate")
        .status(metrics.errorRate() < 0.05)    // NO
        .build();
}
```

Under a genuine incident this restarts every pod simultaneously, which is precisely the wrong response. Error rate belongs in an alert, where a human decides.

16.8 Testing

```
@QuarkusTest
class HealthEndpointTest {

    @Test
    void liveIsUp() {
        given().when().get("/q/health/live")
            .then().statusCode(200)
                .body("status", is("UP"));
    }

    @Test
    void readyIncludesDatabase() {
        given().when().get("/q/health/ready")
            .then().statusCode(200)
                .body("status", is("UP"))
                .body("checks.name", hasItem(containsString("Database")));
    }

    @Test
    void livenessDoesNotDependOnDatabase() {
        // The important test: liveness must stay UP when a dependency is down
        QuarkusMock.installMockForType(brokenDataSource(), AgroalDataSource.class);

        given().when().get("/q/health/live")
            .then().statusCode(200)
                .body("status", is("UP"));
    }
}
```

```
@QuarkusTest
class ReadinessCheckTest {

    @Inject ReadinessCheck check;
    @InjectMock AgroalDataSource dataSource;
```



```

@Test
void reportsDownWhenDatabaseUnavailable() throws Exception {
    when(dataSource.getConnection()).thenReturn(new SQLException("refused"));

    HealthCheckResponse response = check.call();

    assertThat(response.getStatus()).isEqualTo(HealthCheckResponse.Status.DOWN);
    assertThat(response.getData()).hasValueSatisfying(
        data -> assertThat(data).containsEntry("database", "DOWN"));
}

@Test
void staysUpWhenOptionalDependencyIsDown() {
    when(flags.recommendationsHealthy()).thenReturn(false);
    assertThat(check.call().getStatus()).isEqualTo(HealthCheckResponse.Status.UP);
}
}

```

livenessDoesNotDependOnDatabase is the test worth writing. It encodes the chapter's central rule as an executable assertion, and it will catch the well-intentioned pull request that adds a database check to liveness.

16.9 Best Practices

1. **Liveness checks only this process's own state.** Never a dependency.
2. **Readiness checks only what this pod genuinely cannot work without.**
3. **Circuit breakers, not readiness failure,** for downstream degradation.
4. **A startup probe with a generous budget,** so liveness can stay aggressive without penalising boot.
5. **Bound every check** with an explicit timeout.
6. **Serve health on the management port** so it is unaffected by application load and not publicly exposed.
7. **shutdown.delay** plus **terminationGracePeriodSeconds** greater than **delay + timeout**.
8. **Report optional dependencies in data** without letting them affect the status.
9. **Audit the automatic checks** each extension registers.
10. **No business logic in health checks.** They are an actuator interface, not monitoring.
11. **timeoutSeconds** explicitly — the 1-second default is too tight for a JVM under GC.
12. **Test that liveness survives a dependency outage.**

16.10 Common Mistakes

1. **A dependency check in liveness.** A database blip restarts every pod, turning a transient problem into a multi-service outage with a cold-start stampede.
2. **Optional dependencies in readiness.** A recommendation service going down takes the whole service out of rotation when it could have degraded.
3. **No startup probe.** Either liveness is too aggressive and kills slow-starting pods, or it is too lax and takes minutes to detect a genuinely dead one.

4. **Unbounded checks.** The check hangs, the probe times out, Kubernetes acts.
5. **`terminationGracePeriodSeconds` shorter than the drain time.** `SIGKILL` mid-drain drops in-flight requests.
6. **No shutdown delay.** Requests routed during endpoint propagation are dropped on every deploy.
7. **Health checks on the application port.** Unavailable exactly when the thread pool is saturated — the moment you need them.
8. **Business logic or error rates in a health check.** Gives an automated system the power to restart everything during an incident.
9. **Expensive checks.** A count over a large table, run every five seconds on every pod.
10. **Health endpoints publicly exposed.** They enumerate your dependencies for anyone who asks.
11. **`503` with no `data`.** An operator learns nothing.

16.11 Interview Questions

Junior

- Q1. What is a health check?** An endpoint that reports whether the application is functioning, used by an orchestrator to decide whether to restart the container or route traffic to it.
- Q2. Which endpoints does Quarkus expose?** `/q/health`, `/q/health/live`, `/q/health/ready` and `/q/health/started`, plus a `/q/health-ui` in dev mode.
- Q3. How do you write a custom check?** Implement `HealthCheck`, annotate the bean `@Liveness`, `@Readiness` or `@Startup`, and return a `HealthCheckResponse`.
- Q4. What status code does a failing check return?** `503`, with a JSON body listing each check and its status.
- Q5. What is the difference between liveness and readiness?** Liveness failure makes Kubernetes restart the container. Readiness failure removes the pod from Service endpoints without restarting it.

Mid-level

- Q6. Should a liveness probe check the database?** No. Liveness answers “would a restart fix this?”, and restarting cannot fix someone else’s outage. Worse, a database blip would restart every pod across every service, discarding warm caches and pools and hitting the recovering database with a cold-start stampede.
- Q7. What is a startup probe for?** It suppresses liveness and readiness until the application has finished booting. Without it you must choose between a liveness probe lenient enough to tolerate slow starts — and therefore slow to detect a genuinely dead process — or an aggressive one that kills pods during boot. The startup probe removes the trade-off.
- Q8. Which checks does Quarkus register automatically?** Readiness checks for datasources, Kafka, Redis, MongoDB, Elasticsearch, gRPC and others, depending on the extensions present.

They should be audited, because each turns a dependency outage into a pod leaving the load balancer.

Q9. How do you avoid dropping requests during a rolling deploy? `quarkus.shutdown.delay` so the pod keeps serving while endpoint removal propagates, `quarkus.shutdown.timeout` to finish in-flight requests, and `terminationGracePeriodSeconds` set greater than the sum so Kubernetes does not `SIGKILL` mid-drain.

Q10. Why serve health on a separate port? It is not exposed through the Service or Ingress, is unaffected by application authentication filters, and stays reachable when the application port's thread pool is saturated — which is exactly when you need it.

Senior

Q11. Explain how health checks can cause a cascading failure. If service A's readiness check calls service B, then B's degradation makes every A pod report not-ready, so A is removed from its Service and is now completely down — even though most of A's endpoints never touch B. If B in turn checks C, the failure propagates upward through the whole estate, and a partial problem in one leaf service becomes a total outage. The correct model is that readiness reflects whether *this pod* can do its own job. Downstream failure is handled with circuit breakers and graceful degradation, which keeps the unaffected parts of the service serving. I would check a dependency in readiness only when literally no endpoint works without it, and even then I would consider whether returning 503 from the affected endpoints is better than removing the pod entirely.

Q12. Design the probe configuration for a service that takes 90 seconds to start. A startup probe with a budget comfortably above the worst case — `periodSeconds: 10` with `failureThreshold: 18` gives 180 seconds, and costs nothing because the probe stops as soon as it passes. That lets me keep liveness aggressive, at roughly 30 seconds of continuous failure before a restart, without any risk of killing a pod mid-boot. Readiness would be responsive at around 10 seconds, since failing it is cheap and reversible. I would also ask *why* it takes 90 seconds, because that is unusual for Quarkus and usually means work is being done eagerly at startup that could be deferred, or that the pod is waiting on a dependency with a long timeout — and if it is the latter, a slow dependency at boot is a resilience problem the probe configuration is only papering over. For a rollout, a 90-second start also means `maxSurge` and `maxUnavailable` need thought, since replacing all replicas serially takes minutes.

Q13. Health check or circuit breaker for a failing downstream service? Circuit breaker, in almost every case. They answer different questions: a circuit breaker is per-dependency and per-call, so it stops wasting threads and latency budget on a failing service while the rest of the application keeps serving; a readiness failure is all-or-nothing and removes the entire pod. If eighty per cent of my endpoints do not touch the failing dependency, taking the pod out of rotation is a strictly worse outcome for users than degrading the twenty per cent. The circuit breaker also gives the downstream service room to recover by shedding load, which a readiness failure does not — my pods leaving the load balancer does nothing to reduce pressure on the dependency. I would use readiness for a dependency only when there is genuinely no useful work the pod can do without it, and I would still report the dependency's state in the health `data` for visibility.

Principal

Q14. Standardise health checks across 40 services. What do you mandate? Three rules, and they are all about limiting the blast radius of automated decisions. First: liveness

checks process-local state only, never a dependency — enforced by a shared library that provides the liveness check and by a CI test asserting that liveness stays UP with dependencies mocked out, because this is the rule that gets violated with good intentions and the failure is severe and estate-wide. Second: readiness checks only dependencies without which no endpoint functions, with a documented justification per check, and optional dependencies reported in `data` rather than affecting status. Third: a startup probe on every service, so liveness can be aggressive uniformly. Beyond that I would standardise the operational envelope — health on the management port, probe timings as a template with per-service overrides, `shutdown.delay` and grace period defaults that are consistent so rollouts behave predictably — all shipped as an extension plus a Helm library chart rather than as documentation. What I would leave to teams is which checks are meaningful for their domain. The thing I would monitor centrally is restart rate by service, because a rising restart count is the signature of a liveness misconfiguration and it is invisible to the owning team until it becomes an incident.

Q15. Argue for having no readiness checks on dependencies at all. The strong version of the argument is that dependency checks in readiness delegate an availability decision to a mechanism that has no context. Kubernetes cannot know that the failing dependency affects two endpoints out of thirty, or that the cached path would still work, or that the dependency is recovering — it only sees a boolean and removes the pod. Meanwhile the application layer knows all of that and can degrade precisely: serve the reads, queue the writes, return 503 from the specific endpoints that need the dependency, and record a metric so humans can see the degradation. That is strictly more information-preserving than disappearing. There is also a symmetry argument: removing your pods from the load balancer does nothing to help the failing dependency, and it does actively harm your callers, so the action is all cost and no benefit. The counter-case is a service where every code path genuinely requires the dependency — a thin CRUD service over one database — and there, readiness failure and returning 503 are nearly equivalent, so it hardly matters. Where I would stop short of the absolute position is startup: checking that the database is reachable *before* accepting traffic for the first time is reasonable, because a pod that has never worked is different from one that is temporarily impaired. So my practical rule is: dependency checks are acceptable in startup, questionable in readiness, and never in liveness.

16.12 Cheat Sheet

Probes

liveness	-> failure RESTARTS the container	-> process-local state ONLY
readiness	-> failure REMOVES from Service	-> can this pod do its job?
startup	-> suppresses the other two	-> has boot finished?

Endpoints

`/q/health` `/q/health/live` `/q/health/ready` `/q/health/started` `/q/health-ui`
 200 = UP, 503 = DOWN

Checks

```
@Liveness @ApplicationScoped
class C implements HealthCheck {
    public HealthCheckResponse call() {
        return HealthCheckResponse.named("n").status(ok).withData("k", v).build();
    }
}
```

```

}

@Readiness @ApplicationScoped
class A implements AsyncHealthCheck {
    public Uni<HealthCheckResponse> call() { ... }
}

```

Kubernetes

```

startupProbe:  {httpGet: {path: /q/health/started, port: 9000},
                periodSeconds: 5, failureThreshold: 30}
livenessProbe: {httpGet: {path: /q/health/live, port: 9000},
                periodSeconds: 10, failureThreshold: 3, timeoutSeconds: 3}
readinessProbe: {httpGet: {path: /q/health/ready, port: 9000},
                 periodSeconds: 5, failureThreshold: 2, timeoutSeconds: 3}
terminationGracePeriodSeconds: 45

```

Graceful shutdown

```

quarkus.shutdown.delay=5S      # cover endpoint propagation
quarkus.shutdown.timeout=30S   # drain in-flight
# terminationGracePeriodSeconds > delay + timeout

```

Management port

```

quarkus.management.enabled=true
quarkus.management.port=9000

```

Disable automatic checks

```

quarkus.datasource.health.enabled=true
quarkus.kafka.health.enabled=true
quarkus.redis.health.enabled=false

```

16.13 Summary

Three probes, three questions, three very different consequences. **Liveness** asks whether a restart would help, and must therefore examine only this process's own state — a dependency check here turns a five-minute database blip into an estate-wide restart storm. **Readiness** asks whether this pod should receive traffic, and should reflect only what the pod genuinely cannot work without. **Startup** removes the trade-off between tolerating slow boots and detecting dead processes quickly.

The senior-level insight is that readiness checks on downstream dependencies propagate outages upward. A degraded leaf service makes its callers not-ready, which makes *their* callers not-ready, and a partial problem becomes a total one. Circuit breakers and graceful degradation are the right tool: they are per-dependency and per-call, so the unaffected parts of the service keep working, and they shed load from the struggling dependency rather than merely disappearing.

Two operational details prevent dropped requests on every deploy: a shutdown delay long enough to cover Kubernetes endpoint propagation, and a `terminationGracePeriodSeconds` greater than `delay + timeout` so the pod is not killed mid-drain.

Finally, keep health checks boring. They are an actuator interface for an automated system with no judgement — binary, cheap, bounded, side-effect-free. Error rates and business conditions belong in alerts, where a human decides.

Chapter 17 turns to **testing** — the strategy, the tools, the Dev Services integration, and how to build a suite that runs fast enough that people actually run it.

16.14 References

- SmallRye Health — <https://quarkus.io/guides/smallrye-health>
- MicroProfile Health 4.0 — <https://github.com/eclipse/microprofile-health>
- Kubernetes liveness, readiness and startup probes — <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>
- Pod lifecycle and termination — <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- Google SRE Book, on cascading failures — <https://sre.google/sre-book/addressing-cascading-failures/>

Testing

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · JUnit 5 · RestAssured · Mockito · Testcontainers · WireMock · AssertJ

Learning Objectives

By the end of this chapter you will be able to:

1. Choose the right test type for a given question, and explain the cost of each.
2. Use `@QuarkusTest`, `@QuarkusComponentTest` and `@QuarkusIntegrationTest` correctly, and know why they differ.
3. Exploit **Dev Services** so tests run against real PostgreSQL, Kafka and Keycloak rather than substitutes.
4. Mock beans, REST clients and external HTTP services without making tests lie.
5. Manage test data and isolation with `@TestTransaction` and deliberate fixtures.
6. Keep the suite **fast** — the property that determines whether it gets run.
7. Test the things that are usually untested: error paths, authorisation denials, idempotency, N+1 regressions.
8. Run the same suite against a native binary.
9. Diagnose and eliminate flaky tests.
10. Answer 30 interview questions on testing strategy and practice.

17.1 The Test Types

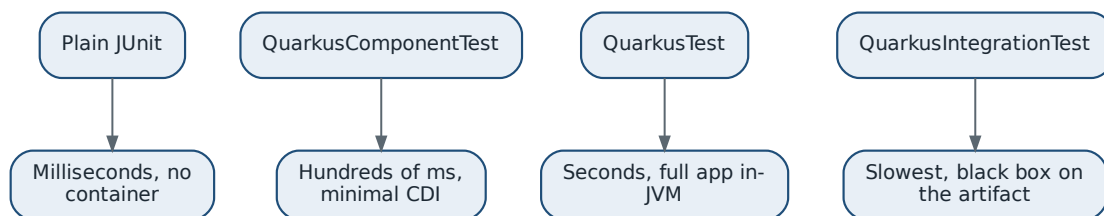


Figure 17.1 — Cost rises sharply. Choose the cheapest type that answers the question.

	JUnit	Component	Quarkus	Integration
Startup	none	~200 ms	1–3 s once	the artifact
CDI	no	listed beans	full	none
<code>@Inject</code>	no	yes	yes	no
<code>@InjectMock</code>	no	yes	yes	no
Dev Services	no	no	yes	yes
HTTP endpoints	no	no	yes	yes
Native mode	no	no	no	yes

Columns: *plain JUnit* · `@QuarkusComponentTest` · `@QuarkusTest` · `@QuarkusIntegrationTest`.

Use the cheapest type that answers the question. A suite of 400 `@QuarkusTest` classes with different profiles takes fifteen minutes and stops being run. A suite of 380 plain unit tests and 20 integration tests takes ninety seconds and gets run on every save.

17.2 Plain JUnit — the Default

Constructor injection (Chapter 3) makes most of your code testable with no framework at all.

```
class PricingServiceTest {

    private final TaxService tax = mock(TaxService.class);
    private final DiscountRepository discounts = mock(DiscountRepository.class);
    private final Clock clock = Clock.fixed(Instant.parse("2026-07-31T10:00:00Z"), UTC);

    private final PricingService service = new PricingService(tax, discounts, clock);

    @Test
    void appliesTaxAtTheDestinationRate() {
        when(tax.rateFor("GB")).thenReturn(new BigDecimal("0.20"));

        BigDecimal price = service.price(orderOf("100.00", "GB"));

        assertThat(price).isEqualByComparingTo("120.00");
    }

    @Test
    void appliesLoyaltyDiscountBeforeTax() {
        when(tax.rateFor("GB")).thenReturn(new BigDecimal("0.20"));
        when(discounts.forCustomer(1L)).thenReturn(new Discount(new BigDecimal("0.10")));

        assertThat(service.price(orderOf("100.00", "GB", 1L)))
            .isEqualByComparingTo("108.00");
    }

    @Test
    void rejectsNegativeQuantities() {
        assertThatThrownBy(() -> service.price(orderWithQuantity(-1)))
```



```

        .isInstanceOf(InvalidOrderException.class)
        .hasMessageContaining("quantity");
    }
}

```

Milliseconds per test, no container, no Docker, runs in the IDE instantly. **If a class cannot be tested this way, that is usually a design signal** — it has too many dependencies, or it reaches out to static state, or it mixes orchestration with logic.

Note the injected `Clock`. Any logic that depends on time should take a `Clock` rather than calling `Instant.now()`, which makes “expire orders older than 30 days” testable without waiting.

17.3 @QuarkusComponentTest

The middle ground: a minimal CDI container with only the beans you name.

```

@QuarkusComponentTest
class OrderValidatorTest {

    @Inject OrderValidator validator;           // the bean under test
    @InjectMock StockService stock;           // its collaborator, mocked

    @Test
    void rejectsWhenStockIsInsufficient() {
        when(stock.availableFor("SKU-1")).thenReturn(0);

        assertThatThrownBy(() -> validator.validate(orderFor("SKU-1", 2)))
            .isInstanceOf(InsufficientStockException.class);
    }
}

```

```

// Configuration and explicit bean lists
@QuarkusComponentTest(value = {OrderValidator.class, PricingService.class},
    addNestedClassesAsComponents = false)
@TestConfigProperty(key = "orders.max-line-items", value = "3")
class ConfiguredValidatorTest {

    @Inject OrderValidator validator;

    @Test
    void enforcesConfiguredLimit() {
        assertThatThrownBy(() -> validator.validate(orderWithLines(4)))
            .isInstanceOf(TooManyLineItemsException.class);
    }
}

```

Roughly ten times faster than `@QuarkusTest` and it exercises real CDI — interceptors, producers, config injection. Use it when the thing you are testing depends on the container but not on HTTP or a database.

17.4 @QuarkusTest

The full application in the same JVM as the test.

```
@QuarkusTest
class OrderResourceTest {

    @Test
    void createsAnOrder() {
        given()
            .contentType(ContentType.JSON)
            .body("""
                {
                    "customerEmail": "buyer@example.com",
                    "items": [{"sku": "SKU-1", "quantity": 2}]
                }
            """)
        .when()
            .post("/orders")
        .then()
            .statusCode(201)
            .header("Location", matchesPattern(".*orders/\\d+"))
            .body("status", is("PENDING"))
            .body("total", is(24.00f))
            .body("items", hasSize(1));
    }

    @Test
    void returnsProblemDetailForInvalidInput() {
        given().contentType(ContentType.JSON)
            .body("""{"customerEmail": "not-an-email", "items": []}""")
        .when().post("/orders")
        .then().statusCode(400)
            .body("type", endsWith("validation-failed"))
            .body("fields.customerEmail", hasItem(containsString("email")))
            .body("fields.items", hasItem("must not be empty"));
    }
}
```

Quarkus starts **once** for the whole suite and reuses the instance across all `@QuarkusTest` classes — unless something forces a restart.

17.4.1 What forces a restart

This is the single biggest lever on suite duration.

```
// Each DISTINCT profile causes a restart
@QuarkusTest
@TestProfile(StrictProfile.class)    // restart
class A { }

@QuarkusTest
@TestProfile(StrictProfile.class)    // SAME profile - no restart
class B { }
```

```
@QuarkusTest
@TestProfile(LenientProfile.class)    // different - restart
class C { }
```

Also restart-inducing: `@QuarkusTestResource` with different resources, and `QuarkusMock.installMockForType` in some configurations.

A suite that takes eight minutes usually has twelve profiles. Group tests by profile, share profiles aggressively, and prefer `@InjectMock` (which does not restart) to a profile that swaps an implementation. Order matters too — JUnit runs classes in a deterministic but arbitrary order, so interleaved profiles restart more often than clustered ones; `@TestProfile` classes that are shared naturally cluster.

```
@QuarkusTest
class CheckoutServiceTest {

    @Inject CheckoutService checkout;

    @InjectMock PaymentGateway gateway;           // replaces the bean
    @InjectMock @RestClient InventoryClient inventory;
    @InjectSpy AuditService audit;                 // real bean, verifiable

    @Test
    void queuesWhenPaymentIsPending() {
        when(gateway.capture(any())).thenReturn(PaymentResult.pending("p-1"));
        when(inventory.reserve(any())).thenReturn(Reservation.ok());

        OrderView result = checkout.checkout(1L);

        assertThat(result.status()).isEqualTo("PAYMENT_PENDING");
        verify(audit).record(eq("PAYMENT_PENDING"), any());
    }
}
```

```
// Global mock for the whole suite - no restart
@Mock
@ApplicationScoped
public class MockPaymentGateway implements PaymentGateway {
    @Override public PaymentResult capture(Order order) {
        return PaymentResult.approved("test-" + order.id);
    }
}
```

`@InjectMock` cannot mock `@Dependent` beans — there is no single contextual instance to replace. Use `@InjectMock(convertScopes = true)` where appropriate, or restructure.

17.5 Dev Services in Tests

This is Quarkus’s strongest testing feature and the reason its integration tests are worth writing.

```
# Do NOT configure a URL in dev/test - let Dev Services provide the real thing
# quarkus.datasource.jdbc.url=...
```

```
%test.quarkus.datasource.devservices.image-name=postgres:16-alpine
%test.quarkus.datasource.devservices.reuse=true
%test.quarkus.hibernate-orm.database.generation=drop-and-create
```

```
@QuarkusTest
@TestTransaction // each test rolls back
class OrderRepositoryTest {

    @Inject OrderRepository repository;

    @Test
    void findsByStatus() {
        repository.persist(orderWith(OrderStatus.PAID));
        repository.persist(orderWith(OrderStatus.PENDING));

        assertThat(repository.findPending()).hasSize(1);
    }

    @Test
    void enforcesUniqueReference() {
        repository.persistAndFlush(orderWithReference("REF-1"));

        assertThatThrownBy(() -> repository.persistAndFlush(orderWithReference("REF-1")))
            .isInstanceOf(PersistenceException.class);
    }
}
```

Testing against real PostgreSQL rather than H2 eliminates an entire class of bug. Dialect differences, **NUMERIC** precision, **TIMESTAMP** semantics, case sensitivity in identifiers, **ON CONFLICT**, **JSONB** — none of these behave the same on H2, and every one of them has produced a “worked in tests, failed in production” incident somewhere. The **enforcesUniqueReference** test above is a good example: on H2 the constraint may behave differently or not exist at all.

Dev Services covers PostgreSQL, MySQL, MariaDB, MS SQL, Oracle, MongoDB, Kafka (Redpanda), Redis, Keycloak, Elasticsearch, RabbitMQ, Pulsar, Vault, Infinispan and more.

```
%test.quarkus.kafka.devservices.provider=redpanda
%test.quarkus.keycloak.devservices.realm-path=acme-realm.json
%test.quarkus.redis.devservices.image-name=redis:7-alpine
```

17.5.1 Custom test resources

For infrastructure Dev Services does not cover:

```
public class WireMockResource implements QuarkusTestResourceLifecycleManager {
```

```

private WireMockServer server;

@Override
public Map<String, String> start() {
    server = new WireMockServer(options().dynamicPort());
    server.start();

    server.stubFor(get(urlPathMatching("/payments/.*"))
        .willReturn(okJson("""
            {"id": "p-1", "status": "APPROVED"}
            """)));

    return Map.of("quarkus.rest-client.payments-api.url", server.baseUrl());
}

@Override
public void stop() {
    if (server != null) server.stop();
}

@Override
public void inject(TestInjector injector) {
    injector.injectIntoFields(server,
        new TestInjector.AnnotatedAndMatchesType(InjectWireMock.class,
            WireMockServer.class));
}
}

```

```

@QuarkusTest
@QuarkusTestResource(WireMockResource.class)
class PaymentIntegrationTest {

    @InjectWireMock WireMockServer wireMock;

    @Test
    void handlesDownstreamTimeout() {
        wireMock.stubFor(post(urlEqualTo("/payments"))
            .willReturn(aResponse().withFixedDelay(5000)));

        given().contentType(JSON).body(validRequest())
            .when().post("/orders")
            .then().statusCode(202) // degraded, not failed
                .body("status", is("PAYMENT_PENDING"));
    }
}

```

WireMock is the right tool for testing **failure modes** of a downstream service — timeouts, 500s, 429s, malformed responses, slow responses. Those paths are almost never exercised otherwise, and they are exactly where resilience bugs live.

17.6 Test Data and Isolation

17.6.1 @TestTransaction

```
@QuarkusTest
@TestTransaction           // class-level: every test method
class OrderServiceTest {

    @Test
    void createsOrder() {
        service.create(validRequest());
        assertThat(Order.count()).isEqualTo(1);
    }                       // rolled back - no cleanup needed
}
```

The cleanest isolation mechanism available: each test runs in a transaction that is rolled back, so tests never see each other's data and there is no teardown code to forget.

Its limitation: you cannot test **actual rollback behaviour**, because the whole test is already inside a transaction destined for rollback.

```
@QuarkusTest
class RollbackTest {

    @Inject OrderService service;

    @AfterEach
    @Transactional
    void cleanup() { Order.deleteAll(); }

    @Test           // deliberately NOT @TestTransaction
    void rollsBackOnInsufficientStock() {
        assertThatThrownBy(() -> service.create(unstockedRequest()))
            .isInstanceOf(InsufficientStockException.class);

        assertThat(Order.count()).isZero();
    }
}
```

17.6.2 Fixtures

```
public final class OrderFixtures {

    public static CreateOrderRequest.Builder aValidOrder() {
        return CreateOrderRequest.builder()
            .customerEmail("buyer@example.com")
            .items(List.of(aLineItem().build()));
    }

    public static LineItemRequest.Builder aLineItem() {
        return LineItemRequest.builder().sku("SKU-1").quantity(1);
    }
}
```

```

@Test
void rejectsExcessiveQuantity() {
    var request = aValidOrder()
        .items(List.of(aLineItem().quantity(10_000).build()))
        .build();

    assertThatThrownBy(() -> service.create(request))
        .isInstanceOf(QuantityLimitExceededException.class);
}

```

A builder-based fixture makes the *relevant* difference visible. A test that constructs a fully-populated request inline buries the one field that matters among twelve that do not, and the reader cannot tell what the test is actually about.

17.7 What Usually Goes Untested

A suite that only covers happy paths gives false confidence. These are the areas where bugs actually reach production.

17.7.1 Error responses

```

@Test
void returnsProblemDetailWithCleanFieldPaths() {
    given().contentType(JSON).body("""
        {"customerEmail": "bad", "items": [{"sku": "S", "quantity": 0}]}
        """)
    .when().post("/orders")
    .then().statusCode(400)
        .body("fields", hasKey("customerEmail"))
        .body("fields", hasKey("items[0].quantity")) // the client depends on this
        .body("fields", not(hasKey(containsString("arg0"))));
}

```

17.7.2 Authorisation denials

```

@Test @TestSecurity(user = "bob", roles = "order-reader")
void readerCannotDelete() {
    given().when().delete("/orders/1").then().statusCode(403);
}

@Test @TestSecurity(user = "bob", roles = "order-reader")
void cannotSeeAnotherUsersOrder() {
    given().when().get("/orders/" + aliceOrderId)
        .then().statusCode(404); // 404, not 403 - no existence leak
}

@Test
void anonymousIsRejected() {
    given().when().get("/orders").then().statusCode(401);
}

```

Every `@RolesAllowed` deserves a test that the *wrong* role is denied. A suite proving only that admins can delete has verified nothing about security.

17.7.3 Idempotency

```
@Test
void duplicateEventIsIgnored() {
    var event = new OrderEvent("evt-1", 1L, "PENDING", TEN, Instant.now());

    connector.source("orders-in").send(event);
    connector.source("orders-in").send(event);           // same eventId

    await().atMost(5, SECONDS).untilAsserted(() ->
        assertThat(Order.count("externalId", 1L)).isEqualTo(1));
}
```

17.7.4 N+1 regressions

```
@QuarkusTest
class QueryCountTest {

    @Inject EntityManager em;
    @Inject OrderQueries queries;

    private Statistics stats() {
        return em.getEntityManagerFactory().unwrap(SessionFactory.class).getStatistics();
    }

    @Test
    @TestTransaction
    void listingIssuesOneQuery() {
        seedOrders(50);
        stats().clear();

        queries.summaries(OrderStatus.PAID);

        assertThat(stats().getPrepareStatementCount())
            .as("N+1 regression: listing should issue exactly one query")
            .isEqualTo(1);
    }
}
```

This test is worth more than any code review. N+1 regressions are introduced by innocuous changes — adding a field to a DTO mapper — and are invisible until the data grows. A query-count assertion catches them the moment they appear.

17.7.5 Cardinality

```
@Test
void metricsUseNoIdentifierTags() {
    service.create(validRequest());

    Set<String> forbidden = Set.of("user_id", "order_id", "request_id", "trace_id");
    assertThat(registry.getMeters())
        .allSatisfy(m -> assertThat(m.getId().getTags())
            .noneMatch(t -> forbidden.contains(t.getKey())));
}
```

17.7.6 Liveness independence

```
@Test
void livenessSurvivesDatabaseOutage() {
    QuarkusMock.installMockForType(brokenDataSource(), AgroalDataSource.class);

    given().when().get("/q/health/live").then().statusCode(200);
}
```

17.8 @QuarkusIntegrationTest and Native

```
@QuarkusTest
class OrderResourceTest {
    // written using only HTTP assertions - no @Inject
    @Test void createsAnOrder() { ... }
}

@QuarkusIntegrationTest
class OrderResourceIT extends OrderResourceTest { }
```

That two-line subclass runs the **same tests** against the packaged artifact: the fast-jar, the container image, or the native binary.

```
./mvnw verify                # JVM artifact
./mvnw verify -Dnative        # native binary
./mvnw verify -Dquarkus.container-image.build=true # container image
```

```
@QuarkusIntegrationTest
class OrderResourceIT extends OrderResourceTest {

    @TestHTTPResource("/orders") URL ordersUrl;    // still injectable

    @Test
    void reportsHealthy() {
        given().when().get("/q/health/ready").then().statusCode(200);
    }
}
```

Constraints, all consequences of it being a black-box test against a separate process: no `@Inject`, no `@InjectMock`, no `@TestTransaction`, no access to application internals.

This is the single most valuable native-mode practice. Reflection, resource loading and serialisation problems that only manifest in native mode are found by your existing test suite rather than by a customer. Write `@QuarkusTest` classes with HTTP-only assertions specifically so they can be reused this way, and run the native `verify` nightly rather than on every PR — the build takes minutes.

17.9 Keeping the Suite Fast

Speed is not a nicety. A suite that takes fifteen minutes is a suite that gets skipped, and a skipped suite has no value at all.

1. Push tests down the pyramid. Most logic should be testable with plain JUnit. Reserve `@QuarkusTest` for wiring, HTTP contracts and persistence.

2. Minimise profiles. Each distinct `@TestProfile` restarts Quarkus. Share them; use `@InjectMock` instead where possible.

3. Reuse containers.

```
%test.quarkus.datasource.devservices.reuse=true
```

```
# ~/.testcontainers.properties
testcontainers.reuse.enable=true
```

4. Parallelise.

```
# junit-platform.properties
junit.jupiter.execution.parallel.enabled=true
junit.jupiter.execution.parallel.mode.default=concurrent
junit.jupiter.execution.parallel.mode.classes.default=same_thread
```

Be careful: parallel `@QuarkusTest` classes share one application instance and one database, so tests that mutate shared state will interfere. `@TestTransaction` helps but does not isolate everything.

5. Split by tag.

```
@Tag("slow")
@QuarkusTest
class FullPipelineTest { }
```

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <excludedGroups>slow</excludedGroups>    <!-- PR builds skip these -->
  </configuration>
</plugin>
```

6. Use continuous testing in development. Press `r` in `quarkus:dev` and only the tests affected by your change run, continuously.

17.10 Flaky Tests

A flaky test is worse than no test: it trains the team to ignore red builds.

Cause 1 — timing. `Thread.sleep` in a test is always wrong.

```
// BAD
service.processAsync(order);
Thread.sleep(1000);
assertThat(repository.count()).isEqualTo(1);

// GOOD
service.processAsync(order);
await().atMost(5, SECONDS)
    .untilAsserted(() -> assertThat(repository.count()).isEqualTo(1));
```

Cause 2 — shared state. Use `@TestTransaction`, or clean up in `@AfterEach`, or generate unique data per test.

Cause 3 — order dependence. If a test only passes when run after another, it depends on shared state. Run the suite in a random order in CI to find these:

```
junit.jupiter.testmethod.order.default=org.junit.jupiter.api.MethodOrderer$Random
```

Cause 4 — real time. Inject a `Clock`. A test asserting on `Instant.now()` will fail at midnight, or across a DST boundary, or on a slow CI agent.

Cause 5 — real randomness. Seed it, or inject a source.

Cause 6 — network. A test that calls a real external service will fail when that service does. Use `WireMock`.

Cause 7 — ports. Never hard-code a port; use `@TestHTTPResource` or a dynamic port.

Quarantine, then fix or delete. A flaky test left in the suite degrades the credibility of every other test. Tag it, exclude it from the gating build, and give it an owner and a deadline. A test nobody will fix should be deleted — an honest gap is better than a false signal.

17.11 Best Practices

1. **Cheapest test type that answers the question.** Plain JUnit first.
2. **Constructor injection** so most classes need no container.
3. **Inject a `Clock`** for any time-dependent logic.
4. **Dev Services, not H2.** Test against the database you deploy against.
5. **`@TestTransaction`** for isolation; explicit cleanup when testing rollback.
6. **Share `@TestProfile` classes** — each distinct one restarts Quarkus.
7. **Write `@QuarkusTest` with HTTP-only assertions**, then subclass with `@QuarkusIntegrationTest` for native.
8. **Builder fixtures** so the relevant difference is visible.
9. **`WireMock` for downstream failure modes** — timeouts, 500s, malformed responses.
10. **Awaitility, never `Thread.sleep`.**
11. **Test the negatives:** wrong role, wrong owner, duplicate event, invalid input, N+1.

12. **Random test order in CI** to surface order dependence.
13. **Tag slow tests** and exclude them from the PR build.
14. **Quarantine flaky tests immediately.**

17.12 Common Mistakes

1. **Everything is a `@QuarkusTest`.** A fifteen-minute suite nobody runs.
2. **A different `@TestProfile` per class.** Each one restarts the application.
3. **H2 instead of the real database.** Dialect and semantic differences produce production-only bugs.
4. **`Thread.sleep`.** Flaky by construction, and slow even when it passes.
5. **Only happy paths.** The 400 is your most frequent response and often untested.
6. **No authorisation-denial tests.** Proving the admin can delete says nothing about the reader.
7. **Asserting on `Instant.now()`.** Fails at midnight or across DST.
8. **`@Inject` in a `@QuarkusIntegrationTest`.** Not available — it is a black-box test.
9. **Testing implementation rather than behaviour.** `verify(repository).save(any())` breaks on every refactor and proves nothing about correctness.
10. **Shared mutable state between tests.** Order-dependent failures that reproduce only in CI.
11. **Ignoring flaky tests.** The whole suite loses credibility.
12. **Never running the native suite.** Reflection failures found by customers instead of CI.

17.13 Interview Questions

Junior

- Q1. How do you test a Quarkus REST endpoint?** `@QuarkusTest` with RestAssured: `given().when().get("/orders").then().statusCode(200)`.
- Q2. What does `@QuarkusTest` do?** Starts the whole application in the same JVM as the test, once for the suite, with CDI injection and mocking available.
- Q3. How do you mock a bean?** `@InjectMock` on the field in a `@QuarkusTest`, or construct the class directly with mock collaborators if it uses constructor injection.
- Q4. What is `@TestTransaction`?** It runs each test method in a transaction that is rolled back afterwards, so tests are isolated and need no cleanup.
- Q5. What are Dev Services in testing?** Quarkus starts real backing services — PostgreSQL, Kafka, Keycloak — in containers for the test run, so tests exercise the real technology rather than a substitute.
- Q6. What is RestAssured?** A fluent library for testing HTTP APIs: `given()` sets up the request, `when()` performs it, `then()` asserts on the response.

Q7. How do you test a secured endpoint? `@TestSecurity(user = "alice", roles = "order-reader")` on the test method.

Mid-level

Q8. `@QuarkusTest` versus `@QuarkusIntegrationTest`? `@QuarkusTest` runs in the same JVM with injection and mocking available. `@QuarkusIntegrationTest` runs the packaged artifact — jar, container or native binary — as a separate process and tests it as a black box, so no injection, no mocking, no `@TestTransaction`. The idiomatic pattern is to write the tests once with HTTP-only assertions and subclass them for the integration run.

Q9. `@QuarkusComponentTest` — what is it for? A minimal CDI container containing only the beans you name. About ten times faster than `@QuarkusTest` while still exercising real CDI — interceptors, producers, config injection. The right choice when the code under test needs the container but not HTTP or a database.

Q10. Why is my test suite slow? Most often too many distinct `@TestProfile` classes, each of which restarts Quarkus. Also: everything written as `@QuarkusTest` when plain JUnit would do, containers not reused between runs, and `Thread.sleep` in asynchronous tests.

Q11. Why test against real PostgreSQL rather than H2? Dialect and semantic differences — `NUMERIC` precision, `TIMESTAMPZ`, identifier case sensitivity, `ON CONFLICT`, `JSONB`, constraint behaviour — mean H2 can pass tests that fail in production. Dev Services makes the real database as convenient as H2, so there is no longer a reason to accept the risk.

Q12. How do you test asynchronous behaviour? Awaitility: `await().atMost(5, SECONDS).untilAsserted(() -> assertThat(...))`. Never `Thread.sleep`, which is both flaky and slow.

Q13. How do you test a REST client? `@InjectMock @RestClient` for unit-level tests of your own logic, and WireMock via a `QuarkusTestResourceLifecycleManager` for integration-level tests — particularly for failure modes such as timeouts, 500s and malformed responses, which are otherwise never exercised.

Q14. What causes flaky tests? Timing assumptions, shared mutable state, order dependence, real clocks, unseeded randomness, real network calls, and hard-coded ports. Running the suite in a random order in CI surfaces the order-dependent ones.

Senior

Q15. Design a testing strategy for a microservice. I would shape it around feedback speed, because a suite that is not run has no value. The bulk is plain JUnit over domain logic, made possible by constructor injection and an injected `Clock` — milliseconds per test, runnable on save. Above that, `@QuarkusComponentTest` for anything needing real CDI but not HTTP or a database. Then a focused `@QuarkusTest` layer covering the HTTP contract, persistence against real PostgreSQL via Dev Services, and the wiring — deliberately kept small, with profiles shared aggressively since each distinct one restarts the application. Downstream services are stubbed with WireMock, specifically so I can test timeouts, 500s and malformed responses, which is where resilience bugs live and which nothing else exercises. Finally a thin `@QuarkusIntegrationTest` layer, written as subclasses of existing HTTP-only tests, run against the native binary nightly rather than per PR. On top of that I would insist on a set of tests that are usually missing: authorisation denials for every protected endpoint, duplicate-message idempotency, query-count assertions to catch N+1,

and a test that liveness survives a database outage. The PR build excludes anything tagged slow and must stay under a few minutes.

Q16. How do you decide what to mock? Mock at the boundaries you do not own, and use the real thing everywhere else. Concretely: mock external HTTP services, because they are slow, unreliable and outside my control; do not mock the database, because Dev Services makes the real one cheap and mocking it means testing my beliefs about SQL rather than SQL. I avoid mocking my own domain objects entirely — if a test needs a mock of a value object, the design is wrong. The failure mode I watch for is over-mocking, where a test asserts `verify(repository).save(any())` and passes regardless of whether the saved object was correct; that test breaks on every refactor and catches nothing. My rule of thumb is that a test should assert on observable behaviour — the response, the persisted state, the published event — and use mocks only to control inputs and to make the unavailable available.

Q17. Your team's test suite takes 20 minutes and people are pushing without running it. What do you do? First measure where the time goes, because the intuition is usually wrong — I would get per-class durations and a count of Quarkus restarts, since in my experience the dominant cost is almost always profile churn rather than the tests themselves. Consolidating twelve profiles into two typically removes more time than any amount of test optimisation. Then I would split the suite: a fast gate under three minutes that runs on every push, and a fuller suite tagged slow that runs on merge and nightly, including the native integration tests. That restores the feedback loop immediately while nothing is deleted. In parallel, I would look at how many `@QuarkusTest` classes are testing pure logic that could be plain JUnit, and migrate the worst offenders — that is a gradual improvement, not a project. Container reuse and parallel execution are worth enabling but usually give less than the profile fix. The thing I would treat as non-negotiable is the three-minute gate, because the behaviour we are trying to change is people skipping tests, and that only changes when running them is cheap.

Q18. How do you test that your service handles a downstream outage correctly? WireMock, driven from a test resource, because this is precisely the case where mocking the client is insufficient — I want the real HTTP stack, the real timeout configuration and the real circuit breaker in the path. I would write a test per failure mode: a fixed delay longer than the configured read timeout, a 500, a 429 with `Retry-After`, a connection reset, and a malformed body. For each, the assertion is about the behaviour the user sees — a 202 with a degraded status, or a 503 with `Retry-After` — and about the side effects, such as the request being queued for later. I would also assert that the circuit breaker opens after the configured volume of failures and that subsequent calls fail fast rather than waiting for a timeout, because a breaker that is configured but never verified is a common source of false confidence. These tests are slow, so they carry a tag and run outside the PR gate. The reason I insist on them is that the resilience configuration — timeouts, retry counts, breaker thresholds — is written once and then never exercised until an incident, and by then it is usually wrong.

Principal

Q19. Standardise testing practice across 40 services. What do you mandate? I would mandate outcomes and provide tooling, rather than mandating techniques. The outcomes: a PR gate under a fixed duration, so feedback stays fast and people do not learn to bypass it; a native integration run before release, so reflection failures are found by CI rather than customers; and a required set of negative tests — an authorisation denial per protected endpoint, idempotency for every message consumer, and a query-count assertion on every list endpoint — because those

three gaps account for a disproportionate share of production incidents and are individually easy to skip. The tooling matters more than the mandate: a shared test library with the `@TestSecurity` helpers, a WireMock test resource, an Awaitility configuration, a query-count assertion helper and a forbidden-metric-tag assertion, so the required tests are three lines rather than thirty. I would also standardise Dev Services image versions so every service tests against the same PostgreSQL as production. What I would not mandate is coverage percentage, because it measures the wrong thing and is reliably gamed — a service can have ninety per cent coverage and no authorisation tests. Instead I would report suite duration and flake rate on a platform dashboard, since those are the two numbers that predict whether the tests are actually protecting anything.

Q20. Make the case against high test coverage targets. Coverage measures which lines executed, not whether anything was verified — a test that calls every method and asserts nothing scores identically to one that checks every edge case. So a target converts into pressure to write tests that raise the number, and the cheapest way to do that is trivial tests over getters, generated code and straight-line delegation, which cost maintenance forever and catch nothing. Worse, it distorts effort away from the places where coverage is hard but valuable: error paths, concurrency, authorisation, and the interaction between components. I have seen codebases at ninety per cent coverage with no test that a non-admin is denied, and codebases at fifty-five per cent where every risky path is thoroughly tested. There is a second-order harm too — a high target makes refactoring expensive, because a large body of implementation-coupled tests must be rewritten, which discourages exactly the design improvement you want. What I would use instead is risk-based: identify the paths where a defect is expensive — money, data loss, security, data corruption — and require tests there specifically, with review checking that the assertions are meaningful. Coverage is useful as a *diagnostic*, in the form of a report that shows an untested package nobody noticed, but the moment it becomes a target it stops being a measurement. If pressed for a number, I would rather set a ceiling on suite duration and a ceiling on flake rate, because both directly protect the property that matters — that the tests get run and are believed.

17.14 Cheat Sheet

Test types

```
class T { } // plain JUnit - milliseconds
@QuarkusComponentTest class T { } // minimal CDI - ~200ms
@QuarkusTest class T { } // full app in-JVM - starts once
@QuarkusIntegrationTest class T { } // black box on the artifact / native
```

Injection and mocking

```
@Inject Service service;
@InjectMock Collaborator collaborator;
@InjectSpy AuditService audit;
@InjectMock @RestClient PaymentClient payments;
@TestHTTPResource("/orders") URL url;
QuarkusMock.installMockForType(mock, Type.class);
```

Isolation

```

@TestTransaction                // rolled back per test
@TestProfile(MyProfile.class)    // RESTARTS Quarkus - share these
@QuarkusTestResource(WireMockResource.class)

```

Security

```

@TestSecurity(user = "alice", roles = {"order-reader"})
@JwtSecurity(claims = @Claim(key = "tenant_id", value = "acme"))

```

RestAssured

```

given().contentType(JSON).header("X-Tenant", "acme").body(payload)
.when().post("/orders")
.then().statusCode(201)
     .header("Location", matchesPattern(".*orders/\\d+"))
     .body("status", is("PENDING"))
     .body("items", hasSize(1));

```

Async

```

await().atMost(5, SECONDS).untilAsserted(() -> assertThat(repo.count()).isEqualTo(1));
// NEVER Thread.sleep

```

Dev Services

```

%test.quarkus.datasource.devservices.image-name=postgres:16-alpine
%test.quarkus.datasource.devservices.reuse=true
%test.quarkus.kafka.devservices.provider=redpanda
%test.quarkus.keycloak.devservices.realm-path=acme-realm.json

```

Native

```

@QuarkusIntegrationTest
class OrderResourceIT extends OrderResourceTest { }

```

```
./mvnw verify -Dnative
```

Speed levers

1. fewer `@TestProfile` classes (each restarts Quarkus)
2. push tests to plain JUnit
3. container reuse
4. parallel execution
5. tag and exclude slow tests from the PR gate

17.15 Summary

The organising principle is cost. Plain JUnit runs in milliseconds and should carry most of your logic; `@QuarkusComponentTest` covers what needs real CDI; `@QuarkusTest` covers wiring, HTTP contracts and persistence; `@QuarkusIntegrationTest` validates the artifact you actually ship. Each step up costs roughly an order of magnitude, so choosing the cheapest type that answers the question is not fussiness — it is what keeps the suite fast enough to be run.

Two Quarkus-specific facts have outsized effects. **Each distinct `@TestProfile` restarts the application**, which is the usual reason a suite takes fifteen minutes; sharing profiles is the highest-leverage optimisation available. And **Dev Services makes testing against real PostgreSQL, Kafka and Keycloak as convenient as using substitutes**, which removes the last justification for H2 and for the class of bug it hides.

The gaps that matter are predictable: error responses, authorisation denials, idempotency under duplicate delivery, N+1 regressions, and the native binary. Each is easy to test once and easy to skip forever. The query-count assertion and the authorisation-denial test in particular catch categories of regression that code review reliably misses.

Finally, treat flakiness as a defect in its own right. A flaky test does not merely fail to protect its own code path — it teaches the team to disbelieve every other test in the suite.

Chapter 18 turns to **native image**: how GraalVM’s closed-world analysis works, what breaks, how to fix it, and how to decide whether native is the right choice for a given service.

17.16 References

- Testing your application — <https://quarkus.io/guides/getting-started-testing>
- Continuous testing — <https://quarkus.io/guides/continuous-testing>
- Dev Services — <https://quarkus.io/guides/dev-services>
- Testing security — <https://quarkus.io/guides/security-testing>
- Testcontainers — <https://testcontainers.com/>
- RestAssured — <https://rest-assured.io/>
- Awaitility — <https://github.com/awaitility/awaitility>
- WireMock — <https://wiremock.org/>

Native Image

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · GraalVM for JDK 21 / Mandrel 23.1 · `native-image`

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what `native-image` actually does — points-to analysis, the closed-world assumption, the image heap.
2. Describe the build pipeline and why it needs 6–12 GB of memory.
3. Predict which Java features break under closed-world analysis, and why.
4. Register reflection, resources, proxies and serialisation correctly.
5. Use the tracing agent to generate configuration for uncooperative libraries.
6. Understand **static initialisation** at image build time and the bugs it causes.
7. Optimise binary size, startup and runtime performance, including PGO and G1.
8. Debug a native binary despite the absent JVM tooling.
9. Decide, per service, whether native is the right choice — including when it is not.
10. Answer 20 interview questions on native compilation.

18.1 What `native-image` Does

A conventional JVM application loads classes on demand, resolves symbols at runtime, and compiles hot methods with the JIT. GraalVM Native Image does the opposite: it performs **ahead-of-time compilation with whole-programme static analysis** and produces a self-contained executable.

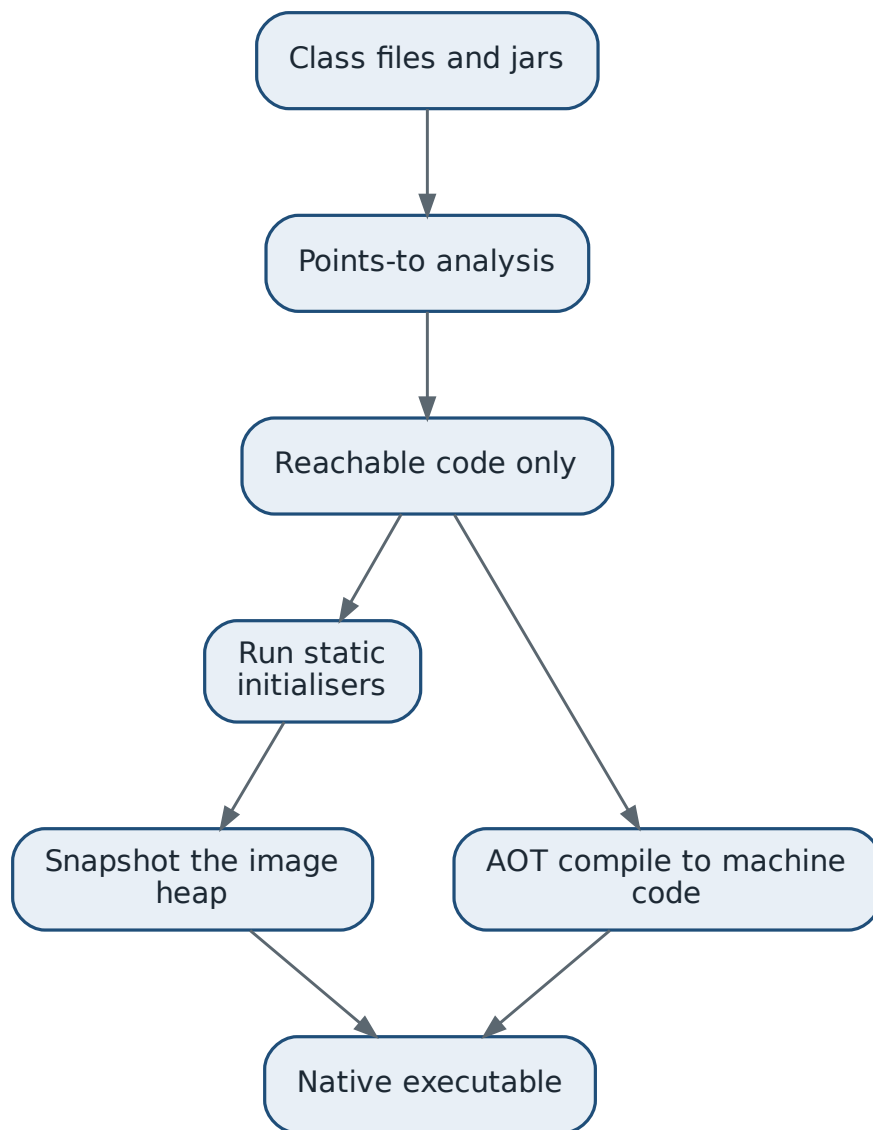


Figure 18.1 — The build determines reachability, executes static initialisers, snapshots the resulting heap, and compiles what remains.

Points-to analysis. Starting from entry points — `main`, registered reflection targets, JNI — the analyser computes every method that could possibly be invoked and every type that could flow to each variable. Everything unreachable is **deleted**, which is where the small binary and small heap come from.

Static initialisation at build time. Class initialisers run during the build, and the resulting objects are serialised into the executable’s **image heap**. At runtime they exist before `main` is

entered — no class loading, no initialisation. This is the largest contributor to sub-millisecond startup.

AOT compilation. The reachable code is compiled to machine code. There is no bytecode at runtime and no JIT, which is why peak throughput is typically 10–30 % below a warmed-up JVM.

A substituted runtime. The executable embeds SubstrateVM — a garbage collector, thread scheduling and memory management — rather than HotSpot.

18.1.1 The closed-world assumption

Everything follows from this: **the set of classes, methods and resources is fixed at build time.** Nothing can be loaded, generated or discovered later.

That single constraint explains every native-mode difficulty in the rest of the chapter.

18.2 Building

```
# Requires a local GraalVM or Mandrel with native-image installed
./mvnw package -Dnative

# Preferred in CI: builds inside a Mandrel builder container
./mvnw package -Dnative -Dquarkus.native.container-build=true

./target/orders-service-1.0.0-SNAPSHOT-runner
```

```
quarkus.native.container-build=true
quarkus.native.builder-image=quay.io/quarkus/ubi-quarkus-mandrel-builder-image:jdk-21
quarkus.native.native-image-xmx=8g
quarkus.native.additional-build-args=\
  --initialize-at-run-time=com.acme.orders.LegacyBootstrap,\
  -H:+ReportExceptionStackTraces,\
  -H:+UnlockExperimentalVMOptions

# Keep the intermediate sources for diagnosis
quarkus.native.debug.enabled=true
quarkus.native.enable-reports=true
```

18.2.1 GraalVM or Mandrel?

Mandrel is Red Hat’s distribution of GraalVM Native Image, stripped to just the native-image capability — no Truffle, no polyglot, no GraalVM JIT. It is what Quarkus is tested against and what the builder images use. Use Mandrel unless you specifically need polyglot support.

18.2.2 Why the build is slow and memory-hungry

Points-to analysis is a whole-programme fixed-point computation: it iterates over the type flow graph until nothing changes. Its memory cost scales with the size of the reachable graph, so 6–12 GB and 2–6 minutes are normal for a real service.

This has a practical consequence worth planning for: **a native build will not fit on a default CI runner**. Budget a larger agent, run native builds on a separate pipeline stage, and do not put them on the per-PR path.

```
# Trade build time for memory, or the reverse
quarkus.native.native-image-xmx=10g
quarkus.native.additional-build-args=-J-XX:ActiveProcessorCount=4
```

18.3 What Breaks, and Why

18.3.1 Reflection

The analyser cannot see through `Class.forName(computedString)` or `getMethod(computedName)`, so the target is unreachable and gets deleted.

```
// Fails at runtime: ClassNotFoundException
Class<?> type = Class.forName(config.handlerClassName());
```

```
// Fix: register explicitly
@RegisterForReflection(targets = {LegacyPayload.class, LegacyResponse.class})
public class ReflectionConfiguration { }

// Register a whole hierarchy
@RegisterForReflection(targets = OrderEvent.class, registerFullHierarchy = true)
public class EventReflection { }

// From a library you do not control, with fine-grained control
@RegisterForReflection(
    targets = ThirdPartyDto.class,
    methods = true,
    fields = true,
    ignoreNested = false,
    serialization = true)
public class ThirdPartyReflection { }
```

Or declaratively, for classes you cannot annotate:

```
// src/main/resources/META-INF/native-image/com.acme/orders/reflect-config.json
[
  {
    "name": "com.thirdparty.LegacyHandler",
    "allDeclaredConstructors": true,
    "allPublicMethods": true,
    "allDeclaredFields": true
  }
]
```

In practice you rarely write these. A Quarkus extension’s job is largely to generate exactly this configuration for the library it wraps. You hit reflection problems when you use a plain Maven dependency with no extension — which is the strongest practical argument for preferring extensions.

18.3.2 Resources

Classpath resources are excluded unless declared:

```
quarkus.native.resources.includes=templates/**,rules/*.json,i18n/*.properties
quarkus.native.resources.excludes=**/*.test.json
```

```
// resource-config.json
{
  "resources": {
    "includes": [{"pattern": "\\Qrules/pricing.json\\E"}],
    "excludes": [{"pattern": ".*\\.dev\\.json$"}]
  },
  "bundles": [{"name": "messages.ValidationMessages"}]
}
```

The symptom is a `null` from `getResourceAsStream` that works perfectly in JVM mode — and it usually appears on a code path exercised only in production.

18.3.3 Dynamic proxies

```
quarkus.native.additional-build-args=\
  -H:DynamicProxyConfigurationResources=proxy-config.json
```

```
[
  ["com.acme.orders.LegacyService", "java.io.Serializable"]
]
```

Quarkus generates these for REST clients and CDI proxies automatically. Hand-rolled `Proxy.newProxyInstance` needs declaring.

18.3.4 Serialisation

```
@RegisterForReflection(targets = SessionState.class, serialization = true)
public class SerializationConfig { }
```

Java serialisation is heavily reflective and needs explicit registration. This is also a good moment to question whether you should be using it at all — JSON or a schema-based format is almost always a better choice, and native mode surfacing the problem is a useful prompt.

18.3.5 Service loaders

`ServiceLoader` works: the build scans `META-INF/services` and registers the implementations. It fails when the service file lives in a JAR that is excluded, or when providers are registered programmatically.

18.3.6 JNI and native libraries

```
quarkus.native.additional-build-args=-H:JNIConfigurationResources=jni-config.json
```

Rare in typical services, but it is why some cryptography and image-processing libraries need work.

18.4 Static Initialisation — The Subtle One

This causes the most confusing native-mode bugs, and it is a favourite senior interview question.

By default, GraalVM runs class initialisers **at build time** and snapshots the result. So this:

```
public class RequestIds {
    private static final Random RANDOM = new Random();           // BUG in native
    private static final Instant BUILT_AT = Instant.now();       // BUG in native
    private static final Connection CONN = openConnection();    // BUG in native

    public static long next() { return RANDOM.nextLong(); }
}
```

...produces an executable in which:

- **RANDOM** was seeded **on the build machine**, so every replica of the image generates the **identical sequence** of “random” numbers. For a session id or a token, that is a security vulnerability.
- **BUILT_AT** is the build timestamp, not the start time.
- **CONN** is a socket handle to a connection closed weeks ago on a host that may not be routable from production.

GraalVM catches some of these:

```
Error: Detected an instance of Random/SplittableRandom class in the image heap.
Instances created during image generation have cached seed values and don't behave
as expected. To see how this object got instantiated use
↪ --trace-object-instantiation=java.util.Random
```

```
Error: Unsupported feature: No instances of java.net.Socket are allowed in the image heap
```

The fix — defer initialisation to runtime:

```
quarkus.native.additional-build-args=\
--initialize-at-run-time=com.acme.orders.RequestIds,\
--initialize-at-run-time=com.thirdparty.CryptoBootstrap
```

Or restructure so there is no static state to snapshot:

```
@ApplicationScoped
public class RequestIds {
    private final SecureRandom random = new SecureRandom();      // per instance, at runtime
    public long next() { return random.nextLong(); }
}
```

The CDI version is better regardless of native mode: it is testable, injectable and has no hidden global state.

This is the concept to be able to explain in an interview. The build error about **Random** is not GraalVM being awkward — it is catching a genuine correctness bug at build time that would otherwise be a subtle security flaw in production. Quarkus’s static-init versus runtime-init split (Chapter 1) exists precisely to manage this boundary for extensions.

18.5 The Tracing Agent

For a library with no extension and no documentation, generate the configuration by observing a JVM run.

```
# 1. Build the JVM version
./mvnw package

# 2. Run it under the agent, exercising every code path
java -agentlib:native-image-agent=config-output-dir=src/main/resources/META-INF/native-image \
-jar target/quarkus-app/quarkus-run.jar

# 3. Drive the application - run the integration test suite against it
./mvnw verify -Dquarkus.http.test-port=8080

# 4. The agent writes reflect-config.json, resource-config.json,
#    proxy-config.json, serialization-config.json, jni-config.json

# 5. Rebuild natively
./mvnw package -Dnative

# Merge across several runs rather than overwriting
java -agentlib:native-image-agent=config-merge-dir=src/main/resources/META-INF/native-image \
-jar target/quarkus-app/quarkus-run.jar
```

The agent records what happened, not what could happen. Configuration is only as complete as your exercise of the application, so a code path you did not hit produces a runtime failure in production. Drive it with your integration test suite rather than by clicking around, and treat the output as a starting point to be reviewed — the agent is generous and will register things you do not need, inflating the binary.

18.6 Optimising

18.6.1 Binary size

```
# Strip debug symbols (default in a release build)
quarkus.native.debug.enabled=false

# A distroless or micro base image for the container
quarkus.native.container-runtime=docker
```

```
# Dockerfile.native-micro - the smallest practical runtime image
FROM quay.io/quarkus/quarkus-micro-image:2.0
WORKDIR /work/
COPY --chown=1001:root target/*-runner /work/application
EXPOSE 8080
USER 1001
ENTRYPOINT ["/application", "-Dquarkus.http.host=0.0.0.0"]
```


A typical native `orders-service` binary is 45–90 MB; the container image on `quarkus-micro-image` is around 50–100 MB, against 200–400 MB for a JVM image. The unused extension is the largest lever: every extension you remove removes its reachable graph.

18.6.2 Runtime performance

```
# G1 instead of SerialGC - worth it above ~1GB heap or with high allocation rates
quarkus.native.additional-build-args=-gc=G1
```

```
# Profile-guided optimisation (GraalVM Enterprise / Oracle GraalVM)
quarkus.native.additional-build-args=-pgo-instrument
# run a representative workload, then:
quarkus.native.additional-build-args=-pgo=default.iprof
```

PGO typically recovers a large part of the throughput gap with the JIT — often bringing native within 10 % of a warmed JVM — at the cost of a two-stage build and a representative workload. It is the answer to “native is slower at peak” when peak matters.

```
# Memory limits inside the executable
quarkus.native.additional-build-args=-R:MaxHeapSize=256m
```

18.6.3 Startup

Native startup is already tens of milliseconds. If it is not, the time is in **runtime init**, not the framework — Chapter 1 §1.17 Q20 covers the diagnosis. The usual culprits are eager connection pools, `@Startup` beans doing real work, DNS resolution, and TLS handshakes.

```
# Do not open the whole pool at boot
quarkus.datasource.jdbc.initial-size=1
quarkus.datasource.jdbc.min-size=1
```

18.7 Debugging

The JVM tooling is largely unavailable, which is the real operational cost of native.

	JVM	Native
Debugger (JDWP)	yes	no — gdb only
Heap dump	yes	limited
JFR	yes	partial
JMX	yes	no
async-profiler	yes	limited (perf)
Thread dump	<code>jstack</code>	<code>SIGQUIT</code> , reduced detail

```
# Build with debug info for gdb and perf
quarkus.native.debug.enabled=true
quarkus.native.additional-build-args=-H:+SourceLevelDebug
```

```
# JFR, partially supported
quarkus.native.monitoring=jfr,heapdump
```

```
./application -XX:+FlightRecorder -XX:StartFlightRecording=duration=60s,filename=rec.jfr
gdb ./target/orders-service-runner
```

The practical strategy: debug in JVM mode. Reproduce the problem with `./mvnw quarkus:dev`, fix it there, then verify natively. This is why the “develop and test in JVM, build native in a release pipeline” workflow is the standard recommendation — you keep the full toolchain for the activity that needs it.

For problems that only manifest natively, `-H:+ReportExceptionStackTraces` and the build reports are the main instruments:

```
quarkus.native.enable-reports=true
# -> target/reports/: reachability, call trees, used methods, image size breakdown
```

The **used-methods report** answers “why is this class in my image?”, which is the question you have when the binary is unexpectedly large.

18.8 Testing Natively

```
@QuarkusTest
class OrderResourceTest {
    // HTTP-only assertions, per Chapter 17
    @Test void createsAnOrder() { ... }
}

@QuarkusIntegrationTest
class OrderResourceIT extends OrderResourceTest { }
```

```
./mvnw verify -Dnative
```

This is the practice that makes native viable. Reflection failures, missing resources and serialisation problems all surface as ordinary test failures instead of production incidents.

```
# CI: native on a schedule, not on every PR
name: native
on:
  schedule: [{cron: "0 2 * * *"}]
  push: {branches: [main]}
jobs:
  native:
    runs-on: ubuntu-latest-8-cores # needs the memory
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: {distribution: temurin, java-version: 21}
      - run: ./mvnw verify -Dnative -Dquarkus.native.container-build=true
```

18.9 Deciding

18.9.1 The comparison

	JVM (fast-jar)	Native
Startup	0.5–1.5 s	0.01–0.05 s
RSS at idle	90–180 MB	20–50 MB
Peak throughput	highest	10–30 % lower (less with PGO)
Build time	10–40 s	2–6 min
Build memory	~1 GB	6–12 GB
Container image	200–400 MB	50–100 MB
Debugging	full	gdb-level
Observability	JFR, JMX, profilers	partial JFR, no JMX
Library compatibility	universal	needs registration without an extension

18.9.2 Choose native when

- **Cold start is user-visible** — serverless, Knative, scale-to-zero, Lambda.
- **Memory is your dominant cost** — many replicas of a small service.
- **The workload is short-lived** — CLI tools, batch jobs, operators, sidecars. A JIT never gets to warm up anyway, so native’s throughput deficit does not apply.
- **Security posture benefits** from a smaller attack surface and no dynamic class loading.
- **Edge or constrained environments** with hard memory limits.

18.9.3 Choose JVM when

- **Throughput per instance is the metric** and instances are long-lived.
- **CI budget cannot absorb multi-minute builds** on every change.
- **A critical dependency has poor native support** and no extension.
- **Your team depends on JVM production tooling** — live profilers, heap dumps, JMX.

The mature answer is “per service, and default to JVM”. Develop and test in JVM mode, run the native build in a nightly or pre-release stage, and promote a service to native when its measurements justify it. Committing the whole estate to native on day one is a common and expensive mistake — you take on the build cost and the debugging cost for every service, including the ones that gain nothing.

And note the possibility of moving *back*: if a service turns out to be throughput-bound, switching it to JVM mode is a build-flag change, not a rewrite. Quarkus is explicitly designed so the same codebase works both ways.

18.10 Best Practices

1. **Prefer extensions to plain dependencies.** Native registration is much of what an extension provides.
2. **Run `@QuarkusIntegrationTest` against the native binary** — nightly, not per PR.
3. **Never hold `Random`, sockets, threads or file handles in static fields.** Restructure into CDI beans.

4. **Build in a container** (`container-build=true`) for reproducibility across machines and CI.
5. **Drive the tracing agent with your integration suite**, not manually, and review its output.
6. **Enable build reports** when the binary is unexpectedly large.
7. **Debug in JVM mode**; verify natively.
8. **Consider PGO** when peak throughput matters and you have a representative workload.
9. **Size CI agents for 8–12 GB**. The build will not fit on a default runner.
10. **quarkus-micro-image** or **distroless** as the runtime base image.
11. **Keep initial-size** small so startup is not dominated by pool creation.
12. **Decide per service**. Measure, do not assume.

18.11 Common Mistakes

1. **Assuming JVM behaviour transfers**. Reflection, resources, proxies and serialisation all need registration.
2. **Random** or **SecureRandom** in a static field. Every replica produces the same sequence. A security bug, not a performance one.
3. A resource that loads in JVM mode and returns **null** natively. Add it to `resources.includes`.
4. **Trusting the tracing agent's output blindly**. It records only what you exercised, and registers more than you need.
5. **Native builds on every PR**. Multi-minute builds and 8 GB agents make the pipeline unusable.
6. **Adopting native everywhere at once**. You pay the build and debugging cost for services that gain nothing.
7. **Expecting the same peak throughput**. Without PGO, expect 10–30 % lower.
8. **Not testing natively at all**. Reflection failures reach customers.
9. **Expecting JMX or a debugger in production**. Plan observability around metrics, traces and logs.
10. **Ignoring the image-heap error messages**. They are catching real bugs.
11. **A static initialiser that reads configuration**. The build machine's environment gets baked in.
12. **--initialize-at-build-time** used broadly to “fix” errors. It moves more code into build-time initialisation and creates the very class of bug you were avoiding.

18.12 Interview Questions

Junior

Q1. What is a native image? A standalone executable produced by GraalVM's ahead-of-time compiler, containing your application and a minimal runtime, with no JVM required.

Q2. How do you build one in Quarkus? `./mvnw package -Dnative`, or `-Dnative -Dquarkus.native.container-build=true` to build inside a Mandrel container without a local GraalVM.

Q3. What are the benefits? Startup in tens of milliseconds instead of a second, RSS of 20–50 MB instead of 100–200, a much smaller container image, and no dynamic class loading.

Q4. What are the drawbacks? Slow, memory-hungry builds; roughly 10–30 % lower peak throughput without PGO; limited debugging and observability tooling; and reflection, resources and proxies must be registered at build time.

Q5. What is Mandrel? Red Hat’s distribution of GraalVM Native Image, stripped to the native-image capability. It is what Quarkus is built and tested against.

Q6. What is `@RegisterForReflection` for? Telling the build to retain a class and its metadata when it is reached only reflectively, since the static analysis cannot see such references.

Mid-level

Q7. What is the closed-world assumption? The set of classes, methods and resources is fixed at build time. Nothing can be loaded, generated or discovered at runtime. Everything the analysis cannot prove reachable is deleted, which is what produces the small binary — and what breaks reflection, dynamic proxies and unregistered resources.

Q8. Why does the build need so much memory? Points-to analysis is a whole-programme fixed-point computation over the type flow graph, iterating until nothing changes. Its memory cost scales with the reachable graph, so 6–12 GB is normal.

Q9. Why can you not have a `Random` in a static field? Static initialisers run at build time and their results are snapshotted into the image heap, so the `Random` is seeded on the build machine. Every container started from that image produces the identical “random” sequence — a security bug if it is used for ids or tokens. GraalVM detects this and fails the build.

Q10. How do you fix a missing resource in native mode? `quarkus.native.resources.includes=path/**`, or a `resource-config.json`. The symptom is a `null` from `getResourceAsStream` that works fine in JVM mode.

Q11. What is the tracing agent? A JVM agent that records reflective access, resource loading, proxy creation and serialisation during a run, and writes the corresponding configuration files. It records only what you exercised, so it must be driven by a thorough workload — ideally the integration test suite — and its output reviewed.

Q12. How do you test a native binary? `@QuarkusIntegrationTest` subclassing an existing `@QuarkusTest` written with HTTP-only assertions, run with `./mvnw verify -Dnative`. The same tests then validate the artifact you actually ship.

Q13. Why is peak throughput lower? There is no JIT. The JVM’s C2 compiler profiles a running application and produces highly-optimised code from real behaviour; AOT compilation has only static information. Profile-guided optimisation closes much of the gap by feeding a recorded profile back into the build.

Senior

Q14. Walk me through the native build pipeline. Quarkus first runs augmentation, producing generated bytecode and the reflection, resource and proxy registrations its extensions know are needed. `native-image` then performs points-to analysis from the entry points — `main`, registered reflection targets, JNI — computing every reachable method and every type that can flow

to each variable, and discarding the rest. Next it executes static initialisers for classes designated build-time-initialised and snapshots the resulting object graph into the image heap, which is why those objects exist before `main` runs. It then AOT-compiles the reachable methods to machine code, links in SubstrateVM — the GC, thread scheduling and memory management — and emits a single executable. The analysis is the expensive part: it is a fixed-point computation whose memory cost scales with the reachable graph, hence the multi-gigabyte requirement.

Q15. A library works in JVM mode and fails natively. How do you diagnose it? The failure type usually names the cause, so I read it first: `ClassNotFoundException` or `NoSuchMethodException` means reflection was not registered; a `null` from `getResourceAsStream` means a missing resource; a `ClassCastException` on a proxy means a missing dynamic proxy declaration. If the message is unclear I rebuild with `-H:+ReportExceptionStackTraces` and enable the build reports, which tell me whether the class is in the image at all. Then the fix depends on ownership: for my own code, `@RegisterForReflection`; for a library, the tracing agent driven by the integration test suite, with the generated configuration reviewed rather than committed wholesale, because the agent over-registers. If the library is central and this is a recurring cost, the right answer may be to look for a Quarkus extension, or to write one, since generating exactly this configuration is much of what an extension is for. And I would add an integration test covering the failing path, because the reason it reached me is that no test exercised it natively.

Q16. When would you not use native? When peak throughput per instance is the metric and instances are long-lived, because there is no JIT and the gap is real without PGO. When a critical dependency has poor native support and no extension, and the registration burden is ongoing rather than one-off. When the team's operational practice depends on JVM tooling — live profilers, heap dumps, JMX — and losing it would materially slow incident response. When the CI budget cannot absorb multi-minute, multi-gigabyte builds, which is a real constraint for a team pushing frequently. And for a long-running monolith on a large machine, where startup time and per-replica memory simply are not costs anyone is paying. The general test is whether the thing native optimises — cold start and per-instance memory — is actually on your critical path. If the answer is no, you are taking on build and debugging costs for no return.

Q17. How do you decide between JVM and native across an estate? Per service, defaulting to JVM, and driven by measurement rather than policy. I would rank services by replica count times memory request, which identifies where native's memory saving is worth real money, and separately by startup sensitivity — anything scale-to-zero, burst-scaled, or with a slow rollout. Those two lists give the candidates. For each candidate I would run the native build, verify with the existing integration suite, and measure both startup and throughput under a representative load, because the throughput deficit matters for some and not others. Services that are short-lived by nature — batch jobs, operators, CLI tools — I would default to native without much analysis, since a JIT never warms up for them anyway. Crucially, I would keep both paths working: the same codebase builds either way, so a service that turns out to be throughput-bound can be moved back with a build flag. I would also be explicit that the decision has an operational cost beyond the build — the on-call team loses JMX and live profiling — so it needs their agreement, not just an architectural one.

Principal

Q18. Your organisation wants to move 40 services to native to cut cloud costs. Evaluate. I would want the number before the plan. The saving is roughly the memory reduction times replica count times price, and it is easy to compute from current requests — if it is not

a compelling figure, the programme should not happen, and saying so is the useful contribution. Assuming it is compelling, I would still not migrate forty services, because the benefit is very unevenly distributed: a handful of high-replica-count services will account for most of the saving, and a long tail will produce almost none while incurring the full build and operational cost. So I would segment, migrate the top decile, measure the realised saving against the projection, and re-decide. The costs to put on the other side of the ledger are concrete: CI agents sized at 8–12 GB and multi-minute builds, which affects pipeline throughput for everyone; the loss of JMX and live profiling, which changes incident response and needs the on-call team’s agreement; and a per-service integration-test burden, since a native build that is not tested natively is a liability. There is also a subtler risk — teams treating native as a target rather than a tool, and accepting a throughput regression on a service where throughput is what matters. I would counter that by requiring a measured before-and-after per service and an explicit decision, and by making it easy and blameless to move a service back to JVM mode, which is a build-flag change. The likely honest outcome is that ten services go native, the saving is most of what was projected, and thirty stay on the JVM — and that is a success, not a partial failure.

Q19. Argue that native image is over-hyped. The benefits are real but narrower than the discourse suggests, and the costs are systematically under-counted. Startup time matters enormously for serverless and scale-to-zero, and very little for a service that runs continuously behind a load balancer — where a one-second JVM start is invisible. Memory matters when you run many replicas, and a Quarkus JVM service at 120 MB is already far below what people are usually comparing against. Against that, you give up the JIT, which is one of the most sophisticated pieces of engineering in the industry and which for a long-running service will beat AOT code on real workloads unless you invest in PGO — and PGO requires a representative profile, which most teams do not have. You give up the debugger, JMX, heap dumps and live profilers, which is a much bigger operational loss than it sounds until the first production incident where you would have attached a profiler. You add multi-minute, multi-gigabyte builds to a pipeline whose speed determines how often people integrate. And you add a class of bug — build-time static initialisation — that has no JVM analogue and that engineers have no intuition for. My honest position is that native is excellent for a specific and identifiable set of workloads and unnecessary for most, and that the memory saving people cite is often being compared against an unoptimised baseline rather than against a tuned JVM deployment. The strongest argument for Quarkus is not native compilation; it is that build-time augmentation makes the JVM version fast and small enough that native is optional.

Q20. How do you make native a sustainable practice rather than a one-off project? The failure mode is that someone gets a service building natively, it works, and then it silently rots — a dependency is added, a reflective path is introduced, and the next native build fails or, worse, produces a binary that fails only on an untested path. So sustainability is mostly about making the feedback continuous. Concretely: a nightly native build and integration test for every service that ships natively, gating the release rather than the PR, so breakage is caught within a day and attributed to a small change set. A shared CI template with correctly-sized agents and container builds, so no team is solving the memory problem individually. A platform rule that new dependencies should prefer extensions, with the native-registration burden made visible at review time rather than discovered at build time. And a documented, blameless path back to JVM mode, because a team that feels trapped will hide problems. I would also invest in the integration test suite specifically, since it is the only mechanism that distinguishes “the build succeeded” from “the binary works” — a native build that compiles but fails at runtime on an unexercised path is the characteristic failure, and it is entirely preventable. Finally, I would keep a small number of

people genuinely fluent in the failure modes, because when a native-only bug appears, the difference between an hour and a week is whether anyone recognises the shape of it.

18.13 Cheat Sheet

Building

```
./mvnw package -Dnative
./mvnw package -Dnative -Dquarkus.native.container-build=true
./mvnw verify -Dnative # integration tests against the binary
./target/app-runner
```

```
quarkus.native.container-build=true
quarkus.native.builder-image=quay.io/quarkus/ubi-quarkus-mandrel-builder-image:jdk-21
quarkus.native.native-image-xmx=8g
quarkus.native.enable-reports=true
quarkus.native.debug.enabled=true
quarkus.native.resources.includes=templates/**,rules/*.json
quarkus.native.additional-build-args=- -gc=G1, -H:+ReportExceptionStackTraces
```

Registration

```
@RegisterForReflection(targets = {A.class, B.class})
@RegisterForReflection(targets = C.class, registerFullHierarchy = true)
@RegisterForReflection(targets = D.class, serialization = true)
```

```
src/main/resources/META-INF/native-image/<group>/<artifact>/
├─ reflect-config.json
├─ resource-config.json
├─ proxy-config.json
├─ serialization-config.json
└─ native-image.properties
```

Static init

```
quarkus.native.additional-build-args=- -initialize-at-run-time=com.acme.Foo
```

Never in a static field: Random, SecureRandom, Socket, Thread, FileDescriptor,
anything reading the environment

Tracing agent

```
java -agentlib:native-image-agent=config-output-dir=src/main/resources/META-INF/native-image \
-jar target/quarkus-app/quarkus-run.jar
```

Decision

```
native -> serverless, scale-to-zero, many small replicas, short-lived jobs
JVM -> throughput-bound, long-lived, needs JVM tooling, tight CI budget
default -> JVM; promote per service on measurement
```


18.14 Summary

Native Image performs whole-programme points-to analysis, deletes everything unreachable, runs static initialisers at build time and snapshots the result, then compiles ahead of time. Every benefit and every difficulty follows from one property: **the closed world**. The set of classes, methods and resources is fixed when the build ends.

That is why reflection, dynamic proxies, resources and serialisation must be registered, and it is why extensions matter — generating that registration is much of what an extension does, and a plain dependency without one is where the work appears.

The subtle failure to understand properly is **build-time static initialisation**. A `Random` in a static field is seeded on the build machine, so every replica produces the same sequence; a socket becomes a dead handle baked into the binary. GraalVM's error messages here are catching genuine correctness bugs, not being awkward, and the right fix is usually to restructure the static state into a CDI bean rather than to reach for `--initialize-at-run-time`.

Operationally, the practice that makes native viable is running `@QuarkusIntegrationTest` against the binary. Reflection and resource failures then appear as ordinary test failures rather than production incidents.

And the decision is per service, defaulting to JVM. Native is excellent where cold start is user-visible or where per-replica memory is the dominant cost, and unnecessary for a long-running throughput-bound service that gains nothing and gives up the JIT and the debugger. Quarkus is designed so the same codebase builds either way — which means the decision is reversible, and should be made on measurement rather than on principle.

Chapter 19 takes the artifact — jar or binary — and turns it into a container image, with the layering, base-image and security decisions that determine build speed and attack surface.

18.15 References

- Building a native executable — <https://quarkus.io/guides/building-native-image>
- Native reference guide — <https://quarkus.io/guides/native-reference>
- Tips for writing native-friendly applications — <https://quarkus.io/guides/writing-native-applications-tips>
- GraalVM Native Image — <https://www.graalvm.org/latest/reference-manual/native-image/>
- Mandrel — <https://github.com/graalvm/mandrel>
- Native Image reachability metadata — <https://www.graalvm.org/latest/reference-manual/native-image/metadata/>
- Profile-guided optimisation — <https://www.graalvm.org/latest/reference-manual/native-image/optimizations-and-performance/PGO/>

Docker and Container Images

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Jib · Docker / Podman · UBI 9 · Distroless

Learning Objectives

By the end of this chapter you will be able to:

1. Explain why the **fast-jar** layout exists and how it maps onto container image layers.
2. Write Dockerfiles for JVM and native modes that rebuild in seconds rather than minutes.
3. Choose a base image on security, size and support criteria rather than habit.
4. Use the Quarkus container-image extensions — Jib, Docker, Buildpacks, OpenShift — and know when each is right.
5. Run containers **as non-root** with a read-only filesystem and a correct signal path.
6. Size JVM memory correctly inside a container, including the cgroup-awareness that people get wrong.
7. Produce reproducible, scannable, signed images with a software bill of materials.
8. Debug a container that will not start.
9. Answer 15 interview questions on containerising Java applications.

19.1 Why Layering Matters

A container image is a stack of read-only layers. On a rebuild, only changed layers are transferred, and only changed layers consume registry storage.

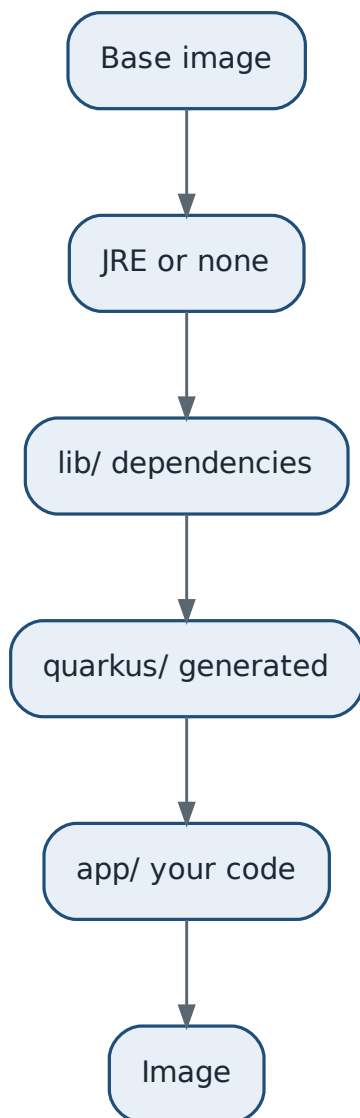


Figure 19.1 — Layers ordered by change frequency. Your code changes every commit; dependencies change monthly.

This is why `quarkus.package.jar.type=fast-jar` is the default. Its output separates concerns exactly along the change-frequency axis:

```
target/quarkus-app/
├─ lib/boot/      rarely changes
├─ lib/main/      changes when dependencies change
├─ quarkus/       changes when config or extensions change
├─ app/           CHANGES EVERY COMMIT (~50 KB)
└─ quarkus-run.jar tiny launcher
```

An uber-jar puts all of that into one 60 MB file, so **every commit transfers 60 MB**. With fast-jar layering, a code-only change transfers about 50 KB. Across a team pushing thirty times a day, that is the difference between a CI pipeline that feels instant and one that does not.

This is the concrete, defensible answer to “why does Quarkus use fast-jar?” — it is a container-image optimisation, not a runtime one.

19.2 The JVM Dockerfile

```
# src/main/docker/Dockerfile.jvm
FROM registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20

ENV LANGUAGE='en_US:en'

# Ordered least-changing to most-changing, so a code edit rebuilds one small layer
COPY --chown=185 target/quarkus-app/lib/ /deployments/lib/
COPY --chown=185 target/quarkus-app/*.jar /deployments/
COPY --chown=185 target/quarkus-app/app/ /deployments/app/
COPY --chown=185 target/quarkus-app/quarkus/ /deployments/quarkus/

EXPOSE 8080
USER 185 # non-root, baked into the UBI image

ENV JAVA_OPTS_APPEND="-Dquarkus.http.host=0.0.0.0"
ENV JAVA_OPTS_APPEND="-Djava.util.logging.manager=org.jboss.logmanager.LogManager"
ENV JAVA_APP_JAR="/deployments/quarkus-run.jar"

ENTRYPOINT ["/opt/jboss/container/java/run/run-java.sh"]
```

```
./mvnw package
docker build -f src/main/docker/Dockerfile.jvm -t acme/orders-service:1.4.0 .
docker run -i --rm -p 8080:8080 acme/orders-service:1.4.0
```

The **COPY** order is the whole trick. **lib/** first because it changes rarely; **app/** last because it changes every commit. Reverse them and every build invalidates the dependency layer.

19.2.1 A multi-stage build

```
# Build and runtime in one file - no local Maven needed
FROM registry.access.redhat.com/ubi9/openjdk-21:1.20 AS build
WORKDIR /code

# Dependency layer: only re-runs when the POM changes
COPY --chown=quarkus mvnw /code/mvnw
COPY --chown=quarkus .mvn /code/.mvn
COPY --chown=quarkus pom.xml /code/
RUN ./mvnw -B dependency:go-offline

# Source layer
```

```

COPY src /code/src
RUN ./mvnw -B package -DskipTests

FROM registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20
COPY --from=build --chown=185 /code/target/quarkus-app/lib/ /deployments/lib/
COPY --from=build --chown=185 /code/target/quarkus-app/*.jar /deployments/
COPY --from=build --chown=185 /code/target/quarkus-app/app/ /deployments/app/
COPY --from=build --chown=185 /code/target/quarkus-app/quarkus/ /deployments/quarkus/
USER 185
EXPOSE 8080
ENTRYPOINT ["/opt/jboss/container/java/run/run-java.sh"]

```

`dependency:go-offline` before copying source is the Maven equivalent of the fast-jar trick: a source-only change does not re-download the dependency tree.

19.3 The Native Dockerfile

```

# src/main/docker/Dockerfile.native
FROM registry.access.redhat.com/ubi9/ubi-minimal:9.4
WORKDIR /work/
RUN chown 1001 /work && chmod "g+rwX" /work && chown 1001:root /work
COPY --chown=1001:root target/*-runner /work/application
EXPOSE 8080
USER 1001
ENTRYPOINT ["/application", "-Dquarkus.http.host=0.0.0.0"]

```

```

# src/main/docker/Dockerfile.native-micro - the smallest practical image
FROM quay.io/quarkus/quarkus-micro-image:2.0
WORKDIR /work/
RUN chown 1001 /work && chmod "g+rwX" /work && chown 1001:root /work
COPY --chown=1001:root target/*-runner /work/application
EXPOSE 8080
USER 1001
ENTRYPOINT ["/application", "-Dquarkus.http.host=0.0.0.0"]

```

```

./mvnw package -Dnative -Dquarkus.native.container-build=true
docker build -f src/main/docker/Dockerfile.native-micro -t acme/orders-service:1.4.0-native .

```

A native build requires a **statically or mostly-statically linked** binary matched to the runtime image's libc. `-Dquarkus.native.container-build=true` builds inside a Mandrel image based on UBI, which is why the resulting binary runs on `ubi-minimal` and `quarkus-micro-image`. Building natively on macOS or Alpine and copying into UBI will fail with a missing-library error at startup — a common and confusing first experience.

```

# Fully static, for a scratch or distroless base
quarkus.native.additional-build-args=--static,--libc=musl

```

19.4 Base Images

Base	Size	Notes
<code>ubi9/openjdk-21-runtime</code>	~230 MB	Red Hat supported, non-root, run-java.sh
<code>eclipse-temurin:21-jre</code>	~230 MB	Widely used, Debian-based
<code>eclipse-temurin:21-jre-alpine</code>	~170 MB	musl libc — verify your native libraries
<code>gcr.io/distroless/java21</code>	~190 MB	No shell, no package manager, minimal CVEs
<code>ubi9/ubi-minimal</code>	~90 MB	For native binaries
<code>quay.io/quarkus/quarkus-micro-image</code>	~40 MB	For native, Quarkus-tuned
<code>scratch</code>	0	Only for fully static native binaries

Criteria, roughly in order of importance:

- 1. Who patches it, and how fast?** A base image with no security update cadence is a liability regardless of size. UBI, Temurin and Distroless all have real processes.
- 2. CVE surface.** Distroless has no shell and no package manager, which removes a large fraction of scanner findings — and removes the tools an attacker would use after a compromise.
- 3. libc.** Alpine’s musl differs from glibc in ways that matter for some native libraries and for DNS resolution. Verify rather than assume.
- 4. Debuggability.** No shell means no `kubectl exec` into a shell. That is a security benefit and an operational cost; ephemeral debug containers mitigate it.
- 5. Size.** Real but usually the least important — layer caching means you transfer the base once per node.

On Alpine specifically: the size saving is genuine, but musl’s DNS resolver has historically behaved differently from glibc’s under some Kubernetes configurations, and some JDK builds have had subtle issues there. For a service, `ubi-minimal` or Distroless gives you most of the size benefit with fewer surprises.

19.5 The Container-Image Extensions

Quarkus can build and push images without a Dockerfile.

```
./mvnw quarkus:add-extension -Dextensions='container-image-jib'
```

```
quarkus.container-image.build=true
quarkus.container-image.push=true
quarkus.container-image.registry=registry.acme.com
```

```

quarkus.container-image.group=platform
quarkus.container-image.name=orders-service
quarkus.container-image.tag=${quarkus.application.version}
quarkus.container-image.additional-tags=latest,${GIT_SHA}
quarkus.container-image.username=${REGISTRY_USER}
quarkus.container-image.password=${REGISTRY_TOKEN}

# Jib specifics
quarkus.jib.base-jvm-image=registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20
quarkus.jib.base-native-image=quay.io/quarkus/quarkus-micro-image:2.0
quarkus.jib.jvm-arguments=-Xmx256m,-XX:MaxRAMPercentage=75
quarkus.jib.ports=8080,9000
quarkus.jib.labels."org.opencontainers.image.source"=https://github.com/acme/orders

./mvnw package -Dquarkus.container-image.build=true
./mvnw package -Dnative -Dquarkus.container-image.build=true
-Dquarkus.container-image.push=true

```

Extension	Needs a daemon	Notes
<code>container-image-jib</code>	no	Layers correctly by default; ideal for CI
<code>container-image-docker</code>	yes	Uses your Dockerfiles
<code>container-image-podman</code>	yes	Rootless
<code>container-image-buildpack</code>	yes	Cloud Native Buildpacks
<code>container-image-openshift</code>	no	S2I inside the cluster

Jib is the right default for CI. It needs no Docker daemon, so no privileged container and no docker-in-docker; it produces correct layering automatically; and it is reproducible, because it sets deterministic timestamps rather than “now”. The reproducibility point matters more than it sounds: with Jib, an unchanged commit produces a byte-identical image, so your registry does not accumulate a new 200 MB image on every rebuild of the same code.

19.6 Running Correctly

19.6.1 Non-root and read-only

```
USER 1001
```

```

securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
capabilities:
  drop: ["ALL"]
seccompProfile:
  type: RuntimeDefault
volumeMounts:

```

```

- name: tmp
  mountPath: /tmp
volumes:
- name: tmp
  emptyDir: {}

```

`readOnlyRootFilesystem: true` requires a writable `/tmp`, because the JVM writes its performance-data file there and Quarkus uses it for file uploads.

```

# Or disable the JVM's /tmp usage
quarkus.jib.jvm-arguments=-XX:-UsePerfData
quarkus.http.body.uploads-directory=/tmp/uploads

```

19.6.2 Signals and PID 1

A process running as PID 1 in a container does **not** get the kernel's default signal handling. If your `ENTRYPOINT` is a shell script that runs `java` as a child, `SIGTERM` goes to the shell and the JVM never sees it — so graceful shutdown (Chapter 16) never happens and Kubernetes `SIGKILLs` after the grace period, dropping in-flight requests.

```

# GOOD: exec form - the binary is PID 1 and receives signals directly
ENTRYPOINT ["/application", "-Dquarkus.http.host=0.0.0.0"]

# BAD: shell form - /bin/sh is PID 1 and does not forward SIGTERM
ENTRYPOINT ./application

# If you must wrap, exec so the JVM replaces the shell
ENTRYPOINT ["/bin/sh", "-c", "exec java $JAVA_OPTS -jar /deployments/quarkus-run.jar"]

```

The UBI `run-java.sh` handles this correctly, which is one of the reasons to use it.

19.6.3 JVM memory in a container

The JVM has been container-aware since JDK 10, but its defaults are conservative in a way that wastes memory.

```

# Percentage-based, so the setting survives a change to the pod's limits
quarkus.jib.jvm-arguments=\
-XX:MaxRAMPercentage=75.0,\
-XX:InitialRAMPercentage=50.0,\
-XX:+UseG1GC,\
-XX:MaxMetaspaceSize=128m,\
-XX:+ExitOnOutOfMemoryError

resources:
  requests: {memory: "256Mi", cpu: "250m"}
  limits:   {memory: "512Mi", cpu: "1000m"}

```

Points that are routinely got wrong:

- **MaxRAMPercentage**, not **-Xmx**. A fixed `-Xmx256m` becomes wrong the moment someone changes the pod's memory limit; a percentage adapts.

- **Leave headroom.** The heap is not the process. Metaspace, thread stacks, code cache, direct buffers and the GC's own structures live outside it — 75 % is a reasonable ceiling, and going to 90 % invites the OOM killer.
- **-XX:+ExitOnOutOfMemoryError.** Without it a heap-exhausted JVM limps on, failing requests while still passing liveness. Better to die and be restarted.
- **CPU limits throttle the JVM.** A limit of `1000m` means one core, and the JVM sizes its GC and compiler threads from `availableProcessors`, which respects the cgroup quota. A very low limit produces surprisingly poor performance because the JIT compiler threads are starved.

Native mode sidesteps most of this. There is no heap sizing to tune beyond `-Xmx`, no metaspace, no JIT threads. That simplification is an underrated operational benefit of native.

19.7 Supply Chain

```
# Provenance labels
quarkus.container-image.labels."org.opencontainers.image.source"=https://github.com/acme/orders
quarkus.container-image.labels."org.opencontainers.image.revision"=${GIT_SHA}
quarkus.container-image.labels."org.opencontainers.image.version"=${quarkus.application.version}
↪ n}
```

SBOM

```
syft acme/orders-service:1.4.0 -o cyclonedx-json > sbom.json
```

Vulnerability scan, failing the build on high severity

```
trivy image --exit-code 1 --severity HIGH,CRITICAL acme/orders-service:1.4.0
```

Sign and verify

```
cosign sign --key cosign.key registry.acme.com/platform/orders-service:1.4.0
cosign verify --key cosign.pub registry.acme.com/platform/orders-service:1.4.0
```

Quarkus can generate an SBOM at build time:

```
./mvnw quarkus:add-extension -Dextensions='cyclonedx'
./mvnw package # -> target/orders-service-1.4.0-cyclonedx.json
```

Tag immutably. `latest` is not a version; it makes rollbacks ambiguous and defeats image caching guarantees. Tag with the version plus the git SHA, and deploy by digest where the platform allows it.

19.8 Debugging Containers

It exits immediately - what did it say?

```
docker logs <id>
```

```
docker run --rm -it --entrypoint /bin/sh acme/orders-service:1.4.0
```

No shell (distroless / micro)? Attach a debug container

```
kubectl debug -it pod/orders-abc --image=busybox --target=orders
```

```
# What is actually in the image?
docker history acme/orders-service:1.4.0
dive acme/orders-service:1.4.0

# Confirm the JVM's view of its own limits
docker run --rm --memory=512m acme/orders-service:1.4.0 \
  java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|ActiveProcessorCount"
```

Common startup failures and their causes:

Symptom	Cause
<code>exec format error</code>	Architecture mismatch — an arm64 image on amd64, or the reverse
<code>no such file or directory</code> on a native binary	libc mismatch; build in the matching container
Exits with code 137	OOM-killed; heap ceiling too high for the memory limit
Exits with code 143	SIGTERM — normal shutdown, or too short a grace period
Permission denied writing a file	<code>readOnlyRootFilesystem</code> without a writable <code>/tmp</code>
Hangs at startup, then SIGKILL	Shell-form ENTRYPOINT swallowing SIGTERM
Connects to <code>localhost</code> and fails	Missing <code>-Dquarkus.http.host=0.0.0.0</code>

19.9 Multi-Architecture Images

```
docker buildx create --use --name multiarch
docker buildx build --platform linux/amd64,linux/arm64 \
  -f src/main/docker/Dockerfile.jvm \
  -t registry.acme.com/platform/orders-service:1.4.0 --push .
```

JVM images are straightforward to build multi-arch because the jar is architecture-neutral — only the base image differs.

Native images are not. A native binary is compiled for one architecture, so a multi-arch native image needs a native build per platform, either on native runners or under emulation (which is very slow). Budget for this if you run a mixed amd64/arm64 fleet — it is a real and often-overlooked cost of adopting native.

19.10 Best Practices

1. **fast-jar with layered `COPY`**, ordered least-changing first.
2. **Jib in CI** — no daemon, correct layering, reproducible timestamps.
3. **Non-root, read-only root filesystem, all capabilities dropped**, with a writable `/tmp`.

4. **Exec-form ENTRYPOINT** so the process is PID 1 and receives `SIGTERM`.
5. **MaxRAMPercentage**, not `-Xmx`, plus `-XX:+ExitOnOutOfMemoryError`.
6. **Build native inside the matching container** so `libc` matches the runtime image.
7. **Immutable tags** — version plus git SHA, never `latest` for deployment.
8. **Scan and sign in CI**, and generate an SBOM.
9. `-Dquarkus.http.host=0.0.0.0`, or the container listens on loopback only.
10. **Provenance labels** so an image can be traced to a commit.
11. **Pin the base image by digest** for reproducibility, and update it on a schedule.
12. **Multi-stage builds** with dependency resolution before the source copy.

19.11 Common Mistakes

1. **Uber-jar in a container.** Every commit pushes 60 MB instead of 50 KB.
2. **COPY ordered wrongly.** Copying `app/` before `lib/` invalidates the dependency layer on every build.
3. **Running as root.** Fails most admission policies and widens the blast radius of a compromise.
4. **Shell-form ENTRYPOINT.** `SIGTERM` is swallowed; graceful shutdown never runs; requests are dropped on every deploy.
5. **Fixed -Xmx.** Wrong as soon as the pod's limit changes.
6. **Heap set to nearly the memory limit.** The OOM killer takes the process — exit code 137.
7. **Native binary built outside the runtime's libc.** `no such file or directory` at startup, which is a genuinely misleading message.
8. **Missing quarkus.http.host=0.0.0.0.** Listens on loopback; health checks fail.
9. **readOnlyRootFilesystem with no /tmp volume.** The JVM cannot write its perf-data file.
10. **Deploying latest.** Rollbacks become ambiguous and nobody can say what is running.
11. **Never scanning.** A base image left un-updated for a year is a CVE list waiting to be found.
12. **Assuming multi-arch is free for native.** It requires a build per architecture.

19.12 Interview Questions

Junior

- Q1. How do you containerise a Quarkus application?** `./mvnw package`, then a Dockerfile that copies the `target/quarkus-app/` directories in layer order, or `-Dquarkus.container-image.build=true` with the Jib extension and no Dockerfile at all.
- Q2. What is fast-jar?** Quarkus's default packaging: output split into `lib/`, `app/`, `quarkus/` and a small launcher jar, so a container image can layer rarely-changing dependencies separately from frequently-changing application code.
- Q3. Why order COPY instructions carefully?** Each `COPY` is a layer, and a changed layer invalidates every layer after it. Copying dependencies before application code means a code change rebuilds only a tiny layer.

Q4. Why run as non-root? It limits the damage of a container escape or a compromise, and most cluster admission policies require it.

Q5. What does `-Dquarkus.http.host=0.0.0.0` do? Binds to all interfaces. Without it the application listens on loopback only and is unreachable from outside the container.

Mid-level

Q6. Jib or a Dockerfile? Jib for CI: it needs no Docker daemon, so no privileged build container; it layers correctly by default; and it produces reproducible images with deterministic timestamps, so an unchanged commit yields a byte-identical image. A Dockerfile when you need custom build steps, extra files, or an existing organisational base image with its own conventions.

Q7. How do you size JVM memory in a container? `-XX:MaxRAMPercentage=75` rather than a fixed `-Xmx`, so it adapts when the pod's memory limit changes. Leave headroom for metaspace, thread stacks, code cache and direct buffers, which live outside the heap. Add `-XX:+ExitOnOutOfMemoryError` so an exhausted JVM dies and is restarted rather than limping while still passing liveness.

Q8. Why does my container ignore `SIGTERM`? A shell-form `ENTRYPOINT` makes `/bin/sh` PID 1, and it does not forward signals to its child. Use the exec form, or `exec` the process from the shell so it replaces it.

Q9. Why does my native binary fail with “no such file or directory”? It is linked against a libc that is not present in the runtime image — typically built on a glibc host and run on Alpine, or vice versa. Build with `-Dquarkus.native.container-build=true` so the builder image matches the runtime base.

Q10. What does exit code 137 mean? `SIGKILL`, almost always the kernel OOM killer. The process exceeded the container's memory limit — usually because the heap ceiling left no headroom for off-heap memory.

Q11. How do you make images reproducible? Jib with deterministic timestamps, base images pinned by digest rather than tag, dependency versions pinned by the BOM and a lock, and no `latest` anywhere. Reproducibility is what makes an SBOM and an image signature meaningful.

Senior

Q12. Design the container build pipeline for 40 services. A shared, versioned base image per runtime — one JVM, one native — owned by the platform team, patched on a schedule and pinned by digest downstream, so a CVE is fixed in one place rather than forty. Builds use Jib, which removes the need for privileged docker-in-docker in CI and gives reproducibility for free. Every image is tagged with the semantic version and the git SHA, never `latest`, and carries provenance labels linking it back to a commit. The pipeline generates an SBOM, scans with a policy that fails on high and critical findings, and signs with cosign; the cluster admission controller verifies the signature, which is what turns signing from a ritual into a control. Native builds run on their own stage with larger agents and, if the fleet is mixed-architecture, on native runners per architecture rather than under emulation. What I would centralise is the base images, the CI template and the policy gates; what I would leave to teams is anything domain-specific in their Dockerfile — and I would measure adoption of the shared base rather than police it.

Q13. Your images are 800 MB and builds take ten minutes. Where do you look? Size first: `dive` or `docker history` shows the layer breakdown, and the usual findings are a full JDK where a JRE would do, a build stage not separated from the runtime stage so the Maven cache is shipped, an uber-jar, or `COPY . .` bringing the entire working tree including `.git` and `target`. Switching to a JRE runtime base with a multi-stage build and fast-jar layering typically takes that 800 MB to around 250 MB without any other change. For build time, the question is what is being redone unnecessarily: dependency resolution on every build because `pom.xml` is copied together with the source rather than before it, no BuildKit cache mount for the Maven repository, or a `COPY` order that invalidates everything. I would also check whether tests are running in the image build when they already ran in the pipeline. The ordering of investigation matters — layer caching almost always yields more than image-size micro-optimisation, and it is what the team actually feels.

Q14. Distroless, UBI, or Alpine? It depends on what the organisation can support, not on size. Distroless has the smallest CVE surface because it has no shell and no package manager, which also removes the tooling an attacker would use post-compromise — the cost is that `kubectrl exec` gives you nothing, so the team needs to be comfortable with ephemeral debug containers. UBI is the pragmatic enterprise choice: Red Hat patches it, it has a support relationship if you have one, it runs non-root by default, and `run-java.sh` handles signals and container memory correctly. Alpine’s saving is real but musl is not glibc, and DNS resolution and some native libraries behave differently in ways that surface late and are hard to diagnose. My default for JVM services would be UBI, moving to Distroless where the security posture demands it and the team is ready; for native binaries, `ubi-minimal` or the Quarkus micro image. The decision I would push back on is choosing Alpine purely for size, because the size difference is amortised by layer caching and the debugging cost is not.

Principal

Q15. How do you secure the container supply chain end to end? The goal is that every running image can be traced to a reviewed commit, and that nothing else can run. Working backwards from the cluster: an admission controller that only admits images signed by our CI’s key and from our registry, which is the control that makes everything upstream meaningful — signing without verification is theatre. CI signs with cosign using a short-lived, workload-identity-issued key rather than a long-lived secret, and attaches an SBOM and a provenance attestation recording the source commit, the builder and the build parameters. Builds are hermetic and reproducible — Jib rather than a daemon, dependencies from an internal proxy repository rather than the public internet, base images pinned by digest — so an image can be rebuilt and compared. Base images are owned centrally, rebuilt on a schedule and on CVE disclosure, with automated PRs to bump the digest in every service. Scanning runs both at build time and continuously against the registry, because a CVE disclosed after the build will not be caught by a build-time gate. And there is a documented, rehearsed path to rebuild and redeploy everything on a base-image CVE, because that capability is the thing you actually need on the day, and organisations discover they lack it during the incident. The part teams most often skip is the admission control, and it is the one that turns the rest from documentation into enforcement.

19.13 Cheat Sheet

Layered JVM Dockerfile

```
FROM registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20
COPY --chown=185 target/quarkus-app/lib/ /deployments/lib/
COPY --chown=185 target/quarkus-app/*.jar /deployments/
COPY --chown=185 target/quarkus-app/app/ /deployments/app/
COPY --chown=185 target/quarkus-app/quarkus/ /deployments/quarkus/
USER 185
EXPOSE 8080
ENTRYPOINT ["/opt/jboss/container/java/run/run-java.sh"]
```

Native

```
FROM quay.io/quarkus/quarkus-micro-image:2.0
COPY --chown=1001:root target/*-runner /work/application
USER 1001
ENTRYPOINT ["/application", "-Dquarkus.http.host=0.0.0.0"]
```

Jib

```
quarkus.container-image.build=true
quarkus.container-image.push=true
quarkus.container-image.registry=registry.acme.com
quarkus.container-image.group=platform
quarkus.container-image.tag=${quarkus.application.version}
quarkus.jib.base-jvm-image=registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20
quarkus.jib.jvm-arguments=-XX:MaxRAMPercentage=75,-XX:+ExitOnOutOfMemoryError
```

Security context

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: {drop: ["ALL"]}
```

Exit codes

```
137 SIGKILL - usually the OOM killer
143 SIGTERM - normal shutdown
1 application error - read the logs
```

Debugging

```
docker logs <id>
docker history <image>          dive <image>
kubectl debug -it pod/x --image=busybox --target=app
java -XX:+PrintFlagsFinal -version | grep MaxHeapSize
```

19.14 Summary

Containerising a Quarkus service well is mostly about **layer ordering**. The fast-jar layout exists specifically so that dependencies, generated code and application code become separate image layers

ordered by change frequency — turning a 60 MB push per commit into a 50 KB one. Getting the `COPY` order wrong silently discards that entire benefit.

Jib is the right default in CI: no daemon, no privileged build container, correct layering, and reproducible images whose digests are stable for unchanged code — which is the precondition for signing and SBOMs meaning anything.

Three runtime details cause most production surprises. **Exec-form** **ENTRYPOINT**, so the process is PID 1 and actually receives **SIGTERM** — otherwise the graceful shutdown from Chapter 16 never runs. **MaxRAMPercentage** rather than **-Xmx**, with real headroom for off-heap memory, plus **ExitOnOutOfMemoryError** so an exhausted JVM dies rather than limping while still passing liveness. And **quarkus.http.host=0.0.0.0**, without which the container listens on loopback and every probe fails.

For native, the constraint to internalise is `libc`: build inside the container that matches your runtime base, and remember that multi-architecture images require a build per architecture rather than one neutral artefact.

Chapter 20 takes these images to **Kubernetes** — manifests, resource sizing, autoscaling, rollouts, and the configuration decisions that determine whether a deploy is invisible or an incident.

19.15 References

- Building a container image — <https://quarkus.io/guides/container-image>
- Deploying to Kubernetes — <https://quarkus.io/guides/deploying-to-kubernetes>
- Red Hat UBI images — <https://catalog.redhat.com/software/base-images>
- Jib — <https://github.com/GoogleContainerTools/jib>
- Distroless — <https://github.com/GoogleContainerTools/distroless>
- Java in containers, resource limits — <https://developers.redhat.com/articles/2022/04/19/java-17-whats-new-openjdk-container-awareness>
- Sigstore cosign — <https://docs.sigstore.dev/cosign/overview/>
- CycloneDX SBOM — <https://cyclonedx.org/>

Kubernetes

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · [quarkus-kubernetes](#) · [quarkus-kubernetes-config](#) · Kubernetes 1.31

Learning Objectives

By the end of this chapter you will be able to:

1. Generate Kubernetes manifests from a Quarkus build and know when to stop generating them.
2. Set requests and limits from measurement, and explain why CPU limits are usually a mistake.
3. Configure rollouts, disruption budgets and anti-affinity so a deploy is invisible.
4. Wire ConfigMaps and Secrets into Quarkus, and reload configuration without a restart.
5. Design autoscaling that reacts to the right signal — including for a Kafka consumer.
6. Trace the full request path from Ingress to pod and reason about where TLS terminates.
7. Deploy database migrations safely alongside a rolling update.
8. Debug the failures that dominate in practice: [CrashLoopBackOff](#), [OOMKilled](#), dropped requests during deploys.
9. Answer 20 interview questions on running Quarkus in Kubernetes.

20.1 Generating Manifests

```
./mvnw quarkus:add-extension -Dextensions='kubernetes,container-image-jib'
```

```
quarkus.kubernetes.namespace=platform
quarkus.kubernetes.name=orders-service
quarkus.kubernetes.version=${quarkus.application.version}
quarkus.kubernetes.replicas=3
```

```
quarkus.kubernetes.resources.requests.memory=256Mi
quarkus.kubernetes.resources.requests.cpu=250m
quarkus.kubernetes.resources.limits.memory=512Mi
```

```
quarkus.kubernetes.liveness-probe.http-action-path=/q/health/live
quarkus.kubernetes.liveness-probe.period=10s
quarkus.kubernetes.liveness-probe.failure-threshold=3
quarkus.kubernetes.readiness-probe.http-action-path=/q/health/ready
quarkus.kubernetes.readiness-probe.period=5s
quarkus.kubernetes.startup-probe.http-action-path=/q/health/started
```



```

quarkus.kubernetes.startup-probe.failure-threshold=30

quarkus.kubernetes.env.vars.deploy-env=production
quarkus.kubernetes.env.secrets=orders-secrets
quarkus.kubernetes.env.configmaps=orders-config

quarkus.kubernetes.labels."app.kubernetes.io/part-of"=commerce
quarkus.kubernetes.annotations."prometheus.io/scrape"=true

quarkus.kubernetes.ingress.expose=true
quarkus.kubernetes.ingress.host=orders.acme.com
quarkus.kubernetes.ingress.tls."acme-tls".enabled=true

./mvnw package          # -> target/kubernetes/kubernetes.yml
./mvnw package -Dquarkus.kubernetes.deploy=true  # apply directly

```

Generated manifests are excellent for getting started and a poor fit for production.

They are opinionated, they cover perhaps 70 % of what a real Deployment needs, and the remaining 30 % — pod disruption budgets, topology spread, network policies, service accounts with workload identity, sidecar configuration — is expressed awkwardly through `quarkus.kubernetes.*` properties or not at all. The healthy pattern is to generate once, commit the output as the starting point for a Helm chart or Kustomize base, and manage it as infrastructure thereafter. What you keep from the extension is the probe configuration defaults and the container-image integration.

20.2 A Production Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
  labels:
    app.kubernetes.io/name: orders-service
    app.kubernetes.io/version: "1.4.0"
    app.kubernetes.io/part-of: commerce
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0          # never dip below the desired count
  selector:
    matchLabels:
      app.kubernetes.io/name: orders-service
  template:
    metadata:
      labels:
        app.kubernetes.io/name: orders-service

```

```

    app.kubernetes.io/version: "1.4.0"
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9000"
    # Force a rollout when config changes - see 20.5
    checksum/config: "{{ include (print $.Template.BasePath \"/configmap.yaml\") . |
    sha256sum }}"
  spec:
    serviceAccountName: orders-service
    terminationGracePeriodSeconds: 45
    securityContext:
      runAsNonRoot: true
      runAsUser: 1001
      fsGroup: 1001
      seccompProfile: {type: RuntimeDefault}

    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels: {app.kubernetes.io/name: orders-service}

    containers:
      - name: orders
        image: registry.acme.com/platform/orders-service:1.4.0
        imagePullPolicy: IfNotPresent

        ports:
          - {name: http,          containerPort: 8080}
          - {name: management, containerPort: 9000}

        env:
          - name: QUARKUS_PROFILE
            value: prod
          - name: DB_PASSWORD
            valueFrom:
              secretKeyRef: {name: orders-secrets, key: db-password}
          - name: POD_NAME
            valueFrom:
              fieldRef: {fieldPath: metadata.name}

        envFrom:
          - configMapRef: {name: orders-config}

        resources:
          requests: {memory: "384Mi", cpu: "250m"}
          limits:   {memory: "768Mi"}           # note: no CPU limit

        startupProbe:
          httpGet: {path: /q/health/started, port: management}
          periodSeconds: 5
          failureThreshold: 30

```

```

    livenessProbe:
      httpGet: {path: /q/health/live, port: management}
      periodSeconds: 10
      timeoutSeconds: 3
      failureThreshold: 3
    readinessProbe:
      httpGet: {path: /q/health/ready, port: management}
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 2

    lifecycle:
      preStop:
        exec: {command: ["sh", "-c", "sleep 5"]}

    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: {drop: ["ALL"]}

    volumeMounts:
      - {name: tmp, mountPath: /tmp}
      - {name: config, mountPath: /deployments/config, readOnly: true}

  volumes:
    - name: tmp
      emptyDir: {}
    - name: config
      configMap: {name: orders-app-config}

```

```

---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: orders-service}
spec:
  minAvailable: 2
  selector:
    matchLabels: {app.kubernetes.io/name: orders-service}

```

20.3 Resources: Requests, Limits and the CPU Argument

20.3.1 What they mean

- **Request** — what the scheduler reserves. It determines placement and the pod's QoS class.
- **Limit** — the hard ceiling. Exceeding a memory limit gets the container OOM-killed; exceeding a CPU limit gets it *throttled*.

20.3.2 Set memory limits; think hard before setting CPU limits

Memory is **incompressible**: exceed it and the kernel kills you. A limit is therefore a necessary guard against a leak taking down the node.

CPU is **compressible**: exceeding your share just makes you slower. A CPU limit is enforced by CFS quota over a 100 ms period, and when a container exhausts its quota it is **throttled for the remainder of that period** — even if the node is completely idle.

For a JVM this is worse than it sounds. Garbage collection, JIT compilation and startup are all bursty and multi-threaded. A limit of **500m** means the container gets 50 ms of CPU per 100 ms window, so a GC pause that wants four threads for 20 ms exhausts the quota and the application stalls for the remaining 50 ms — producing latency spikes that look like a GC problem but are a scheduling artefact.

```
resources:
  requests: {memory: "384Mi", cpu: "250m"}    # scheduling and guaranteed share
  limits:   {memory: "768Mi"}                 # memory only
```

The position to be able to defend: set CPU *requests* — they guarantee a share under contention and drive scheduling — and omit CPU *limits* unless you have a specific reason, such as a multi-tenant cluster where noisy-neighbour isolation is contractual, or a batch workload that would otherwise consume a whole node. Without a limit, a pod can burst into idle capacity, which is exactly what you want during startup and GC. Monitor `container_cpu_cfs_throttled_seconds_total`; if it is non-zero on a latency-sensitive service, your limits are hurting you.

The counter-argument is real and worth acknowledging: without limits, capacity planning is harder and a runaway pod can degrade its neighbours. The mitigation is requests set honestly, plus resource quotas at the namespace level.

```
kubectl top pods -l app.kubernetes.io/name=orders-service --containers
```

```
# Right-size memory: p99 of actual usage, plus headroom
quantile_over_time(0.99,
  container_memory_working_set_bytes{pod=~"orders-.*"}[7d])

# CPU: p95 of real usage
quantile_over_time(0.95, rate(container_cpu_usage_seconds_total{pod=~"orders-.*"}[5m])[7d:])

# Throttling - should be zero for latency-sensitive services
rate(container_cpu_cfs_throttled_seconds_total{pod=~"orders-.*"}[5m])
```

A workable rule for a Quarkus JVM service: memory request at the p99 of working set, memory limit at roughly 1.5–2× the request, and heap at 75 % of the limit via `MaxRAMPercentage` (Chapter 19).

Native mode is dramatically simpler here — a request of **64Mi** and a limit of **128Mi** is often ample, which is where the cost saving actually comes from.

20.4 Rollouts Without Dropped Requests

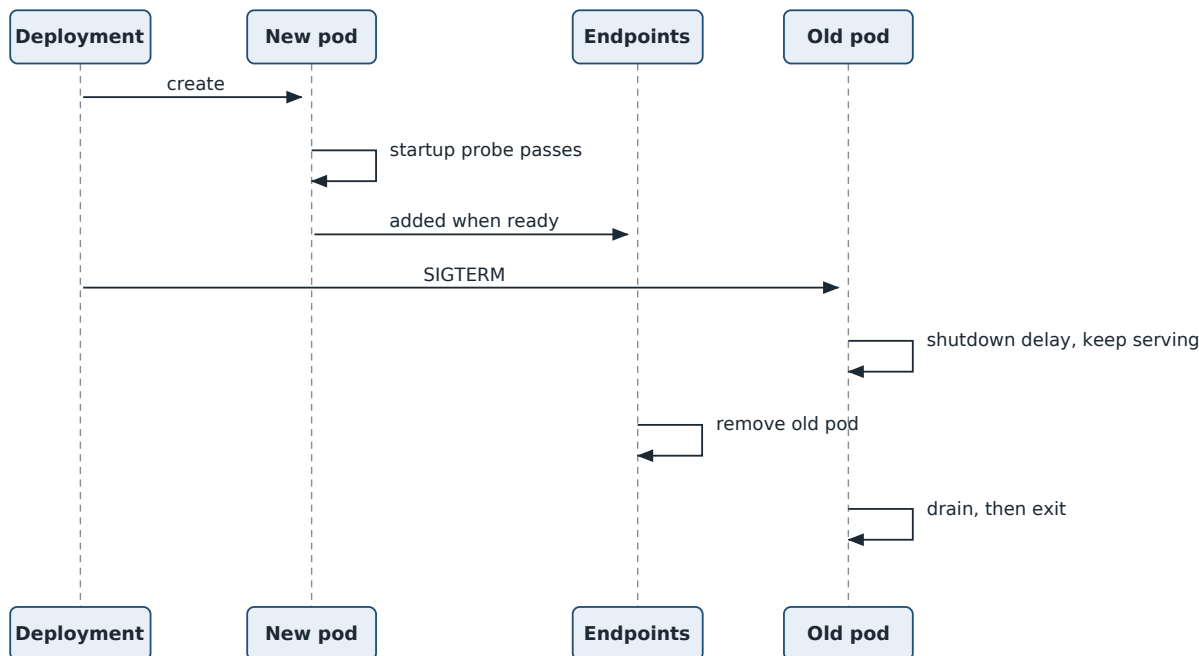


Figure 20.1 — A safe rollout: the new pod is serving before the old one stops.

Four settings must agree, and getting any one wrong drops requests on every deploy:

```

strategy:
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0           # capacity never dips
  terminationGracePeriodSeconds: 45
  lifecycle:
    preStop: {exec: {command: ["sh", "-c", "sleep 5"]}}
```

```

quarkus.shutdown.delay=5S      # keep serving while endpoints propagate
quarkus.shutdown.timeout=30S   # then drain in-flight requests
```

The invariant: `terminationGracePeriodSeconds > shutdown.delay + shutdown.timeout`. Otherwise Kubernetes **SIGKILLs** mid-drain and the requests you were carefully finishing are lost.

The reason the delay is needed at all — worth being able to explain — is that **SIGTERM** and endpoint removal happen **concurrently**, and endpoint propagation to every kube-proxy and ingress takes seconds. Without a delay, a pod that exits promptly on **SIGTERM** is still receiving traffic when it goes.

```

kubectl rollout status deployment/orders-service --timeout=5m
kubectl rollout undo deployment/orders-service
kubectl rollout history deployment/orders-service
```

20.5 Configuration

20.5.1 As environment variables

```
envFrom:
  - configMapRef: {name: orders-config}
  - secretRef: {name: orders-secrets}
```

```
apiVersion: v1
kind: ConfigMap
metadata: {name: orders-config}
data:
  QUARKUS_LOG_LEVEL: "INFO"
  QUARKUS_DATASOURCE_JDBC_URL: "jdbc:postgresql://orders-db:5432/orders"
  ORDERS_MAX_LINE_ITEMS: "100"
```

Simple, but you must get the env-var name mapping right (Chapter 4 §4.2.2), and a change requires a pod restart.

20.5.2 As a mounted properties file — usually better

```
volumeMounts:
  - {name: config, mountPath: /deployments/config, readOnly: true}
volumes:
  - name: config
    configMap: {name: orders-app-config}
```

```
apiVersion: v1
kind: ConfigMap
metadata: {name: orders-app-config}
data:
  application.properties: |
    quarkus.log.level=INFO
    quarkus.datasource.jdbc.url=jdbc:postgresql://orders-db:5432/orders
    quarkus.datasource.jdbc.max-size=20
    orders.max-line-items=100
    orders.retry.max-attempts=5
```

`$PWD/config/application.properties` has ordinal 260 (Chapter 4), above the packaged file. **Real property syntax, no env-var mapping to get wrong** — which removes the single most common Kubernetes configuration bug.

20.5.3 Reading ConfigMaps directly

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-kubernetes-config</artifactId>
</dependency>
```

```

quarkus.kubernetes-config.enabled=true
quarkus.kubernetes-config.config-maps=orders-config,shared-config
quarkus.kubernetes-config.secrets.enabled=true
quarkus.kubernetes-config.secrets=orders-secrets
quarkus.kubernetes-config.fail-on-missing-config=true
quarkus.kubernetes-config.namespace=platform

```

This requires RBAC:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: orders-config-reader, namespace: platform}
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list"]

```

20.5.4 Rolling on config change

A ConfigMap change does **not** restart pods. Mounted files update eventually (up to a minute or two); environment variables never do.

The standard fix is a checksum annotation on the pod template, which changes the template and therefore triggers a rollout:

```

annotations:
  checksum/config: "{{ include (print $.Template.BasePath \"/configmap.yaml\") . | sha256sum
    }}"

```

Do not build a config-watching reload mechanism unless you genuinely need one. A rollout is observable, revertible with `kubectl rollout undo`, and applies the change atomically across replicas. A live reload is none of those, and Chapter 4's build-time properties cannot be reloaded at all.

20.6 Autoscaling

20.6.1 Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: orders-service}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: orders-service}
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 70}
  - type: Pods

```

```

    pods:
      metric: {name: http_server_requests_per_second}
      target: {type: AverageValue, averageValue: "100"}
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies: [{type: Percent, value: 100, periodSeconds: 30}]
    scaleDown:
      stabilizationWindowSeconds: 300      # scale down slowly
      policies: [{type: Percent, value: 25, periodSeconds: 60}]

```

CPU utilisation is a proxy, not the thing you care about. For a service whose bottleneck is a connection pool or a downstream call, CPU stays flat while latency climbs, and the HPA never triggers. A custom metric — requests per second, in-flight requests, or queue depth — reflects load far better.

Asymmetric behaviour matters. Scale up quickly and down slowly: scaling up late costs user-visible latency, whereas scaling down early causes flapping and repeated cold starts.

20.6.2 Scaling a Kafka consumer

CPU is meaningless here; **consumer lag** is the signal.

```

apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: {name: orders-consumer}
spec:
  scaleTargetRef: {name: orders-consumer}
  minReplicaCount: 1
  maxReplicaCount: 6      # NEVER above the partition count
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: kafka:9092
        consumerGroup: orders-processor
        topic: orders
        lagThreshold: "1000"

```

`maxReplicaCount` above the partition count buys nothing — the extra consumers idle (Chapter 11). This is a very common misconfiguration, and it shows up as “we scaled to twenty pods and lag did not improve”.

20.6.3 Where Quarkus helps

Fast startup is what makes aggressive autoscaling viable. A pod that is serving 800 ms after being scheduled can respond to a traffic spike; one that takes 40 seconds cannot, and you are forced to over-provision instead. In native mode the startup is effectively instant, which is what makes scale-to-zero (Knative, KEDA) practical for Java at all.

20.7 Networking

```

apiVersion: v1
kind: Service
metadata:
  name: orders-service
  annotations: {prometheus.io/scrape: "true", prometheus.io/port: "9000"}
spec:
  type: ClusterIP
  selector: {app.kubernetes.io/name: orders-service}
  ports:
    - {name: http, port: 80, targetPort: http}
      # management port deliberately NOT exposed here
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orders-service
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"
spec:
  ingressClassName: nginx
  tls: [{hosts: [orders.acme.com], secretName: orders-tls}]
  rules:
    - host: orders.acme.com
      http:
        paths:
          - {path: /api/orders, pathType: Prefix,
            backend: {service: {name: orders-service, port: {number: 80}}}}

```

Note the Service exposes only the application port. The management port (Chapters 14 and 16) is reachable from Prometheus and the kubelet within the cluster but not through the Ingress — so metrics and health are never publicly exposed.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: orders-service}
spec:
  podSelector:
    matchLabels: {app.kubernetes.io/name: orders-service}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - podSelector: {matchLabels: {app.kubernetes.io/name: api-gateway}}
        - namespaceSelector: {matchLabels: {name: monitoring}}
      ports: [{port: 8080}, {port: 9000}]
  egress:
    - to: [{podSelector: {matchLabels: {app: postgres}}}]
      ports: [{port: 5432}]
    - to: [{namespaceSelector: {matchLabels: {name: kube-system}}}]
      ports: [{port: 53, protocol: UDP}] # DNS - easy to forget

```

Forgetting the DNS egress rule is the classic NetworkPolicy mistake: everything resolves fine until the policy is applied, then every hostname lookup fails and the symptom looks like a general network outage.

20.8 Database Migrations

Chapter 5 §5.10.3 covered why `migrate-at-start` is wrong with multiple replicas. The Kubernetes expression of the fix:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: orders-migrate-1.4.0
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: registry.acme.com/platform/orders-service:1.4.0
          args: ["migrate"]
          env:
            - {name: QUARKUS_PROFILE, value: migrate}
            - name: DB_PASSWORD
              valueFrom: {secretKeyRef: {name: orders-secrets, key: db-password}}
```

```
%prod.quarkus.flyway.migrate-at-start=false
%prod.quarkus.hibernate-orm.database.generation=validate
%prod.quarkus.flyway.migrate-at-start=true
```

```
@QuarkusMain
public class Main implements QuarkusApplication {
    public static void main(String... args) { Quarkus.run(Main.class, args); }

    @Override
    public int run(String... args) {
        if (args.length > 0 && "migrate".equals(args[0])) {
            return 0; // Flyway ran at startup; exit
        }
        Quarkus.waitForExit();
        return 0;
    }
}
```

A failed migration Job stops the rollout before any new pod starts, which is exactly the behaviour you want. And remember the expand–migrate–contract discipline: during the rollout, old and new pods run against the same schema simultaneously.

20.9 Debugging

```
kubectl get pods -l app.kubernetes.io/name=orders-service
kubectl describe pod orders-abc123           # events are at the bottom
kubectl logs orders-abc123 --previous        # the crashed container
kubectl logs -f -l app.kubernetes.io/name=orders-service --max-log-requests=10
kubectl exec -it orders-abc123 -- sh
kubectl debug -it orders-abc123 --image=nicolaka/netshoot --target=orders
kubectl port-forward orders-abc123 9000:9000 # reach /q/metrics locally
kubectl get events --sort-by=.lastTimestamp
```

Symptom	Likely cause
<code>CrashLoopBackOff</code>	Startup failure — <code>logs --previous</code> ; often a config or migration error
<code>OOMKilled</code> (137)	Heap ceiling too high for the memory limit; check <code>MaxRAMPercentage</code>
<code>ImagePullBackOff</code>	Wrong tag, wrong registry, or a missing image pull secret
<code>CreateContainerConfigError</code>	Missing ConfigMap or Secret referenced by the pod
Pod <code>Running</code> but not <code>Ready</code>	Readiness failing — probe the management port directly
Requests dropped on deploy	Missing shutdown delay, or grace period too short
Restart loop under load	Liveness too aggressive, or a dependency check in liveness
Latency spikes, low CPU	CFS throttling — check <code>container_cpu_cfs_throttled_seconds_total</code>
DNS failures after a policy change	NetworkPolicy missing the egress rule to kube-dns

20.10 Best Practices

1. **Generate manifests once, then manage them as infrastructure** in Helm or Kustomize.
2. **Set memory limits; omit CPU limits** unless you have a specific isolation requirement.
3. `maxUnavailable: 0` plus a `PodDisruptionBudget` so capacity never dips.
4. `terminationGracePeriodSeconds > shutdown.delay + shutdown.timeout`.
5. **Mount configuration as `config/application.properties`** rather than a wall of environment variables.
6. **Checksum annotation** so a ConfigMap change triggers a rollout.
7. **Probes on the management port**, which is not exposed through the Service.

8. **Migrations as a pre-upgrade Job**, never `migrate-at-start` with multiple replicas.
9. **Autoscale on a signal that reflects load** — RPS or queue depth, not just CPU; lag for Kafka consumers.
10. **Never scale a Kafka consumer beyond the partition count.**
11. **Topology spread across zones**, and a PDB so a node drain does not take the service down.
12. **NetworkPolicies with an explicit DNS egress rule.**
13. **Right-size from measurement**, and re-check quarterly.

20.11 Common Mistakes

1. **CPU limits on a latency-sensitive JVM service.** CFS throttling produces latency spikes that look like GC problems.
2. **`terminationGracePeriodSeconds` shorter than the drain time.** `SIGKILL` mid-drain drops requests on every deploy.
3. **No shutdown delay.** Requests routed during endpoint propagation are lost.
4. **`migrate-at-start` with three replicas.** Racing migrations; a long one exceeds the readiness deadline and produces a rollback loop.
5. **A ConfigMap change with no rollout trigger.** Environment variables never update; mounted files update eventually and inconsistently.
6. **Wrong env-var name mapping.** Silent — the variable is simply never read.
7. **Autoscaling on CPU for a pool-bound service.** CPU stays flat while latency climbs; the HPA never fires.
8. **Kafka consumer replicas above the partition count.** The extras idle.
9. **`maxUnavailable: 1` with three replicas under load.** A third of capacity disappears during every deploy.
10. **No PodDisruptionBudget.** A node drain can evict every replica at once.
11. **NetworkPolicy without DNS egress.** Every hostname lookup fails after the policy is applied.
12. **Health endpoints exposed through the Ingress.** Leaks dependency topology publicly.
13. **Deploying `latest`.** Nobody can say what is running, and rollback is ambiguous.

20.12 Interview Questions

Junior

Q1. How do you deploy a Quarkus application to Kubernetes? Build a container image, push it to a registry, and apply a Deployment plus a Service. The `quarkus-kubernetes` extension can generate the manifests during the build.

Q2. What is a Deployment? A controller that maintains a desired number of pod replicas and manages rolling updates between versions.

Q3. Requests versus limits? A request is what the scheduler reserves and guarantees; a limit is the hard ceiling. Exceeding a memory limit gets the container killed; exceeding a CPU limit gets it throttled.

Q4. What is a Service? A stable virtual IP and DNS name that load-balances across the pods matching its selector.

Q5. How do you supply configuration? Environment variables from a ConfigMap or Secret, or a mounted `application.properties` file, which Quarkus reads at ordinal 260 if placed in a `config/` directory next to the artifact.

Q6. Which probes should a Quarkus service expose? `/q/health/live`, `/q/health/ready` and `/q/health/started`, ideally on the management port.

Mid-level

Q7. Why might you not set a CPU limit? CPU is compressible, and a limit is enforced by CFS quota over a 100 ms window: a container that exhausts its quota is throttled for the rest of the period even on an idle node. JVM garbage collection, JIT compilation and startup are bursty and multi-threaded, so limits produce latency spikes that look like GC problems. Set CPU requests, which guarantee a share and drive scheduling, and monitor `container_cpu_cfs_throttled_seconds_total`.

Q8. How do you avoid dropping requests during a rolling update? Four things together: `maxUnavailable: 0` so capacity never dips; `quarkus.shutdown.delay` so the pod keeps serving while endpoint removal propagates, since `SIGTERM` and endpoint removal happen concurrently; `quarkus.shutdown.timeout` to drain in-flight requests; and `terminationGracePeriodSeconds` greater than the sum of the two so Kubernetes does not `SIGKILL` mid-drain.

Q9. How do you run database migrations? As a pre-upgrade Job or init container using the same image, with `migrate-at-start` disabled in production. With multiple replicas, `migrate-at-start` races — Flyway’s lock makes it safe but slow, and a long migration can exceed readiness deadlines. A failed Job also stops the rollout before any new pod starts.

Q10. A ConfigMap changed but nothing happened. Why? Environment variables from a ConfigMap are resolved at pod creation and never update. Mounted files do update, but slowly and without notifying the application. The standard fix is a checksum annotation on the pod template so a config change alters the template and triggers a rollout.

Q11. How do you autoscale a Kafka consumer? On consumer lag, using KEDA — CPU is unrelated to whether the consumer is keeping up. Cap `maxReplicaCount` at the partition count, because additional consumers in a group get no partitions and sit idle.

Q12. What does `OOMKilled` mean and how do you fix it? The container exceeded its memory limit and the kernel killed it — exit code 137. For a JVM the usual cause is a heap ceiling that leaves no headroom for metaspace, thread stacks, code cache and direct buffers. Use `MaxRAMPercentage` around 75 rather than a fixed `-Xmx`, and size the limit from the observed p99 working set.

Senior

Q13. Design the Kubernetes deployment for a Quarkus service handling 10 000 requests per second. Start from the bottleneck rather than the traffic number, because 10 000 rps says nothing about what constrains the service. Assuming it is database-backed: replica

count derived from measured per-pod throughput with headroom, `maxUnavailable: 0` and a PDB with `minAvailable` set so a node drain cannot remove too much capacity, and topology spread across zones so a zone failure is survivable. Memory request at the observed p99 working set, memory limit around $1.5\times$ that, CPU request from p95 usage and no CPU limit. Probes on the management port with a generous startup budget and forgiving liveness. Autoscaling on requests per second or in-flight requests rather than CPU, scaling up fast and down slowly. Critically, the connection pool must be sized against the database's capacity times the replica count, not per pod in isolation — this is where high-replica deployments usually fall over, and a PgBouncer layer is often the right answer. Configuration mounted as a properties file, migrations as a pre-upgrade Job, and a NetworkPolicy restricting ingress to the gateway and monitoring namespaces. Then I would load-test the whole thing including a rollout under load, because the deployment settings are only proven by watching a deploy at full traffic.

Q14. Your pods restart under load but the application logs show nothing wrong.

Diagnose it. Nothing in the logs plus restarts points away from the application, so I would look at why the kubelet is killing it. `kubectl describe pod` shows the last termination reason and exit code, which usually settles it immediately: 137 means OOM-killed, and the fix is heap or limit sizing; a liveness probe failure means the probe is timing out. If it is liveness, the most common causes are a probe `timeoutSeconds` of 1 against a JVM in a GC pause, a `failureThreshold` too low, or — the one I would check specifically — a dependency check in the liveness probe, which under load turns a slow database into a restart storm across every pod. I would also check CFS throttling, because a heavily throttled container can miss probe deadlines while doing nothing wrong, and that combination produces exactly this symptom of restarts with clean application logs. The systematic move is to look at the kubelet's view before the application's, since the application by definition has no log line for being killed.

Q15. How do you handle a stateful dependency during a rolling deploy? The core constraint is that old and new pods run simultaneously against the same schema, so every migration must be compatible with both versions for the duration of the rollout. That forces expand–migrate–contract across releases: add a nullable column and dual-write in release one, backfill and switch reads in release two, drop the old column in release three. Renames become add-copy-drop. Migrations run as a pre-upgrade Job so they complete before any new pod starts and a failure halts the deploy. I would also think about the connection pool during the surge: with `maxSurge: 1` there is briefly an extra replica, so total connections exceed the steady-state figure, which matters if the database is near its limit. And for anything with in-flight work — a Kafka consumer mid-batch, a long-running request — the grace period must exceed the longest unit of work, or the drain is theoretical.

Q16. When would you use a StatefulSet for a Quarkus service? Rarely, and I would treat wanting one as a signal to re-examine the design. A StatefulSet gives stable network identity, ordered rollout and per-pod persistent volumes — properties that matter for databases, brokers and consensus systems, not for stateless HTTP services. The cases where it is legitimate for an application: a service that shards work by pod ordinal and needs a stable identity to claim its shard; a service embedding something like Infinispan or Hazelcast in a clustered mode that needs stable peer addresses; or a leader-elected singleton where ordered startup matters. In each of those I would first ask whether the state could move out — to a database, to a broker, to a leader election API — because a StatefulSet materially complicates rollouts, scaling and disaster recovery, and it removes the ability to treat pods as fungible. If the answer is genuinely no, then a StatefulSet with a PodDisruptionBudget and an explicit backup and restore procedure.

Principal

Q17. Design the deployment platform for 40 Quarkus services. The objective is that a team can ship a service safely without knowing much Kubernetes, and that the platform can change a cross-cutting default in one place. Concretely: a Helm library chart encoding the correct defaults — probe timings, shutdown coordination, security context, topology spread, PDB, management-port scraping, NetworkPolicy skeleton — so a service’s own chart is thirty lines of values rather than three hundred lines of YAML. Behind it, a base image per runtime owned centrally, a CI template that builds with Jib and signs, and admission control that only admits signed images from our registry. Configuration convention: mounted properties file rather than environment variables, with the build-time versus runtime distinction from Chapter 4 documented and audited, because that is the one that produces silent misconfiguration. Migrations as a standard pre-upgrade hook in the library chart. Observability wired by default — metrics scraped from the management port, trace ids in logs, a per-service dashboard generated from the same values. What I would explicitly *not* do is build an abstraction that hides Kubernetes entirely, because the first non-standard requirement then has nowhere to go and teams end up fighting the platform; the library chart should be escapable. The measure of success is time from a new repository to production traffic, and the number of services on the current chart version — if the latter decays, the platform is not being maintained.

Q18. Argue against Kubernetes for a team of six running three services. The cost is mostly invisible until you own it. Three services do not need a scheduler, a service mesh, an ingress controller, cert-manager, a metrics stack, a secrets integration and a cluster upgrade cadence — but a Kubernetes deployment brings all of them, and a team of six will spend a meaningful fraction of its capacity operating that rather than building product. Cluster upgrades every few months, CVE response for the control plane, node lifecycle, and debugging failures that are in the platform rather than the application are all real ongoing work. Meanwhile the benefits Kubernetes is famous for — bin-packing across a large fleet, self-healing at scale, uniform deployment across many teams — accrue at a size this team does not have. The alternatives are genuinely good now: a managed container platform such as Cloud Run, App Runner or Container Apps gives you rolling deploys, autoscaling including scale-to-zero, TLS and health checks with almost no operational surface, and Quarkus’s fast startup makes scale-to-zero actually work. My recommendation would be to stay on a managed platform until either the service count or the team size grows enough that the uniformity is worth the operational cost, and to keep the applications container-native so the move is available later. What I would resist is adopting Kubernetes because it is the expected answer; the honest question is who will run it at 3 a.m.

20.13 Cheat Sheet

Quarkus extension

```
quarkus.kubernetes.replicas=3
quarkus.kubernetes.resources.requests.memory=384Mi
quarkus.kubernetes.resources.limits.memory=768Mi
quarkus.kubernetes.readiness-probe.http-action-path=/q/health/ready
quarkus.kubernetes.deploy=true
```

Shutdown coordination

```
quarkus.shutdown.delay=5S
quarkus.shutdown.timeout=30S
```



```
terminationGracePeriodSeconds: 45      # > delay + timeout
strategy: {rollingUpdate: {maxSurge: 1, maxUnavailable: 0}}
```

Resources

```
requests: {memory: "384Mi", cpu: "250m"}
limits:   {memory: "768Mi"}              # no CPU limit
```

Config as a file (ordinal 260)

```
volumeMounts: [{name: config, mountPath: /deployments/config, readOnly: true}]
volumes: [{name: config, configMap: {name: orders-app-config}}]
```

Trigger a rollout on config change

```
annotations:
  checksum/config: "${{ ... | sha256sum }}"
```

KEDA for Kafka

```
triggers:
  - type: kafka
    metadata: {consumerGroup: orders-processor, topic: orders, lagThreshold: "1000"}
# maxReplicaCount <= partition count
```

Debugging

```
kubectl describe pod <p>                # events + last termination reason
kubectl logs <p> --previous
kubectl port-forward <p> 9000:9000
kubectl debug -it <p> --image=nicolaka/netshoot --target=app
```

```
137 OOMKilled   143 SIGTERM   CrashLoopBackOff -> logs --previous
```

20.14 Summary

Kubernetes rewards precision in a small number of settings and punishes defaults everywhere else.

The one that surprises people is **CPU limits**. CPU is compressible, so a limit does not protect you from anything a request does not — but CFS throttling stalls a container for the remainder of a 100 ms window the moment it exhausts its quota, which for a JVM's bursty GC and JIT threads produces latency spikes that are easily misdiagnosed as garbage-collection problems. Set requests, be sceptical of limits, and watch the throttling metric.

The one that causes an incident on every deploy until fixed is the **shutdown handshake**. **SIGTERM** and endpoint removal happen concurrently, so a pod must keep serving through a short delay, then drain, and the grace period must exceed both — otherwise every rollout drops requests.

Beyond that: mount configuration as a properties file rather than a wall of environment variables, because ordinal 260 gives you real property syntax and removes the env-var mapping bug; add a checksum annotation so a config change actually rolls; run migrations as a pre-upgrade Job with

expand–migrate–contract discipline; and autoscale on a signal that reflects load — requests per second for HTTP, consumer lag for Kafka, never beyond the partition count.

Quarkus’s contribution to all of this is startup time. Aggressive autoscaling, fast rollbacks and scale-to-zero are only viable when a pod becomes useful in under a second, and that is the practical payoff of everything in Chapter 1.

Chapter 21 covers **OpenShift** — what it adds over vanilla Kubernetes, S2I builds, Routes, and the security constraints that catch teams migrating from a permissive cluster.

20.15 References

- Deploying to Kubernetes — <https://quarkus.io/guides/deploying-to-kubernetes>
- Kubernetes config extension — <https://quarkus.io/guides/kubernetes-config>
- Kubernetes client — <https://quarkus.io/guides/kubernetes-client>
- Pod lifecycle and termination — <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- Managing resources for containers — <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Horizontal Pod Autoscaler — <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- KEDA — <https://keda.sh/>
- CPU limits and throttling — <https://kubernetes.io/blog/2023/05/08/kubernetes-1-27-cpu-throttling/>

OpenShift

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · [quarkus-openshift](#) · OpenShift 4.16 · S2I

Learning Objectives

By the end of this chapter you will be able to:

1. State precisely what OpenShift adds over vanilla Kubernetes, and what it takes away.
2. Deploy a Quarkus service with the [quarkus-openshift](#) extension using S2I, Docker or binary builds.
3. Work with **Security Context Constraints** — the single most common source of “it worked on my cluster” failures.
4. Use [DeploymentConfig](#) versus [Deployment](#) and know why the former is deprecated.
5. Expose services with [Route](#), including edge, passthrough and re-encrypt TLS.
6. Use ImageStreams for automated rollouts on image change.
7. Integrate with OpenShift’s built-in monitoring, logging and service mesh.
8. Migrate a working Kubernetes deployment to OpenShift without surprises.
9. Answer 15 interview questions on OpenShift specifics.

21.1 What OpenShift Actually Is

OpenShift is Kubernetes plus an opinionated platform layer. Everything from Chapter 20 still applies — Deployments, Services, ConfigMaps, probes, HPAs are all identical. What OpenShift adds:

Capability	Kubernetes	OpenShift
Ingress	Ingress + a controller you install	Route (built in, HAProxy)
Image builds	external CI	BuildConfig , S2I, in-cluster
Image references	registry URLs	ImageStream , with change triggers
Security defaults	permissive	SCC , restricted by default
Multi-tenancy	Namespaces	Projects with quotas and RBAC defaults
Monitoring	install Prometheus	built-in cluster monitoring
Registry	external	integrated internal registry
Console	optional dashboard	full developer and admin console
Service mesh	install Istio	OpenShift Service Mesh (Istio)

Capability	Kubernetes	OpenShift
Serverless	install Knative	OpenShift Serverless (Knative)

The practical summary for an interview: OpenShift is Kubernetes with the batteries you would otherwise assemble yourself, plus a materially stricter security posture. The strictness is where migrations go wrong — a container that runs happily on a vanilla cluster frequently fails on OpenShift because it assumes a fixed UID or wants to write to its own filesystem.

21.2 Deploying with the Extension

```
./mvnw quarkus:add-extension -Dextensions='openshift'
```

```
quarkus.openshift.route.expose=true
quarkus.openshift.route.tls.termination=edge
quarkus.openshift.route.tls.insecure-edge-termination-policy=Redirect
```

```
quarkus.openshift.replicas=3
quarkus.openshift.deployment-kind=Deployment      # not DeploymentConfig - see 21.4
```

```
quarkus.openshift.resources.requests.memory=384Mi
quarkus.openshift.resources.requests.cpu=250m
quarkus.openshift.resources.limits.memory=768Mi
```

```
quarkus.openshift.liveness-probe.http-action-path=/q/health/live
quarkus.openshift.readiness-probe.http-action-path=/q/health/ready
quarkus.openshift.startup-probe.http-action-path=/q/health/started
```

```
quarkus.openshift.env.secrets=orders-secrets
quarkus.openshift.env.configmaps=orders-config
```

```
quarkus.openshift.labels."app.openshift.io/runtime"]=quarkus
quarkus.openshift.annotations."app.openshift.io/connects-to"]=orders-db
```

```
# Build strategy
quarkus.openshift.build-strategy=docker      # docker | s2i | binary
quarkus.container-image.group=platform
quarkus.container-image.name=orders-service
quarkus.container-image.tag=${quarkus.application.version}
```

```
oc login https://api.cluster.acme.com:6443
oc new-project platform
```

```
./mvnw package      # generates target/kubernetes/openshift.yml
./mvnw package -Dquarkus.openshift.deploy=true  # build and deploy in-cluster
```

```
oc get pods -w
oc logs -f deployment/orders-service
oc get route orders-service
```

21.3 Build Strategies

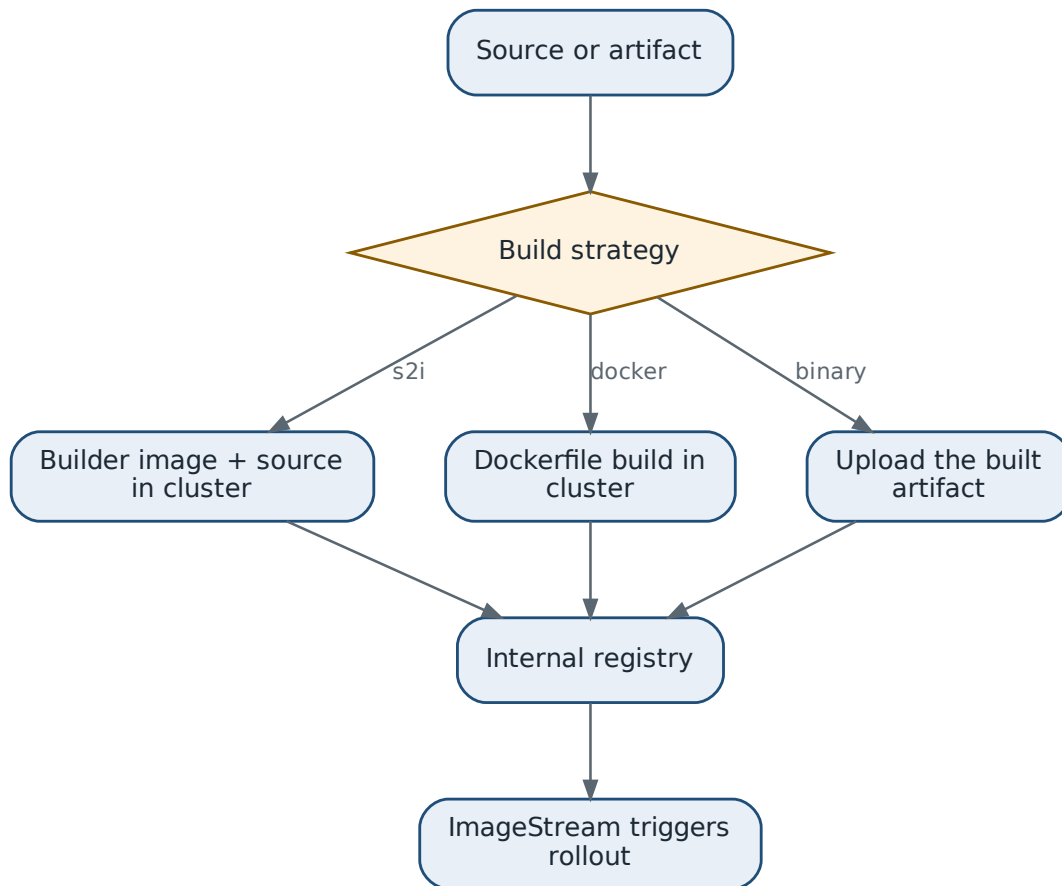


Figure 21.1 — All three strategies produce an image in the internal registry; an ImageStream trigger then rolls the Deployment.

S2I (Source-to-Image). OpenShift pulls your source, injects it into a builder image, runs the build inside the cluster and produces a runtime image. No external CI needed and no Dockerfile.

```
quarkus.openshift.build-strategy=s2i
quarkus.openshift.base-jvm-image=registry.access.redhat.com/ubi9/openjdk-21:1.20
quarkus.openshift.base-native-image=quay.io/quarkus/ubi-quarkus-mandrel-builder-image:jdk-21
```

Docker. OpenShift builds from your Dockerfile inside the cluster.

Binary. You build locally or in CI and upload the artifact; OpenShift only wraps it in an image. Fastest, and the one that fits an existing CI pipeline.

```
oc new-build --binary --name=orders-service \
  --image-stream=openjdk-21-ubi9 -l app=orders-service
oc start-build orders-service --from-dir=target/quarkus-app --follow
```

For a mature organisation, build in CI and use the binary strategy or an external registry. In-cluster builds consume cluster capacity, are harder to cache well, and split your build story across two systems — you end up with some services built by Tekton or GitHub Actions and some by BuildConfigs, with different provenance and scanning stories. S2I is genuinely excellent for getting a developer productive in ten minutes, and for teams without a CI platform. It is not a good place for an organisation's build pipeline to live long-term.

21.4 Deployment versus DeploymentConfig

DeploymentConfig is OpenShift's original controller, predating Kubernetes Deployments. It offers image-change triggers, lifecycle hooks and custom deployment strategies.

```
quarkus.openshift.deployment-kind=Deployment          # recommended
# quarkus.openshift.deployment-kind=DeploymentConfig  # deprecated
```

Use Deployment. **DeploymentConfig** is deprecated as of OpenShift 4.14, its rollouts are driven by a controller in the master rather than by the kubelet (which behaves worse during control-plane disruption), and it is not portable to any other Kubernetes distribution. The features it uniquely offered have equivalents:

DeploymentConfig feature	Deployment equivalent
Image change trigger	<code>image.openshift.io/triggers</code> annotation, or Argo CD / Flux
Pre/post lifecycle hooks	Init containers, or a Job as a Helm hook
Custom deployment strategy	Argo Rollouts, or a service mesh for canary

```
# Image-change trigger on a plain Deployment
metadata:
  annotations:
    image.openshift.io/triggers: |
      [{"from":{"kind":"ImageStreamTag","name":"orders-service:1.4.0"},
        "fieldPath":"spec.template.spec.containers[0].image"}]
```

21.5 Security Context Constraints

This is where migrations fail, and it is the most useful OpenShift-specific knowledge to have.

An SCC is a cluster-level policy controlling what a pod may do: which UIDs it may run as, which volumes it may mount, whether it may run privileged, which capabilities it may hold. Every pod is admitted under an SCC, and the default for application workloads is `restricted-v2`.

```
oc get scc
oc describe scc restricted-v2
oc adm policy who-can use scc restricted-v2
```

`restricted-v2` enforces, among other things:

- **A random UID** from the project's assigned range (something like 1000700000), **not** the UID in your Dockerfile.
- **GID 0** as the primary group.
- No privilege escalation, all capabilities dropped, `RuntimeDefault` seccomp.
- No host network, host ports, host paths.

21.5.1 The random UID consequence

Your image's `USER 1001` is ignored. The container runs as an arbitrary high UID that does not exist in `/etc/passwd`, and it is a member of the **root group (GID 0)**. Therefore every file and directory your application writes to must be **group-writable and owned by GID 0**.

```
# Correct for OpenShift
FROM registry.access.redhat.com/ubi9/openjdk-21-runtime:1.20

COPY --chown=185:root target/quarkus-app/lib/ /deployments/lib/
COPY --chown=185:root target/quarkus-app/*.jar /deployments/
COPY --chown=185:root target/quarkus-app/app/ /deployments/app/
COPY --chown=185:root target/quarkus-app/quarkus/ /deployments/quarkus/

# Make writable paths group-writable - the arbitrary UID is in GID 0
RUN mkdir -p /deployments/tmp && \
    chgrp -R 0 /deployments && \
    chmod -R g=u /deployments

USER 185
EXPOSE 8080
ENTRYPOINT ["/opt/jboss/container/java/run/run-java.sh"]
```

`chmod -R g=u` is the idiom: it gives the group the same permissions as the owner, so whatever UID OpenShift assigns can read and write.

Red Hat UBI images already do this, which is a strong practical reason to use them on OpenShift.

21.5.2 Symptoms and diagnosis

Symptom	Cause
Permission denied writing to a directory	Not group-writable; <code>chmod -R g=u</code>
<code>id: cannot find name for user ID 1000700000</code>	Expected — the UID is not in <code>/etc/passwd</code> ; harmless unless code calls <code>getpwuid</code>
Pod will not schedule, <code>unable to validate against any SCC</code>	Requesting something <code>restricted-v2</code> forbids
Works locally, fails on OpenShift	Almost always the UID or a write path

```
oc get pod orders-abc -o yaml | grep -i scc          # which SCC admitted it
oc describe pod orders-abc | grep -A5 Events
```

21.5.3 If you genuinely need more

```
oc create serviceaccount orders-privileged
oc adm policy add-scc-to-user anyuid -z orders-privileged
```

```
spec:
  serviceAccountName: orders-privileged
```

Treat needing `anyuid` as a bug in your image, not a platform limitation. In nearly every case the real problem is a hard-coded UID assumption or a write to a non-group-writable path, and fixing the image is a ten-minute change. Granting `anyuid` weakens the isolation the platform is providing and tends to spread — once one team has it, it becomes the copied answer. The only routinely legitimate cases are third-party images you cannot rebuild.

21.6 Routes

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: orders-service
  annotations:
    haproxy.router.openshift.io/timeout: 60s
    haproxy.router.openshift.io/rate-limit-connections: "true"
    haproxy.router.openshift.io/rate-limit-connections.rate-http: "100"
spec:
  host: orders.apps.cluster.acme.com
  path: /api/orders
  to: {kind: Service, name: orders-service, weight: 100}
  port: {targetPort: http}
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Three TLS modes, and the distinction matters:

Termination	TLS ends at	Use when
<code>edge</code>	the router	The default; cluster traffic is trusted or meshed
<code>passthrough</code>	your pod	The application must see the client certificate (mTLS)
<code>reencrypt</code>	router, then re-encrypted to the pod	Encryption required end to end, but the router must inspect or route on content

```
quarkus.openshift.route.expose=true
quarkus.openshift.route.tls.termination=edge
```

```
quarkus.openshift.route.tls.insecure-edge-termination-policy=Redirect
quarkus.openshift.route.target-port=http
```

21.6.1 Traffic splitting for canaries

```
spec:
  to:          {kind: Service, name: orders-service-v1, weight: 90}
  alternateBackends:
    - {kind: Service, name: orders-service-v2, weight: 10}
```

A genuinely useful capability that vanilla Ingress lacks without a mesh — a weighted split at the router with no extra components.

21.7 ImageStreams

An ImageStream is a named, versioned pointer to images, decoupling “the image my Deployment runs” from a registry URL.

```
oc import-image orders-service:1.4.0 \
  --from=registry.acme.com/platform/orders-service:1.4.0 \
  --confirm --scheduled

oc get is orders-service
oc tag orders-service:1.4.0 orders-service:production    # promote
oc tag orders-service:production orders-service:rollback # save a rollback point
```

The `oc tag` promotion model is a clean deployment pattern: environments track a tag rather than a version, and promoting to production is a tag move rather than a manifest edit. Combined with a change trigger, `oc tag` alone performs the rollout.

```
oc rollout status deployment/orders-service
oc rollout undo deployment/orders-service
```

21.8 Built-in Monitoring and Logging

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: orders-service
  labels: {app.kubernetes.io/name: orders-service}
spec:
  selector:
    matchLabels: {app.kubernetes.io/name: orders-service}
  endpoints:
    - {port: management, path: /q/metrics, interval: 30s}
```

```
# User-workload monitoring must be enabled at cluster level
oc -n openshift-monitoring get cm cluster-monitoring-config -o yaml
```


Once enabled, your Micrometer metrics (Chapter 14) appear in the built-in Prometheus and the developer console's dashboards with no extra deployment. The same applies to logs via the Logging Operator and to traces via the Distributed Tracing Operator (Tempo/Jaeger), so Chapters 14 and 15 largely configure themselves on OpenShift.

```
quarkus.openshift.annotations."prometheus.io/scrape"=true
quarkus.openshift.annotations."prometheus.io/port"=9000
quarkus.openshift.annotations."prometheus.io/path"=/q/metrics
```

21.9 OpenShift Serverless

Knative, pre-integrated. This is where Quarkus native pays off most visibly.

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata: {name: orders-service}
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "0"           # scale to zero
        autoscaling.knative.dev/maxScale: "20"
        autoscaling.knative.dev/target: "100"           # concurrent requests per pod
    spec:
      containers:
        - image: registry.acme.com/platform/orders-service:1.4.0-native
          resources:
            requests: {memory: "64Mi", cpu: "100m"}
            limits:   {memory: "128Mi"}
          readinessProbe:
            httpGet: {path: /q/health/ready, port: 8080}
```

```
quarkus.knative.min-scale=0
quarkus.knative.max-scale=20
quarkus.knative.revision-auto-timestamp=true
```

Scale-to-zero only works if a cold start is fast enough that the first request does not time out. A JVM Quarkus service at 800 ms is borderline; a native binary at 20 ms is comfortable. **This is the single clearest case where native compilation changes what is architecturally possible**, rather than merely saving money.

21.10 Migrating from Kubernetes

Most of a working Kubernetes deployment transfers unchanged. The differences to plan for:

- 1. Image UID and permissions.** The biggest one. Assume a random UID, make everything group-writable with `chmod -R g=u` and `chgrp -R 0`.
- 2. Ingress → Route.** Ingress works on OpenShift and is converted to a Route automatically, but Route gives you traffic splitting and richer TLS. Both can coexist.
- 3. SCC admission.** Anything requesting privileges, host paths, host network or a specific UID will be rejected. Audit your pod security contexts first.

4. **Registry.** Either push to the internal registry or grant an image pull secret for your external one.
5. **NetworkPolicy.** Works identically, though OpenShift SDN and OVN-Kubernetes differ in some edge behaviours.
6. **PodSecurityPolicy-era assumptions.** OpenShift's SCC predates and outlives PSP; Pod Security Admission labels also apply on recent versions.

```
# A quick pre-migration audit
oc adm policy scc-subject-review -f deployment.yaml
```

That command tells you which SCC would admit your pod, or why none would — the fastest way to find migration problems before you hit them.

21.11 Best Practices

1. **deployment-kind=Deployment**, not **DeploymentConfig**.
2. **Build images in CI**, use binary builds or an external registry; keep S2I for onboarding and prototypes.
3. **Design images for an arbitrary UID**: **chgrp -R 0** and **chmod -R g=u**.
4. **Use Red Hat UBI base images** — they already satisfy **restricted-v2**.
5. **Never reach for **anyuid**** without exhausting image fixes first.
6. **Route with **edge** termination and **Redirect**** unless you need mTLS to the pod.
7. **ImageStream tags as promotion handles** — **oc tag** to promote and to roll back.
8. **ServiceMonitor** rather than a bespoke Prometheus deployment.
9. **scc-subject-review** before migrating any workload.
10. **Native images for Knative**, so scale-to-zero is actually usable.
11. **Keep manifests portable**. Prefer plain Kubernetes objects plus a Route, so the deployment is not welded to OpenShift.

21.12 Common Mistakes

1. **Assuming the Dockerfile's **USER** applies.** OpenShift assigns a random UID; hard-coded UID assumptions break.
2. **Writable directories not group-writable.** **Permission denied** at startup, and the message rarely names the real cause.
3. **Requesting **anyuid** to make it work.** Papers over an image bug and weakens isolation.
4. **Using **DeploymentConfig** on new work.** Deprecated and non-portable.
5. **Building in-cluster for production.** Consumes cluster capacity and splits the build story across two systems.
6. **passthrough** termination when **edge** would do. Adds certificate management inside the pod for no benefit.
7. **Deploying a JVM image to Knative with **minScale: 0**.** Cold starts time out; use native.
8. **Forgetting to enable user-workload monitoring.** ServiceMonitors are silently ignored.

9. Treating OpenShift as entirely different from Kubernetes. It is not — 90 % of Chapter 20 applies verbatim.

10. Not running `scc-subject-review` before migrating. Every SCC problem is discovered one pod at a time.

21.13 Interview Questions

Junior

Q1. What is OpenShift? Red Hat’s Kubernetes distribution, adding Routes, in-cluster builds, ImageStreams, stricter security defaults, integrated monitoring and a registry.

Q2. What is a Route? OpenShift’s built-in ingress object, backed by HAProxy, with TLS termination modes and weighted traffic splitting.

Q3. How do you deploy a Quarkus app to OpenShift? Add the `quarkus-openshift` extension and run `./mvnw package -Dquarkus.openshift.deploy=true`, or apply the generated `openshift.yml`.

Q4. What is S2I? Source-to-Image: OpenShift builds a runtime image from your source inside the cluster using a builder image, with no Dockerfile required.

Q5. What is an ImageStream? A named, versioned pointer to images that decouples the Deployment from a registry URL and can trigger rollouts when a tag changes.

Mid-level

Q6. What is an SCC and why does it matter? A Security Context Constraint is a cluster policy governing what a pod may do — UID range, volumes, capabilities, privilege escalation. The default `restricted-v2` runs containers as a random UID in the project’s range rather than the Dockerfile’s `USER`, which is the most common cause of images that work elsewhere and fail on OpenShift.

Q7. Why does my container get “Permission denied” on OpenShift? Because it runs as an arbitrary UID that is a member of GID 0, and your writable directories are owned by a specific user without group write. Fix the image with `chgrp -R 0 /path && chmod -R g=u /path`.

Q8. `Deployment` or `DeploymentConfig`? `Deployment`. `DeploymentConfig` is deprecated as of OpenShift 4.14, is driven by a master-side controller that behaves worse during control-plane disruption, and is not portable. Image-change triggers are available on Deployments via an annotation, and lifecycle hooks map to init containers or Jobs.

Q9. Explain the Route TLS modes. `edge` terminates TLS at the router and speaks plaintext to the pod. `passthrough` forwards the encrypted connection so the pod terminates TLS — needed when the application must see the client certificate. `reencrypt` terminates at the router and opens a new TLS connection to the pod, for end-to-end encryption where the router still needs to route on content.

Q10. How do you expose Quarkus metrics on OpenShift? Create a `ServiceMonitor` selecting the service and pointing at `/q/metrics` on the management port, with user-workload monitoring enabled at cluster level. The metrics then appear in the built-in Prometheus and console dashboards.

Senior

Q11. Plan a migration of 20 Quarkus services from vanilla Kubernetes to OpenShift.

The manifests largely transfer, so the work concentrates in two places. First, images: I would audit every Dockerfile for UID assumptions and non-group-writable paths, and standardise on UBI bases which already satisfy `restricted-v2` — this is the change that determines whether the migration is smooth or a week of one-pod-at-a-time debugging. Second, admission: run `oc adm policy scc-subject-review` against every existing manifest before moving anything, which surfaces host-path mounts, privileged containers and specific-UID requests as a list rather than as a sequence of surprises. Beyond that: convert Ingress to Route where traffic splitting or richer TLS is wanted, decide the registry story (I would keep the external registry and CI pipeline rather than adopt in-cluster builds), and replace any self-installed Prometheus with ServiceMonitors against the built-in stack. I would migrate one low-risk service end to end first, including its rollout and rollback, and write the findings into a shared base image and Helm values before touching the other nineteen. What I would resist is rewriting manifests into OpenShift-specific objects wholesale — keeping them portable Kubernetes plus a Route preserves the option to move again.

Q12. When is OpenShift worth its cost over vanilla Kubernetes? When you would otherwise assemble and operate the same components yourself and would rather buy that. The concrete value is the integration: ingress, registry, builds, monitoring, logging, tracing, mesh and serverless all installed, upgraded and supported together, plus a security posture that is restrictive by default rather than something a team has to configure. For a regulated organisation, having one vendor accountable for the platform's CVEs and upgrades is worth real money, and the SCC model gives auditors something concrete. Against that: licence cost, a heavier control plane, opinionated components you may not want, and a genuine risk of writing manifests that only run on OpenShift. My test is whether the organisation has a platform team with the capacity to run the equivalent stack — if it does, and it values control and portability, vanilla with a curated set of operators is defensible. If it does not, OpenShift buys a platform team's worth of work, and that is usually the honest comparison.

Q13. Design a canary deployment on OpenShift for a Quarkus service. The simplest version needs no extra components: two Deployments and two Services for v1 and v2, and a single Route with `alternateBackends` splitting traffic by weight — start at 95/5, watch the v2 service's error rate and latency percentiles, and shift weight in steps. Because both versions are labelled distinctly, the metrics from Chapter 14 separate cleanly, which is the part that makes a canary meaningful rather than decorative. The promotion and rollback are both weight changes, so recovery is seconds. Where this is insufficient — if I need header-based routing, automatic analysis, or per-request rather than per-connection splitting — I would move to OpenShift Service Mesh or Argo Rollouts, which can automate the analysis against Prometheus queries and roll back without a human. The precondition for any of it is that the two versions can run concurrently, which brings back the expand-migrate-contract discipline for the schema and the requirement that message consumers tolerate both event versions. Canary is a deployment technique that only works if the data layer was designed for it.

21.14 Cheat Sheet

Extension

```
quarkus.openshift.deployment-kind=Deployment
quarkus.openshift.route.expose=true
```

```
quarkus.openshift.route.tls.termination=edge
quarkus.openshift.route.tls.insecure-edge-termination-policy=Redirect
quarkus.openshift.build-strategy=docker
quarkus.openshift.deploy=true
```

CLI

```
oc login <api>          oc new-project platform
oc get pods -w          oc logs -f deployment/orders-service
oc get route            oc rsh deployment/orders-service
oc rollout status deployment/orders-service
oc rollout undo deployment/orders-service
oc tag orders:1.4.0 orders:production
oc adm policy scc-subject-review -f deployment.yaml
oc get pod x -o yaml | grep scc
```

OpenShift-safe Dockerfile

```
COPY --chown=185:root target/quarkus-app/ /deployments/
RUN chgrp -R 0 /deployments && chmod -R g=u /deployments
USER 185
```

Route TLS

edge	TLS ends at the router	(default)
passthrough	TLS ends at the pod	(mTLS to the app)
reencrypt	router terminates, re-encrypts to the pod	

Canary

```
to: {kind: Service, name: v1, weight: 90}
alternateBackends: [{kind: Service, name: v2, weight: 10}]
```

Knative

```
annotations:
  autoscaling.knative.dev/minScale: "0"
  autoscaling.knative.dev/target: "100"
```

21.15 Summary

OpenShift is Kubernetes plus an integrated platform and a stricter default security posture. Almost everything from Chapter 20 applies unchanged, which means the OpenShift-specific knowledge worth having is small and sharply defined.

The item that matters most is **Security Context Constraints**. Under the default `restricted-v2`, your container runs as an arbitrary UID from the project’s range — not the `USER` in your Dockerfile — and belongs to GID 0. Every writable path must therefore be group-owned by root and group-writable, which is what `chgrp -R 0` and `chmod -R g=u` achieve, and what Red Hat’s UBI images already do. Almost every “works on my cluster, fails on OpenShift” report resolves to this.

The rest is worth knowing but straightforward: prefer `Deployment` to the deprecated `Deployment Config`; use Routes for their TLS modes and weighted traffic splitting, which gives you a canary

with no additional components; use ImageStream tags as promotion handles; and let the built-in monitoring stack scrape your Micrometer metrics via a ServiceMonitor rather than deploying your own Prometheus.

Where OpenShift and Quarkus combine best is **Serverless**: Knative scale-to-zero only works when a cold start is fast enough that the first request does not time out, and that is precisely the case native compilation enables.

Chapter 22 steps up from infrastructure to design, covering the **microservice patterns** that Quarkus makes practical — and the ones that are more often cited than justified.

21.16 References

- Deploying to OpenShift — <https://quarkus.io/guides/deploying-to-openshift>
- Quarkus OpenShift extension — <https://quarkus.io/guides/deploying-to-openshift#openshift-extension>
- Managing SCCs — <https://docs.openshift.com/container-platform/latest/authentication/managing-security-context-constraints.html>
- Routes — <https://docs.openshift.com/container-platform/latest/networking/routes/route-configuration.html>
- Source-to-Image — <https://docs.openshift.com/container-platform/latest/cicd/builds/understanding-image-builds.html>
- OpenShift Serverless — <https://docs.openshift.com/serverless/latest/about/about-serverless.html>
- Red Hat UBI — <https://catalog.redhat.com/software/base-images>

Microservice Patterns

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · SmallRye Fault Tolerance · Kafka · OpenTelemetry

Learning Objectives

By the end of this chapter you will be able to:

1. Decide **service boundaries** using criteria you can defend, rather than by decomposing nouns.
2. Apply the resilience patterns — timeout, retry, circuit breaker, bulkhead, fallback, rate limit — in the right order and the right places.
3. Choose between synchronous and asynchronous integration, and explain the coupling each creates.
4. Implement the data patterns: database per service, saga, outbox, CQRS, event sourcing — and know which are usually overkill.
5. Design an API gateway and BFF layer without recreating a monolith at the edge.
6. Handle service discovery, configuration and secrets consistently across an estate.
7. Recognise the distributed monolith and the anti-patterns that lead to it.
8. Evaluate honestly when a **modular monolith** is the better answer.
9. Answer 20 interview questions on distributed system design.

22.1 Boundaries

Everything else in this chapter is downstream of getting boundaries right, and boundaries are the thing most often got wrong.

22.1.1 Bad criteria

- **By technical layer.** An “API service”, a “business logic service” and a “data service” means every feature touches all three, so nothing can be deployed independently. This is a distributed monolith with extra latency.
- **By entity.** An **Order** service, a **Customer** service, a **Product** service. Entities are not capabilities; almost every operation spans several, so every feature becomes a distributed transaction.
- **By team size.** “Two-pizza teams” describes an outcome, not a decomposition rule.
- **By reuse.** “Everyone needs addresses, let us make an Address service.” Shared code is a library. A service is a *runtime* dependency with an availability cost.

22.1.2 Better criteria

Business capability. What the business does: *order management, payments, inventory, fulfilment*. These names appear in the organisation chart, which is not a coincidence.

Bounded context. Where a term has one unambiguous meaning. “Order” in fulfilment is a list of items to pick; in billing it is a set of charges. Those are different models, and forcing them into one shared type is how a service accumulates conditional logic nobody understands.

Rate of change. Code that changes together belongs together. If two “services” are always deployed as a pair, they are one service.

Independent scaling or availability. A component with genuinely different load characteristics or a different availability requirement is a real candidate.

Compliance boundary. Data with different regulatory handling — PCI, PII, health records — is a legitimate reason to separate.

22.1.3 The tests

Ask three questions about any proposed split:

1. **Can it be deployed without coordinating with another team?** If not, the boundary is wrong.

2. **Does a typical feature touch only this service?** If most features span three services, the split is by layer or by entity.

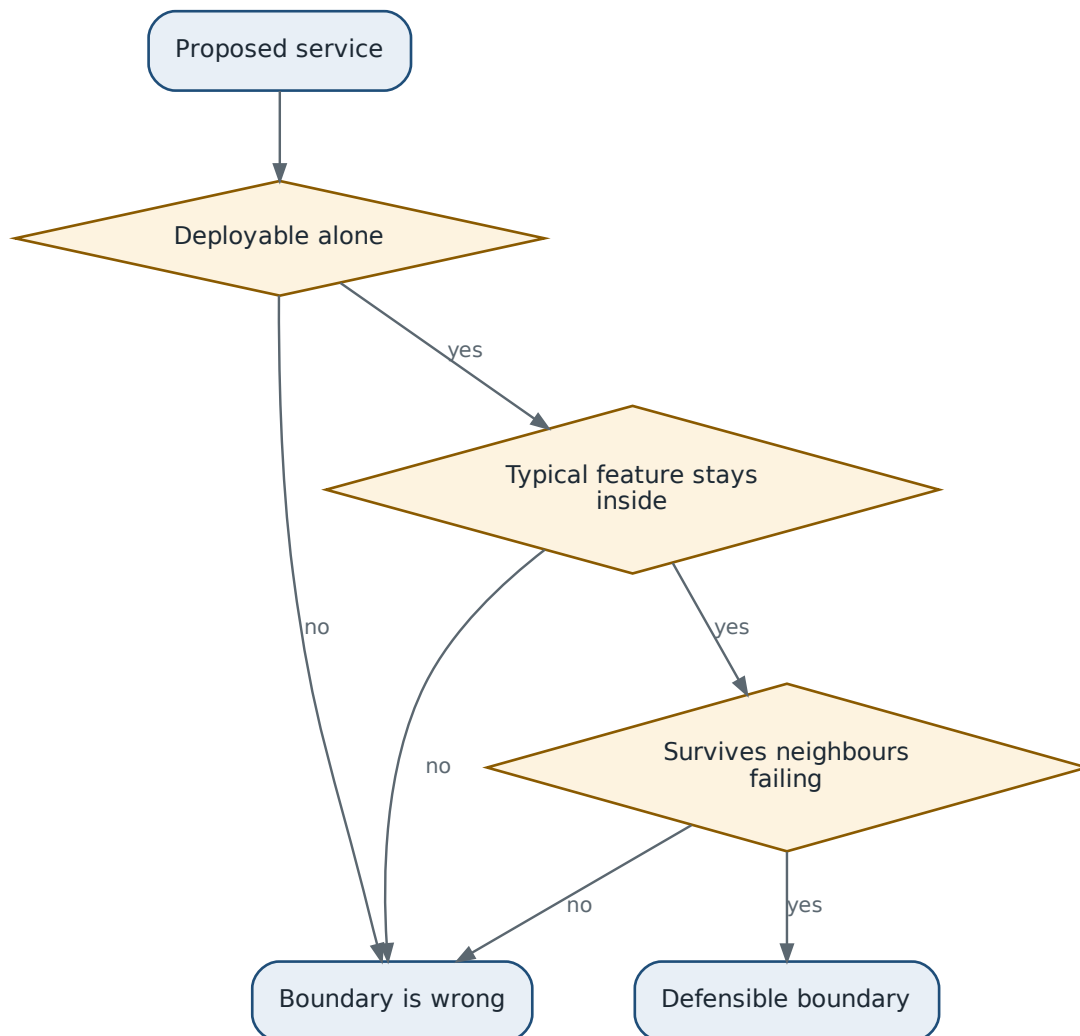


Figure 22.1 — Three questions that catch most bad splits.

22.2 Resilience

22.2.1 The order matters

```

@RegisterRestClient(configKey = "inventory-api")
public interface InventoryClient {

    @GET
    @Path("/stock/{sku}")
    @Timeout(value = 2, unit = ChronoUnit.SECONDS)           // 1. bound the wait
    @Retry(maxRetries = 2, delay = 200, jitter = 100,        // 2. absorb transients
        retryOn = {TimeoutException.class, ServerErrorException.class},

```

```

        abortOn = {BadRequestException.class, NotFoundException.class})
    @CircuitBreaker(requestVolumeThreshold = 20,           // 3. stop hammering
        failureRatio = 0.5,
        delay = 10, delayUnit = ChronoUnit.SECONDS,
        successThreshold = 3)
    @Bulkhead(value = 10)                                // 4. cap concurrency
    @Fallback(fallbackMethod = "cachedStock")            // 5. degrade
    StockLevel stock(@PathParam("sku") String sku);

    default StockLevel cachedStock(String sku) {
        return StockLevel.unknown(sku);
    }
}

```

The interceptors nest outermost-to-innermost: **Fallback** → **CircuitBreaker** → **Timeout** → **Bulkhead** → **Retry**. So a retry happens *inside* the timeout budget, and the circuit breaker counts the final outcome after retries. That has a consequence worth internalising: **your retries must fit inside your timeout**, or the last attempt can never complete.

```
@Timeout(2s) + @Retry(maxRetries=2, delay=200ms)
    attempt 1 (up to ~500ms) + 200ms + attempt 2 + 200ms + attempt 3  <= 2s
```

If each attempt could take 2 seconds, three attempts cannot fit in a 2-second budget and the retry configuration is decorative.

22.2.2 Timeout

The most important and most often omitted. A call with no timeout holds a thread until the TCP stack gives up, which can be minutes.

```
# Per-client, from configuration so it can be tuned without a deploy
quarkus.rest-client.inventory-api.connect-timeout=2000
quarkus.rest-client.inventory-api.read-timeout=3000
```

```
InventoryClient/stock/Timeout/value=2000
InventoryClient/stock/Retry/maxRetries=2
```

Budgets must nest. If your endpoint has a 3-second SLO and calls three services sequentially, they cannot each have a 2-second timeout. Derive per-call budgets from the remaining request deadline.

22.2.3 Retry

```

@Retry(maxRetries = 3,
    delay = 200, delayUnit = ChronoUnit.MILLIS,
    jitter = 100,                                // spread the herd
    retryOn = {TimeoutException.class, ConnectException.class},
    abortOn = {BadRequestException.class, NotFoundException.class})

```

Three rules:

Only idempotent operations. **GET**, **PUT**, **DELETE** are safe. **POST** needs an idempotency key that the server deduplicates on, or you will charge someone twice.

Only retryable failures. Retrying a 400 or a 404 is waste; retrying a 500 or a timeout may help.

Always jitter. Without it, every client that failed at the same instant retries at the same instant — a synchronised thundering herd that can keep a recovering service down.

22.2.4 Circuit breaker

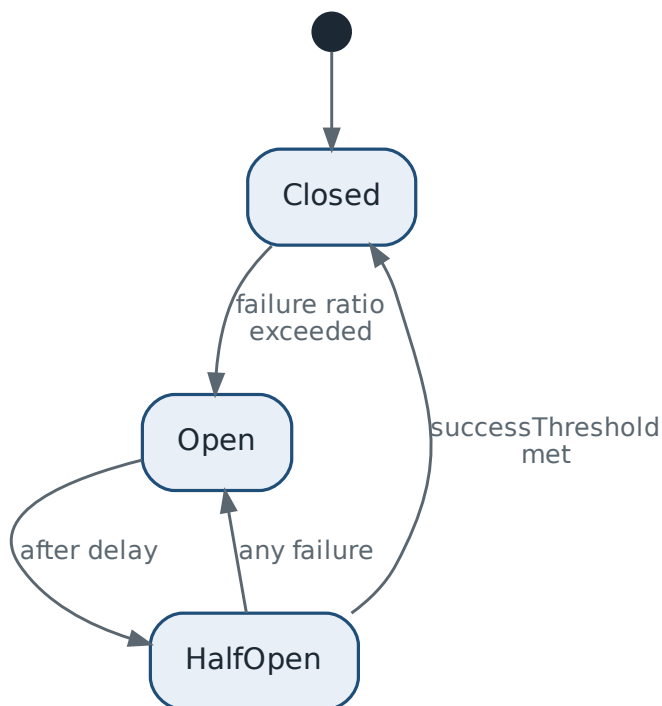


Figure 22.2 — Circuit breaker states.

Closed — calls pass through, outcomes are counted. **Open** — calls fail immediately without touching the network. **Half-open** — a few trial calls decide whether to close.

The purpose is twofold and worth stating precisely: it stops *you* wasting threads and latency budget on a dependency that is already failing, and it sheds load so the dependency can recover. A readiness check does neither (Chapter 16).

```

@CircuitBreaker(
    requestVolumeThreshold = 20,    // minimum sample before judging
    failureRatio = 0.5,           // 50% of the last 20
    delay = 10000,                // stay open 10s
    successThreshold = 3)         // 3 trial successes to close
  
```

Set `requestVolumeThreshold` high enough that a handful of unlucky calls during a deploy do not trip it. A threshold of 4 with a ratio of 0.5 means two failures open the circuit, which will happen by chance.

22.2.5 Bulkhead

```
@Bulkhead(value = 10)                // 10 concurrent calls
@Bulkhead(value = 10, waitingTaskQueue = 20) // async: plus a queue
```

Named after ship compartments: isolate resources per dependency so one slow downstream cannot consume the entire worker pool. Without it, a single degraded dependency saturates every thread and takes down endpoints that never call it — the cascading-failure mechanism from Chapter 2.

22.2.6 Rate limiting

Belongs at the gateway or ingress where possible, because traffic you reject there never reaches the service. In-process, use a token bucket in Redis so the limit is global rather than per-replica, with a **fail-open** policy so a Redis outage does not become a total outage.

22.3 Communication

22.3.1 Synchronous

```
@RegisterRestClient(configKey = "pricing-api")
@AccessToken                // propagate the caller's identity
public interface PricingClient {
    @POST Uni<Price> price(PricingRequest request);
}
```

Simple, immediately consistent, easy to reason about — and it couples availability. If A calls B synchronously, A's availability is at most B's. Chain four services at 99.9 % each and the composite is 99.6 %, which is roughly three hours of downtime a month you did not budget for.

22.3.2 Asynchronous

```
@Transactional
public Order create(CreateOrderRequest request) {
    Order order = persist(request);
    OutboxEvent.record("Order", order.id, "OrderCreated", json(order));
    return order;                // returns without waiting for anyone
}
```

Temporally decoupled — the consumer can be down and the work still happens later. The cost is eventual consistency, which every downstream read path and every user interface must handle.

22.3.3 Choosing

Use synchronous when	Use asynchronous when
The caller needs the answer to proceed	The caller only needs the work to happen
The operation is a query	The operation is a notification of fact
Latency budget is tight and the call is fast	The consumer may be slow or unavailable
Strong consistency is required	Eventual consistency is acceptable
One consumer	Several consumers, or unknown future ones

The strongest single heuristic: if the caller is publishing a *fact that has already happened* — “the order was placed” — that is an event, and it should be asynchronous. If the caller is issuing a *command whose outcome it needs* — “reserve this stock and tell me if you could” — that is a request, and synchronous is honest. Dressing a command up as an event (“OrderPlacementRequested”) to look event-driven produces the worst of both: coupled semantics with asynchronous debugging.

22.4 Data Patterns

22.4.1 Database per service

The defining structural rule: a service owns its data, and no other service touches its tables. Sharing a database is what turns services into a distributed monolith, because a schema change then requires coordinating deployments.

The costs are real and should be named: no cross-service joins, no cross-service transactions, and data duplication across services. Those costs are the reason the pattern is often applied too eagerly.

22.4.2 Saga

Covered in Chapter 7. A sequence of local transactions with compensating actions, replacing a distributed transaction.

```
@ApplicationScoped
public class CheckoutSaga {

    @Transactional
    public void start(CreateOrderRequest request) {
        Order order = Order.create(request);           // local transaction
        order.persist();
        events.fire(new OrderCreated(order.id));
    }

    void reserveStock(@Observes(during = AFTER_SUCCESS) OrderCreated e) {
        stockClient.reserve(e.orderId());              // emits StockReserved or failure
    }

    @Transactional
    public void onStockFailed(StockReservationFailed e) {
        Order order = Order.findById(e.orderId());
        order.cancel();                                // compensation
    }
}
```

```

        events.fire(new OrderCancelled(order.id));
    }
}

```

Choreography (services react to each other's events) is simpler for two or three steps and becomes very hard to follow beyond that — nobody can answer “what happens when an order is placed?” without tooling. Orchestration (a coordinator drives the steps) centralises the flow, which makes it readable and testable, at the cost of a component that knows about everyone.

Compensations are not rollbacks. You cannot un-send an email; you send an apology. They are domain operations and must be designed with the business.

22.4.3 Transactional outbox

Covered in Chapter 7 and Chapter 11 — the answer to “write to the database and publish an event atomically”. This is the pattern with the best value-to-complexity ratio in the chapter, and it is worth building once as shared infrastructure.

22.4.4 CQRS

Separate the write model from the read model.

```

// Write side: normalised, transactional, invariant-enforcing
@Entity
public class Order extends PanacheEntity {
    public OrderStatus status;
    @OneToMany(mappedBy = "order") public List<OrderLine> lines;
}

// Read side: denormalised, shaped for a specific query, no joins
@Entity
@Table(name = "order_summaries")
public class OrderSummary extends PanacheEntityBase {
    @Id public Long orderId;
    public String customerName;
    public String customerEmail;
    public BigDecimal total;
    public int lineCount;
    public String status;
}

@ApplicationScoped
public class OrderSummaryProjector {
    @Incoming("order-events")
    @Blocking @Transactional
    public void project(OrderEvent event) {
        OrderSummary.update("""
            status = ?1, total = ?2 where orderId = ?3 and updatedAt < ?4
            """, event.status(), event.total(), event.orderId(), event.occurredAt());
    }
}

```

Worth it when reads and writes have genuinely different shapes or scaling needs — a write model enforcing invariants versus a read model serving a dashboard. **Not** worth it as a default: you have doubled the models, added a projection pipeline, and introduced replication lag that every consumer must handle.

A lighter-weight version, and usually the right one: keep one database, use DTO projections (Chapter 5) for reads and entities for writes. That captures most of the benefit with none of the infrastructure.

22.4.5 Event sourcing

Store the sequence of events as the source of truth; derive current state by replay.

```
@Entity
public class OrderEventLog extends PanacheEntity {
    public Long aggregateId;
    public String eventType;
    @Column(columnDefinition = "jsonb") public String payload;
    public Instant occurredAt;
    public long sequenceNumber;
}
```

Genuinely valuable where the *history* is the product: ledgers, audit-critical domains, anything needing time-travel or replay into new read models.

Be sceptical. Event sourcing is the most-cited and least-appropriate pattern in this list. It changes everything: schema migration becomes event versioning, debugging requires replay tooling, deleting a customer's data conflicts with an immutable log and therefore with GDPR, and every developer must learn a model with no mainstream framework support. Adopt it because the domain is genuinely event-shaped — accounting, trading, medical records — not

22.5 API Gateway and BFF

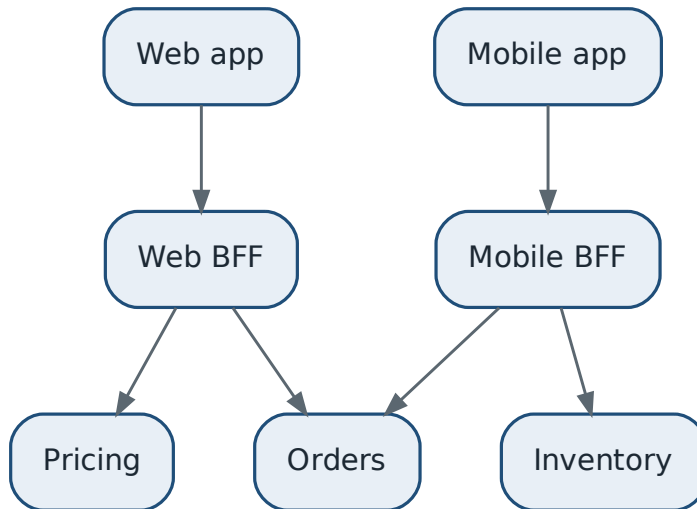


Figure 22.3 — A BFF per client type, each aggregating for its own consumer’s needs.

Gateway — one entry point handling cross-cutting concerns: TLS, authentication, rate limiting, routing, request logging. Use an off-the-shelf one (the ingress, Kong, Envoy) rather than writing one.

Backend-for-frontend — a service per client type that aggregates and shapes responses for that client. A mobile app needs fewer fields and fewer round trips than a web app; a BFF lets each evolve without the downstream services carrying client-specific concerns.

The BFF also solves the token-storage problem from Chapter 9: the browser holds a session cookie, the BFF holds the tokens.

```

@Path("/bff/order-page")
@Authenticated
public class OrderPageResource {

    @RestClient OrderClient orders;
    @RestClient CustomerClient customers;
    @RestClient ShipmentClient shipments;

    @GET
    @Path("/{id}")
    public Uni<OrderPageView> page(@RestPath long id) {
        Uni<Order> order = orders.byId(id);
        return order.chain(o -> Uni.combine().all()
            .unis(customers.byId(o.customerId()), shipments.forOrder(id))
            .with((customer, shipment) -> OrderPageView.of(o, customer, shipment)));
    }
}

```



```
}  
}
```

The BFF anti-pattern is letting it accumulate business logic. A BFF composes and shapes; it does not decide. The moment it contains a pricing rule or a validation the downstream services do not have, you have started rebuilding the monolith at the edge — with the added disadvantage that the logic now lives where no domain team owns it.

22.6 Discovery, Configuration and Identity

Discovery. In Kubernetes, DNS is service discovery. <http://inventory-service.platform.svc.cluster.local> is stable and load-balanced. Do not deploy Consul or Eureka to solve a problem the platform already solves.

Configuration. Chapter 4 plus Chapter 20: mounted ConfigMap at ordinal 260, Secrets or Vault for credentials, build-time properties kept out of environment-specific configuration.

Identity. Chapter 9: propagate the caller's token for user-initiated work, client credentials for system-initiated work, audience per service so a token cannot be replayed. The confused deputy — authenticating a user then calling downstream with a privileged service account — is the failure to design against.

Correlation. A correlation id generated at the gateway, propagated in every call, present in every log line and every span. This is the single highest-value piece of cross-cutting infrastructure in a distributed system, and it costs a filter and a log format.

22.7 The Distributed Monolith

The failure mode this chapter exists to prevent. Symptoms:

- **Services must be deployed together** in a specific order.
- **A schema change requires coordinating several teams.**
- **One service being down takes several others down.**
- **A typical feature touches four services**, so every change is a four-repository pull request.
- **Shared libraries with domain logic** that must be version-bumped everywhere in lockstep.
- **A shared database** that several services write to.

You now have every cost of distribution — network latency, partial failure, operational surface, debugging difficulty — and none of the benefits, because nothing can be changed or deployed independently.

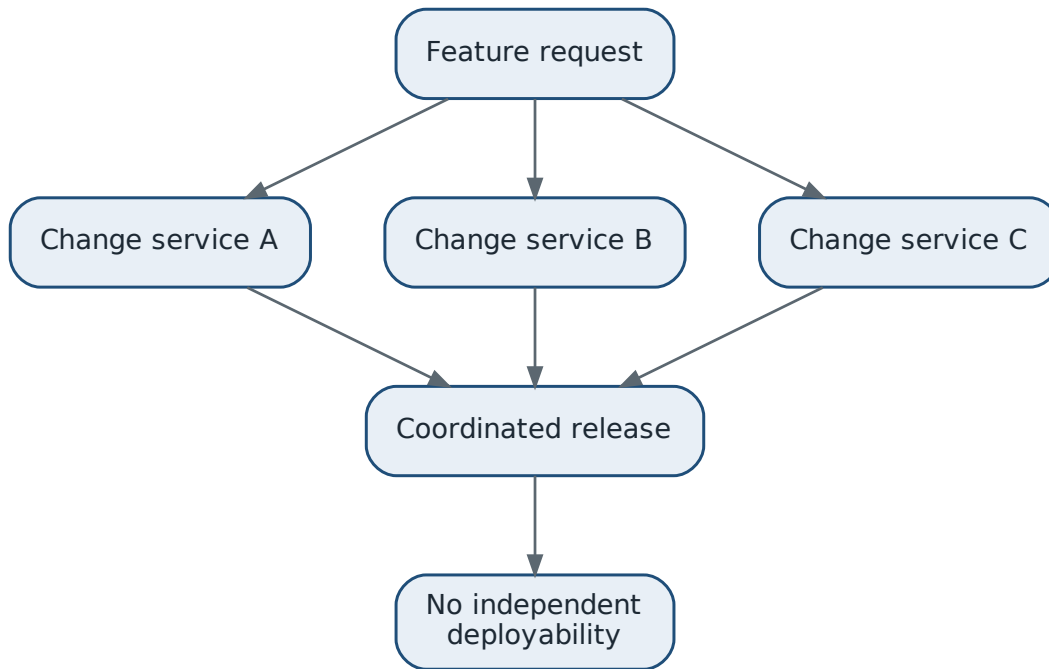


Figure 22.4 — If every feature looks like this, the boundaries are wrong.

The remedy is usually to merge services, not to add more. That is an unpopular recommendation and frequently the correct one.

22.8 The Modular Monolith

Worth stating plainly because the industry’s default advice under-weights it.

A modular monolith is one deployable with strictly enforced internal boundaries — modules with explicit public APIs, no cross-module database access, and communication through in-process interfaces or events.

What you keep:

- **Real transactions.** Invariants enforced by the database rather than by hand-written compensations.
- **One deployment, one log stream, one trace.** Debugging is reading a stack trace.
- **Refactoring across boundaries is a compile-time operation,** not a distributed migration.
- **Massively lower operational cost.**

What you give up:

- Independent deployment and independent scaling.

- Technology heterogeneity.
- Failure isolation between modules.

The honest position: start with a modular monolith and extract services when a specific, named pressure justifies it — a module with genuinely different scaling needs, a team that is blocked by deployment coupling, or a compliance boundary. Extraction from a well-modularised monolith is straightforward precisely because the boundary already exists and has been enforced. Starting with microservices means guessing the boundaries before you understand the domain, and boundaries are the expensive thing to get wrong.

Quarkus supports both equally well. Nothing in the framework pushes you toward distribution.

22.9 Best Practices

1. **Boundaries by business capability**, and apply the three tests before splitting.
2. **Timeout on every outbound call**, with budgets that nest inside the endpoint's deadline.
3. **Retry only idempotent operations, only retryable failures, always with jitter.**
4. **Circuit breaker per dependency**, with a volume threshold high enough to survive noise.
5. **Bulkhead per dependency** so one slow downstream cannot consume the whole pool.
6. **Events for facts, requests for commands.** Do not disguise one as the other.
7. **Outbox for write-plus-publish.** Build it once, share it.
8. **Idempotent consumers everywhere** — delivery is at-least-once.
9. **A BFF composes and shapes; it does not decide.**
10. **Correlation id from the edge**, in every log line and span.
11. **DNS is service discovery.** Do not add a discovery service in Kubernetes.
12. **Consider merging services** when features consistently span several.
13. **Start modular, extract on evidence.**

22.10 Common Mistakes

1. **Splitting by technical layer or by entity.** Every feature spans every service.
2. **No timeout.** One slow dependency exhausts the thread pool and takes the service down.
3. **Retrying non-idempotent operations.** Duplicate charges and duplicate orders.
4. **Retry without jitter.** Synchronised herds prevent recovery.
5. **Circuit breaker threshold too low.** Trips on ordinary noise and turns a blip into an outage.
6. **Synchronous chains four deep.** Availability multiplies downward and latency accumulates.
7. **Shared database across services.** The defining distributed-monolith symptom.
8. **Saga without compensations.** Not a saga — a distributed transaction that silently loses data.
9. **Business logic in the BFF.** The monolith, rebuilt at the edge, owned by nobody.
10. **Event sourcing adopted for its own sake.** Enormous cost, rarely the domain's actual shape.

11. Shared domain libraries version-bumped in lockstep. Compile-time coupling replacing runtime coupling.

12. Adding services to fix problems caused by too many services.

22.11 Interview Questions

Junior

Q1. What is a microservice? An independently deployable service owning a business capability and its own data, communicating over a network.

Q2. What is a circuit breaker? A component that stops calling a failing dependency after a failure threshold, failing fast instead, then periodically testing whether it has recovered.

Q3. Why do you need timeouts? Without one, a call to a hung dependency holds a thread until the TCP stack gives up, which can be minutes. Enough of those exhaust the pool and the service stops serving anything.

Q4. What is an API gateway? A single entry point handling cross-cutting concerns — TLS, authentication, rate limiting, routing — so individual services do not each implement them.

Q5. What is eventual consistency? Data across services converges over time rather than being consistent at every instant. It is the trade you accept when replacing distributed transactions with asynchronous messaging.

Q6. What is a saga? A sequence of local transactions across services, each with a compensating action, replacing a distributed transaction.

Mid-level

Q7. How do you decide service boundaries? By business capability and bounded context, then tested: can it be deployed without coordinating with another team, does a typical feature stay inside it, and can it survive its neighbours being down. Splitting by technical layer or by entity fails all three, because every feature then spans every service.

Q8. Synchronous or asynchronous? Synchronous when the caller needs the answer to proceed — a query, or a command whose outcome matters — accepting that it couples availability. Asynchronous when publishing a fact that has already happened, or when several consumers exist, accepting eventual consistency. The heuristic is fact versus command.

Q9. In what order do the fault-tolerance annotations apply? Fallback, then CircuitBreaker, then Timeout, then Bulkhead, then Retry — outermost to innermost. So retries happen inside the timeout budget, which means the retry schedule must fit within the timeout or the last attempt can never complete.

Q10. What is a bulkhead? A concurrency cap per dependency, so one slow downstream cannot consume the entire worker pool and take down endpoints that never call it.

Q11. What is the distributed monolith? Services that must be deployed together, share a database, or break each other when one is down. You pay every cost of distribution and gain none of the independence. The remedy is usually to merge services rather than add more.

Q12. When is CQRS worth it? When reads and writes have genuinely different shapes or scaling needs — a normalised write model enforcing invariants against a denormalised read model serving dashboards. Not as a default: it doubles the models and adds a projection pipeline with replication lag. DTO projections against one database capture most of the benefit for a fraction of the cost.

Senior

Q13. Design an order-processing system across four services. Boundaries by capability: orders, inventory, payments, fulfilment — each owning its data, with no cross-service database access. Order creation is a local transaction writing the order and an outbox row, so the database write and the event publish are atomic; a CDC publisher drains the outbox to Kafka keyed by customer id, which fixes the ordering guarantee at per-customer. Inventory and payments consume and respond with their own events, so the flow is a choreographed saga with explicit compensations — release the reservation, void the authorisation — designed with the business, since compensations are domain operations rather than rollbacks. Every consumer is idempotent by upsert with a timestamp guard, because delivery is at-least-once and redelivery after a rebalance is inevitable. Synchronous calls exist only where the caller needs an answer — a stock check during checkout — and each carries a timeout, a bulkhead and a circuit breaker with a fallback that degrades rather than fails. A correlation id from the gateway flows through every call, log line and span. The decision I would flag to the team is the partition key, because it determines the ordering guarantee and is expensive to change; and the decision I would flag to the business is how long an order may sit in PENDING, because that is the visible consequence of eventual consistency.

Q14. How do you handle a downstream service that is frequently slow? First establish whether “slow” means high median or a bad tail, because the responses differ: a bad tail is addressed with a tight timeout and a hedged or retried request, whereas a slow median means the dependency is under-provisioned and no client-side pattern will fix it. Assuming it is a tail problem, I would set the timeout at roughly the p99 of the healthy distribution rather than at some round number, so slow calls are cut off before they consume the budget, and add a bulkhead so that even when many calls are slow they cannot take more than a fixed share of the pool. A circuit breaker then stops us paying the timeout at all once failure is sustained, which is the part that protects our own latency. The fallback matters as much as the protection: if a cached or default answer is acceptable, the endpoint degrades instead of failing, and that is a product decision to confirm rather than an engineering one. In parallel I would push back upstream with data — p99 latency and error rate over time — because client-side resilience contains the blast radius but does not fix the dependency, and treating it as permanently acceptable is how a system accumulates degraded modes nobody remembers.

Q15. When would you merge two microservices? When the evidence says they are one service pretending to be two. The signals: they are always deployed together; a typical feature changes both; they share a database or one reads the other’s tables; the calls between them are chatty and synchronous, so neither can survive the other being down; and the team owning them is one team. Any two of those is a strong case. The counter-argument is usually sunk cost or organisational optics — merging looks like a retreat — so I would frame it in terms of what it buys: real transactions instead of a saga, one deployment instead of a coordinated release, and a stack trace instead of a distributed trace. I would keep them as separate modules within one deployable with an enforced boundary, so the split remains available if a genuine pressure appears later. The thing I would want to avoid is merging because integration is painful without first checking whether

the pain is caused by a bad boundary or by missing infrastructure — if the correlation ids and the outbox are missing, the pain is fixable without merging.

Q16. Argue for a modular monolith over microservices. Microservices buy independent deployability, independent scaling and failure isolation, and every one of those is paid for in complexity that lands on the team regardless of whether they needed it. In a modular monolith you keep real transactions, so invariants are enforced by a database that has had decades of work put into them rather than by hand-written compensations that must be designed, tested for every failure ordering and maintained forever. Debugging is a stack trace rather than a correlation-id search across five services and a broker. Refactoring a boundary is a compile-time operation instead of a distributed migration with a deprecation window. And the operational surface is one deployment, one log stream, one dashboard. The critical point is that the expensive mistake in this space is getting boundaries wrong, and boundaries are hardest to judge early, precisely when microservice adoption usually happens. A modular monolith lets you get them wrong cheaply and correct them, then extract a module once a specific pressure names itself — a different scaling profile, a compliance boundary, a team genuinely blocked by deployment coupling. Extraction from an enforced module boundary is straightforward; discovering that your service boundaries are wrong after eighteen months is not. I would still enforce the modules ruthlessly, because a monolith without internal boundaries is the thing everyone is actually trying to escape.

Principal

Q17. Your organisation has 40 services and velocity is falling. Diagnose it. Falling velocity with a growing service count usually means the boundaries are wrong rather than that there is too much code, so I would measure the shape of change before proposing anything. The decisive metric is how many services a typical feature touches: if the median is one, the architecture is working and the problem is elsewhere — tooling, environments, review latency. If it is three or four, we have a distributed monolith and every feature carries coordination cost. I would corroborate with deployment coupling, how often services are released together, cross-team pull requests per feature, and how many services read another's database. Then the remedy is usually consolidation along the seams the data shows: merge services that always change together, and be explicit that this is a correction rather than a failure. Alongside that I would look at whether the pain is boundary-related at all or infrastructural — missing correlation ids, no shared outbox, inconsistent error contracts, and slow environments all masquerade as architectural problems and are much cheaper to fix. I would resist two instincts: adding a platform team to make the current shape bearable without questioning the shape, and adding more services to decouple, which compounds the problem. The uncomfortable recommendation, which is often the right one, is that the estate should be smaller.

Q18. How do you decide which resilience patterns to standardise across an estate? Standardise where an inconsistency costs someone outside the owning team, and make the standard a dependency rather than a document. Timeouts are non-negotiable and should be enforced — a client without one is a latent outage for every caller, so I would ship a REST client wrapper in a platform library that refuses to build without a configured timeout, which turns a review comment into a compile error. Bulkheads and circuit breakers I would provide with sensible defaults per dependency class, configurable per service, because the right thresholds depend on traffic shape and a global number would be wrong everywhere. Retries need the most guardrails, because retrying a non-idempotent operation is a correctness bug rather than a performance one, so the library would default retries off for **POST** and require an explicit idempotency key to enable them.

Fallbacks I would deliberately not standardise: what to serve when a dependency fails is a product decision, and imposing a default produces silent degradation nobody agreed to. On top of that, the platform should provide the observability that makes the patterns legible — circuit-breaker state and bulkhead rejections as metrics with consistent names — because a breaker nobody can see is a breaker nobody tunes. And I would require a test per failure mode, since resilience configuration is written once and never exercised until an incident, at which point it is usually wrong.

22.12 Cheat Sheet

Boundary tests

1. Deployable without coordinating with another team?
2. Does a typical feature stay inside it?
3. Survives its neighbours being down?

Fault tolerance (outermost to innermost)

```
@Fallback → @CircuitBreaker → @Timeout → @Bulkhead → @Retry

@Timeout(value = 2, unit = ChronoUnit.SECONDS)
@Retry(maxRetries = 2, delay = 200, jitter = 100,
      retryOn = TimeoutException.class, abortOn = BadRequestException.class)
@CircuitBreaker(requestVolumeThreshold = 20, failureRatio = 0.5,
               delay = 10000, successThreshold = 3)
@Bulkhead(value = 10)
@Fallback(fallbackMethod = "cached")
```

InventoryClient/stock/Timeout/value=2000

InventoryClient/stock/Retry/maxRetries=2

Sync versus async

fact that happened -> event, asynchronous
 command needing an outcome -> request, synchronous

Data patterns

database per service	structural rule; no cross-service table access	
outbox	write + publish atomically	<- highest value
saga	local transactions + compensations	
CQRS	separate read model	<- often DTO projections suffice
event sourcing	events as source of truth	<- rarely justified

Anti-patterns

split by layer or entity	shared database
sync chain 4 deep	saga without compensations
business logic in the BFF	shared domain library in lockstep

22.13 Summary

Almost every difficulty in a distributed system is downstream of **boundaries**, and boundaries are what teams most often get wrong — by splitting on technical layers, on entities, or on reuse rather

than on business capability and bounded context. The three tests are worth applying literally: independently deployable, features stay inside, survives neighbours failing.

For resilience, the order of the patterns matters as much as their presence. Fallback wraps circuit breaker wraps timeout wraps bulkhead wraps retry, which means **the retry schedule must fit inside the timeout budget** or the configuration is decorative. A timeout on every outbound call is the single highest-value item; retries need idempotency and jitter; circuit breakers need a volume threshold high enough to survive ordinary noise; bulkheads are what stop one slow dependency taking down endpoints that never call it.

For data, one pattern carries most of the value — the **transactional outbox**, which solves “write and publish atomically” and should be built once as shared infrastructure. Sagas replace distributed transactions and require compensations designed with the business. CQRS and event sourcing are frequently cited and rarely justified; DTO projections against one database capture most of CQRS’s benefit for none of its cost.

The failure mode to design against is the **distributed monolith** — services that deploy together, share a database, or break each other. Its remedy is usually merging, not splitting.

And the position worth holding, because the industry’s default advice under-weights it: **start with a modular monolith and extract on evidence**. Boundaries are the expensive thing to get wrong, and a monolith lets you get them wrong cheaply.

Chapter 23 turns to **performance tuning** — measuring properly, the JVM and reactive knobs that actually matter, and the discipline of finding the real bottleneck rather than the convenient one.

22.14 References

- SmallRye Fault Tolerance — <https://quarkus.io/guides/smallrye-fault-tolerance>
- MicroProfile Fault Tolerance — <https://github.com/eclipse/microprofile-fault-tolerance>
- Microservices patterns — <https://microservices.io/patterns/>
- Saga pattern — <https://microservices.io/patterns/data/saga.html>
- Transactional outbox — <https://microservices.io/patterns/data/transactional-outbox.html>
- Backends for Frontends — <https://samnewman.io/patterns/architectural/bff/>
- Modular monolith — <https://www.kamilgrzybek.com/blog/posts/modular-monolith-primer>
- Google SRE Book, addressing cascading failures — <https://sre.google/sre-book/addressing-cascading-failures/>

Performance Tuning

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · JFR · async-profiler · Micrometer · k6 / Gatling

Learning Objectives

By the end of this chapter you will be able to:

1. Apply a **method** to performance work rather than a list of tricks.
2. Design a load test that produces trustworthy numbers, including avoiding coordinated omission.
3. Profile a Quarkus service with JFR and async-profiler, and read a flame graph.
4. Tune the JVM knobs that matter — heap, GC, and the ones that do not.
5. Diagnose the four bottlenecks that account for most real cases: the connection pool, the event loop, N+1, and serialisation.
6. Optimise startup for autoscaling and scale-to-zero.
7. Reduce memory footprint deliberately rather than by guesswork.
8. Decide when native, virtual threads or reactive actually help.
9. Answer 20 interview questions on performance investigation.

23.1 A Method

Performance work goes wrong in a predictable way: someone changes something plausible, the number moves for an unrelated reason, and the change is kept. A method prevents that.

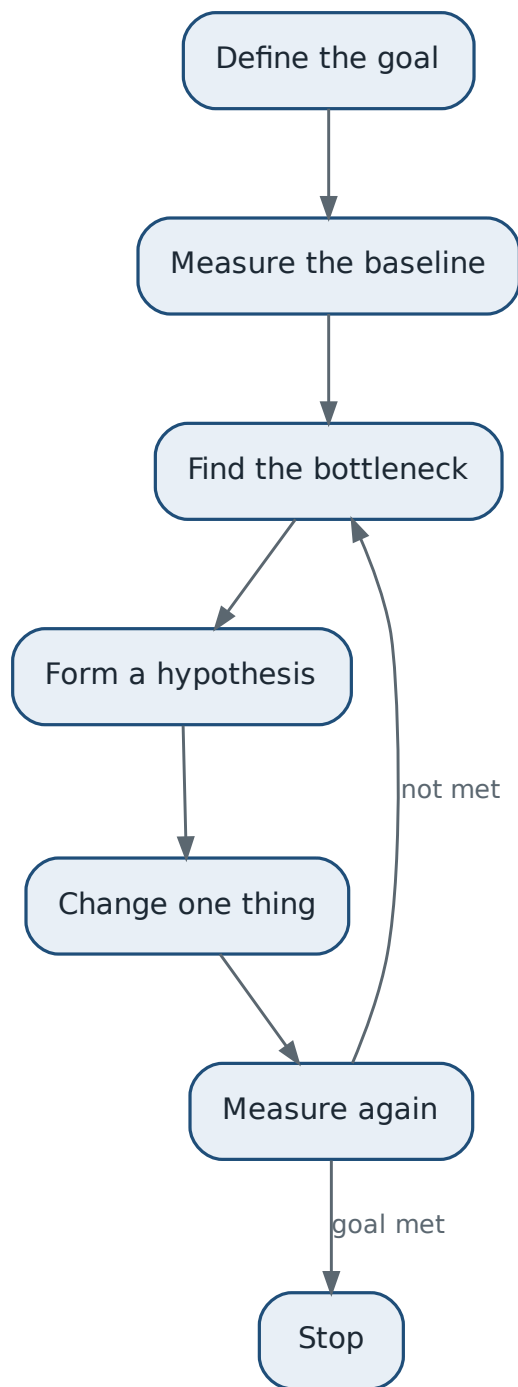


Figure 23.1 — Change one thing, measure, repeat. Stopping is part of the method.

1. **Define the goal numerically.** “Make it faster” is not a goal. “p99 under 200 ms at 2000 rps with 512 MB per pod” is. Without a target you cannot know when to stop, and performance work has no natural end.
2. **Measure a baseline** under realistic load, on realistic hardware, with realistic data volumes.
3. **Find the actual bottleneck.** Profile. Do not guess — intuition about performance is unreliable even for experienced engineers, and the plausible answer is frequently the wrong one.
4. **Change one thing.** Two changes at once means you cannot attribute the result.
5. **Measure again.** Same conditions, same data, enough repetitions to see through noise.
6. **Stop when the goal is met.** Further optimisation costs complexity that will be paid for forever.

The most valuable habit is scepticism about your own hypothesis. The number of teams that have added a cache, switched to reactive or moved to native without first establishing what was slow is large, and the number who discovered afterwards that the bottleneck was a missing index is also large.

23.2 Load Testing Properly

```
// k6: a realistic scenario with a target throughput
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Trend } from 'k6/metrics';

const createLatency = new Trend('create_latency');

export const options = {
  scenarios: {
    steady: {
      executor: 'constant-arrival-rate', // NOT constant-vus - see below
      rate: 2000, timeUnit: '1s',
      duration: '10m',
      preAllocatedVUs: 500, maxVUs: 2000,
    },
  },
  thresholds: {
    'http_req_duration{status:201}': ['p(99)<200'],
    'http_req_failed': ['rate<0.001'],
  },
};

export default function () {
  const res = http.post(`${__ENV.BASE_URL}/orders`, JSON.stringify({
    customerEmail: `user${__VU}@example.com`,
    items: [{ sku: `SKU-${__ITER % 1000}`, quantity: 1 }],
  }), { headers: { 'Content-Type': 'application/json' } });

  check(res, { 'created': (r) => r.status === 201 });
}
```

```
createLatency.add(res.timings.duration);
}
```

23.2.1 Coordinated omission

The most common way a load test lies.

With a closed-loop generator (**constant-vus**), each virtual user sends a request, **waits for the response**, then sends the next. When the server slows down, the generator sends fewer requests — so the slow period is under-sampled and your percentiles look far better than reality.

An open-loop generator (**constant-arrival-rate**) sends at a fixed rate regardless of response times, so a slowdown produces a queue and the latency you measure includes the waiting. That is what your users experience.

Use an open-loop model for anything where you care about latency percentiles. If a report says “p99 was 80 ms” and the test used a fixed number of virtual users, the number is not measuring what it claims to.

23.2.2 Other ways load tests lie

- **Too short.** The JVM needs 30–60 seconds of load before the JIT has compiled the hot path. A two-minute test measures warm-up.
- **Unrealistic data.** A table with 1000 rows behaves nothing like one with 50 million; every index decision changes.
- **No think time.** Real users pause. A test with zero think time measures a different concurrency profile.
- **One endpoint.** Real traffic is a mix, and the mix determines cache behaviour and lock contention.
- **The load generator is the bottleneck.** Check its own CPU before believing its numbers.
- **Testing localhost.** No network latency, no TLS, no ingress, no service mesh.

23.3 Profiling

23.3.1 JFR

```
java -XX:StartFlightRecording=duration=120s,filename=orders.jfr,settings=profile \
-jar target/quarkus-app/quarkus-run.jar
```

Attach to a running process

```
jcmd <pid> JFR.start duration=120s filename=orders.jfr settings=profile
jcmd <pid> JFR.dump filename=orders.jfr
```

```
quarkus.jib.jvm-arguments=-XX:StartFlightRecording=disk=true,maxsize=100m,dumponexit=true
```

JFR is low-overhead (roughly 1–2 % on the **profile** setting) and safe in production. It records allocation, locks, GC, I/O and method samples. Open the recording in JDK Mission Control.

Partial support exists in native mode:

```
quarkus.native.monitoring=jfr
```

23.3.2 async-profiler and flame graphs

```
./profiler.sh -d 60 -f flame.html -e cpu <pid>
./profiler.sh -d 60 -f alloc.html -e alloc <pid>      # allocation
./profiler.sh -d 60 -f lock.html -e lock <pid>        # contention
./profiler.sh -d 60 -f wall.html -e wall <pid>        # includes blocked time
```

Reading a flame graph:

- **Width is time.** A wide frame is where the time goes; depth is only call-stack depth.
- **Look for wide plateaus**, not tall towers.
- **Framework frames at the bottom are normal.** The interesting part is the widest *application* frame.
- **CPU mode misses blocked time.** If the service is slow but CPU is low, use `wall` mode — that is where waiting on I/O, locks and pools shows up.

`-e wall` is the mode to reach for when latency is high and CPU is low, which is the most common shape for a service bottlenecked on a database or a downstream call. A CPU flame graph of such a service looks almost empty and tells you nothing.

23.3.3 What to look at first

```
# Where is the time? Latency by endpoint
histogram_quantile(0.99, sum by (le, uri) (rate(http_server_requests_seconds_bucket[5m])))

# Pool saturation - a leading indicator
agroal_awaiting_count
agroal_blocking_time_average_ms

# GC pressure
rate(jvm_gc_pause_seconds_sum[5m])

# CFS throttling - latency spikes with no application cause
rate(container_cpu_cfs_throttled_seconds_total[5m])
```

23.4 The Four Usual Bottlenecks

In real Quarkus services, most performance problems are one of four things. Check these before anything clever.

23.4.1 The connection pool

Symptom: latency rises with load while CPU stays low; `agroal_awaiting_count` is non-zero.

The pool is a queue in front of the database, and the database has finite concurrency. Enlarging the pool past the database's capacity converts a fast failure into slow queries for everyone.

```
quarkus.datasource.jdbc.max-size=20
quarkus.datasource.jdbc.acquisition-timeout=3S      # fail fast, shed load
quarkus.datasource.jdbc.leak-detection-interval=5M
```

Remember `max-size × replicas` is the real number (Chapter 5). Ten replicas at 20 connections is 200 against a database whose default `max_connections` is 100.

23.4.2 The event loop

Symptom: throughput collapses non-linearly under load; `BlockedThreadChecker` warnings.

```
%dev.quarkus.vertx.max-event-loop-execute-time=1S
```

One blocking call in a reactive path removes a fraction of the server's total I/O capacity, not just one request's (Chapter 2).

23.4.3 N+1 queries

Symptom: latency grows with data volume rather than with traffic; query count scales with result size.

```
quarkus.hibernate-orm.statistics=true
```

Fix with join fetch, entity graphs, `@BatchSize` or — best — DTO projections (Chapters 5 and 6). Assert query counts in tests so the regression cannot come back.

23.4.4 Serialisation and over-fetching

Symptom: high CPU in Jackson frames on a flame graph; large response payloads.

```
// Loads 30 columns and every association to render 4 fields
List<Order> orders = Order.listAll();
return orders.stream().map(OrderSummary::of).toList();

// Loads 4 columns
return Order.find("status", PAID).project(OrderSummary.class).list();
```

```
quarkus.http.enable-compression=true
quarkus.http.compress-media-types=application/json
```

Typically 70–85 % payload reduction on JSON, which is often the largest single latency win for an endpoint returning a list.

23.5 JVM Tuning

23.5.1 Heap

```
quarkus.jib.jvm-arguments=\
-XX:MaxRAMPercentage=75.0,\
-XX:InitialRAMPercentage=50.0,\
-XX:+ExitOnOutOfMemoryError
```

Percentage-based so the setting survives a change to the pod's memory limit; 75 % leaves headroom for metaspace, thread stacks, code cache and direct buffers, which are outside the heap (Chapter 19).

23.5.2 Garbage collector

Collector	Use when
SerialGC	Small heaps (< 512 MB), single CPU, short-lived processes
ParallelGC	Throughput-oriented batch work, pauses acceptable
G1	Default; balanced, heaps above ~1 GB
ZGC	Large heaps and strict pause requirements (sub-millisecond)
Shenandoah	Similar goals to ZGC, different implementation

```
# A small container: SerialGC often beats G1 - less overhead, fewer threads
quarkus.jib.jvm-arguments=-XX:+UseSerialGC,-XX:MaxRAMPercentage=75
```

```
# Large heap, latency-sensitive
quarkus.jib.jvm-arguments=-XX:+UseZGC,-XX:+ZGenerational,-Xmx4g
```

For a typical Quarkus microservice with a 256–512 MB heap, **SerialGC** frequently outperforms **G1**. G1’s concurrent threads and region bookkeeping cost more than they save at that size, and the container may only have one core anyway. This is counter-intuitive enough to be worth measuring rather than assuming — and worth mentioning in an interview, because most candidates reach for G1 or ZGC reflexively.

```
# Look at GC before tuning it
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

23.5.3 Knobs that usually do not matter

- **-XX:+UseStringDeduplication** — helps only if strings dominate the heap; measure first.
- **-XX:+AggressiveOpts** — removed.
- **Tuning NewRatio, SurvivorRatio by hand** — G1 adapts; manual tuning usually makes it worse.
- **Thread stack size** — irrelevant unless you have tens of thousands of platform threads, and if you do, virtual threads are the answer.

23.5.4 Virtual threads

```
@GET
@RunOnVirtualThread
public Dashboard dashboard(@RestPath String id) { ... }
```

```
quarkus.virtual-threads.name-prefix=quarkus-virtual-thread-
```

Virtual threads raise the concurrency ceiling for blocking I/O without a reactive rewrite. What they do **not** do is make anything faster, and they do not remove the connection pool as a constraint — 10 000 concurrent virtual threads still queue on 20 connections. Avoid **synchronized** around blocking I/O, which pins the carrier thread; use **ReentrantLock**.

23.6 Startup Optimisation

Startup matters for autoscaling responsiveness, rollout duration and scale-to-zero.

```
# What is actually slow?
java -Dquarkus.log.category."io.quarkus".level=DEBUG -jar target/quarkus-app/quarkus-run.jar
```

```
@ApplicationScoped
public class StartupTiming {
    void onStart(@Observes StartupEvent event) {
        Log.infof("Startup completed in %d ms",
            ManagementFactory.getRuntimeMXBean().getUptime());
    }
}
```

Levers, roughly in order of impact:

1. Remove unused extensions. Each one adds build steps, beans and startup work. Audit with `quarkus extension list --installed`.

2. Do not open the pool eagerly.

```
quarkus.datasource.jdbc.initial-size=1
quarkus.datasource.jdbc.min-size=1
```

3. Avoid work in `@Startup` beans. Cache warming and schema validation at boot delay readiness; do them lazily or in the background.

4. AppCDS — share pre-parsed class metadata across restarts.

```
quarkus.package.jar.appcfs.enabled=true
```

5. Native. Tens of milliseconds instead of a second (Chapter 18).

If a native binary still takes seconds, the time is in **runtime init** — DNS, TLS handshakes, eager pools, `@Startup` work — not in the framework.

23.7 Memory Optimisation

```
jcmd <pid> GC.heap_info
jcmd <pid> VM.native_memory summary          # needs -XX:NativeMemoryTracking=summary
jcmd <pid> GC.class_histogram | head -30
```

The process is larger than the heap. Budget for:

$RSS \approx \text{heap} + \text{metaspace} + \text{code cache} + \text{thread stacks} + \text{direct buffers} + \text{GC structures}$

Levers:

```
quarkus.datasource.jdbc.max-size=10          # each connection has buffers
quarkus.thread-pool.max-threads=50           # 1 MB of stack each
quarkus.http.limits.max-body-size=1M         # bounds request buffering
quarkus.jib.jvm-arguments=-XX:MaxMetaspaceSize=128m,-XX:MaxDirectMemorySize=64m
```


And the structural ones: DTO projections instead of entity graphs, `Multi` streaming instead of materialised lists, bounded caches (Chapter 13), and pagination everywhere.

Native mode removes metaspace and the JIT's code cache entirely, which is a large part of why RSS drops from ~150 MB to ~30 MB.

23.8 Does It Actually Help?

Change	Typical effect	Actually helps when
Native image	startup 50×, RSS 3–5×	Cold start or memory is the constraint
Virtual threads	higher concurrency	Blocking I/O with many concurrent requests
Reactive rewrite	higher concurrency, backpressure	Threads are the constraint; streaming needed
Caching	fewer expensive operations	Read-heavy, tolerant of staleness
Bigger connection pool	usually nothing	The database has spare capacity
More replicas	linear, up to a shared bottleneck	The bottleneck is per-instance
DTO projections	3–10× on read paths	Loading entities to render a few fields
Compression	70–85 % payload	JSON responses over a real network
GC tuning	0–20 %	Measured GC pressure exists

The two rows to remember are “**bigger connection pool: usually nothing**” and “**more replicas: up to a shared bottleneck**”. Both are the reflexive response to load, and both fail when the constraint is the database — at which point they make things worse by adding contention. Establish *where* the queue is before adding capacity anywhere.

23.9 Best Practices

1. Set a numeric goal, then stop when you reach it.
2. Open-loop load generation, warmed up, with realistic data volumes.
3. Profile before changing anything; use `wall` mode when CPU is low and latency is high.
4. Check the four usual bottlenecks first: pool, event loop, N+1, over-fetching.
5. Change one thing at a time.
6. `MaxRAMPercentage`, not `-Xmx`.
7. Try `SerialGC` for small heaps — measure rather than assume G1.
8. DTO projections on read paths.
9. Compression on JSON responses.
10. Assert query counts in tests so N+1 cannot regress.

11. **Size the pool against the database**, remembering `max-size × replicas`.
12. **Watch `agroal_awaiting_count` and CFS throttling** — both are leading indicators.
13. **Keep the load test in CI** against a fixed environment so regressions are visible.

23.10 Common Mistakes

1. **Optimising without measuring.** The plausible cause is frequently not the real one.
2. **Closed-loop load tests.** Coordinated omission makes percentiles look much better than reality.
3. **Load testing for two minutes.** You measured JIT warm-up.
4. **Testing against 1000 rows.** Index behaviour is completely different at 50 million.
5. **Enlarging the pool to fix slowness.** Moves the queue; adds database contention.
6. **Adding replicas when the bottleneck is shared.** Ten pods contending on one database is slower, not faster.
7. **Caching to hide N+1.** Preserves the design flaw until the data outgrows the cache.
8. **GC tuning by folklore.** Copied flags with no measurement.
9. **CPU flame graph on an I/O-bound service.** Nearly empty; tells you nothing.
10. **Going native or reactive as a performance project.** Neither helps if the constraint is the database.
11. **Not checking CFS throttling.** Latency spikes with no application cause.
12. **Never stopping.** Optimisation past the goal costs complexity forever.

23.11 Interview Questions

Junior

Q1. How do you find a performance problem? Measure first: latency percentiles per endpoint, error rate, and resource metrics. Then profile to find where the time actually goes, rather than guessing.

Q2. What is a percentile and why not use an average? A percentile says what proportion of requests were faster than a value. An average hides the tail — a service with a 20 ms mean can still have a 3-second p99 affecting one request in a hundred, which is what users notice.

Q3. What is garbage collection pause time? The time the application is stopped while the collector works. It appears directly in latency, so a 200 ms pause is a 200 ms outlier for every in-flight request.

Q4. How do you reduce response size? Return only the fields the client needs — DTO projections rather than entities — and enable compression, which typically removes 70–85 % of a JSON payload.

Q5. What is a connection pool and why does its size matter? A set of reusable database connections. Too small and requests queue; too large and you overwhelm the database and exceed its connection limit, especially once multiplied by replica count.

Mid-level

Q6. What is coordinated omission? When a closed-loop load generator waits for each response before sending the next, so a server slowdown causes it to send fewer requests. The slow period is under-sampled and the reported percentiles are far better than what users experience. Use an open-loop, constant-arrival-rate model instead.

Q7. Latency is high but CPU is low. What is happening? Threads are waiting rather than working. The candidates are the connection pool (check `agroal_awaiting_count`), a slow downstream call, lock contention, a blocked event loop, or CFS throttling from a CPU limit. A CPU flame graph will look empty; use `wall` mode, which includes blocked time.

Q8. How do you read a flame graph? Width is time spent, depth is stack depth. Look for wide plateaus rather than tall towers, and focus on the widest application frame rather than the framework frames underneath it. Remember CPU mode excludes blocked time.

Q9. Which garbage collector for a Quarkus microservice? It depends on heap size. For a typical 256–512 MB container heap, SerialGC often beats G1 because G1’s concurrent threads and region bookkeeping cost more than they save at that size, and the container may have only one core. G1 above roughly 1 GB, ZGC when pause requirements are strict and the heap is large. Measure rather than assume.

Q10. How do you reduce startup time? Remove unused extensions, avoid eagerly opening connection pools, move work out of `@Startup` beans, and consider AppCDS. Native compilation takes it to tens of milliseconds. If a native binary is still slow to start, the time is in runtime init — DNS, TLS, eager pools — not the framework.

Q11. Why does a bigger connection pool often not help? Because the pool is a queue in front of the database, and the database has finite concurrency. Past that point a larger pool converts a fast acquisition timeout into slow queries for every client, and total connections are `max-size × replicas`, which can exceed the server’s limit.

Senior

Q12. Walk me through diagnosing a service whose p99 degraded from 100 ms to 800 ms after a release. The fact that it correlates with a release is the strongest signal available, so I would start with the diff rather than with a profiler. First confirm the degradation is real and not a measurement artefact — same load, same data, and check whether traffic or data volume also changed. Then look at query count per request, because the most common release-shaped regression is an N+1 introduced by an innocuous change to a mapper or a DTO; Hibernate statistics answer that in minutes. Next, traces: a distributed trace of a slow request shows immediately whether the extra time is in our service, in a downstream call, or in the database, which narrows forty possibilities to one. If it is in our own code, a wall-clock flame graph before and after the release compared side by side usually identifies the frame. I would also check the boring causes that a release can introduce: a new synchronous call added to a hot path, a blocking call added to a reactive endpoint, a changed pool or timeout setting, or a new eager fetch on an entity. If nothing

in the diff explains it, I would question whether the release is actually the cause — a data-volume threshold crossed at the same time is a classic coincidence.

Q13. How do you design a load test you would trust? It has to reproduce the conditions that matter, and the four things most often wrong are load model, duration, data and topology. Open-loop, constant-arrival-rate generation, because closed-loop hides exactly the degradation I am testing for. Long enough that the JIT has compiled the hot path and the caches have reached steady state — ten minutes rather than two. A dataset of production-like size, because index behaviour and cache hit rates are entirely different at a thousand rows and at fifty million. A traffic mix rather than one endpoint, since the mix determines contention and cache behaviour. Beyond that: run against a realistic topology including the ingress and TLS, not localhost; verify the load generator is not itself saturated; and record server-side metrics alongside client-side latency so I can attribute a slowdown rather than just observe it. I would keep it in CI against a fixed environment so regressions are visible as a trend, and I would treat the absolute numbers as less important than the delta, because a test environment is never production.

Q14. When would you tune the JVM versus change the code? Almost always the code, and I would be suspicious of a proposal that starts with flags. JVM tuning yields perhaps ten to twenty per cent in the good cases, and only where there is measured GC pressure or a genuine mismatch between the collector and the workload; code and query changes routinely yield multiples. So the sequence is: find the bottleneck, and if it is a query, a fetch strategy or an algorithm, fix that first. The cases where JVM tuning is genuinely the right lever are narrow but real — heap sized wrongly for the container so the process is OOM-killed or thrashing; the default collector mismatched to a very small heap, where SerialGC can beat G1; direct memory or metaspace unbounded and growing. Each of those is identifiable from metrics rather than from intuition. What I would push back on hardest is copied flag sets, because they encode someone else’s workload and are impossible to reason about later — every flag in a production command line should have a measurement that justified it.

Q15. Your service is fast in isolation and slow in production. Where do you look? The gap is in everything the isolated test removed, so I would enumerate that rather than re-profile the service. Network and TLS between the client and the pod, and between the pod and its dependencies — a service mesh adds a hop each way. Real data volumes, which change query plans and cache hit rates. Real concurrency, which introduces lock contention and pool queuing that a single-threaded test never sees. Noisy neighbours and CPU throttling, which is invisible locally and shows up as latency spikes with no application cause. Cold caches and cold JIT on freshly-scheduled pods, which matters more if autoscaling is churning. And real dependency latency, since a mocked downstream in a test responds in microseconds. Distributed tracing is the tool that separates these quickly, because it attributes time across the whole path rather than within one process. The general lesson is that “fast in isolation” usually means the test removed the bottleneck, and the useful question is which part of production the test failed to represent.

Principal

Q16. How do you build a performance culture across 40 services? The goal is that performance is a property teams own continuously rather than a project someone runs after an incident, and that requires making the feedback routine and cheap. Concretely: every service publishes latency and error SLIs in a standard form, so a regression is visible on a shared dashboard without anyone instrumenting anything special. Load tests run in CI against a fixed environment on merge, reporting the delta rather than absolute numbers, so a change that doubles query count

fails visibly and immediately — that single control catches more regressions than any amount of review. A shared library provides the assertions that are easy to skip and expensive to miss: query counts, forbidden high-cardinality metric tags, and a check that no endpoint returns an unbounded result set. Platform-level dashboards surface the four usual bottlenecks — pool saturation, event-loop blocking, CFS throttling, GC pressure — with consistent metric names, so a team can self-diagnose without a specialist. Beyond tooling, I would invest in a small number of people who can profile properly and pair with teams during incidents, because flame-graph literacy spreads by apprenticeship rather than documentation. What I would avoid is a central performance team that owns the numbers, because ownership then leaves the teams who can actually fix things, and a quarterly optimisation exercise, which finds problems long after they were cheap to fix.

Q17. A team wants to rewrite in native and reactive to fix performance. Evaluate. Both proposals are solutions in search of a diagnosis, so my first question is what the measured bottleneck is. Native improves startup time and memory footprint and slightly *reduces* peak throughput without profile-guided optimisation, so if the complaint is throughput or latency under sustained load, native is the wrong direction — it would help if the pain is cold starts during autoscaling or per-replica memory cost. Reactive raises the concurrency ceiling when threads are the scarce resource, which is a real but specific condition; if the constraint is a twenty-connection pool, reactive moves the queue and throughput is unchanged, and I have seen a quarter spent discovering that. So before either, I would want profiles and pool metrics, and I would expect them to point at something much cheaper: an N+1, a missing index, over-fetching, or a blocking call on an event loop. If threads genuinely are the constraint, on Java 21 the first thing to try is `@RunOnVirtualThread`, which is a one-line change with none of the readability cost of a reactive rewrite and can be reverted. I would support a reactive rewrite where the service genuinely streams or needs backpressure, and native where cold start or memory is the measured cost — and I would say so explicitly, because the point is not to block the proposals but to ensure they are aimed at something real.

23.12 Cheat Sheet

Method

goal (numeric) -> baseline -> find bottleneck -> one change -> measure -> stop

Load testing

open-loop (constant-arrival-rate)	NOT constant-vus
>= 10 min	realistic data volume
traffic mix	check the generator is not saturated

Profiling

```
jcmd <pid> JFR.start duration=120s filename=o.jfr settings=profile
./profiler.sh -d 60 -f flame.html -e cpu <pid>
./profiler.sh -d 60 -f wall.html -e wall <pid> # low CPU, high latency
./profiler.sh -d 60 -f alloc.html -e alloc <pid>
```

The four usual bottlenecks

- | | |
|--------------------|---|
| 1. connection pool | <code>agroal_awaiting_count > 0</code> |
| 2. event loop | BlockedThreadChecker warnings |
| 3. N+1 | query count scales with result size |
| 4. over-fetching | wide Jackson frames; large payloads |

JVM

```
-XX:MaxRAMPercentage=75.0
-XX:+ExitOnOutOfMemoryError
-XX:+UseSerialGC           # small heaps often beat G1
-XX:+UseZGC                 # large heap, strict pauses
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

Metrics to watch

```
histogram_quantile(0.99, sum by (le,uri) (rate(http_server_requests_seconds_bucket[5m])))
agroal_awaiting_count
rate(jvm_gc_pause_seconds_sum[5m])
rate(container_cpu_cfs_throttled_seconds_total[5m])
```

Does it help?

native	cold start / memory	NOT peak throughput
virtual threads	blocking I/O concurrency	NOT the pool limit
reactive	thread-bound concurrency, backpressure	
projections	3-10x on read paths	<- usually the biggest win
compression	70-85% payload	
bigger pool	usually nothing	

23.13 Summary

Performance work is a method, not a list of settings. Set a numeric goal, measure a baseline under honest load, find the bottleneck by profiling rather than guessing, change one thing, measure again, and stop when the goal is met.

Two measurement traps invalidate more performance work than any tuning mistake. **Coordinated omission** — a closed-loop load generator sends fewer requests when the server slows, so the reported percentiles describe a system under less load than you think. And **CPU profiling an I/O-bound service**, which produces an almost-empty flame graph; when latency is high and CPU is low, the time is in waiting, and `wall` mode is what shows it.

In practice, most Quarkus performance problems are one of four things: the connection pool queueing, a blocked event loop, $N+1$ queries, or over-fetching and serialising data nobody asked for. Check those before anything clever, and note that the two reflexive responses — a bigger pool and more replicas — help only when the bottleneck is not shared, and make things worse when it is.

The single highest-leverage code change on read paths remains DTO projections: not loading entities you will not mutate is routinely a multiple, where GC tuning is a percentage. And be honest about the big architectural levers — native helps cold start and memory but slightly reduces peak throughput; virtual threads and reactive raise the concurrency ceiling but do not widen a twenty-connection pool.

Chapter 24 gathers the book’s practices into a coherent set, and covers migration from Spring Boot with the same evenhandedness.

23.14 References

- Performance measurement in Quarkus — <https://quarkus.io/guides/performance-measure>
- Native image performance — <https://quarkus.io/guides/native-reference>
- JDK Flight Recorder — <https://docs.oracle.com/en/java/javase/21/jfapi/>
- async-profiler — <https://github.com/async-profiler/async-profiler>
- Gil Tene on coordinated omission — <https://www.azure.com/presentation/how-not-to-measure-latency/>
- k6 — <https://k6.io/docs/>
- Brendan Gregg, flame graphs — <https://www.brendangregg.com/flamegraphs.html>
- G1 tuning — <https://docs.oracle.com/en/java/javase/21/gctuning/>

Best Practices and Migration

Stack for this chapter — Java 21 · Quarkus 3.33 LTS · Spring Boot 3.x (for comparison)

Learning Objectives

By the end of this chapter you will be able to:

1. Structure a Quarkus codebase so it stays legible at 50 000 lines.
2. Apply a coherent set of conventions across configuration, errors, observability, security and testing.
3. Keep dependencies and extensions healthy, and upgrade Quarkus without drama.
4. Map Spring Boot concepts to Quarkus accurately.
5. Plan and execute a **migration** — including deciding not to.
6. Use the Spring compatibility extensions as a transitional device rather than a destination.
7. Build a “paved road” so 40 services get the defaults right without 40 decisions.
8. Recognise the anti-patterns this book has accumulated, in one place.
9. Answer 20 interview questions on practice and migration.

24.1 Structuring a Codebase

24.1.1 Package by feature, not by layer

```
com.acme.orders
├── OrdersApplication.java
├── order/                                # a feature owns its whole vertical slice
│   ├── OrderResource.java
│   ├── OrderService.java
│   ├── OrderRepository.java
│   ├── Order.java
│   ├── OrderStatus.java
│   └── api/
│       ├── CreateOrderRequest.java
│       └── OrderView.java
├── pricing/
├── shipping/
├── config/
│   └── OrdersConfig.java
└── shared/
    ├── error/ProblemDetail.java
    └── correlation/CorrelationFilter.java
```


Compare the layered alternative — `resource/`, `service/`, `repository/`, `dto/` — where every feature is smeared across four packages and adding a field means touching all of them. Package-by-feature keeps a change local, makes ownership obvious, and marks the seam along which a service could later be extracted (Chapter 22).

Quarkus is indifferent to package structure — ArC discovers beans from a Jandex index of the whole application (Chapter 3) — so the discipline is entirely yours to impose.

24.1.2 Enforce boundaries

```
@AnalyzeClasses(packages = "com.acme.orders")
class ArchitectureTest {

    @ArchTest
    static final ArchRule featuresDoNotDependOnEachOther =
        SlicesRuleDefinition.slices()
            .matching("com.acme.orders.*..")
            .should().notDependOnEachOther()
            .ignoreDependency(alwaysTrue(), resideInAPackage("..shared.."));

    @ArchTest
    static final ArchRule resourcesDoNotTouchRepositories =
        noClasses().that().haveSimpleNameEndingWith("Resource")
            .should().dependOnClassesThat().haveSimpleNameEndingWith("Repository");

    @ArchTest
    static final ArchRule entitiesStayOutOfTheApi =
        noClasses().that().resideInAPackage("..api..")
            .should().dependOnClassesThat().areAnnotatedWith(Entity.class);

    @ArchTest
    static final ArchRule servicesDoNotThrowWebApplicationException =
        noClasses().that().haveSimpleNameEndingWith("Service")
            .should().dependOnClassesThat()
                .areAssignableTo(WebApplicationException.class);
}
```

ArchUnit turns a convention into a failing build. Without enforcement, boundaries decay — not through malice but because the shortcut is always locally reasonable.

24.2 Code Conventions

Constructor injection. Explicit dependencies, `final` fields, testable without a container (Chapter 3).

Records for DTOs, never entities on the wire (Chapters 2 and 5).

Unchecked domain exceptions, so the default transaction rollback rule is correct (Chapter 7).

Inject a `Clock` wherever behaviour depends on time, so it is testable.

Text blocks for JPQL, SQL and JSON fixtures. A multi-line query you can read is a query with fewer bugs.

Keep **@Scheduled**, **@Incoming** and resource methods **thin** — a two-line adapter over a testable service.

```
// The shape to aim for
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
public class OrderResource {

    private final OrderService orders;

    public OrderResource(OrderService orders) { this.orders = orders; }

    @POST
    @RolesAllowed("order-writer")
    public RestResponse<OrderView> create(@Valid CreateOrderRequest request) {
        OrderView view = orders.create(request);
        return RestResponse.ResponseBuilder
            .create(RestResponse.Status.CREATED, view)
            .location(URI.create("/orders/" + view.id()))
            .build();
    }
}
```

Everything interesting is in **OrderService**; the resource maps HTTP to a call and back.

24.3 Convention Checklist

A consolidated set, drawn from the preceding chapters.

Configuration

```
# @ConfigMapping over @ConfigProperty; validated at startup
# environment-specific values NEVER in build-time properties
# secrets from Secrets or Vault, never in application.properties
quarkus.http.port=8080
quarkus.shutdown.delay=5S
quarkus.shutdown.timeout=30S
%prod.quarkus.log.console.json=true
%prod.quarkus.hibernate-orm.database.generation=validate
%prod.quarkus.flyway.migrate-at-start=false
```

Errors

RFC 7807 problem details with a stable **type** namespace; domain exceptions in services, HTTP knowledge only at the boundary; a catch-all mapper that logs with the trace id and returns it as the customer reference.

Observability

```
quarkus.management.enabled=true
quarkus.management.port=9000
```

```
quarkus.log.console.format=%d{HH:mm:ss} %-5p traceId=%X{traceId} spanId=%X{spanId} [%c{2.}]
↪ %s%e%n
quarkus.otel.traces.sampler=parentbased_traceidratio
```

RED metrics per endpoint, at least one business metric, no identifier-valued metric tags, alerts on symptoms with runbook links.

Security

```
quarkus.security.jaxrs.deny-unannotated-endpoints=true
quarkus.oidc.token.audience=orders-service
```

Token propagation for user-initiated calls; 404 rather than 403 where existence is secret; negative tests for every protected endpoint.

Persistence

`@Enumerated(String)`, lazy `@ManyToOne`, `@Version`, projections on read paths, Flyway with expand-migrate-contract, query-count assertions in tests.

Testing

Plain JUnit by default, shared `@TestProfile` classes, Dev Services rather than H2, `@QuarkusIntegrationTest` against the native binary nightly, a PR gate under a few minutes.

24.4 Dependency and Extension Hygiene

```
quarkus extension list --installed
./mvnw dependency:tree -Dincludes=org.jboss
./mvnw versions:display-dependency-updates
```

Never override a version the BOM manages. The `quarkus-bom` pins a combination that has been tested together; overriding one artifact produces a combination nobody has run (Chapter 1).

```
<!-- WRONG -->
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-hibernate-orm</artifactId>
  <version>3.20.1</version>      <!-- do not do this -->
</dependency>
```

Prefer an extension to a plain dependency. An extension moves initialisation to build time and makes the library native-safe (Chapters 1 and 18). A plain dependency works in JVM mode and becomes your problem when you compile natively.

Remove unused extensions. Each one adds build steps, beans, startup work and native-image surface.

Index shared modules with the Jandex plugin so their beans and entities are discoverable (Chapter 3).

24.5 Upgrading Quarkus

```
quarkus update                                # applies OpenRewrite migration recipes
quarkus update --stream=3.33
./mvnw quarkus:update
```

`quarkus update` does more than bump a property: it applies mechanical migrations — renamed artifacts (the 3.9 `resteasy-reactive` → `rest` rename), deprecated APIs, changed configuration keys. Editing the version property by hand skips all of that and leaves you to discover the differences at runtime.

Follow the LTS stream unless you need something specific from a newer release. LTS versions get 12 months of support and a much calmer upgrade path than tracking every monthly release.

A safe upgrade sequence:

1. Read the release notes and the migration guide for every intervening version.
2. `quarkus update` on a branch.
3. Run the full test suite, including the native integration tests.
4. Deploy to a non-production environment and watch startup logs for deprecation and build-time-property warnings.
5. Canary in production before a full rollout.

Upgrade little and often. A team that upgrades every quarter does a small, mechanical piece of work each time; a team that has skipped two years faces a project. This is the single most common source of “we are stuck on an old version because upgrading is too risky”, and the risk is created by the delay.

24.6 Spring Boot to Quarkus: the Mapping

Spring Boot	Quarkus
<code>@SpringBootApplication</code>	Nothing — no annotation needed
<code>@RestController</code>	<code>@Path</code>
<code>@GetMapping("/x")</code>	<code>@GET @Path("/x")</code>
<code>@RequestBody</code>	the method parameter
<code>@PathVariable</code>	<code>@PathParam</code> / <code>@RestPath</code>
<code>@RequestParam</code>	<code>@QueryParam</code> / <code>@RestQuery</code>
<code>@Component</code> , <code>@Service</code>	<code>@ApplicationScoped</code>
<code>@Autowired</code>	<code>@Inject</code> , or constructor injection
<code>@Value("\${x}")</code>	<code>@ConfigProperty(name = "x")</code>
<code>@ConfigurationProperties</code>	<code>@ConfigMapping</code>
<code>@Profile("dev")</code>	<code>@IfBuildProfile("dev")</code>
<code>@Transactional</code> (Spring)	<code>@Transactional</code> (Jakarta)
<code>JpaRepository<T, ID></code>	<code>PanacheRepository<T></code>

Spring Boot	Quarkus
<code>RestTemplate / WebClient</code>	<code>@RegisterRestClient</code> interface
<code>@Scheduled(cron=)</code>	<code>@Scheduled(cron=)</code>
<code>@Cacheable</code>	<code>@CacheResult</code>
<code>@ControllerAdvice</code>	<code>@ServerExceptionHandler</code>
<code>@Valid</code>	<code>@Valid</code> (identical)
Actuator <code>/health</code>	<code>/q/health</code>
Actuator <code>/metrics</code>	<code>/q/metrics</code>
<code>application.yml</code>	<code>application.properties</code> or <code>.yaml</code>
<code>@SpringBootTest</code>	<code>@QuarkusTest</code>
<code>@MockBean</code>	<code>@InjectMock</code>
Testcontainers by hand	Dev Services

24.6.1 A worked comparison

```
// Spring Boot
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService service;

    public OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    public ResponseEntity<OrderView> get(@PathVariable Long id) {
        return service.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<OrderView> create(@Valid @RequestBody CreateOrderRequest request) {
        OrderView view = service.create(request);
        return ResponseEntity.created(URI.create("/orders/" + view.id())).body(view);
    }
}
```

```
// Quarkus
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class OrderResource {

    private final OrderService service;

    public OrderResource(OrderService service) { this.service = service; }
```

```

@GET @Path("/{id}")
public OrderView get(@RestPath Long id) {
    return service.findById(id)
        .orElseThrow(() -> new OrderNotFoundException(id));
}

@POST
public RestResponse<OrderView> create(@Valid CreateOrderRequest request) {
    OrderView view = service.create(request);
    return RestResponse.ResponseBuilder
        .create(RestResponse.Status.CREATED, view)
        .location(URI.create("/orders/" + view.id()))
        .build();
}
}

```

The shapes are close enough that the mechanical translation is fast. What is *not* mechanical is what makes migrations take longer than expected: build-time versus runtime configuration, bean removal, the blocking/non-blocking execution model, and native-image constraints.

24.7 The Compatibility Extensions

```

<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-di
</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-web
</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-data-jpa
</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-security
</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-cache
</artifactId></dependency>
<dependency><groupId>io.quarkus</groupId><artifactId>quarkus-spring-scheduled
</artifactId></dependency>

```

```

// This compiles and runs on Quarkus
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired OrderRepository repository;

    @GetMapping("/{id}")
    public Order get(@PathVariable Long id) {
        return repository.findById(id).orElseThrow();
    }
}

public interface OrderRepository extends JpaRepository<Order, Long> {

```

```
List<Order> findByStatus(OrderStatus status);    // derived queries work  
}
```

They work by mapping Spring annotations onto Quarkus mechanisms **at build time**, so there is no runtime cost. What they are not is a full Spring implementation — the supported surface is a subset, and anything relying on Spring’s application context, [BeanPostProcessor](#), AOP proxies or auto-configuration is not covered.

24.8 Planning a Migration

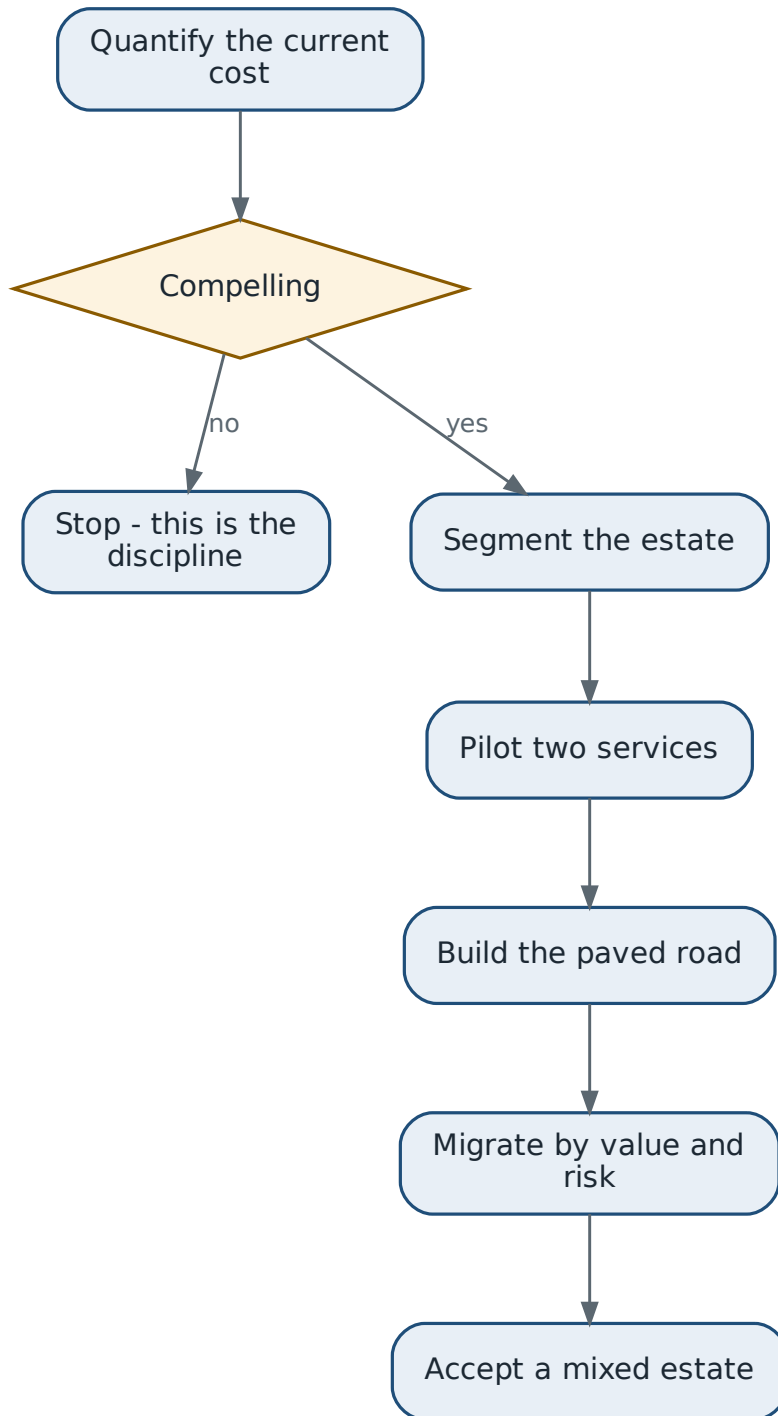


Figure 24.1 — Migration as a measured programme, with an explicit stopping criterion.

1. **Quantify.** Replica counts, memory requests, cold-start impact on autoscaling, deploy duration. Convert to a monthly figure. If it is not compelling, stop — and being willing to say so is the difference between an architect and an enthusiast.
2. **Segment.** Rank by memory footprint $\times$ replica count, and by startup sensitivity. Serverless and burst-scaled services benefit most; a low-traffic admin service benefits least.
3. **Pilot two services.** One straightforward (REST + JPA), one that exercises a hard dependency — a proprietary client, an unusual integration. The second tells you the real cost.
4. **Build the paved road** before service three: an archetype, shared extensions for organisational cross-cutting concerns, CI templates including native builds, dashboards, and a playbook written from the pilots.
5. **Sequence by value and risk.** High-benefit, low-risk first, to bank savings and build confidence.
6. **Accept a permanent mixed estate.** Some services will not be worth migrating. Set the stopping criterion up front so the programme has a defined end.

24.8.1 What actually takes the time

Not the annotation mapping. In order of surprise:

- **Configuration**, especially discovering which properties are build-time fixed (Chapter 4).
- **Testing**, rewriting `@SpringBootTest` suites and learning Dev Services.
- **Bean removal** — `@Unremovable` on anything looked up dynamically (Chapter 3).
- **The execution model** — blocking versus non-blocking, and the incidents caused by getting it wrong (Chapter 2).
- **Libraries with no extension**, which work in JVM mode and need registration for native (Chapter 18).
- **Operational knowledge** — new logs, new metric names, new runbooks, new failure modes for the on-call team.

24.9 The Paved Road

For an estate of any size, the highest-leverage engineering is making the right thing the default.

A shared extension carrying the organisation's cross-cutting concerns: correlation-ID propagation, the RFC 7807 exception mappers, security headers, the OIDC baseline with `deny-unannotated-endpoints`, common metric tags and a cardinality guard, and log redaction. Adoption becomes a dependency rather than a document — which matters, because conventions that live in a wiki decay.

A starter archetype with the structure, conventions, ArchUnit rules and a working test suite.

CI templates doing container build with Jib, signing, SBOM, scanning, and a nightly native verify.

A Helm library chart encoding probes, shutdown coordination, security context, PDB, topology spread and management-port scraping (Chapter 20).

A **shared test library** with the assertions that are easy to skip: query counts, forbidden metric tags, authorisation denials, liveness independence.

Measure adoption rather than police it. A dashboard showing which services are on the current chart and extension versions tells you whether the paved road is being maintained. If adoption decays, the road is not good enough — that is information about the platform, not about the teams.

24.10 Anti-Pattern Catalogue

The book's accumulated failure modes, in one place.

Architecture

- Splitting services by technical layer or by entity.
- A shared database across services.
- Business logic in a BFF.
- Event sourcing adopted without an event-shaped domain.
- Adding services to fix problems caused by too many services.

Quarkus mechanics

- Expecting a build-time property to be runtime-overridable.
- A bean in an unindexed JAR.
- A dynamically-looked-up bean without `@Unremovable`.
- `quarkus.arc.remove-unused-beans=none` as a fix.
- Self-invocation expecting `@Transactional`, `@CacheResult` or `@Timed` to apply.

Runtime

- Blocking an event loop.
- An external call inside a transaction.
- No timeout on a REST client.
- Retrying a non-idempotent operation.
- `@Retry` without jitter.

Data

- `@Enumerated` left at `ORDINAL`.
- Eager `@ManyToOne`.
- N+1 queries.
- Entities as REST DTOs.
- `hbm2ddl=update` in production.
- Unbounded queries.

Operations

- A dependency check in a liveness probe.

- No shutdown delay, so every deploy drops requests.
- CPU limits on a latency-sensitive JVM service.
- `migrate-at-start` with multiple replicas.
- Health endpoints exposed publicly.
- Deploying `latest`.

Observability

- Identifier-valued metric tags.
- Averaging percentiles across pods.
- Alerting on CPU rather than symptoms.
- No business metric.
- No trace id in logs.

Security

- No audience validation.
- Endpoints public by default.
- The confused deputy — user's request, service's credentials.
- Tokens in `localStorage`.
- Roles that encode policy.

Testing

- Everything a `@QuarkusTest`, with a profile each.
- H2 instead of the real database.
- Only happy paths.
- Flaky tests tolerated.

24.11 Interview Questions

Junior

Q1. How should you structure a Quarkus project? By feature rather than by layer, so a change stays local and ownership is obvious. Quarkus does not care about packages — ArC discovers beans from a Jandex index — so the discipline is yours.

Q2. How do you upgrade Quarkus? `quarkus update`, which applies OpenRewrite recipes for renamed artifacts, deprecated APIs and changed config keys — not by editing the version property by hand.

Q3. What is the Quarkus BOM for? It pins a tested combination of versions for every extension and its transitive dependencies. Overriding one by hand produces a combination nobody has run.

Q4. What replaces `@Autowired`? `@Inject`, or better, constructor injection — which needs no annotation when there is a single constructor.

Q5. What replaces `@Value`? `@ConfigProperty`, or `@ConfigMapping` for a group of related properties.

Mid-level

Q6. How do the Spring compatibility extensions work? They map Spring annotations onto Quarkus mechanisms during build-time augmentation, so `@RestController`, `@Autowired` and `JpaRepository` work with no runtime cost. They cover a subset of Spring — nothing relying on the application context, `BeanPostProcessor` or auto-configuration — and are intended to shorten a migration, not to be permanent.

Q7. What is the hardest part of a Spring Boot to Quarkus migration? Not the annotations. It is discovering which configuration properties are build-time fixed, rewriting the test suite, adding `@Unremovable` where beans are looked up dynamically, learning the blocking versus non-blocking execution model, and handling libraries that have no extension when you compile natively.

Q8. How do you keep architectural boundaries from decaying? Make them a failing build. ArchUnit rules asserting that features do not depend on each other, resources do not touch repositories, entities do not appear in API packages, and services do not throw `WebApplicationException`. Conventions that are only documented decay, because the shortcut is always locally reasonable.

Q9. What should be in a `@ConfigMapping` versus `@ConfigProperty`? `@ConfigMapping` for anything beyond a couple of keys: it validates the whole group at startup and reports every failure at once, so in Kubernetes a misconfiguration halts the rollout rather than surfacing at first use.

Q10. Why upgrade little and often? Because the risk of an upgrade grows superlinearly with how far behind you are. A quarterly upgrade is a mechanical `quarkus update` and a test run; a two-year gap is a project, and teams that reach that point conclude upgrading is too risky — a state their own delay created.

Senior

Q11. Plan a migration of 40 Spring Boot services to Quarkus. I would not migrate forty. First quantify the current cost — replica counts, memory requests, deploy duration, cold-start impact on autoscaling — and convert it to a monthly figure; if that number is not compelling, the honest recommendation is to stop, and being willing to say so is the point. If it is compelling, segment by memory footprint times replica count and by startup sensitivity, because the benefit is very unevenly distributed and a handful of services will account for most of it. Pilot two: one straightforward, and one that exercises a hard dependency, because the second tells you the real cost. Then build the paved road — archetype, shared extension, CI templates with native verify, Helm library chart, runbooks — before service three, or every subsequent team re-solves the same problems. Sequence by value and risk to bank savings early, use the compatibility extensions with an explicit removal deadline rather than as a destination, and set a stopping criterion up front so a permanent mixed estate is a planned outcome rather than a failure. Throughout, I would treat the operational learning as a first-class workstream, because the on-call team inherits new logs, new metric names and new failure modes, and that is where migrations actually hurt.

Q12. When would you advise against migrating? When the numbers do not justify it. If services are long-lived, throughput-bound and running at modest replica counts, the memory and startup savings that Quarkus optimises for are not costs anyone is paying, and the migration spends real engineering capacity for a rounding error. Also when the team has deep Spring expertise and the organisation's tooling, security scanning and runbooks are all built around it — fragmenting the

estate into two stacks doubles the surface area of shared libraries, patching, observability integration and onboarding, and that cost is diffuse and permanent while the saving is bounded. And when a critical dependency has no extension and poor native support, if native was the point. My general position is that the strongest case for Quarkus is a new service or a service with a specific measured constraint, not a wholesale replacement — and that a partial migration ending deliberately is a better outcome than a complete one justified after the fact.

Q13. How do you build a paved road that teams actually use? By making the paved road the fastest path, not the mandated one. Concretely that means shipping capability as dependencies rather than documents — a shared extension that registers the exception mappers, correlation filter, security defaults and metric tags automatically, so a service gets the conventions by adding a line to its POM rather than by copying a class. A starter archetype that produces a working service with tests, ArchUnit rules and a Helm values file in one command. CI templates that already do the container build, signing, scanning and nightly native verify. The failure mode I would design against is the platform becoming a cage: every part of it must be escapable, because the first team with a legitimate non-standard requirement will otherwise fork it, and a fork is worse than an exception. I would measure adoption on a dashboard — which services are on the current extension and chart versions — and treat decay as feedback about the platform rather than as non-compliance. And I would keep the platform team small and embedded, pairing with product teams during migrations, because a paved road built without daily contact with its users becomes an abstraction over problems nobody has.

Principal

Q14. Your organisation is split between Spring Boot and Quarkus and it is causing friction. What do you do? The first question is whether the friction is inherent or accidental, because the answer changes the recommendation entirely. Inherent friction — two sets of framework knowledge, two upgrade cadences, two idioms — is real but bounded, and mostly costs at onboarding and when someone moves teams. Accidental friction is much larger and much more fixable: shared libraries that must exist twice, observability conventions that differ so dashboards do not compose, error contracts that vary so clients special-case, and CI pipelines that diverge. In my experience most of the reported pain is the second kind, and it is addressable without consolidating on one framework — standardise the *contracts* rather than the *implementations*. The error envelope, the correlation header, the metric naming, the health endpoints, the container and deployment conventions, the token audience scheme: all of those can be identical across both stacks, and once they are, a Spring service and a Quarkus service look the same to everyone outside the owning team. That is usually the right answer, because consolidating forty services onto one framework is a very expensive way to fix a naming problem. I would only recommend consolidation if the estate is small enough that the migration is weeks rather than quarters, or if one stack has a specific blocking deficiency. And I would be explicit that “two frameworks” is a permanent state to be managed well, not a temporary condition to be resolved — organisations that treat it as temporary tend to under-invest in making it work.

Q15. How do you decide what to standardise across an engineering organisation? The test I apply is whether an inconsistency costs someone outside the owning team. Error response shape does — clients special-case it. Metric names do — dashboards and alerts cannot compose. Correlation headers do — an incident spanning three services becomes a search. Deployment conventions do — the platform team carries the variance. Those get standardised, enforced through shared libraries and CI gates rather than documents, because a convention with no enforcement

mechanism decays to the mean of individual judgement. Conversely, package structure, DTO field naming, whether a service uses Panache active record or repositories, test framework preferences, and internal design all stay local, because the cost of getting them wrong is borne by the team that chose, and standardising them produces lowest-common-denominator outcomes plus resentment. There is a second criterion worth naming: standardise things that are expensive to change later and cheap to get right early — an error contract or a token audience scheme is nearly free to define on day one and painful to retrofit across forty services. Finally, every standard should have an owner, a version and an escape hatch; a standard nobody maintains becomes an obstacle, and one with no exception process gets routed around invisibly, which is worse than an explicit exception.

24.12 Cheat Sheet

Structure

package by feature, not by layer
 ArchUnit rules to enforce boundaries
 constructor injection; records for DTOs; entities never on the wire
 unchecked domain exceptions; inject a Clock

Spring → Quarkus

@RestController	-> @Path
@GetMapping	-> @GET @Path
@Component/@Service	-> @ApplicationScoped
@Autowired	-> @Inject / constructor
@Value	-> @ConfigProperty
@ConfigurationProperties	-> @ConfigMapping
@Profile	-> @IfBuildProfile
JpaRepository	-> PanacheRepository
RestTemplate	-> @RegisterRestClient
@Cacheable	-> @CacheResult
@ControllerAdvice	-> @ServerExceptionMapper
@SpringBootTest	-> @QuarkusTest
@MockBean	-> @InjectMock

Compatibility extensions

quarkus-spring-di -web -data-jpa -security -cache -scheduled
 transitional only, with a removal deadline

Upgrading

```
quarkus update --stream=3.33
# follow LTS; upgrade quarterly; run the native suite before merging
```

Migration programme

quantify -> segment -> pilot 2 -> paved road -> sequence -> accept a mixed estate

Paved road

shared extension archetype CI template Helm library chart test library
 measure adoption, do not police it; every part escapable

24.13 Summary

The practices in this book converge on a few ideas. **Structure by feature and enforce it**, because boundaries decay silently and ArchUnit turns a convention into a failing build. **Keep the framework at the edges** — thin resources, thin scheduled methods, thin consumers over services that can be tested without a container. **Make configuration fail at startup**, so a bad value halts a rollout instead of surfacing at 3 a.m.

On dependencies, two rules carry most of the value: never override a version the BOM manages, and prefer an extension to a plain dependency — the extension is what moves initialisation to build time and makes the library native-safe. And upgrade quarterly with `quarkus update` rather than by editing a version property, because the risk of an upgrade is created almost entirely by the delay.

For migration, the mapping from Spring Boot is largely mechanical and is not what takes the time. The cost is in build-time configuration, the test suite, bean removal, the execution model, and the operational knowledge the on-call team has to rebuild. So the programme should start by **quantifying whether it is worth doing at all**, pilot two services including a hard one, build the paved road before the third, and set a stopping criterion — a deliberately mixed estate is a successful outcome, not a failed migration.

Above all: standardise the things whose inconsistency costs someone outside the owning team, ship them as dependencies rather than documents, and leave everything else alone.

Chapter 25 closes the book with fifty scenario-based questions of the kind asked at senior and principal level — the ones that test judgement rather than recall.

24.14 References

- Quarkus update and OpenRewrite — <https://quarkus.io/guides/update-quarkus>
- Quarkus release and support policy — <https://quarkus.io/blog/quarkus-3-15-lts/>
- Spring DI compatibility — <https://quarkus.io/guides/spring-di>
- Spring Web compatibility — <https://quarkus.io/guides/spring-web>
- Spring Data JPA compatibility — <https://quarkus.io/guides/spring-data-jpa>
- ArchUnit — <https://www.archunit.org/>
- Quarkus extension development — <https://quarkus.io/guides/writing-extensions>

Senior Interview Scenarios

| Fifty scenario questions of the kind asked at senior, staff and principal level — where the interviewer is testing judgement, not recall.

25.1 How to Answer a Scenario Question

Scenario questions have no single correct answer. What is being assessed is whether you reason like someone who has operated systems.

A structure that works:

- 1. Clarify before solving.** “What is the error rate?” “Is this affecting all endpoints or one?” “When did it start?” A candidate who begins fixing before understanding is the one who takes down production twice.
- 2. State your hypothesis and how you would falsify it.** “My first guess is the connection pool, and I would check `agroal_awaiting_count` — if it is zero, I am wrong and I would look at the event loop next.”
- 3. Order by likelihood and cost of checking.** Cheap, likely checks first. Reading a metric beats reading a heap dump.
- 4. Separate mitigation from root cause.** Stop the bleeding first — roll back, scale, shed load. Diagnose afterwards. Conflating them prolongs incidents.
- 5. Name the trade-off.** Every real decision has one. A candidate who presents a solution with no costs has not thought about it hard enough.
- 6. Say what you would do differently next time.** The systemic fix — a test, an alert, a guard rail — is what distinguishes senior from mid-level.

| The single strongest signal you can give is being willing to say “I would not do that, and here is why.” Interviewers ask about microservices, event sourcing, native compilation and caching partly to see whether you reach for them reflexively.

25.2 Production Incidents

Scenario 1 — Throughput collapses under load

Your Quarkus service handles 500 rps comfortably. At around 700 rps, throughput collapses to 50 rps and latency goes to 30 seconds. CPU is at 40 %.

A non-linear collapse with low CPU is the signature of a **blocked event loop**, not of saturation — saturation degrades gracefully. I would look for `BlockedThreadChecker` warnings first; a blocking call in a reactive path removes a fraction of total I/O capacity, and once enough event loops are stalled the server stops accepting.

If the loops are clean, the second candidate is a bounded resource behind a queue — connections, a bulkhead, a downstream with limited concurrency — where past a threshold everything waits. `agroal_awaiting_count` distinguishes them.

Mitigation is to shed load: reduce the ingress rate limit or scale out to buy time. The fix depends on the finding — `@Blocking` or `@RunOnVirtualThread` on the offending endpoint, or right-sizing the pool. Systemically, `max-event-loop-execute-time` set low in dev so this fails locally rather than in production.

Scenario 2 — Pods restart under load, logs are clean

Under peak traffic, pods restart every few minutes. Application logs show nothing wrong.

Clean logs plus restarts means something outside the application is killing it, so I would look at the kubelet's view first: `kubectl describe pod` gives the last termination reason and exit code, which usually settles it. 137 is OOM-killed, and the fix is heap sizing — `MaxRAMPercentage` leaving headroom for metaspace, stacks and direct buffers.

If it is a liveness failure, the causes in order: `timeoutSeconds` of 1 against a JVM in a GC pause; `failureThreshold` too low; or — the one I would check specifically — a dependency check in the liveness probe, which under load turns a slow database into a fleet-wide restart storm.

I would also check CFS throttling, because a throttled container misses probe deadlines while doing nothing wrong, and that produces exactly this symptom.

Scenario 3 — Every deploy drops requests

Users see a burst of 502s during each rolling update.

This is the shutdown handshake. `SIGTERM` and endpoint removal happen concurrently, and endpoint propagation to every kube-proxy takes seconds — so a pod that exits promptly is still receiving traffic when it goes.

Four settings must agree: `quarkus.shutdown.delay` to keep serving during propagation, `quarkus.shutdown.timeout` to drain in-flight work, `terminationGracePeriodSeconds` greater than their sum so Kubernetes does not `SIGKILL` mid-drain, and `maxUnavailable: 0` so capacity never dips.

I would also check the `ENTRYPOINT`: shell form makes `/bin/sh` PID 1, which does not forward `SIGTERM`, so graceful shutdown never runs at all regardless of configuration.

Scenario 4 — Intermittent stale reads

After updating a setting, users see the new value, then the old one, then the new one again.

They are hitting different pods. A local Caffeine cache is per-JVM, so `@CacheInvalidate` clears the entry on the pod that handled the write while the others serve stale values until their TTL expires.

The options are a short TTL that bounds the inconsistency, a distributed cache so invalidation is global, a two-tier arrangement with a short local TTL in front of a shared one, or broadcast

invalidation over Kafka with the TTL as a backstop. Which one depends on how much staleness the product can tolerate and how latency-sensitive the read path is — and that staleness window is a decision someone should agree to explicitly rather than inherit from a default.

Scenario 5 — Consumer lag grows continuously

A Kafka consumer's lag climbs steadily and never recovers.

Lag growing means arrival exceeds processing, so first establish which side changed — the consumed-rate metric distinguishes a producer spike from a consumer slowdown.

If consumption slowed: a blocking call on an event loop, which shows as very low throughput with low CPU; a slower database or downstream; a rebalance loop caused by processing exceeding `max.poll.interval.ms`, visible as repeated partition reassignments and producing zero net progress; or a stuck message being retried forever, which shows as lag on one partition rather than all.

If it is genuinely capacity, more consumers help only up to the partition count. Beyond that the answer is repartitioning or reducing per-message work — not more pods.

Scenario 6 — A downstream service is slow and takes you down with it

A dependency's p99 went from 50 ms to 8 seconds. Your service is now failing entirely, including endpoints that do not call it.

The endpoints that do not call it failing is the diagnostic: worker threads are all parked on the slow dependency, so nothing else can run. That is missing bulkheads.

Immediate mitigation: a tight timeout on that client so threads are released, which restores the unrelated endpoints within seconds. Then a bulkhead capping concurrency per dependency, so it can never consume more than a fixed share. Then a circuit breaker so we stop paying the timeout at all once failure is sustained, and a fallback that degrades the affected feature rather than failing it.

Systemically: no REST client should be creatable without a timeout, which I would enforce in a shared library rather than in review.

Scenario 7 — Native binary works in tests, fails in production

A native build passes CI and throws `ClassNotFoundException` in production on one code path.

The tracing agent or the extension registrations covered the paths CI exercised, and this one was not among them — the closed-world analysis deleted a class reached only reflectively.

Immediate mitigation is a rollback to the JVM image, which is a build-flag change, not a rewrite. Then register the class with `@RegisterForReflection` or a `reflect-config.json`, and — more importantly — add an integration test covering that path, because the reason it reached production is that nothing exercised it natively.

Systemically, this argues for `@QuarkusIntegrationTest` coverage being a precondition for shipping natively, and for preferring extensions over plain dependencies, since generating exactly this configuration is what an extension does.

Scenario 8 — Duplicate emails after a crash

Following a pod restart, some customers received two order-confirmation emails.

At-least-once delivery working as designed, plus a non-idempotent consumer. The pod crashed after sending the email but before committing the offset, so the message was redelivered.

The fix is idempotency, and I would prefer to make it structural rather than bookkeeping: record the send in the same transaction as the business write, keyed on the event id with a unique constraint, so a redelivery violates the constraint and is skipped. The constraint, not a preceding existence check, is what makes it correct under concurrency.

There is a second lesson: sending an email is an external side effect that should be behind an after-commit observer or an outbox, so it never fires for work that then rolls back.

Scenario 9 — Latency spikes with no application cause

p99 shows regular 100–200 ms spikes. Profiling shows nothing; CPU is low.

Regular spikes with an idle-looking profile point at the platform rather than the application. My first check is CFS throttling — `container_cpu_cfs_throttled_seconds_total` — because a CPU limit throttles the container for the remainder of a 100 ms window once its quota is exhausted, and a JVM's bursty GC and JIT threads exhaust it easily. The 100 ms periodicity is the tell.

If throttling is zero, the next candidates are GC pauses, which would show in `jvm_gc_pause_seconds`, and a noisy neighbour on the node.

The fix for throttling is to remove the CPU limit and keep the request, which lets the container burst into idle capacity — and to be able to explain why that is safe.

Scenario 10 — Configuration change has no effect

An environment variable was set correctly in the Deployment and the application ignores it.

Three candidates, checked in order of likelihood. The name mapping — `kubectl exec` and print the environment, since quoted and hyphenated keys produce doubled underscores and fail silently. Whether the property is build-time fixed, in which case no runtime override will ever work and the answer is a rebuild. Or a higher-ordinal source overriding it — a system property in `JAVA_OPTS`, or a `.env` baked into the image.

The decisive move is to ask the application rather than reason about it: `SmallRyeConfig.getConfigValue(key)` returns the value *and* its source name and ordinal, which resolves all three at once. That is why I put a startup config banner in every service.

Scenario 11 — Memory grows until the pod is killed

RSS climbs steadily over hours until `OOMKilled`, then repeats.

A steady climb rather than a step change suggests unbounded growth rather than a sizing error. I would take a heap histogram over time — `jcmd GC.class_histogram` at intervals — and look for the type that grows. The usual suspects in a Quarkus service: an unbounded cache with no `maximum-size`, a `Multi` with no overflow strategy buffering behind a slow consumer, an unbounded query materialising a growing table, or a listener registered per request and never removed.

If the heap is flat and RSS still grows, it is off-heap: direct buffers, metaspace from dynamic class generation, or a native library. `VM.native_memory summary` distinguishes them.

Mitigation is a memory limit with `ExitOnOutOfMemoryError` so restarts are clean, while the leak is found.

Scenario 12 — A scheduled job silently stopped running

Nobody noticed for three weeks that the nightly reconciliation had not run.

This is the defining scheduled-job failure: the failure mode is absence, and absence produces no error, no log line and no alert. The cause could be a property set to `off` during an incident and never restored, a stuck clustered lock, an unscheduled trigger, or a job silently skipping because the previous run never finished.

The fix is not to find this particular cause but to make the class of failure visible: a gauge recording seconds since the last successful completion, alerted above the expected interval. That one metric catches every variant.

I would also ask why three weeks passed before anyone noticed, because that suggests nothing downstream depended on it visibly — which is worth knowing either way.

25.3 Architecture and Design

Scenario 13 — Design an order-processing system

Four services: orders, inventory, payments, fulfilment.

Boundaries by capability, each owning its data with no cross-service table access. Order creation writes the order and an outbox row in one local transaction, because a database commit and a Kafka publish cannot be atomic; CDC drains the outbox to a topic keyed by customer id, which fixes ordering at per-customer while allowing parallelism across customers.

Inventory and payments consume and emit their own events, making this a choreographed saga with explicit compensations — release the reservation, void the authorisation — designed with the business, since compensations are domain operations, not rollbacks.

Every consumer is idempotent by upsert with a timestamp guard, since delivery is at-least-once and out-of-order redelivery is possible. Synchronous calls exist only where the caller needs an answer, each with timeout, bulkhead, circuit breaker and a degrading fallback.

Two decisions I would surface explicitly: the partition key, because it is expensive to change; and how long an order may sit `PENDING`, because that is the user-visible face of eventual consistency.

Scenario 14 — Should this be one service or three?

A team proposes splitting a service into “API”, “business logic” and “data access” services.

I would say no, and explain why rather than just refusing. Splitting by technical layer means every feature touches all three, so nothing can be deployed independently, every change is a three-repository pull request, and each hop adds latency and a failure mode. That is a distributed monolith — all the costs of distribution and none of the independence.

The three tests: can each be deployed without coordinating? No. Does a typical feature stay inside one? No. Can each survive the others being down? No.

If the underlying motivation is that the codebase is hard to work in, the answer is internal modularity — packages by feature with enforced boundaries — which addresses the real problem without distributing it. If the motivation is that two parts have genuinely different scaling profiles, that is a real reason, and the split should follow *that* line, not the layer line.

Scenario 15 — 40 services, velocity falling

Feature delivery has slowed markedly as the service count grew.

I would measure the shape of change before proposing anything. The decisive metric is how many services a typical feature touches: a median of one means the architecture works and the problem is elsewhere; three or four means a distributed monolith where every feature carries coordination cost. Corroborate with deployment coupling, cross-team pull requests per feature, and how many services read another's database.

If it is boundaries, the remedy is consolidation along the seams the data shows — merging services that always change together — and I would frame that as a correction rather than a failure.

But I would first check whether the pain is infrastructural rather than architectural: missing correlation ids, no shared outbox, inconsistent error contracts and slow environments all masquerade as architecture problems and are far cheaper to fix.

Scenario 16 — Microservices or a modular monolith for a new product?

Modular monolith, unless there is a specific named pressure. The expensive mistake in this space is getting boundaries wrong, and boundaries are hardest to judge at the start, which is exactly when microservices are usually adopted.

A monolith with enforced module boundaries keeps real transactions, so invariants are enforced by the database rather than by hand-written compensations; debugging is a stack trace rather than a correlation-id search; and refactoring a boundary is a compile-time operation instead of a distributed migration. Extraction later is straightforward *because* the boundary already exists and has been enforced.

I would extract a module when something names itself: a genuinely different scaling profile, a compliance boundary, or a team blocked by deployment coupling. What I would insist on from day one is the module enforcement, because a monolith without internal boundaries is what everyone is actually trying to escape.

Scenario 17 — Design a multi-tenant SaaS on Quarkus

Tenancy is not a role, so it travels as a claim in the token, populated into a request-scoped `TenantContext` by an identity augmentor.

The critical design decision is that the tenant filter must **fail closed**. A manual `where tenant_id = ?` in forty query methods will eventually be omitted once, and the consequence is a cross-tenant data leak — so I would use a Hibernate filter or PostgreSQL row-level security, where forgetting produces no rows rather than everyone's rows.

Within a tenant, permissions rather than roles, with roles as customer-composable packaging. Support access is a separate, explicitly-audited path with its own permission and a logged justification, never an ambient super-admin role.

For scheduled per-tenant work, a sweeper job over tenants due for work scales better than a job per tenant once the count grows. And the test I would run against every resource type is that a user from tenant A gets 404 on tenant B's data.

Scenario 18 — Design an API for external partners

The error contract matters as much as the happy path, because that is what partners integrate against. RFC 7807 with a stable `type` URI per error class, a machine-readable code per field violation rather than only prose, and cleaned property paths that map onto the request body's JSON structure. All violations returned at once, since partners integrate in batch.

Versioning in the URI for major versions, because it is unambiguous in logs and routing; additive-only within a version, with clients required to ignore unknown fields.

The OpenAPI document is the contract, generated at build time, committed, and diffed in CI so an accidental breaking change fails the build rather than a partner's integration.

Rate limiting at the gateway with `Retry-After` and `X-RateLimit-*` headers. Authentication by client credentials with per-partner audiences. And contract tests on the error responses, because field paths are exactly what a refactor silently breaks.

Scenario 19 — Choose between Kafka and RabbitMQ

The deciding question is whether anyone will ever need to re-read the stream.

Kafka is a log: retained, replayable, ordered per partition, high throughput. If the messages are facts other services may want to consume later, or in a way not yet designed, that is Kafka — a new consumer can read the last month of history.

RabbitMQ is a queue that forgets. Its advantages are per-message acknowledgement and redelivery, so a poison message does not block its peers the way it blocks a Kafka partition; plus priorities, delayed delivery, TTLs and flexible routing. For a work queue of independent tasks executed once and forgotten, it is simpler and a better fit.

The failure modes differ in a way that matters operationally: Kafka needs a dead-letter strategy or one bad message halts a partition; RabbitMQ needs a dead-letter exchange but degrades far more gracefully.

Scenario 20 — Design for a hard 99.99 % availability requirement

First I would test whether the requirement is real, because 99.99 % is 4.3 minutes a month and it constrains everything — deploys, dependencies, maintenance. Frequently the number was chosen aspirationally and the business would accept 99.9 % for a fraction of the cost.

If it is real: the composite availability of a synchronous chain is the product of its links, so four services at 99.99 % give 99.96 % — meaning synchronous dependencies must be minimised, and every remaining one needs a fallback that degrades rather than fails. Asynchronous where possible, because temporal decoupling means a dependency's outage is a delay, not an error.

Multi-zone, ideally multi-region, with data replication whose consistency implications are understood. Zero-downtime deploys with the shutdown handshake correct, expand–migrate–contract schema changes, and canary releases with automated rollback. And an error budget that actually governs prioritisation, or the target is decoration.

Scenario 21 — A team wants to add a shared “Address service”

I would push back. Everyone needing addresses is an argument for a **library**, not a service. A service is a runtime dependency with an availability cost: every consumer’s availability is now capped by it, every address lookup is a network hop, and a change to its API requires coordinating with every consumer.

The three tests fail: it is not a business capability, features do not stay inside it, and consumers cannot function without it.

If the motivation is a shared address *format* and validation, that is a shared library or a schema. If it is a shared address *dataset* — a canonical registry that must be consistent — that is a legitimate service, but then it needs the availability treatment of a critical dependency: caching in consumers, a fallback, and an SLO.

The general form of this mistake is treating reuse as a decomposition criterion, and it produces services that nothing can run without.

Scenario 22 — Design an audit trail

The requirement usually contains two different things, and separating them is the useful move. Compliance audit — who did what, immutable, retained for years, queryable by regulators — is a different system from operational observability, which is traces and logs with short retention.

For compliance: write audit records in the same transaction as the business change, so they cannot diverge; or, if they must survive a rollback, **REQUIRES_NEW**, accepting the second connection. Append-only storage with no update or delete path, and access controls stricter than the application’s own data. The record needs the actor’s identity from the token rather than a client-supplied field, the correlation and trace ids, the before and after state, and a timestamp from a trusted source.

I would resist implementing this as an event stream consumed asynchronously unless loss is acceptable, because an audit trail with at-least-once delivery and no dedup is neither complete nor exact.

25.4 Performance

Scenario 23 — p99 degraded after a release

Correlation with a release is the strongest signal available, so I would start with the diff, not a profiler. First confirm it is real and not a measurement artefact — same load, same data volumes.

Then query count per request, because the most common release-shaped regression is an N+1 introduced by a change to a mapper or DTO, and Hibernate statistics answer that in minutes. Then traces, which show immediately whether the extra time is in our service, a downstream, or the database.

If it is our code, wall-clock flame graphs before and after, compared side by side. I would also check the boring causes a release introduces: a new synchronous call on a hot path, a blocking call added to a reactive endpoint, a changed pool or timeout, a new eager fetch.

If nothing in the diff explains it, I would question the correlation — a data-volume threshold crossed at the same time is a classic coincidence.

Scenario 24 — Worker pool exhausted

All 200 worker threads are busy; requests queue and time out.

A thread dump answers it in one step, and almost always they are all parked on the same thing.

If they are on `getConnection`, the JDBC pool is the real bottleneck and the worker pool is just the queue in front of it. If they are in a socket read, a downstream slowed and I would check whether that client has a read timeout — a missing timeout turns a downstream slowdown into a total outage. If they are in application code, profile it.

Fixes in priority order: add timeouts so threads are released; add a bulkhead per dependency so one slow downstream cannot take the whole pool; right-size the connection pool against the database rather than against request concurrency; and consider `@RunOnVirtualThread` so blocked requests do not each hold a platform thread.

Scenario 25 — Service is fast in staging, slow in production

The gap is everything staging removed, so I would enumerate that rather than re-profile. Real data volumes, which change query plans and cache hit rates. Real concurrency, which introduces lock contention and pool queuing. Network and TLS hops, including a service mesh. CPU throttling and noisy neighbours, invisible in staging. Cold caches and cold JIT on freshly-scheduled pods. And real dependency latency, since staging mocks respond in microseconds.

Distributed tracing separates these fastest, because it attributes time across the whole path rather than within one process.

The general lesson is that “fast in staging” usually means staging removed the bottleneck, and the useful question is which part of production it failed to represent — often it is simply the size of the data.

Scenario 26 — Team proposes going reactive for performance

I would ask what number they are trying to move. Reactive does not make a single request faster; it raises the concurrency ceiling when *threads* are the scarce resource. If the constraint is a twenty-connection pool, reactive moves the queue and throughput is unchanged — and I have seen a quarter spent discovering that.

On Java 21 the first thing to try is `@RunOnVirtualThread`, which addresses the same constraint with a one-line change, keeps the code imperative and debuggable, and can be reverted.

The costs of a reactive rewrite are real and diffuse: stack traces stop corresponding to logical flow, onboarding slows, and the team acquires a class of bug — unsubscribed pipelines, missing overflow strategies, accidental blocking — that they have no experience detecting.

I would support it where the service genuinely streams, needs backpressure, or fans out with cancellation semantics. Not as a performance project without a measurement showing threads are the constraint.

Scenario 27 — Database is the bottleneck across the estate

One PostgreSQL cluster serves 40 services and is at its connection limit.

The arithmetic first: connections are `max-size × replicas × services`, and that number almost certainly exceeds what the database can usefully serve. Short term, audit pool sizes — most were copied defaults, and halving them typically costs nothing because the database was never executing that many queries concurrently.

Then PgBouncer in transaction mode, which is the standard fix and decouples application-side pool sizing from server-side limits; note that transaction mode forbids session state, so prepared statements and temporary tables need checking.

Medium term the real issue is architectural: 40 services on one database is a shared-database anti-pattern and connection pressure is a symptom. I would give the highest-cost, least-coupled services their own database and route reporting to read replicas.

Throughout I would resist raising `max_connections`, which trades a hard failure for degraded performance everywhere.

Scenario 28 — Cold starts hurting autoscaling

Scaling events take 45 seconds to add capacity, so spikes cause errors.

I would decompose the 45 seconds before optimising, because most of it is usually not the application. Image pull time, scheduling delay, init containers, and `initialDelaySeconds` on the readiness probe all count toward it and are often the majority.

For the application's own share: remove unused extensions, avoid eagerly opening the connection pool, move work out of `@Startup` beans, and consider AppCDS. Native compilation takes startup to tens of milliseconds and is the strongest lever if cold start is genuinely the constraint — this is the case where native changes what is possible rather than merely saving money.

Alongside that: pre-pull images to nodes, scale on a leading indicator rather than a lagging one so capacity arrives earlier, and keep a warm buffer of replicas if the spike shape justifies it.

Scenario 29 — Native is slower than JVM under load

After moving to native, peak throughput dropped 25 %.

That is expected, and the team may have chosen the wrong mode. Native has no JIT, so it runs statically-compiled code rather than code optimised from observed behaviour; 10–30 % below a warmed JVM is the normal range.

If the service is long-lived and throughput-bound, native was optimising something they were not paying for — and switching back is a build-flag change, not a rewrite. That reversibility is worth stating explicitly, because teams often feel committed.

If they need native for cold start or memory, profile-guided optimisation recovers much of the gap by feeding a recorded profile into the build, at the cost of a two-stage build and a representative workload.

The framing I would leave them with is that native is a tool for cold start and memory, not a general performance upgrade.

Scenario 30 — Design a load test you would trust

Four things are usually wrong and each invalidates the result.

Load model: open-loop, constant-arrival-rate. A closed-loop generator sends fewer requests when the server slows, so coordinated omission makes the percentiles describe a system under less load than you think.

Duration: ten minutes rather than two, so the JIT has compiled the hot path and caches have reached steady state.

Data: production-like volumes, because index behaviour and cache hit rates are completely different at a thousand rows and at fifty million.

Topology: through the real ingress and TLS, not localhost, and with a traffic mix rather than one endpoint.

Plus: verify the generator is not itself saturated, record server-side metrics alongside client latency so a slowdown can be attributed, and keep it in CI so the *delta* is visible — the delta matters more than the absolute number.

25.5 Data and Consistency

Scenario 31 — Move from a monolith database to per-service databases

The first move is to inventory the cross-boundary transactions and challenge each one, because in a monolith it is free to wrap unrelated work in one transaction and a large fraction of the existing boundaries are accidental.

For each genuine one, ask what invariant it protects and whether it must hold *immediately* or merely *eventually* — a business conversation, not a technical one. Most tolerate eventual consistency once someone states plainly what the intermediate state looks like to a user. Those become sagas with compensations designed alongside the domain experts.

Where an invariant genuinely must be immediate, that is strong evidence the split is wrong and those entities should stay together — not a reason to reach for XA.

I would introduce the outbox pattern early and platform-wide, since every service will need atomic write-and-publish, and mandate idempotent consumers with a shared helper. The failure I would guard hardest against is a saga implemented without compensations, which is a distributed transaction that silently loses data.

Scenario 32 — Guarantee exactly-once processing

I would start by correcting the frame: exactly-once *delivery* is not achievable, because the consumer and broker cannot atomically agree that a message was both processed and committed. What is achievable is exactly-once *effect*.

Producer side, `enable.idempotence=true` deduplicates the producer's own retries. Consumer side, at-least-once delivery plus an idempotent operation. I would prefer structural idempotency — an upsert keyed on the business id with a timestamp guard, which also handles out-of-order redelivery — over a dedup table, because it is cheaper and cannot drift.

Where a dedup table is needed, the insert goes in the same transaction as the business write, with a unique constraint on the event id; the constraint, not a preceding check, is what makes it correct under concurrency.

Kafka transactions extend exactly-once to consume-transform-produce within Kafka, but stop at the database boundary.

Scenario 33 — Schema change during a rolling deploy

Old and new pods run concurrently against one schema for the rollout's duration, so every migration must be compatible with both versions.

That forces expand–migrate–contract across releases: release one adds a nullable column and writes both; release two backfills and switches reads; release three drops the old column. Renames become add-copy-drop.

Migrations run as a pre-upgrade Job rather than `migrate-at-start`, so they complete before any new pod starts and a failure halts the deploy before it begins — with ten replicas racing, Flyway's lock makes it safe but slow, and a long migration can exceed readiness deadlines and cause a rollback loop.

I would also account for the surge: `maxSurge: 1` briefly adds a replica, so total connections exceed steady state, which matters if the database is near its limit.

Scenario 34 — Intermittent `OptimisticLockException`

First establish whether this is a bug or the system working correctly, because optimistic locking exists to surface concurrent modification and occasional conflicts are expected.

A low rate spread across many rows is normal, and the fix is at the boundary: map it to a 409 with a clear message so clients retry, and make sure it is not logged as an error paging someone.

A high rate concentrated on a few rows is real contention needing a design answer. Optimistic locking is the wrong tool there, because every conflict wastes a whole transaction — I would move to a pessimistic lock on that row, or better, restructure so the contended value is not a single row: an append-only ledger with periodic rollup, or a database-side atomic decrement.

I would also check whether the aggregate is too coarse, since unrelated fields sharing one `@Version` makes users editing different parts of an order collide unnecessarily.

Scenario 35 — Should we adopt event sourcing?

Probably not, and I would want to hear the motivation before answering properly.

It is genuinely valuable where the history *is* the product — ledgers, trading, medical records, anything needing time travel or replay into new read models. Outside that, the costs are severe and permanent: schema migration becomes event versioning, debugging requires replay tooling, every developer must learn a model with little mainstream framework support, and deleting a customer's data conflicts with an immutable log and therefore with GDPR.

If the motivation is auditability, an audit table gives that for a fraction of the cost. If it is temporal queries, a history table does. If it is decoupling, an outbox and events do — without making events the source of truth.

The distinction worth holding is between *publishing* events, which is cheap and useful, and *sourcing state from* events, which is a different and much larger commitment.

Scenario 36 — Report needs data from five services

I would first ask how fresh the report must be, because the answer changes the design entirely.

If minutes of staleness are acceptable — which reporting usually tolerates — the right answer is a read model: each service publishes events, a reporting service projects them into a denormalised store shaped for the queries, and the report reads one database with no joins across services. That also isolates analytical load from transactional systems, which matters more as the report grows.

If it must be real-time, fanning out to five services synchronously multiplies latency and availability, and I would push back on the requirement before building it.

What I would avoid is the tempting middle option of letting the reporting service read five databases directly. It reinstates the shared-database coupling those services were split to escape, and every schema change becomes a cross-team negotiation.

Scenario 37 — GDPR deletion across an event-driven estate

This is where event-driven architectures hurt, and it should be designed for rather than discovered.

The technique is crypto-shredding: personal data is encrypted with a per-subject key, only the ciphertext appears in events and read models, and deletion means destroying the key — which renders every copy unreadable without mutating an immutable log. That is the only approach I have seen work at scale.

Alternatives are worse: retroactively rewriting a Kafka log is impractical, and tombstone-plus-compaction only helps for keyed state, not for the full event history.

Operationally, deletion also needs a registry of every system holding a copy, since read models, caches, search indexes, backups and analytics warehouses all count. And backups need a retention policy short enough that the obligation is satisfiable.

If crypto-shredding is not already in place, adopting it retroactively is a large programme, which is the argument for designing for deletion before the estate exists.

25.6 Security

Scenario 38 — A token for another service was accepted

A token issued for the marketing app authenticated successfully against the orders API.

Missing audience validation — without `quarkus.oidc.token.audience`, any token the IdP issued for any client is accepted. It fails silently and your own tokens work fine, so testing never catches it.

Immediate fix is to set the audience and confirm the issuer check is enabled. Then assess the exposure: what could that token's holder have done, and did they? That is a log-analysis question against the token's `azp` or `client_id` claim.

Systemically, every service's audience should be its own name, so a token cannot be replayed across services, and that should be set by a shared extension rather than by each team remembering. I would also add a test that a token with the wrong audience is rejected, because this is precisely the kind of setting that gets removed to “make it work in dev” and never restored.

Scenario 39 — A new endpoint shipped unauthenticated

Code review does not reliably catch an *absent* annotation, which is why the fix is structural rather than procedural: `quarkus.security.jaxrs.deny-unannotated-endpoints=true`, so an endpoint with no security annotation is denied rather than public.

I would turn that on, which will immediately break any other endpoints in the same state — and finding those is the point.

Beyond that, a CI test enumerating endpoints and asserting each has an explicit security annotation, which also documents intent for the deliberately-public ones. And for an estate, a periodic automated sweep across all services flagging unauthenticated endpoints, because the same mistake will be made elsewhere.

The framing worth stating: security-by-default means the failure mode of forgetting is denial, not exposure.

Scenario 40 — Users can read each other's orders

Any authenticated user can fetch any order by id.

An authorisation gap: authentication was checked, ownership was not. Annotations cannot express “the caller owns this row”, so that check must be in code, and its absence is invisible in review because the endpoint *looks* protected.

The fix is an ownership check in the service, returning **404 rather than 403**, because 403 confirms the resource exists and lets an attacker enumerate valid ids.

Systemically, this is the test that should have existed: for every resource, a user from one account gets 404 on another's. I would add that to a shared test library so it is three lines rather than thirty, and require it for every resource type — because this vulnerability class recurs and is entirely preventable by one test pattern.

Scenario 41 — Should the SPA store the access token?

No. `localStorage` is readable by any JavaScript on the origin, so a single XSS anywhere — including in a third-party script or a dependency — exfiltrates a credential valid until expiry and usable from anywhere. That turns a bounded vulnerability into account takeover.

The current recommendation is a backend-for-frontend: the SPA authenticates against a Quarkus service in `web-app` mode, tokens stay server-side, and the browser holds an `HttpOnly`, `Secure`, `SameSite` cookie that JavaScript cannot read.

I would name the costs rather than pretend the choice is free: a stateful component, and a CSRF consideration that bearer tokens do not have — mitigated by `SameSite=Lax` plus a CSRF token.

What I would not accept is the halfway position of an in-memory access token with a refresh token in a cookie; better than `localStorage`, but the access token is still exposed to any script on the page.

Scenario 42 — Service A calls B with its own service account

This is the confused deputy, and it is a privilege escalation waiting to happen. B sees A's identity, not the user's, so it cannot enforce user-level rules — which means any bug in A that lets a user trigger a call becomes full access to whatever A is permitted.

The fix is token propagation: forward the caller's access token so B sees the original user and applies its own authorisation. Client credentials are for genuinely system-initiated work — a scheduled reconciliation — not for user-initiated calls.

Where B should see a narrower identity than A holds, token exchange produces a token scoped to the specific operation.

Systemically I would make client-credentials usage an explicit, reviewable exception, ideally a lint rule, because the pattern is easy to introduce, looks like a convenience, and is nearly invisible in review.

Scenario 43 — Rotate a leaked credential across 40 services

The immediate priority is containment, and the order matters: revoke the credential first, then rotate, then assess. Revoking before you have a replacement causes an outage, so in practice it is issue-new, deploy, then revoke-old — which is why credentials should support two valid values during rotation, and discovering they do not is the common blocker.

Then assess exposure: what did that credential permit, and what was done with it? That is a log question, and the answer depends on whether the credential's use was logged with enough identity to distinguish legitimate from not.

Afterwards, the systemic fix is to remove the class of problem: workload identity rather than shared secrets, so there is nothing to leak or rotate; short-lived credentials so a leak has a small window; and a rehearsed rotation runbook, because organisations reliably discover during the incident that they cannot rotate quickly. The rehearsal is the part most often skipped and the part that determines the outcome.

25.7 Team, Process and Judgement

Scenario 44 — Test suite takes 20 minutes; people push without running it

I would measure where the time goes first, because intuition is usually wrong — per-class durations and a count of Quarkus restarts, since the dominant cost is almost always `@TestProfile` churn

rather than the tests themselves. Consolidating twelve profiles into two typically removes more time than any test optimisation.

Then split: a fast gate under three minutes on every push, and a fuller suite tagged slow on merge and nightly including the native run. That restores the feedback loop immediately without deleting anything.

In parallel, migrate the worst `@QuarkusTest` classes that are testing pure logic down to plain JUnit — a gradual improvement, not a project.

The three-minute gate is the non-negotiable part, because the behaviour we are changing is people skipping tests, and that only changes when running them is cheap.

Scenario 45 — A team wants to adopt a new framework for one service

I would ask what problem it solves that the current stack does not, and then weigh that against costs that are diffuse and permanent rather than local and one-off.

One service in a new stack means: a second set of base images, CI templates, observability integration, security patching and runbooks; an on-call rota that must know two stacks; and a hiring and onboarding surface that has doubled for a single service's benefit. Those costs land on people who did not choose it.

That said, blanket refusal is how organisations ossify. I would be open to it as a deliberate, time-boxed evaluation with success criteria agreed up front and an explicit decision point — either it becomes a supported second stack with the platform work funded, or it is migrated back. What I would not accept is adoption by accident, where one service becomes three before anyone decided.

Scenario 46 — Disagreement about microservices with a senior colleague

I would try to find the empirical question underneath the architectural one, because those arguments are usually unresolvable in the abstract and quite tractable in specifics.

Rather than debating microservices in general, I would ask what we expect to be true in each design six months in: how many services a typical feature touches, whether teams are blocked deploying, whether the invariants can tolerate eventual consistency. Those are checkable claims, and often the disagreement dissolves once someone traces three real features through both designs.

I would also try to identify what each of us is optimising for — they may be weighting team autonomy, I may be weighting operational simplicity, and both are legitimate. Naming that is usually more productive than trading examples.

If we still disagree, I would advocate for the reversible option: a modular monolith with enforced boundaries can become services; the reverse is much harder. And then commit fully to whichever is chosen, because a half-hearted implementation of either is worse than a wholehearted implementation of the other.

Scenario 47 — Justify Quarkus adoption to a sceptical CTO

I would lead with the number, not the technology. The case is a measurable reduction in memory per replica and in cold-start time, which converts to a monthly figure from current replica counts and memory requests — and if that figure is not compelling, I would say so and not propose the migration.

I would then be honest about what it costs: a second stack unless we migrate broadly, a learning curve, and CI capacity for native builds if we go that far. And about where Spring is better: ecosystem depth, hiring pool, and the volume of existing answers to any given problem.

The framing that tends to land is scope. I would propose Quarkus for *new* services and for the specific existing services where the numbers justify it, with an explicit criterion for stopping — not a wholesale replacement. That makes it a bounded bet with a measurable outcome rather than a strategic commitment, and a CTO's scepticism is usually about the latter.

Scenario 48 — Inherit a service with no tests and frequent incidents

I would resist the instinct to start adding tests broadly, because untargeted coverage on unfamiliar code is slow and low-yield.

Instead I would start from the incidents: read the last three months, classify them, and find where they cluster. That tells you which parts of the system are actually dangerous, which is rarely uniform. Then add characterisation tests around those paths specifically — tests that pin current behaviour, so refactoring becomes safe — starting at the HTTP boundary where behaviour is observable without understanding the internals.

In parallel, the cheap operational wins that reduce incident *impact* while the tests are being built: correlation ids and trace ids in logs, alerts on symptoms rather than causes, and a runbook per recurring incident.

I would negotiate explicit time for this rather than doing it invisibly, framed in terms of incident rate, because that is the number the business already cares about.

Scenario 49 — Asked to ship something you believe is unsafe

I would separate the technical judgement from the decision right, because those belong to different people and conflating them is how these conversations go wrong.

My job is to make the risk legible: what specifically could go wrong, how likely, what the blast radius is, and what it would cost to mitigate — including a cheaper partial mitigation, since “ship it safely later” and “do not ship” are rarely the only options. A feature flag, a canary to 1 %, or shipping to internal users first often reduces the risk enough to be acceptable.

I would put that in writing, not for self-protection but because a verbal risk assessment does not survive the conversation.

If the decision is still to ship, and it is not unethical or unlawful, it is a legitimate business call and I would support it — while making sure monitoring and a rollback path exist so we find out quickly. If it crosses into safety, security of user data, or legality, that is different, and I would escalate rather than comply.

Scenario 50 — Your recommended architecture failed in production

I would separate the incident from the retrospective, and be careful about the order.

First, fix it — my ownership of the decision is not relevant while users are affected, and there is a real failure mode where the person who chose the design is slow to consider that the design is the problem. I would say out loud that it might be, so others feel able to raise it.

Afterwards, the honest question is whether the decision was wrong given what was known, or right given what was known and wrong given what emerged. Those are different, and conflating them either produces false blame or prevents learning. If information was available that I did not seek — a load characteristic, an operational constraint, a dependency’s real behaviour — that is a process improvement about how I evaluate designs.

Then I would write it up plainly, including what I would decide differently, and share it. Visibly owning a wrong call is the thing that makes it safe for everyone else to raise concerns about the next one — which is worth more than the individual lesson.

25.8 Summary

Fifty scenarios, and a small number of patterns run through nearly all of them.

Clarify before solving. Almost every scenario above is under-specified on purpose, and the first move is a question. Which endpoints? What error rate? When did it start? A candidate who begins fixing before understanding is the one who takes production down twice.

Separate mitigation from root cause. Roll back, shed load, add a timeout — then diagnose. Conflating the two prolongs incidents.

Check the cheap and likely first. A metric beats a heap dump. In Quarkus specifically, four causes account for most production problems: the connection pool queueing, a blocked event loop, N+1 queries, and something outside the application entirely — CFS throttling, OOM kills, probe misconfiguration.

Name the trade-off. Every real decision has one, and a solution presented without costs has not been thought about hard enough.

Finish with the systemic fix. The test, the alert, the guard rail, the default that makes the failure impossible next time. That is the difference between fixing an incident and improving a system, and it is what senior means.

And the answer that consistently distinguishes the strongest candidates: **“I would not do that, and here is why.”** Microservices, event sourcing, native compilation, caching, a service mesh — each is asked partly to see whether you reach for it reflexively. The engineer who can argue against the fashionable option, and change their mind when the numbers say otherwise, is the one you want making these decisions.

25.9 References

- Google SRE Book — <https://sre.google/sre-book/table-of-contents/>
- Google SRE Workbook — <https://sre.google/workbook/table-of-contents/>
- Release It!, Michael Nygard — resilience patterns and failure modes
- Designing Data-Intensive Applications, Martin Kleppmann — consistency and distributed data
- Microservices patterns — <https://microservices.io/patterns/>
- OWASP API Security Top 10 — <https://owasp.org/API-Security/>
- Quarkus guides — <https://quarkus.io/guides/>

Quarkus Annotation Cheat Sheet

Every annotation you are likely to need, grouped by concern, with a worked example and the options that matter. Java 21 · Quarkus 3.33 LTS · Jakarta EE 10.

Reading the tables: *Package* tells you where the annotation comes from, which matters when two libraries define similar names (`@Timed` exists in both Micrometer and the deprecated MicroProfile Metrics). *Applies to* tells you where it is legal. The notes column carries the trap.

A.1 REST — Server

quarkus-rest · quarkus-rest-jackson

A.1.1 Resource and routing

Annotation	Package	Applies to	Purpose
<code>@Path</code>	<code>jakarta.ws.rs</code>	class, method	Declares the URI template
<code>@GET @POST @PUT @PATCH @DELETE @HEAD @OPTIONS</code>	<code>jakarta.ws.rs</code>	method	HTTP verb
<code>@Produces</code>	<code>jakarta.ws.rs</code>	class, method	Response media type
<code>@Consumes</code>	<code>jakarta.ws.rs</code>	class, method	Accepted request media type
<code>@ApplicationPath</code>	<code>jakarta.ws.rs</code>	class	Base path for the whole application

```
@Path("/orders")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class OrderResource {

    @GET @Path("/{id}")          public OrderView get(@RestPath long id) { ... }
    @POST                          public RestResponse<OrderView> create(@Valid
        CreateOrderRequest r) { ... }
    @PATCH @Path("/{id}/status") public OrderView patch(@RestPath long id, StatusPatch p)
        { ... }
```

```
@DELETE @Path("/{id}") public void delete(@RestPath long id) { ... }
}
```

Options that matter

- `@Produces` accepts several types for content negotiation: `@Produces({APPLICATION_JSON, APPLICATION_XML})`.
- `@Path` supports regex templates: `@Path("/{id: \\d+}")` matches numeric ids only.
- Prefer `quarkus.http.root-path` over `@ApplicationPath` — but remember it is **build-time fixed** (Chapter 4).

A.1.2 Parameter binding

Annotation	Quarkus shorthand	Binds
<code>@PathParam("id")</code>	<code>@RestPath</code>	URI template variable
<code>@QueryParam("q")</code>	<code>@RestQuery</code>	Query string
<code>@HeaderParam("X-Tenant")</code>	<code>@RestHeader</code>	Request header
<code>@CookieParam("session")</code>	<code>@RestCookie</code>	Cookie
<code>@FormParam("name")</code>	<code>@RestForm</code>	Form field
<code>@MatrixParam("v")</code>	<code>@RestMatrix</code>	Matrix parameter
<code>@BeanParam</code>	—	Groups parameters into a holder
<code>@DefaultValue("20")</code>	—	Default when absent
<code>@Context</code>	—	Injects <code>UriInfo</code> , <code>HttpHeaders</code> , <code>SecurityContext</code>

```
@GET
public List<OrderSummary> search(
    @RestQuery List<String> status,           // repeatable -> List
    @RestQuery @DefaultValue("0") @Min(0) int page,
    @RestQuery @DefaultValue("20") @Max(100) int size,
    @RestQuery Optional<OrderStatus> filter, // enum, optional
    @RestHeader("X-Tenant") String tenant,
    @BeanParam SortParams sort,
    @Context UriInfo uriInfo) { ... }

public class SortParams {
    @RestQuery @DefaultValue("createdAt") public String sortBy;
    @RestQuery @DefaultValue("desc")     public String direction;
}
```

The `@Rest*` shorthand infers the name from the parameter, which requires `-parameters` on the compiler. The Quarkus archetype sets it; if you see `arg0` in an error message, it is missing.

A.1.3 Execution model

Annotation	Package	Effect
<code>@Blocking</code>	<code>io.smallrye.common.annotation</code>	Force a worker thread
<code>@NonBlocking</code>	<code>io.smallrye.common.annotation</code>	Force the event loop
<code>@RunOnVirtualThread</code>	<code>io.smallrye.common.annotation</code>	Java 21 virtual thread per request

```

public List<Order> a()           // plain return -> worker thread (default)
public Uni<Order> b()           // reactive return -> event loop (default)
@Blocking public Uni<Order> c()  // override: worker
@NonBlocking public String d()  // override: event loop
@RunOnVirtualThread public Order e() // virtual thread

```

The classification is decided **at build time** from the return type. Getting it wrong — a JDBC call in a `Uni` method without `@Blocking` — blocks an event loop and degrades the whole server (Chapter 2).

A.1.4 Filters, mappers and streaming

Annotation	Package	Applies to	Purpose
<code>@ServerRequestFilter</code>	<code>org.jboss.resteasy.reactive.server</code>	method	Inbound filter
<code>@ServerResponseFilter</code>	<code>org.jboss.resteasy.reactive.server</code>	method	Outbound filter
<code>@ServerExceptionMapper</code>	<code>org.jboss.resteasy.reactive.server</code>	method	Map an exception to a response
<code>@Provider</code>	<code>jakarta.ws.rs.ext</code>	class	Register a JAX-RS provider
<code>@NameBinding</code>	<code>jakarta.ws.rs</code>	annotation	Bind a filter to specific endpoints
<code>@RestStreamElementType</code>	<code>org.jboss.resteasy.reactive</code>	method	Element type for SSE
<code>@CacheControl</code>	<code>org.jboss.resteasy.reactive</code>	method	HTTP cache headers

```

public class ApiFilters {

    @ServerRequestFilter(preMatching = true)           // runs before route matching
}

```

```

    public void correlation(ContainerRequestContext ctx) { ... }

    @ServerRequestFilter                                // abort by returning a response
    public Optional<RestResponse<ProblemDetail>> requireTenant(ContainerRequestContext ctx) {
        ... }

    @ServerResponseFilter
    public void addTraceId(ContainerResponseContext res) { ... }

    @ServerExceptionHandler(priority = 1)
    public RestResponse<ProblemDetail> notFound(OrderNotFoundException e) { ... }
}

@GET @Path("/events")
@Produces(MediaType.SERVER_SENT_EVENTS)
@RestStreamElementType(MediaType.APPLICATION_JSON)
public Multi<OrderEvent> events() { ... }

```

Options: `@ServerRequestFilter(preMatching = true)` to rewrite the path or method; `priority` on both filters and mappers to control ordering (lower runs first).

A.1.5 REST client

Annotation	Package	Applies to	Purpose
<code>@RegisterRestClient</code>	<code>org.eclipse.microprofile.rest.client.inject</code>	interface	Declares a typed client
<code>@RestClient</code>	same	injection point	Injects the client
<code>@RegisterClientHeaders</code>	<code>org.eclipse.microprofile.rest.client.annotation</code>	interface	Header factory
<code>@ClientHeaderParam</code>	same	interface, method	Static or computed header
<code>@ClientExceptionMapper</code>	<code>io.quarkus.rest.client.reactive</code>	static method	Map a response to an exception
<code>@AccessToken</code>	<code>io.quarkus.oidc.token.propagation</code>	interface	Forward the caller's token
<code>@OidcClientFilter</code>	<code>io.quarkus.oidc.client.filter</code>	interface	Attach the service's own token

```

@Path("/payments")
@RegisterRestClient(configKey = "payments-api")
@RegisterClientHeaders(CorrelationHeaderFactory.class)
@ClientHeaderParam(name = "X-Client", value = "orders-service")

```

```

@AccessToken
public interface PaymentClient {

    @POST PaymentResult capture(CaptureRequest request);

    @ClientExceptionMapper
    static RuntimeException toDomain(Response r) {
        return r.getStatus() == 402 ? new PaymentDeclinedException() : null;
    }
}

```

```

quarkus.rest-client.payments-api.url=https://payments.internal:8443
quarkus.rest-client.payments-api.read-timeout=5000
quarkus.rest-client.payments-api.scope=jakarta.inject.Singleton

```

`configKey` decouples the interface from the property name — use it, or the property key becomes the fully-qualified class name.

A.2 CDI and Dependency Injection

quarkus-arc (transitively present)

A.2.1 Scopes

Annotation	Package	Instances	Proxied	Lazy
@ApplicationScoped	jakarta.enterprise.context	one per app	yes	yes
@Singleton	jakarta.inject	one per app	no	no
@RequestScoped	jakarta.enterprise.context	one per request	yes	yes
@SessionScoped	jakarta.enterprise.context	one per HTTP session	yes	yes
@Dependent	jakarta.enterprise.context	one per injection point	no	n/a

```

@ApplicationScoped // the default choice
public class OrderService { }

@Singleton // needs final fields, or a hot path
public class Metrics { }

@RequestScoped
public class TenantContext { }

```

`@ApplicationScoped` beans must not be `final` and need a no-arg constructor, because they are injected as generated client proxies (Chapter 3).

A.2.2 Injection and lifecycle

Annotation	Package	Purpose
<code>@Inject</code>	<code>jakarta.inject</code>	Injection point
<code>@Named("x")</code>	<code>jakarta.inject</code>	String-based qualifier
<code>@Qualifier</code>	<code>jakarta.inject</code>	Declares a typed qualifier
<code>@Any</code>	<code>jakarta.enterprise.inject</code>	Matches all beans of a type
<code>@Default</code>	<code>jakarta.enterprise.inject</code>	The implicit qualifier
<code>@Produces</code>	<code>jakarta.enterprise.inject</code>	Producer method or field
<code>@Disposes</code>	<code>jakarta.enterprise.inject</code>	Disposer for a produced bean
<code>@PostConstruct</code>	<code>jakarta.annotation</code>	After construction and injection
<code>@PreDestroy</code>	<code>jakarta.annotation</code>	Before the context ends
<code>@Startup</code>	<code>io.quarkus.runtime</code>	Force eager instantiation
<code>@Priority(n)</code>	<code>jakarta.annotation</code>	Ordering
<code>@Unremovable</code>	<code>io.quarkus.arc</code>	Exempt from unused-bean removal
<code>@DefaultBean</code>	<code>io.quarkus.arc</code>	Replaceable default
<code>@Identifier("x")</code>	<code>io.smallrye.common.annotation</code>	Typed alternative to <code>@Named</code>

```

@ApplicationScoped
public class ClientProducers {

    @Produces @ApplicationScoped
    S3Client s3(StorageConfig config) { return S3Client.builder()...build(); }

    void close(@Disposes S3Client client) { client.close(); }    // ALWAYS pair these
}

@Qualifier @Retention(RUNTIME)
@Target({FIELD, PARAMETER, METHOD, TYPE})
public @interface Channel { Type value(); enum Type { EMAIL, SMS } }

@ApplicationScoped @Channel(EMAIL)
public class EmailChannel implements NotificationChannel { }

@ApplicationScoped
public class Dispatcher {
    @Inject @Channel(EMAIL) NotificationChannel email;
}

```

```
@Inject @Any Instance<NotificationChannel> all;    // also keeps them unremovable
}
```

A producer without a **@Disposes** is a resource leak for anything holding a socket, file handle or thread pool — and nothing fails immediately.

A.2.3 Conditional beans

Annotation	Package	Decided	Loser
<code>@IfBuild Profile("prod")</code>	<code>io.quarkus.arc. properties</code>	build time	removed from the image
<code>@UnlessBuild Profile("prod")</code>	same	build time	removed
<code>@IfBuild Property(name=, stringValue=)</code>	same	build time	removed
<code>@UnlessBuild Property(...)</code>	same	build time	removed
<code>@LookupIfProperty(. ..)</code>	same	runtime lookup	kept
<code>@LookupUnless Property(...)</code>	same	runtime lookup	kept
<code>@Alternative + @Priority</code>	<code>jakarta.enterprise. inject</code>	build time	kept, not injected

```
@ApplicationScoped @IfBuildProfile("prod")
public class StripeGateway implements PaymentGateway { }
```

```
@ApplicationScoped @UnlessBuildProfile("prod")
public class FakeGateway implements PaymentGateway { }
```

Build-time selection means the test double is **absent from the production binary**. That is a security property, not just a size one.

A.2.4 Interceptors, decorators and events

Annotation	Package	Purpose
<code>@InterceptorBinding</code>	<code>jakarta.interceptor</code>	Declares a binding annotation
<code>@Interceptor</code>	<code>jakarta.interceptor</code>	Marks the interceptor class

Annotation	Package	Purpose
<code>@AroundInvoke</code>	<code>jakarta.interceptor</code>	Wraps a method call
<code>@AroundConstruct</code>	<code>jakarta.interceptor</code>	Wraps construction
<code>@Decorator</code>	<code>jakarta.decorator</code>	Type-aware wrapper
<code>@Delegate</code>	<code>jakarta.decorator</code>	The wrapped instance
<code>@Observes</code>	<code>jakarta.enterprise.event</code>	Synchronous observer
<code>@ObservesAsync</code>	<code>jakarta.enterprise.event</code>	Asynchronous observer

```

@InterceptorBinding @Retention(RUNTIME) @Target({TYPE, METHOD})
public @interface Audited { }

@Audited @Interceptor @Priority(Interceptor.Priority.APPLICATION)
public class AuditInterceptor {
    @AroundInvoke
    Object audit(InvocationContext ctx) throws Exception { return ctx.proceed(); }
}

// Events
@Inject Event<OrderPlaced> events; events.fire(new OrderPlaced(id));

void sync(@Observes OrderPlaced e) { }
void afterCommit(@Observes(during = TransactionPhase.AFTER_SUCCESS) OrderPlaced e) { }
void async(@ObservesAsync OrderPlaced e) { }
void ordered(@Observes @Priority(100) OrderPlaced e) { }

```

Transaction phases: `IN_PROGRESS` (default), `BEFORE_COMPLETION`, `AFTER_COMPLETION`, `AFTER_SUCCESS`, `AFTER_FAILURE`.

Interceptors do not fire on self-invocation. `this.method()` bypasses the proxy — true for `@Transactional`, `@CacheResult`, `@Timed` and every other binding.

A.3 Configuration

quarkus-core · SmallRye Config

Annotation	Package	Applies to	Purpose
<code>@ConfigProperty</code>	<code>org.eclipse.microprofile.config.inject</code>	field, parameter	Injects one property
<code>@ConfigMapping</code>	<code>io.smallrye.config</code>	interface	Binds a whole prefix
<code>@WithDefault</code>	<code>io.smallrye.config</code>	mapping method	Default value
<code>@WithName</code>	<code>io.smallrye.config</code>	mapping method	Override the derived key

Annotation	Package	Applies to	Purpose
<code>@WithParentName</code>	<code>io.smallrye.config</code>	mapping method	Flatten a nested group
<code>@WithConverter</code>	<code>io.smallrye.config</code>	mapping method	Custom converter
<code>@ConfigRoot</code>	<code>io.quarkus.runtime.annotations</code>	class	Extension config root
<code>@StaticInitSafe</code>	<code>io.smallrye.config</code>	config source	Usable during static init

```

@ConfigMapping(prefix = "orders")
public interface OrdersConfig {

    @WithDefault("50") @Min(1) @Max(500)
    int maxLineItems();

    @WithDefault("30s")
    Duration checkoutTimeout();

    @NotEmpty
    List<@Email String> notifyEmails();

    Optional<String> legacyEndpoint();
    Map<String, String> regionCodes();

    Retry retry();

    interface Retry {
        @WithDefault("3") int maxAttempts();
        @WithName("jitter-ms") @WithDefault("100") int jitterMillis();
    }
}

```

```

@ConfigProperty(name = "orders.max-line-items", defaultValue = "50") int max;
@ConfigProperty(name = "orders.legacy") Optional<String> legacy;    // optional
@ConfigProperty(name = "orders.flag") Provider<Boolean> flag;      // re-read per call

```

Prefer `@ConfigMapping`. It validates the whole group at startup and reports every failure at once, so in Kubernetes a misconfiguration halts the rollout instead of surfacing at first use.

A.4 Validation

quarkus-hibernate-validator

A.4.1 Constraints

Group	Annotations
Presence	<code>@NotNull @NotEmpty @NotBlank @Null</code>
Size / range	<code>@Size(min,max) @Min @Max @Positive @PositiveOrZero @Negative @NegativeOrZero @DecimalMin @DecimalMax @Digits(integer, fraction)</code>
Format	<code>@Email @Pattern(regex,flags) @URL @UUID @CreditCardNumber @ISBN</code>
Time	<code>@Past @PastOrPresent @Future @FutureOrPresent @DurationMin @DurationMax</code>
Boolean	<code>@AssertTrue @AssertFalse</code>
Hibernate extras	<code>@Length @Range @CodePointLength @ScriptAssert</code>
Cascade	<code>@Valid</code>

```

public record CreateOrderRequest(
    @NotBlank @Email String customerEmail,
    @NotEmpty @Valid List<LineItemRequest> items,           // @Valid cascades
    @Size(max = 500) String note,
    @Future LocalDate deliverBy) { }

public record LineItemRequest(
    @NotBlank @Pattern(regex = "[A-Z]{3}-\\d{4}$") String sku,
    @Positive @Max(999) int quantity) { }

```

A.4.2 Applying and customising

Annotation	Package	Purpose
<code>@Valid</code>	<code>jakarta.validation</code>	Trigger / cascade validation
<code>@Validated</code>	<code>jakarta.validation.executable</code>	Class-level method validation
<code>@Constraint(validatedBy=)</code>	<code>jakarta.validation</code>	Declares a custom constraint
<code>@GroupSequence</code>	<code>jakarta.validation</code>	Ordered validation groups
<code>@ConvertGroup(to=)</code>	<code>jakarta.validation.groups</code>	Switch group on cascade

```

@Constraint(validatedBy = SkuValidator.class)
@Target({FIELD, PARAMETER, RECORD_COMPONENT, TYPE_USE}) @Retention(RUNTIME)
public @interface ValidSku {
    String message() default "invalid SKU";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
    String family() default ""; // your own option
}

public class SkuValidator implements ConstraintValidator<ValidSku, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) {
        return v == null || PATTERN.matcher(v).matches(); // null -> true
    }
}

```

Two rules: **@Valid** never cascades implicitly — every nested object and collection needs it; and a custom validator should **return true for null**, leaving nullability to **@NotNull**.

A.5 Hibernate ORM and Panache

quarkus-hibernate-orm-panache

A.5.1 JPA mapping

Annotation	Purpose	Notes
<code>@Entity</code>	Maps a class to a table	Needs <code>@Id</code> and a no-arg constructor
<code>@Table(name, indexes, uniqueConstraints)</code>	Table metadata	Declare indexes here
<code>@Id</code>	Primary key	
<code>@GeneratedValue(strategy, generator)</code>	Key generation	<code>SEQUENCE</code> batches; <code>IDENTITY</code> disables JDBC batching
<code>@SequenceGenerator(allocationSize)</code>	Sequence tuning	Default 50 — gaps are normal
<code>@Column(name, nullable, length, precision, scale, updatable)</code>	Column metadata	
<code>@Enumerated(EnumType.STRING)</code>	Enum mapping	Never <code>ORDINAL</code> — silent data corruption
<code>@Version</code>	Optimistic locking	Should be your default
<code>@Embedded</code> / <code>@Embeddable</code>	Value object	Hibernate 6 supports record embeddables

Annotation	Purpose	Notes
<code>@AttributeOverride</code>	Remap embedded columns	For two embeddables of one type
<code>@Transient</code>	Exclude from persistence	
<code>@Lob</code>	Large object	
<code>@Convert(converter=)</code>	Attribute converter	

A.5.2 Relationships

Annotation	Default fetch	Recommended
<code>@ManyToOne</code>	EAGER	always set LAZY
<code>@OneToOne</code>	EAGER	always set LAZY
<code>@OneToMany(mappedBy=)</code>	LAZY	correct by default
<code>@ManyToMany</code>	LAZY	use <code>Set</code> , prefer a join entity
<code>@JoinColumn(name, nullable)</code>	—	on the owning side
<code>@JoinTable(name, joinColumns, inverseJoinColumns)</code>	—	for <code>@ManyToMany</code>
<code>@OrderBy("createdAt DESC")</code>	—	database-side ordering
<code>@BatchSize(size=50)</code>	—	N+1 mitigation
<code>@Cacheable / @Cache(usage=)</code>	—	second-level cache opt-in
<code>@NamedEntityGraph</code>	—	declarative fetch plan

```

@Entity
@Table(name = "orders", indexes = @Index(columnList = "status,created_at"))
@NamedEntityGraph(name = "Order.withLines", attributeNodes = @NamedAttributeNode("lines"))
public class Order extends PanacheEntity {

    @Enumerated(EnumType.STRING) @Column(nullable = false, length = 32)
    public OrderStatus status = OrderStatus.PENDING;

    @Version public int version;

    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "customer_id")
    public Customer customer;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    @BatchSize(size = 50)
    public List<OrderLine> lines = new ArrayList<>();

    @PrePersist void onCreate() { createdAt = Instant.now(); }
    @PreUpdate void onUpdate() { updatedAt = Instant.now(); }
}

```

Lifecycle callbacks: `@PrePersist` `@PostPersist` `@PreUpdate` `@PostUpdate` `@PreRemove` `@PostRemove` `@PostLoad`.

Cascade types: `PERSIST` `MERGE` `REMOVE` `REFRESH` `DETACH` `ALL`. `orphanRemoval` is separate — it deletes a child removed from the collection. **Never put `REMOVE` on a `@ManyToOne`.**

A.5.3 Panache and datasources

Annotation	Package	Purpose
<code>@ProjectedFieldName("customer.email")</code>	<code>io.quarkus.hibernate.orm.panache.common</code>	Map a projection field
<code>@PersistenceUnit("name")</code>	<code>io.quarkus.hibernate.orm</code>	Select a persistence unit
<code>@DataSource("name")</code>	<code>io.quarkus.agroal</code>	Select a datasource
<code>@WithTransaction</code>	<code>io.quarkus.hibernate.reactive.panache.common</code>	Reactive transaction
<code>@WithSession</code>	same	Reactive session, no transaction

```
public record OrderSummary(Long id, String status, BigDecimal total,
                           @ProjectedFieldName("customer.email") String email) { }

List<OrderSummary> rows = Order.find("status", PAID)
                              .project(OrderSummary.class)
                              .list();
```

`.project()` is the highest-value Panache feature: fewer columns, no persistence-context bookkeeping, no lazy-loading risk.

A.6 Transactions

quarkus-narayana-jta

Annotation	Package	Purpose
<code>@Transactional</code>	<code>jakarta.transaction</code>	Wrap in a transaction
<code>@Transaction Configuration(timeout=)</code>	<code>io.quarkus.narayana.jta.runtime</code>	Per-method timeout
<code>@TransactionScoped</code>	<code>jakarta.transaction</code>	Bean scoped to the transaction
<code>@Transactional(rollbackOn=)</code>	<code>jakarta.transaction</code>	Roll back on listed exceptions

Annotation	Package	Purpose
<code>@Transactional(dontRollbackOn=)</code>	same	Commit despite listed exceptions

```

@Transactional                                // REQUIRED (default)
@Transactional(TxType.REQUIRES_NEW)           // suspend + new
@Transactional(TxType.MANDATORY)              // throw if none active
@Transactional(TxType.SUPPORTS)               // join if present
@Transactional(TxType.NOT_SUPPORTED)          // suspend, run without
@Transactional(TxType.NEVER)                  // throw if one is active
@Transactional(rollbackOn = BusinessException.class)
@TransactionalConfiguration(timeout = 30)

```

Three ways `@Transactional` silently does nothing: **self-invocation** through `this`, a **non-public** method, or the object **not being a CDI bean**. And by default, **checked exceptions commit** — make domain exceptions unchecked (Chapter 7).

A.7 Caching

quarkus-cache

Annotation	Package	Purpose
<code>@CacheResult(cacheName=)</code>	<code>io.quarkus.cache</code>	Cache the return value
<code>@CacheInvalidate(cacheName=)</code>	<code>io.quarkus.cache</code>	Evict one key
<code>@CacheInvalidateAll(cacheName=)</code>	<code>io.quarkus.cache</code>	Clear the cache
<code>@CacheKey</code>	<code>io.quarkus.cache</code>	Restrict the composite key
<code>@CacheName("n")</code>	<code>io.quarkus.cache</code>	Inject a <code>Cache</code> handle

```

@ApplicationScoped
public class TaxRateService {

    @CacheResult(cacheName = "tax-rates", lockTimeout = 2000)
    public TaxRate rateFor(@CacheKey String country, RequestContext ignored) { ... }

    @CacheInvalidate(cacheName = "tax-rates")
    public void refresh(String country) { ... }

    @CacheInvalidateAll(cacheName = "tax-rates")
    public void refreshAll() { ... }
}

@Inject @CacheName("tax-rates") Cache cache;
cache.get(key, k -> load(k));           // atomic per key - stampede-safe

```

```
quarkus.cache.caffeine."tax-rates".maximum-size=10000      # ALWAYS set this
quarkus.cache.caffeine."tax-rates".expire-after-write=1H
quarkus.cache.caffeine."tax-rates".metrics-enabled=true
```

Options: `lockTimeout` on `@CacheResult` bounds how long a caller waits for another thread's computation. `@CacheKey` matters when a parameter varies per call — a timestamp in the key means a 0 % hit rate.

A.8 Scheduling

quarkus-scheduler · quarkus-quartz

Annotation	Package	Purpose
<code>@Scheduled</code>	<code>io.quarkus.scheduler</code>	Schedule a method
<code>@ActivateRequestContext</code>	<code>io.quarkus.arc</code>	Activate a request context

```
@Scheduled(every = "30s")
@Scheduled(every = "1m", delayed = "30s")
@Scheduled(cron = "0 0 2 * * ?")           // Quartz: 6 fields, leading seconds
@Scheduled(cron = "{orders.job.cron:off}")  // configurable; "off" disables
@Scheduled(cron = "0 0 9 * * ?", timeZone = "Europe/London")
@Scheduled(every = "10s", concurrentExecution = ConcurrentExecution.SKIP)
@Scheduled(every = "10s", skipExecutionIf = MaintenancePredicate.class)
@Scheduled(every = "5m", identity = "reconciliation")
```

Option	Values	Notes
<code>every</code>	"30s", "5m", "1h"	Interval from the end of the previous run
<code>cron</code>	Quartz expression or <code>{property}</code>	? required in one day field
<code>delayed</code>	duration	Delay the first execution
<code>concurrentExecution</code>	PROCEED (default), SKIP	Use SKIP for anything stateful
<code>skipExecutionIf</code>	a <code>SkipPredicate</code> class	<code>Scheduled.ApplicationNotRunning</code> is built in
<code>timeZone</code>	IANA zone	Prefer UTC — DST causes double or missed runs
<code>identity</code>	string	Stable id, needed for clustering

Lifecycle events: `@Observes SuccessfulExecution`, `FailedExecution`, `SkippedExecution`.

With multiple replicas the default scheduler runs on **every pod**. Use clustered Quartz, an application lock with a TTL, or a Kubernetes `CronJob` (Chapter 12).

A.9 Messaging and Kafka

quarkus-messaging-kafka · SmallRye Reactive Messaging

Annotation	Package	Purpose
<code>@Incoming("channel")</code>	<code>org.eclipse.microprofile.reactive.messaging</code>	Consume
<code>@Outgoing("channel")</code>	same	Produce
<code>@Channel("channel")</code>	same	Inject an <code>Emitter</code> / <code>Multi</code>
<code>@Acknowledgment(Strategy)</code>	same	Ack policy
<code>@Broadcast</code>	<code>io.smallrye.reactive.messaging.annotations</code>	Fan out to several consumers
<code>@Merge</code>	same	Merge several producers
<code>@Blocking("pool")</code>	<code>io.smallrye.common.annotation</code>	Worker thread + optional pool
<code>@RunOnVirtualThread</code>	same	Virtual thread per message
<code>@Identifier("name")</code>	<code>io.smallrye.common.annotation</code>	Name a serde or config bean

```
@Inject @Channel("orders-out") MutinyEmitter<OrderEvent> emitter;

@Incoming("orders-in")
@Blocking @Transactional
public void consume(OrderEvent event) { ... }

@Incoming("orders-in")
@Acknowledgment(Acknowledgment.Strategy.MANUAL)
public CompletionStage<Void> manual(Message<OrderEvent> m) { return m.ack(); }

@Incoming("orders-in") @Outgoing("enriched-out")
public OrderEvent enrich(OrderEvent e) { return e.withCustomer(...); }

@Outgoing("heartbeats")
public Multi<String> beat() { return Multi.createFrom().ticks().every(ofSeconds(30)); }
```

Acknowledgment strategy	Meaning
<code>POST_PROCESSING</code> (default)	Ack after the method succeeds — at-least-once
<code>PRE_PROCESSING</code>	Ack before — at-most-once, loses on crash
<code>MANUAL</code>	You call <code>ack()</code> / <code>nack()</code>

Acknowledgment strategy	Meaning
<code>NONE</code>	No acknowledgement

```
mp.messaging.incoming.orders-in.enable.auto.commit=false
mp.messaging.incoming.orders-in.commit-strategy=throttled
mp.messaging.incoming.orders-in.failure-strategy=dead-letter-queue
mp.messaging.outgoing.orders-out.acks=all
mp.messaging.outgoing.orders-out.enable.idempotence=true
```

An `@Incoming` method returning a plain type runs on an **event loop** by default. Add `@Blocking` for JDBC work. And `@Blocking(ordered = false)` discards Kafka's ordering guarantee — a correctness decision, not a tuning one.

A.10 Security

quarkus-oidc · quarkus-smallrye-jwt

Annotation	Package	Purpose
<code>@Authenticated</code>	<code>io.quarkus.security</code>	Require any valid identity
<code>@RolesAllowed({"a","b"})</code>	<code>jakarta.annotation.security</code>	Require a role (OR)
<code>@PermitAll</code>	<code>jakarta.annotation.security</code>	Explicitly public
<code>@DenyAll</code>	<code>jakarta.annotation.security</code>	Block entirely
<code>@PermissionsAllowed(value, inclusive)</code>	<code>io.quarkus.security</code>	Permission-based
<code>@Claim("name")</code>	<code>org.eclipse.microprofile.jwt</code>	Inject a JWT claim
<code>@NoCache</code>	<code>io.quarkus.cache (REST)</code>	Cache-Control: no-cache
<code>@Tenant("name")</code>	<code>io.quarkus.oidc</code>	Select an OIDC tenant

```
@Path("/orders")
public class OrderResource {

    @Inject SecurityIdentity identity;
    @Inject JsonWebToken jwt;
    @Inject @Claim("tenant_id") String tenantId;

    @GET @RolesAllowed("order-reader") public List<OrderView> list() { }
```

```

@POST @PermissionsAllowed("orders:write")      public void create() { }
@PUT  @PermissionsAllowed(value = {"orders:write", "payments:refund"},
                                     inclusive = true) public void refund() { } // AND
@GET  @Path("/public") @PermitAll              public Health health() { }
}

```

```

quarkus.security.jaxrs.deny-unannotated-endpoints=true # secure by default
quarkus.oidc.token.audience=orders-service           # DO NOT OMIT
quarkus.oidc.roles.role-claim-path=realm_access/roles

```

Two settings prevent most real vulnerabilities: `deny-unannotated-endpoints` (a missing annotation becomes a denial, not a public endpoint) and `token.audience` (without it, any token your IdP issued is accepted).

A.11 Fault Tolerance

`quarkus-smallrye-fault-tolerance`

Annotation	Package	Purpose
<code>@Timeout(value, unit)</code>	<code>org.eclipse.microprofile.faulttolerance</code>	Bound the wait
<code>@Retry(maxRetries, delay, jitter, retryOn, abortOn)</code>	same	Retry on failure
<code>@CircuitBreaker(request VolumeThreshold, failure Ratio, delay, success Threshold)</code>	same	Fail fast when failing
<code>@Bulkhead(value, waiting TaskQueue)</code>	same	Cap concurrency
<code>@Fallback(fallbackMethod / value)</code>	same	Degrade
<code>@Asynchronous</code>	same	Run on another thread
<code>@ApplyGuard("name")</code>	<code>io.smallrye.faulttolerance.api</code>	Reusable guard
<code>@ExponentialBackoff</code>	<code>io.smallrye.faulttolerance.api</code>	Backoff strategy
<code>@RateLimit</code>	<code>io.smallrye.faulttolerance.api</code>	Rate limiting

```

@Timeout(value = 2, unit = ChronoUnit.SECONDS)
@Retry(maxRetries = 2, delay = 200, jitter = 100,
      retryOn = {TimeoutException.class, ServerErrorException.class},
      abortOn = {BadRequestException.class, NotFoundException.class})
@CircuitBreaker(requestVolumeThreshold = 20, failureRatio = 0.5,
               delay = 10, delayUnit = ChronoUnit.SECONDS, successThreshold = 3)
@Bulkhead(value = 10)
@Fallback(fallbackMethod = "cachedStock")
StockLevel stock(String sku);

```

```

# Override any annotation parameter without a rebuild
InventoryClient/stock/Timeout/value=3000
InventoryClient/stock/Retry/maxRetries=1
MP_Fault_Tolerance_NonFallback_Enabled=true

```

Nesting order, outermost to innermost: `@Fallback` → `@CircuitBreaker` → `@Timeout` → `@Bulkhead` → `@Retry`. Retries therefore happen *inside* the timeout budget, so the retry schedule must fit within it or the last attempt can never complete.

A.12 Observability

quarkus-micrometer-* · quarkus-opentelemetry · quarkus-smallrye-health

Annotation	Package	Purpose
<code>@Timed(value, percentiles, extraTags)</code>	<code>io.micrometer.core.annotation</code>	Time a method
<code>@Counted(value, record FailuresOnly)</code>	<code>io.micrometer.core.annotation</code>	Count invocations
<code>@MeterTag</code>	<code>io.micrometer.core.annotation</code>	Tag from a parameter
<code>@WithSpan("name")</code>	<code>io.opentelemetry.instrumentation.annotations</code>	Create a span
<code>@SpanAttribute("key")</code>	same	Parameter as a span attribute
<code>@AddingSpanAttributes</code>	same	Attributes without a new span
<code>@Liveness</code>	<code>org.eclipse.microprofile.health</code>	Liveness check
<code>@Readiness</code>	<code>org.eclipse.microprofile.health</code>	Readiness check

Annotation	Package	Purpose
<code>@Startup</code>	<code>org.eclipse.microprofile.health</code>	Startup check
<code>@HealthCheck</code> (interface)	same	Implement <code>call()</code>

```

@Timed(value = "orders.create", percentiles = {0.5, 0.95, 0.99},
      extraTags = {"layer", "service"})
@Counted("orders.attempts")
@WithSpan("orders.create")
public Order create(@SpanAttribute("order.customer") String email) { ... }

@Liveness @ApplicationScoped
public class DeadlockCheck implements HealthCheck {
    public HealthCheckResponse call() {
        return HealthCheckResponse.named("deadlock").status(ok)
            .withData("threads", n).build();
    }
}

```

Cardinality inverts between the two. Metric tags must be low-cardinality — never a user id or a raw URI. Span attributes may be high-cardinality; identifiers belong there. Note also `@Startup` exists in two packages: `io.quarkus.runtime.Startup` (eager bean) and `org.eclipse.microprofile.health.Startup` (health check).

A.13 Testing

quarkus-junit5 · quarkus-junit5-mockito

Annotation	Package	Purpose
<code>@QuarkusTest</code>	<code>io.quarkus.test.junit</code>	Full app in-JVM
<code>@QuarkusComponentTest</code>	<code>io.quarkus.test.component</code>	Minimal CDI container
<code>@QuarkusIntegrationTest</code>	<code>io.quarkus.test.junit</code>	Black box on the artifact
<code>@QuarkusMainTest</code>	<code>io.quarkus.test.junit.main</code>	Test a <code>QuarkusApplication</code>
<code>@TestTransaction</code>	<code>io.quarkus.test</code>	Roll back each test
<code>@TestProfile(P.class)</code>	<code>io.quarkus.test.junit</code>	Config override — restarts Quarkus
<code>@TestSecurity(user, roles)</code>	<code>io.quarkus.test.security</code>	Simulated identity

Annotation	Package	Purpose
<code>@JwtSecurity(claims=)</code>	<code>io.quarkus.test.security.jwt</code>	JWT claims
<code>@InjectMock</code>	<code>io.quarkus.test.junit.mockito</code>	Replace a bean with a mock
<code>@InjectSpy</code>	same	Real bean, verifiable
<code>@Mock</code>	<code>io.quarkus.test</code>	Global alternative for tests
<code>@QuarkusTestResource(R.class)</code>	<code>io.quarkus.test.common</code>	External resource lifecycle
<code>@TestHTTPResource("/path")</code>	<code>io.quarkus.test.common.http</code>	Inject a test URL
<code>@TestHTTPEndpoint(R.class)</code>	same	Base path from a resource
<code>@TestConfigProperty(key, value)</code>	<code>io.quarkus.test.component</code>	Config for a component test
<code>@DisabledOnIntegrationTest</code>	<code>io.quarkus.test.junit</code>	Skip when running as IT

```

@QuarkusTest
@TestProfile(StrictProfile.class)           // share profiles - each one restarts
class OrderResourceTest {

    @Inject OrderService service;
    @InjectMock PaymentGateway gateway;
    @InjectSpy AuditService audit;
    @TestHTTPResource("/orders") URL ordersUrl;

    @Test @TestTransaction
    void createsOrder() { ... }

    @Test @TestSecurity(user = "alice", roles = "order-reader")
    @JwtSecurity(claims = @Claim(key = "tenant_id", value = "acme"))
    void readerCanList() { ... }
}

@QuarkusIntegrationTest
class OrderResourceIT extends OrderResourceTest { }    // runs natively too

```

`@QuarkusIntegrationTest` is a **black box**: no `@Inject`, no `@InjectMock`, no `@TestTransaction`. Write the base test with HTTP-only assertions so it can be reused.

A.14 Native Image

Annotation	Package	Purpose
<code>@RegisterForReflection</code>	<code>io.quarkus.runtime.annotations</code>	Retain a class and its metadata
<code>@RegisterForProxy</code>	<code>io.quarkus.runtime.annotations</code>	Declare a dynamic proxy
<code>@Recorder</code>	<code>io.quarkus.runtime.annotations</code>	Bytecode-recording class
<code>@BuildStep</code>	<code>io.quarkus.deployment.annotations</code>	Augmentation step
<code>@Record(ExecutionTime)</code>	<code>io.quarkus.deployment.annotations</code>	STATIC_INIT / RUNTIME_INIT
<code>@ConfigRoot(phase=)</code>	<code>io.quarkus.runtime.annotations</code>	Extension config root
<code>@Substitute</code> / <code>@TargetClass</code>	<code>com.oracle.svm.core.annotate</code>	Replace code in the image

```

@RegisterForReflection(targets = {LegacyPayload.class, LegacyResponse.class})
@RegisterForReflection(targets = OrderEvent.class, registerFullHierarchy = true)
@RegisterForReflection(targets = SessionState.class, serialization = true)
@RegisterForReflection(targets = Dto.class, methods = true, fields = true,
    ignoreNested = false, reason = "Serialised by the audit writer")
public class ReflectionConfiguration { }

```

```

quarkus.native.resources.includes=templates/**/*.json
quarkus.native.additional-build-args=-initialize-at-run-time=com.acme.RequestIds

```

A.15 Quick Reference by Task

Expose an endpoint	<code>@Path</code> + <code>@GET</code> / <code>@POST</code> + <code>@Produces</code>
Bind a parameter	<code>@RestPath</code> <code>@RestQuery</code> <code>@RestHeader</code> <code>@BeanParam</code>
Force a thread model	<code>@Blocking</code> <code>@NonBlocking</code> <code>@RunOnVirtualThread</code>
Map an exception	<code>@ServerExceptionMapper</code>
Call another service	<code>@RegisterRestClient</code> + <code>@RestClient</code> (+ <code>@AccessToken</code>)
Make a bean	<code>@ApplicationScoped</code> (default) / <code>@Singleton</code>
Inject	constructor, or <code>@Inject</code>
Produce a third-party bean	<code>@Produces</code> + <code>@Disposes</code>
Choose by environment	<code>@IfBuildProfile</code> / <code>@LookupIfProperty</code>
Keep a looked-up bean	<code>@Unremovable</code>
Read config	<code>@ConfigMapping</code> (preferred) / <code>@ConfigProperty</code>
Validate input	<code>@Valid</code> + constraints on a record
Map an entity	<code>@Entity</code> <code>@Id</code> <code>@Enumerated(STRING)</code> <code>@Version</code>
Avoid N+1	<code>@BatchSize</code> / <code>@NamedEntityGraph</code> / <code>.project()</code>
Transaction	<code>@Transactional</code> (public, not self-invoked)
Cache	<code>@CacheResult</code> / <code>@CacheInvalidate</code> (+ maximum-size)

Schedule	@Scheduled(cron = "{prop}", concurrentExecution = SKIP)
Consume Kafka	@Incoming + @Blocking + @Transactional (idempotent!)
Produce Kafka	@Channel + MutinyEmitter
Secure	@RolesAllowed / @PermissionsAllowed (+ deny-unannotated)
Resilience	@Timeout @Retry @CircuitBreaker @Bulkhead @Fallback
Measure	@Timed @Counted @WithSpan
Health	@Liveness @Readiness @Startup
Test	@QuarkusTest @InjectMock @TestTransaction @TestSecurity
Native	@RegisterForReflection

A.16 Annotations That Are Easy to Confuse

Pair	Difference
<code>io.quarkus.runtime.Startup</code> vs <code>org.eclipse.microprofile.health.Startup</code>	Eager bean creation vs a startup health check
<code>@Singleton</code> vs <code>@ApplicationScoped</code>	No proxy, eager vs proxied, lazy
<code>@Produces</code> (CDI) vs <code>@Produces</code> (JAX-RS)	Producer method vs response media type
<code>@Timed</code> (Micrometer) vs <code>@Timed</code> (MicroProfile Metrics)	Micrometer is current; MP Metrics is deprecated
<code>@Transactional</code> (Jakarta) vs <code>@WithTransaction</code> (reactive Panache)	Blocking vs reactive sessions
<code>@Blocking</code> (SmallRye) vs <code>@Blocking</code> (messaging)	Same annotation, different defaults per context
<code>@Valid</code> vs <code>@Validated</code>	Cascade/trigger vs class-level method validation
<code>@RestPath</code> vs <code>@PathParam</code>	Shorthand needing <code>-parameters</code> vs explicit name
<code>@InjectMock</code> vs <code>@Mock</code>	Per-test mock vs global test alternative
<code>@CacheKey</code> vs <code>@CacheName</code>	Narrow the key vs inject the cache handle

Glossary

Terms used in this book, with the meaning they carry in a Quarkus context. Cross-references point to the chapter where the term is developed.

A

AOT (ahead-of-time) compilation — Compiling to machine code before execution rather than at runtime. GraalVM Native Image AOT-compiles the reachable code of an application; there is no bytecode and no JIT at runtime, which is why startup is fast and peak throughput is somewhat lower. (*Ch 18*)

AppCDS (Application Class Data Sharing) — A JVM feature that stores pre-parsed class metadata in an archive shared across restarts, reducing JVM-mode startup time. Enabled with `quarkus.package.jar.appcds.enabled`. (*Ch 23*)

ArC — Quarkus’s implementation of CDI 4.1. It resolves the entire bean graph during the Maven build and generates a class per bean, so runtime injection involves no reflection and no bean resolution. Also removes unused beans and generates client proxies. (*Ch 3*)

Agroal — The connection pool Quarkus uses instead of HikariCP. Designed for Quarkus’s transaction integration and native-image compatible. Its metrics (`agroal_active_count`, `agroal_awaiting_count`) are among the most useful diagnostics available. (*Ch 5*)

At-least-once delivery — A messaging guarantee in which a message is delivered one or more times. This is what Kafka gives you in practice, and no configuration changes it — which is why consumers must be idempotent. (*Ch 11*)

At-most-once delivery — A message is delivered zero or one times. Achieved by committing the offset before processing, and therefore loses messages on a crash. Rarely what you want. (*Ch 11*)

Augmentation — Quarkus’s build-time phase, run by the Maven or Gradle plugin, in which build steps analyse the application and generate bytecode, metadata and a recorded bootstrap sequence. The machinery that performs it is discarded before the artefact is packaged. (*Ch 1*)

B

Backpressure — A protocol in which a subscriber requests a bounded number of items and the publisher may not exceed it. Demand flows upstream, data flows downstream. This is what distinguishes Reactive Streams from callbacks, and it is the capability virtual threads do **not** provide. (*Ch 10*)

BFF (Backend for Frontend) — A service per client type that aggregates and shapes responses for that client. Also the recommended place to hold OAuth tokens so a browser never does. A BFF composes and shapes; it must not accumulate business logic. (*Ch 9, Ch 22*)

BOM (Bill of Materials) — `quarkus-bom`, a Maven POM that pins consistent, tested versions for every extension and its transitive dependencies. Overriding a single version by hand produces a combination nobody has tested. (*Ch 1*)

Build item — A typed value passed between build steps during augmentation. `SimpleBuildItem` has at most one instance; `MultiBuildItem` has many. (*Ch 1*)

Build step — A method annotated `@BuildStep` in an extension's deployment module. Consumes and produces build items; the engine sorts them topologically and runs independent branches in parallel. (*Ch 1*)

Build-time-fixed property — A configuration property consumed during augmentation to generate code, and therefore immutable afterwards. `quarkus.datasource.db-kind` and `quarkus.http.root-path` are examples. Setting one at runtime has no effect and often no warning — the most common source of silent misconfiguration. (*Ch 4*)

Bulkhead — A concurrency cap per dependency, so one slow downstream cannot consume the entire worker pool and take down endpoints that never call it. Named after ship compartments. (*Ch 22*)

C

CDI (Contexts and Dependency Injection) — The Jakarta EE component model: beans, contexts (scopes) and injection. Quarkus implements CDI 4.1 through ArC. (*Ch 3*)

CFS throttling — Completely Fair Scheduler quota enforcement. When a container exhausts its CPU quota within a 100 ms window it is throttled for the remainder of that window, even on an idle node. For a JVM's bursty GC and JIT threads this produces latency spikes easily mistaken for garbage-collection problems. (*Ch 20, Ch 23*)

Circuit breaker — A component that stops calling a failing dependency after a failure threshold, failing fast instead, then periodically testing recovery. States: closed, open, half-open. Its purpose is to stop *you* wasting threads and to shed load so the dependency can recover. (*Ch 22*)

Client proxy — A generated subclass injected in place of a normal-scoped bean, forwarding each call to the current contextual instance. Why `@ApplicationScoped` beans must not be `final`, need a no-arg constructor, and why self-invocation bypasses interceptors. (*Ch 3*)

Closed-world assumption — The premise of native compilation: the set of classes, methods and resources is fixed at build time and nothing can be loaded or generated later. Everything unreachable is deleted. Every native-mode difficulty follows from this. (*Ch 18*)

Coordinated omission — A measurement error in which a closed-loop load generator sends fewer requests when the server slows, so the slow period is under-sampled and reported percentiles are far better than reality. Avoided with an open-loop, constant-arrival-rate model. (*Ch 23*)

Confused deputy — A vulnerability in which service A authenticates a user then calls service B with its own privileged credentials, so B cannot enforce user-level rules. Any bug in A becomes a privilege escalation. (*Ch 9*)

CQRS (Command Query Responsibility Segregation) — Separating the write model from the read model. Worth it when reads and writes have genuinely different shapes or scaling needs; DTO projections against one database capture most of the benefit for a fraction of the cost. (*Ch 22*)

D

Dead-letter queue (DLQ) — A topic or queue receiving messages that could not be processed, so the partition can make progress. Not optional at scale, because a Kafka partition's offset is a single high-water mark and a poison message otherwise blocks everything behind it. (*Ch 11*)

Dev Mode — `quarkus:dev`. Runs the application with live reload, continuous testing, Dev UI and Dev Services. (*Ch 1*)

Dev Services — Quarkus automatically starting containerised backing services — PostgreSQL, Kafka, Keycloak, Redis — in dev and test when no connection URL is configured. Its most valuable property is that tests run against the real technology rather than a substitute. (*Ch 1, Ch 17*)

Dev UI — The per-extension console at `/q/dev-ui`, showing resolved configuration with sources, the CDI bean graph including removed beans, Hibernate persistence units and more. (*Ch 1*)

Distributed monolith — Services that must be deployed together, share a database, or break each other when one is down. Every cost of distribution and none of the independence. The remedy is usually merging, not splitting. (*Ch 22*)

DTO (Data Transfer Object) — A type shaped for the wire rather than for persistence. Java 21 records are the idiomatic choice. Exposing entities instead couples the API to the schema and triggers lazy-loading failures during serialisation. (*Ch 2, Ch 5*)

E

Entity graph — A declarative fetch plan (`@NamedEntityGraph`) specifying which associations to load, applied per query. One of the remedies for N+1. (*Ch 5*)

Event loop — A thread in Vert.x's small fixed pool (default `2 × cores`) that must never block. Every connection is pinned to one for its lifetime, so blocking one stalls every connection assigned to it — not just the offending request. (*Ch 2*)

Event sourcing — Storing the sequence of events as the source of truth and deriving current state by replay. Valuable where the history *is* the product; frequently cited and rarely justified elsewhere. (*Ch 22*)

Exactly-once effect — Achievable, unlike exactly-once *delivery*: at-least-once delivery combined with an idempotent consumer. The distinction is worth insisting on in an interview. (*Ch 11*)

Extension — A Quarkus module integrating a library: a runtime module that ships, and a `-deployment` module containing build steps that is discarded after the build. Its job is to move initialisation to build time and make the library native-safe. (*Ch 1*)

F

Fast-jar — Quarkus’s default packaging: `lib/`, `app/`, `quarkus/` and a small launcher jar. Exists so container images can layer rarely-changing dependencies separately from frequently-changing application code — a container optimisation, not a runtime one. (*Ch 1*, *Ch 19*)

Flame graph — A profile visualisation where width is time spent and depth is stack depth. Look for wide plateaus, not tall towers. CPU mode excludes blocked time; use wall mode when latency is high and CPU is low. (*Ch 23*)

Flyway — The schema-migration tool Quarkus integrates. Versioned SQL migrations with a history table, as opposed to `hbm2ddl`, which is not schema management. (*Ch 5*)

G

GraalVM — The Oracle/community JDK distribution whose Native Image component compiles Java ahead of time. (*Ch 18*)

H

hbm2ddl — Hibernate’s schema generation (`quarkus.hibernate-orm.database.generation`). Values `create`, `drop-and-create`, `update`, `validate`, `none`. `update` must never be used in production: additive only, no history, no review, version-dependent behaviour. (*Ch 5*)

I

Idempotency — An operation producing the same effect however many times it is applied. Non-negotiable for message consumers, because delivery is at-least-once. Best achieved structurally — an upsert with a timestamp guard — rather than with a dedup table. (*Ch 7*, *Ch 11*)

Image heap — The serialised object graph produced by running static initialisers during a native build and embedded in the executable. Why a `Random` in a static field is a security bug: it is seeded on the build machine, so every replica produces the identical sequence. (*Ch 18*)

ImageStream — An OpenShift object: a named, versioned pointer to images that decouples the Deployment from a registry URL and can trigger rollouts on tag change. (*Ch 21*)

J

Jandex — An offline bytecode indexer producing a queryable index of classes, annotations and hierarchies **without loading any classes**. Everything Quarkus discovers comes from it — which is why a bean in an unindexed JAR is invisible. (*Ch 1*, *Ch 3*)

Jib — A container-image builder needing no Docker daemon. Layers correctly by default and produces reproducible images with deterministic timestamps. The right default for CI. (*Ch 19*)

JFR (JDK Flight Recorder) — A low-overhead JVM profiler (~1–2 % on the `profile` setting), safe in production. Partially supported in native mode. (*Ch 23*)

K

KEDA — Kubernetes Event-Driven Autoscaling. Scales workloads on external signals such as Kafka consumer lag, which is the correct signal for a consumer where CPU is meaningless. (*Ch 20*)

L

LazyInitializationException — Thrown when an uninitialised Hibernate proxy is touched after the persistence context has closed — typically while mapping an entity to a DTO outside the transaction. Fix by fetching what you need in the query, or by projecting straight into a DTO. Never by enabling `enable_lazy_load_no_trans`. (*Ch 5*)

Liveness probe — Answers “would a restart fix this?” Failure causes Kubernetes to restart the container. Must therefore check only this process’s own state — **never a dependency**, since restarting cannot fix someone else’s outage. (*Ch 16*)

Live reload — Quarkus’s request-triggered reload: on an incoming request it checks source timestamps, recompiles changed files, re-runs augmentation and swaps in a fresh application classloader. Not JVM HotSwap, which is why it can pick up new extensions and build-time config changes. (*Ch 1*)

LRA (Long Running Actions) — A MicroProfile specification for orchestrated sagas over HTTP, with `@LRA`, `@Compensate` and `@Complete`. (*Ch 7*)

M

Mandrel — Red Hat’s distribution of GraalVM Native Image, stripped to the native-image capability. What Quarkus is built and tested against. (*Ch 18*)

Micrometer — A vendor-neutral metrics facade — the SLF4J of metrics. Instrument once, choose Prometheus, Datadog or OTLP by dependency. (*Ch 14*)

MicroProfile — A set of specifications for Java microservices — Config, Health, Metrics, Fault Tolerance, REST Client, JWT — implemented in Quarkus largely through SmallRye. (*Ch 1*)

Multi — Mutiny’s type for a stream of zero to many items with backpressure. Compare `Uni`. (*Ch 10*)

Mutiny — Quarkus’s reactive library. Two types (`Uni`, `Multi`) and an event-driven API where you name the event group first (`onItem()`, `onFailure()`) and then the action. (*Ch 10*)

N

N+1 problem — Loading N parents with one query, then issuing one query per parent for an association. Scales linearly with data volume and is invisible on small datasets. Remedied with join fetch, entity graphs, `@BatchSize` or — best — DTO projections. (*Ch 5*)

Native image — A standalone executable produced by GraalVM containing the application and a minimal runtime. Starts in tens of milliseconds with 20–50 MB RSS, at the cost of build time, tooling and roughly 10–30 % peak throughput. (*Ch 18*)

Narayana — The JTA transaction manager Quarkus uses. Also provides `QuarkusTransaction`, the modern programmatic API. (*Ch 7*)

O

OIDC (OpenID Connect) — An identity layer over OAuth 2.0 defining discovery, the authorization code flow, refresh and logout. Distinct from JWT, which is a token *format*. `quarkus-oidc` speaks the protocol; `quarkus-smallrye-jwt` only verifies tokens. (*Ch 9*)

Optimistic locking — Concurrency control using a `@Version` column: the update carries the expected version and fails with `OptimisticLockException` if another transaction won. No locks held, no contention — the right default. (*Ch 5, Ch 7*)

Ordinal — A config source’s precedence. System properties 400, environment variables 300, `.env` 295, `config/application.properties` 260, `application.yaml` 255, `application.properties` 250. Highest wins outright; there is no merging. (*Ch 4*)

Outbox pattern — Writing an event to an outbox table in the same local transaction as a business change, then publishing it separately. The answer to “update the database and publish an event atomically”, which is otherwise impossible. The highest value-to-complexity ratio pattern in the book. (*Ch 7, Ch 11*)

OTLP (OpenTelemetry Protocol) — The wire protocol for exporting traces, metrics and logs to a collector. (*Ch 15*)

P

Panache — A layer of build-time-generated syntactic sugar over Hibernate ORM, in active-record (`PanacheEntity`) and repository (`PanacheRepository`) styles. Reduces boilerplate; does not reduce the JPA you need to understand. (*Ch 6*)

Parent-based sampler — An OpenTelemetry sampler that honours the incoming `traceparent` sampled flag and only decides independently for root spans. Essential in production, or traces come back with holes. (*Ch 15*)

Persistence context — The set of entities Hibernate is currently managing, scoped to a transaction. Also the first-level cache and the source of dirty checking. (*Ch 5*)

PGO (Profile-Guided Optimisation) — Feeding a recorded runtime profile back into a native build, recovering much of the throughput gap with the JIT. Requires a two-stage build and a representative workload. (*Ch 18*)

Points-to analysis — The whole-programme static analysis in a native build computing every reachable method and every type that can flow to each variable. A fixed-point computation whose memory cost is why builds need 6–12 GB. (*Ch 18*)

Projection — Selecting only the columns you need directly into a DTO, rather than loading entities. Routinely a 3–10× improvement on read paths and the single highest-leverage query change available. (*Ch 5, Ch 6*)

Q

Quarkus REST — The current name of RESTEasy Reactive, renamed in Quarkus 3.9 because “Reactive” wrongly implied you had to write reactive code. Artifact `quarkus-rest`. (*Ch 2*)

R

Readiness probe — Answers “should this pod receive traffic?” Failure removes the pod from Service endpoints without restarting it. Should reflect only what *this pod* cannot work without — checking downstream dependencies here propagates their outages upward. (*Ch 16*)

Recorder — A `@Recorder`-annotated class whose method calls during augmentation are intercepted and written as bytecode rather than executed. The mechanism bridging build time and runtime; returns `RuntimeValue<T>` handles that cannot be inspected at build time. (*Ch 1*)

RED method — Rate, Errors, Duration: the three signals to measure for any request-driven service. Compare USE. (*Ch 14*)

RESTEasy Classic — The original blocking, servlet-derived JAX-RS implementation, now `quarkus-resteasy-classic`. Superseded by Quarkus REST for most purposes. (*Ch 2*)

RFC 7807 (Problem Details) — A standard JSON error format with `type`, `title`, `status`, `detail` and `instance`. The right default for API error bodies, because clients can switch on a stable `type` URI. (*Ch 2*, *Ch 8*)

Runtime init — The bootstrap phase executing environment-dependent setup — opening sockets and pools, reading environment variables — always at process start. Compare static init. (*Ch 1*)

S

Saga — A sequence of local transactions across services, each with a compensating action, replacing a distributed transaction. Compensations are domain operations, not rollbacks — you cannot un-send an email. (*Ch 7*, *Ch 22*)

SCC (Security Context Constraint) — An OpenShift policy governing what a pod may do. The default `restricted-v2` runs containers as a **random UID** in the project’s range rather than the Dockerfile’s `USER`, which is the most common cause of images that work elsewhere and fail on OpenShift. (*Ch 21*)

SLI / SLO / error budget — Service Level Indicator (a measured signal), Objective (a target for it), and the budget of failure the objective permits. Alerting on budget burn rate rather than on instantaneous thresholds is current best practice. (*Ch 14*)

SmallRye — The open-source project providing Quarkus’s MicroProfile implementations: SmallRye Config, Health, Fault Tolerance, Reactive Messaging, Mutiny and more. (*Ch 1*)

Span — One unit of work within a trace, with an id, a parent, a name, a duration, attributes and a status. Unlike metric tags, span attributes **may** be high-cardinality. (*Ch 15*)

Static init — The bootstrap phase executing deterministic, environment-independent setup — building an ORM metamodel, compiling serialisers. In native mode this runs **at image build time** and its object graph is serialised into the executable. (*Ch 1*)

Stampede (cache) — A hot key expiring and many concurrent requests all executing the expensive load. Mitigated by atomic per-key loading (`cache.get(key, loader)`), jittered TTLs and refresh-ahead. (*Ch 13*)

Startup probe — Suppresses liveness and readiness until boot completes, removing the trade-off between tolerating slow starts and detecting dead processes quickly. (*Ch 16*)

T

Tail-based sampling — Deciding whether to keep a trace after it completes, in the collector, so every error and slow trace can be retained while most successful ones are discarded. Far more useful per byte stored than a flat head-based percentage. (*Ch 15*)

Testcontainers — The library underlying Dev Services, starting real backing services in containers for development and tests. (*Ch 17*)

Trace context (W3C) — The `traceparent` header carrying trace id, parent span id and sampled flag across process boundaries. (*Ch 15*)

U

UBI (Universal Base Image) — Red Hat’s freely redistributable base images. Already satisfy OpenShift’s `restricted-v2` SCC, which is a strong practical reason to use them there. (*Ch 19, Ch 21*)

Uni — Mutiny’s type for at most one item, then completion. Compare `Multi`. Lazy: nothing happens until subscription. (*Ch 10*)

Unremovable — `@Unremovable` exempts a bean from Arc’s unused-bean removal. Needed for beans obtained only by programmatic lookup, which the build cannot see. (*Ch 3*)

USE method — Utilisation, Saturation, Errors: the three signals to measure for any resource. Saturation is the leading indicator. Compare RED. (*Ch 14*)

V

Vert.x — The reactive toolkit, built on Netty, underneath Quarkus’s entire I/O layer. Both the imperative and reactive programming models sit on one engine. (*Ch 1, Ch 2*)

Virtual thread — A JVM-managed lightweight thread (Java 21, JEP 444). `@RunOnVirtualThread` gives high concurrency for blocking I/O with imperative, debuggable code — but provides **no backpressure**, and does not widen a connection pool. (*Ch 2, Ch 10*)

W

Worker thread — A thread from Quarkus’s elastic pool (default max 200) for blocking work: JDBC, file I/O, synchronous HTTP clients. Compare event loop. (*Ch 2*)

Write skew — A concurrency anomaly in which two transactions each read a consistent snapshot, each make a locally valid decision, and together violate an invariant. Prevented by `SERIALIZABLE`; permitted by PostgreSQL’s `REPEATABLE READ`. (*Ch 7*)

X

XA (eXtended Architecture) — Two-phase commit across multiple resources. Supported by Narayana and the wrong answer for microservices: synchronous coupling, locks held across the network, a coordinator single point of failure, and most modern participants do not support it. (*Ch 7*)

Z

Zero-width space (U+200B) — Not a Quarkus term, but worth recording: this book's PDF pipeline injects it into long inline code spans so that identifiers such as `src/main/resources/application.properties` can wrap rather than overrun the margin. See `tools/build.py`.

Abbreviations

AOT	Ahead-of-time compilation
ArC	Quarkus's CDI container
BFF	Backend for Frontend
BOM	Bill of Materials
CDC	Change Data Capture
CDI	Contexts and Dependency Injection
CQRS	Command Query Responsibility Segregation
DLQ	Dead-letter queue
DTO	Data Transfer Object
HPA	Horizontal Pod Autoscaler
JFR	JDK Flight Recorder
JIT	Just-in-time compilation
JTA	Jakarta Transactions
JWT	JSON Web Token
LRA	Long Running Actions
LTS	Long-Term Support
MDC	Mapped Diagnostic Context
OIDC	OpenID Connect
OTLP	OpenTelemetry Protocol
PDB	Pod Disruption Budget
PGO	Profile-Guided Optimisation
RSS	Resident Set Size
S2I	Source-to-Image
SBOM	Software Bill of Materials
SCC	Security Context Constraint
SLI/SLO	Service Level Indicator / Objective
SSE	Server-Sent Events

UBI	Universal Base Image
XA	eXtended Architecture (two-phase commit)